

Redis

The Complete Guide

From first principles to production: how Redis works inside, every data structure, caching and scaling patterns, Java and Spring Boot implementation, where Redis is used in real systems, and complete end-to-end projects.

Fundamentals · Data structures · Internals · Persistence · Replication & Cluster · Spring Boot
Patterns · Use cases · Projects · Operations · Interview preparation

user:42	cart:7	sess:a9	rank	stock:9	feed	lock:o1
rate:ip	hits	user:42	cart:7	sess:a9	rank	stock:9
feed	lock:o1	rate:ip	hits	user:42	cart:7	sess:a9

16384 hash slots · keys · values · microseconds

Version basis

Redis Open Source 8.x (the 8.6 line was current at the time of writing, October 2026)

Spring Boot 4.1 with Spring Data Redis 4.1 and Lettuce (Spring Framework 7; examples use Java 21)

Jedis and Redisson are covered briefly; Testcontainers for tests

How to use this guide

- **Read it in order the first time.** The opening parts build the mental model (what Redis is and why it is fast), then every data structure, how Redis works inside, persistence, replication and Cluster, the Java and Spring Boot implementation, the patterns that matter (caching, locks, rate limiting, queues, leaderboards), the use-case catalogue, complete projects, operations and an interview kit. Use the contents page to jump to what you need.
- **Every chapter follows the same rhythm:** what it is, why it exists, how it works, how to use it (with commands and code), what goes wrong, and a short key-points box. Coloured boxes mean: NOTE (extra detail), TIP (practical advice), WATCH OUT and GOTCHA (mistakes that cost people days), REAL WORLD (how it appears in production), INTERVIEW ANGLE (what interviewers like to ask).
- **Labs are hands-on.** A dark box is a terminal session: lines beginning with `$` are shell commands, lines beginning with `127.0.0.1:6379>` are what you type inside `redis-cli`, and the other lines are the output you should see. Output is shown as typical output; timings, IDs and memory numbers on your machine will differ.
- **About the code.** Every command and listing was written against the official documentation for the versions named on the cover and checked for syntax. They were not executed against a live server while this guide was produced, so treat the first run on your machine as part of the lab: if something behaves differently, the documentation for your exact version is the final word.
- **Running example.** Most examples use one fictional online shop, *ShopStream*, with a product catalogue, carts, sessions, flash sales and notifications. Seeing the same domain across chapters lets you focus on Redis, not on the business problem.
- **Conventions.** `monospace` is code, a command or a configuration name. Command names are written in upper case (`SET`) although Redis accepts any case. Key names use colons as separators (`user:42:profile`). Defaults quoted are those of Redis 8 documentation unless stated.

Contents

Part 1: Fundamentals	6
1.1 The Problem Redis Solves	6
1.2 What Redis Is	9
1.3 Why Redis Is Fast	12
1.4 Keys, Values, Types and Naming	16
1.5 The First Ten Minutes: A Tour of the Types	20
1.6 Guarantees and Trade-offs	23
1.7 Redis versus the Alternatives	26
1.8 History, Licensing and the Redis 8 Platform	28
1.9 Roadmap of This Book and the ShopStream Project	30
Part 2: Hands-On Labs	34
2.1 Installing Redis	34
2.2 <code>redis-cli</code> in Depth	37
2.3 Configuration Basics	43
2.4 The Lab Workbook	46
Part 3: Data Structures	69
3.1 Strings	69
3.2 Lists	75
3.3 Hashes	79

3.4 Sets	83
3.5 Sorted sets	86
3.6 Choosing the right structure	90
3.7 Streams	92
3.8 Pub/Sub and Keyspace Notifications	98
3.9 Bitmaps and Bitfields	101
3.10 HyperLogLog	103
3.11 Geospatial Indexes	105
3.12 Iterating Safely: SCAN and Friends	106
3.13 JSON Documents	109
3.14 The Query Engine: Secondary Indexes and Search	111
3.15 Vector Search and Vector Sets	115
3.16 Probabilistic Data Structures	117
3.17 Time Series	120

Part 4: Redis Inside 123

4.1 The Server Architecture	123
4.2 Memory Layout and Encodings	128
4.3 Expiration	134
4.4 Memory Limits and Eviction	138
4.5 RDB Snapshots	142
4.6 The Append-Only File (AOF)	145
4.7 Choosing a Persistence and Durability Strategy	149
4.8 Transactions: MULTI, EXEC and WATCH	151
4.9 Lua Scripting with EVAL	154
4.10 Redis Functions	159
4.11 Pipelining, Blocking Commands and Latency	161
4.12 RESP3 and Client-Side Caching	166
4.13 The Redis Security Model	169

Part 5: Scaling and High Availability 175

5.1 Replication	175
5.2 Sentinel: automatic failover	182
5.3 Why Cluster: sharding basics	188
5.4 Redis Cluster	189
5.5 Atomic slot migration and operating a cluster	196
5.6 Choosing a topology	200
5.7 Multi-region and managed services	202

Part 6: Redis from Java 205

6.1 Choosing a Java client	205
6.2 Lettuce	206

6.3 Jedis	211
6.4 Redisson	214
6.5 Connection management and resilience	216

Part 7: Spring Boot and Spring Data Redis 220

7.1 Setting up Spring Boot with Redis	220
7.2 StringRedisTemplate, RedisTemplate and serializers	224
7.3 Hash mapping and Redis repositories	230
7.4 Spring Cache with Redis	236
7.5 Pub/Sub with Spring	241
7.6 Streams with Spring	244
7.7 Reactive Redis	249
7.8 Spring Session with Redis	252
7.9 Transactions, Scripts and Pipelines in Spring	254
7.10 Testing Redis Code	257
7.11 Observability and a Production Checklist	260

Part 8: Patterns 263

8.1 Caching fundamentals	263
8.2 Cache-aside	268
8.3 Read-through, write-through, write-behind and refresh-ahead	273
8.4 Cache failure modes	276
8.5 Keeping cache and database consistent	282
8.6 Multi-level caching	285
8.7 Distributed locks	288
8.8 Rate limiting	296
8.9 Idempotency keys and deduplication	302
8.10 Queues and background jobs	306
8.11 Delayed and scheduled tasks	310
8.12 Counters, analytics and unique counts	312
8.13 Leaderboards and ranking	315
8.14 Sessions, carts and ephemeral state	318
8.15 Feature flags, configuration and flash-sale stock	320
8.16 Real-time messaging patterns	324
8.17 Search, geo and AI: semantic caching and RAG	326
8.18 Anti-patterns catalogue	329

Part 9: Where Redis Is Used 333

9.1 The Use-Case Map	333
9.2 E-Commerce	334
9.3 Gaming and Social Applications	337
9.4 Fintech, Ride-Hailing and Logistics	338

9.5 Real-Time Analytics and IoT	339
9.6 Microservices Glue	341
9.7 AI Applications	342
9.8 Decision Guide: When to Use Redis and When Not To	344
9.9 Reference Architectures for ShopStream	346

Part 10: Projects 350

10.1 How to Work Through the Projects	350
10.2 Project A: A Production-Grade Catalogue Cache	351
10.3 Project B: An API Rate Limiter Service	363
10.4 Project C: A Flash-Sale Service That Never Oversells	373
10.5 Project D: A Live Leaderboard with Notifications	385

Part 11: Operations and Security 398

11.1 Deployment Options and a Production redis.conf	398
11.2 Sizing and Capacity Planning	402
11.3 Monitoring Redis	405
11.4 Troubleshooting Playbook	408
11.5 Security Hardening	412
11.6 Backup, Restore and Disaster Recovery	416
11.7 Upgrades and Migrations	418

Part 12: Revision Kit and Interview Preparation 422

12.1 The One-Page Mental Model	422
12.2 Command Cheat Sheets by Structure	423
12.3 Configuration and Metrics Cheat Sheet	426
12.4 Decision Guides	429
12.5 Interview Q&A: Fundamentals	430
12.6 Interview Q&A: Data Structures	432
12.7 Interview Q&A: Internals and Persistence	433
12.8 Interview Q&A: Scaling and High Availability	435
12.9 Interview Q&A: Caching and Patterns	437
12.10 Interview Q&A: Java and Spring	438
12.11 System-Design Scenarios	440
12.12 Common Mistakes and Myths	441
12.13 A 30-Day Study Plan	442

PART 1

Fundamentals

What Redis is, why it is fast, what it promises and what it does not, and how this book is organised.

In this part

- 1.1 The Problem Redis Solves
- 1.2 What Redis Is
- 1.3 Why Redis Is Fast
- 1.4 Keys, Values, Types and Naming
- 1.5 The First Ten Minutes: A Tour of the Types
- 1.6 Guarantees and Trade-offs
- 1.7 Redis versus the Alternatives
- 1.8 History, Licensing and the Redis 8 Platform
- 1.9 Roadmap of This Book and the ShopStream Project

1.1 The Problem Redis Solves

Every application that grows past a handful of users meets the same wall. The code is correct, the database schema is sensible, and yet pages that felt instant on a laptop take a second or two under real traffic. Servers are busy, the database CPU is at 90 percent, and adding more application servers makes things worse, not better. Redis exists because of that wall. This chapter explains where the wall comes from, so that when we introduce Redis you already know which problem it is aimed at.

1.1.1 Where the time goes in a request

Take a request to ShopStream, the online shop we will build throughout this book: a customer opens a product page. The browser asks the gateway, the gateway forwards the call to `catalog-service`, and that service has to produce a JSON document with the product name, price, stock level and a few reviews. A simplified timeline looks like this.

Where one product-page request spends its time (illustrative numbers)

```

Browser → Gateway → catalog-service → Database → catalog-service → Browser

network      routing      business code    SQL query      JSON mapping
20-60 ms     1-2 ms           1-3 ms          5-40 ms       1-2 ms
  
```

The database step is usually the largest slice, and it grows when the database is busy.

The numbers above are illustrative, not measurements, but the shape is realistic. Your own Java code is rarely the slow part. The slow part is almost always **waiting for something else**: a network hop, a disk read, a query that has to plan, scan an index, lock rows and serialise a result. When the database is idle, 5 ms is typical for a simple indexed lookup. When a few hundred requests arrive at the same moment, every one of them competes for the same connections, the same buffer pool and the same disk, and each query gets slower.

1.1.2 Latency and throughput are different problems

Two words come up constantly in this book, so we define them now.

Term	Meaning	Question it answers
Latency	How long one operation takes, from request sent to answer received	"How fast does this single page feel?"
Throughput	How many operations the system completes per second	"How many users can we serve at once?"

Latency and throughput.

They are related but not the same. A system can answer one request in 2 ms (low latency) and still collapse at 500 requests per second (low throughput) because requests queue behind each other. Redis helps with both: an in-memory lookup is very fast for one caller, and a server that answers each request in microseconds can serve a very large number of callers before a queue builds.

1.1.3 Read-heavy systems and repeated work

Most web workloads are heavily skewed toward reads, and within the reads a small set of items is requested over and over. A shop has thousands of products, but the home page, the ten best sellers and the current promotion are shown to nearly everyone. If one thousand visitors open the same product page in a minute, the database runs the same query one thousand times and returns the same answer one thousand times. That is **repeated work**, and repeated work is the cheapest thing to remove.

The standard remedy is a **cache**: a small, fast store that remembers the answer to a question you already paid to compute. The first request goes to the database and stores the result; the following 999 read the stored copy. Even a modest hit rate changes the economics. If 90 percent of reads are answered by the cache, the database sees one tenth of the load.

Without and with a cache (the arrows show who answers the read)



1.1.4 A worked example: what a cache does to the database

Numbers make the benefit concrete. Suppose **catalog-service** receives 2,000 product-page requests per second at peak, and each one needs one database query. Suppose the database can comfortably sustain 500 such queries per second before latency climbs. Without a cache, the database is four times over capacity; the service is effectively down.

Now put a cache in front and assume (illustratively) a 95 percent hit ratio. Only 5 percent of requests, 100 per second, reach the database, a fifth of its capacity. The average latency also improves: if a hit takes 1 ms and a miss takes 30 ms including the cache write, the average is $0.95 \times 1 + 0.05 \times 30 = 2.45$ ms, compared with 30 ms before.

Metric	No cache	95 percent hit ratio	80 percent hit ratio
Requests per second	2,000	2,000	2,000
Database queries per second	2,000	100	400
Average latency	30 ms	2.45 ms	7 ms
Database load vs 500 q/s capacity	400%	20%	80%

The same traffic with and without a cache (illustrative).

Two lessons follow. First, the hit ratio is the number that matters: going from 95 to 80 percent quadruples the database load. Second, a cache protects the database only while it is warm and healthy. If the cache restarts empty, all 2,000 requests per second hit the database at once. That "cold cache" and its cousins (stampede, penetration, avalanche) are the subject of the caching failure chapters in the Patterns Part.

1.1.5 Why not just keep a map in the application?

A natural first idea is a **ConcurrentHashMap** inside the Java process. It is the fastest possible cache, and for a single instance it can be a fine choice. It breaks down as soon as the system is real, for five reasons.

1. **Many instances.** You run four copies of **catalog-service**. Each has its own map, so each pays the cold-start cost separately, and they can disagree about the price of a mug for a few minutes.
2. **Invalidation.** When a price changes, you must remove the old value from every copy in every instance. A local map offers no way to do that.
3. **Memory.** The map lives inside the Java heap. A big cache means big garbage collection pauses, and every restart of the service empties it.
4. **Size and eviction.** You must write the logic for what to throw away when the map is too large. Getting that right is hard work.
5. **Shared state beyond caching.** Sessions, shopping carts, rate-limit counters and job queues all need state that every instance sees. A per-process map cannot provide that.

What we want is a fast, memory-based store that sits **outside** the application, is shared by every instance, understands expiry, and offers useful operations on the data (not just "get" and "put"). That is the gap Redis fills.

1.1.6 What Redis brings

In one sentence: Redis keeps data in memory, in rich data structures, on a server that many applications can talk to over the network, and it answers most commands in well under a millisecond. We will unpack each part of that sentence in the next two chapters. For now it is enough to see the kinds of problems it is used for.

Problem	ShopStream example	Redis feature used
Repeated expensive reads	Product details shown to many users	Strings or hashes with a TTL (cache)
State shared by all app instances	Login sessions, shopping carts	Hashes with expiry
Counting very fast	Page views, API calls per minute	Atomic INCR
Ranking	Best-selling products today	Sorted sets
Work handed from one service to another	"Order placed" events for the notifier	Lists or streams
Protecting a resource	Limit 100 requests per minute per client	Counters with expiry, Lua scripts
Coordination	Only one worker may process order 1001	Locks with SET ... NX

Problems that shaped Redis usage, with ShopStream examples.

1.1.7 Beyond caching

Redis spread from caching to many other jobs because a fast shared store with atomic operations solves coordination problems too. When four instances of **cart-service** must agree on how many requests a client has made in the last minute, they cannot each count locally; they need one place to increment. When **order-service** must tell **notification-service** that an order was placed, they need a shared place to put the message. In both cases the requirement is the one that made caches attractive: **a shared, very fast, in-memory place that everyone can reach**. The Patterns Part builds each of these roles carefully.

1.1.8 Common misconceptions

- **"Redis is just a cache."** It is often used as one, but it is a general data structure server with persistence, replication and messaging.
- **"A cache makes the system more reliable."** It makes the system faster. It can also make it more fragile, because a system that only survives load thanks to its cache fails when the cache does. Plan for a cold or missing cache.
- **"In-memory means data is lost on restart."** Only if you configure no persistence. With RDB or AOF a restart reloads the data set from disk (which takes time proportional to its size).
- **"Faster hardware solves the database problem."** It postpones it. A cache attacks repeated work directly, which scales better than buying a bigger database server.
- **"Redis is always sub-millisecond."** The command is; your end-to-end call includes the network, queueing behind other commands and your client library.

GOTCHA

Redis is not a magic "make it fast" switch. A cache adds a second copy of the data, and two copies can disagree. Every use of Redis in front of a database brings the problem of keeping them consistent. Later chapters (the caching chapters in the Patterns Part) are largely about managing exactly that trade-off. Use a cache because you measured a problem, not because caches are fashionable.

INTERVIEW ANGLE

Why do applications put a cache like Redis in front of a database? To reduce latency and database load for data that is read often and changes rarely. A lookup in memory over a fast network takes a fraction of a millisecond, whereas a database query involves query planning, index access and possibly disk I/O. The cost is a second copy of the data that must be expired or invalidated, so you accept bounded staleness in exchange for speed and lower load.

KEY POINTS

Key points

- Request time is dominated by waiting (network, disk, queries), not by your Java code.
- Latency is how long one operation takes; throughput is how many complete per second. Caching helps both.
- Read-heavy workloads repeat the same work; a cache removes the repetition.
- A map inside the JVM cannot be shared, invalidated centrally, or survive a restart; an external store can.
- A cache is a second copy of the data, so staleness and invalidation become your problem.

1.2 What Redis Is

1.2.1 A data structure server

The name **Redis** stands for *REmote DIctionary Server*. A dictionary (a map) is a collection of key-value pairs: give it a key, get back the value. Redis is that, running as a separate server process that you reach over a network connection. What sets it apart from a plain key-value store is that the value is not just a blob of bytes. A value can be a **string**, a **list**, a **hash** (a small map of fields), a **set**, a **sorted set**, a **stream**, a bitmap, a HyperLogLog, a geospatial index, and more. Each type comes with its own commands that work **inside the server**.

That last point matters. With a plain key-value store you fetch a whole value, change it in your program, and write it back. With Redis you say "add this member to the set", "increment this field", or "give me the top three scores", and the server does it atomically without moving the whole value across the network. Because of this, Redis is usually described as a **data structure server**.

String SET GET INCR	List LPUSH RPOP	Hash HSET HGET	Set SADD SINTER
Sorted set ZADD ZRANGE	Stream XADD XREADGROUP	Bitmap SETBIT BITCOUNT	HyperLogLog PFADD PFCOUNT
Geo GEOADD GEOSEARCH	JSON JSON.SET JSON.GET	Search + vectors FT.SEARCH	Prob. + TimeSeries BF.ADD TS.ADD

Figure 1.1. The Redis data types. Notice that every one of them lives under a key, and each has its own commands.

1.2.2 In-memory, with optional persistence

Redis holds its data set in RAM. Reading and writing memory takes nanoseconds, where an SSD read takes tens of microseconds and a spinning disk several milliseconds. This is the single largest reason for its speed (the next chapter looks at the others).

RAM is volatile, so the obvious question is what happens to the data when the process stops. Redis can write data to disk in two ways, which we cover in depth in the Part on Redis internals: **RDB** snapshots (a point-in-time copy of the whole data set) and the **AOF** (append-only file, a log of write commands). You choose whether to use none, one, or both. The data still lives in RAM while running; disk is only for recovery after a restart. This also means your data set must fit in memory, which is a design constraint worth remembering.

1.2.3 Clients, server and commands

Redis follows the classic client-server model. The server listens on TCP port **6379** by default. Any program that can open a socket can be a client: `redis-cli` (the command line tool that ships with Redis), your Spring Boot service, a Python script, a monitoring agent.

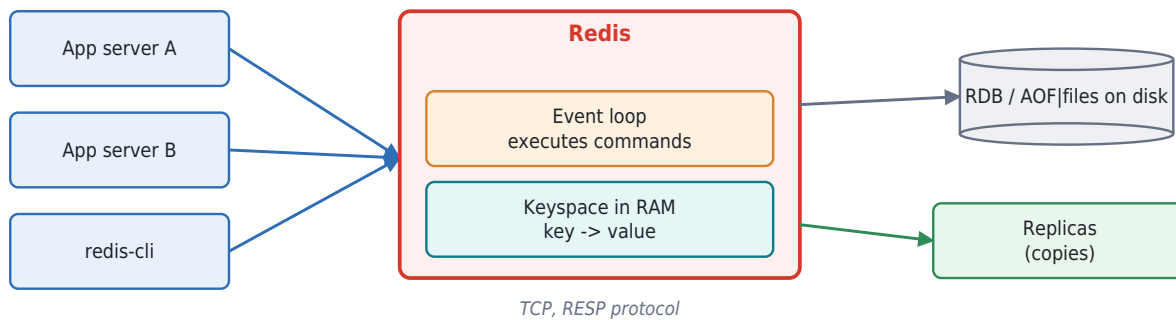


Figure 1.2. Redis in the middle: many clients connect over TCP, send commands, and read replies. Persistence files are written on the side.

A client sends a **command** (a name followed by arguments) and the server sends back a **reply**. That is the whole interaction model. Commands are short, upper-case by convention (though Redis accepts any case), and fast to read:

Your first commands

TERMINAL

```

127.0.0.1:6379> SET greeting "hello"
OK
127.0.0.1:6379> GET greeting
"hello"
127.0.0.1:6379> INCR visits
(integer) 1
127.0.0.1:6379> INCR visits
(integer) 2
127.0.0.1:6379> EXISTS greeting
(integer) 1
  
```

Notice what you did not have to do: create a database, define a table, declare a type. Redis creates a key the first time you write it. **INCR visits** created the key with value 0 and then added one. Redis is **schemaless**: it knows the type of each key (string, list, ...) but nothing about what you store inside.

1.2.4 Keys point at values

Every piece of data is reached through a **key**, which is a binary-safe string (any sequence of bytes, so **"user:42"** and an image hash are both fine). The key chooses the value; the value has a type; the type decides which commands work on it. Calling a list command on a string key is an error:

Types are enforced

TERMINAL

```

127.0.0.1:6379> SET greeting "hello"
OK
127.0.0.1:6379> LPUSH greeting "x"
(error) WRONGTYPE Operation against a key holding the wrong kind of value
127.0.0.1:6379> TYPE greeting
string
  
```

1.2.5 The protocol: RESP

How do the command and reply travel? Over TCP using a very simple text-friendly protocol called **RESP** (REdis Serialization Protocol). A command is sent as an array of strings, and each reply is tagged with a type character. You can see it by hand with **printf** and a socket, which is a good way to convince yourself there is no magic:

What goes over the wire for SET greeting hello (\r\n shown as visible)

```

Client sends: *3\r\n$3\r\nSET\r\n$8\r\ngreeting\r\n$5\r\nhello\r\n
               |           |                               |
               |           |                               |
               | "SET" is 3 bytes                         | "hello" is 5 bytes
               | an array of 3 elements
Server replies: +OK\r\n          (a "simple string")
  
```

The first byte of a RESP value tells you its type: **+** simple string, **-** error, **:** integer, **\$** bulk string (length-prefixed, binary safe), ***** array. Two versions exist. **RESP2** is the classic one and still the default for new connections. **RESP3**, enabled per connection with the **HELLO 3** command, adds richer types (maps, sets, doubles, booleans) and **push** messages,

which the server can send without being asked. Push messages make features like client-side caching possible; the chapter on RESP3 in the Redis internals Part covers it.

Because the protocol is this simple, client libraries exist for virtually every language. In Java you will mostly meet Lettuce, Jedis and Redisson (see the Java clients Part), and Spring Data Redis on top of them (see the Spring Part).

1.2.6 One server, many databases

A single Redis server has 16 numbered **logical databases** (0 to 15), selected with `SELECT n`. They are separate key spaces inside one process. Real systems rarely use them, for reasons covered in the chapter on keys. Remember only that the default is database 0 and that Redis Cluster supports database 0 alone.

1.2.7 Anatomy of a command

Every command has the same shape: a name, then arguments. Some arguments are keys, some are values, some are options (flags such as `NX` or `EX 60`). Two commands that do similar things can differ in what they return, so learn to read a reply. `redis-cli` prints each RESP type in a recognisable way:

Reply shown	RESP type	Example
OK	Simple string	<code>SET k v</code> replies OK
"hello"	Bulk string	<code>GET k</code>
(integer) 5	Integer	<code>INCR counter</code>
(nil)	Null	<code>GET missing</code>
1) "a" and 2) "b"	Array	<code>LRANGE list 0 -1</code>
(error) ERR ...	Error	A wrong argument or wrong type
(empty array)	Empty array	<code>LRANGE</code> on a missing key

How redis-cli displays replies.

Notice that a missing key does not raise an error for reads: `GET missing` returns `(nil)` and `LRANGE missing 0 -1` returns an empty array. A missing key behaves like an empty collection, and Redis deletes a collection automatically when its last element is removed, so you never have to create or clean up containers by hand. That convenience is also a reminder that Redis has no way to tell "empty" from "never existed" for collections.

The documentation for each command lists its **syntax**, **complexity**, which **version** introduced it, the **ACL categories** it belongs to, and the **reply** it returns. Get used to reading those boxes; they answer most "will this hurt in production?" questions.

1.2.8 A single node is a process, not a cluster

When we say "a Redis server", we mean one operating-system process, `redis-server`, with one set of data in memory. Everything else in this book (replication, Sentinel, Cluster) is built by running several such processes and having them cooperate. This matters when you reason about failure: if that one process dies, everything it held in memory is gone unless it was persisted, and until you add replicas nothing will take over.

1.2.9 Redis in one picture

Property	Value
Model	Key to typed value, with commands specific to each type
Storage	In RAM, optionally persisted to disk (RDB and/or AOF)
Process model	One server process; commands executed one at a time (next chapter)
Protocol	RESP over TCP, default port 6379
Extras	Replication, Sentinel, Cluster, Pub/Sub, Lua scripting, transactions
Built-in since 8.0	JSON, Query Engine (search), time series, probabilistic structures

Redis at a glance.

NOTE

Redis is not only a cache. Many people meet Redis as "the cache". That is the most common job, but sessions, queues, leaderboards, rate limiters, geo lookups and streams are just as typical. We will use Redis in all of these roles in ShopStream.

INTERVIEW ANGLE

What is Redis and how is it different from a relational database or Memcached? Redis is an in-memory data structure server: keys map to typed values (strings, lists, hashes, sets, sorted sets, streams and more) manipulated by server-side commands. Unlike a relational database it has no tables, joins or SQL and keeps its working set in RAM. Unlike Memcached, which stores opaque values, it understands structure, supports persistence and replication, and offers atomic operations on the data.

KEY POINTS

Key points

- Redis is a server holding key-value data in RAM; values are typed data structures with their own commands.
- Commands run on the server, so you modify data without shipping it back and forth.
- Persistence (RDB, AOF) is optional and exists for recovery; the live data set must fit in memory.
- Clients speak RESP over TCP (default port 6379); RESP3 adds richer types and server push.
- Redis is schemaless: it enforces the type of a key, nothing about its contents.

1.3 Why Redis Is Fast

Redis routinely answers simple commands in well under a millisecond, and a single server on ordinary hardware can handle very large numbers of commands per second. (Exact figures depend on hardware, command mix, value size and network, so measure your own with `redis-benchmark`; we do that in a lab.) This speed is not one trick. It is the product of several design decisions that reinforce each other. Understanding them helps you predict when Redis is fast and, just as important, when it is not.

1.3.1 Reason 1: the data lives in RAM

The first reason is the obvious one. A lookup in main memory costs on the order of 100 nanoseconds. Reading a record from an SSD costs tens of microseconds, hundreds of times slower, and a random read from a spinning disk costs milliseconds. Redis never has to go to disk to answer a read. Even writes are fast because, by default, the write is applied to memory and the disk work (if you enabled persistence) happens in the background or in batches.

1.3.2 Reason 2: efficient data structures

Redis is written in C, and each data type is backed by a structure chosen for its commands. A hash lookup in a hash table is $O(1)$. Adding to the head or tail of a list is $O(1)$. A sorted set uses a skip list plus a hash table, so adding a score is $O(\log N)$ and reading a rank is $O(\log N)$. Small collections use a compact memory layout called a **listpack** that is friendly to the CPU cache. Every command in the official documentation states its time complexity, and it is the most useful thing to check before you use a command on a large collection. The chapter on memory layout in the Redis internals Part opens these structures up.

1.3.3 Reason 3: a single-threaded command loop

Redis executes commands on **one main thread**, one command at a time. For many developers this sounds like a weakness. It is actually the heart of Redis's simplicity and speed.

- **No locks.** If only one thread touches the data, there is nothing to lock. No lock contention, no deadlocks, no context switches between threads fighting over a hash table.
- **Atomic by construction.** Because commands never interleave, every command (and every Lua script or `MULTI/EXEC` block) is atomic. `INCR` can never lose an update.
- **Cache friendly.** One thread working through a single data set keeps the CPU caches warm.
- **Simple to reason about.** The behaviour of the server is easy to predict, which is why Redis is so stable.

The trade-off is plain: **while one command runs, all others wait**. A command that takes 10 ms stalls everybody for 10 ms. This is why you must avoid slow commands on large collections (for example `KEYS *` on millions of keys, or `SMEMBERS` on a huge set), and why we repeatedly say "O(N) commands are dangerous at scale".

1.3.4 Reason 4: I/O multiplexing

If Redis uses one thread, how does it serve thousands of connected clients? Not by dedicating a thread to each (that is the model that burdens traditional servers), but by **I/O multiplexing**. The operating system lets one thread ask "which of these ten thousand sockets have something to read?" through a call such as `epoll` on Linux, and the thread then handles only the sockets that are ready. Redis wraps this in a small event library and runs an **event loop**:

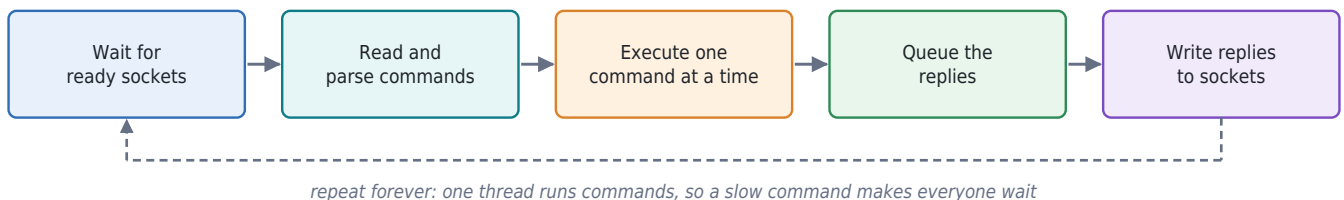


Figure 1.3. The Redis event loop: wait for ready sockets, read and parse commands, execute each one, queue the replies, write them back.

In plain steps, one turn of the loop looks like this:

1. Ask the OS which client sockets are readable.
2. Read the bytes for each ready client and parse them into commands.
3. Execute each command against the in-memory data set (the only step that touches data).
4. Put each reply in that client's output buffer and write the buffers to the sockets.
5. Run time-based chores (expiring keys, other cron-like work), then go back to step 1.

Since version 6.0, Redis can optionally use extra **I/O threads** to read and write sockets and parse commands in parallel, because network I/O can become the bottleneck at high rates. Command execution itself stays on the main thread, so the no-locks and atomicity properties remain. Version 8.0 reworked this I/O threading; the Redis internals Part discusses the details and how to configure `io-threads`. A handful of operations also run in **background threads**: closing files, flushing the AOF to disk and freeing large objects lazily.

1.3.5 What "one command at a time" means in practice

You can see the single thread at work with the slow log. Redis records any command whose execution time exceeds a threshold (10 milliseconds by default, set by `slowlog-log-slower-than` in microseconds) and keeps the most recent entries. A command on a million-element collection shows up like this:

Catching a slow command
TERMINAL

```

127.0.0.1:6379> SLOWLOG GET 1
1) 1) (integer) 14
   2) (integer) 1760000100
   3) (integer) 41873
   4) 1) "SMEMBERS"
      2) "tags:all"
   5) "127.0.0.1:52144"
   6) ""

```

The third item, `41873`, is the execution time in microseconds: about 42 ms. For those 42 ms no other client got a reply. A thousand clients each waiting for 1 ms of work do not notice a 1 ms command; they do notice a 42 ms one. The fix is almost always to use an incremental command (`SSCAN` instead of `SMEMBERS`), a smaller structure, or to move the heavy read elsewhere.

1.3.6 Reading the complexity column

The documentation labels each command with its time complexity, where **N** is the size of the collection and **M** the number of elements touched. A small table of the most important categories:

Label	Meaning	Example commands
O(1)	Cost does not depend on size	GET, SET, HGET, SADD, LPUSH
O(log N)	Grows slowly	ZADD, ZRANK
O(log N + M)	Seek, then read M items	ZRANGE with a small range
O(N)	Cost grows with size: dangerous on large collections	LRANGE 0 -1, HGETALL, SMEMBERS, KEYS
O(N) with N being the whole key space	Walks everything	KEYS, FLUSHALL

What the complexity labels mean for you.

A rule of thumb that will save you incidents: an O(N) command is fine when N is bounded and small (a 20-field hash, a 100-item cart) and a problem when N can grow without limit. Bound your collections.

1.3.7 Reason 5: a lean protocol and pipelining

RESP is cheap to parse: lengths are prefixed, so the server never scans for delimiters in binary data. Beyond that, Redis supports **pipelining**, and understanding it explains many real-world performance problems.

A normal call is a round trip: the client sends a command, waits, then reads the reply. Even if Redis answers in 20 microseconds, the packet must cross the network. On a local network a round trip often takes around 0.2 to 0.5 ms (illustrative), which is ten or more times the cost of the command itself. If you issue 1,000 commands one after another, you pay 1,000 round trips.

With a **pipeline** the client sends many commands without waiting for individual replies, then reads all the replies at the end. You pay roughly one round trip for the whole batch.

Round-trip cost: 1,000 commands, 0.3 ms per round trip (illustrative)

```
One at a time: 1,000 x (0.3 ms network + 0.02 ms Redis) ≈ 320 ms
Pipelined:    1 x 0.3 ms network + 1,000 x 0.02 ms Redis ≈ 20 ms
```

Here is the same idea in the shell. `redis-cli --pipe` and `redis-benchmark -P` use it; a lab later in the book measures the difference on your machine.

A tiny pipeline from the shell

TERMINAL

```
$ printf 'SET a 1\r\nSET b 2\r\nINCR a\r\nGET b\r\n' | redis-cli --pipe
All data transferred. Waiting for the last reply...
Last reply received from server.
errors: 0, replies: 4
```

Pipelining reduces network waiting, but it does not make commands atomic: other clients' commands may run between yours. Atomicity is the job of **MULTI/EXEC** and Lua scripts. The Java clients Part shows how to pipeline from Java clients.

1.3.8 Pipelining from Java

You will use a client library rather than raw sockets, and every serious one exposes pipelining. With Spring Data Redis it looks like this. The `executePipelined` call queues every command, sends them together and returns the replies in order.

PipelineDemo.java

JAVA

```
package com.shopstream.demo;

import java.util.List;
import org.springframework.data.redis.core.RedisCallback;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Component;

@Component
public class PipelineDemo {

    private final StringRedisTemplate redis;
```

PipelineDemo.java (continued)

JAVA

```

public PipelineDemo(StringRedisTemplate redis) {
    this.redis = redis;
}

public List<Object> loadStock(List<String> skus) {
    RedisCallback<Object> batch = connection -> {
        for (String sku : skus) {
            connection.stringCommands().get(("stock:" + sku).getBytes());
        }
        return null;
    };
    return redis.executePipelined(batch);
}
}

```

What to notice: the loop issues one **GET** per SKU but nothing is sent one by one; the client flushes the batch and waits once. The method returns a list with one reply per command, in the order issued. The Spring Part returns to templates, serializers and the callback types in detail.

1.3.9 Measuring it yourself

Do not trust any number in this chapter, including the illustrative ones. Redis ships a benchmark tool, **redis-benchmark**, that makes the effect visible in seconds. The lab on benchmarking in the next Part walks through it; here is the idea:

Pipeline depth changes throughput

TERMINAL

```

$ redis-benchmark -t set,get -n 100000 -q
SET: 85000.00 requests per second
GET: 90000.00 requests per second
$ redis-benchmark -t set,get -n 100000 -q -P 16
SET: 650000.00 requests per second
GET: 800000.00 requests per second

```

The figures above are examples of the shape of the result (many times higher with **-P 16**), not benchmarks of any machine. Run both commands yourself and compare.

1.3.10 Putting the speed in context

Operation	Typical order of magnitude
Read from RAM	~100 ns
Redis command execution (simple)	~1 to 20 µs
Network round trip inside a data centre	~0.1 to 0.5 ms
SSD random read	~20 to 100 µs
Database query with index (warm)	~1 to 10 ms
Round trip across continents	~100 ms and up

Latency scale (order-of-magnitude, illustrative).

Notice that for a client in your own network the **round trip dominates**. This is why pipelining, connection reuse and keeping Redis close to the application matter as much as the speed of the server itself.

1.3.11 Honest limits

- **Slow commands block everyone.** The single thread is the shared resource. Large **O(N)** commands, huge Lua scripts and very large values all become latency spikes for every client.
- **One main thread means one core for commands.** A single Redis process cannot use all cores to execute commands. To use more cores you run more processes: shards in Redis Cluster, or several instances.
- **Memory is the limit.** The data set must fit in RAM (and you need headroom, for example for fork during snapshots).
- **The network is the floor.** The client cannot be faster than its round trip.

- **Persistence work can add pauses.** Taking a snapshot forks the process, and with a large data set the fork itself can stall for a short time. We examine this under persistence.

WATCH OUT

Never run **KEYS *** or **MONITOR** on a busy production server. **KEYS** walks every key in one uninterruptible command, stalling all clients for as long as it takes. **MONITOR** streams every command to you and can cut throughput substantially. Use **SCAN** to iterate keys, and use **SLOWLOG** or the latency tools to investigate.

INTERVIEW ANGLE

If Redis is single-threaded, how can it be so fast, and how does it handle many clients? Data is in RAM, structures are efficient, and with one thread there are no locks or context switches. An event loop built on I/O multiplexing (epoll) lets that one thread serve thousands of sockets by handling only those that are ready. Since 6.0 optional I/O threads help with network reads and writes, but commands still execute on the main thread. The cost is that a slow command blocks everyone.

KEY POINTS

Key points

- Speed comes from RAM, good data structures, a lock-free single execution thread, I/O multiplexing and a lean protocol.
- Commands execute one at a time, which makes every command atomic and removes locking.
- I/O threads (optional since 6.0) parallelise socket work, not command execution.
- Pipelining batches many commands per round trip; it improves throughput but is not atomic.
- Slow **O(N)** commands, huge values and long scripts hurt all clients. Always check a command's complexity.

1.4 Keys, Values, Types and Naming

Because every operation starts with a key, how you name and organise keys is the first design decision in any Redis system. Bad key design is hard to fix later because keys are scattered through application code, scripts and dashboards. This chapter gives you a practical set of rules.

1.4.1 What a key is

A key is a **binary-safe string**: any bytes from an empty string up to 512 MB. In practice you want short, readable, ASCII keys. The same applies to values of the string type, which are also limited to 512 MB, although values that large are almost always a mistake (see "big keys" in the operations Part).

Keys are **case sensitive** (**User:1** and **user:1** are different keys) and there is no namespace or table concept: all keys in a database sit side by side. You create your own namespaces through naming.

1.4.2 Naming convention: colon-separated segments

The community convention, and the one used in this book, is **object-type:id:field**, with colons separating the segments:

Key naming in ShopStream

product:42	the catalogue entry for product 42 (hash or JSON)
product:42:stock	a counter for the stock of product 42
session:9f8a1c	a login session (hash with TTL)
cart:1001	the shopping cart of user 1001 (hash)
rl:203.0.113.7:/api/cart	a rate-limit counter per IP and route
lb:bestsellers:daily	today's best-seller leaderboard (sorted set)
stream:orders	the order-event stream
lock:order:5521	a lock for processing order 5521

The colon is a convention, not a feature; Redis treats it as an ordinary character. Its value is that tools, humans and SCAN patterns can group by prefix (**product:***), that RedisInsight shows keys as a folder tree, and that you avoid collisions between teams. Some practical rules:

- **Be consistent.** Document the key schema like you document a database schema, ideally in one Java class (a **Keys** helper) so the format is written down exactly once.

- **Put the most general segment first**, then narrow down: `cart:1001`, not `1001:cart`.
- **Keep keys short but readable**. Every key is stored in memory, and with ten million keys, 20 extra bytes each is 200 MB. Do not go cryptic either: `u:1:n` saves a few bytes and costs hours of debugging.
- **Do not put unbounded or sensitive data in keys**. Raw user input, email addresses or tokens in keys leak into logs, `MONITOR` output and slow-log entries.
- **Version keys when the format changes**. For cached JSON, `product:v2:42` lets you deploy a new format without reading stale old-format values.

A Java helper keeps the convention in one place:

```

Keys.java
package com.shopstream.common;

public final class Keys {

    private Keys() {
    }

    public static String product(long id) {
        return "product:" + id;
    }

    public static String cart(long userId) {
        return "cart:" + userId;
    }

    public static String rateLimit(String ip, String route) {
        return "rl:" + ip + ":" + route;
    }
}

```

1.4.3 A worked key design: the product catalogue

Suppose `catalog-service` must cache products, their stock and a "related products" list. A sensible first layout:

Key	Type	Why
<code>product:{id}</code>	Hash	Name, price, description fields; one key per product, TTL 10 minutes
<code>product:{id}:stock</code>	String counter	Changes far more often than the rest, so it is separate and has its own lifetime
<code>product:{id}:related</code>	List	Ids of related products; short TTL
<code>category:{slug}:products</code>	Sorted set	Product ids ordered by popularity, for category pages

Key design for the catalogue (a draft you could review in a team).

Look at the choices. Splitting `stock` out means a stock change does not force you to rewrite or invalidate the whole product. Keeping related products in their own key means a bad recommendation run only affects one key family. Each family gets its own TTL. This is the habit to build: **group data that is read and expired together, separate data that changes at different rates**.

1.4.4 Key size and memory cost

Each key costs memory twice: for its bytes and for the bookkeeping Redis keeps per entry (a few dozen bytes of structure, which varies by version). Ten million keys of the form `product:1234567:stock` use a few hundred megabytes before counting any values. Two practical consequences: prefer fewer, richer keys when the access pattern allows it (one hash holding ten fields costs less than ten string keys), and do not store a million tiny values as separate keys if one hash or a bitmap would do. The memory chapters in the Redis internals Part show how to measure with `MEMORY USAGE`.

1.4.5 A preview: keys and Cluster

If you later run Redis Cluster, keys decide which shard stores them, through a hash of the key. A multi-key command only works if all its keys live on the same shard. You can force related keys together by putting a **hash tag** in braces: `{user:1001}:cart` and `{user:1001}:session` hash on the text `user:1001` only. You do not need this now, but it is a reason to think of key names as a design asset from the very first day.

1.4.6 Value types and why they shape keys

Choosing the type is the other half of key design. You can store a product as one string holding JSON, as a hash with a field per attribute, or as a set of separate keys. Each choice has different costs and abilities, and the chapter on choosing the right structure in the data structures Part gives a decision table. The immediate point: **a key has exactly one type**, and `TYPE key` tells you which. Writing a different type under an existing key with a command like `SET` replaces it.

1.4.7 Time to live (TTL) basics

Every key may carry an **expiry**, after which Redis deletes it automatically. This is the feature that makes Redis a natural fit for caches, sessions and temporary tokens. You set it with `EXPIRE`, with `SET ... EX`, or at the same moment as the write:

```
Setting and reading a TTL TERMINAL
127.0.0.1:6379> SET session:9f8a1c "user:1001" EX 1800
OK
127.0.0.1:6379> TTL session:9f8a1c
(integer) 1798
127.0.0.1:6379> EXPIRE session:9f8a1c 3600
(integer) 1
127.0.0.1:6379> PERSIST session:9f8a1c
(integer) 1
127.0.0.1:6379> TTL session:9f8a1c
(integer) -1
127.0.0.1:6379> TTL no:such:key
(integer) -2
```

`TTL` returns the remaining seconds, `-1` for a key that exists with no expiry, and `-2` if the key does not exist. A subtle but important fact: **overwriting a key with a plain `SET` removes its TTL**, so a session refreshed with a naive `SET` becomes immortal. Use `SET key value EX seconds` again, or `KEEPTTL`, depending on what you want. The expiry chapter in the Redis internals Part explains how Redis actually removes expired keys (lazily and in the background).

WATCH OUT

Keys without a TTL grow forever. If you use Redis as a cache and forget TTLs, memory usage creeps up until `maxmemory` is reached or the server runs out of RAM. Decide for every key family whether it expires, and write that decision down next to its name.

1.4.8 Database numbers: usually leave them alone

Redis offers `SELECT 0` to `SELECT 15`. They look like separate databases but are not isolated: they share one process, one memory budget, one set of persistence files, and a slow command in database 3 blocks database 5. Redis Cluster supports only database 0. Therefore the usual advice is: **use database 0 and separate applications with key prefixes or separate Redis instances**. The one place you will use a database number deliberately is tests, where an isolated database makes clean-up easy (and even there a throwaway container is better).

1.4.9 Environments, tenants and shared servers

Teams often share a Redis server between environments or services. Two sound options exist: separate instances (best isolation, simplest ACLs) or a leading prefix. If you use a prefix, put it first and configure it in one place:

```
application.yml YAML
shopstream:
  redis:
    key-prefix: "shopstream:prod:"
```

With that, a key is `shopstream:prod:cart:1001`. The prefix also lets an ACL user be restricted to `shopstream:prod:*` keys (covered under security), so a bug in a test environment cannot overwrite production data. Spring's cache support applies its own prefix (the cache name) automatically; a later chapter shows how to configure it.

1.4.10 Common key-design mistakes

Mistake	Better
Whole user table in one hash key	One key per user; one huge key is slow to read, expire and delete
Timestamps or random ids in the middle of a cached key (never reused)	Use stable ids so the key can be hit again
Same TTL on millions of keys created at once	Add random jitter so they do not all expire together
Keys built from unsanitised input	Validate and bound the segments
No documented schema	Keep a <code>Keys</code> class and a table like the one above
Using a key as a queue of one item	Use a list or stream

Mistakes seen again and again, and the fix.

1.4.11 Listing keys: SCAN, not KEYS

Beginners ask "how do I list all keys?" The wrong answer is `KEYS pattern`, which walks the whole key space in one blocking call. The right answer is `SCAN`, which returns a small batch plus a `cursor` you pass back to continue until the cursor returns to 0:

Iterating with SCAN

TERMINAL

```
127.0.0.1:6379> SCAN 0 MATCH product:* COUNT 100
1) "0"
2) 1) "product:42"
   2) "product:43"
```

`COUNT` is a hint about the work per call, not a guarantee of the number of keys returned, and `MATCH` filters after the batch is chosen, so a call can legitimately return zero keys with a non-zero cursor. `SCAN` can show a key more than once, so make processing idempotent. The data structures Part has a full chapter on safe iteration.

TIP

Inspecting a key quickly: `TYPE key`, `TTL key`, `OBJECT ENCODING key` and `MEMORY USAGE key` answer "what is this, when does it expire, how is it stored and how big is it?" in four commands.

INTERVIEW ANGLE

How should you design keys in Redis? Use a consistent, readable scheme such as `entity:id:field` with colons, keep keys reasonably short, put a TTL on anything that is not permanent, avoid unbounded user input in keys, and version them when formats change. Avoid multiple database numbers; use prefixes or separate instances. Iterate with `SCAN`, never `KEYS`, in production.

KEY POINTS

Key points

- Keys are binary-safe strings up to 512 MB; keep them short, readable and consistent: `entity:id:field`.
- A key holds exactly one type; commands for the wrong type return `WRONGTYPE`.
- Give every temporary key a TTL; a plain `SET` over an existing key clears its TTL.
- Use database 0 and prefixes; the numbered databases share one process and do not exist in Cluster.
- Iterate with `SCAN`, not `KEYS`.

1.5 The First Ten Minutes: A Tour of the Types

This chapter is a quick, hands-on walk through the main data types, one command at a time, so that the later Parts do not feel abstract. If you have no Redis yet, the next Part (Hands-On Labs) shows how to install it in a few minutes with Docker; you can also read this chapter first and run the commands afterwards. All sessions use `redis-cli`, which prints a prompt of `127.0.0.1:6379>`.

1.5.1 Strings: values and counters

A string is the simplest type: bytes, text or numbers. If the content looks like an integer, you can do arithmetic on it atomically.

```
Strings TERMINAL
127.0.0.1:6379> SET product:42:name "Ceramic Mug"
OK
127.0.0.1:6379> GET product:42:name
"Ceramic Mug"
127.0.0.1:6379> SET product:42:stock 40
OK
127.0.0.1:6379> DECRBY product:42:stock 3
(integer) 37
127.0.0.1:6379> SET lock:order:5521 worker-1 NX EX 30
OK
127.0.0.1:6379> SET lock:order:5521 worker-2 NX EX 30
(nil)
```

`NX` means "only set if the key does not exist"; the second call fails with `(nil)`. That one command, with a TTL, is the foundation of simple distributed locks.

1.5.2 Lists: ordered, pushable at both ends

```
Lists TERMINAL
127.0.0.1:6379> LPUSH recent:searches:1001 "mug" "teapot"
(integer) 2
127.0.0.1:6379> LRANGE recent:searches:1001 0 -1
1) "teapot"
2) "mug"
127.0.0.1:6379> LTRIM recent:searches:1001 0 9
OK
```

`LPUSH` puts new items at the head, `LRANGE 0 -1` reads the whole list, and `LTRIM 0 9` keeps only the first ten: a "recent searches" list that cannot grow without bound.

1.5.3 Hashes: small objects

```
Hashes TERMINAL
127.0.0.1:6379> HSET product:42 name "Ceramic Mug" price 1299 stock 40
(integer) 3
127.0.0.1:6379> HGET product:42 price
"1299"
127.0.0.1:6379> HINCRBY product:42 stock -1
(integer) 39
127.0.0.1:6379> HGETALL product:42
1) "name"
2) "Ceramic Mug"
3) "price"
4) "1299"
5) "stock"
6) "39"
```

A hash maps field names to values inside one key. It is the natural way to store an object, and you can change one field without rewriting the rest.

1.5.4 Sets: unique, unordered

```
Sets
127.0.0.1:6379> SADD tags:product:42 kitchen ceramic gift
(integer) 3
127.0.0.1:6379> SADD tags:product:42 gift
(integer) 0
127.0.0.1:6379> SISMEMBER tags:product:42 ceramic
(integer) 1
```

TERMINAL

Adding `gift` again returns `0` because a set keeps each member once. Sets also offer union, intersection and difference across keys.

1.5.5 Sorted sets: members ranked by score

```
Sorted sets
127.0.0.1:6379> ZADD lb:bestsellers:daily 120 sku-1 95 sku-2 40 sku-3
(integer) 3
127.0.0.1:6379> ZINCRBY lb:bestsellers:daily 10 sku-2
"105"
127.0.0.1:6379> ZRANGE lb:bestsellers:daily 0 1 REV WITHSCORES
1) "sku-1"
2) "120"
3) "sku-2"
4) "105"
```

TERMINAL

Every member has a score and the set stays ordered by it. `REV` reads from the highest score. This is the leaderboard structure.

1.5.6 Streams, HyperLogLog, geo, bitmaps

```
A taste of the other types
127.0.0.1:6379> XADD stream:orders * orderId 1001 total 2599
"1760000000000-0"
127.0.0.1:6379> PFADD uv:2026-10-10 user:1 user:2 user:1
(integer) 1
127.0.0.1:6379> PFCOUNT uv:2026-10-10
(integer) 2
127.0.0.1:6379> GEOADD geo:stores 77.5946 12.9716 blr-central
(integer) 1
127.0.0.1:6379> SETBIT dau:2026-10-10 1001 1
(integer) 0
127.0.0.1:6379> BITCOUNT dau:2026-10-10
(integer) 1
```

TERMINAL

The stream returns a generated ID (a millisecond timestamp and a sequence, so yours will differ). The HyperLogLog counts distinct members approximately using a tiny fixed amount of memory; the bitmap stores one bit per user ID.

1.5.7 Atomicity in practice: a tiny flash sale

Before the map, one more experiment that shows why Redis is used for counters. Imagine 100 customers click "buy" on the last item at the same moment. If the application reads the stock, checks it, and writes it back, two customers can read `1` and both succeed (a **race condition**). With Redis, the decrement itself is atomic, so each caller gets a distinct answer:

```
Atomic decrement
127.0.0.1:6379> SET stock:sku-7 1
OK
127.0.0.1:6379> DECR stock:sku-7
(integer) 0
127.0.0.1:6379> DECR stock:sku-7
(integer) -1
```

TERMINAL

The first caller sees `0` (it got the last item). The second sees `-1` and knows it lost, so it must undo its decrement or the application must avoid going negative. A real flash sale solves this properly with a Lua script that checks and decrements in one step, which the Patterns Part and a project build in full. The point here is that **atomic server-side operations remove whole classes of race conditions**.

1.5.8 Try it yourself

Without looking back, do these on a throwaway Redis, predicting each reply before pressing Enter.

1. Create `cart:1001` as a hash with two products and quantities, then increase one quantity by 2.
2. Give the cart a TTL of 7 days and read it back. What does `TTL` return after `PERSIST`?
3. Add three product ids to a list `recent:viewed:1001`, then keep only the two most recent.
4. Create a sorted set of three SKUs with sales counts and read the top two.
5. Call `LPUSH` on a string key and read the error. Which command shows the key's type?

1.5.9 A one-page map

Type	Example command	Typical ShopStream use	Cost
String	SET, GET, INCR	Cache entry, counter, lock	O(1)
List	LPUSH, RPOP, LRange	Recent searches, simple queue	O(1) at ends
Hash	HSET, HGET, HINCRBY	Product, session, cart	O(1) per field
Set	SADD, SISMEMBER, SINTER	Tags, unique buyers	O(1) per member
Sorted set	ZADD, ZRange, ZINCRBY	Best sellers, delay queue	O(log N)
Stream	XADD, XREADGROUP, XACK	Order events with consumer groups	O(1) append
HyperLogLog	PFADD, PFCOUNT	Unique visitors per day	O(1), 12 KB max
Geo	GEOADD, GEOSEARCH	Nearest store	O(log N)
Bitmap	SETBIT, BITCOUNT	Daily active users	O(1) per bit

One command per type, and its main use.

1.5.10 Inspecting what you made

```

Type, encoding and cleaning up
127.0.0.1:6379> TYPE product:42
hash
127.0.0.1:6379> OBJECT ENCODING product:42
"listpack"
127.0.0.1:6379> DEL product:42 tags:product:42
(integer) 2
127.0.0.1:6379> DBSIZE
(integer) 12

```

The `OBJECT ENCODING` result shows that a small hash is stored compactly as a listpack; it switches to a hash table when it grows. You will use this command often when studying memory.

TIP

When you meet a command you do not know, `redis-cli` can explain it: type `HELP SET` or `HELP @sorted_set`. The official site documents each command with its complexity and an example.

KEY POINTS

Key points

- Each type has its own commands; pick the type by what you need to do with the data.
- `SET ... NX EX` writes only if absent and attaches a TTL in one atomic step.
- `LTRIM` after `LPUSH` caps a list; `HINCRBY` and `INCR` are atomic counters.
- `OBJECT ENCODING`, `TYPE` and `TTL` are your inspection tools.
- The labs in the next Part let you repeat all of this and go further.

1.6 Guarantees and Trade-offs

Every data store makes promises, and every promise has a price. Before you trust Redis with a piece of data you should know exactly what it promises, what it quietly does not, and which settings move the line. This chapter is a map; the later Parts on persistence, replication and clusters fill in the mechanics.

1.6.1 Atomicity and isolation

Redis gives you strong guarantees at the level of a **single command**. Because one thread executes commands one at a time, every command is atomic and isolated: another client never sees a half-finished **HSET** or loses an **INCR**. The same is true for a whole Lua script and for a **MULTI/EXEC** block, whose commands run back to back with nothing in between.

What Redis does **not** do is roll back. If a command inside a **MULTI/EXEC** block fails at run time (say, **INCR** on a non-numeric string), the other commands still run and keep their effects. A script that fails halfway keeps the writes it already made. Design operations so that a partial result is acceptable or detectable. We return to this in the chapters on transactions and scripting.

1.6.2 Durability: how much can you lose?

Durability means that once Redis tells you a write succeeded, the write survives a crash. For Redis this depends entirely on your persistence configuration, and the honest summary is that the default is not what a database person would call durable.

Setup	Data you may lose	Notes
No persistence	Everything on restart	Fine for a pure cache
RDB snapshots only	Writes since the last snapshot (minutes)	Compact files, fast restart
AOF with appendfsync everysec	About the last second of writes	A common production choice
AOF with appendfsync always	Almost nothing acknowledged	Slower; each write waits for fsync

What a process crash or power loss can cost (guidance; details in the persistence chapters).

Redis ships with RDB snapshotting enabled by default and the AOF disabled, so a fresh installation can lose several minutes of writes on a crash. If Redis holds data that you cannot rebuild, enable the AOF and decide the fsync policy deliberately.

1.6.3 Replication is asynchronous

Redis scales reads and survives machine failure by copying data from a **primary** to one or more **replicas**. That copying is **asynchronous** by default: the primary acknowledges your write to the client first and sends it to replicas afterwards.

The window in which an acknowledged write can disappear

```

Client — SET x 1 —> Primary — "OK" —> Client      (write acknowledged)
                        |
                        | (a moment later) —> Replica (not there yet)

```

If the primary crashes inside that window and the replica is promoted, the write is gone although the client was told OK.

Consequences you must plan for:

- A failover (Sentinel or Cluster promoting a replica) can lose the most recent acknowledged writes.
- A read from a replica can be **stale**: it may not yet show a write you just made on the primary.
- The **WAIT** command makes a client block until a given number of replicas have acknowledged the write, which narrows the loss window but does not turn Redis into a strongly consistent store, because a failover can still choose a replica that missed the write in some failure scenarios.
- **min-replicas-to-write** and **min-replicas-max-lag** make a primary refuse writes when too few replicas are reachable, trading availability for a smaller loss window.

1.6.4 Consistency and the CAP view

The **CAP theorem** says that when a network partition splits a distributed system, you must choose between **consistency** (every read sees the latest write) and **availability** (every request gets an answer). A single Redis node has no partition problem; it is simply consistent. Once you add replicas, Sentinel or Cluster, you have a distributed system with asynchronous replication, and the documentation is candid that Redis Cluster does not guarantee strong consistency: under partitions it can accept writes on a minority side that will later be discarded. In practice Redis leans toward **availability and speed** over strict consistency, and offers knobs (`WAIT`, `min-replicas-to-write`, cluster settings) that move you part of the way back.

That is the right default for caches, sessions and counters, where a rare lost write is acceptable. It is the wrong default for a bank ledger.

1.6.5 Memory is the capacity limit

All data lives in RAM, so capacity is bounded by memory, and memory costs more per gigabyte than disk. You decide what happens when memory runs out through `maxmemory` and `maxmemory-policy`: by default (`noeviction`) writes start to fail with an error; with an eviction policy, Redis deletes keys to make room. Using Redis as a cache means choosing an eviction policy; using it as a store means making sure it never gets there.

1.6.6 A worked failover: what is lost?

Imagine a primary with one replica, using the default asynchronous replication, and these events within a few milliseconds:

Timeline of a failover with asynchronous replication

```
t0 Client A: INCR stock:sku-7      → primary replies 37, sends to replica
t1 Client B: INCR stock:sku-7      → primary replies 38, replication not yet sent
t2 Primary crashes
t3 Sentinel promotes the replica; it has applied the t0 write but not t1
t4 stock:sku-7 on the new primary says 37; Client B was told 38
```

Client B's acknowledged write has vanished and the counter has moved backwards. For a view counter nobody cares. For stock, it can mean overselling by one item. The mitigations are all trade-offs: reconcile with the database after a failover, use `WAIT 1 100` on writes that matter, require replicas with `min-replicas-to-write`, or keep the authoritative number in a durable database. The right choice depends on what a lost write costs.

1.6.7 Five questions before you store data in Redis

1. If this data disappeared right now, could I rebuild it, and how long would that take?
2. How much staleness is acceptable (seconds, minutes, never)?
3. How big will it grow, and does it have a TTL or another bound?
4. Do I need to find it by something other than its key?
5. What happens to the application if Redis is unreachable for a minute?

1.6.8 The guarantees at a glance

Feature	Guarantee	Caveat
Single command	Atomic and isolated	None
Lua script / Function	Atomic and isolated	A long script blocks all clients; no rollback
<code>MULTI/EXEC</code>	Commands run together without interleaving	No rollback; <code>WATCH</code> gives optimistic checks
Pipelining	Order of replies preserved	Not atomic
AOF <code>everysec</code>	At most about a second lost on crash	Disk stalls can delay it
Replication	Eventually consistent copy	Asynchronous; replicas can be stale
Pub/Sub	At-most-once delivery	No storage; absent subscribers miss messages
Streams with groups	At-least-once with acknowledgements	Consumers must handle duplicates
Cluster	Automatic failover and sharding	Not strongly consistent under partitions

What you can rely on, by feature (defaults; each is tunable).

Keep this table in mind; most "Redis lost my data" stories are a mismatch between a default and an expectation listed in one of these rows.

1.6.9 What Redis is not

- **Not a replacement for your system of record by default.** An order, a payment or a customer account belongs in a database with transactions, constraints, point-in-time recovery and years of operational tooling. Redis holds copies, derived data, temporary state and fast counters.
- **Not a general query engine.** You look data up by key. There are no joins, no ad hoc **WHERE**. The built-in Query Engine (see the data structures Part) adds secondary indexes and search for the data you choose to index, but it does not turn Redis into a relational database.
- **Not infinite.** Datasets of terabytes are expensive in RAM.
- **Not a message broker with every feature.** It has lists, Pub/Sub and streams; it does not replace the long retention and ecosystem of Kafka.

1.6.10 When Redis can be the primary store

"Not by default" is not "never". Teams do keep authoritative data in Redis when the data set fits in memory, the loss window is acceptable or the AOF with **always** or **everysec** plus replicas and backups is configured, and the access pattern is key-based. Typical cases are session stores, leaderboards, rate-limit state, real-time counters, feature flags and job queues. A good test: **if the Redis data vanished, could you rebuild it or tolerate its loss?** If yes, treat it as a fast cache or working store. If no, either configure persistence and replication seriously and test restores, or keep a durable database as the system of record.

REAL WORLD

In a typical e-commerce platform the order database is the truth. Redis holds the product cache (rebuildable), sessions (loss means a re-login), carts (sometimes copied to a database periodically), and the flash-sale stock counter (initialised from the database before the sale). Each of these has a different answer to "what happens if Redis loses it?", and that answer decides how it is configured.

INTERVIEW ANGLE

Is Redis durable and consistent? Not by default. Durability depends on persistence: RDB snapshots can lose minutes of data, AOF with **everysec** about a second, AOF with **always** almost nothing at a throughput cost. Replication is asynchronous, so a failover can lose recent acknowledged writes and replicas can serve stale reads; **WAIT** and **min-replicas-to-write** reduce but do not remove that risk. A single command, script or transaction is atomic, but there is no rollback.

KEY POINTS

Key points

- Each command, script and **MULTI/EXEC** block is atomic and isolated; there is no rollback.
- Durability is a configuration choice: none, RDB, AOF **everysec** or AOF **always**.
- Replication is asynchronous: acknowledged writes can be lost on failover and replicas can be stale.
- Redis favours availability and speed; it offers knobs to tighten consistency but is not a strongly consistent store.
- Treat Redis as the source of truth only when the data fits in RAM, loss is tolerable or well protected, and access is key-based.

1.7 Redis versus the Alternatives

"Which tool should I use?" is rarely answered by "the fastest". This chapter compares Redis with the options you will be asked about, honestly, including the cases where Redis is the wrong choice. The comparisons are about design differences; any product evolves, so check current documentation before a decision, especially for features of commercial and fast-moving projects.

1.7.1 Redis versus Memcached

Memcached is a distributed memory cache. It stores opaque values under keys with a time to live, evicts with an LRU policy, and is multi-threaded. It is simple, mature and fast.

Aspect	Redis	Memcached
Value model	Typed structures (hash, list, set, zset, stream, ...)	Opaque byte values
Operations	Server-side commands, scripts, transactions	Get, set, delete, increment, a few more
Persistence	RDB and AOF	None
Replication / HA	Replicas, Sentinel, Cluster	Not built in (client-side sharding)
Threads	Main thread for commands, optional I/O threads	Multi-threaded
Pub/Sub, streams	Yes	No

Redis and Memcached.

Choose Memcached if all you need is a plain, horizontally sharded string cache and you value its simplicity. Choose Redis when you want more than get and set, which is true for most modern systems. Many teams standardise on Redis simply so one technology covers caching, counters, queues and sessions.

1.7.2 Redis versus a relational database

A relational database (PostgreSQL, MySQL) offers durable storage, SQL, joins, constraints and full ACID transactions. Redis offers speed and structure-specific operations. They are partners, not competitors: the relational database is the system of record, Redis absorbs repeated reads, short-lived state and counters.

Aspect	Redis	Relational database
Typical latency	Sub-millisecond	Milliseconds and up
Data model	Keys and structures	Tables, rows, relations
Queries	By key (plus indexed search if you set it up)	Arbitrary SQL, joins, aggregations
Transactions	Atomic commands, scripts, MULTI/EXEC , no rollback	Full ACID with rollback
Capacity	Bounded by RAM	Bounded by disk
Durability	Configurable, weaker by default	Strong by design

Redis and a relational database.

1.7.3 Redis versus Kafka and RabbitMQ

Redis can move messages (lists, Pub/Sub, streams), but message brokers are built for it.

Aspect	Redis (streams)	Kafka	RabbitMQ
Model	Append-only log in memory, consumer groups	Partitioned, replicated disk log	Broker with exchanges and queues
Retention	Bounded by memory; you trim	Long, disk-based; replay	Until consumed (queues)
Throughput focus	Low latency, moderate volume	Very high volume, durable	Flexible routing, per-message delivery
Ecosystem	Small, simple	Large (connectors, stream processing)	Mature routing and protocols
Operations	You already run Redis	A cluster to operate	A broker to operate

Redis, Kafka and RabbitMQ for messaging.

Use Redis streams or lists when the volume is moderate, you want low latency, you already run Redis and a bounded history is fine, such as background jobs or notification events in ShopStream. Use Kafka when you need durable long-term retention, replay by many independent consumers and very large throughput. Use RabbitMQ when you want rich routing and per-message acknowledgement semantics from a mature broker. Redis Pub/Sub is different again: it is fire-and-forget with no storage, suitable for ephemeral signals only.

1.7.4 Redis versus Valkey, KeyDB and Dragonfly

Several projects speak the Redis protocol, so existing clients work. They differ in origin and design.

- **Valkey** is a fork of Redis 7.2.4 created in 2024 under the Linux Foundation after Redis changed its license, and kept under the BSD license. It tracks Redis closely in command set and has its own roadmap. Moving between the two is often easy at the command level, though newer features diverge, and the Redis 8 built-in modules (JSON, Search, time series and so on) are Redis features.
- **KeyDB** is a multi-threaded fork of Redis.
- **Dragonfly** is a from-scratch, multi-threaded, shared-nothing server that aims to be compatible with the Redis and Memcached protocols and to use many cores in one process.

The honest view: for most applications the data model and commands matter more than which engine hosts them, and compatibility is high but not total. Verify the commands and modules you rely on before you switch, and check the license terms for your use. This book targets Redis Open Source 8.x.

1.7.5 Redis versus Hazelcast and other in-memory data grids

Hazelcast is a JVM-based in-memory data grid. It can run embedded inside your Java application and distributes data across application nodes, offering distributed maps, locks, and compute. Redis is a separate, non-JVM server shared by clients in any language. The embedded model removes a network hop but couples cache memory and lifecycle to your application (garbage collection, restarts). A standalone Redis is easier to scale and operate separately and is usually the default choice, while a data grid fits teams that want distributed Java data structures in-process.

1.7.6 Managed services and DynamoDB

You do not have to operate Redis yourself. Cloud providers offer managed services built on Redis or compatible engines: Amazon ElastiCache and MemoryDB, Azure Cache for Redis and its successors, Google Memorystore, and Redis Cloud from Redis itself. Managed service features and supported versions vary and change; check each vendor's documentation for the version, modules and persistence they provide. A later Part discusses managed services in more detail.

Amazon DynamoDB is a durable, serverless key-value and document database with single-digit millisecond latency by design. It is a primary store, not a cache. AWS pairs it with a separate in-memory cache service (DAX) for microsecond reads. In an AWS design you may use DynamoDB as the database and Redis (via ElastiCache) next to it for hot reads, counters and sessions; the two address different needs.

1.7.7 Three small design reviews

A. "We want to show the number of items in stock quickly." Reads dominate, a few seconds of staleness is fine: a Redis counter or cached value with a short TTL, sourced from the database. Redis fits.

B. "We want to record every payment." Payments are authoritative, must be durable and auditable, and need transactions and constraints: a relational database. Redis may sit alongside to rate-limit or deduplicate requests, but is not the ledger.

C. "We want to send an email whenever an order is placed." The volume is moderate, losing an event is unacceptable, and consumers may crash and retry: a Redis stream with a consumer group is reasonable (acknowledgements and a pending list give at-least-once delivery); a broker such as Kafka or RabbitMQ is equally defensible if the organisation already runs one. Plain Pub/Sub would be wrong because it forgets messages for absent subscribers.

Notice that none of these answers is "Redis is best". They are "Redis is best for this shape of problem".

1.7.8 Migrating from Memcached or a local cache

Teams commonly start with a local map or with Memcached and later move to Redis. The mechanics are small: replace `get` and `set` calls with `GET` and `SET ... EX`, and serialise values (JSON or another format) because Redis stores bytes. The deeper change is mental: once the cache is a data structure server, you start moving logic into it (counters, sorted rankings, queues). Do this gradually, one role at a time, and keep the original data source authoritative until you have measured the result. The caching chapters in the Patterns Part and the Spring cache chapter show a migration-friendly layering in which the cache is behind a service interface.

1.7.9 A quick decision table

You need	Reach for
A shared cache with expiry	Redis (or Memcached for plain strings)
Sessions, carts, rate limits, leaderboards	Redis
Authoritative business data with ACID	A relational database
Durable event log with replay and huge volume	Kafka
Complex routing, per-message acks	RabbitMQ
Simple job queue or notification stream	Redis lists or streams
Embedded distributed Java data structures	A data grid such as Hazelcast
Hands-off operations	A managed Redis-compatible service

Which tool for which job.

INTERVIEW ANGLE

When would you choose Redis over Memcached, and Redis streams over Kafka? Redis over Memcached when you need structures beyond strings, atomic server-side operations, persistence or replication; Memcached suffices for a plain cache. Redis streams over Kafka when volume is moderate, latency matters, you already operate Redis and bounded retention is acceptable; Kafka when you need durable long retention, replay by many consumers and very high throughput.

KEY POINTS

Key points

- Redis pairs with, rather than replaces, a relational database.
- Versus Memcached: Redis adds structure, persistence, replication and scripting; Memcached is simpler.
- Redis streams cover moderate-volume queues; Kafka is the choice for large durable logs.
- Valkey, KeyDB and Dragonfly are protocol-compatible alternatives; check feature and license differences.
- Managed services remove operations work; check which version and features they support.

1.8 History, Licensing and the Redis 8 Platform

1.8.1 A short history

Redis was created by Salvatore Sanfilippo (known online as antirez), whose first release came in 2009. He built it to solve a scaling problem in a real-time web analytics tool he was working on, where a relational database could not keep up with the write rate. The idea of exposing in-memory lists and sets through simple commands turned out to be widely useful, and the project grew quickly, with the design values that remain visible today: simplicity, predictable performance and a small, readable code base in C.

Milestones worth knowing (the dates and version numbers are the stable ones; look at the release notes for details):

Version	What it brought
2.x	Replication, Pub/Sub, expiry maturing, the AOF and RDB persistence formats
2.6	Lua scripting (EVAL)
2.8	Partial resynchronisation, Sentinel (first form)

Version	What it brought
3.0	Redis Cluster
4.0	Modules API, MEMORY commands, lazy freeing
5.0	Streams
6.0	ACLs, TLS, RESP3, optional I/O threads, client-side caching support
7.0	Functions, sharded Pub/Sub, multi-part AOF, ACL selectors
7.4	Hash field expiration (HEXPIRE and friends)
8.0	Redis Stack merged into the core product; new data types and commands; I/O threading rework

Selected Redis milestones.

Do not try to memorise this. The pattern matters: Redis started as a cache-like key-value server and steadily added the capabilities of a small data platform (persistence, high availability, scaling out, messaging, scripting, security) while keeping the same command model.

1.8.2 Licensing

The license is a topic where you should read the current text yourself, but you should know the outline. For many years Redis was released under the permissive BSD license. In March 2024 the project moved to a dual source-available arrangement (RSALv2 and SSPLv1) starting with version 7.4. In 2025, with Redis 8, the project added the **GNU AGPLv3** as a further option, so that Redis Open Source is again available under an OSI-approved open source license. The Linux Foundation fork **Valkey** (from Redis 7.2.4, BSD) exists because of the 2024 change.

For a developer building an application that simply connects to a Redis server, these terms matter mostly to companies that offer Redis itself as a hosted service or that distribute modified versions. Still: **check the license terms for your own use**, and your organisation's policy, rather than relying on a book (including this one).

1.8.3 Redis 8: one platform

Before Redis 8, the open source product came in two shapes. **Redis Community Edition** was the core server with the classic data types. **Redis Stack** bundled that core with extra modules: RedisJSON, RedisSearch, RedisTimeSeries and RedisBloom, plus the RedisInsight GUI. Developers had to decide which to install, and module commands were not part of the same documentation, security model or release train.

Redis 8.0 (released in 2025) merged these. Redis Open Source 8 includes, in one download:

- **JSON**: a native document type with path-based updates (**JSON.SET**, **JSON.GET**, **JSON.NUMINCRBY**).
- **The Query Engine** (search and query): secondary indexes over hashes and JSON documents, full-text search, aggregation and vector similarity (**FT.CREATE**, **FT.SEARCH**, **FT.AGGREGATE**).
- **Time series**: **TS.ADD**, **TS.RANGE** and retention and downsampling rules.
- **Probabilistic data structures**: Bloom and Cuckoo filters, Count-Min Sketch, Top-K and t-digest.
- **Vector sets**: a new data type (introduced in beta with 8.0) for storing and searching embedding vectors.
- Performance work (the release notes quote large gains in command latency and replication efficiency and memory savings), and module commands now belong to the standard ACL categories, so the same permission model controls them.

Subsequent minor releases continue to add features. As of this writing the current line includes 8.2, 8.4 and 8.6. Examples you will meet later in the book:

Version	Examples
8.0	HGETEX , HSETEX , HGETDEL ; modules built in; vector sets (beta)
8.2	XDELEX and XACKDEL for streams; new BITOP operators; CLUSTER SLOT-STATS
8.4	Atomic slot migration in Cluster; DELEX , DIGEST , MSETEX ; hybrid search (FT.HYBRID)
8.6	Further performance and memory reductions; idempotent XADD ; volatile-lrm and allkeys-lrm eviction; HOTKEYS

Notable additions in the 8.x line (see the release notes for the full lists).

1.8.4 Editions and tools around Redis

You will meet several names. **Redis Open Source** is what this book uses: the server you can download and run yourself. Redis the company also sells commercial products (a cloud service and self-managed enterprise software) built on it, with extra operational features such as multi-region replication; these are optional. **RedisInsight** is a free graphical tool for browsing keys, running commands and inspecting memory. It is helpful for beginners and for exploring a data set, although `redis-cli` remains the tool you will use in scripts, containers and production. The Hands-On Labs Part shows how to connect RedisInsight to a local server.

The ecosystem around the server is large: client libraries in every language, the Spring and Quarkus integrations, monitoring exporters, and managed offerings from the main cloud providers. Because the protocol is shared, a skill learned on one is almost always portable to the others.

NOTE

Version targeting in this book. Everything here is written for Redis Open Source 8.x. Where a feature needs a recent 8.x release, the text says so. The classic commands (strings, lists, hashes, sets, sorted sets, expiry, replication) behave the same on Redis 6 and 7, so the first Parts remain useful if you are stuck on an older version, but check any 7.4+ or 8.x command against your server's version with `INFO server` before using it.

Check what you are talking to

TERMINAL

```
127.0.0.1:6379> INFO server
# Server
redis_version:8.6.6
redis_mode:standalone
os:Linux 6.8.0 x86_64
```

(The output continues with many more fields. Your version and OS lines will differ.)

INTERVIEW ANGLE

What changed in Redis 8? Redis Stack and Community Edition were merged: JSON, the Query Engine, time series and probabilistic structures became part of Redis Open Source, vector sets were added as a new data type, module commands joined the standard ACL categories, and the I/O threading and many commands were optimised. Hash field expiration (from 7.4) gained companion commands such as `HGETEX` and `HSETEX`. The license also gained the AGPLv3 option.

KEY POINTS

Key points

- Redis began in 2009 as a fast key-value server and grew into a data platform while keeping its command model.
- The license changed in 2024 and gained an AGPLv3 option with Redis 8; Valkey is the BSD-licensed fork. Check the terms for your use.
- Redis 8 merged Redis Stack into the core: JSON, search, time series, probabilistic structures and vector sets.
- The 8.x line keeps adding commands; verify the version with `INFO server`.

1.9 Roadmap of This Book and the ShopStream Project

1.9.1 How the book is organised

The book moves from "what and why" to "build and operate". Each Part answers one question:

Part	Question it answers
Fundamentals	What is Redis, why is it fast, and what does it promise?
Hands-On Labs	How do I install it and try every major feature on my machine?
Data Structures	Which structure fits which problem, and how do the commands work?
Redis Inside	How does the engine, memory, expiry, persistence and scripting work?
Scaling and High Availability	How do replication, Sentinel and Cluster keep it available and large?
Redis from Java	How do Lettuce, Jedis and Redisson talk to Redis from Java 21?
Spring Boot and Spring Data Redis	How do I use Redis cleanly from Spring Boot 4?

Part	Question it answers
Patterns	How do I build caches, locks, rate limiters, queues and leaderboards correctly?
Where Redis Is Used	Which real systems use which patterns, and when should I avoid Redis?
Projects	Can I build four complete services end to end?
Operations and Security	How do I run, monitor, secure, back up and upgrade it?
Revision Kit and Interview Preparation	How do I revise quickly and answer interview questions?

The Parts of this book and the question each one answers.

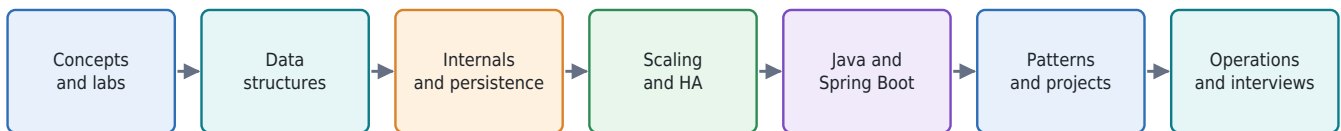


Figure 1.4. A suggested learning path. The first three Parts give you the working vocabulary; the rest you can take in the order that matches your goals.

1.9.2 Three ways to read it

- **Straight through** if you are new to Redis. Do the labs when you reach them; typing the commands is how the model sticks.
- **Interview preparation**: read Fundamentals, skim Data Structures and Redis Inside, then work through the Revision Kit and the interview questions, returning to chapters when an answer feels thin.
- **Building an application**: read Fundamentals, do the install lab, jump to the Spring Boot Part and the Patterns you need, then follow one of the Projects.

Each chapter ends with a **Key points** box you can use for revision. Boxes labelled Warn and Gotcha mark mistakes that cost real time; Interview boxes contain a question and a crisp answer; Real boxes describe how things look in production.

1.9.3 Tools and versions

All examples assume the following. You do not need to match them exactly, but the Spring chapters in particular depend on the library versions.

Component	Version / note
Redis	Redis Open Source 8.x (Docker image <code>redis:8</code>)
Java	21
Spring Boot	4.1.x with Spring Data Redis 4.1.x (Lettuce by default)
Build tool	Maven
Tests	JUnit 5 with Testcontainers
Command line	<code>redis-cli</code> (ships with Redis)

Environment used in this book.

1.9.4 The running example: ShopStream

To keep the examples connected, nearly everything uses one fictional product, **ShopStream**, an online store built from small Spring Boot services behind a gateway. It is deliberately ordinary, because the problems it has (slow product pages, shared logins, flash sales, notifications) are the ones real teams meet.

Service	Responsibility
<code>gateway</code>	Entry point, authentication, rate limiting
<code>catalog-service</code>	Product catalogue and search
<code>cart-service</code>	Shopping carts

Service	Responsibility
<code>order-service</code>	Placing and tracking orders
<code>inventory-service</code>	Stock levels, flash-sale reservations
<code>notification-service</code>	Emails and push messages triggered by order events

ShopStream services.

The services use Redis in the roles below, and the book builds each one at the point where the matching theory has been taught.

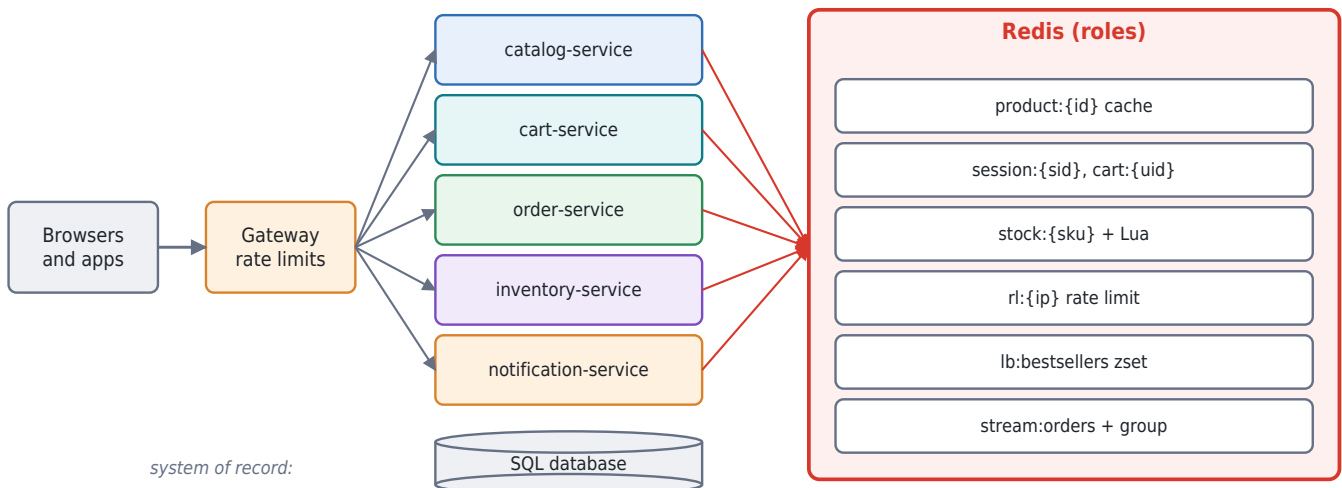


Figure 1.5. ShopStream and the jobs Redis does in it. The database stays the system of record; Redis serves the fast, shared, short-lived parts.

Key pattern	Structure	Role
<code>product:{id}</code>	Hash or JSON	Catalogue cache (cache-aside)
<code>session:{sid}</code>	Hash with TTL	Login sessions
<code>cart:{userId}</code>	Hash	Shopping cart
<code>stock:{sku}</code>	String counter plus Lua	Flash-sale stock
<code>rl:{ip}:{route}</code>	String counter	Rate limits
<code>lb:bestsellers:daily</code>	Sorted set	Best-seller leaderboard
<code>stream:orders</code>	Stream	Order events for the notifier
<code>uv:{date}</code>	HyperLogLog	Unique visitors per day
<code>geo:stores</code>	Geo index	Store locator
<code>idem:{key}</code>	String with TTL	Idempotency keys
<code>lock:order:{id}</code>	String with TTL	Locks
<code>ff:{name}</code>	String or hash	Feature flags

Redis roles in ShopStream and their keys.

All Java code lives under the package root `com.shopstream`. The Projects Part assembles four of these roles into complete, tested services that you can run on your own machine.

1.9.5 What you need to start

You should be comfortable reading Java 21, understand what a Spring Boot service with a REST controller and a service class looks like, and know basic SQL. You do not need prior distributed-systems knowledge; terms such as replication, sharding and consistency are defined where they first matter. Docker is the easiest way to run Redis, and the Hands-On Labs Part has the installation steps for Docker, Linux, macOS and Windows.

A realistic pace is one chapter per sitting of 30 to 60 minutes, with commands typed rather than read. The Revision Kit at the end contains a 30-day plan for those who prefer a schedule.

1.9.6 Conventions in the text

- **Monospace** marks commands, keys, class names and configuration keys.
- A terminal box starting with `127.0.0.1:6379>` is a `redis-cli` session; lines starting with `$` are shell commands. Output shown is typical output; yours may differ in IDs, timings and version.
- Numbers described as illustrative are for building intuition, not measurements. Measure your own system.
- Cross-references are by chapter name, because numbering can change between editions.

TIP

A good habit while reading: keep a `redis-cli` window open and run every command you see. Predict the output before you press Enter. Wrong predictions are where the learning happens.

KEY POINTS

Key points

- The book goes from fundamentals and labs, through data structures and internals, to scaling, Java and Spring, patterns, projects, operations and revision.
- Pick a reading path: straight through, interview-focused, or build-focused.
- ShopStream is the running example; its Redis roles map to the Parts that teach them.
- Target: Redis Open Source 8.x, Java 21, Spring Boot 4.1.x.

PART 2

Hands-On Labs

Everything in this Part is meant to be typed. Reading about Redis teaches you vocabulary; running it teaches you how it behaves.

In this part

- 2.1 Installing Redis
- 2.2 redis-cli in Depth
- 2.3 Configuration Basics
- 2.4 The Lab Workbook

2.1 Installing Redis

You cannot learn a server by reading about it. In the next hour you will have a real Redis running on your machine, and by the end of this Part you will have broken it, restarted it, replicated it, and clustered it on purpose. This chapter gets the server and its tools installed. Pick **one** route below; the Docker route is the one the rest of the book assumes because it behaves the same on Linux, macOS and Windows, and because it is trivial to throw away and recreate.

2.1.1 What you are installing

Redis Open Source 8 ships as one package. Since version 8.0 the JSON, Query Engine (search), time series and probabilistic data types, and the new vector set type are bundled with the server instead of living in a separate "Redis Stack" download. You get three programs that matter:

- **redis-server**: the server process itself.
- **redis-cli**: the command-line client you will use constantly (next chapter).
- **redis-benchmark**: a load generator, used in the last lab. Also shipped: **redis-check-aof** and **redis-check-rdb**, two repair tools used in the persistence lab.

Redis listens on TCP port **6379** by default. It talks plain text over that port, with no password and no encryption unless you configure them. That is acceptable on your laptop and a disaster on a public network. We come back to this in the security chapters; for now remember that a lab Redis should only be reachable from **localhost**.

2.1.2 Route 1: Docker (recommended)

If you have Docker Desktop (macOS, Windows) or Docker Engine (Linux), one command is enough. The image **redis** on Docker Hub is the official image; **redis:8** follows the 8.x line, so you get the latest 8 release when you pull. For anything you care about, pin an exact tag instead of a moving one (see the tip below).

Start Redis in a container

TERMINAL

```
$ docker run -d --name redis -p 6379:6379 redis:8
$ docker ps --filter name=redis --format "{{.Names}} {{.Image}} {{.Status}}"
redis redis:8 Up 5 seconds
$ docker exec -it redis redis-cli
127.0.0.1:6379> PING
PONG
```

What the flags mean: **-d** runs the container in the background, **--name redis** gives it a name you can refer to later, and **-p 6379:6379** publishes the container's port 6379 on your machine's port 6379. **docker exec -it redis redis-cli** starts the CLI **inside** the container, so you do not need to install any client on the host.

Containers are disposable: **docker rm -f redis** deletes the container and, unless you mounted a volume, its data. To keep data between container re-creations, mount a volume at **/data**, which is where the image writes its files:

Persist data in a named volume

TERMINAL

```
$ docker run -d --name redis -p 6379:6379 -v redis-data:/data redis:8 \
redis-server --appendonly yes
$ docker logs redis 2>&1 | tail -n 3
1:M 10 Oct 2026 09:14:02.411 * Ready to accept connections tcp
```

Everything after the image name replaces the container's default command. Here it starts `redis-server` with AOF persistence enabled (explained properly in the persistence lab). Without that, the default is RDB snapshots only.

2.1.3 Docker Compose: the shape we use for most labs

Compose describes the same container in a file you can commit and reuse. This is the single-node file you will start from throughout the book. Save it as `docker-compose.yml`.

```
docker-compose.yml YAML

services:
  redis:
    image: redis:8
    container_name: shopstream-redis
    ports:
      - "6379:6379"
    command: ["redis-server", "--appendonly", "yes"]
    volumes:
      - redis-data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

  redisinsight:
    image: redis/redisinsight:latest
    container_name: shopstream-insight
    ports:
      - "5540:5540"
    depends_on:
      redis:
        condition: service_healthy

volumes:
  redis-data:
```

Start it with `docker compose up -d`, stop it with `docker compose down` (add `-v` to delete the volume too). The `healthcheck` runs `redis-cli ping` inside the container so that other services can wait until Redis actually answers.

GOTCHA

"localhost" inside a container is the container itself. In the file above, RedisInsight runs in its own container. When you add a database in its web UI (<http://localhost:5540>), the host name is `redis` (the Compose service name), not `localhost` or `127.0.0.1`. From your own terminal or from a Spring Boot application running on the host, use `localhost:6379`. Mixing these up is the most common "Redis is down" report from new users.

TIP

Pin the version for anything you share. `redis:8` is convenient for learning. In a project, a team repository or a CI pipeline, pin an exact patch tag, for example `redis:8.6.6` (check Docker Hub for the tags that exist today). A floating tag means two colleagues can run different server versions without noticing.

2.1.4 Route 2: Linux packages and building from source

Ubuntu and Debian (official APT repository)

Redis publishes its own package repository, which carries current releases (distribution repositories often lag by years). These are the commands from the Redis documentation:

```
install-ubuntu.sh BASH

sudo apt-get install lsb-release curl gpg
curl -fsSL https://packages.redis.io/gpg | sudo gpg --dearmor \
  -o /usr/share/keyrings/redis-archive-keyring.gpg
sudo chmod 644 /usr/share/keyrings/redis-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/redis-archive-keyring.gpg] \
https://packages.redis.io/deb $(lsb_release -cs) main" \
  | sudo tee /etc/apt/sources.list.d/redis.list
sudo apt-get update
sudo apt-get install redis
```

The `redis` package pulls in `redis-tools` (the CLI and friends) and registers a systemd service. It normally starts on boot; if not, run `sudo systemctl enable --now redis-server`. Check it with `systemctl status redis-server` and `redis-cli ping`. On Red Hat family distributions the Redis documentation describes an equivalent RPM repository; follow the page for your distribution.

Building from source

Building from source is worthwhile once, because it shows you how small the project is and gives you the exact binaries the documentation talks about. Download a release tarball from the Redis releases page (verify it against the published digests), unpack it, and run `make`:

```
build-from-source.sh BASH
cd /usr/src
tar xvf redis-8.x.y.tar.gz      # the release you downloaded
cd redis-8.x.y
export BUILD_TLS=yes           # TLS support (needs OpenSSL headers)
export BUILD_WITH_MODULES=yes  # JSON, search, time series, ... (needs a Rust toolchain)
make -j "$(nproc)" all
./src/redis-server --version
sudo make install              # optional: copies binaries to /usr/local/bin
```

The module build needs extra tooling (a C/C++ toolchain, CMake and a Rust toolchain; the exact list is on the per-distribution build pages in the documentation). If that is more than you want, a plain `make` still builds the core server with all the classic data types, and the Docker route gives you the full package with no toolchain at all.

2.1.5 Route 3: macOS and Windows

macOS with Homebrew

```
install-macos.sh BASH
brew install redis
brew services start redis    # start now and at login
redis-cli ping              # PONG
brew services stop redis     # when you are done
```

The Redis documentation says the Homebrew formula tracks the latest Open Source release. Whether a given formula build includes the bundled modules (JSON, search and so on) depends on the version, so after installing run `redis-cli MODULE LIST`. If it is empty and you want those features, use Docker for the labs that need them. Also note that after a `brew upgrade` Homebrew keeps your old `redis.conf` and writes the new default next to it as `redis.conf.default`; an old config file may lack the directives that load the modules.

Windows

Redis is built for Linux and macOS; there is no native Windows server from the Redis project. You have two good options:

- **Docker Desktop:** open PowerShell and use the Docker commands above. This is the route the Redis documentation points to.
- **WSL2:** install Ubuntu from the Microsoft Store (`wsl --install`), open the Ubuntu shell, and follow the Ubuntu steps above. Redis is then reachable from Windows at `localhost:6379`, because WSL2 forwards localhost ports by default.

Avoid old unofficial Windows ports of Redis 3.x that still turn up in search results. They are decades behind and miss everything this book teaches.

2.1.6 RedisInsight, the graphical client

RedisInsight is Redis's free desktop and web GUI. It lets you browse keys with their types and TTLs, run commands with auto-complete, watch memory and slow commands, and inspect streams and JSON. It is a convenience, not a replacement for `redis-cli`: everything it does is a command underneath, and you will be asked in interviews about the commands, not the buttons. Install the desktop application from the Redis site, or run the container shown in the Compose file above and open `http://localhost:5540`. Add a database with host `localhost` (or `redis` from another container) and port `6379`.

2.1.7 Verify the installation

Whatever route you took, these four checks should pass before you continue.

Verification checklist

TERMINAL

```
$ redis-server --version
Redis server v=8.6.6 sha=00000000:0 malloc=jemalloc-5.3.0 bits=64 build=3f2a1c9e7d4b8a10
$ redis-cli ping
PONG
$ redis-cli INFO server | head -n 4
# Server
redis_version:8.6.6
redis_git_sha1:00000000
redis_git_dirty:0
$ redis-cli JSON.SET install:test $ '{"ok":true}'
OK
$ redis-cli JSON.GET install:test
"{\"ok\":true}"
$ redis-cli DEL install:test
(integer) 1
```

Your version number and build id will differ; the shape of the output should not. The `JSON.SET` line proves that the bundled modules are loaded (Redis 8 distributions have them). If it answers `ERR unknown command 'JSON.SET'`, you are running a core-only build, which is fine for everything except the JSON, search and time series chapters.

KEY POINTS

- Use Docker for the labs: one command to start, one to destroy, identical on every operating system.
- Redis listens on port 6379 with no password by default; keep lab instances on `localhost`.
- Inside Compose, services reach each other by service name (`redis`), not by `localhost`.
- Pin exact image tags for anything shared; use `redis:8` only for personal learning.
- Verify with `redis-cli ping`, `INFO server` and (for the bundled modules) a `JSON.SET` round trip.

2.2 redis-cli in Depth

`redis-cli` is the screwdriver of Redis work. You use it to learn commands, to debug production, to load data, to measure latency, to find big keys, and to build clusters. Developers who master it solve problems in minutes that others solve with a day of logging code. This chapter covers the parts you will actually use.

2.2.1 Two ways to run it

Interactive mode is what you get when you type `redis-cli` with no command. You see a prompt, type a command, and read the reply. The CLI knows every command's syntax, so it shows hints while you type and keeps a history (up-arrow works; the history lives in `~/.rediscli_history`).

One-shot mode is when you pass the command as arguments. It prints the reply and exits, which is what you want in scripts.

Interactive versus one-shot

TERMINAL

```
$ redis-cli
127.0.0.1:6379> SET product:1001:name "Steel Mug"
OK
127.0.0.1:6379> GET product:1001:name
"Steel Mug"
127.0.0.1:6379> EXIT
$ redis-cli GET product:1001:name
"Steel Mug"
$ redis-cli GET product:1001:name > /tmp/name.txt && cat /tmp/name.txt
Steel Mug
```

Look at the last command: when the output is not a terminal, `redis-cli` switches to **raw** output and drops the quotes. That is deliberate (scripts want clean values) and it surprises people exactly once. Use `--no-raw` to force the decorated format, `--raw` to force raw output at a terminal.

2.2.2 Connecting: host, port, user, database

Option	Meaning
-h host	Server host name or IP (default 127.0.0.1)
-p port	Server port (default 6379)
-a password	Password. Prints a warning because the password shows in ps ; prefer the next two
--askpass	Prompts for the password with masked input
REDISCLI_AUTH	Environment variable holding the password (not visible in the process list)
--user name	ACL user name; with -a or --askpass it sends AUTH name password
-u uri	A URI: redis://user:pass@host:6379/2 , or rediss:// for TLS
-n 2	Select logical database 2 (databases are numbered 0 to 15)
-s path	Connect through a Unix socket instead of TCP
-3	Use the RESP3 protocol for the session
-c	Cluster mode: follow MOVED and ASK redirects automatically
--tls, --cacert, --cert, --key	TLS connections, covered in the security chapter

Connection options.

Connecting to a protected, remote server

TERMINAL

```
$ export REDISCLI_AUTH='s3cret'
$ redis-cli -h cache.internal -p 6380 --user shop-app -n 2 PING
PING
$ redis-cli -u redis://shop-app:s3cret@cache.internal:6380/2 DBSIZE
(integer) 1204
```

WATCH OUT

Do not put real passwords on the command line. Anything after **-a** is visible to every user on the machine through **ps**, and it lands in your shell history. Use **REDISCLI_AUTH** or **--askpass**. The CLI itself warns you with "Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe." (**--no-auth-warning** silences it; do not use that as an excuse).

2.2.3 Getting help and reading output

Inside the CLI, **HELP <command>** shows syntax, summary, version and group; **HELP @string** lists all commands of a group (@list, @hash, @set, @sorted_set, @stream, ...). The same reference lives at redis.io/docs/latest/commands/.

The help command

TERMINAL

```
127.0.0.1:6379> HELP SET

SET key value [NX|XX] [GET] [EX seconds|PX milliseconds|EXAT unix-time-seconds|
PXAT unix-time-milliseconds|KEEPTTL]
summary: Set the string value of a key
since: 1.0.0
group: string
```

The exact **SET** syntax line grows with each release (Redis 8.4 added further options), so do not be alarmed if yours differs. The CLI decorates replies according to their **type**, and learning to read these decorations is half of using it:

Reply type	Looks like	Example
Simple string	<code>`OK`</code>	reply to SET
Integer	<code>`(integer) 42`</code>	reply to INCR , DEL
Bulk string	<code>`"Steel Mug"`</code> (quoted, escaped)	reply to GET
Null	<code>`(nil)`</code>	GET of a missing key
Array	numbered lines <code>`1) "a"`</code>	reply to LRANGE , SMEMBERS
Nested array	indented numbering <code>`1) 1) "a"`</code>	reply to XRANGE
Empty array	<code>`(empty array)`</code>	LRANGE of a missing list

Reply type	Looks like	Example
Error	<code>`(error) ERR ...`</code> or <code>`(error) WRONGTYPE ...`</code>	misuse of a command

How redis-cli prints reply types.

```
Types in action TERMINAL
127.0.0.1:6379> SET views 41
OK
127.0.0.1:6379> INCR views
(integer) 42
127.0.0.1:6379> GET nosuchkey
(nil)
127.0.0.1:6379> RPUSH recent:42 "mug" "kettle"
(integer) 2
127.0.0.1:6379> LRange recent:42 0 -1
1) "mug"
2) "kettle"
127.0.0.1:6379> LRange nosuchlist 0 -1
(empty array)
127.0.0.1:6379> INCR recent:42
(error) WRONGTYPE Operation against a key holding the wrong kind of value
```

Quoting and binary data

Arguments with spaces need quotes. Inside double quotes you can use C-style escapes such as `\n`, `\t` and `\x00`; single quotes are literal. Redis strings are binary safe, so the CLI shows non-printable bytes as escapes: a stored UTF-8 character such as the rupee sign appears as `"\xe2\x82\xb9"` unless you start the CLI with `--raw`, which prints the real characters.

2.2.4 Output formats for scripts

Three options change the shape of the output; they matter when you pipe replies into other tools.

```
--raw, --csv and --json TERMINAL
$ redis-cli RPUSH tags:42 kitchen steel gift
(integer) 3
$ redis-cli --no-raw LRange tags:42 0 -1
1) "kitchen"
2) "steel"
3) "gift"
$ redis-cli LRange tags:42 0 -1
kitchen
steel
gift
$ redis-cli --csv LRange tags:42 0 -1
"kitchen","steel","gift"
$ redis-cli --json LRange tags:42 0 -1
["kitchen","steel","gift"]
```

`--json` and `--quoted-json` (ASCII-safe escapes instead of raw Unicode) exist in current versions of `redis-cli`; they use RESP3 replies by default (add `-2` if you want RESP2). With JSON output you can feed results to `jq`.

2.2.5 Repeating commands, reading from files and stdin

`-r N` repeats a command `N` times and `-i seconds` waits between repetitions. This turns the CLI into a tiny monitoring tool and a data generator:

```
Repeat and watch TERMINAL
$ redis-cli -r 3 -i 1 INFO clients | grep connected_clients
connected_clients:1
connected_clients:1
connected_clients:1
$ redis-cli -r 1000 LPUSH queue:emails "welcome"
(integer) 1
...
```

The `-x` flag reads the **last argument** from standard input, which is the easy way to store a file or the output of another program:

Store a file as a value

TERMINAL

```
$ cat banner.html | redis-cli -x SET page:home:banner
OK
$ redis-cli STRLEN page:home:banner
(integer) 2048
$ redis-cli --raw GET page:home:banner > banner-copy.html
```

To run a list of commands from a file, redirect it into the CLI: `redis-cli < commands.txt`. Each line is a command, exactly as you would type it. The next section shows the faster, safer way for bulk loads.

2.2.6 Bulk loading with --pipe

Sending one million `SET` commands with one process per command would take hours; sending them one at a time over a socket still wastes a round trip per command. `redis-cli --pipe` reads raw Redis protocol (RESP) from standard input and streams it to the server as fast as the network allows, without waiting for each reply. It then reports how many replies it received.

You must feed it RESP, not plain commands. This small shell function generates the format (each command is `*<arg>` then, for every argument, `<length>` and the bytes, all lines ended with `\r\n`):

gen_resp.sh

BASH

```
#!/usr/bin/env bash
# usage: gen_resp.sh SET product:1 "Steel Mug"
cmd=("$@")
printf '%d\r\n' "${#cmd[@]}"
for arg in "${cmd[@]"; do
    printf '%d\r\n%s\r\n' "${#arg}" "$arg"
done
```

Load 100,000 products

TERMINAL

```
$ for i in $(seq 1 100000); do ./gen_resp.sh SET "product:$i:name" "item-$i"; done \
| redis-cli --pipe
All data transferred. Waiting for the last reply...
Last reply received from server.
errors: 0, replies: 100000
$ redis-cli DBSIZE
(integer) 100000
```

In real projects you would generate the protocol from a script in Python or Java instead of Bash, but the principle is the same. Notice the final message: `errors: 0, replies: 100000`. Always check that the two numbers match your expectation; `--pipe` is how large imports are done safely (the official `redis-cli --pipe` documentation calls this "mass insertion"). The same idea in application code is called **pipelining**, covered in the engine and client chapters.

2.2.7 Scanning the keyspace

Never run `KEYS *` on a real server. `KEYS` walks the whole keyspace in one blocking command, and since Redis runs commands one at a time on a single thread, every other client waits. The safe alternative is `SCAN`, which returns a small batch plus a cursor. `redis-cli --scan` runs the cursor loop for you:

Safe key listing

TERMINAL

```
$ redis-cli --scan --pattern 'product:100*:name' | head -n 3
product:1000:name
product:10001:name
product:10002:name
$ redis-cli --scan --pattern 'product:*' --count 1000 | wc -l
100000
$ redis-cli --scan --pattern 'session:*' | xargs -n 100 redis-cli UNLINK
(integer) 100
(integer) 100
```

`--pattern` filters by a glob (`*`, `?`, `[a-c]`) and `--count` is the batch size hint passed to `SCAN`. The last line deletes matching keys in batches of 100 with `UNLINK`, which frees the memory in a background thread instead of blocking. The `SCAN` guarantees (and its duplicates) are covered in the chapter on iterating safely in Part 3.

2.2.8 Finding big keys and memory hogs

Three sampling modes answer "what is eating my memory?". They all use **SCAN** internally, so they are safe on a live server (but still add load; use **-i 0.1** to pause 0.1 seconds per 100 scan steps on busy systems).

```
--bigkeys (abridged) TERMINAL

$ redis-cli --bigkeys

# Scanning the entire keyspace to find biggest keys as well as
# average sizes per key type. You can use -i 0.1 to sleep 0.1 sec
# per 100 SCAN commands (not usually needed).

[00.00%] Biggest string found so far "product:77:name" with 11 bytes
[00.00%] Biggest list found so far "queue:emails" with 1000 items
[00.00%] Biggest hash found so far "cart:u-17" with 3 fields

----- summary -----

Sampled 100003 keys in the keyspace!
Total key length in bytes is 1788915 (avg len 17.89)

Biggest string found "product:77:name" has 11 bytes
Biggest list found "queue:emails" has 1000 items
Biggest hash found "cart:u-17" has 3 fields

100000 strings with 1099000 bytes (99.99% of keys, avg size 10.99)
1 lists with 1000 items (00.00% of keys, avg size 1000.00)
2 hashes with 5 fields (00.00% of keys, avg size 2.50)
```

--bigkeys reports the biggest key of each type **by element count** (items, fields, members; string length for strings). **--memkeys** reports the same by **bytes of memory** (it calls **MEMORY USAGE** per key), which is usually the question you really have. Newer versions add **--keystats**, which combines both and adds a distribution of key sizes; check **redis-cli --help** for the options in your version. **--hotkeys** finds frequently accessed keys, but only works when **maxmemory-policy** is one of the LFU policies. Redis 8.6 additionally introduces a server-side **HOTKEYS** command; the troubleshooting chapter in the operations Part uses it.

2.2.9 Latency tools

```
Measuring latency from the client side TERMINAL

$ redis-cli --latency
min: 0, max: 1, avg: 0.11 (1893 samples)^C
$ redis-cli --latency-history -i 5
min: 0, max: 1, avg: 0.10 (402 samples) -- 5.01 seconds range
min: 0, max: 3, avg: 0.13 (398 samples) -- 5.01 seconds range
$ redis-cli --stat -i 1
----- data ----- load ----- child -
keys      mem      clients blocked requests      connections
100003    12.35M    1         0      10243 (+0)      5
100003    12.35M    1         0      10244 (+1)      5
```

--latency sends **PING** in a loop and reports min, max and average round trip in **milliseconds**; press Ctrl-C to stop. **--latency-history** prints one line per interval so you can see change over time. **--stat** is a rolling one-line-per-second summary of keys, memory, clients and request rate, ideal during a load test. Remember that these numbers include your network: run them from the application host, not from the server, to see what the application sees. **--intrinsic-latency 5** (run on the server) measures the jitter of the machine itself with no Redis involved, which separates "slow hardware or noisy virtualisation" from "slow Redis".

2.2.10 MONITOR, a taste of the firehose

MONITOR streams every command the server executes, in real time. It is the quickest way to see what a Spring application really sends.

```
MONITOR (run in a second terminal, then use another client) TERMINAL

127.0.0.1:6379> MONITOR
OK
1791623402.114872 [0 127.0.0.1:52144] "SET" "product:1001:name" "Steel Mug"
1791623402.116031 [0 127.0.0.1:52144] "GET" "product:1001:name"
1791623403.002294 [0 127.0.0.1:52150] "HGETALL" "cart:u-17"
```

Each line shows a Unix timestamp with microseconds, the database number, the client address and the command.

WATCH OUT

Do not leave **MONITOR** running on a busy production server. **MONITOR** copies every command to a client output buffer. The Redis documentation warns that a running **MONITOR** can cut throughput substantially (more than half on a busy instance, in their benchmark). Use it for a few seconds on staging; in production prefer **SLOWLOG**, the latency tools, or sampling. Also remember that it prints arguments, including passwords sent with **AUTH** and any sensitive values, to whoever runs it.

2.2.11 Other options worth knowing

- **--rdb dump.rdb** downloads an RDB snapshot from a running server (it asks the server to produce one and streams it). A cheap way to take a backup copy or to seed a test instance; **--functions-rdb** fetches only the Functions.
- **--replica** makes the CLI pretend to be a replica and print every write command the primary propagates. Useful for seeing exactly what gets replicated.
- **-e** makes **redis-cli** exit with a non-zero status when a command fails, which you need in CI scripts.
- **--eval script.lua key1 key2 , arg1 arg2** runs a Lua script, with a comma separating the **KEYS** from the **ARGV** items (explained in the scripting chapter). **--ldb** starts the Lua debugger.
- **--cluster ...** is the cluster manager: **create**, **check**, **reshard**, **rebalance**, **add-node**, **del-node**, **fix**. You will use it in the cluster lab.
- **--version** prints the CLI version.

2.2.12 Scripting with redis-cli

Because one-shot mode prints raw values and sets exit codes, **redis-cli** fits naturally in shell scripts. A typical health check for a deployment pipeline:

redis-health.sh

BASH

```
#!/usr/bin/env bash
set -euo pipefail
HOST="${REDIS_HOST:-127.0.0.1}"
PORT="${REDIS_PORT:-6379}"

if [ "$(redis-cli -h "$HOST" -p "$PORT" PING)" != "PONG" ]; then
    echo "redis not answering" >&2
    exit 1
fi

used=$(redis-cli -h "$HOST" -p "$PORT" INFO memory \
| awk -F: '/^used_memory:/ {print $2}' | tr -d '\r')
role=$(redis-cli -h "$HOST" -p "$PORT" INFO replication \
| awk -F: '/^role:/ {print $2}' | tr -d '\r')
echo "role=$role used_memory_bytes=$used"
```

The **tr -d '\r'** matters: **INFO** replies use **\r\n** line endings, and a stray carriage return inside a shell variable breaks comparisons in confusing ways. For multi-command scripts that must run atomically, use a Lua script or a **MULTI/EXEC** block instead of a shell loop; shell loops are not atomic.

INTERVIEW ANGLE

How do you find the biggest keys on a production Redis without blocking it? Use **redis-cli --bigkeys** (by element count) or **--memkeys** (by memory), which iterate with **SCAN** rather than **KEYS**, so they never block the server for long. Sampling is incremental and approximate, so on very busy systems add **-i 0.1**. For one key, **MEMORY USAGE key** gives its bytes. Never use **KEYS *** for this.

KEY POINTS

- Interactive mode is for exploring; one-shot mode (with raw output when piped) is for scripts. Use **-e** for exit codes.
- Pass secrets with **REDISCLI_AUTH** or **--askpass**, never with **-a** on the command line.
- Learn to read reply types: (**integer**), quoted bulk strings, numbered arrays, (**nil**), (**error**).
- Use **--scan**, **--bigkeys**, **--memkeys** instead of **KEYS ***; use **--pipe** with RESP for bulk loads and check its error count.
- **--latency**, **--latency-history** and **--stat** measure what the client sees; **MONITOR** is a debugging tool, not a production tool.

2.3 Configuration Basics

A freshly installed Redis runs with sensible defaults, which is why the first chapters needed no configuration at all. Sooner or later you will need to change something: bind to another interface, set a memory limit, turn on the append-only file, add a password. This chapter shows where settings live, the three ways to change them, and the dozen directives that cover most real situations. Later Parts return to individual directives in context (persistence, eviction, replication, security); here you learn the mechanics.

2.3.1 Where configuration comes from

Redis reads configuration from four places, applied in this order, with later ones overriding earlier ones:

1. **Built-in defaults** compiled into the server.
2. **A configuration file**, usually named `redis.conf`, passed as the first argument: `redis-server /etc/redis/redis.conf`. If you start `redis-server` with no file, only defaults and command-line options apply.
3. **Command-line options**, which use the same names as the file with two dashes: `redis-server /etc/redis/redis.conf --port 6380 --maxmemory 256mb`.
4. **Runtime changes** made with `CONFIG SET` while the server is running.

The file format is one directive per line, a name followed by its arguments, with `#` starting a comment. Memory sizes accept units (`1k`, `1kb`, `1m`, `1mb`, `1g`, `1gb`; the `k/m/g` forms mean powers of 1000, and the `kb/mb/gb` forms mean powers of 1024). Booleans are `yes` and `no`. A file can pull in others with `include /path/to/other.conf`, which is handy for keeping environment-specific overrides separate.

The configuration file shipped with the source tree is heavily commented and is the best documentation of each directive; the version for your release is also published on redis.io. Do not edit it in place for a new project. Write a **small** file of your own that contains only what you changed, so a reviewer can see your decisions at a glance.

2.3.2 Inspecting and changing configuration at runtime

`CONFIG GET` reads a setting (glob patterns work), `CONFIG SET` changes it immediately without a restart, and `CONFIG REWRITE` writes the current running configuration back to the file that started the server. `INFO server` tells you which file that was.

```

CONFIG GET, SET and REWRITE
127.0.0.1:6379> CONFIG GET maxmemory
1) "maxmemory"
2) "0"
127.0.0.1:6379> CONFIG SET maxmemory 100mb
OK
127.0.0.1:6379> CONFIG GET maxmemory
1) "maxmemory"
2) "104857600"
127.0.0.1:6379> CONFIG SET maxmemory-policy allkeys-lru
OK
127.0.0.1:6379> CONFIG SET nosuch 1
(error) ERR Unknown option or number of arguments for CONFIG SET - 'nosuch'
127.0.0.1:6379> CONFIG GET save
1) "save"
2) "3600 1 300 100 60 10000"
127.0.0.1:6379> CONFIG REWRITE
OK
127.0.0.1:6379> INFO server
# Server
...
config_file:/usr/local/etc/redis/redis.conf

```

Three details deserve attention. First, `CONFIG GET` shows values in their canonical form: `100mb` became `104857600` bytes. Second, many parameters can be changed at runtime, but a few are **immutable** (for example the `daemonize` setting, and in practice anything that needs a new socket or a restart); `CONFIG SET` replies with an error for those. Third, `CONFIG SET` is **not persistent**: restart the server and the change is gone unless you ran `CONFIG REWRITE` or edited the file yourself.

GOTCHA

CONFIG REWRITE needs a file, and it rewrites the file. If the server was started without a config file, CONFIG REWRITE fails with "The server is running without a config file". If it was started with one, Redis rewrites it: it keeps your comments and other lines where possible, but it appends the values it changed, and it may add lines you never wrote (for example `user default on nopass ~* &* +@all`). In containers the file is often mounted read-only or owned by another user, and the rewrite then fails. Treat the file as the source of truth in version control, and use CONFIG SET for emergencies and experiments only.

2.3.3 The essential directives

Redis has hundreds of settings. The table lists the ones you will meet in almost every deployment, with the default (as shipped for current Redis releases). Where I am not sure a default is stable across versions I say "see docs" instead of quoting one.

Directive	Default	What it does and when to change it
bind	<code>* -::*</code>	Network interfaces to listen on. Production: list only the private addresses the app uses. Inside a container you need <code>0.0.0.0</code>
port	6379	TCP port. <code>0</code> disables TCP (use with <code>unixsocket</code>)
protected-mode	yes	If there is no <code>bind</code> and no password, Redis answers only loopback clients. A safety net, not a security design
requirepass	(none)	A password for the <code>default</code> user. Legacy style; prefer ACL users (security chapter)
databases	16	Number of logical databases. Cluster uses only database 0
dir	current directory	Working directory where RDB and AOF files are written. In Docker: <code>/data</code>
save	<code>3600 1 300 100 60 10000</code>	RDB snapshot rules: after N seconds if at least M keys changed. <code>save ""</code> disables snapshots
appendonly	no	Turns on the append-only file for durability
appendfsync	everysec	AOF flush policy: <code>always</code> , <code>everysec</code> , <code>no</code>
maxmemory	0 (no limit)	Memory ceiling for data. Always set it for a cache
maxmemory-policy	noeviction	What to evict at the ceiling: <code>allkeys-lru</code> , <code>allkeys-lfu</code> , <code>volatile-ttl</code> , ...
maxclients	10000	Maximum simultaneous client connections
timeout	0	Close idle clients after N seconds (0 = never)
tcp-keepalive	300	Seconds between TCP keepalive probes, to detect dead peers
loglevel	notice	<code>debug</code> , <code>verbose</code> , <code>notice</code> , <code>warning</code>
logfile	<code>""</code> (stdout)	Empty means standard output, which is what containers and systemd want
io-threads	1	Extra threads for socket reads and writes; see the engine chapter before touching
slowlog-log-slower-than	10000	Log commands slower than this many microseconds (10 ms)
slowlog-max-len	128	How many slow entries to remember
notify-keyspace-events	<code>""</code>	Which keyspace events to publish through Pub/Sub; empty means none
replicaof	(none)	Make this server a replica of <code>host port</code>

Directives you will actually touch.

Two of these deserve a sentence more. `dir` decides where `dump.rdb` and the AOF files land. People lose data by starting Redis from a different working directory and wondering where their snapshot went; set it explicitly. `maxmemory` defaults to unlimited; a Redis used as a cache and left at that default will keep growing until the operating system kills it. The eviction chapter in the engine Part explains how to pick the policy.

2.3.4 A small configuration file for the labs

This is a complete, minimal file for a single Redis in a container. It reflects the pattern recommended earlier: say only what you decided.

```
redis.conf CONF
# --- network ---
bind 0.0.0.0
port 6379
protected-mode yes

# --- access (lab only; use ACL users for real systems) ---
requirepass lab-secret

# --- data ---
dir /data
save 3600 1 300 100 60 10000
appendonly yes
appendfsync everysec

# --- memory ---
maxmemory 256mb
maxmemory-policy allkeys-lru

# --- diagnostics ---
slowlog-log-slower-than 10000
slowlog-max-len 256
```

Notice `bind 0.0.0.0` inside a container: the container has its own network namespace, and Docker's published port reaches it through a non-loopback address. The real protection is that you publish the port only to your own machine (`-p 127.0.0.1:6379:6379`) and use a password.

2.3.5 Running with a config in Docker

Mount the file into the container and pass its path as the first argument. Mount a volume at `/data` so the files from `dir /data` survive.

```
Start Redis with your own redis.conf TERMINAL
$ docker run -d --name redis-conf -p 127.0.0.1:6380:6379 \
  -v "$(pwd)/redis.conf":/usr/local/etc/redis/redis.conf:ro \
  -v conf-data:/data \
  redis:8 redis-server /usr/local/etc/redis/redis.conf
$ redis-cli -p 6380 PING
(error) NOAUTH Authentication required.
$ redis-cli -p 6380 -a lab-secret --no-auth-warning CONFIG GET maxmemory
1) "maxmemory"
2) "268435456"
$ redis-cli -p 6380 -a lab-secret --no-auth-warning CONFIG GET appendonly
1) "appendonly"
2) "yes"
$ docker rm -f redis-conf
```

The `:ro` suffix mounts the file read-only, so `CONFIG REWRITE` would fail here; that is a feature, since the file in Git is the truth. In Docker Compose the same thing is `command: ["redis-server", "/usr/local/etc/redis/redis.conf"]` plus a `volumes:` entry.

TIP

Configuration is code: keep `redis.conf` in the same repository as your Compose file or Helm chart, and review changes to it like source changes. A silent `CONFIG SET maxmemory-policy noeviction` made by hand during an incident and never written back is a classic way for the next restart to behave differently.

INTERVIEW ANGLE

What is the difference between `CONFIG SET` and editing `redis.conf`? `CONFIG SET` changes the running server immediately and is lost on restart; editing `redis.conf` is durable but only takes effect on the next start (unless you also run `CONFIG SET`). `CONFIG REWRITE` bridges the two by writing the running configuration back to the file. A few settings cannot be changed at runtime at all.

KEY POINTS

- Order of precedence: defaults, then the config file, then command-line options, then **CONFIG SET** at runtime.
- **CONFIG GET / SET / REWRITE** inspect, change and persist settings; **INFO server** shows the active config file.
- Always set **dir**, **maxmemory** (for caches) and an explicit persistence mode; do not rely on defaults you have not read.
- Keep a small config file of only your decisions, in version control; mount it read-only in containers.
- Never expose a Redis with **protected-mode** and a wide **bind** as your only defences; add a password or ACL and network rules.

2.4 The Lab Workbook

The rest of this Part is a sequence of labs. Each has a goal, numbered steps with typical output, things to observe, and a cleanup. Work through them in order the first time; later labs assume you can already use **redis-cli**. The first four labs use the single instance from the installation chapter; the last ones use Docker Compose to build replication, Sentinel and Cluster topologies.

LAB 2.1 · Lab: Strings, counters and expiry

Goal. Use **SET** options, atomic counters and TTLs the way ShopStream does for page-view counters, sessions, per-IP rate limits and a simple lock. About 15 minutes.

Setup. Start from a clean single instance (Chapter on installing) and open a CLI:

```
Setup TERMINAL
$ docker run -d --name redis -p 6379:6379 redis:8
$ docker exec -it redis redis-cli
127.0.0.1:6379> FLUSHALL
OK
```

FLUSHALL deletes every key in every database. It is fine on a lab instance you just created; never type it on a server that holds data you care about.

Step 1: a counter that cannot lose updates

INCR reads, adds one and writes back as **one** command. Because Redis runs commands one at a time, two clients calling it at once never overwrite each other, which is the failure of "read the value in the app, add one, write it back".

```
Page views and stock TERMINAL
127.0.0.1:6379> INCR views:product:1001
(integer) 1
127.0.0.1:6379> INCRBY views:product:1001 10
(integer) 11
127.0.0.1:6379> SET stock:sku-1001 50
OK
127.0.0.1:6379> DECRBY stock:sku-1001 3
(integer) 47
127.0.0.1:6379> SET stock:sku-1001 "fifty"
OK
127.0.0.1:6379> DECR stock:sku-1001
(error) ERR value is not an integer or out of range
127.0.0.1:6379> OBJECT ENCODING views:product:1001
"int"
```

Redis stores numbers as strings, but it knows when a string holds an integer: **OBJECT ENCODING** reports **int**. Incrementing a non-numeric string is an error, not a silent zero.

Step 2: expiry for sessions

A session that disappears

TERMINAL

```
127.0.0.1:6379> SET session:s-9f2 "u-17" EX 30
OK
127.0.0.1:6379> TTL session:s-9f2
(integer) 29
127.0.0.1:6379> SET session:s-9f2 "u-17-v2" KEEPTTL
OK
127.0.0.1:6379> TTL session:s-9f2
(integer) 24
127.0.0.1:6379> SET session:s-9f2 "u-17-v3"
OK
127.0.0.1:6379> TTL session:s-9f2
(integer) -1
127.0.0.1:6379> TTL nosuchkey
(integer) -2
```

Observe the three return values of **TTL**: a positive number of seconds, **-1** meaning "key exists with no expiry", and **-2** meaning "key does not exist". Observe also that a plain **SET** on an existing key **removes** its TTL, because it replaces the whole value; **KEEPTTL** is how you update a value without extending or dropping its life. Now slide the expiry forward on each access, the way a session works, and then make a key permanent:

Sliding expiry

TERMINAL

```
127.0.0.1:6379> SET session:s-9f2 "u-17" EX 30
OK
127.0.0.1:6379> GETEX session:s-9f2 EX 30
"u-17"
127.0.0.1:6379> PERSIST session:s-9f2
(integer) 1
127.0.0.1:6379> TTL session:s-9f2
(integer) -1
127.0.0.1:6379> SET temp:x 1 PX 1500
OK
127.0.0.1:6379> GET temp:x
"1"
127.0.0.1:6379> GET temp:x
(nil)
```

The last two **GET** commands were issued a couple of seconds apart: after 1.5 seconds the key is simply gone. Expiry and how Redis implements it (lazily on access plus an active background sweep) are covered in the engine Part.

Step 3: a fixed-window rate limit in two commands

Combine **INCR** with an expiry that is set only once. **EXPIRE ... NX** sets a TTL only if the key has none:

Allow N requests per minute per IP and route

TERMINAL

```
127.0.0.1:6379> INCR rl:203.0.113.7:/login
(integer) 1
127.0.0.1:6379> EXPIRE rl:203.0.113.7:/login 60 NX
(integer) 1
127.0.0.1:6379> INCR rl:203.0.113.7:/login
(integer) 2
127.0.0.1:6379> EXPIRE rl:203.0.113.7:/login 60 NX
(integer) 0
127.0.0.1:6379> TTL rl:203.0.113.7:/login
(integer) 52
```

The second **EXPIRE** returns **0** because the key already has a TTL, so the window does not slide forward on every request. The application rejects the request when the counter exceeds its limit. (A crash between **INCR** and **EXPIRE** could leave a counter without TTL; the rate-limiting chapter shows the Lua version that closes that gap.)

Step 4: `SET NX` as a lock

Only one worker wins

TERMINAL

```
127.0.0.1:6379> SET lock:order:9001 worker-1 NX PX 5000
OK
127.0.0.1:6379> SET lock:order:9001 worker-2 NX PX 5000
(nil)
127.0.0.1:6379> GET lock:order:9001
"worker-1"
127.0.0.1:6379> PTTL lock:order:9001
(integer) 3118
```

NX means "only if the key does not exist". The reply (**nil**) for worker-2 means it did not get the lock. This is the heart of a distributed lock; the lock chapter explains why releasing it correctly takes a little more.

Step 5: `OBJECT ENCODING` and binary safety

Encodings

TERMINAL

```
127.0.0.1:6379> SET name:short "Asha"
OK
127.0.0.1:6379> OBJECT ENCODING name:short
"embstr"
127.0.0.1:6379> APPEND name:short " Rao"
(integer) 8
127.0.0.1:6379> OBJECT ENCODING name:short
"raw"
127.0.0.1:6379> STRLEN name:short
(integer) 8
```

Short strings are stored as **embstr** (the header and bytes in one allocation). Modifying one converts it to **raw**. You never need to manage this; it is a first look at the memory-saving encodings the engine Part explains.

What to observe. Counters are atomic; **TTL** has two special negative values; **SET** without options clears a TTL; **NX** turns **SET** into a try-lock.

Cleanup. **FLUSHALL** (lab instance only), or leave the keys for the next lab.

LAB 2.2 · Lab: Lists and sets

Goal. Build a work queue with a reliable hand-off, a capped "recent searches" list, and a tag system with set algebra. About 20 minutes. Start a fresh **redis-cli** with **FLUSHALL**.

Part A: a queue of emails to send

Producers push on the left, consumers take from the right, so the list behaves first-in, first-out.

Producer side

TERMINAL

```
127.0.0.1:6379> LPUSH queue:emails "welcome:u-1"
(integer) 1
127.0.0.1:6379> LPUSH queue:emails "welcome:u-2"
(integer) 2
127.0.0.1:6379> LPUSH queue:emails "receipt:o-9001"
(integer) 3
127.0.0.1:6379> LRANGE queue:emails 0 -1
1) "receipt:o-9001"
2) "welcome:u-2"
3) "welcome:u-1"
```

The newest item is at index 0, the oldest is at the right end. A plain **RPOP** would take **welcome:u-1**, but if the worker crashed right after popping, the email would be lost. **LMOVE** pops and pushes to a second "processing" list in one atomic step, so the item is never in limbo:

Consumer side with a processing list

TERMINAL

```
127.0.0.1:6379> LMOVE queue:emails queue:emails:processing RIGHT LEFT
"welcome:u-1"
127.0.0.1:6379> LLEN queue:emails
(integer) 2
127.0.0.1:6379> LRANGE queue:emails:processing 0 -1
1) "welcome:u-1"
127.0.0.1:6379> LREM queue:emails:processing 1 "welcome:u-1"
(integer) 1
```

After sending the email the worker removes it from the processing list with **LREM**. A janitor process would look at **queue:emails:processing** for items that have been there too long and move them back. This is the classic "reliable queue" pattern; the queue chapter in the Patterns Part builds a full one.

Part B: blocking pop

Workers should not poll in a loop. **BLMOVE** waits until an item arrives or the timeout expires. Open a second terminal for the producer.

Terminal 1: worker waits up to 30 seconds

TERMINAL

```
127.0.0.1:6379> DEL queue:emails queue:emails:processing
(integer) 2
127.0.0.1:6379> BLMOVE queue:emails queue:emails:processing RIGHT LEFT 30
```

Terminal 2: producer

TERMINAL

```
127.0.0.1:6379> LPUSH queue:emails "welcome:u-3"
(integer) 1
```

Terminal 1 returns **"welcome:u-3"** at once (and prints the wait time, for example **(2.41s)**). If nothing arrives within the timeout it returns **(nil)**. A blocked client consumes almost no server resources, which is why this beats polling.

Part C: a capped list of recent searches

Keep only the last five searches

TERMINAL

```
127.0.0.1:6379> LPUSH recent:search:u-17 mug kettle toaster blender teapot grater peeler
(integer) 7
127.0.0.1:6379> LTRIM recent:search:u-17 0 4
OK
127.0.0.1:6379> LRANGE recent:search:u-17 0 -1
1) "peeler"
2) "grater"
3) "teapot"
4) "blender"
5) "toaster"
```

LTRIM key 0 4 keeps indexes 0 to 4 and discards the rest. Run **LPUSH** then **LTRIM** on every search and the list never grows beyond five entries. (Pushed several values at once, **LPUSH** inserts them one after another at the head, so the last argument ends up first.) If a user searches the same word twice you will want to remove the old copy with **LREM** first; try it.

Part D: tags with sets

Products and their tags

TERMINAL

```
127.0.0.1:6379> SADD tags:product:1001 kitchen steel gift
(integer) 3
127.0.0.1:6379> SADD tags:product:1002 kitchen plastic kids
(integer) 3
127.0.0.1:6379> SADD tags:product:1001 gift
(integer) 0
127.0.0.1:6379> SISMEMBER tags:product:1001 steel
(integer) 1
127.0.0.1:6379> SINTER tags:product:1001 tags:product:1002
1) "kitchen"
127.0.0.1:6379> SUNION tags:product:1001 tags:product:1002
1) "kitchen"
2) "steel"
3) "gift"
4) "plastic"
5) "kids"
127.0.0.1:6379> SDIFF tags:product:1001 tags:product:1002
1) "steel"
2) "gift"
127.0.0.1:6379> SCARD tags:product:1001
(integer) 3
```

A set stores each member once, so adding **gift** again returns **0**. The order of members in **SMEMBERS**, **SUNION** and friends is **not part of the contract**; your output may list them differently. Now flip the index around for lookups from the tag side:

A reverse index and a query

TERMINAL

```
127.0.0.1:6379> SADD tag:kitchen 1001 1002
(integer) 2
127.0.0.1:6379> SADD tag:steel 1001 1003
(integer) 2
127.0.0.1:6379> SINTER tag:kitchen tag:steel
1) "1001"
127.0.0.1:6379> SINTERCARD 2 tag:kitchen tag:steel
(integer) 1
```

"Kitchen products that are steel" is one command. In a relational database that is a join across a tag table; here it is a set intersection, done in memory. **SINTERCARD** returns only the size of the intersection, cheaper when you need just a count.

What to observe. **LPUSH/LMOVE/LTRIM** give you queues and capped lists with atomic steps; blocking pops avoid polling; sets give uniqueness and fast algebra, with no ordering promise.

Cleanup. **FLUSHALL** on the lab instance.

LAB 2.3 · Lab: Hashes and sorted sets

Goal. Model a user profile and a shopping cart with hashes, then build the daily bestsellers leaderboard with a sorted set. About 20 minutes. Use the instance from the earlier labs and run **FLUSHALL** first (lab instance only).

Part A: a profile as a hash

A hash stores several named fields under one key. It is the natural shape for an object whose fields you read and update separately.

Create and read a profile

TERMINAL

```
127.0.0.1:6379> HSET user:17:profile name Asha city Pune plan free
(integer) 3
127.0.0.1:6379> HGET user:17:profile city
"Pune"
127.0.0.1:6379> HMGET user:17:profile name plan nosuchfield
1) "Asha"
2) "free"
3) (nil)
127.0.0.1:6379> HGETALL user:17:profile
1) "name"
2) "Asha"
3) "city"
```

Create and read a profile (continued)

TERMINAL

```

4) "Pune"
5) "plan"
6) "free"
127.0.0.1:6379> HINCRBY user:17:profile logins 1
(integer) 1
127.0.0.1:6379> HDEL user:17:profile plan
(integer) 1
127.0.0.1:6379> HLEN user:17:profile
(integer) 3

```

HSET returns the number of **new** fields it created (three here); updating existing fields returns **0** for them. **HGETALL** returns a flat alternating list of field, value, field, value. **HINCRBY** works on one field atomically, so **logins** can be counted without reading the whole profile. Check how small hashes are stored:

Encoding

TERMINAL

```

127.0.0.1:6379> OBJECT ENCODING user:17:profile
"listpack"

```

Small hashes use a compact **listpack** encoding, then switch to a real hash table above a size threshold. That switch is invisible to your code but matters for memory, and it is the subject of the memory chapter in the engine Part.

Part B: a cart, with a field that expires

The cart is a hash of sku to quantity

TERMINAL

```

127.0.0.1:6379> HSET cart:u-17 sku-1001 2 sku-1002 1
(integer) 2
127.0.0.1:6379> HINCRBY cart:u-17 sku-1001 1
(integer) 3
127.0.0.1:6379> HGETALL cart:u-17
1) "sku-1001"
2) "3"
3) "sku-1002"
4) "1"
127.0.0.1:6379> EXPIRE cart:u-17 86400
(integer) 1

```

EXPIRE on the cart key expires the whole hash after a day. Since Redis 7.4 you can also give an individual **field** a lifetime, for example a flash-sale price that applies for 30 seconds:

Field-level expiry (Redis 7.4 and later, so also Redis 8)

TERMINAL

```

127.0.0.1:6379> HSET product:1001 name "Steel Mug" price 9.90 sale_price 7.90
(integer) 3
127.0.0.1:6379> HEXPIRE product:1001 30 FIELDS 1 sale_price
1) (integer) 1
127.0.0.1:6379> HTTL product:1001 FIELDS 1 sale_price
1) (integer) 28
127.0.0.1:6379> HTTL product:1001 FIELDS 1 price
1) (integer) -1

```

The reply is an array with one result per named field: **1** means the TTL was set, and **HTTL** returns **-1** for a field without expiry, as with keys. After 30 seconds **sale_price** disappears while **name** and **price** stay.

Part C: a leaderboard in a sorted set

A sorted set keeps unique members ordered by a numeric **score**. For bestsellers, the member is the SKU and the score is units sold today.

Build and query the leaderboard

TERMINAL

```

127.0.0.1:6379> ZADD lb:bestsellers:daily 120 sku-1001 95 sku-1002 300 sku-1003
(integer) 3
127.0.0.1:6379> ZINCRBY lb:bestsellers:daily 5 sku-1002
"100"
127.0.0.1:6379> ZRANGE lb:bestsellers:daily 0 -1 REV WITHSCORES
1) "sku-1003"
2) "300"
3) "sku-1001"
4) "120"
5) "sku-1002"
6) "100"

```

Build and query the leaderboard (continued)

TERMINAL

```
127.0.0.1:6379> ZRANGE lb:bestsellers:daily 0 0 REV
1) "sku-1003"
127.0.0.1:6379> ZREVRANK lb:bestsellers:daily sku-1002
(integer) 2
127.0.0.1:6379> ZSCORE lb:bestsellers:daily sku-1002
"100"
127.0.0.1:6379> ZRANGE lb:bestsellers:daily 100 200 BYSCORE WITHSCORES
1) "sku-1002"
2) "100"
3) "sku-1001"
4) "120"
127.0.0.1:6379> ZCOUNT lb:bestsellers:daily 100 +inf
(integer) 3
```

Things to notice. Ranks start at 0 (the third place has rank 2, not 3). **REV** flips the order so the biggest score comes first; this replaces the older **ZREVRANGE**. Scores are returned as strings because they are floating-point numbers. **ZINCRBY** is atomic, so thousands of order handlers can increment concurrently with no lost updates. Finally, ties are broken by the member name in byte order, which is stable but not always what you want; the leaderboard chapter deals with that.

Try it. Add **sku-1004** with the same score as **sku-1001** and see which one comes first. Then use **ZPOPMAX lb:bestsellers:daily** and notice that it removes the leader.

What to observe. Hashes are objects with per-field operations; sorted sets are ordered by score with $O(\log N)$ inserts; the same key can be expired as a whole, and in recent versions per field.

Cleanup. **FLUSHALL**.

LAB 2.4 · Lab: Streams and a consumer group

Goal. Publish order events to a stream, let a **notifier** group read them, acknowledge them, and recover a message from a worker that "crashed". About 25 minutes. This is the mechanism ShopStream's notification-service uses.

Step 1: append events

XADD appends an entry. ***** asks Redis to generate the ID, which is **<millisecond-timestamp>-<sequence>**.

Three orders arrive (your IDs will differ)

TERMINAL

```
127.0.0.1:6379> XADD stream:orders * orderId 9001 userId u-17
"1760086441000-0"
127.0.0.1:6379> XADD stream:orders * orderId 9002 userId u-18
"1760086441015-0"
127.0.0.1:6379> XADD stream:orders * orderId 9003 userId u-17
"1760086441031-0"
127.0.0.1:6379> XLEN stream:orders
(integer) 3
```

Step 2: read without a group

Read a range

TERMINAL

```
127.0.0.1:6379> XRANGE stream:orders - + COUNT 2
1) 1) "1760086441000-0"
   2) 1) "orderId"
   2) "9001"
   3) "userId"
   4) "u-17"
2) 1) "1760086441015-0"
   2) 1) "orderId"
   2) "9002"
   3) "userId"
   4) "u-18"
```

- and + mean "from the smallest ID" and "to the largest". Unlike a list, reading does **not** remove anything: the stream is a log that many readers can read independently.

Step 3: create a consumer group

A group remembers, for the stream, which entries have been delivered and which are still unacknowledged. `0` means "start from the beginning"; `$` would mean "only entries added from now on".

Group and first delivery

TERMINAL

```
127.0.0.1:6379> XGROUP CREATE stream:orders notifier 0
OK
127.0.0.1:6379> XREADGROUP GROUP notifier worker-1 COUNT 2 STREAMS stream:orders >
1) 1) "stream:orders"
   2) 1) 1) "1760086441000-0"
        2) 1) "orderId"
           2) "9001"
           3) "userId"
           4) "u-17"
       2) 1) "1760086441015-0"
        2) 1) "orderId"
           2) "9002"
           3) "userId"
           4) "u-18"
```

The `>` at the end means "entries never delivered to anyone in this group". Worker `worker-1` now owns two entries. Redis recorded them in the group's **pending entries list** (PEL) until someone acknowledges them.

What is pending?

TERMINAL

```
127.0.0.1:6379> XPENDING stream:orders notifier
1) (integer) 2
2) "1760086441000-0"
3) "1760086441015-0"
4) 1) 1) "worker-1"
    2) "2"
127.0.0.1:6379> XPENDING stream:orders notifier - + 10
1) 1) "1760086441000-0"
   2) "worker-1"
   3) (integer) 41872
   4) (integer) 1
2) 1) "1760086441015-0"
   2) "worker-1"
   3) (integer) 41872
   4) (integer) 1
```

The summary shows the count, the smallest and largest pending IDs, and how many each consumer holds. The detailed form shows, per entry, the owning consumer, the **idle time** in milliseconds and how many times it has been delivered.

Step 4: acknowledge, then simulate a crash

Worker-1 finishes the first order and acknowledges it. It "crashes" before handling the second.

Acknowledge one entry

TERMINAL

```
127.0.0.1:6379> XACK stream:orders notifier 1760086441000-0
(integer) 1
127.0.0.1:6379> XPENDING stream:orders notifier
1) (integer) 1
2) "1760086441015-0"
3) "1760086441015-0"
4) 1) 1) "worker-1"
    2) "1"
```

Meanwhile worker-2 reads new work. It gets only order 9003, because the other two were already delivered to this group.

Another consumer in the same group

TERMINAL

```
127.0.0.1:6379> XREADGROUP GROUP notifier worker-2 COUNT 5 STREAMS stream:orders >
1) 1) "stream:orders"
   2) 1) 1) "1760086441031-0"
        2) 1) "orderId"
           2) "9003"
           3) "userId"
           4) "u-17"
```

Step 5: take over the stuck entry

Entry 9002 is still pending for the dead worker. **XAUTOCLAIM** transfers ownership of entries that have been idle at least a given number of milliseconds. A real system uses a few seconds or more; use a small value in the lab.

```
Claim abandoned work TERMINAL

127.0.0.1:6379> XAUTOCLAIM stream:orders notifier worker-2 1000 0 COUNT 10
1) "0-0"
2) 1) 1) "1760086441015-0"
    2) 1) "orderId"
        2) "9002"
        3) "userId"
        4) "u-18"
    3) (empty array)
127.0.0.1:6379> XACK stream:orders notifier 1760086441015-0 1760086441031-0
(integer) 2
127.0.0.1:6379> XPENDING stream:orders notifier
1) (integer) 0
2) (nil)
3) (nil)
4) (nil)
```

The reply has three parts: the cursor for the next call (**0-0** means the scan is complete), the claimed entries, and a list of IDs that no longer exist in the stream. Wait at least a second after step 4 before running it, or nothing is idle long enough to be claimed. After acknowledging, the PEL is empty.

Step 6: block for new entries

```
Wait up to five seconds for new events TERMINAL

127.0.0.1:6379> XREADGROUP GROUP notifier worker-1 BLOCK 5000 STREAMS stream:orders >
(nil)
(5.01s)
```

Run **XADD stream:orders * orderId 9004 userId u-19** from a second terminal while this waits, and it returns the new entry at once. That is the loop of a real consumer: block, process, acknowledge, repeat.

What to observe. A group gives each entry to exactly one consumer, tracks it until **XACK**, and lets another consumer take over after a crash: at-least-once delivery. Nothing is deleted when acknowledged; trim the stream with **XTRIM** or **XADD ... MAXLEN ~ N** to bound memory, as the streams chapter explains.

Cleanup. **DEL stream:orders** (this also removes its groups).

LAB 2.5 · Lab: Persistence, RDB versus AOF, and a crash

Goal. Prove with a **kill -9** what each persistence mode loses, inspect the files Redis writes, back up and restore a snapshot, and meet a damaged AOF. About 30 minutes. We use Docker volumes so data survives container restarts. **docker kill** sends **SIGKILL** by default, the same as a power failure for the process: no chance to flush or save.

Part A: RDB only

Start a container whose only persistence is a snapshot rule ("save after 3600 seconds if at least 1 key changed"), and store two orders.

```
RDB: the data written after the last snapshot is lost TERMINAL

$ docker run -d --name rdb-lab -v rdb-data:/data redis:8 \
  redis-server --save "3600 1" --appendonly no
$ docker exec rdb-lab redis-cli SET order:1 "paid"
OK
$ docker exec rdb-lab redis-cli SET order:2 "paid"
OK
$ docker exec rdb-lab redis-cli BGSAVE
Background saving started
$ docker exec rdb-lab redis-cli SET order:3 "paid"
OK
$ docker kill rdb-lab
rdb-lab
$ docker start rdb-lab
rdb-lab
$ docker exec rdb-lab redis-cli DBSIZE
```

RDB: the data written after the last snapshot is lost (continued)

TERMINAL

```
(integer) 2
$ docker exec rdb-lab ls -l /data
total 4
-rw-r--r-- 1 redis redis 109 Oct 10 09:31 dump.rdb
```

Make sure **BGSAVE** has finished before the third **SET** (a second or less for this size; **INFO persistence** shows **rdb_bgsave_in_progress:0**). The result: **order:3** is gone, because it was written after the snapshot and the process died before the next one. The snapshot rule **3600 1** would only have fired an hour later. That is the data-loss window of RDB-only persistence: everything since the last snapshot.

Part B: AOF with fsync on every write

AOF with appendfsync always

TERMINAL

```
$ docker run -d --name aof-lab -v aof-data:/data redis:8 \
  redis-server --appendonly yes --appendfsync always
$ docker exec aof-lab redis-cli SET order:1 "paid"
OK
$ docker exec aof-lab redis-cli SET order:2 "paid"
OK
$ docker kill aof-lab
aof-lab
$ docker start aof-lab
aof-lab
$ docker exec aof-lab redis-cli DBSIZE
(integer) 2
$ docker exec aof-lab sh -c 'ls /data/appendonlydir; cat /data/appendonlydir/*.manifest'
appendonly.aof.1.base.rdb
appendonly.aof.1.incr.aof
appendonly.aof.manifest
file appendonly.aof.1.base.rdb seq 1 type b
file appendonly.aof.1.incr.aof seq 1 type i
```

Both keys survived: with **appendfsync always**, Redis calls **fsync** before replying **OK**, so acknowledged writes are on disk. Since Redis 7 the AOF is a **directory** of files described by a manifest: a **base** file (an RDB snapshot written at the last rewrite) and one or more **incr** files (the commands since). The default policy is **everysec**, which flushes once per second: a crash then loses up to about a second of writes, and it is dramatically faster than **always**. You can see the log of commands in the **incr** file:

The AOF is a readable command log

TERMINAL

```
$ docker exec aof-lab sh -c 'cat /data/appendonlydir/appendonly.aof.1.incr.aof'
*2
$6
SELECT
$1
0
*3
$3
SET
$7
order:1
$4
paid
```

That is the RESP protocol you met with **redis-cli --pipe**: ***3** means three arguments follow, **\$3** means the next one is three bytes long. The file contains the write commands in order; the rest of the output (the second **SET**) is omitted here.

Part C: back up and restore a snapshot

Taking a copy of an RDB file is a valid backup, because the file is written to a temporary name and renamed atomically. The **rdb-lab** container is running again with two keys, so write the lost order once more, take a snapshot, and copy it out:

Backup

TERMINAL

```
$ docker exec rdb-lab redis-cli SET order:3 "paid"
OK
$ docker exec rdb-lab redis-cli SAVE
OK
$ docker cp rdb-lab:/data/dump.rdb ./backup-dump.rdb
Successfully copied 3.07kB to /home/asha/backup-dump.rdb
```

SAVE blocks the server until the snapshot is written (acceptable on a lab instance; use **BGSAVE** for real servers). To restore, start a **new** container with that file placed in **/data** before the server starts:

Restore into a fresh instance

TERMINAL

```
$ docker run -d --name restored -v "$(pwd)/backup-dump.rdb":/data/dump.rdb redis:8
$ docker exec restored redis-cli DBSIZE
(integer) 3
$ docker exec restored redis-cli KEYS 'order:*'
1) "order:2"
2) "order:3"
3) "order:1"
```

The restored instance has all three keys, because the **SAVE** ran after **order:3** was written. (The order of **KEYS** replies is arbitrary, and **KEYS** is acceptable here only because the instance is tiny and ours.) The lesson: a backup is worth exactly as much as the last time you restored from it successfully.

Part D: a damaged AOF

Simulate a crash in the middle of writing a command by appending half of a **SET** to the incr file of the stopped AOF container, then start it again.

Truncated tail

TERMINAL

```
$ docker stop aof-lab
$ docker run --rm -v aof-data:/data redis:8 sh -c \
  'printf "%3\r\n$3\r\nSET\r\n$7\r\norder:9\r\n$5\r\nhel" \
  >> /data/appendonlydir/appendonly.aof.1.incr.aof'
$ docker start aof-lab
$ docker logs aof-lab 2>&1 | grep -i -E "truncat|short read|ready"
Ready to accept connections tcp
$ docker exec aof-lab redis-cli DBSIZE
(integer) 2
```

With the default **aof-load-truncated yes**, Redis notices that the file ends in the middle of a command, discards the incomplete tail, writes a warning to the log and starts. The half-written **order:9** was never acknowledged to a client, so dropping it is correct. Corruption in the **middle** of the file is different: Redis refuses to start and tells you to run the repair tool. It works on the manifest:

Check and repair (only after taking a copy of the files)

TERMINAL

```
$ docker run --rm -v aof-data:/data redis:8 \
  redis-check-aof /data/appendonlydir/appendonly.aof.manifest
$ docker run --rm -it -v aof-data:/data redis:8 \
  redis-check-aof --fix /data/appendonlydir/appendonly.aof.manifest
```

Without **--fix** the tool only reports the first invalid offset. With **--fix** it truncates the file at that point and asks for confirmation; everything after the damage is discarded, so copy the directory first.

What to observe. RDB-only loses the window since the last snapshot; AOF **always** loses nothing acknowledged but costs throughput; the AOF is a directory in Redis 7 and later; restore is only real if you have practised it.

Cleanup.

Remove lab containers and volumes

TERMINAL

```
$ docker rm -f rdb-lab aof-lab restored
$ docker volume rm rdb-data aof-data
$ rm backup-dump.rdb
```

NOTE

The numbers you see (file sizes, durations, exact log wording) will differ from the listings, which are typical rather than recorded from your machine. What should match is the behaviour: which keys survive, and which files exist.

LAB 2.6 · Lab: Replication with a primary and two replicas

Goal. Run one primary and two replicas with Docker Compose, watch data flow, see why replicas refuse writes, observe that replication is asynchronous, survive a replica restart, and promote a replica by hand. About 35 minutes. Make a folder `lab-replication` and put this file in it.

lab-replication/docker-compose.yml

YAML

```
services:
  primary:
    image: redis:8
    container_name: shop-primary
    command: ["redis-server", "--appendonly", "yes"]
    ports:
      - "6379:6379"

  replica1:
    image: redis:8
    container_name: shop-replica1
    command: ["redis-server", "--replicaof", "primary", "6379"]
    depends_on:
      - primary
    ports:
      - "6380:6379"

  replica2:
    image: redis:8
    container_name: shop-replica2
    command: ["redis-server", "--replicaof", "primary", "6379"]
    depends_on:
      - primary
    ports:
      - "6381:6379"
```

`--replicaof primary 6379` makes each replica connect to the host named `primary`, which Compose resolves to the primary container. The replicas publish different host ports (6380, 6381) only so you can also reach them from your machine; between containers they always use 6379.

Step 1: start and inspect

Bring the topology up

TERMINAL

```
$ cd lab-replication && docker compose up -d
$ docker exec shop-primary redis-cli INFO replication
# Replication
role:master
connected_slaves:2
slave0:ip=172.19.0.3,port=6379,state=online,offset=210,lag=0
slave1:ip=172.19.0.4,port=6379,state=online,offset=210,lag=0
master_failover_state:no-failover
master_replid:9c1b3e5f0a7d42e8b6f1a3c5d7e9f0a1b2c3d4e5
master_replid2:0000000000000000000000000000000000000000
master_repl_offset:210
second_repl_offset:-1
repl_backlog_active:1
repl_backlog_size:1048576
repl_backlog_first_byte_offset:1
repl_backlog_histlen:210
```

Even in Redis 8 the output still uses the old word `slave` in field names (`connected_slaves`, `slave0`); the commands use `replica`. Read the key fields: the **replication id** (`master_replid`) names this dataset's history; the **offset** (`master_repl_offset`) counts bytes of the replication stream; each replica line shows its own acknowledged **offset** and **lag** in seconds. Equal offsets mean the replica is fully caught up. The backlog is a buffer of recent stream data (1 MB by default) that lets a briefly disconnected replica resume without a full copy.

The replica's view

TERMINAL

```
$ docker exec shop-replica1 redis-cli INFO replication | head -n 9
# Replication
role:slave
master_host:primary
master_port:6379
master_link_status:up
master_last_io_seconds_ago:0
master_sync_in_progress:0
slave_read_repl_offset:210
slave_repl_offset:210
```

Step 2: writes flow one way

Write on the primary, read from a replica

TERMINAL

```
$ docker exec shop-primary redis-cli SET stock:sku-1001 50
OK
$ docker exec shop-replica1 redis-cli GET stock:sku-1001
"50"
$ docker exec shop-replica2 redis-cli GET stock:sku-1001
"50"
$ docker exec shop-replica1 redis-cli SET stock:sku-1001 49
(error) READONLY You can't write against a read only replica.
```

Replicas are read-only by default (**replica-read-only yes**). The write must go to the primary, which sends it down the replication stream. Replication normally takes well under a millisecond on one machine, so the first **GET** seemed instant. On a real network that gap is the **replication lag**, and reading from a replica can return stale data. A cart that shows yesterday's quantity is annoying; a stock check against stale data can oversell. Decide per use case.

Step 3: replication is asynchronous, `WAIT` makes it conditional

By default the primary answers **OK** before the replicas have the write. **WAIT numreplicas timeout-ms** blocks until that many replicas have acknowledged everything the connection wrote, or the timeout passes, and returns how many did.

WAIT in one interactive session (terminal 1)

TERMINAL

```
$ docker exec -it shop-primary redis-cli
127.0.0.1:6379> SET order:9001:status paid
OK
127.0.0.1:6379> WAIT 2 100
(integer) 2
```

Now stop one replica from a second terminal and repeat in the same session:

Terminal 2, then terminal 1 again

TERMINAL

```
$ docker stop shop-replica2
shop-replica2
```

Terminal 1

TERMINAL

```
127.0.0.1:6379> INCR stock:sku-1001
(integer) 51
127.0.0.1:6379> WAIT 2 100
(integer) 1
127.0.0.1:6379> EXIT
```

WAIT applies to the writes made on **its own connection**, which is why both commands ran in one interactive session; from a fresh **redis-cli** call it would have nothing to wait for. The second **WAIT** waited the full 100 ms and reported that only one replica had the write. Even so, **WAIT** is not a durability guarantee: it tells you the write reached memory on N replicas, and a failover can still lose writes that reached fewer. For real protection you also set **min-replicas-to-write**:

Refuse writes when too few replicas are healthy

TERMINAL

```
$ docker exec shop-primary redis-cli CONFIG SET min-replicas-to-write 2
OK
$ docker exec shop-primary redis-cli SET order:9002:status paid
(error) NOREPLICAS Not enough good replicas to write.
```

With **shop-replica2** still stopped, only one replica is connected, and the primary now refuses writes. This trades availability for safety: a primary cut off from its replicas would otherwise keep accepting writes that nobody else has.

(The companion setting `min-replicas-max-lag` defines how recent a replica's last acknowledgement must be to count as good.) Put the setting back and restart the replica:

Recovery by partial resynchronisation

TERMINAL

```
$ docker exec shop-primary redis-cli CONFIG SET min-replicas-to-write 0
OK
$ docker start shop-replica2
shop-replica2
$ docker logs shop-replica2 2>&1 | grep -i resynchronization
* Trying a partial resynchronization (request 9c1b3e5f0a7d42e8b6f1a3c5d7e9f0a1b2c3d4e5:211).
* Successful partial resynchronization with master.
$ docker exec shop-replica2 redis-cli GET stock:sku-1001
"51"
```

The replica remembered the replication id and offset it had reached, asked the primary to continue from there, and the primary still had the missing bytes in its backlog. If the replica had been away so long that the backlog no longer covered its offset, you would see a **full** resynchronisation instead: the primary writes a new RDB snapshot and sends it, which is much more expensive. Sizing `repl-backlog-size` is how you avoid that.

Step 4: promote a replica by hand

Now kill the primary and perform the failover that Sentinel automates in the next lab.

Manual failover

TERMINAL

```
$ docker stop shop-primary
shop-primary
$ docker exec shop-replica1 redis-cli REPLICAOF NO ONE
OK
$ docker exec shop-replica2 redis-cli REPLICAOF replica1 6379
OK
$ docker exec shop-replica1 redis-cli SET order:9003:status paid
OK
$ docker exec shop-replica2 redis-cli GET order:9003:status
"paid"
$ docker exec shop-replica1 redis-cli INFO replication | head -n 4
# Replication
role:master
connected_slaves:1
slave0:ip=172.19.0.4,port=6379,state=online,offset=322,lag=0
```

`REPLICAOF NO ONE` turns a replica into a primary. Then the other replica is told to follow it. Every client now has to be pointed at the new primary; with plain Redis nothing does that for you. That is exactly the job of Sentinel and of Cluster.

Finally, bring the old primary back. It restarts with the command in the Compose file, which does **not** say `--replicaof`, so it believes it is still the primary: two primaries now exist (a split brain). Demote it:

Rejoin the old primary as a replica

TERMINAL

```
$ docker start shop-primary
shop-primary
$ docker exec shop-primary redis-cli GET order:9003:status
(nil)
$ docker exec shop-primary redis-cli REPLICAOF replica1 6379
OK
$ docker exec shop-primary redis-cli GET order:9003:status
"paid"
```

Before the demotion the old primary knew nothing about `order:9003`. After `REPLICAOF` it discarded its own dataset and copied the new primary's, so any write that had reached only the old primary during the split would be lost for good. This is the core of why asynchronous replication is not a durability guarantee.

What to observe. One-way replication; offsets and replication ids; `READONLY`; `WAIT` and `min-replicas-to-write` as partial safety nets; partial versus full resync; the need to repoint clients after a manual failover.

Cleanup.

Terminal

TERMINAL

```
$ docker compose down -v
```

LAB 2.7 · Lab: High availability with Sentinel

Goal. Add three Sentinel processes that watch the primary and promote a replica automatically. You will kill the primary and watch the election in the logs. About 30 minutes. Make a folder `lab-sentinel` with two files.

Sentinel needs a configuration file, and it **rewrites** that file as it learns the topology, so each Sentinel must have its own writable copy. The Compose file copies a template to `/tmp` on startup.

lab-sentinel/sentinel.conf

CONF

```
port 26379
sentinel resolve-hostnames yes
sentinel monitor shopmaster primary 6379 2
sentinel down-after-milliseconds shopmaster 5000
sentinel failover-timeout shopmaster 15000
sentinel parallel-syncs shopmaster 1
```

The key line is `sentinel monitor <name> <host> <port> <quorum>`. The name `shopmaster` is what clients ask for. The **quorum** of 2 is the number of Sentinels that must agree the primary is unreachable before it is marked as objectively down; electing the Sentinel that carries out the failover additionally needs a majority of all Sentinels (2 of 3). `down-after-milliseconds` is how long a node must stay silent before one Sentinel considers it down. `resolve-hostnames yes` lets us use the Compose service name `primary` instead of an IP address.

lab-sentinel/docker-compose.yml

YAML

```
services:
  primary:
    image: redis:8
    container_name: s-primary
    command: ["redis-server", "--appendonly", "yes"]

  replica1:
    image: redis:8
    container_name: s-replica1
    command: ["redis-server", "--replicaof", "primary", "6379"]
    depends_on: [primary]

  replica2:
    image: redis:8
    container_name: s-replica2
    command: ["redis-server", "--replicaof", "primary", "6379"]
    depends_on: [primary]

  sentinel1: &sentinel
    image: redis:8
    container_name: s-sentinel1
    command: >
      sh -c "cp /conf/sentinel.conf /tmp/sentinel.conf &&
        redis-sentinel /tmp/sentinel.conf"
    volumes:
      - ./sentinel.conf:/conf/sentinel.conf:ro
    depends_on: [primary, replica1, replica2]
    ports:
      - "26379:26379"

  sentinel2:
    <<: *sentinel
    container_name: s-sentinel2
    ports:
      - "26380:26379"

  sentinel3:
    <<: *sentinel
    container_name: s-sentinel3
    ports:
      - "26381:26379"
```

Step 1: look at what Sentinel knows

Ask Sentinel for the primary

TERMINAL

```
$ docker compose up -d
$ docker exec s-sentinel1 redis-cli -p 26379 SENTINEL get-master-addr-by-name shopmaster
1) "172.20.0.2"
2) "6379"
$ docker exec s-sentinel1 redis-cli -p 26379 SENTINEL ckquorum shopmaster
OK 3 usable Sentinels. Quorum and failover authorization can be reached
```

This is how a Sentinel-aware client finds the primary: it asks any Sentinel for the address instead of having it hard-coded. Sentinels discover each other and the replicas automatically through the primary, so the configuration only mentions the primary. `SENTINEL ckquorum` is the health check you run after deploying a Sentinel setup: it confirms that enough Sentinels are alive and agree.

Step 2: kill the primary and watch

Open a second terminal and follow a Sentinel's log, then stop the primary in the first one.

Terminal 2

TERMINAL

```
$ docker logs -f s-sentinel1 2>&1 | grep -E "sdown|odown|elected|promoted|switch-master"
```

Terminal 1

TERMINAL

```
$ docker stop s-primary
s-primary
```

After roughly the `down-after-milliseconds` interval, the log shows the stages of a failover. The lines below are typical; addresses and timings differ:

Sentinel log (abridged)

TERMINAL

```
+sdown master shopmaster 172.20.0.2 6379
+odown master shopmaster 172.20.0.2 6379 #quorum 2/2
+elected-leader master shopmaster 172.20.0.2 6379
+selected-slave slave 172.20.0.3:6379 172.20.0.3 6379 @ shopmaster 172.20.0.2 6379
+promoted-slave slave 172.20.0.3:6379 172.20.0.3 6379 @ shopmaster 172.20.0.2 6379
+switch-master shopmaster 172.20.0.2 6379 172.20.0.3 6379
```

Read it as a story. `+sdown` is **subjectively down**: this Sentinel can no longer reach the primary. `+odown` is **objectively down**: enough Sentinels (the quorum) agree. One Sentinel is elected leader, picks the best replica (by priority, replication offset and run id), promotes it, and `+switch-master` announces the new address. (The log keeps the old terms `slave` and `master` for these event names.)

The answer has changed

TERMINAL

```
$ docker exec s-sentinel2 redis-cli -p 26379 SENTINEL get-master-addr-by-name shopmaster
1) "172.20.0.3"
2) "6379"
$ docker exec s-replica1 redis-cli INFO replication | head -n 3
# Replication
role:master
connected_slaves:1
```

Step 3: the old primary comes back

It rejoins as a replica, without any manual command

TERMINAL

```
$ docker start s-primary
s-primary
$ sleep 10
$ docker exec s-primary redis-cli INFO replication | head -n 4
# Replication
role:slave
master_host:172.20.0.3
master_port:6379
```

Sentinel noticed the old primary return and reconfigured it as a replica of the new one. That is the difference from the manual lab, where you had to do it yourself.

GOTCHA

Sentinel returns addresses, and addresses inside Docker are private. `get-master-addr-by-name` returned `172.20.0.3`, an address on the Compose network. An application running on your host machine cannot reach it, so a Sentinel-aware client on the host would discover a primary it cannot connect to. Run the application in the same Docker network, or configure the nodes to announce reachable addresses (`replica-announce-ip`, `replica-announce-port`). This trips up almost everyone who tries Sentinel with containers.

What to observe. Quorum versus majority; the sequence `sdown` then `odown` then election then promotion; clients discover the primary through Sentinel; the returned address must be reachable by the client; failover time is dominated by `down-after-milliseconds`.

Cleanup.

Terminal

TERMINAL

```
$ docker compose down -v
```

INTERVIEW ANGLE

What does the quorum in a Sentinel setup control, and why run three Sentinels? The quorum is how many Sentinels must agree that the primary is down before a failover can start. Actually performing the failover also needs authorization from a **majority** of all Sentinels, so you want an odd number, at least three, on independent machines or zones. With two Sentinels a single failure can leave no majority and no failover; with one Sentinel, Sentinel itself is the single point of failure.

LAB 2.8 · Lab: A six-node Redis Cluster

Goal. Build a cluster of three primaries and three replicas, see how keys map to hash slots, trigger and obey a `MOVED` redirect, learn why multi-key commands need hash tags, and kill a primary. About 40 minutes. Make a folder `lab-cluster`. The theory (slots, gossip, failure detection) belongs to the cluster chapters in the scaling Part; here you just need to know that Redis Cluster splits the keyspace into **16384 hash slots** and each primary owns a range of them.

Cluster nodes announce their own IP addresses to each other and to clients, so we give each container a fixed address on a private network instead of relying on port mappings.

lab-cluster/docker-compose.yml

YAML

```
x-node: &node
  image: redis:8
  command:
    - redis-server
    - --cluster-enabled
    - "yes"
    - --cluster-config-file
    - /data/nodes.conf
    - --cluster-node-timeout
    - "5000"
    - --appendonly
    - "yes"

services:
  node1:
    <<: *node
    container_name: c-node1
    networks:
      shopnet: { ipv4_address: 172.30.0.11 }
  node2:
    <<: *node
    container_name: c-node2
    networks:
      shopnet: { ipv4_address: 172.30.0.12 }
  node3:
    <<: *node
    container_name: c-node3
    networks:
      shopnet: { ipv4_address: 172.30.0.13 }
  node4:
    <<: *node
    container_name: c-node4
```

lab-cluster/docker-compose.yml (continued)

YAML

```

networks:
  shopnet: { ipv4_address: 172.30.0.14 }
node5:
  <<: *node
  container_name: c-node5
  networks:
    shopnet: { ipv4_address: 172.30.0.15 }
node6:
  <<: *node
  container_name: c-node6
  networks:
    shopnet: { ipv4_address: 172.30.0.16 }

networks:
  shopnet:
    ipam:
      config:
        - subnet: 172.30.0.0/24

```

--cluster-enabled yes switches the node into cluster mode; --cluster-config-file is where the node stores the cluster state it learns (never edit it by hand); --cluster-node-timeout 5000 means a node unreachable for 5 seconds starts being suspected (the default is 15 seconds; we shorten it so the lab does not take long). Right after `docker compose up -d` the six nodes are running but know nothing about each other.

Step 1: create the cluster

Form the cluster

TERMINAL

```

$ docker compose up -d
$ docker exec -it c-node1 redis-cli --cluster create \
  172.30.0.11:6379 172.30.0.12:6379 172.30.0.13:6379 \
  172.30.0.14:6379 172.30.0.15:6379 172.30.0.16:6379 \
  --cluster-replicas 1
>>> Performing hash slots allocation on 6 nodes...
Master[0] -> Slots 0 - 5460
Master[1] -> Slots 5461 - 10922
Master[2] -> Slots 10923 - 16383
Adding replica 172.30.0.15:6379 to 172.30.0.11:6379
Adding replica 172.30.0.16:6379 to 172.30.0.12:6379
Adding replica 172.30.0.14:6379 to 172.30.0.13:6379
M: 017211931d591398de0d0c9d89a81360c56c9c5c 172.30.0.11:6379
  slots:[0-5460] (5461 slots) master
M: 198c8a72c9f80089f11ec584a646cbe84d9a483f 172.30.0.12:6379
  slots:[5461-10922] (5462 slots) master
M: 6bb3ea7c89f86b5fe8a37fe9af2d23a5dbc63034 172.30.0.13:6379
  slots:[10923-16383] (5461 slots) master
S: ffb6a65467706a689ef64009dfe60766436f85821 172.30.0.14:6379
  replicates 6bb3ea7c89f86b5fe8a37fe9af2d23a5dbc63034
S: de0cf6ef0c01fb5a99e92f5a50a14515aa0ee82e 172.30.0.15:6379
  replicates 017211931d591398de0d0c9d89a81360c56c9c5c
S: 944007d7c09ed4d79b55ca26c5a4bc500afc482f 172.30.0.16:6379
  replicates 198c8a72c9f80089f11ec584a646cbe84d9a483f
Can I set the above configuration? (type 'yes' to accept): yes
>>> Nodes configuration updated
>>> Assign a different config epoch to each node
>>> Sending CLUSTER MEET messages to join the cluster
Waiting for the cluster to join
.
>>> Performing Cluster Check (using node 172.30.0.11:6379)
[OK] All nodes agree about slots configuration.
>>> Check for open slots...
>>> Check slots coverage...
[OK] All 16384 slots covered.

```

This listing is abridged, and the exact wording differs slightly between versions. Read it: --cluster-replicas 1 asked for one replica per primary, so with six nodes you get three primaries (M:) and three replicas (S:). The 16384 slots were divided into three ranges. Node IDs are random 40-hex-digit names that will differ on your machine. The closing `[OK] All 16384 slots covered` is the line that matters: if any slot has no owner, the cluster refuses to serve it.

Inspect the cluster

TERMINAL

```
$ docker exec c-node1 redis-cli CLUSTER INFO | head -n 7
cluster_state:ok
cluster_slots_assigned:16384
cluster_slots_ok:16384
cluster_slots_pfail:0
cluster_slots_fail:0
cluster_known_nodes:6
cluster_size:3
$ docker exec c-node1 redis-cli CLUSTER NODES
ffba6546... 172.30.0.14:6379@16379 slave 6bb3ea7c... 0 1760086500000 3 connected
198c8a72... 172.30.0.12:6379@16379 master - 0 1760086500500 2 connected 5461-10922
944007d7... 172.30.0.16:6379@16379 slave 198c8a72... 0 1760086499500 2 connected
017211931... 172.30.0.11:6379@16379 myself,master - 0 0 1 connected 0-5460
de0cf6ef... 172.30.0.15:6379@16379 slave 017211931... 0 1760086500500 1 connected
6bb3ea7c... 172.30.0.13:6379@16379 master - 0 1760086499000 3 connected 10923-16383
```

(The node IDs are shortened here for width.) In each **CLUSTER NODES** line you see the address **ip:port@busport**, the role, the master it replicates, a connection state, and for primaries the slots owned. The second port, **16379**, is the **cluster bus** (data port plus 10000) over which nodes gossip; it must be reachable between nodes, which in real firewalls is an often-forgotten rule.

Step 2: which slot is my key in?

The slot is **CRC16(key) mod 16384**. You can ask any node:

Slots of ShopStream keys

TERMINAL

```
$ docker exec c-node1 redis-cli CLUSTER KEYSLOT product:1001
(integer) 181
$ docker exec c-node1 redis-cli CLUSTER KEYSLOT stock:sku-1001
(integer) 6106
$ docker exec c-node1 redis-cli CLUSTER KEYSLOT cart:u-17
(integer) 14402
```

Slot 181 lives on the first primary (0 to 5460), 6106 on the second, 14402 on the third. So these three keys land on three different machines.

Step 3: get redirected

Connect to node 1 **without** cluster mode and write a key that belongs elsewhere:

A plain client meets MOVED

TERMINAL

```
$ docker exec -it c-node1 redis-cli
127.0.0.1:6379> SET stock:sku-1001 50
(error) MOVED 6106 172.30.0.12:6379
127.0.0.1:6379> SET product:1001:name "Steel Mug"
OK
127.0.0.1:6379> EXIT
```

MOVED 6106 172.30.0.12:6379 means: "slot 6106 is not mine; ask 172.30.0.12". The server never forwards the request for you. A cluster-aware client keeps a map of slots to nodes, learns it at startup, sends each command straight to the right node, and updates the map when it receives **MOVED**. **redis-cli -c** does the following for you:

Cluster mode follows redirects

TERMINAL

```
$ docker exec -it c-node1 redis-cli -c
127.0.0.1:6379> SET stock:sku-1001 50
-> Redirected to slot [6106] located at 172.30.0.12:6379
OK
172.30.0.12:6379> GET stock:sku-1001
"50"
172.30.0.12:6379> GET cart:u-17
-> Redirected to slot [14402] located at 172.30.0.13:6379
(nil)
172.30.0.13:6379> EXIT
```

Notice the prompt changing to show which node you are talking to. Spring Data Redis and Lettuce do the same thing programmatically with a topology refresh; the Java chapters show how to configure it.

Step 4: multi-key commands and hash tags

A command touching several keys works only if all the keys are in the same slot. Otherwise:

```
CROSSSLOT
$ docker exec -it c-node1 redis-cli -c
127.0.0.1:6379> MSET cart:u-17 x stock:sku-1001 49
(error) CROSSSLOT Keys in request don't hash to the same slot
```

A **hash tag** fixes this: if a key contains `{...}`, only the text between the first `{` and the next `}` is hashed. Put the entity id in braces and everything about that entity lands in one slot:

```
Hash tags force co-location
127.0.0.1:6379> CLUSTER KEYSLOT {cart:u-17}:items
(integer) 14402
127.0.0.1:6379> CLUSTER KEYSLOT {cart:u-17}:meta
(integer) 14402
127.0.0.1:6379> MSET {cart:u-17}:items 3 {cart:u-17}:meta "updated"
-> Redirected to slot [14402] located at 172.30.0.13:6379
OK
172.30.0.13:6379> EXIT
```

Both keys hash exactly like the string `cart:u-17` (same slot 14402 as the plain key above), so a **MULTI/EXEC** transaction or a Lua script over them is allowed. The price is that all of one customer's data lives on one node, so one very hot tag is a hot shard. Choose tags to group things that are used together, not "everything of a tenant".

Step 5: kill a primary

Write some data, then stop the node that owns slots 0 to 5460 and watch the cluster heal.

```
Failover
$ docker stop c-node1
c-node1
$ sleep 10
$ docker exec c-node2 redis-cli CLUSTER NODES | grep -E "172.30.0.1[15]"
de0cf6ef... 172.30.0.15:6379@16379 master - 0 1760086620000 7 connected 0-5460
017211931... 172.30.0.11:6379@16379 master,fail - 1760086608000 1760086606000 1 disconnected
$ docker exec c-node2 redis-cli CLUSTER INFO | head -n 1
cluster_state:ok
```

The replica on `172.30.0.15` noticed (through the other primaries voting) that its primary had been unreachable for longer than the node timeout and was promoted; it now owns slots 0 to 5460 and the cluster state is **ok** again. The old primary is shown as **master,fail**. Bring it back:

```
The old primary returns as a replica
$ docker start c-node1
c-node1
$ sleep 5
$ docker exec c-node2 redis-cli CLUSTER NODES | grep 172.30.0.11
017211931... 172.30.0.11:6379@16379 slave de0cf6ef... 0 1760086640000 7 connected
$ docker exec c-node1 redis-cli --cluster check 172.30.0.11:6379 | tail -n 3
[OK] All nodes agree about slots configuration.
>>> Check for open slots...
>>> Check slots coverage...
```

It rejoined by itself, learned from gossip that it had lost its slots, and became a replica of its former replica. No operator command was needed.

Try it. Stop **both** a primary and its replica (`c-node2` and `c-node6`) and run **CLUSTER INFO** on another node: with the default `cluster-require-full-coverage yes` the state becomes **fail** and the whole cluster stops answering queries, because slots 5461 to 10922 have no owner. That default is a deliberate consistency choice, and the cluster chapters explain when to relax it.

What to observe. Three ranges of slots; **MOVED** and **-c**; **CROSSSLOT** and hash tags; the bus port; automatic promotion; the old primary returning as a replica.

Cleanup.

```
Terminal
$ docker compose down -v
```

GOTCHA

Reach the cluster from inside the network, or announce reachable addresses. The nodes tell clients to connect to `172.30.0.x`, an address that exists only inside the Docker network. From your laptop `redis-cli -c -p ...` against a published port will be redirected to unreachable IPs and hang. Run clients as containers on `shopnet`, as in this lab, or configure `cluster-announce-ip` and port mappings consciously. The same issue appears whenever a cluster sits behind NAT or a load balancer.

LAB 2.9 · Lab: Benchmarking and the pipelining effect

Goal. Measure throughput and latency with `redis-benchmark`, then see what pipelining, payload size, client count and fsync policy do to them. About 25 minutes.

The numbers below are **illustrative** (a laptop-class machine, Redis and the benchmark on the same host through the loopback interface). Yours will differ a lot; what you should compare are the ratios. Benchmarks that run on the same machine as the server are good for relative comparisons and bad as capacity figures.

Start a clean server to measure

TERMINAL

```
$ docker run -d --name bench -p 6390:6379 redis:8
$ docker exec bench redis-benchmark -q -n 100000 -c 50 -t set,get
SET: 118343.20 requests per second, p50=0.239 msec
GET: 136612.02 requests per second, p50=0.207 msec
```

`-n` is the total number of requests, `-c` the number of parallel client connections (default 50), `-t` selects which tests to run, and `-q` prints one summary line per test. `p50` is the median latency. Redis commands are small; the cost per request is dominated by the network round trip, not by Redis. Now batch 16 commands per round trip with `-P 16`:

Pipelining

TERMINAL

```
$ docker exec bench redis-benchmark -q -n 200000 -c 50 -t set,get -P 16
SET: 574712.69 requests per second, p50=0.863 msec
GET: 1190476.25 requests per second, p50=0.495 msec
```

Throughput rose four to nine times. Latency per **batch** went up (each reply now waits for 16 commands), but the cost per command fell, because the client no longer pays a network round trip for each one and the server does fewer system calls. This is the effect the pipelining chapter explains with numbers, and the reason the Java clients let you batch commands.

Vary one thing at a time**Payload size, random keys and a custom command**

TERMINAL

```
$ docker exec bench redis-benchmark -q -n 100000 -t set -d 4096
SET: 98039.22 requests per second, p50=0.263 msec
$ docker exec bench redis-benchmark -q -n 100000 -t set -r 1000000
SET: 117647.06 requests per second, p50=0.247 msec
$ docker exec bench redis-benchmark -q -n 100000 LPUSH queue:test __rand_int__
LPUSH: 124223.60 requests per second, p50=0.231 msec
$ docker exec bench redis-cli DBSIZE
(integer) 95388
```

`-d 4096` uses 4 KB values instead of the default 3 bytes (larger payloads cost more bandwidth and copying). `-r 1000000` makes each request use a random key among a million; without `-r` every request hits the same key, which flatters caching in the CPU. The last benchmark runs any command you give it and replaces `__rand_int__` with a random number. Look at `DBSIZE` afterwards: the benchmark **wrote tens of thousands of keys** into your server (100000 requests spread over a million possible keys). Never run `redis-benchmark` against a server whose data you care about.

The full latency picture

Without `-q` you get the whole report:

Full report (abridged)

TERMINAL

```
$ docker exec bench redis-benchmark -n 100000 -t get -c 50
===== GET =====
100000 requests completed in 0.78 seconds
50 parallel clients
3 bytes payload
```

Full report (abridged) (continued)

TERMINAL

```
keep alive: 1
multi-thread: no
```

Latency by percentile distribution:

```
0.000% <= 0.087 milliseconds (cumulative count 1)
50.000% <= 0.207 milliseconds (cumulative count 51004)
99.219% <= 0.431 milliseconds (cumulative count 99250)
100.000% <= 2.847 milliseconds (cumulative count 100000)
```

Summary:

```
throughput summary: 128205.13 requests per second
latency summary (msec):
      avg      min      p50      p95      p99      max
    0.208    0.040    0.207    0.287    0.399    2.847
```

Averages hide the tail. For SLAs people watch p95, p99 and the maximum, because a user who hits the slow 1 percent notices. The occasional multi-millisecond maximum is typical of a shared machine.

What durability costs

Start a second server that fsyncs the AOF on every write and compare:

appendfsync always versus everysec

TERMINAL

```
$ docker run -d --name bench-always -p 6391:6379 redis:8 \
  redis-server --appendonly yes --appendfsync always
$ docker run -d --name bench-sec -p 6392:6379 redis:8 \
  redis-server --appendonly yes --appendfsync everysec
$ docker exec bench-always redis-benchmark -q -n 20000 -t set
SET: 9876.54 requests per second, p50=4.911 msec
$ docker exec bench-sec redis-benchmark -q -n 100000 -t set
SET: 105263.16 requests per second, p50=0.263 msec
```

The ratio is the point (illustrative figures): with **always**, every write waits for the disk, so throughput is limited by your storage's **fsync** latency; **everysec** runs near memory speed and accepts up to about a second of loss. Run it on a cloud disk and on a laptop SSD and you will see very different **always** numbers, which is why persistence settings should be benchmarked on production-like hardware. Pipelining softens the cost of **always** considerably, because many commands share one **fsync** (try **-P 16**).

Reading benchmarks honestly

- The load generator is a program too. With a fast server you can saturate the **benchmark's** CPU first; use **--threads 4** (available in current versions) or run it on a separate machine.
- Default tests use tiny values and one key shape. Your workload has bigger values, mixed commands and Lua scripts. Benchmark your own mix, with your client library, from where your application runs.
- Containers, virtual networks and noisy neighbours change latency. Compare like with like and repeat runs.
- **redis-benchmark** shows peak behaviour, not the effect of persistence rewrites, replication or eviction; run it while those are active if they matter.
- For richer workloads the Redis project maintains **memtier_benchmark**, which supports ratios of reads to writes and key distributions.

What to observe. Pipelining multiplies throughput; payload size and key randomness change results; the tail latency tells more than the average; **fsync** policy dominates write speed.

Cleanup.

Terminal

TERMINAL

```
$ docker rm -f bench bench-always bench-sec
```

REAL WORLD

Benchmark numbers you publish internally should always state: hardware, Redis version and config, client count, pipeline depth, payload size, key space, and whether the client ran on the same machine. Without those, nobody can reproduce or compare them, and a number like "Redis does a million operations per second" travels without its context and misleads the next team.

KEY POINTS

- The labs give you muscle memory: **SET/INCR/EXPIRE**, lists and sets, hashes and sorted sets, streams with groups, persistence modes, replication, Sentinel and Cluster.
- Persistence is a trade between speed and the data-loss window: RDB loses since the last snapshot, AOF **everysec** about a second, AOF **always** almost nothing at a throughput cost.
- Asynchronous replication does not guarantee that acknowledged writes survive a failover; **WAIT** and **min-replicas-to-write** narrow the risk.
- Sentinel automates failover for one dataset; Cluster shards the keyspace into 16384 slots and needs a cluster-aware client, hash tags for multi-key operations, and reachable node addresses.
- Benchmark relative changes (pipelining, payload, fsync) on realistic hardware, and never against data you care about.

PART 3

Data Structures

Strings, lists, hashes, sets and sorted sets: the five workhorses, with their commands, internals and patterns.

In this part

- 3.1 Strings
- 3.2 Lists
- 3.3 Hashes
- 3.4 Sets
- 3.5 Sorted sets
- 3.6 Choosing the right structure
- 3.7 Streams
- 3.8 Pub/Sub and Keyspace Notifications
- 3.9 Bitmaps and Bitfields
- 3.10 HyperLogLog
- 3.11 Geospatial Indexes
- 3.12 Iterating Safely: SCAN and Friends
- 3.13 JSON Documents
- 3.14 The Query Engine: Secondary Indexes and Search
- 3.15 Vector Search and Vector Sets
- 3.16 Probabilistic Data Structures
- 3.17 Time Series

3.1 Strings

The string is the simplest Redis type and the one you will use most. A string value is a sequence of bytes, nothing more: it can hold text, a number, a serialized Java object, a JSON document, a JPEG. Redis does not care. Almost every other feature in this book (caching, counters, locks, sessions, rate limits) is built on strings, so it pays to know every corner of them.

Strings solve three everyday problems. They let you store a value under a name and read it back in one round trip (a cache entry). They let you change a number atomically on the server without a read-modify-write cycle (a counter). And, with the right **SET** options, they let you make a decision atomically ("create this key only if nobody else has", which is the heart of a lock).

3.1.1 Reading and writing: SET and GET

The basic pair is **SET** **key** **value** and **GET** **key**. **SET** always overwrites whatever was there, even a value of a different type, and it also discards any TTL (time to live) the key had. **GET** returns the value or **(nil)** when the key does not exist.

Store and read a key

TERMINAL

```
127.0.0.1:6379> SET user:1:name "Asha"
OK
127.0.0.1:6379> GET user:1:name
"Asha"
127.0.0.1:6379> GET user:2:name
(nil)
127.0.0.1:6379> SET user:1:name "Asha K"
OK
127.0.0.1:6379> STRLEN user:1:name
(integer) 6
```

SET takes options that turn it into a small toolbox. They are the reason the older commands **SETNX**, **SETEX**, **PSETEX** and **GETSET** are considered legacy: a single **SET** can express all of them.

Option	Meaning	Replaces legacy
NX	Set only if the key does not exist	SETNX

Option	Meaning	Replaces legacy
XX	Set only if the key already exists	-
GET	Return the old value (nil if there was none) instead of OK	GETSET
EX seconds	Expire after this many seconds	SETEX
PX milliseconds	Expire after this many milliseconds	PSETEX
EXAT / PXAT	Expire at an absolute Unix time (seconds / milliseconds)	-
KEEPTTL	Keep the key's current TTL instead of clearing it	-

Options of SET (conditions and expiry are two independent groups).

You may combine one condition with one expiry option. Two conditions together (**NX** with **XX**), or two expiry options together, are syntax errors.

```

SET with NX, XX, expiry and KEEPTTL
127.0.0.1:6379> SET session:abc "u42" EX 1800
OK
127.0.0.1:6379> TTL session:abc
(integer) 1800
127.0.0.1:6379> SET session:abc "u42-v2" KEEPTTL
OK
127.0.0.1:6379> TTL session:abc
(integer) 1800
127.0.0.1:6379> SET session:abc "u42-v3"
OK
127.0.0.1:6379> TTL session:abc
(integer) -1
127.0.0.1:6379> SET lock:order:7 "worker-1" NX PX 30000
OK
127.0.0.1:6379> SET lock:order:7 "worker-2" NX PX 30000
(nil)
127.0.0.1:6379> SET missing "x" XX
(nil)
127.0.0.1:6379> SET user:1:name "Asha"
OK
127.0.0.1:6379> SET user:1:name "Asha K" GET
"Asha"

```

Look at the last three commands. A **NX** that loses the race returns **(nil)**, which is how a lock attempt reports "someone else has it". **XX** on a missing key also returns **(nil)**. And **SET ... GET** returns the previous value while writing the new one, in one atomic step. The third **SET** of **session:abc** shows the trap that **KEEPTTL** exists for: a plain **SET** wipes the TTL (**-1** means "no expiry"), so a session you merely "touch" would live forever.

GOTCHA

A plain **SET** on an existing key removes its TTL. This is the classic way to create an immortal key by accident: you cache something with **EX 600**, later refresh it with a bare **SET**, and it never expires again. Always pass the expiry again, or use **KEEPTTL** when you want to preserve it.

3.1.2 Reading with side effects: GETEX and GETDEL

Two read commands change the key as they read it, both atomically.

- GETEX key [EX seconds | PX ms | EXAT ts | PXAT ts | PERSIST]** returns the value and sets or clears its TTL. It is the right tool for a sliding session: read the session and extend its life in one command.
- GETDEL key** returns the value and deletes the key. It is perfect for one-time tokens such as an e-mail verification code or a "consume once" download link, because two concurrent callers can never both receive the value.

GETEX and GETDEL

TERMINAL

```
127.0.0.1:6379> SET session:abc "u42" EX 60
OK
127.0.0.1:6379> GETEX session:abc EX 1800
"u42"
127.0.0.1:6379> TTL session:abc
(integer) 1800
127.0.0.1:6379> SET otp:asha "493021" EX 300
OK
127.0.0.1:6379> GETDEL otp:asha
"493021"
127.0.0.1:6379> GETDEL otp:asha
(nil)
```

3.1.3 Many keys at once: MSET and MGET

MSET *k1 v1 k2 v2 ...* writes many keys and **MGET** *k1 k2 ...* reads many. Each is one round trip instead of *N*, which matters because network latency usually dominates the cost of a Redis command (see the chapter on pipelining later in the book). **MSET** is atomic: other clients see all of the writes or none. **MGET** returns *(nil)* in the slot of any missing or non-string key.

MSET and MGET

TERMINAL

```
127.0.0.1:6379> MSET product:1:price 1999 product:2:price 2499 product:3:price 899
OK
127.0.0.1:6379> MGET product:1:price product:3:price product:9:price
1) "1999"
2) "899"
3) (nil)
```

In a Redis Cluster all keys of one **MSET** or **MGET** must live in the same hash slot, otherwise the command fails with a **CROSSSLOT** error. The cluster chapter explains how hash tags (*{...}* in a key name) solve this.

3.1.4 Counters: the INCR family

Strings that look like integers can be incremented in place. **INCR** and **DECR** add or subtract one, **INCRBY** and **DECRBY** take an amount, and **INCRBYFLOAT** works with decimals. A key that does not exist is treated as *0*, so the first **INCR** creates it with value *1*. The result is the new value.

Atomic counters

TERMINAL

```
127.0.0.1:6379> INCR page:home:views
(integer) 1
127.0.0.1:6379> INCR page:home:views
(integer) 2
127.0.0.1:6379> INCRBY page:home:views 10
(integer) 12
127.0.0.1:6379> DECR stock:sku-42
(integer) -1
127.0.0.1:6379> INCRBYFLOAT wallet:asha 12.5
"12.5"
127.0.0.1:6379> INCRBYFLOAT wallet:asha -2.25
"10.25"
127.0.0.1:6379> SET greeting "hello"
OK
127.0.0.1:6379> INCR greeting
(error) ERR value is not an integer or out of range
```

The value must be a base-10 signed 64-bit integer. Anything else gives the error you see in the last line. **INCR** past the 64-bit limit also fails instead of wrapping around.

Why is this valuable? Consider the alternative: the application does **GET**, adds one, then **SET**. Two servers doing that at the same moment both read *41* and both write *42*; one increment is lost. **INCR** runs inside the single-threaded command loop, so it is a single indivisible step and no update is ever lost. This is the idea behind page counters, rate limiters, ID generators and stock counters.

TIP

Use one **INCR** on a key as a cheap unique ID generator. **INCR seq:order** returns a number nobody else will ever receive from that key. Reserve a block with **INCRBY seq:order 100** if you want to hand out IDs locally without a round trip each time.

3.1.5 Working with parts of a string

Because strings are byte arrays you can treat them as buffers.

APPEND, GETRANGE and SETRANGE

TERMINAL

```
127.0.0.1:6379> APPEND log:today "login;"
(integer) 6
127.0.0.1:6379> APPEND log:today "view:42;"
(integer) 14
127.0.0.1:6379> GET log:today
"login;view:42;"
127.0.0.1:6379> GETRANGE log:today 0 4
"login"
127.0.0.1:6379> GETRANGE log:today -3 -1
"42;"
127.0.0.1:6379> SETRANGE log:today 0 "LOGIN"
(integer) 14
127.0.0.1:6379> GET log:today
"LOGIN;view:42;"
```

GETRANGE **key start end** takes inclusive offsets, and negative offsets count from the end (**-1** is the last byte). **SETRANGE** overwrites bytes in place and pads with zero bytes if you write beyond the current end. **APPEND** returns the new length. Appending in a loop is cheap in Redis (amortized), but a key that grows without bound is a memory leak; see the section on big values below.

Redis 7.0 added **LCS**, which computes the longest common subsequence of two string values on the server. It is occasionally useful for diffing two stored versions of a text without downloading them.

LCS: longest common subsequence

TERMINAL

```
127.0.0.1:6379> MSET k1 ohmytext k2 mynewtext
OK
127.0.0.1:6379> LCS k1 k2
"mytext"
127.0.0.1:6379> LCS k1 k2 LEN
(integer) 6
```

3.1.6 Compare-and-set and DELEX in Redis 8.4

Locks and optimistic updates need "do this only if the value is still what I think it is". Until recently that required a Lua script or a transaction with **WATCH**. Redis 8.4 added conditions for this directly to **SET** and a new command **DELEX**. The syntax below is quoted from the command reference for 8.4:

```
SET key value [NX | XX | IFEQ ifeq-value | IFNE ifne-value |
               IFDEQ ifdeq-digest | IFDNE ifdne-digest]
[GET] [EX seconds | PX ms | EXAT ts | PXAT ts | KEeptTL]

DELEX key [IFEQ ifeq-value | IFNE ifne-value |
           IFDEQ ifdeq-digest | IFDNE ifdne-digest]
```

- **IFEQ value**: apply only if the current value equals **value**. If the key does not exist, **SET** does not create it.
- **IFNE value**: apply only if the current value differs. If the key does not exist, **SET** creates it.
- **IFDEQ** and **IFDNE** do the same with a digest: a fixed-size XXH3 hash of the value, as a hexadecimal string, which you can obtain with the new **DIGEST** command. Use these when the value is large and you do not want to send it twice.

DELEX returns **1** if it deleted the key and **0** if the key was missing or the condition was false. With a condition, the key must hold a string, otherwise it returns an error. The classic use is releasing a lock safely: delete the lock only if it still holds my token.

REDIS

```
# acquire (works on every Redis version)
SET lock:order:7 "worker-1-token" NX PX 30000
# release, Redis 8.4 and later: delete only if I still own it
DELEX lock:order:7 IFEQ "worker-1-token"
# refresh the value only while it is still mine
SET lock:order:7 "worker-1-token" IFEQ "worker-1-token" PX 30000
```

NOTE

These options exist only in Redis 8.4 and later. On older servers, or on a managed service that has not upgraded, the same release step needs a small Lua script (compare, then **DEL**). The scripting chapter shows it, and the patterns Part uses it for distributed locks. If your code must run on several versions, keep the Lua version.

3.1.7 What a string really is inside

A string in Redis is stored by an SDS ("simple dynamic string"), a length-prefixed byte buffer. Two consequences follow. It is **binary safe**: it may contain zero bytes and any encoding, because the length is stored explicitly instead of relying on a terminator. And **STRLEN** is $O(1)$. The maximum size of a string value is **512 MB**, but you should never come near that. Large values block the single command thread while they are read, written and sent, and they hurt replication and memory fragmentation. Values of a few kilobytes are comfortable; values over about 100 KB deserve a second look.

Redis picks one of three internal encodings per value, and you can see it with **OBJECT ENCODING**:

Encoding	When	Why it matters
int	Value is an integer that fits in a 64-bit signed long	No separate buffer; the number is stored in the object itself
embstr	Short string, up to 44 bytes	Object header and buffer live in one allocation: fewer cache misses
raw	Everything longer, or any string that has been modified in place	Header and buffer are separate allocations

String encodings.

Observing encodings

TERMINAL

```
127.0.0.1:6379> SET counter 100
OK
127.0.0.1:6379> OBJECT ENCODING counter
"int"
127.0.0.1:6379> SET name "hello"
OK
127.0.0.1:6379> OBJECT ENCODING name
"embstr"
127.0.0.1:6379> APPEND name " world"
(integer) 11
127.0.0.1:6379> OBJECT ENCODING name
"raw"
127.0.0.1:6379> SET page:html "<html>...a long document of more than forty-four bytes...</html>"
OK
127.0.0.1:6379> OBJECT ENCODING page:html
"raw"
```

An **embstr** value is read-only: any in-place modification (**APPEND**, **SETRANGE**) converts it to **raw**, as the third line of the session shows. You will rarely tune this, but the sizes explain why millions of tiny string keys cost more memory than you expect: every key has per-entry overhead (the dictionary entry, the key string, the value object) on top of the bytes you store. When you have very many small related values, a hash often costs less; the chapter on hashes explains why.

3.1.8 A worked example: a fixed-window rate limiter

Counters plus expiry already give you a usable rate limiter. Allow each client IP 5 requests per 60 seconds on the login route. The key encodes the client, the route and the current minute; the first request in a window creates it, and the TTL cleans it up.

Five requests per minute, per IP

TERMINAL

```

127.0.0.1:6379> INCR rl:1.2.3.4:/login:29333
(integer) 1
127.0.0.1:6379> EXPIRE rl:1.2.3.4:/login:29333 60
(integer) 1
127.0.0.1:6379> INCR rl:1.2.3.4:/login:29333
(integer) 2
127.0.0.1:6379> INCR rl:1.2.3.4:/login:29333
(integer) 3
127.0.0.1:6379> INCR rl:1.2.3.4:/login:29333
(integer) 4
127.0.0.1:6379> INCR rl:1.2.3.4:/login:29333
(integer) 5
127.0.0.1:6379> INCR rl:1.2.3.4:/login:29333
(integer) 6
127.0.0.1:6379> TTL rl:1.2.3.4:/login:29333
(integer) 60

```

The sixth **INCR** returns 6, which is above the limit of 5, so the application rejects the request (HTTP 429) until the window key expires. The number 29333 is the current Unix time divided by 60, so every minute uses a fresh key. Here is the same idea in Spring code, with **StringRedisTemplate** (the Spring Data Redis Part explains the template in depth):

FixedWindowLimiter.java

JAVA

```

package com.shopstream.gateway;

import java.time.Duration;
import java.time.Instant;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Component;

@Component
public class FixedWindowLimiter {

    private static final long WINDOW_SECONDS = 60;
    private final StringRedisTemplate redis;

    public FixedWindowLimiter(StringRedisTemplate redis) {
        this.redis = redis;
    }

    public boolean allow(String ip, String route, long limit) {
        long window = Instant.now().getEpochSecond() / WINDOW_SECONDS;
        String key = "rl:" + ip + ":" + route + ":" + window;
        Long count = redis.opsForValue().increment(key);
        if (count != null && count == 1L) {
            redis.expire(key, Duration.ofSeconds(WINDOW_SECONDS));
        }
        return count != null && count <= limit;
    }
}

```

What to notice: **increment** maps to **INCR**, so concurrent requests from many gateway instances all see distinct counts. The **expire** call runs only when the count is 1. There is still the small gap mentioned above: a crash between the two calls would leave a key without TTL. For production, use the Lua version from the rate-limiting chapter. This simple form also allows a burst of up to twice the limit across a window boundary (5 at the end of one minute and 5 at the start of the next), a weakness that the sliding window and token bucket variants remove.

3.1.9 Strings in practice

What to store

A string can hold a plain scalar or a serialized object. For a product cache entry you can store JSON text under **product:42**. This is simple, fast to read in one command, and works with any language. The price is that you replace the whole value to change one field. If you need to read or update single fields, use a hash or the JSON type instead. The Spring Data Redis Part shows how serializers turn a Java object into bytes and back.

Typical uses in ShopStream

- **Catalogue cache:** `SET product:42 "{...json...}" EX 600` is written by the cache-aside code.
- **Counters:** `INCR page:home:views` for analytics, `DECR stock:sku-42` for inventory (with the safety checks shown in the Lua chapter).
- **Idempotency keys and locks:** `SET idem:req-9f3 "done" NX EX 86400`. A `(nil)` reply tells the second caller that the request was already seen.
- **Feature flags:** `GET ff:new-checkout` returns "on" or "off".
- **Rate limit windows:** `INCR rl:1.2.3.4:/login` followed by `EXPIRE` on the first hit.

Pitfalls

- Forgetting the TTL. Every cache or session key should expire; set it in the same command as the value (`SET ... EX`) so there is no moment when the key exists without one.
- Doing `INCR` and `EXPIRE` as two separate commands. If the client dies between them, the counter never expires. Wrap both in a transaction or a Lua script, as the rate-limiting chapter shows.
- Treating `INCRBYFLOAT` as exact money arithmetic. It uses floating point. Store money as integer minor units (cents) with `INCRBY`.
- Storing huge blobs. Keep values small; store large files in object storage and keep the URL in Redis.

INTERVIEW ANGLE

How do you implement a safe distributed lock with plain Redis strings? Acquire with a single command, `SET lock:key <random-token> NX PX <ttl>`. `NX` makes the creation atomic and `PX` guarantees the lock expires even if the holder crashes. Release only if the stored token is still yours, because otherwise you could delete a lock that expired and was re-acquired by someone else. That compare-and-delete must be atomic: a Lua script, or `DELEX key IFEQ token` on Redis 8.4 and later. Mention that a TTL-based lock can still be lost if the holder pauses longer than the TTL, which is why critical work needs fencing tokens.

KEY POINTS

- A string is a binary-safe byte buffer up to 512 MB; keep real values small.
- `SET` with `NX`, `XX`, `GET`, `EX`, `PX`, `KEEPTTL` replaces `SETNX`, `SETEX`, `GETSET`; a bare `SET` clears the TTL.
- `INCR`, `INCRBY`, `DECR` and `INCRBYFLOAT` are atomic, which makes them safe counters under concurrency.
- `GETEX` and `GETDEL` read and change a key in one step; `MSET` and `MGET` save round trips.
- Redis 8.4 adds `IFEQ`, `IFNE`, `IFDEQ`, `IFDNE` to `SET` and the `DELEX` command for compare-and-set and compare-and-delete.
- Encodings are `int`, `embstr` and `raw`; check with `OBJECT ENCODING`.

3.2 Lists

A Redis list is an ordered sequence of strings, optimized for adding and removing at the two ends. Think of it as a double-ended queue (a "deque") that lives on the server and is shared by every client. Pushing to either end is $O(1)$ no matter how long the list is. That single property makes lists the natural structure for queues, stacks, activity feeds and "latest N items" views.

Lists solve the problem of ordering and hand-off between processes. A web request pushes a job; a worker in another process pops it. The list holds the work in between, and Redis guarantees that each element is popped by exactly one caller.

3.2.1 The basic commands

Elements are added with `LPUSH` (to the head, the left end) and `RPUSH` (to the tail, the right end). They are removed with `LPOP` and `RPOP`, which also accept a count since Redis 6.2. `LRANGE key start stop` reads a slice using inclusive indexes, where negative indexes count from the tail, so `0 -1` means "everything". A list that becomes empty is deleted automatically, and popping from a missing key returns `(nil)`.

Push, pop and range

TERMINAL

```

127.0.0.1:6379> RPUSH queue:emails a b c
(integer) 3
127.0.0.1:6379> LPUSH queue:emails z
(integer) 4
127.0.0.1:6379> LRANGE queue:emails 0 -1
1) "z"
2) "a"
3) "b"
4) "c"
127.0.0.1:6379> LPOP queue:emails
"z"
127.0.0.1:6379> RPOP queue:emails 2
1) "c"
2) "b"
127.0.0.1:6379> LLEN queue:emails
(integer) 1
127.0.0.1:6379> LPOP queue:emails
"a"
127.0.0.1:6379> LPOP queue:emails
(nil)
127.0.0.1:6379> EXISTS queue:emails
(integer) 0

```

Notice the order produced by pushing several values at once. **LPUSH** recent **a b c** pushes **a**, then **b**, then **c** to the head, so the list reads **c b a**. Each value goes to the head in turn.

Two cheap helpers are worth knowing. **RPUSHX** and **LPUSHX** push only if the list already exists (they return **0** otherwise). **LLEN** returns the length in $O(1)$.

3.2.2 Reaching into the middle

Lists are not designed for random access, but Redis lets you do it, and you should know what it costs. All of the commands below walk the list from an end, so they are $O(N)$ in the distance travelled.

Command	What it does	Cost
LINDEX key i	Element at index i (negative counts from tail)	$O(N)$
LSET key i v	Replace element at index i; error if out of range	$O(N)$
LINSERT key BEFORE AFTER pivot v	Insert next to the first element equal to pivot	$O(N)$
LREM key count v	Remove up to count occurrences (count 0 means all; negative counts from the tail)	$O(N)$
LPOS key v [RANK r] [COUNT n] [MAXLEN m]	Find the index (or indexes) of matching elements	$O(N)$
LTRIM key start stop	Keep only the given range, drop the rest	$O(N)$, N removed

Index-based list commands.

LPOS, LINSERT, LREM

TERMINAL

```

127.0.0.1:6379> RPUSH l x y z y
(integer) 4
127.0.0.1:6379> LPOS l y
(integer) 1
127.0.0.1:6379> LPOS l y RANK 2
(integer) 3
127.0.0.1:6379> LPOS l y COUNT 0
1) (integer) 1
2) (integer) 3
127.0.0.1:6379> LINSERT l BEFORE z mid
(integer) 5
127.0.0.1:6379> LRANGE l 0 -1
1) "x"
2) "y"
3) "mid"
4) "z"
5) "y"
127.0.0.1:6379> LREM l 1 y
(integer) 1

```

LPOS, LINSERT, LREM (continued)

TERMINAL

```
127.0.0.1:6379> LRange l 0 -1
1) "x"
2) "mid"
3) "z"
4) "y"
127.0.0.1:6379> LSet l 100 a
(error) ERR index out of range
```

LPOS arrived in Redis 6.0.6 and replaces the old pattern of scanning with **LRange** on the client. **RANK** selects the *n*-th match, **COUNT 0** returns all matches, and **MAXLEN** bounds the scan so a huge list cannot hold up the server.

WATCH OUT

Avoid **LINDEX**, **LSET**, **LINSERT**, **LREM** and **LPOS** on long lists in hot code paths. Redis executes commands one at a time on a single thread. A scan across a list of a million elements blocks every other client for as long as it runs. If you find yourself indexing into a list a lot, you probably want a sorted set (ordered by score) or a hash (addressed by field) instead.

3.2.3 Pattern 1: a capped "latest N" list

A very common use of lists is a rolling window of recent items: the last 10 searches of a user, the last 100 log lines, the newest 20 notifications. The recipe is **LPUSH** followed by **LTRIM**, which together keep the list bounded.

Recent searches, capped at 5

TERMINAL

```
127.0.0.1:6379> LPUSH recent:asha "red shoes"
(integer) 1
127.0.0.1:6379> LPUSH recent:asha "usb cable"
(integer) 2
127.0.0.1:6379> LPUSH recent:asha "desk lamp"
(integer) 3
127.0.0.1:6379> LPUSH recent:asha "phone case"
(integer) 4
127.0.0.1:6379> LPUSH recent:asha "monitor"
(integer) 5
127.0.0.1:6379> LPUSH recent:asha "keyboard"
(integer) 6
127.0.0.1:6379> LTRIM recent:asha 0 4
OK
127.0.0.1:6379> LRange recent:asha 0 -1
1) "keyboard"
2) "monitor"
3) "phone case"
4) "desk lamp"
5) "usb cable"
```

If two clients run this at the same moment the list can briefly exceed the cap, but the next **LTRIM** fixes it. To make the pair atomic, send both commands in a **MULTI/EXEC** transaction or a pipeline. A newer item that repeats an old one should first be removed with **LREM recent:asha 0 "monitor"** so the list does not hold duplicates.

3.2.4 Pattern 2: a work queue

The producer pushes to one end, the consumer pops from the other. Use **LPUSH** + **RPOP** (or **RPUSH** + **LPOP**); the direction does not matter as long as you are consistent.

Polling in a loop wastes CPU and adds latency. The blocking variants wait on the server until data arrives:

- **BLPOP key [key ...] timeout** and **BRPOP** block until one of the lists has an element, or the timeout in seconds passes (0 means wait forever). They return a two-element array: the key name and the value, so that you know which of several lists woke you.
- **BLMPOP timeout numkeys key [key ...] LEFT|RIGHT [COUNT n]** (Redis 7.0) is the modern general form: it can pop several elements at once from the first non-empty list.
- **LMPOP numkeys key [key ...] LEFT|RIGHT [COUNT n]** is the non-blocking twin.

A blocked consumer and a producer (two terminals)

TERMINAL

```
# terminal 1: waits, no CPU used, no busy loop
127.0.0.1:6379> BLPOP queue:emails 30
# terminal 2:
127.0.0.1:6380> RPUSH queue:emails "welcome:u42"
(integer) 1
# terminal 1 wakes immediately:
1) "queue:emails"
2) "welcome:u42"
(4.21s)
```

Blocking commands block only the calling client, not the server. If several consumers wait on the same list, Redis serves them in the order in which they started waiting, one element each. This is the "competing consumers" model: add workers to add throughput.

The weakness: a popped job is gone

BLPOP removes the element before your code has processed it. If the worker crashes after the pop, the job is lost. For anything that matters you want a reliable queue, and lists support one with **LMOVE**.

LMOVE source destination LEFT|RIGHT LEFT|RIGHT atomically pops from one list and pushes onto another, and returns the element. Move each job from **jobs** to a per-worker **jobs:processing** list, do the work, then remove it from the processing list with **LRM** when finished. If a worker dies, a janitor process can find stale entries in the processing lists and push them back. **BLMOVE** is the blocking version. The older **RPOPLUSH** and **BRPOPLUSH** do the same thing with fixed directions and are deprecated.

Reliable queue with LMOVE

TERMINAL

```
127.0.0.1:6379> RPUSH jobs "email:101" "email:102" "email:103"
(integer) 3
127.0.0.1:6379> LMOVE jobs jobs:processing:w1 LEFT RIGHT
"email:101"
127.0.0.1:6379> LRANGE jobs:processing:w1 0 -1
1) "email:101"
# ... the worker sends the e-mail, then acknowledges:
127.0.0.1:6379> LREM jobs:processing:w1 1 "email:101"
(integer) 1
127.0.0.1:6379> LRANGE jobs:processing:w1 0 -1
(empty array)
127.0.0.1:6379> LRANGE jobs 0 -1
1) "email:102"
2) "email:103"
```

This pattern gives at-least-once delivery: after a crash the same job may run twice, so job handlers must be idempotent. It has no built-in retry counter, delay, or visibility timeout. When you need those, or several independent consumer groups reading the same data, use a stream instead; the streams chapter and the queues chapter in the Patterns Part compare the options.

3.2.5 Pattern 3: a stack, and a message inbox

A stack is the same list used LIFO: **LPUSH** and **LPOP** on the same end. "Undo history" and "back button" data fit this. For a per-user inbox that is read often and capped, **LPUSH** to add, **LRANGE inbox:u42 0 19** for the first page, and **LTRIM** to bound it. Beyond a few hundred entries or when you need paging by time, prefer a sorted set whose score is a timestamp.

3.2.6 How lists are stored

Internally, a list is stored as a **quicklist**: a linked list whose nodes each hold a compact array called a **listpack** (the memory-efficient successor of the **ziplist**). Small lists in recent versions are kept as a single listpack with no linked-list overhead at all. You can check with **OBJECT ENCODING key**, which returns **listpack** or **quicklist** depending on the version and size. Redis 7.0 reports **quicklist** even for a tiny list, while newer versions can report **listpack** for small ones.

The design is a compromise. A plain linked list wastes memory on a pointer pair per element; a plain array makes pushes at the front expensive. A linked list of small arrays gives O(1) pushes at both ends with low overhead per element.

Directive	Default	Meaning
<code>list-max-listpack-size</code>	-2	Maximum size of one node. A negative value is a byte limit: -2 means 8 KB per node (-1 is 4 KB, up to -5 for 64 KB). A positive value limits the number of entries
<code>list-compress-depth</code>	0	Number of nodes at each end left uncompressed. 0 disables compression; 1 keeps the head and tail nodes plain and compresses the middle

List configuration directives.

Compression pays off only for very long lists whose middle is rarely touched (because the ends are what you read). For a typical queue it is better left off.

3.2.7 Big-list gotchas

- **Unbounded growth.** A queue whose consumers are slower than its producers grows until memory runs out. Cap lists with `LTRIM` where the content is disposable, and alert on `LLEN` for queues.
- **LRANGE 0 -1 on a long list.** It copies the whole list into one reply. Page with ranges such as `0 99`, then `100 199`.
- **One giant key.** A list of ten million elements lives in one key, on one shard, and deleting it can stall the server. `UNLINK` frees it in a background thread; `DEL` does it inline. In a cluster, consider splitting a queue across several keys (`queue:emails:{0}`, `queue:emails:{1}`) with a client-side choice.
- **Treating it as a database.** Lists cannot answer "which job has id 7" without a scan. Keep an index (a hash from job id to payload) if you need lookups.
- **Lost jobs on BLPOP.** Covered above: use `LMOVE`, or a stream with acknowledgements.

INTERVIEW ANGLE

How would you build a reliable task queue on Redis lists, and where does it fall short? Use `LMOVE` (or `BLMOVE`) to atomically move each job from the pending list to a per-worker processing list, delete it with `LREM` after success, and run a reaper that requeues entries stuck in processing too long. That gives at-least-once delivery, so handlers must be idempotent. It falls short on retries with backoff, delayed jobs, per-job visibility timeouts, dead-letter handling and multiple independent consumers. Streams with consumer groups or a sorted-set delay queue handle those better.

REAL WORLD

In ShopStream, the notification-service keeps `recent:{userId}` as a capped list of the last 20 product views for the "recently viewed" strip: `LPUSH`, `LREM` of a duplicate, `LTRIM 0 19`, all in one pipeline. Order events themselves go to a stream, not a list, because several services must read them independently and acknowledge them.

KEY POINTS

- Lists are dequeues: $O(1)$ push and pop at both ends, $O(N)$ access in the middle.
- `LPUSH` + `LTRIM` gives a capped "latest N" list; `LPUSH` + `BRPOP` gives a simple blocking queue.
- `LMOVE`/`BLMOVE` plus a processing list gives an at-least-once queue; streams are better when you need more.
- `LMPOP` and `BLMPOP` (7.0) pop several elements from the first non-empty of many lists.
- Storage is a quicklist of listpacks; tune `list-max-listpack-size` rarely, and never let a list grow without a bound.
- Page large reads and delete big lists with `UNLINK`.

3.3 Hashes

A hash maps field names to values inside a single key, like a small dictionary or a row of a table. Where a string holds one value, a hash holds a record: `product:42` can carry `name`, `price` and `stock` as separate fields, and you can read or change one field without touching the others.

Hashes solve the "object" problem. Storing a product as one JSON string forces you to download, parse, change and re-upload the entire document to adjust one number, and two clients doing that at once overwrite each other. With a hash, `HINCRBY product:42 stock -1` changes exactly one field, atomically, on the server.

3.3.1 The core commands

HSET *key field value [field value ...]* creates or updates fields and returns how many fields were newly created. (The old **HMSET** does the same job and only differs in returning **OK**; it is deprecated, use **HSET**.) **HGET** reads one field, **HMGET** reads several, **HGETALL** reads everything as a flat list of alternating field and value.

A product as a hash

TERMINAL

```
127.0.0.1:6379> HSET product:42 name "USB-C Cable" price 799 stock 120
(integer) 3
127.0.0.1:6379> HGET product:42 name
"USB-C Cable"
127.0.0.1:6379> HMGET product:42 name price color
1) "USB-C Cable"
2) "799"
3) (nil)
127.0.0.1:6379> HGETALL product:42
1) "name"
2) "USB-C Cable"
3) "price"
4) "799"
5) "stock"
6) "120"
127.0.0.1:6379> HINCRBY product:42 stock -1
(integer) 119
127.0.0.1:6379> HINCRBYFLOAT product:42 price 0.5
"799.5"
127.0.0.1:6379> HSETNX product:42 name "other"
(integer) 0
127.0.0.1:6379> HEXISTS product:42 color
(integer) 0
127.0.0.1:6379> HLEN product:42
(integer) 3
127.0.0.1:6379> HDEL product:42 price color
(integer) 1
127.0.0.1:6379> HKEYS product:42
1) "name"
2) "stock"
```

A few details worth remembering. Field values are strings (like everything in Redis), so **price 799** is stored as the text **799**; **HINCRBY** parses it as an integer when you ask. **HSETNX** sets a field only if it does not exist, a handy "initialize once". **HSTRLEN** gives the length of a field's value. **HRANDFIELD** (6.2) returns random fields, optionally with values. Prices in cents as integers keep arithmetic exact.

Command	What it does	Cost
HSET / HGET	Write or read one or more fields	O(1) per field
HMGET	Read several fields	O(N) in fields asked
HGETALL, HKEYS, HVALS	Read the whole hash	O(N) in all fields
HINCRBY, HINCRBYFLOAT	Atomic numeric update of a field	O(1)
HDEL	Delete fields; the key disappears when the last field goes	O(N) in fields given
HSCAN	Iterate fields in small batches with a cursor	O(1) per call

Hash commands and their cost.

WATCH OUT

HGETALL on a hash with hundreds of thousands of fields blocks the server. It builds one huge reply on the single command thread. For big hashes read the fields you need with **HMGET**, or walk the hash incrementally with **HSCAN** *key cursor [MATCH pattern] [COUNT n]*. The scan commands are covered in their own chapter.

3.3.2 Field expiration (Redis 7.4 and later)

A key can expire, but historically a field could not. If you wanted per-field lifetimes (the items of a shopping cart, the temporary flags of a session) you had to split data across many keys. Redis 7.4 added **hash field expiration**, and Redis 8.0 added commands that combine reading or writing with an expiry.

Command	Purpose
HEXPIRE key seconds [NX XX GT LT] FIELDS n f1 ...	Set a relative TTL (seconds) on fields. HPEXPIRE is the millisecond form; HEXPIREAT and HPEXPIREAT take an absolute Unix time
HTTL / HPTTL key FIELDS n f1 ...	Remaining TTL per field, in seconds / milliseconds. -1 means the field has no TTL, -2 means the field (or key) does not exist
HPERSIST key FIELDS n f1 ...	Remove the TTL from fields
HEXPIRETIME / HPEXPIRETIME	The absolute expiry time per field
HGETEX key [EX PX EXAT PXAT PERSIST] FIELDS n f1 ...	Read fields and set or clear their TTL in one step (8.0)
HSETEX key [FNX FXX] [EX PX EXAT PXAT KEEPTTL] FIELDS n f v ...	Write fields with a TTL; FNX only if none exist, FXX only if all exist (8.0)
HGETDEL key FIELDS n f1 ...	Read fields and delete them (8.0)

Hash field expiration commands.

All of these take the form **FIELDS numfields field [field ...]**: you state how many fields follow, then list them. Commands that set a TTL return one integer per field: **1** the TTL was set, **0** a condition (**NX**, **XX**, **GT**, **LT**) was not met, **-2** no such field, **2** the field was deleted because the TTL was zero or already in the past.

Field TTLs on a cart (Redis 7.4 or later)

TERMINAL

```
127.0.0.1:6379> HSET cart:u42 sku-1 2 sku-2 1 sku-3 5
(integer) 3
127.0.0.1:6379> HEXPIRE cart:u42 3600 FIELDS 2 sku-1 sku-9
1) (integer) 1
2) (integer) -2
127.0.0.1:6379> HTTL cart:u42 FIELDS 3 sku-1 sku-2 sku-9
1) (integer) 3600
2) (integer) -1
3) (integer) -2
127.0.0.1:6379> HGETEX cart:u42 EX 7200 FIELDS 1 sku-1
1) "2"
127.0.0.1:6379> HSETEX cart:u42 FXX EX 600 FIELDS 1 sku-2 3
(integer) 1
127.0.0.1:6379> HGETDEL cart:u42 FIELDS 1 sku-3
1) "5"
```

When the last field of a hash expires or is deleted, the key itself disappears. Expired fields are removed lazily when touched and by a background process, exactly as keys are (see the chapter on expiration); they are never returned by reads once their time has passed. Expiry of fields is also replicated and persisted like key expiry.

NOTE

Field expiry does not exist before Redis 7.4. Many managed services lag behind the open source release. If your code must run against an older server, keep the per-field time in the hash as a normal field (an expiry timestamp) and filter on read, or use one key per item.

3.3.3 A worked example: a session store

A web session is a bundle of small attributes (user ID, locale, CSRF token, last seen) that is read on every request and must disappear after inactivity. A hash with a key TTL fits exactly: one key per session, fields for attributes, the whole thing expiring together.

Session hash with a sliding TTL

TERMINAL

```

127.0.0.1:6379> HSET session:9f2c userId 42 locale en-IN csrf a81f
(integer) 3
127.0.0.1:6379> EXPIRE session:9f2c 1800
(integer) 1
127.0.0.1:6379> HGET session:9f2c userId
"42"
127.0.0.1:6379> HSET session:9f2c lastSeen 1760000100
(integer) 1
127.0.0.1:6379> EXPIRE session:9f2c 1800
(integer) 1
127.0.0.1:6379> TTL session:9f2c
(integer) 1800
127.0.0.1:6379> HGETALL session:9f2c
1) "userId"
2) "42"
3) "locale"
4) "en-IN"
5) "csrf"
6) "a81f"
7) "lastSeen"
8) "1760000100"

```

Each request reads the fields it needs and pushes the expiry forward with **EXPIRE** (a pipeline sends the two commands in one round trip). Only the changed field is written, so concurrent requests from the same browser do not overwrite each other's attributes. When the user is inactive for 30 minutes the key vanishes by itself. Spring Session uses a hash per session in exactly this way, as described in the Spring Part.

3.3.4 Objects: hash or JSON string?

Both work; they have different strengths.

Aspect	Hash	JSON in a string
Read one field	HGET, O(1), tiny reply	Fetch and parse whole value
Update one field	HSET or HINCRBY, atomic	Read, modify, write; races
Nested data, arrays	Not supported; flatten to field names	Natural
Field-level TTL	Yes (7.4+)	No
Memory, small objects	Very compact (listpack)	Compact string, plus serialization cost
Cross-language reading	Easy, plain strings	Easy

Storing an object as a hash or as one serialized string.

Rule of thumb: use a hash for flat records that you update by field (product stock, user profile, cart lines, session attributes). Use a string holding JSON for a read-mostly document that you always load whole (a cached API response). If you need nested documents with partial updates and indexing, the JSON data type of Redis 8 (covered later in this Part) is designed for that.

3.3.5 Memory: listpack to hashtable

A small hash is stored as a **listpack**, a single compact array of field and value bytes, with no pointers or per-entry allocations. Lookup in a listpack is a linear scan, which is fast for a handful of entries because they sit together in cache. When the hash exceeds **hash-max-listpack-entries** (default 512 in Redis 7.0 and later) or any field or value is longer than **hash-max-listpack-value** bytes (default 64), Redis converts it to a real **hashtable**, which has O(1) lookup but more memory per field. The conversion happens automatically and is one-way for a given hash (a hash does not shrink back when it loses fields in the usual case).

Seeing the encoding

TERMINAL

```
127.0.0.1:6379> HSET product:42 name "USB-C Cable" price 799
(integer) 2
127.0.0.1:6379> OBJECT ENCODING product:42
"listpack"
127.0.0.1:6379> HSET product:42 desc \
  "Braided nylon cable, 2 m, 100 W power delivery, data up to 480 Mbps"
(integer) 1
127.0.0.1:6379> OBJECT ENCODING product:42
"hashtable"
```

This is why Redis documentation recommends hashes for many small objects: a million users as a million `user:ID` hashes of five fields costs far less than five million separate string keys, because each Redis key carries fixed bookkeeping overhead. A hash that sits just above the thresholds loses that advantage, so keep fields short where you can, and measure with `MEMORY USAGE` key. Release 8.6 further reduces the memory of hashtable-encoded hashes, but the guidance stays the same: small flat hashes are the cheapest representation.

TIP

Do not put a million fields in one hash just to save memory. A single huge hash is one key on one shard, it can only expire as a whole key (unless you use field TTLs), and commands like `HGETALL` and `DEL` become latency spikes. Shard by a natural boundary (per user, per product) and keep each hash small to medium.

INTERVIEW ANGLE

Why store a user profile as a hash instead of JSON in a string? A hash lets you read and update individual fields with $O(1)$ commands and atomic numeric updates (`HINCRBY`) without a read-modify-write cycle, avoids lost updates between concurrent writers, and a small hash uses a compact listpack encoding. JSON in a string is better when the object is nested and always read whole. Add that since 7.4 individual fields can even have their own TTL.

KEY POINTS

- A hash is a record under one key: fields are read and written individually, `HINCRBY` is atomic.
- Use `HSET` (not `HMSET`); avoid `HGETALL` on big hashes and use `HSCAN` or `HMGET`.
- Redis 7.4 added field TTLs (`HEXPIRE`, `HTTL`, `HPERSIST`); 8.0 added `HGETEX`, `HSETEX` and `HGETDEL`.
- Small hashes are listpacks; beyond 512 entries or 64-byte values they convert to hashtables.
- Choose hash for flat, field-updated records; JSON string or the JSON type for nested documents.

3.4 Sets

A set is an unordered collection of unique strings. Adding the same member twice has no effect, checking membership is $O(1)$, and Redis can combine sets on the server with intersection, union and difference. Where a list answers "what is in order", a set answers "is X in this group, and what do these groups have in common".

Sets solve problems of uniqueness and relationships: the tags on a product, the users who liked a post, the IDs of items already shown to a visitor, the set of users currently online.

3.4.1 Adding, testing, removing

`SADD key member [member ...]` returns how many members were new. `SISMEMBER` tests one member and `SMISMEMBER` (6.2) tests several in one call. `SREM` removes, `SCARD` returns the size, `SMEMBERS` returns everything. Sets have no defined order, so never depend on the order of the replies shown.

Tags as sets

TERMINAL

```
127.0.0.1:6379> SADD tags:p1 red cotton sale
(integer) 3
127.0.0.1:6379> SADD tags:p1 red
(integer) 0
127.0.0.1:6379> SISMEMBER tags:p1 sale
(integer) 1
127.0.0.1:6379> SMISMEMBER tags:p1 sale blue
1) (integer) 1
2) (integer) 0
127.0.0.1:6379> SCARD tags:p1
(integer) 3
```

Tags as sets (continued)

TERMINAL

```
127.0.0.1:6379> SREM tags:p1 cotton
(integer) 1
127.0.0.1:6379> SCARD tags:p1
(integer) 2
```

WATCH OUT

SMEMBERS on a large set is like **HGETALL** on a large hash. It returns every member in a single reply and blocks the server while doing so. For big sets use **SSCAN** *key cursor [MATCH pattern] [COUNT n]*, which returns a few members per call.

3.4.2 Combining sets

The set algebra commands are the reason to choose a set. Take two products and their tags:

Intersection, union, difference

TERMINAL

```
127.0.0.1:6379> SADD tags:p1 red cotton sale
(integer) 3
127.0.0.1:6379> SADD tags:p2 sale new cotton
(integer) 3
127.0.0.1:6379> SINTER tags:p1 tags:p2
1) "cotton"
2) "sale"
127.0.0.1:6379> SUNION tags:p1 tags:p2
1) "new"
2) "cotton"
3) "sale"
4) "red"
127.0.0.1:6379> SDIFF tags:p1 tags:p2
1) "red"
127.0.0.1:6379> SINTERCARD 2 tags:p1 tags:p2
(integer) 2
127.0.0.1:6379> SINTERCARD 2 tags:p1 tags:p2 LIMIT 1
(integer) 1
127.0.0.1:6379> SINTERSTORE tags:common tags:p1 tags:p2
(integer) 2
127.0.0.1:6379> SCARD tags:common
(integer) 2
```

- **SINTER** returns members present in all the sets; **SUNION** in any; **SDIFF** returns members of the first set that are in none of the others (the order of the arguments matters).
- Each has a **...STORE** form (**SINTERSTORE**, **SUNIONSTORE**, **SDIFFSTORE**) that saves the result into a destination key instead of returning it. Use these for large results and then page through them.
- **SINTERCARD** *numkeys key [key ...] [LIMIT n]* (7.0) returns only the size of the intersection, and with **LIMIT** it stops counting early. It answers "how many common tags" without building the result.
- **SMOVE** *source destination member* moves a member atomically.

The cost of an intersection is bounded by the smallest set times the number of sets, which is why intersections of a small set with large ones are cheap. A union or difference is proportional to the total number of elements. In a Redis Cluster all the keys of one of these commands must hash to the same slot.

3.4.3 Random members

SRANDMEMBER *key [count]* returns random members without removing them. A negative count allows repeats. **SPOP** *key [count]* removes and returns random members. Together they cover lotteries, random sampling and "pick any available worker".

SRANDMEMBER and SPOP

TERMINAL

```
127.0.0.1:6379> SADD raffle:2026 u1 u2 u3 u4 u5
(integer) 5
127.0.0.1:6379> SCARD raffle:2026
(integer) 5
127.0.0.1:6379> SPOP raffle:2026
"u3"
127.0.0.1:6379> SCARD raffle:2026
(integer) 4
```

The replies of **SPOP** and **SRANDMEMBER** are random, so your session will show different members.

3.4.4 Storage

Sets have three encodings. A set that contains only integers, and not more than `set-max-intset-entries` (default 512), is an **intset**: a sorted array of integers that uses 2, 4 or 8 bytes per number depending on the largest value. In Redis 7.2 and later, a small set of non-integer members (up to `set-max-listpack-entries`, default 128, each at most `set-max-listpack-value` bytes, default 64) is a **listpack**. Everything else is a **hashtable** with the members as keys. Conversion is automatic, and `OBJECT ENCODING` shows the current one.

Set encodings (Redis 7.2 and later)

TERMINAL

```
127.0.0.1:6379> SADD nums 1 2 3
(integer) 3
127.0.0.1:6379> OBJECT ENCODING nums
"intset"
127.0.0.1:6379> SADD nums apple
(integer) 1
127.0.0.1:6379> OBJECT ENCODING nums
"listpack"
```

Add enough members (or one member longer than 64 bytes) and the encoding becomes **hashtable**. Before Redis 7.2 the second answer would already be **hashtable**.

3.4.5 Patterns for ShopStream

- **Product tags and filters:** keep `tag:cotton` as a set of product IDs, then `SINTER tag:cotton tag:sale` is the list of cotton products on sale. This inverted index is the basis of many small catalogues.
- **Unique counting for small groups:** `SADD viewers:product:42 u1` and `SCARD`. When the group has millions of members and an estimate is fine, use HyperLogLog instead (its chapter follows in this Part).
- **Recommendations:** "customers who bought this also bought" via `SINTER` of buyers of two products, or `SINTERCARD` to count the overlap.
- **Online users and presence:** `SADD online:users u42` on connect, `SREM` on disconnect. Remember a crash leaves stale members, so pair it with a heartbeat sorted set when accuracy matters.
- **Already seen:** `SADD seen:u42 article-9` returns 0 if the user has seen it before, giving a one-command "is this new" test.
- **Allow and deny lists:** a set of IDs that may (or may not) use a feature.

GOTCHA

Sets do not expire per member. A member lives until you `SREM` it or the whole key expires. If you need "members that disappear after N minutes" (online presence, recent activity), use a sorted set with the expiry time as the score and delete the old range with `ZREMRANGEBYSCORE`.

KEY POINTS

- A set holds unique, unordered strings with $O(1)$ add, remove and membership test.
- `SINTER`, `SUNION`, `SDIFF`, `SINTERCARD` and the `STORE` forms combine sets on the server; keep the keys in one slot on a cluster.
- Use `SSCAN`, not `SMEMBERS`, on big sets; `SPOP` and `SRANDMEMBER` give random picks.
- Encodings: intset for small integer sets, listpack for small mixed sets (7.2+), hashtable otherwise.
- Great for tags, uniqueness, relationships and seen-lists; use a sorted set when you need order or per-member expiry.

3.5 Sorted sets

A sorted set (often written "zset" after the command prefix **Z**) is a set of unique members in which every member carries a numeric **score**, and Redis keeps the members ordered by score at all times. You can add a member, change its score, ask for its rank, and fetch the top ten or the range between two scores, each in $O(\log N)$ time. No other structure in Redis, and few in any database, gives you a live ranking this cheaply.

That makes sorted sets the answer to a long list of problems: leaderboards, "most popular", time-ordered feeds, delayed job queues, sliding-window rate limiters, priority queues, and any index where you need "everything between A and B".

3.5.1 Adding members: ZADD and its flags

ZADD key [NX|XX] [GT|LT] [CH] [INCR] score member [score member ...]. A new member is inserted; an existing member has its score updated. By default the reply is the number of **new** members, which is why updating a score returns **0**.

Flag	Meaning
NX	Only add new members; never update existing ones
XX	Only update existing members; never add
GT	Update an existing member only if the new score is greater (still adds new members)
LT	Update an existing member only if the new score is less (still adds new members)
CH	Reply with the number of members changed (added plus updated) instead of added
INCR	Add the score to the existing one, like ZINCRBY; one pair only, reply is the new score

ZADD flags.

NX and **XX** exclude each other, and **GT**, **LT** and **NX** cannot be combined with each other. **GT** is the right flag for "keep the best score": a player submits many results and only a higher one counts.

```

Scores, flags and the best-score rule
127.0.0.1:6379> ZADD lb:bestsellers 100 asha 250 ravi 175 mei
(integer) 3
127.0.0.1:6379> ZADD lb:bestsellers 90 asha
(integer) 0
127.0.0.1:6379> ZADD lb:bestsellers GT CH 120 asha
(integer) 1
127.0.0.1:6379> ZADD lb:bestsellers GT CH 80 asha
(integer) 0
127.0.0.1:6379> ZADD lb:bestsellers NX 500 asha
(integer) 0
127.0.0.1:6379> ZADD lb:bestsellers INCR 25 mei
"200"
127.0.0.1:6379> ZSCORE lb:bestsellers mei
"200"
127.0.0.1:6379> ZMSCORE lb:bestsellers mei nobody
1) "200"
2) (nil)
127.0.0.1:6379> ZCARD lb:bestsellers
(integer) 3

```

Read the sequence carefully. **ZADD ... 90 asha** changed the score from 100 to 90 and still replied **0**, because no new member was created. The two **GT CH** calls show the flag at work: 120 is greater than 90 so it changed (**1**), 80 is not so nothing changed (**0**). Scores are 64-bit floating point numbers. Integers are exact up to 2^{53} ; beyond that, or with fractions, rounding applies. **+inf** and **-inf** are valid scores.

3.5.2 Reading ranges: ZRANGE

Since Redis 6.2 one command, **ZRANGE**, does all range queries; the older **ZREVRANGE**, **ZRANGEBYSCORE**, **ZREVRANGEBYSCORE**, **ZRANGEBYLEX** are deprecated spellings of it.

```
ZRANGE key start stop [BYSCORE | BYLEX] [REV] [LIMIT offset count] [WITHSCORES]
```

- Plain **ZRANGE key 0 9** selects by **rank** (position): index 0 is the lowest score. Negative indexes count from the end.
- **REV** reverses the order, so **ZRANGE key 0 9 REV** is the top ten. This is the leaderboard query.
- **BYScore** interprets start and stop as scores. A leading **(** makes a bound exclusive, and **-inf/+inf** mean unbounded. With **REV** the arguments swap: you give the high bound first.
- **BYLEX** selects by member name when all scores are equal, using **[** for inclusive and **(** for exclusive bounds, and **-/+** for the extremes. This gives you prefix search and autocomplete over a lexicographic index.
- **LIMIT offset count** pages the result, and is only allowed with **BYScore** or **BYLEX**.
- **WITHSCORES** adds each member's score to the reply.

Rank, score and lexicographic ranges

TERMINAL

```
127.0.0.1:6379> ZADD lb:bestsellers 120 asha 300 ravi 200 mei
(integer) 3
127.0.0.1:6379> ZRANGE lb:bestsellers 0 -1
1) "asha"
2) "mei"
3) "ravi"
127.0.0.1:6379> ZRANGE lb:bestsellers 0 -1 WITHSCORES
1) "asha"
2) "120"
3) "mei"
4) "200"
5) "ravi"
6) "300"
127.0.0.1:6379> ZRANGE lb:bestsellers 0 1 REV WITHSCORES
1) "ravi"
2) "300"
3) "mei"
4) "200"
127.0.0.1:6379> ZRANGE lb:bestsellers 150 +inf BYSCORE
1) "mei"
2) "ravi"
127.0.0.1:6379> ZRANGE lb:bestsellers (150 +inf BYSCORE LIMIT 0 1
1) "mei"
127.0.0.1:6379> ZRANGE lb:bestsellers +inf -inf BYSCORE REV LIMIT 0 2 WITHSCORES
1) "ravi"
2) "300"
3) "mei"
4) "200"
127.0.0.1:6379> ZCOUNT lb:bestsellers 100 250
(integer) 2
127.0.0.1:6379> ZADD names 0 alice 0 bob 0 carol 0 dave
(integer) 4
127.0.0.1:6379> ZRANGE names [b (d BYLEX
1) "bob"
2) "carol"
```

3.5.3 Rank, increments and popping

ZRANK key member is the 0-based position from the lowest score and **ZREVRANK** from the highest; both are $O(\log N)$. Since 7.2 either accepts **WITHSCORE** to also return the score in one call. A missing member gives **(nil)**. **ZINCRBY key amount member** is the atomic counter of sorted sets: it creates the member if needed and returns the new score.

ZPOPMIN and **ZPOPMAX** remove and return the lowest or highest members (with an optional count), and their blocking forms **BZPOPMIN** and **BZPOPMAX** wait for data, which makes a sorted set a priority queue. **ZMPOP** and **BZMPOP** (7.0) do the same across several keys. **ZREM** removes by member, and **ZREMRANGEBYSCORE**, **ZREMRANGEBYRANK** remove whole ranges.

Rank, increment, pop

TERMINAL

```
127.0.0.1:6379> ZADD lb:bestsellers 120 asha 300 ravi 200 mei
(integer) 3
127.0.0.1:6379> ZRANK lb:bestsellers mei
(integer) 1
127.0.0.1:6379> ZREVRANK lb:bestsellers mei
(integer) 1
127.0.0.1:6379> ZINCRBY lb:bestsellers 10 asha
"130"
127.0.0.1:6379> ZPOPMIN lb:bestsellers
1) "asha"
2) "130"
127.0.0.1:6379> ZPOPMAX lb:bestsellers
1) "ravi"
2) "300"
127.0.0.1:6379> ZRANK lb:bestsellers nobody
(nil)
```

ZRANK with WITHSCORE (Redis 7.2 or later)

TERMINAL

```
127.0.0.1:6379> ZRANK lb:bestsellers mei WITHSCORE
1) (integer) 0
2) "200"
```

3.5.4 Ties and ordering

When two members have the same score, Redis orders them lexicographically by member (byte comparison). That is deterministic, which is what you want. It is also why the **BYLEX** index works: give every member score 0 and the order is purely alphabetical.

For a leaderboard where ties should be broken by who reached the score first, encode the time into the score itself. One way: keep the real score as a whole number and add a small fraction that shrinks as time passes, for example $\text{score} + (1 - \text{timestampMillis} / 10^{14})$. An earlier timestamp gives a larger fraction, so the earlier player ranks higher among equals. Check that the combined number stays exactly representable as a double (about 15 to 16 significant digits) before adopting any such scheme, or keep the tie-break data in a separate hash.

3.5.5 Set algebra on sorted sets

Sorted sets can be combined like sets, with a rule for merging the scores of members that appear in several inputs.

- **ZUNIONSTORE** *dest numkeys key [key ...] [WEIGHTS w ...] [AGGREGATE SUM|MIN|MAX] and **ZINTERSTORE** write the result into *dest*.*
- **ZUNION**, **ZINTER**, **ZDIFF** (6.2) return the result directly, and **ZINTERCARD** (7.0) returns only the size of the intersection.
- **ZRANGESTORE** *dest src min max [BYScore|BYLEX] [REV] [LIMIT ...]* (6.2) saves a slice of a sorted set into another key.

Merging daily leaderboards into a weekly one

TERMINAL

```
127.0.0.1:6379> ZADD lb:day:mon 10 asha 20 ravi
(integer) 2
127.0.0.1:6379> ZADD lb:day:tue 15 asha 5 mei
(integer) 2
127.0.0.1:6379> ZUNIONSTORE lb:week 2 lb:day:mon lb:day:tue
(integer) 3
127.0.0.1:6379> ZRANGE lb:week 0 -1 WITHSCORES
1) "mei"
2) "5"
3) "ravi"
4) "20"
5) "asha"
6) "25"
127.0.0.1:6379> ZINTERSTORE lb:both 2 lb:day:mon lb:day:tue WEIGHTS 1 2 AGGREGATE MAX
(integer) 1
127.0.0.1:6379> ZRANGE lb:both 0 -1 WITHSCORES
1) "asha"
2) "30"
127.0.0.1:6379> ZRANGESTORE lb:top1 lb:week 0 0 REV
(integer) 1
127.0.0.1:6379> ZRANGE lb:top1 0 -1
1) "asha"
```


The default aggregate is **SUM**, so **asha** has 25 in the weekly set. In the intersection **WEIGHTS 1 2** multiplies the second set's scores by 2 and **MAX** keeps the larger value. These commands cost roughly the sum of the input sizes times log, so they are for periodic jobs, not for every request.

3.5.6 Pattern: a leaderboard

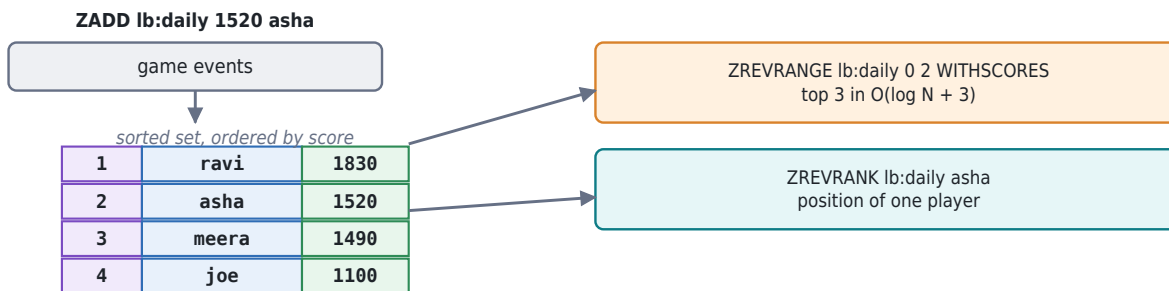


Figure 3.1. The sorted set keeps members ordered by score; rank and range queries touch only the part you ask for.

A game or a "bestsellers today" page needs three operations: record a result, show the top N, and show where one entry stands. They map to one command each.

REDIS

```
# a purchase of sku-42 happened: add one to its sales count
ZINCRBY lb:bestsellers:daily 1 sku-42
# top 10 products with their counts
ZRANGE lb:bestsellers:daily 0 9 REV WITHSCORES
# position of one product (0 = best)
ZREVRANK lb:bestsellers:daily sku-42
# a window "around me": 2 above and 2 below rank 14
ZRANGE lb:bestsellers:daily 12 16 REV WITHSCORES
# end of day: keep a copy, start fresh
RENAME lb:bestsellers:daily lb:bestsellers:2026-10-09
```

All of these are $O(\log N)$ plus the size of the result. Hundreds of thousands of members are not a problem; tens of millions in one key are, because one key lives on one node and a huge **ZUNIONSTORE** will stall it. The later chapter on leaderboards in the Patterns Part covers sharding and pagination for that scale.

3.5.7 Pattern: a time-ordered index

Use a Unix timestamp as the score and an ID as the member, and you have an index of "things by time" that supports range queries, which a list cannot do. **ZADD events:u42 1760000000 evt-9** adds, **ZRANGE events:u42 (1759990000 +inf BYSCORE LIMIT 0 50** reads everything newer than a moment, and **ZREMRANGEBYSCORE events:u42 -inf 1759000000** trims the old entries. The payloads live elsewhere, in hashes or strings keyed by the member ID; the sorted set is the ordered index.

3.5.8 Pattern: a sliding-window rate limiter

Keep one sorted set per client. Each request is a member (made unique with a request ID) whose score is the timestamp in milliseconds. To check the limit, delete members older than the window, count what is left, and add the new request if there is room. Doing all this in one transaction or script keeps it atomic.

REDIS

```
# limit: 100 requests per 60 seconds, now = 1760000100000 ms
ZREMRANGEBYSCORE rl:1.2.3.4:/login 0 1760000040000
ZCARD rl:1.2.3.4:/login
# if the count is below 100:
ZADD rl:1.2.3.4:/login 1760000100000 req-7f3a
PEXPIRE rl:1.2.3.4:/login 60000
```

This "sliding window log" is exact but stores one member per request, so memory grows with the limit. The rate-limiting chapter in the Patterns Part compares it with cheaper counters and token buckets and gives the Lua version that makes the steps atomic.

3.5.9 Pattern: a delay queue

A job that must run at a future time goes into a sorted set with its due time as the score. Workers poll for jobs whose score is at or below the current time.

REDIS

```
# schedule: send reminder in 10 minutes (due = now + 600000 ms)
ZADD delayed:jobs 1760000700000 job:remind:1001
# worker: fetch one due job
ZRANGE delayed:jobs -inf 1760000100000 BYSCORE LIMIT 0 1
# claim it; only the worker whose ZREM returns 1 owns the job
ZREM delayed:jobs job:remind:1001
```

The claim step matters: two workers may read the same member, but **ZREM** returns **1** for exactly one of them. The chapter on delayed and scheduled tasks turns this into a robust loop with a Lua script and a retry path.

3.5.10 How sorted sets are stored

A small sorted set (up to **zset-max-listpack-entries**, default 128, with members no longer than **zset-max-listpack-value**, default 64 bytes) is a **listpack** of member and score pairs kept in order. Larger ones use a **skiplist** together with a **hashtable**. The skiplist is a layered linked list that gives $O(\log N)$ insertion, deletion and range access with simpler code than a balanced tree; the hashtable maps member to score so **ZSCORE** is $O(1)$. Each member is stored once and referenced from both structures. The cost is a pointer-rich entry per member on top of the member itself, so a large sorted set is one of the heavier structures (measure with **MEMORY USAGE**); Redis 8.6 brought a memory reduction for skiplist-encoded sorted sets.

WATCH OUT

A sorted set key grows forever unless you trim it. The time-ordered index, the log-style rate limiter and the leaderboard history all need an explicit **ZREMRANGEBYSCORE** or **ZREMRANGEBYRANK**, or a TTL on the whole key, as part of the same code path that adds members. Otherwise the key becomes a slow memory leak and, eventually, a very big key.

INTERVIEW ANGLE

How would you build a real-time leaderboard for ten million players, and what are the limits? One sorted set with the player ID as the member and the score as the score. **ZINCRBY** or **ZADD GT** to record, **ZRANGE key 0 99 REV WITHSCORES** for the top 100, **ZREVRANK** for a player's position, and a window with **ZRANGE ... REV** around that rank. Every operation is $O(\log N)$. At ten million members the set holds a few GB and sits on one node, so a hot key concentrates load; split by region or season, or keep the top N only. Mention tie-breaking and that rank queries stay cheap while bulk **ZUNIONSTORE** does not.

KEY POINTS

- A sorted set is unique members ordered by a floating-point score; ties break lexicographically by member.
- Updating a score replies **0** for **ZADD** unless you use **CH**; use **GT** for "best score wins" and **ZINCRBY** for counters.
- **ZRANGE** with **REV**, **BYSCORE**, **BYLEX** and **LIMIT** is the single read command; **ZRANK**, **ZREVRANK**, **ZPOPMIN** are $O(\log N)$.
- Typical uses: leaderboards, time-ordered indexes, sliding-window limiters, delay queues, priority queues.
- Storage is a listpack when small, otherwise a skiplist plus a hashtable; trim every sorted set that can grow.

3.6 Choosing the right structure

The most useful skill when designing with Redis is picking the structure whose operations match your questions. Start from what you need to do, not from the data. Ask: do I read the whole value or part of it? Do I need order, uniqueness, ranges, counting, expiry per item? How big can it grow, and who writes concurrently?

3.6.1 From need to structure

I need to ...	Use	Key commands	Typical cost
Cache a value or a serialized object under a key	String	SET EX, GET	O(1)
Count atomically (views, stock, IDs)	String	INCR, DECR, INCRBY	O(1)
Lock or "do this once" marker	String	SET NX PX, DELEX	O(1)
Store a record and update single fields	Hash	HSET, HGET, HINCRBY	O(1) per field
Give fields their own lifetime	Hash (7.4+)	HEXPIRE, HGETEX	O(N) fields given
Queue jobs, newest-N list, stack	List	LPUSH, BLMOVE, LTRIM	O(1) at ends
Test membership, keep uniques, tag/relate	Set	SADD, SISMEMBER, SINTER	O(1) per member
Rank by score, top-N, range by score or time	Sorted set	ZADD, ZRANGE, ZRANK	O(log N)
Delay queue, sliding window, priority queue	Sorted set	ZADD, ZRANGE BYSCORE, ZPOPMIN	O(log N)
Durable log with consumer groups	Stream	XADD, XREADGROUP, XACK	O(1) append
Count unique items approximately, in tiny memory	HyperLogLog	PFADD, PFCOUNT	O(1)
Yes/no flag per integer ID, huge ranges	Bitmap	SETBIT, BITCOUNT	O(1) / O(N)
Nearby places	Geo (sorted set)	GEOADD, GEOSEARCH	O(log N + M)
Nested documents with partial updates, indexing	JSON, Query Engine	JSON.SET, FT.SEARCH	varies

Decision table: need to structure to key commands to cost.

Streams, HyperLogLog, bitmaps, geo, JSON and the search structures are covered in the next chapters of this Part.

3.6.2 Worked examples

Shopping cart

Per-user carts with line quantities: a hash `cart:u42` with the SKU as field and the quantity as value. `HINCRBY cart:u42 sku-1 1` adds one, `HDEL` removes a line, `HGETALL` renders it (carts are small). Set a key TTL (`EXPIRE cart:u42 604800`) so abandoned carts disappear. A list would make "change the quantity of sku-1" an O(N) search; a JSON string would force a read-modify-write.

Online users

A set `online` is simplest, but a crash leaves ghosts. A sorted set `online:zset` with the last-seen timestamp as the score fixes that: `ZADD online:zset <now> u42` on every heartbeat, `ZREMRANGEBYSCORE online:zset -inf <now - 60000>` to evict the silent, `ZCARD` to count.

"Customers also viewed"

For each product keep a sorted set `also:product:42` where the member is another product and the score the number of co-views: `ZINCRBY`. The top five is `ZRANGE also:product:42 0 4 REV`. A set would only say that two products were viewed together, not how often.

Unique visitors

A set gives exact counts and costs memory in proportion to the number of visitors. When millions of visitors per day only need an approximate number, a HyperLogLog is about 12 KB regardless of size. Decide whether you need the members back (set) or only the count (HyperLogLog).

3.6.3 Mistakes to avoid

- Using a **list** as a database table and searching it with **LPOS**. Use a hash or sorted set.
- Packing a whole object into one **string** and then updating one field from many clients.
- Using a **set** where you need order or expiry per member; use a sorted set.
- Using a **sorted set** where a list is enough (a plain FIFO queue): you pay $O(\log N)$ and extra memory for ordering you do not need.
- One enormous key (a hash with ten million fields, a list that never drains). Split by a natural boundary.
- Forgetting that every key and structure needs a lifetime plan: a TTL, a trim, or a scheduled cleanup.

KEY POINTS

- Choose by the question you must answer: lookup by key, order, uniqueness, range, counting, or hand-off.
- Strings for values and counters, hashes for records, lists for queues at the ends, sets for membership, sorted sets for anything ranked or time-ordered.
- Prefer many medium keys over a few giant ones; give each key a TTL or a trim rule.
- If two structures can do the job, pick the one with the cheaper operations for your hottest query.

3.7 Streams

ShopStream emits an event every time something happens to an order: created, paid, packed, shipped. Several services care about those events. The notification service sends e-mails, the inventory service adjusts stock, the analytics job counts revenue. You need a place to put the events so that each service can read them at its own pace, none of them loses an event when it restarts, and a slow reader does not hold up a fast one.

A list can act as a queue, but one consumer pops an item and it is gone. Pub/Sub can fan out to many readers, but it keeps nothing: a subscriber that is down misses the message forever (the next chapter shows this). What you want is a **log**: an append-only sequence of entries that stays in Redis, that readers walk through at their own position, and that can be shared out among a team of workers. That is the Redis **stream**, introduced in Redis 5.0 and extended in nearly every release since.

3.7.1 The core idea: an append-only log with IDs

A stream is a key whose value is an ordered sequence of **entries**. Each entry has a unique **ID** and a set of field-value pairs, much like a small hash. You only ever append to the end (and trim from the front); entries are not updated in place.

```
stream:orders
  ID          fields
  1760000000000-0  orderId=1001  userId=7  total=59.90
  1760000000000-1  orderId=1002  userId=9  total=12.00
  1760000000500-0  orderId=1003  userId=7  total=5.25    ← newest, appended at the tail
```

The ID has two parts, `<milliseconds>-<sequence>`. When you let Redis generate it (by passing `*`), the first part is the server time in milliseconds and the second part is a counter that distinguishes entries created in the same millisecond. IDs always increase, so the stream is automatically ordered by time of insertion, and you can ask for "everything since this ID" cheaply. Internally a stream is a radix tree of listpacks (compact blocks of entries), which gives $O(1)$ append and fast lookup by ID.

Adding entries with XADD

```
Append and measure TERMINAL
127.0.0.1:6379> XADD stream:orders * orderId 1001 userId 7 total 59.90
"1760000000000-0"
127.0.0.1:6379> XADD stream:orders * orderId 1002 userId 9 total 12.00
"1760000000000-1"
127.0.0.1:6379> XLEN stream:orders
(integer) 2
```

`XADD key id field value [field value ...]` returns the ID it assigned. You may supply the ID yourself, but it must be greater than the last one or you get an error (`ERR The ID specified in XADD is equal or smaller than the`

`target stream top item`). A partial ID such as `5-*` fixes the millisecond part and lets Redis choose the sequence. Explicit IDs are useful when importing data or in tests; production code almost always passes `*`.

`XADD` is $O(1)$ when it only appends. Adding `NOMKSTREAM` prevents the command from creating the stream if the key does not exist.

Reading by range: `XRANGE` and `XREVRANGE`

`XRANGE key start end [COUNT n]` returns entries between two IDs, inclusive. The special IDs `-` and `+` mean "the smallest possible" and "the largest possible". A millisecond-only bound such as `1760000000000` matches every sequence number in that millisecond.

```

Range queries
127.0.0.1:6379> XRANGE stream:orders - + COUNT 2
1) 1) "1760000000000-0"
   2) 1) "orderId"
      2) "1001"
      3) "userId"
      4) "7"
      5) "total"
      6) "59.90"
2) 1) "1760000000000-1"
   2) 1) "orderId"
      2) "1002"
      3) "userId"
      4) "9"
      5) "total"
      6) "12.00"
127.0.0.1:6379> XRANGE stream:orders (1760000000000-1 + COUNT 100

```

A `(` in front of an ID makes the bound **exclusive**. That is how you page through a stream: remember the last ID you processed and ask for `(lastId +. XREVRANGE key end start` walks backwards, which is the easy way to fetch the latest `N` entries (`XREVRANGE stream:orders + - COUNT 10`).

3.7.2 Reading as a stream: `XREAD`

`XRANGE` is a query. To **follow** a stream, like `tail -f`, use `XREAD`, which takes one or more keys and, for each, the last ID you have seen. It returns entries with greater IDs.

```

Follow a stream, with blocking
127.0.0.1:6379> XREAD COUNT 2 STREAMS stream:orders 1760000000000-1
1) 1) "stream:orders"
   2) 1) 1) "17600000000500-0"
      2) 1) "orderId"
         2) "1003"
         3) "userId"
         4) "7"
         5) "total"
         6) "5.25"
127.0.0.1:6379> XREAD BLOCK 5000 STREAMS stream:orders $
(nil)

```

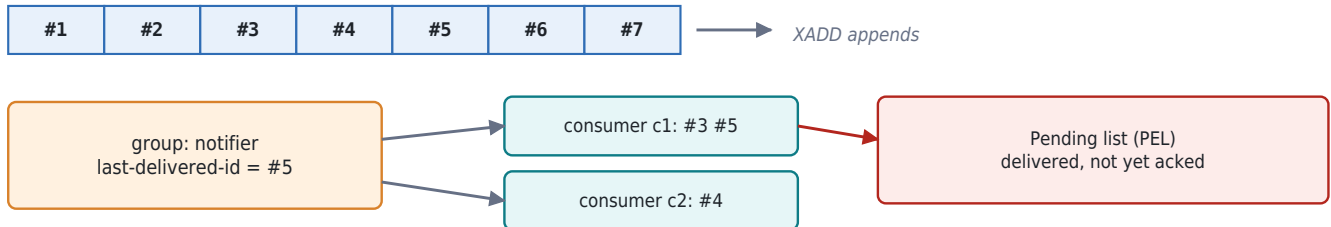
Two details matter. First, `BLOCK 5000` makes the command wait up to 5000 ms for new data instead of returning immediately with an empty result; `BLOCK 0` waits forever. A blocked client costs almost nothing, so many consumers can wait on one stream. Second, the special ID `$` means "only entries added after this command starts". Use `$` for the first call, then switch to the real last-seen ID for every call after that. A classic bug is to pass `$` on every loop iteration, which silently skips every entry that arrived between two calls.

Plain `XREAD` has no memory: every reader keeps its own cursor and every reader sees every entry. For fan-out of the full stream to independent readers (an audit logger, a cache warmer), that is exactly right. For sharing work among a team it is not, and that is what consumer groups are for.

3.7.3 Consumer groups: sharing the work

A **consumer group** is a named cursor on a stream, plus bookkeeping, shared by a set of **consumers**. The group remembers the ID of the last entry it handed out. Each new entry is delivered to exactly one consumer in the group, so adding consumers scales processing out. Different groups on the same stream are independent: the **notifier** group and the **inventory** group each see every order.

stream:orders



*XREADGROUP > delivers new entries; XACK removes from the PEL; XAUTOCLAIM takes over entries a dead consumer left
an entry stays in the stream after ack until you trim it (XTRIM / XDEL)*

Figure 3.2. A stream with two consumer groups. Notice that each group has its own last-delivered position and its own list of pending (delivered but not yet acknowledged) entries.

The delivery cycle

1. Create the group once with `XGROUP CREATE key group id [MKSTREAM]`. The ID says where the group starts: `0` means "from the beginning of the stream", `$` means "only new entries". `MKSTREAM` creates the stream if it does not exist yet.
2. Each worker calls `XREADGROUP GROUP group consumer ... STREAMS key >`. The special ID `>` means "entries never delivered to any consumer in this group". Consumers do not need to be registered in advance; the first call creates them.
3. Redis records every delivered entry in the group's **Pending Entries List (PEL)**, with the consumer name, the delivery time and a delivery counter.
4. When the worker has finished the job it calls `XACK key group id [id ...]`, which removes the entries from the PEL.

Create a group, read, inspect, acknowledge

TERMINAL

```
127.0.0.1:6379> XGROUP CREATE stream:orders notifier 0
OK
127.0.0.1:6379> XREADGROUP GROUP notifier worker-1 COUNT 2 STREAMS stream:orders >
1) 1) "stream:orders"
   2) 1) 1) "1760000000000-0"
        2) 1) "orderId"
           2) "1001"
           3) "userId"
           4) "7"
           5) "total"
           6) "59.90"
        2) 1) "1760000000000-1"
           2) 1) "orderId"
              2) "1002"
              3) "userId"
              4) "9"
              5) "total"
              6) "12.00"
127.0.0.1:6379> XPENDING stream:orders notifier
1) (integer) 2
2) "1760000000000-0"
3) "1760000000000-1"
4) 1) 1) "worker-1"
      2) "2"
127.0.0.1:6379> XACK stream:orders notifier 1760000000000-0
(integer) 1
```

The summary form of `XPENDING` shows how many entries are pending, the smallest and largest pending IDs, and how many each consumer holds. The extended form `XPENDING key group [IDLE min-idle-ms] start end count`

[**consumer**] lists individual entries: ID, consumer, milliseconds since the last delivery, and the number of times it has been delivered.

Pending entries in detail

TERMINAL

```
127.0.0.1:6379> XPENDING stream:orders notifier - + 10
1) 1) "1760000000000-1"
   2) "worker-1"
   3) (integer) 184233
   4) (integer) 1
```

Read that as: entry ...-1 was given to **worker-1** 184 seconds ago and has been delivered once. If worker-1 crashed, this entry will sit there forever unless someone takes it over.

3.7.4 At-least-once delivery and recovery

The PEL is what turns a stream into a reliable queue. A message is only forgotten when a worker explicitly acknowledges it. If the worker dies before acknowledging, the entry stays pending and can be handed to another worker. This gives **at-least-once** delivery: every entry is processed at least once, and sometimes twice (a worker may finish the work and crash before sending **XACK**). Your handler must therefore be idempotent, a topic the patterns part of the book returns to.

Taking over abandoned entries

XAUTOCLAIM **key group consumer min-idle-time start [COUNT n]** scans the PEL starting at **start**, transfers every entry that has been idle at least **min-idle-time** ms to **consumer**, and returns them with a cursor for the next call. It replaces the older two-step recipe of **XPENDING** followed by **XCLAIM** for most uses.

Claim entries idle for more than 60 seconds

TERMINAL

```
127.0.0.1:6379> XAUTOCLAIM stream:orders notifier worker-2 60000 0-0 COUNT 10
1) "0-0"
2) 1) 1) "1760000000000-1"
   2) 1) "orderId"
   2) "1002"
   3) "userId"
   4) "9"
   5) "total"
   6) "12.00"
3) (empty array)
```

The first element is the cursor to pass as **start** on the next call; **0-0** means the scan reached the end of the PEL. The last element lists IDs that were in the PEL but no longer exist in the stream (trimmed or deleted); Redis removes those dangling references for you. Claiming resets the idle timer and increments the delivery counter, so an entry that keeps crashing workers shows an ever-growing counter.

Claiming while reading (Redis 8.4)

Redis 8.4 added **CLAIM min-idle-time** to **XREADGROUP**. With it, a single call first returns pending entries that have been idle at least that long (longest idle first), then fills the rest with new entries when you read with **>**. Claimed entries carry two extra values: the milliseconds since last delivery and the delivery count. This lets one worker loop do both jobs:

REDIS

```
XREADGROUP GROUP notifier worker-2 COUNT 10 BLOCK 2000 CLAIM 60000 STREAMS stream:orders >
```

The reply layout for claimed entries is slightly different from a normal reply, so verify it against your client library before parsing it by hand. On servers older than 8.4 use **XAUTOCLAIM** in a periodic housekeeping task instead.

Re-reading your own pending entries

If a worker restarts it should first look at what it already owns. **XREADGROUP ... STREAMS stream:orders 0** (an ID instead of **>**) returns that consumer's own pending entries rather than new ones. A typical worker loop reads with **0** until the reply is empty, then switches to **>**.

Poison messages and dead letters

Some entries can never succeed: malformed payloads, a bug that throws every time. The delivery counter in **XPENDING** tells you how many attempts an entry has had. Redis has no built-in dead-letter queue, so the usual pattern is: when the counter passes a limit (say 5), the worker copies the entry to a second stream such as **stream:orders:dead** with **XADD**, then acknowledges the original. A human or a repair job looks at the dead stream later.

WATCH OUT

Never skip the acknowledgement path. A worker that reads with **XREADGROUP** and never calls **XACK** makes the PEL grow without bound, and the stream cannot be trimmed safely. Acknowledge after the work is done (not before), and alert on **XPENDING** counts and on the oldest pending idle time.

3.7.5 Keeping a stream from growing forever

A stream keeps its entries until you remove them. Reading, even acknowledging, does not delete anything: acknowledgement only affects the group's PEL. Left alone, a busy stream eats all your memory. There are three tools.

Tool	What it does	Typical use
XADD ... MAXLEN ~ N	Keep roughly the newest N entries	Capped event log
XADD ... MINID ~ id	Drop entries with IDs older than id	Time-based retention
XTRIM key MAXLEN or MINID	Trim on a schedule instead of on every add	Housekeeping job
XDEL key id	Delete specific entries (O(1) each)	Privacy / cleanup

Ways to bound a stream.

The **~** in **MAXLEN ~ 1000** asks for **approximate** trimming. Redis stores entries in macro nodes and only removes whole nodes, so the stream may keep slightly more than 1000 entries but trimming is far cheaper. Exact trimming (**=**, the default) can cut inside a node and costs more. For retention by time, compute the cutoff yourself, since an ID starts with milliseconds: to keep 7 days, pass **MINID ~ <nowMillis minus 604800000>**.

GOTCHA

Trimming does not know about your consumers. Plain **MAXLEN** and **MINID** remove entries even when a consumer group has not read them yet or still has them pending. A slow group then finds its entries gone, and **XAUTOCLAIM** reports them as deleted. Size the cap with the slowest consumer in mind, or use the group-aware options below.

Group-aware deletion (Redis 8.2)

Redis 8.2 added three ways to decide what happens to consumer group references when entries are deleted or trimmed. **XADD** and **XTRIM** accept an optional **KEEPREF | DELREF | ACKED** flag (default **KEEPREF**), and two new commands require one:

- **XDELEX key [KEEPREF | DELREF | ACKED] IDS numids id [id ...]** deletes entries.
- **XACKDEL key group [KEEPREF | DELREF | ACKED] IDS numids id [id ...]** acknowledges in the given group and deletes in one atomic step.

The three policies mean:

- **KEEPREF** (default): delete the entry but leave existing references in every group's PEL, like plain **XDEL**.
- **DELREF**: delete the entry and also remove its references from all PELs.
- **ACKED**: delete only entries that every consumer group has read and acknowledged. Entries still pending in some group stay.

Both commands reply with one integer per ID: **1** deleted, **-1** no such entry, **2** not deleted because references still exist (**ACKED** only).

Acknowledge and delete only when all groups are done

TERMINAL

```
127.0.0.1:6379> XACKDEL stream:orders notifier ACKED IDS 1 1760000000000-1
1) (integer) 2
```

With two groups, the **notifier** group acknowledges the entry, but the **inventory** group has not, so the reply is **2**: acknowledged, not deleted. When **inventory** later runs the same command, the entry is gone. If you use several groups on one stream, **XACKDEL ... ACKED** replaces the error-prone pair **XACK** plus **XDEL**.

3.7.6 Monitoring a stream with XINFO

XINFO STREAM key reports the length, first and last entries, the last generated ID and group count. **XINFO GROUPS** key shows each group's state and **XINFO CONSUMERS** key group shows each consumer.

```

Group health at a glance TERMINAL
127.0.0.1:6379> XINFO GROUPS stream:orders
1) 1) "name"
   2) "notifier"
   3) "consumers"
   4) (integer) 2
   5) "pending"
   6) (integer) 1
   7) "last-delivered-id"
   8) "1760000000000-1"
   9) "entries-read"
  10) (integer) 2
  11) "lag"
  12) (integer) 1

```

pending is the size of the PEL; **lag** is how many entries the group has not been delivered yet. Together they say "how far behind is this group" and "is anything stuck". **XINFO CONSUMERS** adds an **idle** time per consumer, useful for noticing dead workers.

3.7.7 Idempotent producers (Redis 8.6)

Producers also fail: a client sends **XADD**, the network drops before the reply arrives, and the client retries. Without protection the event appears twice. Redis 8.6 added producer-side deduplication to **XADD**:

```

REDIS
XADD stream:orders IDMP order-service ord-1001-created * orderId 1001 total 59.90
XADD stream:orders IDMPAUTO order-service * orderId 1001 total 59.90
XCFGSET stream:orders IDMP-DURATION 300 IDMP-MAXSIZE 1000

```

IDMP producer-id idempotent-id tells Redis that this producer has a unique key for this message. If the same pair was used recently, **XADD** adds nothing and returns the ID of the original entry. **IDMPAUTO producer-id** lets Redis derive the idempotent ID from the content. Both work only with an auto-generated entry ID (*). **XCFGSET** sets how long (**IDMP-DURATION**, seconds) and how many (**IDMP-MAXSIZE**) idempotent IDs the stream remembers per producer. The window is bounded, so this protects against retries, not against duplicates hours apart.

3.7.8 Streams, lists and Kafka

	List (LMOVE queue)	Stream + group	Kafka
Retention	Gone once popped	Kept until trimmed	Kept by time or size policy
Several independent readers	No	Yes, one group each	Yes
Work sharing in a group	Yes	Yes	Yes (partitions)
Acknowledge and retry	Manual with a second list	Built in (PEL, XAUTOCLAIM)	Offsets
Replay from the past	No	Yes, by ID	Yes
Scale limit	One node's RAM	One node's RAM per key	Disk, many brokers
Operational weight	Tiny	Tiny (already have Redis)	A cluster to run

Choosing a messaging structure.

Streams sit between a simple list and a full log platform. They are an excellent fit when the volume fits in memory, retention is hours or days, and you already run Redis. They are not a substitute for Kafka when you need terabytes of retained history, strict ordering across many partitions, or a large ecosystem of connectors. One more caveat: persistence and replication follow the usual Redis rules, so with default settings a crash can lose the most recent entries (see the persistence chapters).

REAL WORLD

ShopStream's order events `order-service` appends to `stream:orders` with `MAXLEN ~ 100000`. The `notification-service` runs three worker instances in group `notifier`; `inventory-service` has its own group. A scheduled task in each service calls `XAUTOCLAIM` every 30 seconds with a 60-second idle limit, and moves entries delivered more than five times to `stream:orders:dead`.

INTERVIEW ANGLE

How do Redis Streams give at-least-once delivery, and what happens when a consumer crashes? A group's PEL records every entry delivered to a consumer until it calls `XACK`. If the consumer crashes, its entries stay pending. Another consumer takes them over with `XAUTOCLAIM` (or `XCLAIM`, or `XREADGROUP ... CLAIM` on 8.4+) after a minimum idle time, and the delivery counter increases. Because a crash after the work but before the `XACK` causes reprocessing, handlers must be idempotent.

INTERVIEW ANGLE

What is the difference between a Redis stream and Pub/Sub? Pub/Sub is fire-and-forget: messages are pushed to whoever is connected at that moment and are not stored. A stream stores entries, supports replay by ID, remembers each group's position and tracks acknowledgements.

KEY POINTS

- A stream is an append-only log of entries with increasing IDs of the form `ms-seq`; append with `XADD`, read ranges with `XRANGE`, follow with `XREAD BLOCK`.
- A consumer group shares entries among workers; every delivered entry sits in the PEL until `XACK`, which gives at-least-once delivery.
- Recover from dead workers with `XAUTOCLAIM`; watch `XPENDING`, `XINFO GROUPS` lag and delivery counts, and send poison messages to a dead-letter stream yourself.
- Streams grow until you trim them: use `MAXLEN ~` or `MINID ~`, and on 8.2+ `XACKDEL ... ACKED` when several groups share a stream.
- Redis 8.6 adds `IDMP` and `IDMPAUTO` on `XADD` so producer retries do not create duplicates.

3.8 Pub/Sub and Keyspace Notifications

Sometimes you do not need a log. You need to tell whoever is listening right now that something happened: a cache entry changed, a chat message was typed, a configuration flag flipped. **Publish/subscribe** (Pub/Sub) is Redis's lightweight broadcast mechanism for exactly that. Senders publish a message to a named **channel**; every client currently subscribed to that channel receives a copy. The sender and receivers never know about each other.

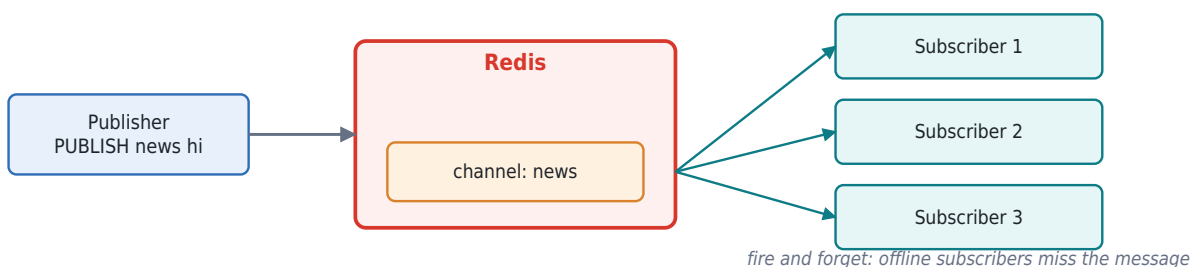


Figure 3.3. Publishers and subscribers meet only at a channel name. The server keeps no copy of the message after delivering it.

3.8.1 The basic commands

`SUBSCRIBE channel [channel ...]` puts a connection into subscriber mode. `PUBLISH channel message` sends a message and returns the number of subscribers that received it.

Terminal 1: subscriber

TERMINAL

```
127.0.0.1:6379> SUBSCRIBE shop:events
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "shop:events"
3) (integer) 1
1) "message"
2) "shop:events"
3) "price-changed:sku-42"
```

Terminal 2: publisher

TERMINAL

```
127.0.0.1:6379> PUBLISH shop:events "price-changed:sku-42"
(integer) 1
127.0.0.1:6379> PUBLISH shop:nobody-listens "hello"
(integer) 0
```

The subscriber sees a three-element push: the type (**message**), the channel and the payload. The publisher's reply is the receiver count; **0** is not an error, it means the message was dropped because nobody was listening.

Pattern subscriptions

PSUBSCRIBE pattern [pattern ...] subscribes to every channel matching a glob-style pattern. **PSUBSCRIBE shop:*** hears **shop:events**, **shop:alerts** and so on. Messages arrive as four elements, type **pmessage**, then the pattern that matched, the channel and the payload.

Pattern subscriber

TERMINAL

```
127.0.0.1:6379> PSUBSCRIBE shop:*
Reading messages... (press Ctrl-C to quit)
1) "psubscribe"
2) "shop:*"
3) (integer) 1
1) "pmessage"
2) "shop:*"
3) "shop:events"
4) "price-changed:sku-42"
```

Pattern matching costs CPU on every publish, because Redis checks the channel against every active pattern. A handful of patterns is fine; thousands is a design smell. Prefer exact channel names where you can.

UNSUBSCRIBE and **PUNSUBSCRIBE** end subscriptions. Introspect with **PUBSUB CHANNELS [pattern]** (active channels), **PUBSUB NUMSUB channel ...** (subscriber counts) and **PUBSUB NUMPAT** (active pattern count).

3.8.2 Delivery semantics: at most once

Pub/Sub provides **at-most-once** delivery, and the Redis documentation says so plainly: a message is delivered once if at all. There is no queue, no acknowledgement and no replay. Consider the failure cases:

- A subscriber that is disconnected, restarting or slow to reconnect misses everything published in the gap.
- A subscriber whose connection is too slow does not slow the publisher down; instead Redis closes the connection when that client's output buffer grows beyond the limit set by **client-output-buffer-limit pubsub**. The default is moderate (check the configuration reference for your version) and the effect is a silent loss of that subscriber.
- Messages are not written to the AOF or RDB and are not stored anywhere on the server. (They are propagated to replicas and, in a classic cluster, to all nodes.)

WATCH OUT

Do not use Pub/Sub for events you cannot lose Orders, payments and anything that must be processed exactly once belong in a stream (or a real message broker), not in Pub/Sub. Use Pub/Sub for hints and live updates where the receiver can recover by other means, for example by re-reading state from Redis when it reconnects.

A good mental rule: if the subscriber could fetch the truth afterwards with a normal read, Pub/Sub is a fine "something changed" nudge. If the message itself is the truth, store it.

3.8.3 Subscriber mode and connections

In the classic protocol (RESP2), a connection that has subscribed can only issue **SUBSCRIBE**, **PSUBSCRIBE**, **SSUBSCRIBE** and their un-variants, plus **PING**, **QUIT** and **RESET**. It cannot run **GET** or **SET**. So an application needs a **dedicated connection** for subscriptions, separate from the connection it uses for commands. Client libraries handle this for you (Lettuce uses a separate pub/sub connection; Spring's listener container manages one). With RESP3 (**HELLO 3**) a connection may run other commands while subscribed, because pushes are a distinct message type.

Pub/Sub ignores database numbers. Publishing while connected to database 10 is heard by a subscriber on database 1. To keep environments apart, put the environment in the channel name (**prod:shop:events**), or better, use separate Redis instances.

3.8.4 Sharded Pub/Sub for clusters

In a classic Pub/Sub on a Redis Cluster, every **PUBLISH** is forwarded to every node over the cluster bus, because any node might have a subscriber. That does not scale: more nodes means more bus traffic per message.

Redis 7.0 added **sharded Pub/Sub**: **SSUBSCRIBE**, **SUNSUBSCRIBE** and **SPUBLISH**. A shard channel is hashed to a slot with the same CRC16 algorithm as keys, and the message stays inside the shard (the primary and its replicas) that owns that slot. A client subscribes through the node that owns the slot, or one of its replicas.

REDIS

```
SSUBSCRIBE shop:cart:7
SPUBLISH shop:cart:7 "item-added"
```

Consequences: in a cluster, sharded channels scale by adding shards, and a client needs a connection per shard it subscribes through. Pattern subscription does not exist for sharded channels. On a single node, sharded and classic Pub/Sub behave alike, so only use the sharded form when you run (or plan to run) Cluster.

3.8.5 Keyspace notifications

Redis can publish a Pub/Sub message whenever a key changes. This is called **keyspace notification**, and it lets you react to events such as "this key expired" or "this hash was modified" without polling. It is off by default because it costs CPU. You turn it on with the **notify-keyspace-events** setting, which takes a string of flag characters.

Flag	Meaning
K	Keyspace channel: __keyspace@<db>__:<key> , payload is the event name
E	Keyevent channel: __keyevent@<db>__:<event> , payload is the key
g	Generic commands (DEL , EXPIRE , RENAME , ...)
\$	String commands
l, s, h, z, t	List, set, hash, sorted set and stream commands
x	Expired events (generated when Redis deletes an expired key)
e	Evicted events (a key removed by maxmemory eviction)
m, n	Key miss and new key events (not part of A)
A	Alias for all the main classes except m and n (and the newer o, c)

Common flags for notify-keyspace-events.

At least one of **K** or **E** must be present, or nothing is published. The newer **o** (overwritten) and **c** (type changed) flags exist in recent releases; they are not included in **A**.

Watch for expirations

TERMINAL

```
127.0.0.1:6379> CONFIG SET notify-keyspace-events Ex
OK
127.0.0.1:6379> PSUBSCRIBE __keyevent@0__:expired
Reading messages... (press Ctrl-C to quit)
1) "psubscribe"
2) "__keyevent@0__:expired"
3) (integer) 1
1) "pmessage"
2) "__keyevent@0__:expired"
3) "__keyevent@0__:expired"
4) "session:abc123"
```

Before this, another client ran `SET session:abc123 x EX 5`. Five seconds later (roughly) the subscriber learns that `session:abc123` expired. With `K` instead of `E` you would subscribe to `__keyspace@0__:session:abc123` and receive the event name `expired` as the payload.

GOTCHA

expired events are not on time Redis generates `expired` when it actually deletes the key, either lazily when a command touches it or when the background expiry cycle finds it. If a key is never accessed and the server has many keys with TTLs, the event can arrive well after the TTL passed. You also lose events if no subscriber is connected at that moment. Never build a precise scheduler or an exactly-once cleanup on `expired` events. The chapter on delayed tasks in the patterns part shows a sorted-set design that is reliable.

Keyspace notifications are also not stored: they have all the limits of Pub/Sub. They are good for cache invalidation hints and for debugging ("who is touching this key?"), and poor for anything where a missed event is a bug. Hash field expiration (Redis 7.4+) generates its own `hexpired` events as fields expire.

3.8.6 Practical limits and tuning

- **Fan-out cost.** One `PUBLISH` to 10,000 subscribers means 10,000 socket writes done by the single command thread. Large fan-outs can raise latency for everyone. Keep messages small, and shard channels across keys or use several servers for very large audiences.
- **Message size.** The payload is a binary-safe string; keep it to identifiers and small JSON, and let subscribers fetch details.
- **Slow subscribers.** Watch `CLIENT LIST` for subscriber connections with large output buffers (`omem`). The `pubsub` limit protects the server by disconnecting them.
- **Reconnects.** Client libraries re-subscribe automatically after a reconnect, but the messages in between are lost. Decide what the subscriber should do on reconnect (usually: refresh its state).
- **Naming.** Use hierarchical channel names (`shop:inventory:sku-42`) so patterns stay useful.

INTERVIEW ANGLE

Why might a Redis Pub/Sub subscriber miss messages, and what would you use instead? Pub/Sub is at-most-once: the server pushes each message only to connections that are subscribed at that instant and keeps no copy. A disconnect, a restart, or being disconnected for a full output buffer drops messages. When messages must not be lost, use Redis Streams with a consumer group (or a broker), which store entries and track acknowledgements.

KEY POINTS

- `PUBLISH` sends to all current subscribers of a channel and returns how many there were; `SUBSCRIBE` and `PSUBSCRIBE` (glob patterns) receive.
- Delivery is at-most-once with no storage or replay; use it for nudges and live updates, not for critical events.
- In RESP2 a subscribed connection cannot run normal commands, so use a dedicated connection.
- In a cluster prefer sharded Pub/Sub (`SPUBLISH`, `SSUBSCRIBE`) so traffic stays within a shard.
- Keyspace notifications (`notify-keyspace-events`) publish key events such as `expired`, but expiry events can be late and are never guaranteed.

3.9 Bitmaps and Bitfields

Suppose ShopStream wants to know which of its ten million registered users were active today. A set of user IDs would work, but at tens of bytes per member it costs hundreds of megabytes. A **bitmap** answers the same question in about one bit per user: ten million users fit in roughly 1.2 MB.

A bitmap in Redis is not a separate data type. It is an ordinary **string** that you treat as an array of bits. Bit number 0 is the highest bit of the first byte, bit 8 is the highest bit of the second byte, and so on. Because strings are binary-safe and can reach 512 MB, a bitmap can hold up to 2^{32} bits (about 4.3 billion).

3.9.1 The basic commands

Command	What it does	Cost
SETBIT key offset 0/1	Set one bit, return the old value	O(1)
GETBIT key offset	Read one bit	O(1)
BITCOUNT key [start end [BYTE BIT]]	Count bits set to 1	O(N)
BITPOS key bit [start [end [BYTE BIT]]]	Position of the first 0 or 1	O(N)
BITOP op dest key [key ...]	Bitwise operation across strings	O(N)
BITFIELD key ...	Read/write integers packed in the string	O(1) per op

Core bitmap commands.

```

Daily active users, one bit per user ID
127.0.0.1:6379> SETBIT dau:2026-10-10 7 1
(integer) 0
127.0.0.1:6379> SETBIT dau:2026-10-10 1001 1
(integer) 0
127.0.0.1:6379> GETBIT dau:2026-10-10 7
(integer) 1
127.0.0.1:6379> GETBIT dau:2026-10-10 8
(integer) 0
127.0.0.1:6379> BITCOUNT dau:2026-10-10
(integer) 2
127.0.0.1:6379> BITPOS dau:2026-10-10 1
(integer) 7
127.0.0.1:6379> STRLEN dau:2026-10-10
(integer) 126

```

SETBIT returns the previous bit value, so one call tells you both "mark user 7 active" and "was this the first time today". Reading a bit past the end of the string returns 0. Setting a high offset allocates the whole string up to that byte, filled with zeros: setting bit 1001 made the key 126 bytes long. That is why user IDs used as offsets should be dense, small integers. If your IDs are sparse (UUIDs, or 64-bit snowflake IDs), a bitmap wastes memory or fails; map users to a compact sequential number first, or use a set or a HyperLogLog.

3.9.2 Combining bitmaps with BITOP

BITOP AND|OR|XOR|NOT dest key [key ...] combines whole bitmaps and stores the result in **dest**. Shorter inputs are treated as padded with zero bytes. The reply is the length in bytes of the stored result.

```

Active on both days, and on either day
127.0.0.1:6379> SETBIT dau:2026-10-11 7 1
(integer) 0
127.0.0.1:6379> SETBIT dau:2026-10-11 9 1
(integer) 0
127.0.0.1:6379> BITOP AND dau:both dau:2026-10-10 dau:2026-10-11
(integer) 126
127.0.0.1:6379> BITCOUNT dau:both
(integer) 1
127.0.0.1:6379> BITOP OR dau:either dau:2026-10-10 dau:2026-10-11
(integer) 126
127.0.0.1:6379> BITCOUNT dau:either
(integer) 3

```

With a key per day you can answer retention questions with two commands: AND of today and yesterday gives returning users, **BITCOUNT** counts them. A weekly active count is an OR of seven keys.

New operators in Redis 8.2

Redis 8.2 added four **BITOP** operators for set-style logic. With **X** the first source key and **Y1, Y2, ...** the rest:

- **DIFF**: bits set in **X** but in none of the **Y** keys. "Active today but never before" when **X** is today.
- **DIFF1**: bits set in at least one **Y** but not in **X**.
- **ANDOR**: bits set in **X** and also in at least one **Y**.
- **ONE**: bits set in exactly one of the source keys.

REDIS

```
# new users today: active today, not active on any of the previous 6 days
BITOP DIFF dau:new dau:2026-10-10 dau:2026-10-09 dau:2026-10-08 dau:2026-10-07 \
dau:2026-10-06 dau:2026-10-05 dau:2026-10-04
BITCOUNT dau:new
```

On servers before 8.2 you build the same result from **NOT** and **AND** with temporary keys.

3.9.3 BITFIELD: small integers in a string

BITFIELD treats the string as a sequence of fixed-width integers and lets you read, write and increment them atomically. A type is written **u8** (unsigned 8-bit) or **i16** (signed 16-bit); widths go up to **u63** and **i64**. An offset like **#2** means "the third field of this width", while a plain number is a bit offset.

Packed counters with overflow control

TERMINAL

```
127.0.0.1:6379> BITFIELD stats:u1 SET u8 0 200 INCRBY u8 0 100 OVERFLOW SAT INCRBY u8 0 100 GET u8 0
1) (integer) 0
2) (integer) 44
3) (integer) 144
4) (integer) 144
```

Four sub-commands, four replies. **SET u8 0 200** returned the old value 0. The first **INCRBY u8 0 100** pushed 200 past the 8-bit maximum of 255 and, with the default **OVERFLOW WRAP**, wrapped around to 44. **OVERFLOW SAT** then changed the policy to saturate: $44 + 100 = 144$ fits, so the increment returned 144. The **OVERFLOW** option applies only to the **INCRBY** operations after it and can be **WRAP**, **SAT** or **FAIL** (the operation returns nil). Use bitfields for many tiny counters, for example per-user 4-bit feature flags, where a hash would cost far more memory.

WATCH OUT

Big offsets allocate big strings **SETBIT dau:today 4000000000 1** makes Redis allocate roughly 500 MB at once, and zero-filling it blocks the server for a noticeable time. Validate offsets in your application and keep user numbers dense. Also remember that **BITCOUNT** and **BITOP** on huge strings are $O(N)$ and run on the single command thread; limit key size, or split one bitmap per day into chunks (for example per million users) for very large user bases.

INTERVIEW ANGLE

How would you count daily active users and retention for 50 million users in Redis? Use one bitmap per day with the user's sequential ID as the offset: about 6 MB per day. **SETBIT** on each activity, **BITCOUNT** for the daily number, **BITOP AND** of two days for retention. If exact membership is not needed and IDs are sparse, a HyperLogLog gives a count in 12 KB with about 0.81% error.

KEY POINTS

- A bitmap is a string treated as bits: **SETBIT**, **GETBIT**, **BITCOUNT**, **BITPOS**, **BITOP**, all on one ordinary key.
- It is extremely compact for dense integer IDs (about 1 bit per user) and wasteful for sparse ones.
- **BITOP** combines days or segments; Redis 8.2 adds **DIFF**, **DIFF1**, **ANDOR** and **ONE**.
- **BITFIELD** packs small integers with explicit overflow behaviour (**WRAP**, **SAT**, **FAIL**).
- Large offsets allocate large strings and block while zero-filling; bound them.

3.10 HyperLogLog

Counting **distinct** things is expensive. How many different visitors came to ShopStream today? With a set you add each visitor ID and call **SCARD**, but memory grows with every new visitor: millions of members, hundreds of megabytes per day. Often you do not need the exact number, a figure within a percent is enough for a dashboard. That is the trade a **HyperLogLog** (HLL) makes.

3.10.1 The idea

A HyperLogLog is a probabilistic data structure. Each element is hashed to a long bit string. Looking at the hash values, the algorithm tracks how rare the patterns are: seeing a hash that starts with many zero bits is unlikely unless you have seen very many different elements, roughly 2^k distinct elements are needed before a run of k leading zeros shows up. Redis splits the hash space into 16,384 registers, keeps in each register the longest run seen, and combines them with a harmonic mean to estimate the count. The registers are all you store, and elements themselves are never stored.

The results:

- **Fixed memory.** At most about 12 KB per HLL, whether it counted a hundred or a billion elements. Redis stores small HLLs in a sparse form that is smaller still, and converts to the dense 12 KB form as it fills.
- **Standard error 0.81%.** The estimate is typically within a percent or so of the truth, never exact.
- **No membership queries.** You cannot ask "have I seen user 42?" or list the members. You can only add and count.

3.10.2 Commands

Command	What it does
PFADD key element [element ...]	Add elements; returns 1 if the estimate changed, else 0
PFCOUNT key [key ...]	Estimated distinct count (several keys: count of their union)
PFMERGE dest source [source ...]	Store the union of HLLs in dest

HyperLogLog commands.

(The **PF** prefix honours Philippe Flajolet, who co-invented the algorithm.) A HyperLogLog is stored as a string with a special header, so **TYPE** reports **string**.

```

Unique visitors per day
127.0.0.1:6379> PFADD uv:2026-10-10 u1 u2 u3
(integer) 1
127.0.0.1:6379> PFADD uv:2026-10-10 u3
(integer) 0
127.0.0.1:6379> PFCOUNT uv:2026-10-10
(integer) 3
127.0.0.1:6379> PFADD uv:2026-10-11 u3 u4
(integer) 1
127.0.0.1:6379> PFCOUNT uv:2026-10-10 uv:2026-10-11
(integer) 4
127.0.0.1:6379> PMERGE uv:week uv:2026-10-10 uv:2026-10-11
OK
127.0.0.1:6379> PFCOUNT uv:week
(integer) 4

```

Adding **u3** a second time returned 0 because the sketch did not change. Counting two keys at once returns the number of distinct elements across both, so **u3** is counted once. This union property is what makes HLLs so useful for rolling windows: keep one per day with a TTL, and **PFCOUNT** over the last seven keys, or **PFMERGE** them into a weekly sketch. Small counts like these are in practice exact; the error shows up only at large cardinalities.

Contrast with the bitmap: a bitmap gives exact counts and membership but needs dense integer IDs and about one bit per possible user; an HLL accepts any string, uses at most 12 KB, and gives an estimate only.

GOTCHA

An HLL is a string in disguise **PFCOUNT** caches the computed estimate inside the key, so even a read may modify the stored value. Counting several keys merges them on the fly, which is slower than counting one. Because an HLL is a string, **GET** on it returns binary junk, and an accidental **SET** or **APPEND** on the key corrupts it; later **PF** commands then fail with an error about a corrupted HLL object. Give HLL keys their own prefix and never write to them with string commands.

REAL WORLD

Unique visitors at ShopStream The gateway runs **PFADD uv:<date> <visitorId>** on each page view (one command, no matter how many visitors) and sets a 40-day TTL on the key. The dashboard calls **PFCOUNT** for today, and for the month passes the daily keys to one **PFCOUNT**. Billing or fraud decisions never use this number, because it is approximate.

KEY POINTS

- A HyperLogLog counts distinct elements in at most about 12 KB with a standard error of 0.81%.
- Use **PFADD**, **PFCOUNT** and **PFMERGE**; counting several keys gives the union without double counting.
- It cannot list or test members, and results are estimates; use sets or bitmaps when exactness matters.
- Keep one HLL per time bucket with a TTL to build rolling unique-visitor windows.

3.11 Geospatial Indexes

"Which ShopStream pickup stores are within 5 km of me?" is a question about places on the surface of the Earth. Redis answers it with **geospatial indexes**, and the most interesting thing about them is how little there is to them: a geo index is just a **sorted set**.

3.11.1 How geo works

GEOADD converts each longitude/latitude pair into a single 52-bit integer using a **geohash**-style interleaving of the bits of both coordinates. That integer is used as the score of a sorted set member. Points close to each other on the map tend to share a prefix and so have nearby scores, which means a radius query becomes a handful of sorted set range queries over a few neighbouring cells, followed by an exact distance filter. All the usual sorted set tools (**ZRANGE**, **ZREM**, **ZCARD**, **TYPE**) work on the key, and removing a location is a **ZREM**.

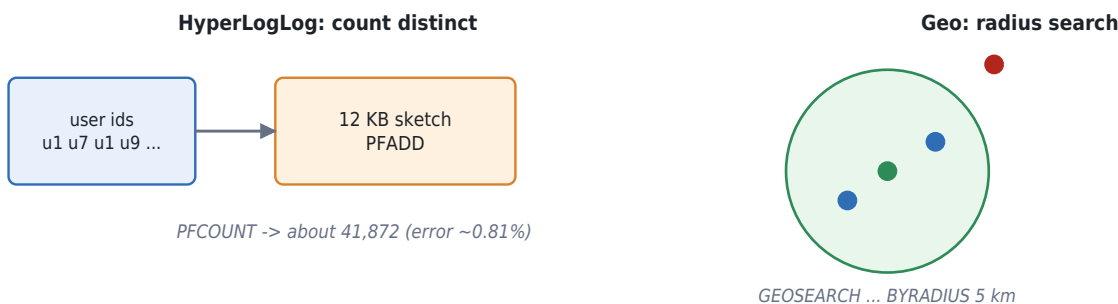


Figure 3.4. Left: HyperLogLog registers estimating a count. Right: points on a map encoded into one-dimensional geohash scores.

3.11.2 Commands

A store locator

TERMINAL

```
127.0.0.1:6379> GEOADD geo:stores 77.5946 12.9716 blr-central 77.6408 12.9784 blr-indiranagar \
72.8777 19.0760 mumbai-bkc
(integer) 3
127.0.0.1:6379> GEOPOS geo:stores blr-central
1) 1) "77.59459823369979858"
2) "12.97159902243186025"
127.0.0.1:6379> GEODIST geo:stores blr-central blr-indiranagar km
"5.0647"
127.0.0.1:6379> GEOSEARCH geo:stores FROMLONLAT 77.60 12.97 BYRADIUS 10 km ASC WITHDIST COUNT 5
1) 1) "blr-central"
2) "0.6119"
2) 1) "blr-indiranagar"
2) "4.5200"
127.0.0.1:6379> GEOSEARCH geo:stores FROMMEMBER blr-central BYBOX 20 20 km ASC
1) "blr-central"
2) "blr-indiranagar"
```

What to notice:

- **Order is longitude first, then latitude.** This is the opposite of how most people say coordinates. Swapping them is the most common geo bug.
- **GEOPOS** returns slightly different digits than you stored, because coordinates are quantised into 52 bits (error well under a metre).
- **GEODIST** key m1 m2 [m|km|ft|mi] gives the distance along the Earth's surface assuming a sphere.

- `GEOSEARCH key FROMMEMBER member | FROMLONLAT lon lat BYRADIUS r unit | BYBOX w h unit [ASC|DESC] [COUNT n [ANY]] [WITHCOORD] [WITHDIST] [WITHHASH]` is the search command (Redis 6.2+). It searches from a member or a point, inside a circle or a rectangle.
- `GEOSEARCHSTORE dest source ...` writes results into another key; add `STOREDIST` to store distances as scores.
- `GEOHASH` returns the standard 11-character geohash strings, which you can paste into other tools.

`GEORADIUS` and `GEORADIUSBYMEMBER` are the older query commands and are deprecated; use `GEOSEARCH`. `GEOADD` also accepts `NX`, `XX` and `CH` like `ZADD`.

3.11.3 Limits to know

- Valid longitudes are -180 to 180, and latitudes -85.05112878 to 85.05112878; points near the poles are rejected.
- Distance uses a spherical Earth model, with an error up to about 0.5%, fine for a store locator and not for surveying.
- A search must examine all points in the covered cells. `COUNT n` stops early only with `ANY` (any `n` matches, not the nearest); without `ANY`, Redis still has to look at all candidates, so a huge radius over a dense area is slow.
- A geo index is a single key, which lives on one shard. Millions of points are fine, but a single hot key can become a bottleneck. For global data, shard by region in the key name (`geo:stores:in`, `geo:stores:us`).
- You can attach other information to a member by keeping a hash such as `store:blr-central` next to the geo key.
- For polygons, geofencing with shapes and rich filtering, use the Query Engine's `GEO` and `GEOSHAPE` field types (see the chapter on secondary indexes and search).

TIP

Moving objects For vehicles or couriers, `GEOADD` repeatedly overwrites the same member with its new position, and a TTL is not possible per member. Delete stale entries with `ZREM`, or keep a second sorted set scored by last update time and `ZREMRANGEBYSCORE` the old ones periodically.

KEY POINTS

- A geo index is a sorted set whose score is a 52-bit geohash; `TYPE` shows `zset`.
- `GEOADD` takes longitude before latitude; `GEOSEARCH` (by radius or box, from a member or a point) replaces the deprecated `GEORADIUS` family.
- Distances assume a sphere; coordinates near the poles are not accepted.
- Everything lives in one key: shard by region if the data or traffic is huge.

3.12 Iterating Safely: SCAN and Friends

Sooner or later you want to ask "which keys exist?" Maybe to clean up `session:` keys, or find everything that belongs to one user. The obvious command is `KEYS pattern`. It is also an outage waiting to happen.

3.12.1 Why not KEYS

`KEYS product:*` walks the entire keyspace in one go. Redis runs commands one at a time on a single thread, so while it scans ten million keys nothing else gets served: every other client waits, and latency spikes into seconds. `KEYS` is fine on your laptop and in a tiny test database; in production it is an anti-pattern. The same applies to commands that return a whole big collection at once: `SMEMBERS`, `HGETALL` and `LRange 0 -1` on a million-element value.

3.12.2 SCAN: a cursor over the keyspace

`SCAN cursor [MATCH pattern] [COUNT hint] [TYPE type]` returns a small batch of keys plus a new cursor. You start with cursor `0` and call again with the cursor you got back, until the server returns cursor `0`.

Iterating with SCAN

TERMINAL

```
127.0.0.1:6379> SCAN 0 MATCH product:* COUNT 5
1) "10"
2) 1) "product:2"
   2) "product:11"
   3) "product:4"
   4) "product:7"
127.0.0.1:6379> SCAN 10 MATCH product:* COUNT 5
1) "7"
2) 1) "product:5"
   2) "product:1"
```

Each call does a small, bounded amount of work, so other clients get served between calls. The command line has a wrapper: `redis-cli --scan --pattern 'product:*` loops for you.

Properties you must understand

- **COUNT is a hint, not a limit.** It sets roughly how many hash-table slots to examine per call (default 10). A call can return fewer keys than `COUNT`, even none, and still not be finished. A cursor of `0` is the only end signal.
- **MATCH filters after the work is done.** Redis still visits keys and discards non-matching ones, so a rare pattern means many empty replies. The filter reduces network traffic, not server work.
- **TYPE type** (6.0+) filters by type, for example `SCAN 0 TYPE hash`.
- **Guarantee:** every element that was present for the whole iteration is returned at least once. Elements added or removed during the iteration may or may not appear.
- **Duplicates are possible.** The same key can come back more than once (for example while the hash table is being resized). Your code must tolerate it, such as by collecting into a set or by making the action idempotent.
- **The cursor is opaque.** It is not an index. Do not do arithmetic on it, and do not store it across a server restart.
- **There is no snapshot.** If you need an exactly consistent view, `SCAN` is the wrong tool.

3.12.3 SSCAN, HSCAN and ZSCAN

Big collections get the same treatment: `SSCAN key cursor [MATCH pattern] [COUNT n]`, `HSCAN key cursor [MATCH pattern] [COUNT n] [NOVALUES]` and `ZSCAN key cursor [MATCH pattern] [COUNT n]`. `HSCAN` returns field-value pairs (`NOVALUES`, from 7.4, returns only fields) and `ZSCAN` returns member-score pairs.

Scanning a hash

TERMINAL

```
127.0.0.1:6379> HSCAN product:h 0
1) "0"
2) 1) "a"
   2) "1"
   3) "b"
   4) "2"
   5) "c"
   6) "3"
```

Small collections (listpack or intset encoded, see the chapters on each type) are stored compactly in one block, so a scan returns everything in a single call with cursor `0`. The cursor behaviour only appears once the value has become a real hash table or skiplist.

3.12.4 Practical advice

```

KeyCleaner.java
package com.shopstream.cleanup;

import java.util.HashSet;
import java.util.Set;
import org.springframework.data.redis.core.Cursor;
import org.springframework.data.redis.core.ScanOptions;
import org.springframework.data.redis.core.StringRedisTemplate;

public class KeyCleaner {

    private final StringRedisTemplate redis;

    public KeyCleaner(StringRedisTemplate redis) {
        this.redis = redis;
    }

    /** Deletes all keys matching a pattern, in small batches. */
    public long deleteSessions(String pattern) {
        ScanOptions options = ScanOptions.scanOptions().match(pattern).count(500).build();
        long deleted = 0;
        Set<String> batch = new HashSet<>();
        try (Cursor<String> cursor = redis.scan(options)) {
            while (cursor.hasNext()) {
                batch.add(cursor.next());
                if (batch.size() >= 500) {
                    deleted += flush(batch);
                }
            }
        }
        return deleted + flush(batch);
    }

    private long flush(Set<String> batch) {
        if (batch.isEmpty()) {
            return 0;
        }
        Long n = redis.unlink(batch);
        batch.clear();
        return n == null ? 0 : n;
    }
}

```

The loop collects keys into small batches and removes them with **UNLINK** (an asynchronous **DEL** that frees memory in the background). In a Redis Cluster, **SCAN** covers only the node it is sent to, so a full cleanup must scan every primary (how to do that depends on your client library), and multi-key commands such as **UNLINK** with keys from different slots need the client to split them; deleting key by key is the simple, safe option there.

Better than scanning is to avoid needing it. If you must find "all keys of one user", maintain an index: add each key to a set **user:7:keys** when you create it. If you need to expire a family of keys at once, include a version in the key name (**catalog:v12:product:3**) and bump the version.

WATCH OUT

Do not scan in a hot path A **SCAN** over a large database is a background job, not something to call per request. Run it from a scheduled task, add a small pause between batches, and make it resumable. If a user-facing feature needs lookup by attribute, build a secondary index (sets, sorted sets or the Query Engine).

INTERVIEW ANGLE

Why is **KEYS** * dangerous in production, and what do you use instead? **KEYS** is $O(N)$ over all keys and blocks the single command thread until it completes, stalling every client. Use **SCAN** with a cursor, which does a bounded amount of work per call (**COUNT** as a hint), tolerates concurrent changes, may return duplicates, and ends when the cursor returns to 0. For collections, use **SSCAN**, **HSCAN** and **ZSCAN**.

KEY POINTS

- **KEYS**, **SMEMBERS** on huge sets, **HGETALL** on huge hashes block the server; avoid them on big data.
- **SCAN** iterates incrementally with a cursor; **COUNT** is a hint, **MATCH** filters after the work, and the end is cursor **0**.
- Expect duplicates and concurrent changes; make the per-key action idempotent.
- **SSCAN**, **HSCAN** and **ZSCAN** do the same for big collections; small ones come back in one call.
- Prefer an explicit index or key versioning over scanning for lookups.

3.13 JSON Documents

Everything so far stored flat values: strings, or hashes whose fields are strings. Real application data is often nested. A ShopStream product has a name and a price, but also a list of tags, a set of variants (size, colour, stock) and a nested **seller** object. You can serialize that to a JSON string and **SET** it, which is simple, but then changing one field means reading the whole document, parsing it in your application, modifying it and writing it back. Two services doing that at the same time overwrite each other.

The **JSON** data type stores a JSON document natively in Redis and lets you read and change parts of it atomically on the server. Before Redis 8 it was a separate module (RedisJSON, part of Redis Stack). In Redis Open Source 8.0 and later it is built in: no module to load.

3.13.1 Storing and reading

A JSON value lives under a normal key. You address parts of it with a **path**. A path starting with **\$** uses JSONPath syntax: **\$** is the root, **.name** or **["name"]** picks a field, **[0]** an array element, **[*]** all elements, **..** searches recursively, and **[?(@.price < 20)]** filters.

Create and read a product

TERMINAL

```
127.0.0.1:6379> JSON.SET product:42 $ '{"name":"Trail Shoe","price":59.9,"stock":120}'
OK
127.0.0.1:6379> JSON.SET product:42 $.tags '["running","outdoor"]'
OK
127.0.0.1:6379> JSON.SET product:42 $.seller '{"id":7,"city":"Pune"}'
OK
127.0.0.1:6379> JSON.GET product:42 $.name
["Trail Shoe"]
127.0.0.1:6379> JSON.GET product:42 $.seller.city $.price
{"$.seller.city":["Pune"],\"$.price\":[59.9]}
127.0.0.1:6379> JSON.GET product:42 $.seller
[{\"id\":7,\"city\":\"Pune\"}]
127.0.0.1:6379> TYPE product:42
ReJSON-RL
```

What to notice:

- With a **\$** path the reply is always an **array**, because a JSONPath can match many places. **\$.name** returns **[\"Trail Shoe\"]**, not **\"Trail Shoe\"**. If you ask for several paths, you get an object keyed by path. Older "legacy" paths that do not start with **\$** (such as **.name**) return a bare value and only act on the first match; use **\$** paths in new code.
- **redis-cli** shows the reply as a quoted string, so inner quotes are escaped. Your client library will give you the JSON text.
- **TYPE** reports **ReJSON-RL**, the internal name of the type.

3.13.2 Changing parts of a document

```

Atomic partial updates
127.0.0.1:6379> JSON.NUMINCRBY product:42 $.price -5
"[54.9]"
127.0.0.1:6379> JSON.NUMINCRBY product:42 $.stock -1
"[119]"
127.0.0.1:6379> JSON.ARRAPPEND product:42 $.tags "sale"
1) (integer) 3
127.0.0.1:6379> JSON.SET product:42 $.seller.city "Mumbai"
OK
127.0.0.1:6379> JSON.SET product:42 $.discount 0.1 NX
OK
127.0.0.1:6379> JSON.ARRLEN product:42 $.tags
1) (integer) 3
127.0.0.1:6379> JSON.STRLEN product:42 $.name
1) (integer) 10
127.0.0.1:6379> JSON.DEL product:42 $.discount
(integer) 1

```

TERMINAL

Each command is atomic, like every Redis command, and changes only the part you address. `JSON.NUMINCRBY` is the JSON counterpart of `HINCRBYFLOAT`: two services decrementing `stock` never lose an update. `JSON.SET` has `NX` (only if the path does not exist) and `XX` (only if it does). Other useful commands:

Command	Purpose
<code>JSON.SET key path value [NX XX]</code>	Write a value at a path (path <code>\$</code> replaces the whole document)
<code>JSON.GET key [path ...]</code>	Read one or more paths
<code>JSON.MGET key [key ...] path</code>	Read the same path from many documents
<code>JSON.DEL key [path]</code>	Delete a path, or the whole key
<code>JSON.NUMINCRBY</code> / <code>JSON.NUMMULTBY</code>	Arithmetic on numbers
<code>JSON.STRAPPEND</code> / <code>JSON.STRLEN</code>	Append to or measure strings
<code>JSON.ARRAPPEND</code> / <code>ARRINSERT</code> / <code>ARRPOP</code> / <code>ARRTRIM</code> / <code>ARRLEN</code> / <code>ARRINDEX</code>	Array operations
<code>JSON.OBJKEYS</code> / <code>JSON.OBJLEN</code>	Inspect objects
<code>JSON.TYPE key path</code>	The JSON type at a path
<code>JSON.TOGGLE key path</code>	Flip a boolean

JSON commands you will use most.

Filters let one command touch many elements. Given an `items` array of order lines, `JSON.NUMINCRBY order:9 '$.items[?(@.sku=="A1")].qty' 1` adds one to the quantity of every line with that SKU. Quote the path in the shell, because it contains characters the shell would interpret.

3.13.3 JSON from Java

With Spring Data Redis you can store JSON text through `StringRedisTemplate` and send the JSON commands as raw commands through the connection's `execute` method, but a client with native support is more convenient: Jedis, for example, has JSON methods such as `jsonSet` and `jsonGet`. The simplest approach that always works is to keep the document as a JSON string in a plain key (or hash field) and accept read-modify-write; use the native type when you need partial updates, server-side arithmetic, or indexing with the Query Engine (next chapter).

3.13.4 JSON, hash or string?

	String holding JSON	Hash	JSON type
Nesting	Yes, opaque to Redis	No (flat fields)	Yes
Update one field	Read, parse, write whole	HSET, atomic	JSON.SET path, atomic
Atomic numeric update	No	HINCRBY	JSON.NUMINCRBY
Memory	Smallest for read-only blobs	Very compact when small	Larger (tree of nodes)
Field TTL	No	Yes (7.4+)	No
Indexing by field	No	Yes, Query Engine	Yes, Query Engine
Arrays inside	In the blob	No	Yes

Three ways to store a product.

A good default: use a **hash** for flat records (sessions, carts, simple products) because it is smaller and has per-field TTL; use **JSON** when the data is genuinely nested or contains arrays you need to modify in place, or when you plan to search into nested fields; use a **string** when the whole value is always read and written together (a cached API response). Resist the temptation to store everything as JSON just because it is convenient, since the native JSON type uses noticeably more memory than a compact hash for the same fields.

GOTCHA

Wrong-type surprises `JSON.GET` on a key that holds a string returns a `WRONGTYPE` error, and `GET` on a JSON key does too. If your path matches nothing, many commands return an empty array or `nil` instead of an error. Check what your client returns before relying on it.

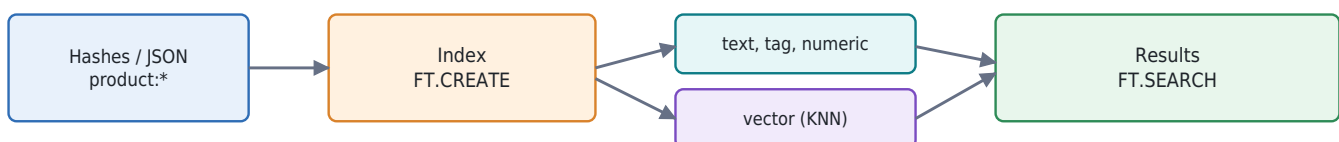
KEY POINTS

- The JSON type stores nested documents natively and is built into Redis 8 (no module needed).
- Use `$` paths (JSONPath): replies are arrays; `JSON.SET`, `JSON.GET`, `JSON.NUMINCRBY` and `JSON.ARRAPPEND` change or read only the addressed part, atomically.
- Choose hash for flat records, JSON for nested or array data and indexing, a plain string for blobs read whole.
- JSON documents can be indexed and searched by the Query Engine.

3.14 The Query Engine: Secondary Indexes and Search

Redis finds data by key. That is why it is fast, and also its biggest limitation. "Give me product 42" is easy. "Give me running shoes under 60 dollars from sellers in Pune, cheapest first" is not: you would have to maintain sets and sorted sets by hand for every field you might filter on (tags, price, city) and intersect them yourself. A **secondary index** removes that burden: you declare which fields of which keys to index, and Redis keeps the index up to date as the data changes.

In Redis 8 this capability, called the **Query Engine** (formerly RediSearch), is part of the server. It indexes hashes and JSON documents and supports full-text search, exact matching, numeric ranges, geo filters, vector similarity, and aggregations. The commands start with `FT..`



new keys with the indexed prefix are indexed automatically

Figure 3.5. An index sits beside the data. The application queries the index, which returns matching keys and fields.

3.14.1 Creating an index

An index is defined with **FT.CREATE**. You give it a name, say whether it covers hashes or JSON, choose a key prefix so it only watches the keys you care about, and list the fields (the schema) with their types.

Type	Use it for
TEXT	Full-text search: tokenization, stemming, ranking (names, descriptions)
TAG	Exact-match categories and IDs (colour, brand, status, city). Fast, no stemming
NUMERIC	Range filters and sorting (price, stock, timestamp)
GEO	Radius queries on a "longitude,latitude" point
VECTOR	Similarity search on embeddings (next chapter)
GEOSHAPE	Polygon and shape queries

Field types.

REDIS

```
# Index the product hashes (keys product:<id>)
FT.CREATE idx:product ON HASH PREFIX 1 product: SCHEMA \
  name TEXT WEIGHT 2.0 \
  description TEXT \
  brand TAG \
  city TAG \
  price NUMERIC SORTABLE \
  stock NUMERIC

# Index JSON documents; the identifier is a JSON path, AS gives it a name
FT.CREATE idx:productjson ON JSON PREFIX 1 pj: SCHEMA \
  $.name AS name TEXT \
  $.tags[*] AS tags TAG \
  $.price AS price NUMERIC SORTABLE \
  $.seller.city AS city TAG
```

Existing keys with the prefix are indexed in the background right after creation, and every later **HSET**, **JSON.SET**, **DEL** or expiry updates the index automatically. **SORTABLE** keeps the field in a column-like structure so sorting and aggregating on it is fast; add it only to fields you sort or group by, because it costs memory. Indexing a field you never query wastes memory too, so index only what you need. Run **FT.INFO idx:product** to see the schema, the number of documents and indexing progress, and **FT.DROPINDEX idx:product** to remove an index (the data stays unless you add **DD**).

3.14.2 Querying with FT.SEARCH

TERMINAL

```
Search products

127.0.0.1:6379> HSET product:1 name "Trail Running Shoe" brand nike city Pune price 59.9 stock 120
(integer) 5
127.0.0.1:6379> HSET product:2 name "Road Running Shoe" brand adidas city Mumbai price 79 stock 40
(integer) 5
127.0.0.1:6379> FT.SEARCH idx:product "@name:running @city:{Pune} @price:[0 70]" \
  RETURN 2 name price SORTBY price ASC LIMIT 0 10
1) (integer) 1
2) "product:1"
3) 1) "name"
   2) "Trail Running Shoe"
   3) "price"
   4) "59.9"
```

The reply starts with the total number of matches, followed by each matching key and a flat list of its returned fields. The query string has its own small language:

Query	Meaning
running shoe	Both words anywhere in any TEXT field (implicit AND)
@name:running	Word in the name field
@name:(trail road)	Both words inside the name field
@name:"running shoe"	Exact phrase
@name:run*	Prefix match

Query	Meaning
%runnig%	Fuzzy match (edit distance 1)
~@name:road	Exclude documents matching the term
@brand:{nike}	TAG field equals this value
@price:[10 70]	Numeric range, inclusive (use (for exclusive: [(10 70])
@price:[-inf 70]	Open-ended range
@loc:[77.6 12.97 5 km]	GEO radius (longitude latitude radius unit)

Query syntax.

The pipe character means OR: `@name:(trail|road)` matches either word and `@brand:{nike|adidas}` matches either brand. Spaces between clauses mean AND.

Important options of `FT.SEARCH`:

- `RETURN n field [AS alias] ...` limits which fields come back (and may take JSON paths). `NOCONTENT` returns only keys.
- `SORTBY field [ASC|DESC]` sorts by a field; declare it `SORTABLE` for speed.
- `LIMIT offset count` pages the results. The default is `LIMIT 0 10`, which surprises people who expect all matches. `LIMIT 0 0` only counts.
- `PARAMS n name value ...` passes values by name (`$name` in the query), which avoids string concatenation and is required for vector queries. It needs `DIALECT 2` or higher.
- `DIALECT n` chooses the query language version. Newer dialects change syntax details and multi-value JSON results, so state it explicitly in queries you write, and check the configured default on your server with `FT.CONFIG GET DEFAULT_DIALECT`.

WATCH OUT

Never build queries by concatenating user input Characters like `|`, `@`, `{`, `-` and `*` have meaning in the query language. A product name typed by a user can change the query, in the same way SQL injection does. Pass user text as a `PARAMS` value, or escape punctuation with a backslash, and limit the allowed characters. Likewise, tokenization splits on punctuation: searching for an e-mail address or a SKU such as `AB-1002` in a `TEXT` field needs care, and a `TAG` field with escaped punctuation is usually the right choice for IDs.

3.14.3 Aggregations

`FT.AGGREGATE` runs a pipeline over the matching documents: group, reduce, sort, transform.

Average price and count per brand

TERMINAL

```
127.0.0.1:6379> FT.AGGREGATE idx:product "*" GROUPBY 1 @brand \
  REDUCE COUNT 0 AS items REDUCE AVG 1 @price AS avg_price \
  SORTBY 2 @items DESC LIMIT 0 5
1) (integer) 2
2) 1) "brand"
   2) "nike"
   3) "items"
   4) "1"
   5) "avg_price"
   6) "59.9"
3) 1) "brand"
   2) "adidas"
   3) "items"
   4) "1"
   5) "avg_price"
   6) "79"
```

Each clause feeds the next: `GROUPBY 1 @brand` groups by one property, `REDUCE` computes per-group values (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX` and more), `SORTBY` orders the groups, and `APPLY expr AS name` creates computed properties. `LOAD` can pull in fields that are not `SORTABLE`, but it is much slower, so mark aggregated fields `SORTABLE` instead. For paging through huge aggregation results, use `WITHCOURSE`.

3.14.4 Hybrid search (Redis 8.4)

Keyword search (BM25-style text scoring) finds documents that contain your words; vector search finds documents that mean something similar. Real search boxes want both. Redis 8.4 added **FT.HYBRID**, which runs a text query and a vector similarity query together and fuses the two ranked lists, by default with **Reciprocal Rank Fusion (RRF)**, which scores each document by its rank in each list instead of trying to compare incomparable scores.

REDIS

```
FT.HYBRID idx:docs \
  SEARCH "@title:(return policy)" \
  VSIM @embedding $query_vec \
  COMBINE RRF 2 WINDOW 40 \
  LOAD 2 title url \
  LIMIT 0 10 \
  PARAMS 2 query_vec "<binary float32 blob>"
```

The clauses are **SEARCH** (the text part, same syntax as above), **VSIM** (the vector field and the query vector parameter), **COMBINE** (RRF, or **LINEAR** with weights) and the usual **LIMIT**, **SORTBY**, **LOAD** and **PARAMS**. Note that the numbers after **COMBINE RRF** and similar options count the tokens that follow, a convention of this command. The exact grammar is detailed in the command reference and may change between 8.x releases; vector blobs are covered in the next chapter.

3.14.5 What the Query Engine is not

- **It is not SQL.** There are no joins across indexes, no foreign keys and no multi-document transactions. It indexes one document at a time.
- **Indexes cost memory and write time.** Every indexed write updates the index. Measure with **FT.INFO** before you index a big dataset.
- **The key is the source of truth.** Treat the index as a derived view: after you write a document, do not assume a search issued at the same instant sees it, and fetch the document by key when you need the exact current value.
- **Scaling.** On a single node it just works. Check the documentation for how the Query Engine behaves on Redis Cluster before you rely on it there; many teams run search on a dedicated instance.
- **Deep paging is expensive.** Large **LIMIT** offsets get slower and slower; use **FT.AGGREGATE** with **WITHCURSOR**, or sort by a unique field and continue from the last value.

INTERVIEW ANGLE

How do you query Redis data by something other than the key? Either maintain your own secondary indexes with sets and sorted sets (tag sets for equality, a sorted set for range queries), which is fast but you must keep them consistent in application code or Lua; or use the Query Engine: declare an index with **FT.CREATE** on hashes or JSON, and query with **FT.SEARCH** or **FT.AGGREGATE**. The engine keeps the index in sync with writes, and supports text, tag, numeric, geo and vector fields.

REAL WORLD

ShopStream's catalogue search The catalogue cache keeps **product:{id}** hashes for fast lookups by ID. A search index **idx:product** on the same hashes backs the "filter by brand, price and city" sidebar. The authoritative data still lives in the relational database; if the index is ever lost, **FT.CREATE** rebuilds it from the cached hashes.

KEY POINTS

- The Query Engine adds secondary indexes to Redis 8: define them with **FT.CREATE** on hashes or JSON, with **TEXT**, **TAG**, **NUMERIC**, **GEO** and **VECTOR** fields.
- **FT.SEARCH** filters, sorts and pages; the default limit is 10 results; pass user input via **PARAMS**.
- **FT.AGGREGATE** runs group, reduce and sort pipelines; mark aggregated fields **SORTABLE**.
- **FT.HYBRID** (8.4) fuses keyword and vector results, by default with RRF.
- Index only what you query; indexes cost memory and are not a substitute for a relational database.

3.15 Vector Search and Vector Sets

Keyword search matches words. Ask a keyword index for "waterproof jacket for hiking" and it will miss a product described as "rain shell for trekking", because the words differ even though the meaning is the same. **Vector search** matches meaning. A machine-learning model called an **embedding model** turns a piece of content (a sentence, a product description, an image) into a list of numbers, a **vector**, typically a few hundred to a few thousand floats long. Content with similar meaning ends up as nearby points in that space. To search, you embed the query with the same model and ask for the stored vectors closest to it. That operation is called **nearest-neighbour** search, or **KNN** when you want the K closest.

Redis can store the vectors and run this search in two ways: as a **VECTOR** field in a Query Engine index (flexible, combined with filters and text), and as the dedicated **vector set** data type (simple, new, in beta in Redis 8). Redis does not create embeddings; your application calls an embedding model (a hosted API or a local model) and sends the numbers to Redis.

3.15.1 Distance metrics and index types

"Close" needs a definition. The Query Engine supports three metrics:

- **COSINE**: compares the angle between vectors and ignores their length. Distance 0 means the same direction. A common default for text embeddings.
- **L2**: straight-line (Euclidean) distance.
- **IP**: inner product (dot product). For vectors normalised to length 1 it gives the same ranking as cosine.

Use the metric your embedding model recommends. An index also needs an algorithm:

Algorithm	How it works	When to choose it
FLAT	Compares the query with every vector (exact, brute force)	Small data sets (thousands to a few hundred thousand), or when you need perfect recall
HNSW	Builds a layered graph of neighbours and walks it (approximate)	Large data sets; trades a little accuracy for much lower latency

Vector index algorithms.

HNSW (Hierarchical Navigable Small World) is **approximate**: it may miss a true neighbour. You tune the trade-off with **M** (edges per node, default 16), **EF_CONSTRUCTION** (effort while building, default 200) and **EF_RUNTIME** (effort while searching, default 10; raise it for better recall and slower queries).

3.15.2 Vectors in a Query Engine index

Store the vector as a binary string of little-endian 32-bit floats in a hash field, or as an array of numbers in a JSON document. The index declares its type and size, and every vector must match exactly.

REDIS

```
FT.CREATE idx:docs ON HASH PREFIX 1 doc: SCHEMA \
  title TEXT \
  brand TAG \
  embedding VECTOR HNSW 6 TYPE FLOAT32 DIM 384 DISTANCE_METRIC COSINE
```

The **6** after **HNSW** is the number of attribute tokens that follow (three name-value pairs). **DIM** must equal your model's output size. The query pattern is "filter, then rank by distance":

REDIS

```
# Top 5 nearest products, restricted to brand nike. $vec is a binary blob passed in PARAMS.
FT.SEARCH idx:docs "(@brand:{nike})=>[KNN 5 @embedding $vec AS dist]" \
  PARAMS 2 vec "<384 float32 values as bytes>" \
  RETURN 2 title dist SORTBY dist DIALECT 2
```

The part before **=>** is an ordinary filter (***** means no filter); the part in brackets is the nearest-neighbour request. **AS dist** names the distance so you can return and sort by it, a smaller value meaning closer for **COSINE** and **L2**. **DIALECT 2** (or higher) is required. Remember that the default **LIMIT** is 10, so ask for more than that with **LIMIT 0 n** when you use a larger K.

Producing the blob in Java

```

VectorBytes.java
package com.shopstream.search;

import java.nio.ByteBuffer;
import java.nio.ByteOrder;

public final class VectorBytes {

    private VectorBytes() {}

    /** Encodes floats as the little-endian FLOAT32 blob Redis expects. */
    public static byte[] toBlob(float[] vector) {
        ByteBuffer buffer = ByteBuffer.allocate(vector.length * Float.BYTES)
            .order(ByteOrder.LITTLE_ENDIAN);
        for (float value : vector) {
            buffer.putFloat(value);
        }
        return buffer.array();
    }

    public static float[] fromBlob(byte[] blob) {
        ByteBuffer buffer = ByteBuffer.wrap(blob).order(ByteOrder.LITTLE_ENDIAN);
        float[] vector = new float[blob.length / Float.BYTES];
        for (int i = 0; i < vector.length; i++) {
            vector[i] = buffer.getFloat();
        }
        return vector;
    }
}

```

What to notice: the byte order must be little-endian (Java's `ByteBuffer` defaults to big-endian), and the blob length must be exactly **DIM** times 4 bytes. A wrong size gives an error; a wrong byte order gives no error but nonsense results. Store the blob with a binary-safe write (a `byte[]` hash field), not through a string serializer.

3.15.3 Vector sets (beta)

Redis 8.0 introduced **vector sets**, a data type resembling a sorted set, but each element carries a vector instead of a score. There is no schema or index to declare: you add elements and query by similarity. The set is built on the HNSW algorithm. Vector sets are marked beta in the Redis 8 materials, so treat the API as subject to change and re-check the command pages when you upgrade.

```

VADD products VALUES 3 0.1 1.2 0.5 sku:1001
VADD products VALUES 3 0.2 1.1 0.4 sku:1002
VADD products VALUES 3 0.9 0.1 0.0 sku:2001
VSETATTR products sku:1001 '{"brand":"nike","price":59}'
VSIM products VALUES 3 0.1 1.2 0.4 WITHSCORES COUNT 2
VSIM products ELE sku:1001 FILTER '.brand == "nike" && .price < 100' COUNT 5
VCARD products
VDIM products

```

- **VADD** key [REDUCE dim] (FP32 | VALUES num) vector element [Q8 | NOQUANT | BIN] ... adds an element, either from a binary float blob (FP32) or from num numbers written out. **REDUCE** projects to fewer dimensions to save memory. The quantization options store vectors in 8 bits, full precision, or one bit per dimension; lower precision saves memory and costs some accuracy.
- **VSIM** key (ELE element | FP32 blob | VALUES num ...) returns the most similar elements: use **ELE** to search by an element already in the set. **WITHSCORES** returns similarity from 1 (identical) down to 0, **COUNT** limits results, and **EF** raises search effort.
- **VSETATTR**/**VGETATTR** attach a JSON attribute to an element, and **FILTER** in **VSIM** restricts results with a small expression over those attributes.
- **VCARD** gives the element count, **VDIM** the dimensions, **VREM** removes an element, and **TYPE** reports **vectorset**.

Choose by need. A Query Engine index gives you text + tags + numbers + vectors in one query, hybrid search and aggregation. A vector set is lighter when you only need "nearest neighbours of this vector, optionally with simple attribute filters", for example recommendations by item similarity.

3.15.4 Semantic cache and RAG, a preview

Two popular uses show why vectors live in Redis:

- **Semantic cache.** Calling a large language model (LLM) is slow and costs money. Many users ask the same thing in different words. Store each question's embedding with the model's answer (and a TTL). For a new question, run a KNN query with $K = 1$; if the nearest cached question is closer than a threshold you chose, return the stored answer instead of calling the model. Choose the threshold by testing with real questions: too loose returns wrong answers, too strict never hits.
- **Retrieval-augmented generation (RAG).** To answer questions about your own documents, split them into chunks, embed each chunk, and store chunk text plus vector. At question time embed the question, fetch the top K chunks with a vector (or hybrid) query, and put them into the model's prompt as context. Redis is the fast retrieval layer; filters (tenant, language, access rights) narrow what the model may see.

Later parts of the book build on this in the patterns and use-case chapters; here the point is the building block.

WATCH OUT

Dimensions, model and metric are a contract. If you change the embedding model, every stored vector is invalid, since vectors from different models (or different sizes) cannot be compared. Version the index name and the key prefix (`doc:v2:`), re-embed in the background, and switch over when ready. Also keep memory in mind: 1,000,000 vectors of 1536 float32 numbers take about 6 GB for the vectors alone, before the HNSW graph overhead.

KEY POINTS

- An embedding turns content into a vector; nearby vectors mean similar meaning. Redis stores vectors and searches them, but your application produces the embeddings.
- In the Query Engine, declare a **VECTOR** field (**FLAT** exact or **HNSW** approximate) with **TYPE**, **DIM** and **DISTANCE_METRIC**, and query with `=>[KNN k @field $blob AS dist]` and **DIALECT 2**.
- Vectors are little-endian float32 blobs; byte order and length must match.
- Vector sets (**VADD**, **VSIM**, **VSETATTR**) are a simpler beta type for similarity search with attribute filters.
- Typical uses: semantic caching of LLM answers and retrieval for RAG; changing the model means re-embedding everything.

3.16 Probabilistic Data Structures

Sets are exact, and exactness costs memory. Sometimes "almost right, in tiny space" is the better bargain. Probabilistic data structures answer questions about large streams of data using small, fixed memory in return for a bounded, known error. You have already met one: the HyperLogLog counts distinct items. Redis 8 includes five more families:

Bloom filter, **Cuckoo filter**, **Count-Min Sketch**, **Top-K** and **t-digest**. Before Redis 8 these came from the separate RedisBloom module; now the commands exist in the server, with the prefixes **BF.**, **CF.**, **CMS.**, **TOPK.** and **TDIGEST.**

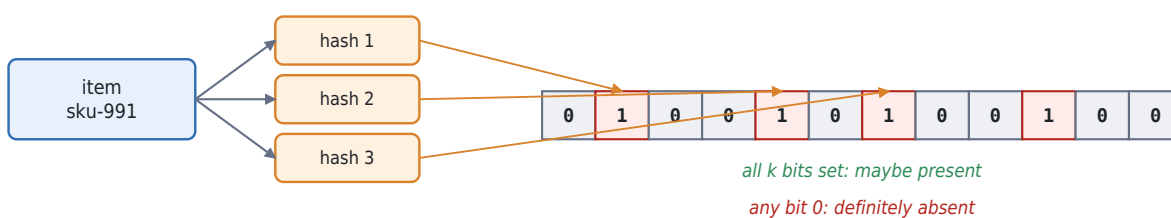


Figure 3.6. A Bloom filter: each item sets several bits chosen by hash functions. A lookup that finds any of its bits at 0 is definitely absent; all bits at 1 means probably present.

3.16.1 Bloom filter: "have I seen this?"

A Bloom filter answers set membership: is X in the set? Its answer is one of two things: **definitely not** or **probably yes**. It never gives a false negative; it sometimes gives a false positive, at a rate you choose. Internally it is a bit array and several hash functions: adding an item sets a few bits, and a lookup checks the same bits.

ShopStream's catalogue service gets requests for product IDs that do not exist (typos, scrapers). Each one misses the cache and hits the database. A Bloom filter holding every valid product ID rejects most of those requests before they reach the database, a defence against cache penetration discussed in the patterns part.

```

Create, add, check
127.0.0.1:6379> BF.RESERVE bf:products 0.01 1000000
OK
127.0.0.1:6379> BF.ADD bf:products product:42
(integer) 1
127.0.0.1:6379> BF.ADD bf:products product:42
(integer) 0
127.0.0.1:6379> BF.EXISTS bf:products product:42
(integer) 1
127.0.0.1:6379> BF.EXISTS bf:products product:99999
(integer) 0
127.0.0.1:6379> BF.MADD bf:products product:1 product:2 product:3
1) (integer) 1
2) (integer) 1
3) (integer) 1
127.0.0.1:6379> BF.MEXISTS bf:products product:1 product:77
1) (integer) 1
2) (integer) 0

```

BF.RESERVE key error_rate capacity [EXPANSION n] [NONSCALING] creates a filter: here a 1% false-positive rate for an expected one million items. **BF.ADD** returns 1 when the item was newly added and 0 when it probably existed already. If you call **BF.ADD** without reserving first, Redis creates a filter with default settings, which is rarely what you want, so always reserve explicitly.

Sizing. A Bloom filter needs roughly 9.6 bits per item at 1% error and about 14.4 bits per item at 0.1%. One million items at 1% error therefore take about 1.2 MB, compared with tens of megabytes for a set of IDs. If you add more items than **capacity**, a scaling filter (the default) adds another, larger sub-filter, growing by **EXPANSION** (default 2) each time, and the real error rate drifts up slowly; **NONSCALING** returns an error instead. You cannot delete items from a Bloom filter, and you cannot list them.

3.16.2 Cuckoo filter: membership with deletes

A Cuckoo filter answers the same question as a Bloom filter but stores small fingerprints of items in a table, which allows **CF.DEL**. It is often a bit faster on lookups and can use less space at low error rates, but inserts can fail when the table is full.

```

CF.RESERVE cf:active-sessions 100000
CF.ADD cf:active-sessions session:abc
CF.EXISTS cf:active-sessions session:abc
CF.DEL cf:active-sessions session:abc

```

Choose Bloom when you only add; choose Cuckoo when items must be removed (and delete only items you know you added, since deleting an item that was never added can remove a different item's fingerprint).

3.16.3 Count-Min Sketch: "how often?"

A Count-Min Sketch estimates how many times each item occurred in a stream, in fixed memory, using a small matrix of counters and several hash functions. It can **over-estimate** (hash collisions add to counters) but never under-estimates.

```

CMS.INITBYDIM cms:views 2000 5
CMS.INCRBY cms:views product:42 1 product:7 3
CMS.QUERY cms:views product:42 product:7

```

`CMS.INITBYDIM` key `width` `depth` sets the matrix size: more width reduces the size of the error, more depth reduces the chance of a large error. `CMS.INITBYPROB` key `error probability` lets you state the accuracy instead. `CMS.INCRBY` adds counts and `CMS.QUERY` reads estimates. Use a sketch when the number of distinct items is too large to keep a counter for each.

3.16.4 Top-K: the heavy hitters

Top-K keeps the K most frequent items of a stream. It uses a compact structure that remembers only candidates likely to be in the top K. Which products are viewed most, right now?

Track the top 3 viewed products

TERMINAL

```
127.0.0.1:6379> TOPK.RESERVE topk:views 3
OK
127.0.0.1:6379> TOPK.ADD topk:views p42 p42 p7 p99 p42 p7
1) (nil)
2) (nil)
3) (nil)
4) (nil)
5) (nil)
6) (nil)
127.0.0.1:6379> TOPK.LIST topk:views
1) "p42"
2) "p7"
3) "p99"
127.0.0.1:6379> TOPK.QUERY topk:views p42 p1000
1) (integer) 1
2) (integer) 0
```

`TOPK.ADD` returns, for each added item, the item that was pushed out of the top K (or nil if none was). `TOPK.LIST` returns the current list and `TOPK.QUERY` tells whether items are in it. The counts are approximate; the width, depth and decay (the probability that a counter shrinks when a competing item arrives) can be tuned in `TOPK.RESERVE`, and the defaults are fine to start. For exact top lists over bounded data, a sorted set with `ZINCRBY` is simpler and exact; use Top-K when the item universe is unbounded.

3.16.5 t-digest: percentiles

Averages hide pain; engineers want the 95th and 99th percentile of a response time. Keeping every sample is expensive. A **t-digest** keeps a compressed summary of a distribution that is very accurate near the extremes (the tails, where percentiles like p99 live).

REDIS

```
TDIGEST.CREATE td:latency
TDIGEST.ADD td:latency 12 15 11 230 14 13 16 18
TDIGEST.QUANTILE td:latency 0.5 0.95 0.99
TDIGEST.CDF td:latency 100
TDIGEST.MIN td:latency
TDIGEST.MAX td:latency
```

`TDIGEST.QUANTILE` takes fractions from 0 to 1 and returns one estimated value for each; `TDIGEST.CDF` answers the reverse question (what fraction of observations are at or below 100?). Digests can be merged with `TDIGEST.MERGE`, so you can keep one per minute or per instance and combine them later.

3.16.6 Choosing among them

Question	Structure	Guarantee
Have I seen this item?	Bloom (or Cuckoo if deletes)	No false negatives, few false positives
How many distinct items?	HyperLogLog	About 0.81% standard error
How often did this item occur?	Count-Min Sketch	Never under-counts
Which items are most frequent?	Top-K	Approximate heavy hitters
What is the p99 of this metric?	t-digest	Accurate tails

Which structure for which question.

GOTCHA

Probabilistic means plan for the error. A false positive from a Bloom filter in front of a database only costs an unnecessary lookup, which is harmless. The same false positive used to decide "this user has already redeemed the coupon" would wrongly refuse a customer. Use these structures where an error is cheap and the savings are large, and keep an exact check behind them wherever a wrong answer matters. Also reserve the filter with realistic capacity and error rate up front, because resizing means rebuilding it.

INTERVIEW ANGLE

How does a Bloom filter protect a database from cache penetration? The filter holds every valid key (here, product IDs). A request for an ID is first checked with **BF.EXISTS**. A "no" is certain, so the request is rejected without touching the cache or database. A "yes" may be a false positive, so it continues the normal path and may then find nothing. The filter must be filled at start-up and updated when new products are created because it cannot remove items, and it must be rebuilt periodically if deletions are common.

KEY POINTS

- Probabilistic structures trade exactness for small, bounded memory; they are built into Redis 8 (**BF.**, **CF.**, **CMS.**, **TOPK.**, **TDIGEST.**).
- Bloom: membership with no false negatives and no deletes; Cuckoo adds deletes. Reserve with a realistic capacity and error rate.
- Count-Min Sketch estimates frequencies (only over-estimates); Top-K tracks heavy hitters; t-digest estimates quantiles such as p99.
- Use them where an error is cheap, and keep an exact check where it is not.

3.17 Time Series

A time series is a sequence of measurements stamped with a time: the temperature of a warehouse sensor every ten seconds, the number of orders per minute, the response time of an API. The workload has a clear shape. Writes arrive in time order, almost never change, and the questions are about ranges ("the last hour"), aggregates ("average per five minutes") and retention ("keep fine detail for a day, summaries for a year"). You can model this with sorted sets, but a dedicated type does the compression, aggregation and expiry for you. Redis 8 includes the **time series** data type (formerly RedisTimeSeries), with commands starting with **TS.**

3.17.1 Creating a series and adding samples

A series is a key holding **samples**: a timestamp (milliseconds since the Unix epoch, a 64-bit integer) and a floating-point value. Create it with options, or let the first **TS.ADD** create it with defaults.

REDIS

```
TS.CREATE sensor:wh1:temp RETENTION 86400000 DUPLICATE_POLICY LAST \
LABELS site wh1 metric temp
```

- **RETENTION** is in milliseconds: samples older than that (relative to the newest sample) are dropped. **0** (the default unless the server config says otherwise) keeps samples forever.
- **ENCODING COMPRESSED** is the default and compresses chunks heavily; **UNCOMPRESSED** is rarely needed.
- **DUPLICATE_POLICY** says what to do if a sample arrives with an existing timestamp: **BLOCK** (error, the default), **FIRST**, **LAST**, **MIN**, **MAX** or **SUM**.
- **LABELS** attach key-value metadata (**site wh1**) used to find series later, like a tiny secondary index.
- **CHUNK_SIZE** sets the internal chunk size in bytes (default 4096); you rarely change it.

Add and read samples

TERMINAL

```
127.0.0.1:6379> TS.ADD sensor:wh1:temp 1760000000000 21.5
(integer) 1760000000000
127.0.0.1:6379> TS.ADD sensor:wh1:temp * 21.7
(integer) 1760000012345
127.0.0.1:6379> TS.MADD sensor:wh1:temp 1760000060000 21.9 sensor:wh1:temp 176000120000 22.4
1) (integer) 1760000060000
2) (integer) 176000120000
127.0.0.1:6379> TS.GET sensor:wh1:temp
1) (integer) 176000120000
2) 22.4
127.0.0.1:6379> TS.INCRBY orders:count 1 TIMESTAMP *
(integer) 176000130001
```

TS.ADD *key timestamp value* returns the timestamp of the sample; the timestamp *** means "now" according to the server. **TS.MADD** adds samples to several series in one call. **TS.INCRBY** and **TS.DECRBY** add to the latest sample's value, handy for counters. Samples normally arrive in time order; inserting an older sample is possible within the retention window, subject to the duplicate policy.

3.17.2 Querying ranges and aggregating

TS.RANGE *key from to* returns samples between two timestamps, where *-* and *+* mean earliest and latest.

TS.REVRANGE returns them newest first. Add **AGGREGATION** to get one value per time bucket:

Average temperature per five minutes

TERMINAL

```
127.0.0.1:6379> TS.RANGE sensor:wh1:temp - + AGGREGATION avg 300000
1) 1) (integer) 1759999800000
   2) 21.7
2) 1) (integer) 1760000100000
   2) 22.4
```

AGGREGATION *avg 300000* groups samples into 300,000 ms (5 minute) buckets, aligned to the Unix epoch by default, and averages each. Other aggregators include *sum*, *count*, *min*, *max*, *range*, *first*, *last*, *std.p*, *std.s*, *var.p*, *var.s* and *tw* (time-weighted average). Useful options: **COUNT** *n* limits the number of samples or buckets returned, **FILTER_BY_VALUE** *min max* drops outliers (like a sensor's error value) before aggregating, **ALIGN** changes how buckets line up, **BUCKETTIMESTAMP** chooses whether a bucket is labelled by its start, middle or end, and **EMPTY** reports empty buckets. You can also pass several aggregators separated by commas with no spaces (*avg,min,max*).

Many series at once

Because each series is its own key, a dashboard often needs the same metric from all sensors. **TS.MRANGE** queries many series selected by their **labels**, and can group them:

REDIS

```
TS.MRANGE - + AGGREGATION avg 300000 FILTER metric=temp site=wh1
TS.MRANGE - + AGGREGATION max 3600000 FILTER metric=temp GROUPBY site REDUCE max
TS.QUERYINDEX metric=temp
```

FILTER takes label expressions such as *metric=temp*, *site!=wh2* or *site=(wh1,wh2)*. At least one equality filter is required. **TS.QUERYINDEX** lists the keys that match, without data. In a Redis Cluster each series is a separate key on some slot, so check how multi-series queries behave on your cluster setup before depending on them.

3.17.3 Retention and compaction rules

Raw samples are expensive to keep for a year. The usual design is a **downsampling** cascade: keep raw data for a short time, plus automatically computed summaries kept longer. A compaction rule does that on write.

REDIS

```
TS.CREATE sensor:wh1:temp:1m RETENTION 2592000000 LABELS site wh1 metric temp res 1m
TS.CREATERULE sensor:wh1:temp sensor:wh1:temp:1m AGGREGATION avg 60000
```

The destination series must exist first. Each time a bucket (here a minute) completes in the source, the aggregate is appended to the destination, which has its own, longer retention (30 days in this sketch, written in milliseconds). Queries over long ranges then read the small, summarised series instead of the raw one. Create one rule per resolution and per aggregation type you need (an average rule does not give you the maximum).

3.17.4 When to use Redis time series

Need	Good fit	Why
Recent metrics, dashboards, rate and counters, sub-millisecond reads	Redis time series	In memory, compressed, built-in aggregation
Years of history, huge cardinality, ad hoc SQL	A dedicated time-series or analytics database	Disk-based storage and richer query languages
Just counting events per minute	INCR on bucketed keys, or TS . INCRBY	No extra structure needed
Event log you replay	Streams	Ordering and consumer groups, not aggregation

Time series storage options.

Memory is the limit: everything lives in RAM, so combine short retention with compaction rules, and measure with **MEMORY USAGE** on a realistic series. Compression works well for regular timestamps and slowly varying values.

NOTE

Samples are doubles. Values are 64-bit floating-point numbers, so keep money out of time series as raw floats if exactness matters (store cents as integers; they fit exactly below 2^{53}). Newer versions also support **NaN** samples and the aggregators **countNaN** and **countAll**.

INTERVIEW ANGLE

How would you keep one year of per-second sensor data in Redis without running out of memory? You wouldn't keep the raw seconds for a year. Keep raw samples with a retention of hours or days, and add compaction rules (**TS . CREATERULE**) that produce minute and hourly aggregates into destination series with longer retention. Dashboards read the aggregated series. If the raw data must be kept, archive it to a disk-based store.

KEY POINTS

- The time series type stores timestamp-value samples with compression, retention and labels; built into Redis 8, commands begin with **TS .**
- **TS . ADD** (timestamp * = now), **TS . MADD**, **TS . INCRBY**, **TS . GET**, **TS . RANGE** with **AGGREGATION**, and **TS . MRANGE** with label **FILTER**.
- **DUPLICATE_POLICY** decides what happens on equal timestamps; **RETENTION** bounds the age of samples.
- Compaction rules (**TS . CREATERULE**) downsample automatically so long ranges are cheap.
- Everything is in RAM: plan retention and resolutions, and archive long history elsewhere.

PART 4

Redis Inside

How the engine really works: threads, memory layout, expiry, eviction, persistence, transactions, scripting and security.

In this part

- 4.1 The Server Architecture
- 4.2 Memory Layout and Encodings
- 4.3 Expiration
- 4.4 Memory Limits and Eviction
- 4.5 RDB Snapshots
- 4.6 The Append-Only File (AOF)
- 4.7 Choosing a Persistence and Durability Strategy
- 4.8 Transactions: MULTI, EXEC and WATCH
- 4.9 Lua Scripting with EVAL
- 4.10 Redis Functions
- 4.11 Pipelining, Blocking Commands and Latency
- 4.12 RESP3 and Client-Side Caching
- 4.13 The Redis Security Model

4.1 The Server Architecture

Up to now we treated Redis as a black box that answers commands quickly. This Part opens the box. Understanding what the server does between "bytes arrive on a socket" and "reply leaves the socket" lets you predict behaviour instead of memorising it: why one slow command stalls every client, why **UNLINK** exists next to **DEL**, why a **BGSAVE** can make latency spike, and why more CPU cores do not make a single Redis instance run commands in parallel.

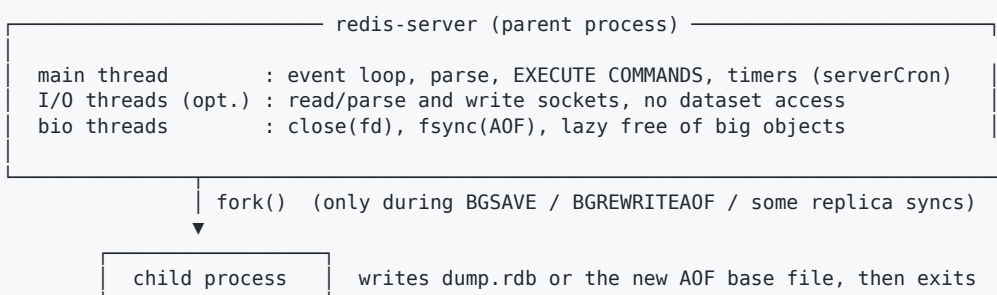
This chapter covers the process anatomy (threads and child processes), the event loop, the life of one command, the I/O threads, and the practical consequences for latency. The chapters that follow zoom into memory, expiry, eviction and persistence.

4.1.1 One process, a few threads, occasional children

A running **redis-server** is one operating-system process. Inside it you will find:

- **The main thread.** It runs the event loop, parses client requests, executes every command against the dataset, and (in the classic design) writes replies. All reads and writes of your keys happen here, one command at a time. This is why Redis needs no locks around its data structures.
- **I/O threads (optional).** Helper threads that move bytes between sockets and buffers. They never touch the dataset. Disabled by default.
- **Background threads (always there).** Small worker threads for jobs that would block the main thread: closing files, calling **fsync** on the AOF, and freeing big objects (lazy free). Redis calls this subsystem *bio* (background I/O).
- **Child processes (occasionally).** When Redis writes an RDB snapshot or rewrites the AOF, it calls **fork()**. The child process writes the file while the parent keeps serving clients.

Threads and processes of one Redis server



The key sentence to remember: **commands run on one thread**. Everything else in the picture exists to keep that one thread from waiting.

4.1.2 The event loop

Redis is *event driven*. Instead of one thread per client, which would need locks and many stacks, a single loop asks the operating system "which of my sockets are ready?" and handles only those. The library that implements this is called **ae** (a simple event library written for Redis). On Linux it uses **epoll**; on BSD and macOS it uses **kqueue**; other systems fall back to **evport** or **select**. The technique is called I/O multiplexing.

The loop handles two kinds of events:

- **File events:** a socket became readable (a client sent bytes, or a new client is connecting) or writable (room to send a reply).
- **Time events:** periodic jobs, chiefly **serverCron**, which runs **hz** times per second (default **hz 10**, so every 100 ms). It does housekeeping: the active expire cycle, incremental rehashing of big dictionaries, client timeouts, checking if a background save finished, and updating statistics.

One turn of the event loop (simplified pseudocode)

```
while (!stop) {
    beforeSleep();                // flush replies, fsync policy, fast expire cycle,
                                // handle blocked clients that can now be served
    events = epoll_wait(timeout); // sleep until some socket is ready (or a timer is due)
    for (e in events) {
        if (e.readable) readQueryFromClient(e.client); // read -> parse -> EXECUTE
        if (e.writable) writeToClient(e.client);       // send pending reply bytes
    }
    processTimeEvents();          // serverCron when due
}
```

Note that "execute" sits inside the loop body. While a command runs, the loop does not turn: no other client is read, no reply is sent, no timer fires. A command that takes 10 ms therefore adds up to 10 ms of waiting to every other client that arrived during it.

4.1.3 The life of one command

Take the command **HSET product:1001 name "Mug" price 1299** sent by a ShopStream catalog-service. These are the steps on the server.

1. **Accept and register.** When the connection was opened, Redis created a *client* structure for it: a query buffer (bytes received but not yet consumed), an output buffer (replies waiting to be sent), the selected database, the authenticated user, and so on.
2. **Read.** The socket becomes readable. Redis reads available bytes into the client's query buffer. A single read can contain half a command or many commands (pipelining).
3. **Parse.** The RESP protocol is length-prefixed, so parsing is cheap: the array header tells Redis how many arguments follow and each bulk string states its own length. A request that is incomplete simply waits for more bytes. Because every argument has a length, values may contain any bytes, including newlines.
4. **Look up the command.** The name **HSET** is looked up in the command table (a dictionary). Redis checks the arity (number of arguments), that the client is authenticated, that the ACL user may run this command on these keys, and a few state checks: is the server out of memory (**maxmemory** with **noeviction**) while this is a write command, is this a read-only replica, is the client inside **MULTI**.
5. **Execute.** The command's C function runs. It finds the key in the keyspace dictionary, creates or updates the hash, and prepares the reply (here the integer **2**). It also notes side effects: mark the key as modified for **WATCH**, emit a keyspace notification if enabled, and tell the AOF and replication layers to log the command.
6. **Reply.** The reply bytes are appended to the client's output buffer. They are usually sent just before the loop sleeps again (**beforeSleep**), which batches replies from many clients into fewer system calls. Only if the socket cannot take everything does Redis register a writable event and finish later.
7. **Propagate.** The write is appended to the replication stream (and to the AOF buffer when AOF is on), so replicas and the log see the same commands in the same order.

Two details explain real behaviour. First, because step 6 batches replies, the average latency under load stays low even though the loop is shared. Second, step 7 happens at the end of the command, so replicas and the AOF see exactly the sequence the main thread executed; there is no concurrent interleaving to reason about.

4.1.4 I/O threads

At high request rates the main thread spends a large share of its time not on your data but on system calls: reading bytes from sockets, parsing, and writing replies. **I/O threads** let helper threads do that part in parallel while the main thread keeps executing commands.

- Redis 6.0 introduced threaded I/O, mainly for writing replies, with an additional option for reads.
- Redis 8.0 reworked the I/O threading implementation as part of its performance work; the release notes describe large command-throughput and latency gains from many optimisations together. Treat the headline percentages as best-case vendor numbers measured on specific hardware and workloads, not as a promise for your system.
- The model is unchanged where it matters: **I/O threads never execute commands**. The main thread still runs each command, in order, alone.

You enable it with one directive:

redis.conf

CONF

```
# Total number of threads used for I/O, counting the main thread.
# 1 (the default) means threading is off. Set it in redis.conf, then restart.
io-threads 4
```

Guidance that holds across versions:

- Leave it at the default until you have measured. A typical single-node ShopStream cache serving a few tens of thousands of commands per second does not need it.
- Consider it when a single Redis process saturates one CPU core on network handling and you have spare cores. Set it clearly below the number of cores (the sample `redis.conf` suggests starting with `io-threads 4` only on machines with at least 4 cores).
- Verify the exact directive set for your version in its `redis.conf`; older releases also had `io-threads-do-reads`, and later versions changed how reads are threaded.
- If you are CPU-bound on command execution itself, I/O threads do not help. Scale out with replicas for reads or with Cluster for writes (see the Part on scaling).

GOTCHA

More threads is not "multi-core Redis". A common misunderstanding is that `io-threads 8` makes Redis execute eight commands at once. It does not. Commands, Lua scripts and transactions remain strictly serialized on the main thread. If you want to use eight cores for data work, run several Redis processes, which is exactly what Redis Cluster does.

4.1.5 Background threads and lazy free

Some jobs are too slow to run on the main thread, so Redis hands them to the bio threads:

- **Close file descriptors.** Closing a big file can block, for example when the kernel has to release its pages.
- **AOF fsync.** With `appendfsync everysec`, a background thread calls `fsync` once per second, so a slow disk does not stall commands (more on this in the AOF chapter).
- **Lazy free.** Deleting a key that holds a million elements means freeing a million small allocations. Done on the main thread, that is a visible pause. With lazy free the main thread unlinks the key from the keyspace in constant time and a background thread frees the memory.

You decide when to use lazy free in two ways. Explicitly, by choosing **UNLINK** instead of **DEL**, or **FLUSHALL ASYNC/FLUSHDB ASYNC** instead of the synchronous versions. Or by configuration, so even normal paths free in the background:

Directive	When it applies	Default
<code>lazyfree-lazy-user-del</code>	A plain DEL behaves like UNLINK	no
<code>lazyfree-lazy-user-flush</code>	FLUSHALL / FLUSHDB without a modifier behave as ASYNC	no
<code>lazyfree-lazy-eviction</code>	Keys removed by maxmemory eviction are freed in the background	no
<code>lazyfree-lazy-expire</code>	Keys removed because their TTL elapsed are freed in the background	no
<code>lazyfree-lazy-server-del</code>	Implicit deletes, for example when SET or RENAME overwrites a big value	no

Directive	When it applies	Default
replica-lazy-flush	A replica flushes its old data asynchronously before a full sync	no

Lazy free directives. Defaults shown are those in the shipped redis.conf template; check your version.

For big collections, make **UNLINK** your habit and consider turning on the lazy-free options if your values can get large. Freeing is not the only cost of big values, but it is the one that surprises people.

4.1.6 Fork and copy-on-write, briefly

To produce an RDB snapshot without pausing clients, Redis calls `fork()`. The operating system gives the child a copy of the parent's memory *lazily*: both processes share the same physical pages until one of them writes. When the parent modifies a page after the fork, the kernel copies that page first (**copy-on-write**, COW) so the child keeps seeing the frozen snapshot.

Two costs follow, and both matter for latency:

- The `fork()` call itself must copy the page tables, which grows with dataset size. On a large instance this can block the main thread for tens or hundreds of milliseconds, and noticeably longer on some virtualised environments. **INFO stats** reports the last duration as `latest_fork_usec`.
- While the child runs, every first write to a shared page allocates a new page. In a write-heavy workload memory use can rise sharply, up to almost double in the worst case. Leave headroom in `maxmemory` and on the machine for this.

The persistence chapters later in this Part return to fork in detail. For now remember: the main thread is blocked only during the `fork()` call, not for the whole save.

4.1.7 What this means for latency

The single-thread model gives Redis its simplicity and its sharpest edge. Latency of a command is its own execution time plus the time spent waiting behind the commands ahead of it.

Most commands take microseconds. The problem commands are those whose cost grows with the size of a collection or the size of the reply:

Command	Why it is slow	Safer alternative
KEYS pattern	Walks the entire keyspace, $O(N)$	SCAN with COUNT
SMEMBERS, HGETALL on huge collections	Builds and sends one giant reply	SSCAN / HSCAN, or smaller collections
LRANGE 0 -1 on a long list	Same, plus $O(N)$ copying	Page with LRANGE start stop
DEL on a key with millions of elements	Frees every element synchronously	UNLINK
SUNION / SINTER on big sets	$O(\text{total size})$ work in one command	Store results, split the work, or compute elsewhere
FLUSHALL (synchronous)	Frees the whole dataset	FLUSHALL ASYNC
Long Lua script or EVAL loop	Holds the loop for its whole run	Keep scripts short and bounded
SORT on large inputs	$O(N \log N)$ work in the main thread	Pre-sorted structure such as a sorted set

Commands that can block the loop on big data.

Two facts help you reason about cost. Complexity is documented on every command page as $O(1)$, $O(\log N)$, $O(N)$ and so on. And the slow log (**SLOWLOG GET**) records commands whose execution time exceeded `slowlog-log-slower-than` (default 10000 microseconds). It measures the execution step only, not waiting in the queue or network time, so a healthy slow log does not prove clients see low latency.

WATCH OUT

One slow command is everyone's slow command. If one worker issues **HGETALL** on a hash with two million fields and it takes 400 ms, then during those 400 ms the cart service, the session store and the rate limiter, all sharing that Redis, are frozen. The symptom in the application is a burst of timeouts across unrelated features. Find these commands with **SLOWLOG GET** and **INFO commandstats** and fix the data model; do not simply raise client timeouts.

A related queueing effect is client buffering. Each client has an input buffer (bounded by `client-query-buffer-limit`, 1 GB by default) and an output buffer. A slow consumer, for example a subscriber that does not read, makes its output buffer grow in Redis memory. Redis protects itself with `client-output-buffer-limit` rules per client class (normal, replica, pubsub) and will disconnect offenders. The chapter on pipelining and latency later in this Part covers this in context.

4.1.8 Seeing it on a live server

You can observe the effects described here with a few commands. `INFO commandstats` shows, for each command, how often it ran and how much time it used in total, which is the quickest way to find the expensive one.

Which commands consume the time?

TERMINAL

```
127.0.0.1:6379> INFO commandstats
# Commandstats
cmdstat_set:calls=120000,usec=210000,usec_per_call=1.75,rejected_calls=0,failed_calls=0
cmdstat_get:calls=980000,usec=1150000,usec_per_call=1.17,rejected_calls=0,failed_calls=0
cmdstat_hgetall:calls=450,usec=9800000,usec_per_call=21777.78,rejected_calls=0,failed_calls=0
```

In this illustrative output `HGETALL` was called only 450 times but consumed more total time than a million `GET`s, at about 22 ms per call. That is the signature of a big-key problem. The command to inspect the cause is `MEMORY USAGE` (next chapter) or `redis-cli --bigkeys`.

4.1.9 Connections, buffers and blocking commands

Several limits and behaviours around clients follow from the architecture.

- `maxclients` (default 10000) caps simultaneous connections. Redis also needs file descriptors for persistence files and replication, so it raises its own limit when it can; if the operating system limit is lower, Redis lowers `maxclients` and logs a warning. When the cap is hit new connections are refused with `ERR max number of clients reached`. A Spring service with a connection pool of 50 per instance and 40 instances already uses 2000 connections, so count before you scale out.
- `timeout` (default 0, never) closes idle clients after N seconds; `tcp-keepalive` (default 300 seconds) probes dead peers. Connection pools should validate or refresh connections rather than rely on the server to clean up.
- **Input and output buffers** belong to each client and live in Redis memory. The input buffer is limited by `client-query-buffer-limit`; output buffers by `client-output-buffer-limit`, with separate default rules for normal clients, replicas and pub/sub subscribers. A subscriber that stops reading is disconnected when its buffer passes the pub/sub limit.
- **Blocking commands** such as `BLPOP`, `BRPOP`, `XREAD BLOCK` and `WAIT` do *not* block the event loop. Redis parks that client in a table of blocked clients keyed by what it waits for, keeps serving everyone else, and wakes the client when the data arrives or the timeout elapses. That is why thousands of idle workers waiting on a queue cost almost nothing, whereas a thousand clients each running an expensive command would not.
- **MULTI/EXEC and Lua** run their commands back to back without yielding to the loop. This gives atomicity and also means a long transaction or script blocks everyone. The chapters on transactions and Lua scripting in this Part show how to keep them short.

4.1.10 Experiment: feel the single thread

You can convince yourself of the one-command-at-a-time rule in two terminals. In the first, create a large set and ask for all of it; in the second, measure latency continuously with `redis-cli --latency`, which sends `PING` in a loop.

Terminal 1: build a big set, then read all of it

TERMINAL

```
$ seq 1 1000000 | sed 's/^/SADD big:set /' | redis-cli --pipe
$ redis-cli SMEMBERS big:set > /dev/null
```

Terminal 2: latency measured while terminal 1 runs the heavy command

TERMINAL

```
$ redis-cli --latency
min: 0, max: 1, avg: 0.11 (4521 samples)
min: 0, max: 118, avg: 0.62 (9714 samples)
```

The numbers above are illustrative: the exact maximum depends on your machine, but the pattern is reliable. While the reply of a million elements is built and sent, the `PING` in terminal 2 waits in the queue and its measured latency jumps from a fraction of a millisecond to many milliseconds. Replace `SMEMBERS` with a loop of `SSCAN big:set <cursor>`

`COUNT 1000` and the maximum falls back, because each call is tiny and other clients interleave between calls. That is the philosophy of the `SCAN` family: turn one long command into many short ones.

4.1.11 Where the time goes in practice

For a typical small command such as `GET product:1001`, Redis spends a handful of microseconds on parsing and execution. The rest of what the client measures is the network round trip (tens to hundreds of microseconds in a data centre, milliseconds across zones) and the client library. Three consequences shape how you write code:

1. **Round trips dominate.** A loop of 100 sequential `GET`s costs 100 round trips. Pipelining (sending commands without waiting for each reply) or multi-key commands such as `MGET` cut that cost by an order of magnitude. See the chapter on pipelining later in this Part.
2. **The server is rarely the bottleneck for small commands.** A well-tuned single instance handles very high rates of simple operations on one core; before adding I/O threads, look at client concurrency and round trips.
3. **Tail latency matters more than the average.** One heavy command per second can leave the average untouched while the 99.9th percentile of every other client climbs. Watch percentiles, not means.

INTERVIEW ANGLE

If Redis is single-threaded, how can it handle so many clients, and can it use multiple cores? Redis runs commands on one thread, so no locking is needed and each command is atomic. It serves many connections by I/O multiplexing (`epoll/kqueue`): the single loop only touches sockets that are ready. Each command is fast because the data is in RAM and the structures are efficient. Since 6.0 optional I/O threads can parallelise socket reads and writes, and background threads handle `fsync`, file closing and lazy free, but command execution is still on one thread. To use more cores for data you run several instances: replicas for reads, Redis Cluster shards for writes.

KEY POINTS

- Commands execute one at a time on the main thread, which makes every command atomic and removes the need for locks.
- The event loop (`ae`) multiplexes sockets with `epoll/kqueue`, and runs periodic work in `serverCron` at `hz` per second.
- A command goes through read, parse, look up, execute, reply and propagate; replies are batched before the loop sleeps.
- I/O threads (`io-threads`) speed up socket work only; background threads handle `fsync`, closing files and lazy free (`UNLINK`).
- `fork()` for RDB/AOF rewrite blocks briefly, and copy-on-write can raise memory use under heavy writes.
- Any $O(N)$ command on a big collection delays every client; use `SCAN`-family commands, `UNLINK` and small keys.

4.2 Memory Layout and Encodings

Redis keeps the whole dataset in RAM, so memory is both its speed and its bill. The same logical hash can cost 100 bytes or 10 000 bytes depending on how Redis stores it internally, and the difference is the kind of thing that decides whether a cache fits on a 4 GB instance or needs a 16 GB one. This chapter explains how the keypace is organised, which internal data structures back each Redis type, how to see which one you are using, and how to measure and reduce memory.

The central idea: a Redis type (string, list, hash, set, sorted set, stream) is a *contract*, not a data structure. Behind the contract Redis picks an **encoding**, an actual in-memory representation, and switches encodings as a value grows. Small values get compact, cache-friendly encodings; large values get structures with better asymptotic complexity.

4.2.1 The keypace is one big hash table

Every database (the numbered ones, 0 to 15) is a dictionary from key to value. The dictionary is a classic hash table with chaining: an array of buckets, each bucket a short linked list of entries. Looking up `product:1001` hashes the key name, picks a bucket, and compares keys along the chain. That is $O(1)$ on average.

When the table gets too full, it must grow (about doubling). Copying all entries at once would freeze the server, so Redis does **incremental rehashing**: it allocates the new, larger table, keeps both, and moves a few buckets on every dictionary operation and in `serverCron` (the `activeres hashing yes` setting lets cron help). During the move, lookups check both tables. Two practical consequences:

- During rehashing of a huge dictionary, memory briefly holds two bucket arrays (just the pointer arrays, not the values).
- Growing is smooth in latency terms, but the array allocation itself is a large block of memory, which is one reason an instance with 100 million keys needs headroom.

Each entry in the keyspace carries a **redisObject**: a small header (16 bytes on 64-bit builds) storing the type, the encoding, an LRU/LFU field used for eviction, a reference count, and a pointer to the actual data. That header, the key string, and the dictionary entry are the fixed per-key overhead, typically several dozen bytes before any of your data. Keep that in mind when you consider storing 50 million tiny values as separate keys.

4.2.2 Strings: SDS and three encodings

Redis does not use C strings. It uses **SDS** (Simple Dynamic Strings): a small header with the current length and allocated capacity, followed by the bytes. This gives O(1) length, safe appending (spare capacity avoids repeated reallocation), and binary safety (a value may contain zero bytes, since length is stored rather than found by scanning for a terminator).

A string value is stored in one of three encodings, which you can see with **OBJECT ENCODING**:

- **int**: the value is a number that fits in a 64-bit signed integer. Redis stores the number directly in the object's pointer field, with no separate allocation. This is why counters are cheap.
- **embstr**: a short string (44 bytes or less on the usual 64-bit build) stored in the *same* memory block as its redisObject. One allocation, good cache locality. Embedded strings are read-only; modifying one converts it.
- **raw**: longer strings, or any string that has been modified in place (**APPEND**, **SETRANGE**), get a separate SDS allocation.

```

The three string encodings
127.0.0.1:6379> SET views 100
OK
127.0.0.1:6379> OBJECT ENCODING views
"int"
127.0.0.1:6379> SET sku:1 "abc"
OK
127.0.0.1:6379> OBJECT ENCODING sku:1
"embstr"
127.0.0.1:6379> APPEND sku:1 x
(integer) 4
127.0.0.1:6379> OBJECT ENCODING sku:1
"raw"

```

The practical lesson: a string you only **SET** and **GET** is cheaper than one you keep mutating with **APPEND**. For counters use **INCR** on integers rather than storing numbers as text you edit yourself.

4.2.3 Compact encodings for small collections

For small hashes, sorted sets, sets and lists, a pointer-heavy structure (hash table, skip list, linked list) wastes memory: each element needs its own allocation plus pointers, and jemalloc rounds every allocation up to a size class. Redis therefore stores small collections in a **listpack**, and switches to the big structure when a limit is crossed.

Listpack

A **listpack** is a single contiguous block of bytes holding a sequence of entries, each encoded with a small header and its own length so you can walk forward and backward. There are no pointers between entries and just one allocation for the whole collection. Integers are stored in 1 to 9 bytes depending on magnitude, strings with a length prefix.

A hash with three fields in listpack encoding (conceptual layout)

total bytes	num entries	"name" len+bytes	"Mug" len+bytes	"price" len+bytes	1299 int	...	end mark
----------------	-------------	---------------------	--------------------	----------------------	-------------	-----	-------------

one malloc'd block; field/value pairs are just adjacent entries; lookup = linear scan

The trade-off is plain: lookup in a listpack is a linear scan, O(N), and inserting in the middle may move bytes. That is a fine bargain when N is small (the scan touches one or two cache lines) and a bad one when N is large. Hence the thresholds below. Listpack replaced the older **ziplist** in Redis 7.0; ziplist could trigger a "cascading update" when one entry's size change forced its neighbours' length headers to grow, and listpack was designed to avoid that.

Intset

A set whose members are all integers fits in an **intset**: a sorted array of integers of the smallest width (16, 32 or 64 bit) that holds all of them. Membership is a binary search, $O(\log N)$. Add one non-integer member, or exceed `set-max-intset-entries`, and it converts to a hash table. Newer versions also allow small sets of non-integers to use a listpack (the directive is `set-max-listpack-entries`); check your version's `redis.conf`.

Quicklist

Lists can be long, and a single listpack would make pushes at the head costly. A **quicklist** is a doubly linked list whose nodes are listpacks. Pushing and popping at either end is $O(1)$, and the per-node packing keeps pointer overhead low. `list-max-listpack-size` controls node size: a positive number limits entries per node, a negative number limits bytes (-2, the default, means 8 KB per node). `list-compress-depth` can LZF-compress the middle nodes of very long lists, keeping the nodes near both ends uncompressed because those are the hot ones. A very short list is stored as one plain listpack.

Hash table

The big encoding for hashes and sets is the same kind of dictionary as the keypace: $O(1)$ average access, but each entry costs allocations and pointers, which is where the memory goes.

Skiplist

Sorted sets that outgrow their limits use a **skiplist** plus a hash table. The skiplist is a linked list with extra "express lanes" of random heights, so a search can skip ahead and find any element by score in $O(\log N)$; it also supports range queries by walking the bottom level. The companion dictionary maps member to score so `ZSCORE` is $O(1)$. The two structures share the member strings. This is why `ZADD` is $O(\log N)$ while `ZSCORE` is $O(1)$.

Skiplist idea: higher levels skip further

```
level 3: head → 70 → nil
level 2: head → 30 → 70 → 90 → nil
level 1: head → 10 → 30 → 45 → 60 → 70 → 80 → 90 → nil
search for 60: head→(L3: 70 too far)→L2: 30→(70 too far)→L1: 45→60 found
```

Radix tree for streams

A stream is a **radix tree** (rax) keyed by entry ID. Each leaf holds a *macro node*: a listpack containing many entries, with field names stored once per node and repeated entries compressed as deltas. `stream-node-max-bytes` (default 4096) and `stream-node-max-entries` (default 100) bound a node. This makes appending and range-reading by ID fast and compact. Consumer groups add their own rax of pending entries (the PEL), covered in the chapter on streams.

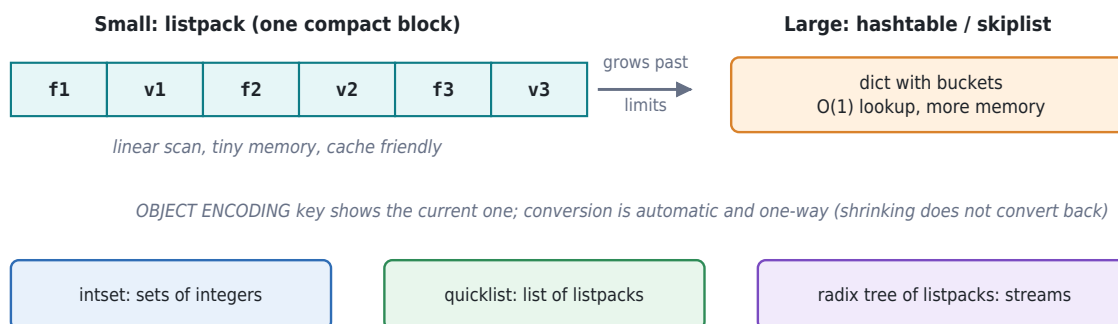


Figure 4.1. Each type starts in a compact encoding and converts to a larger structure at a threshold. Notice that the conversion is one-way for most types: shrinking the value later does not convert it back automatically.

4.2.4 Which encoding do I have, and when does it change?

The table lists the encodings you can see with `OBJECT ENCODING`, and the default thresholds that trigger a change. Defaults below were confirmed against a Redis 7.0 server and the 8.0 configuration template. They can change between versions, so read them from your own server with `CONFIG GET`.

Type	Encodings	Switch to large form when	Directive (default)
string	int, embstr, raw	not an integer; length above 44 bytes; modified in place	none

Type	Encodings	Switch to large form when	Directive (default)
list	listpack, quicklist	list outgrows one node	list-max-listpack-size (-2)
hash	listpack, hashtable	more fields or longer values than limits	hash-max-listpack-entries (512), hash-max-listpack-value (64)
set	intset, listpack, hashtable	non-integer member, or too many members	set-max-intset-entries (512)
zset	listpack, skiplist	more members or longer members than limits	zset-max-listpack-entries (128), zset-max-listpack-value (64)
stream	stream	always a radix tree of listpacks	stream-node-max-bytes (4096), stream-node-max-entries (100)

Types, encodings and the config that switches them.

Hashes with per-field expiration (**HEXPIRE**, Redis 7.4 and later) use listpack and hashtable variants that also store the expiry times; depending on version you may see an encoding name such as **listpackex** for them. That is the same idea with an extra column.

You can watch a conversion happen. Here a hash starts as a listpack and becomes a hashtable once a value is longer than **hash-max-listpack-value**:

```

Encoding conversion by value length TERMINAL
127.0.0.1:6379> HSET product:1001 name "Mug" price 1299 stock 40
(integer) 3
127.0.0.1:6379> OBJECT ENCODING product:1001
"listpack"
127.0.0.1:6379> HSET product:1001 description \
"Hand-glazed ceramic mug, 350 ml, dishwasher and microwave safe, six colours"
(integer) 1
127.0.0.1:6379> OBJECT ENCODING product:1001
"hashtable"

```

A product description pushing the whole hash over the 64-byte value limit is a realistic ShopStream accident: one long field makes *all* the fields of that product use the memory-hungry encoding. If most of your hashes are small except one fat field, consider keeping the long text in its own key.

NOTE

Conversions are one-way Redis converts a collection to the large encoding when a threshold is crossed, but does not convert it back when elements are removed (a rebuild such as **RESTORE**, a restart that reloads from RDB, or copying into a new key can produce the compact form again). Never rely on a particular encoding in application logic; use it only for diagnosis.

4.2.5 Measuring memory

Three tools cover most needs.

MEMORY USAGE **key** [**SAMPLES** **n**] returns the bytes a key costs, including its key name and internal overhead. For nested values it samples some elements and extrapolates (default 5 samples; pass **SAMPLES 0** for an exact walk, which costs more CPU).

INFO memory gives the instance view. The fields to know are **used_memory** (what Redis asked the allocator for), **used_memory_rss** (what the operating system reports resident), **used_memory_peak**, **used_memory_dataset** versus **used_memory_overhead**, and **mem_fragmentation_ratio**.

MEMORY STATS and **MEMORY DOCTOR** give a breakdown and plain-language hints.

Cost of a small product, as a hash versus separate string keys

TERMINAL

```

127.0.0.1:6379> HSET product:1 name "Mug" price 1299 stock 40
(integer) 3
127.0.0.1:6379> MEMORY USAGE product:1
(integer) 96
127.0.0.1:6379> SET product:1:name "Mug"
OK
127.0.0.1:6379> MEMORY USAGE product:1:name
(integer) 72
127.0.0.1:6379> SET product:1:price 1299
OK
127.0.0.1:6379> MEMORY USAGE product:1:price
(integer) 56

```

These byte counts come from a test on Redis 7.0; yours will differ by version and key length, but the shape is stable. The one hash costs about as much as one or two of the three separate keys, so the three-key layout is roughly twice as large (and needs three lookups and three TTLs). Small objects as hashes in listpack encoding are the most memory-efficient way to store records.

Fragmentation and the allocator

Redis uses **jemalloc** by default on Linux. Allocators serve requests from size classes, so some bytes inside each block are wasted (internal fragmentation) and freed memory can be scattered in pieces too small to reuse (external fragmentation). The ratio `mem_fragmentation_ratio` is `used_memory_rss / used_memory`.

- Around 1.0 to 1.5 on a busy instance: normal.
- Much above 1.5: memory is resident but not in use. Typical causes are a large delete (or expiry wave) followed by new keys of different sizes, or a freshly started tiny instance (the ratio is meaningless when `used_memory` is a few megabytes).
- Below 1.0: the operating system has swapped part of Redis to disk. This is an emergency; latency will collapse.

If fragmentation is real and large, options are to restart through a replica failover, or to enable **active defragmentation** (`activedefrag yes`), which compacts memory in the background with bounded CPU (`active-defrag-cycle-min/max`). It is supported with the bundled jemalloc and is off by default.

4.2.6 Reducing memory: what actually works

1. **Use the right structure.** Small records as hashes (listpack) beat one key per field. Counters as integers. IDs as integers in sets (intset).
2. **Shorten keys and values where it matters.** A key like `shopstream:production:catalog:product:1001:name` repeats 40 bytes for every entry. Keep names readable, but do not be verbose at 50 million keys. Compress large JSON on the client when CPU is cheaper than RAM.
3. **Use expiry.** Data that should not live forever must have a TTL (next chapter).
4. **Avoid huge keys.** One key with 20 million elements is hard to delete, hard to migrate in a cluster, and spikes latency; split by bucket (for example `cart:{userId}` instead of a single `carts` hash).
5. **Tune thresholds with care.** Raising `hash-max-listpack-entries` keeps bigger hashes compact but makes every access scan more bytes. Measure with `MEMORY USAGE` and `redis-benchmark` before changing it in production.
6. **Upgrade.** Recent releases reduce memory by themselves. The 8.0 notes cite memory savings on replicas, and the 8.6 notes say hash and sorted-set operations use unified field/value and score/value structures, delivering a "substantial memory reduction". The notes give no fixed percentages for 8.6, so benchmark your own data.

4.2.7 A worked estimate: the ShopStream catalogue

Suppose ShopStream has 200 000 products and the catalogue cache keeps each as ten short fields, around 400 bytes of payload. Compare three layouts.

Layout A: one hash per product (listpack)

Each hash stays under 512 entries and every value under 64 bytes, so it is a listpack. Cost per product is roughly the payload plus a key name (`product:100000`, 14 bytes), the dictionary entry and redisObject (several dozen bytes), and allocator rounding. A planning figure is 500 to 600 bytes per product, so 200 000 products need about 100 to 120 MB. This is an estimate for planning, not a measurement; sample real data with `MEMORY USAGE` before you trust it.

Layout B: ten string keys per product

Now there are 2 000 000 keys. Each carries its own dictionary entry, redisObject and key string, and a long key name such as `product:100000:description`. The overhead per key is commonly 50 to 70 bytes plus rounding, so overhead alone is 100 to 140 MB before any payload, and every product needs ten lookups and ten TTLs. It is typically at least double the size of layout A.

Layout C: one JSON string per product

A serialised JSON document in a single string key uses one key and one value. It is compact when you always read the whole object, but you cannot change one field without rewriting everything. The Redis JSON type (covered in the data structures Part) keeps the document in a native binary tree and allows path updates, with its own overhead.

The choice depends on access pattern more than size. If the application nearly always reads the full product, a single value (hash or JSON string) wins. If it frequently updates a single field such as `stock`, a hash is the natural fit.

4.2.8 Bucketing: a classic memory trick

Because small hashes are so compact, there is an old technique for storing a very large number of tiny values: split the key space into buckets. Instead of 50 million keys `user:avatar:{id}`, store them as fields in hashes whose name comes from the id divided by a fixed number.

Instead of one key per user, one hash per 500 users

REDIS

```
# user id 123456789 -> bucket = id / 500 = 246913, field = id % 500 = 289
HSET avatar:246913 289 "u/ab12.png"
HGET avatar:246913 289
```

With a bucket size that keeps each hash within `hash-max-listpack-entries`, every bucket remains a listpack, saving the per-key overhead. The costs are real: you cannot give an individual field a key-level TTL (per-field TTL from 7.4 softens this), keys are harder to inspect, and hot buckets in a cluster can unbalance shards. Use it only when you have measured that key overhead dominates, typically beyond tens of millions of tiny entries.

4.2.9 Finding the big and the wasteful

Two `redis-cli` scans help find outliers. Both use `SCAN` internally, so they are safe on a live server, but they add load, so run them off peak or against a replica.

Find the biggest key of each type (shortened, illustrative)

TERMINAL

```
$ redis-cli --bigkeys

[00.00%] Biggest string found so far "session:abc123" with 212 bytes
[00.00%] Biggest hash   found so far "product:1001" with 9 fields
[28.57%] Biggest zset   found so far "lb:bestsellers:daily" with 51234 members

----- summary -----

Sampled 8402 keys in the keyspace!
Total key length in bytes is 129344 (avg len 15.39)

Biggest hash found "product:1001" has 9 fields
Biggest zset found "lb:bestsellers:daily" has 51234 members
```

`--bigkeys` counts elements; `--memkeys` reports bytes using `MEMORY USAGE`. Redis 8.6 adds a `HOTKEYS` command for finding the keys that receive the most traffic, which is a different problem from keys that use the most memory.

WATCH OUT

Memory counted by `maxmemory` is not the whole process `maxmemory` limits roughly the memory Redis uses for data. Output buffers for replicas, the replication backlog and AOF buffers are not counted toward eviction decisions the same way, and a fork for a snapshot may double memory under heavy writes. Setting `maxmemory` to 100% of the machine's RAM invites the out-of-memory killer. A common starting rule is to leave 25 to 50 percent of RAM free when persistence is on; the operations Part gives more precise sizing advice.

INTERVIEW ANGLE

Why does Redis store small hashes as a listpack, and what is the downside? A listpack is one contiguous allocation with no per-element pointers, so it is far smaller and cache friendly than a hash table with a node per field. The downside is $O(N)$ lookup. That is acceptable only while N is small, so Redis converts to a real hash table once the entry count or value size crosses `hash-max-listpack-entries` or `hash-max-listpack-value`. Same pattern for sorted sets (listpack to skiplist), lists (quicklist of listpacks) and sets (intset to hash table).

KEY POINTS

- A type is a contract; the encoding (**OBJECT ENCODING**) is the representation and changes with size.
- Small collections use listpack or intset; large ones use hashtable, skiplist or quicklist. Thresholds are configurable and conversion is one-way.
- Per-key overhead is real (dictionary entry, redisObject, key string); many tiny keys cost far more than a few small hashes.
- Measure with **MEMORY USAGE**, **INFO memory** and **MEMORY STATS**; read `mem_fragmentation_ratio` with the context above.
- Reduce memory by choosing structures, bucketing, TTLs, shorter keys and upgrading; leave RAM headroom beyond `maxmemory`.

4.3 Expiration

A cache without expiry slowly turns into a second copy of your database that is always a little wrong. Sessions that never expire fill memory with logged-out users. Redis solves both with a **time to live** (TTL): you attach a deadline to a key and Redis deletes the key when the deadline passes. This chapter shows the commands, then explains how Redis actually enforces them (it is more subtle than "a timer per key") and how expiry interacts with replicas, the AOF and hashes.

4.3.1 Setting, reading and removing a TTL

A TTL belongs to the key, not the value. Most keys have none and live until deleted or evicted. The relevant commands:

Command	What it does	Notes
<code>EXPIRE key seconds</code> <code>[NX XX GT LT]</code>	Set TTL in seconds	Options since 7.0: NX only if no TTL, XX only if TTL exists, GT only if new TTL is greater, LT only if smaller
<code>PEXPIRE key ms</code>	Same in milliseconds	Millisecond precision
<code>EXPIREAT / PEXPIREAT key ts</code>	Expire at an absolute Unix time (seconds / ms)	Good for "end of day" deadlines
<code>TTL / PTTL key</code>	Remaining time	-1 = key exists but has no TTL, -2 = key does not exist
<code>EXPIRETIME / PEXPIRETIME key</code>	Absolute expiry time	Since 7.0; same -1 / -2 codes
<code>PERSIST key</code>	Remove the TTL	Returns 1 if a TTL was removed
<code>SET key v EX s / PX ms / EXAT ts / PXAT ts</code>	Write and set TTL atomically	Prefer this over SET followed by EXPIRE
<code>SET key v KEPTTL</code>	Overwrite value, keep the old TTL	Without it, SET clears the TTL
<code>GETEX key EX s</code>	Read a value and refresh its TTL	Useful for sliding session expiry

Expiry commands.

Working with TTLs on a ShopStream session

TERMINAL

```
127.0.0.1:6379> SET session:abc123 "user:42" EX 1800
OK
127.0.0.1:6379> TTL session:abc123
(integer) 1800
127.0.0.1:6379> GETEX session:abc123 EX 1800
"user:42"
127.0.0.1:6379> EXPIRE session:abc123 100 GT
(integer) 0
```

Working with TTLs on a ShopStream session (continued)

TERMINAL

```
127.0.0.1:6379> PERSIST session:abc123
(integer) 1
127.0.0.1:6379> TTL session:abc123
(integer) -1
127.0.0.1:6379> TTL no:such:key
(integer) -2
```

Three behaviours catch people out:

- **Overwriting clears the TTL.** A plain `SET key newvalue` removes any existing expiry. If you refresh a cached product with `SET product:1001 json` and forget `EX`, the key becomes immortal. Use `SET ... EX` every time, or `KEEPTTL` if you really want to keep the old deadline.
- **Other commands keep it.** `INCR`, `APPEND`, `HSET`, `LPUSH` and similar modify the value and leave the TTL alone. `RENAME` moves the TTL to the new name. `DEL` removes the key and its TTL.
- **A past deadline deletes immediately.** `EXPIRE key 0`, `EXPIRE key -1`, or `EXPIREAT` with a timestamp in the past removes the key right away.

TIP

Keep the TTL atomic with the write `SET key value EX 60` is one command. `SET key value` followed by `EXPIRE key 60` is two; if the client crashes between them the key lives forever. The same reasoning is why `SETEX`, `PSETEX` and `SETNX` are legacy: the `SET` options cover them.

4.3.2 How Redis expires keys: lazy plus active

Redis stores a deadline per key in a second dictionary (the *expires* dictionary), a mapping from key to absolute millisecond timestamp. It would be too expensive to keep every deadline in a sorted structure and wake up exactly on time. Instead, Redis combines two cheap strategies.

Passive (lazy) expiration. Every time a command touches a key, Redis first checks the key's deadline. If it has passed, the key is deleted on the spot and the command sees "no such key". So a read never returns a logically expired value. This costs nothing for keys no one accesses, which is exactly the problem: an expired key that nobody asks for would sit in memory forever.

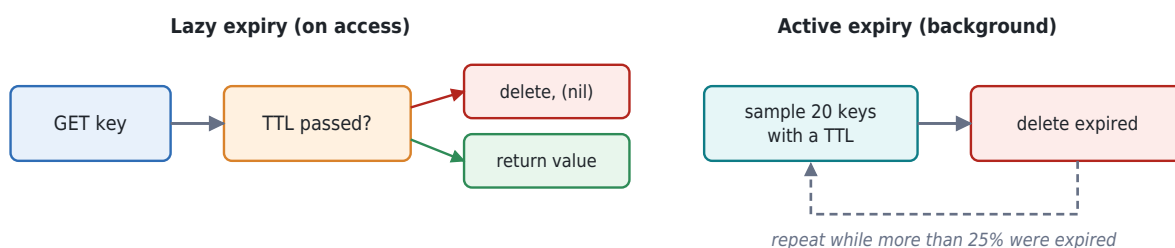
Active expiration. To reclaim those, Redis runs an *active expire cycle* many times per second, inside `serverCron` (and a short "fast" cycle before the loop sleeps). The classic algorithm, which is still the basis today:

1. Pick 20 random keys from the set of keys that have a TTL.
2. Delete every one of them that has expired.
3. If more than 25 percent of the sample was expired, go back to step 1.

The loop is time-bounded, by default to a small slice of CPU per second, so expiry can never take over the server. The directive `active-expire-effort` (1 to 10, default 1) lets you trade CPU for more aggressive cleanup if expired keys pile up.

Why 25 percent? A feedback loop that scales with the garbage

```
sample 20 keys with TTL → delete expired ones
├─ expired ≤ 25% of sample → stop; few dead keys, check again next tick
└─ expired > 25% of sample → sample again (likely lots of dead keys left)
```



both together keep memory bounded without scanning every key

Figure 4.2. A key is removed either when a command touches it after its deadline (lazy) or when the sampling cycle finds it (active). Notice that memory is reclaimed late for keys that are neither accessed nor sampled soon.

What follows from this design:

- **Expired keys can still use memory for a while.** After a mass expiry (for example thousands of sessions created in the same minute with the same TTL), the sampling cycle may take several seconds to clear them. `INFO stats` shows `expired_keys` (a running count) and `expired_stale_perc` indicates how stale the TTL set currently is.
- **Mass expiry at the same instant is a latency risk.** If a million keys expire in the same second, the active cycle can spend its full time budget repeatedly, and the main thread has less time for clients. Add random jitter to TTLs in caches (the caching patterns Part uses 10 to 20 percent jitter).
- **Lazy free applies.** With `lazyfree-lazy-expire yes` the memory of expired keys is released on a background thread.

GOTCHA

TTL does not mean "memory freed at that instant" `DBSIZE` and `INFO keyspace` count keys that are logically expired but not yet deleted. Do not size capacity from the number of keys you expect to be alive at a moment; look at `used_memory`.

4.3.3 Expiry with replicas, AOF and RDB

Expiry has to give the same result on every node, but clocks differ and replicas receive commands later. Redis therefore keeps one rule: **only the primary decides that a key has expired.**

- When the primary deletes an expired key (lazily or actively), it sends a `DEL` (or `UNLINK` with lazy free) to its replicas and appends it to the AOF. Replicas do not delete keys on their own clock.
- A replica still hides keys that are logically expired for `reads`: a `GET` on a replica returns nil for a key whose deadline has passed, even before the `DEL` arrives. Memory is freed only when the `DEL` comes.
- In the AOF, relative commands such as `EXPIRE key 100` are rewritten as absolute `PEXPIREAT` commands, so replaying the AOF later does not extend lifetimes. Likewise `SET ... EX 60` is propagated with an absolute deadline.
- An RDB file stores absolute expiry timestamps. When a primary loads an RDB it skips keys that have already expired; a replica loads them and waits for `DEL` from the primary. If the server was stopped for an hour, all keys with a TTL shorter than an hour are gone on restart.
- Inside a Lua script or `MULTI`, time is effectively frozen to the start of the script, so a key does not expire half way through; scripts see a consistent clock.

This means a primary and replica with badly skewed clocks can behave oddly with absolute timestamps (`PEXPIREAT`). Run NTP on all nodes.

4.3.4 Expiry on hash fields

Before Redis 7.4 a TTL applied only to a whole key, which made "one hash per user with some fields expiring early" impossible. Now fields can have their own TTL: `HEXPIRE`, `HPEXPIRE`, `HEXPIREAT`, `HPEXPIREAT`, `HTTL`, `HPTTL`, `HEXPIRETIME`, `HPERSIST`. Redis 8.0 added `HGETEX` (read and set field TTL), `HSETEX` (write and set TTL) and `HGETDEL` (read and delete). Field commands take a `FIELDS numfields field [field ...]` clause and return one status per field.

Per-field TTL on a ShopStream product hash (syntax from the 7.4+ command pages)

REDIS

```
HSET product:1001 name "Mug" price 1299 flash_price 999
HEXPIRE product:1001 3600 FIELDS 1 flash_price
HTTL product:1001 FIELDS 2 flash_price name
```

The `HTTL` reply for that call is a list with the remaining seconds for `flash_price` and `-1` for `name` (the field exists with no TTL); `-2` means the field does not exist. When the last field of a hash expires the key itself is removed. Field expiry is also implemented with a passive part and an active part, but it is tracked separately from key-level expiry, so a key can have its own TTL and also fields with TTLs. Use it for things like a flash-sale price or a "verified" flag that must lapse; do not use it as a substitute for a separate key when you need keyspace notifications per item.

4.3.5 Patterns: session expiry in a ShopStream service

Two designs for sessions show why the TTL commands have so many options.

- **Absolute expiry:** the session ends a fixed time after login no matter what. Set the TTL once with `SET ... EX` and never refresh it. Use `EXPIRE ... NX` when you re-save the session and must not extend it.
- **Sliding expiry:** the session lives as long as the user stays active. Refresh the TTL on every request with `GETEX key EX seconds` (read and refresh in one round trip), or `EXPIRE key seconds`. Combine with `EXPIRE ... LT` or `GT` if you need to enforce a rule such as "never shorten" or "never exceed a cap".

A practical combination is a sliding 30-minute idle timeout with an absolute 12-hour maximum: store the login time in the session, refresh the idle TTL on each request, and refuse to refresh once the absolute limit passed. Here it is with the Spring `StringRedisTemplate`:

```
SessionStore.java JAVA

package com.shopstream.session;

import java.time.Duration;
import java.util.Optional;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Component;

@Component
public class SessionStore {

    private static final Duration IDLE_TIMEOUT = Duration.ofMinutes(30);

    private final StringRedisTemplate redis;

    public SessionStore(StringRedisTemplate redis) {
        this.redis = redis;
    }

    public void create(String sessionId, String userId) {
        // value and TTL in one atomic command: SET key value EX seconds
        redis.opsForValue().set("session:" + sessionId, userId, IDLE_TIMEOUT);
    }

    public Optional<String> touch(String sessionId) {
        // GETEX: read the value and slide the expiry in a single round trip
        String userId = redis.opsForValue()
            .getAndExpire("session:" + sessionId, IDLE_TIMEOUT);
        return Optional.ofNullable(userId);
    }

    public void destroy(String sessionId) {
        redis.delete("session:" + sessionId);
    }
}
```

What to notice: `set(key, value, Duration)` maps to a single `SET ... PX` (or `EX`) command, so there is no window with a TTL-less key, and `getAndExpire` maps to `GETEX`. Never write `set(key, value)` followed by `expire(key, ...)` in two calls.

4.3.6 TTL jitter and keyspace notifications

If you load 100 000 products at start-up with the same 10-minute TTL, they all expire in the same few seconds, the cache goes cold at once, and the database sees a burst of misses (a cache avalanche, covered in the caching failures chapter). Add random jitter when you set the TTL:

```
TtlJitter.java JAVA

package com.shopstream.cache;

import java.time.Duration;
import java.util.concurrent.ThreadLocalRandom;

public final class TtlJitter {

    private TtlJitter() {
    }
}
```

TtlJitter.java (continued)

JAVA

```

/** Returns base plus a random 0..20 percent, so entries written together expire apart. */
public static Duration withJitter(Duration base) {
    long extra = (long) (base.toMillis() * 0.2 * ThreadLocalRandom.current().nextDouble());
    return base.plusMillis(extra);
}
}

```

Redis can also publish an event when a key expires, using **keyspace notifications**. They are off by default (`notify-keyspace-events ""`) because they cost CPU. To receive **expired** events set `notify-keyspace-events Ex` (E selects keyevent channels, x the expired events) and subscribe to `__keyevent@0__:expired`. The events are emitted at the moment Redis actually deletes the key, not the instant the TTL elapsed, and because they travel through pub/sub nobody who was disconnected receives them.

WATCH OUT

Do not build workflows on expiry notifications. Keyspace notifications with the **expired** event fire when Redis *deletes* the key, which may be seconds after the deadline, and are delivered by fire-and-forget pub/sub, so they can be lost. The chapter on delayed tasks in the Patterns Part shows a safer design with a sorted set.

INTERVIEW ANGLE

How does Redis expire keys, and why can an expired key still be in memory? Two mechanisms. Lazy expiration checks the deadline whenever a key is accessed and deletes it if it is past due. Active expiration repeatedly samples 20 random keys that have a TTL, deletes the expired ones, and repeats if more than 25 percent were expired, within a CPU time budget. Keys that are never touched and not yet sampled stay in memory until a sample finds them. Replicas do not expire on their own; the primary sends **DEL** for each expired key.

KEY POINTS

- A TTL is per key; **SET** without **EX/KEEPTTL** clears it, so set value and TTL in one command.
- Expiry = lazy check on access + active random sampling (20 keys, repeat while over 25 percent are expired).
- Expired keys can linger in memory, and synchronized TTLs can cause latency bumps: add jitter.
- Only the primary expires keys; replicas and the AOF receive explicit deletes and absolute timestamps.
- Hash fields can have their own TTL since 7.4 (**HEXPIRE**), and 8.0 added **HGETEX/HSETEX/HGETDEL**.

4.4 Memory Limits and Eviction

Expiry removes data you declared temporary. **Eviction** removes data when Redis runs out of the memory you allowed it, even though you never said the data was disposable. It is what lets Redis serve as a cache: a full cache keeps working by dropping the least valuable entries to make room for new ones.

4.4.1 maxmemory and what happens when it is reached

Set a ceiling with **maxmemory** (bytes, or with suffixes like **2gb**). The default **0** means no limit on 64-bit systems, so Redis will grow until the operating system kills it. Always set a limit in production, below physical RAM with room for fork copy-on-write, buffers and the OS.

When a command that may add data arrives and memory use is above the limit, Redis tries to free memory according to **maxmemory-policy**. If it cannot (or the policy is **noeviction**), commands that would increase memory are rejected with an error, while reads and deletes still work.

The default policy refuses writes at the limit

TERMINAL

```

127.0.0.1:6379> CONFIG SET maxmemory 2mb
OK
127.0.0.1:6379> CONFIG GET maxmemory-policy
1) "maxmemory-policy"
2) "noeviction"
127.0.0.1:6379> SET product:99999 "x"
(error) OOM command not allowed when used memory > 'maxmemory'.
127.0.0.1:6379> GET product:1001
"{...cached json...}"

```

An application that gets `OOM command not allowed` from Redis usually surfaces it as a 500. For a cache this is the wrong default. For a primary datastore (queues, locks you cannot lose) it is the right one, because silently deleting data would be worse.

4.4.2 The eviction policies

Policy	Candidates	Chooses the victim by
noeviction	none	Does not evict; writes that need memory fail
allkeys-lru	all keys	Least recently used
allkeys-lfu	all keys	Least frequently used
allkeys-random	all keys	Random
allkeys-lrm	all keys	Least recently modified (Redis 8.6)
volatile-lru	keys with a TTL	Least recently used
volatile-lfu	keys with a TTL	Least frequently used
volatile-random	keys with a TTL	Random
volatile-ttl	keys with a TTL	Shortest remaining TTL
volatile-lrm	keys with a TTL	Least recently modified (Redis 8.6)

maxmemory-policy values.

The **volatile-*** policies only consider keys that have an expiry; if none do, they behave like **noeviction**. That makes them a trap: a cache whose writers forgot **EX** will start failing writes even though it is "full of cache data".

LRM (least recently modified), new in 8.6, orders by the last time a key was *written* rather than read. The release notes describe it as giving better control for write-heavy workloads; it suits data where being read often does not prove it is valuable, but being updated recently does. If you run an older server, the **CONFIG SET** for these policy names fails with an error that lists the policies your version accepts.

4.4.3 Approximated LRU and LFU

True LRU needs a linked list updated on every access: extra pointers per key and extra work per read. Redis approximates instead. Each key's `redisObject` holds a 24-bit field with a coarse last-access clock. To evict, Redis **samples** `maxmemory-samples` random keys (default 5), compares them, and evicts the best victim. Since version 3.0 it also keeps a small pool of good candidates between rounds, which improves accuracy. Raising `maxmemory-samples` to 10 gets close to true LRU at a modest CPU cost; lowering it saves CPU.

LFU uses the same 24 bits differently: 16 bits for the time of the last decrement and 8 bits for a logarithmic access counter (a Morris counter). The counter grows probabilistically, so it saturates only after a very large number of accesses:

- `lfu-log-factor` (default 10): higher means the counter grows more slowly. With 10, it saturates around one million accesses.
- `lfu-decay-time` (default 1): minutes without access before the counter drops by one. `0` disables decay.

LFU protects keys that are *consistently* popular from being flushed by a one-time scan of many rarely used keys, which pure LRU would let through. You can see the counter with **OBJECT FREQ** (only when an LFU policy is selected; otherwise Redis answers with an error).

```

The LFU counter rises with access
127.0.0.1:6379> CONFIG SET maxmemory-policy allkeys-lfu
OK
127.0.0.1:6379> SET hot 1
OK
127.0.0.1:6379> OBJECT FREQ hot
(integer) 5
127.0.0.1:6379> GET hot
"1"
127.0.0.1:6379> OBJECT FREQ hot
(integer) 6

```

(New keys start at a small initial counter value, 5, so they are not evicted immediately.)

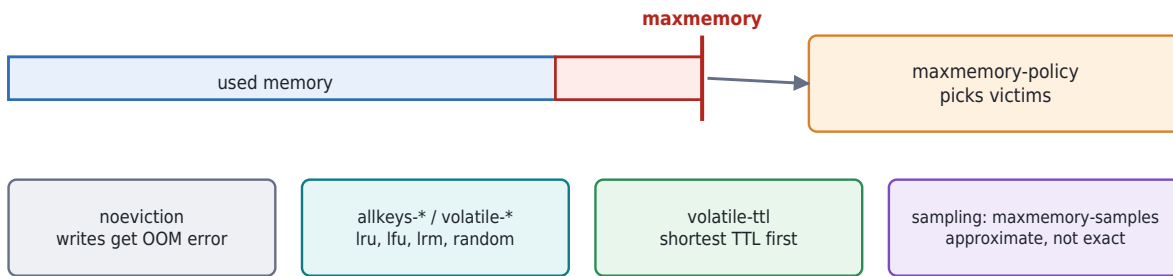


Figure 4.3. On each write at the memory limit Redis samples a few keys, picks the best victim per policy, evicts it, and repeats until memory is under the limit. Notice that the sample is random, so the result is an approximation.

4.4.4 Eviction is work on the main thread

Eviction happens before a command is executed: Redis checks memory, evicts until it is under the limit, then runs the command. A single big write may need to evict many keys, and freeing a large value takes time. Two knobs help:

- `maxmemory-eviction-tenacity` (default 10) bounds how long a single eviction round may run; higher values evict more aggressively per call, lower values protect latency but let memory overshoot temporarily.
- `lazyfree-lazy-eviction yes` frees evicted values on a background thread.

Replicas ignore `maxmemory` by default (`replica-ignore-maxmemory yes`): the primary evicts and sends `DEL` for evicted keys to replicas, so replicas stay identical. Memory used by replication output buffers and AOF buffers is not meant to count toward the `maxmemory` data limit, so leave spare RAM for them.

4.4.5 A worked scenario: sizing the catalogue cache

ShopStream's catalogue has 200 000 products at roughly 600 bytes each in Redis, about 120 MB for all of them, but the traffic is skewed: 5 percent of products receive most of the page views. The team sets `maxmemory 100mb` on a cache instance with `allkeys-lfu`. What happens?

1. The first misses fill the cache with hot and cold products alike until memory reaches 100 MB (about 83 percent of the whole catalogue).
2. A new write now triggers eviction: Redis samples keys and removes the one with the lowest LFU counter. Hot products have high counters (they are read constantly) and survive; a product seen once has a low counter and goes first.
3. The hit ratio stays high because the working set (the hot 5 percent, around 6 MB) fits comfortably. `evicted_keys` ticks up slowly; that is the cache doing its job, not an incident.

Now imagine a nightly report job that scans every product exactly once. Under `allkeys-lru` that scan makes every product look "recently used" and can push out the genuinely hot keys. Under `allkeys-lfu` the scan produces counters of 1 and the hot keys stay protected, which is why LFU is the better default for catalogue-style data. It is not free: LFU needs a few accesses to learn that a new key is valuable, and a key that was hot yesterday decays slowly (`lfu-decay-time`).

You can reproduce the basic behaviour on a laptop. With `maxmemory 2mb` and `allkeys-lru`, writing 30 000 keys of 100 bytes succeeds for every command, and `INFO stats` afterwards shows tens of thousands of `evicted_keys` while `DBSIZE` stays near what fits in memory:

```
Writes keep succeeding, old keys disappear (typical output) TERMINAL
127.0.0.1:6379> CONFIG SET maxmemory-policy allkeys-lru
OK
127.0.0.1:6379> DBSIZE
(integer) 5810
127.0.0.1:6379> INFO stats
# Stats
evicted_keys:30494
```

The `DBSIZE` and `evicted_keys` values are from a test on a 2 MB limit, so they will differ on your machine. The point is the pair: the dataset size is capped, and evictions counted instead of errors.

4.4.6 Choosing a policy

Situation	Use
Pure cache; every key can be rebuilt; all have TTLs or not	allkeys-lru, or allkeys-lfu if some keys are hot for long periods (product pages) and you fear scan pollution
Cache and durable data in the same instance (not recommended)	volatile-lru or volatile-lfu, and set TTLs only on the cache keys
Redis as primary store, queues, streams, locks	noeviction; plan capacity and monitor memory instead
Write-heavy data where recent writes matter (8.6+)	allkeys-lrm or volatile-lrm; benchmark first
Strict ordering by deadline (for example short-lived tokens)	volatile-ttl
Uniform access, no skew	allkeys-random is cheap and often adequate

Which policy for which job.

The cleanest design for a ShopStream deployment is to separate roles. The catalogue cache runs on an instance with `allkeys-lfu`; sessions, carts and the order stream run on another with `noeviction` and persistence. Mixing both on one instance forces compromises in both directions.

Monitoring eviction

Watch `evicted_keys` in `INFO stats`, `used_memory` against `maxmemory`, and the cache hit ratio from `keyspace_hits` and `keyspace_misses`. A steadily rising `evicted_keys` with a falling hit ratio means the cache is too small for its working set. If evicted keys include things you expected to be permanent (sessions), your policy or your separation of roles is wrong.

WATCH OUT

Evicting queue data is data loss. With any `allkeys-*` policy a list or stream used as a job queue is a candidate for eviction like any other key. One memory spike and the whole queue key can vanish, with no error. Keep queues and locks on a `noeviction` instance, or use `volatile-*` policies and never set a TTL on them.

INTERVIEW ANGLE

What happens when Redis reaches `maxmemory`, and how does it choose what to evict? With the default `noeviction`, write commands that need memory fail with an OOM error. With any other policy Redis evicts keys before executing the command: it samples `maxmemory-samples` keys (default 5) from all keys or only from keys with a TTL, depending on the policy, and removes the one that is least recently used, least frequently used, random, nearest to expiry (or least recently modified, in 8.6). LRU and LFU are approximations built from a per-key 24-bit field, avoiding the cost of exact bookkeeping.

KEY POINTS

- Always set `maxmemory`; the default of 0 lets Redis grow until the OS intervenes.
- `noeviction` rejects writes (reads still work); the other policies evict by LRU, LFU, random, TTL or (8.6) LRM, over all keys or only keys with a TTL.
- LRU and LFU are sampled approximations; `maxmemory-samples` trades accuracy for CPU.
- `volatile-*` policies do nothing for keys without TTL, so a cache without TTLs can still hit OOM.
- Do not put queues, locks or sessions you cannot lose on an instance that evicts; separate cache from durable roles.

4.5 RDB Snapshots

Everything we have discussed so far lives in RAM, and RAM is erased when the process stops. If Redis only ever held disposable cache data that would be fine, but sessions, carts, and order events must survive a restart. Redis offers two persistence mechanisms, which can be used alone or together: **RDB snapshots** (this chapter) and the **append-only file** (next chapter). The question to keep in mind for both is: *if the server dies right now, how much do I lose, and how long does the restart take?*

4.5.1 The idea: a point-in-time photograph

An RDB (Redis DataBase) file is a compact, binary dump of the entire dataset at one moment. Redis writes it to a file called `dump.rdb` (the default name). On restart, Redis reads that file back into memory. Think of it as a photograph of the dataset: small, fast to load, and perfect for backups, but anything that changed after the photo was taken is not in it.

The main trigger is configuration. Rules of the form `save <seconds> <changes>` mean: take a snapshot if at least `<changes>` write operations happened in the last `<seconds>`. The commonly shipped default (Redis 7.x and later templates) is:

```
redis.conf CONF
# Snapshot if: 3600s and >=1 change, or 300s and >=100 changes, or 60s and >=10000 changes
save 3600 1 300 100 60 10000
dbfilename dump.rdb
dir /data
```

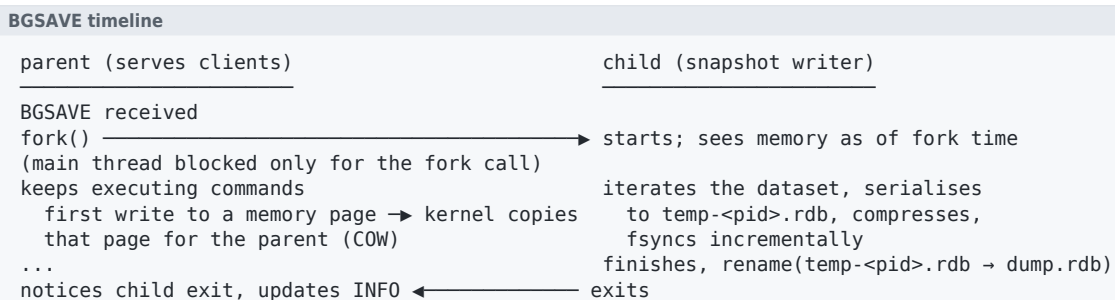
This rule set bounds data loss in a crash to minutes in quiet periods and still saves quickly under heavy writes. `save ""` disables snapshots entirely. You can also trigger one by hand:

- **BGSAVE** forks a child process and returns immediately. This is what you want.
- **SAVE** writes the snapshot **on the main thread** and blocks all clients until it finishes. Use it only in a maintenance window or tests.

Redis also snapshots on a clean shutdown (**SHUTDOWN**, or **SIGTERM**) if any save rule is configured, and a primary does a **BGSAVE**-style dump when a replica needs a full resynchronization (see the Part on replication).

4.5.2 How BGSAVE works: fork and copy-on-write

The trick that makes snapshots cheap is `fork()` plus **copy-on-write** (COW), introduced in the server architecture chapter. Here is the full sequence.



Points to notice:

1. **The snapshot is consistent** because the child sees frozen memory. It does not matter that the parent keeps changing data.
2. **The file is replaced atomically.** The child writes to a temporary file and renames it over `dump.rdb` at the end, so a crash during the save leaves the previous good snapshot in place.
3. **Memory can grow during the save.** Each page the parent modifies is duplicated. A mostly-read workload adds little; a workload that touches many keys can nearly double memory in the worst case.
4. **Only one child at a time.** Redis will not start a second **BGSAVE** or AOF rewrite while one is running; a request is deferred or refused.

WATCH OUT

Fork cost and Transparent Huge Pages `fork()` copies the process page tables, so the pause grows with dataset size (tens of milliseconds for a few GB on bare metal, more on some virtual machines). On Linux, Transparent Huge Pages (THP) make COW much worse because a one-byte write copies a 2 MB page: latency spikes and memory blow-up during saves. Redis logs a warning at startup if THP is enabled; disable it as the warning advises. Also set `vm.overcommit_memory = 1`, otherwise `fork()` can fail on a busy machine even though the child would barely use memory.

4.5.3 Observing and tuning

`INFO persistence` is the place to look. The fields that matter:

Field	Meaning
<code>rdb_changes_since_last_save</code>	Writes since the last successful snapshot: your current "at risk" count
<code>rdb_last_save_time</code>	Unix time of the last successful save (also available as <code>LASTSAVE</code>)
<code>rdb_last_bgsave_status</code>	ok or err; alert if not ok
<code>rdb_bgsave_in_progress</code>	1 while a child is running
<code>rdb_last_bgsave_time_sec</code>	How long the last save took
<code>latest_fork_usec</code> (in <code>INFO stats</code>)	Duration of the last fork in microseconds: the main-thread pause

Key `INFO persistence` fields for RDB.

Trigger and check a snapshot

TERMINAL

```
127.0.0.1:6379> BGSAVE
Background saving started
127.0.0.1:6379> LASTSAVE
(integer) 1791613516
127.0.0.1:6379> INFO persistence
loading:0
rdb_changes_since_last_save:0
rdb_bgsave_in_progress:0
rdb_last_save_time:1791613516
rdb_last_bgsave_status:ok
```

Directives worth knowing:

Directive	Default	Purpose
<code>save</code>	<code>3600 1 300 100 60 10000</code>	Automatic snapshot rules; "" disables
<code>dbfilename</code>	<code>dump.rdb</code>	File name inside dir
<code>dir</code>	working dir	Directory for RDB and AOF files; set it to a persistent volume
<code>rdbcompression</code>	yes	LZF-compress strings in the file; costs a little CPU
<code>rdbchecksum</code>	yes	CRC64 checksum at the end of the file; disabling saves a bit of CPU but removes corruption detection
<code>stop-writes-on-bgsave-error</code>	yes	Reject writes if the last background save failed, so you notice
<code>rdb-save-incremental-fsync</code>	yes	fsync the file every 32 MB while writing, avoiding one huge disk flush at the end

RDB-related configuration.

`stop-writes-on-bgsave-error yes` deserves attention. If the disk is full or read-only, the next write is refused with `MISCONF Redis is configured to save RDB snapshots, but it's currently unable to persist to disk`. That error is Redis protecting you from silently running without a backup. Fix the disk problem; turning the setting off just hides it.

4.5.4 What is inside the file

You do not need to parse RDB files by hand, but knowing the layout explains why loading is fast. The file starts with the magic string **REDIS** followed by a four-digit format version (for example **REDIS0010** on 7.0). Then come auxiliary fields (Redis version, creation time, memory used), and for each database a selector followed by the key-value pairs. Each pair is stored as an optional expiry (absolute time), a one-byte type, the key, and the value in a serialised form that reuses the compact encodings (a listpack is dumped almost as it is in memory). The file ends with an end marker and the CRC64 checksum.

Because the format is dense and mostly a straight copy, loading an RDB is typically much faster than replaying a log of commands. The same format is used for replica synchronization and for the **DUMP/RESTORE** commands of a single key. Tools: **redis-check-rdb** verifies a file, and **redis-cli --rdb out.rdb** downloads a snapshot from a running server over the network.

Restoring is simple: stop Redis, put the file at **dir/dbfilename**, start Redis. If AOF is enabled, Redis loads the AOF instead (see the next chapter), so remember to account for that.

4.5.5 A restore drill

The only way to know a backup works is to restore it. This short drill uses a snapshot from a test server and checks it with the bundled tool before loading.

Verify an RDB file before you rely on it

TERMINAL

```
$ redis-check-rdb dump.rdb
[offset 0] Checking RDB file dump.rdb
[offset 27] AUX FIELD redis-ver = '7.0.15'
[offset 41] AUX FIELD redis-bits = '64'
[offset 53] AUX FIELD ctime = '1791613649'
[offset 68] AUX FIELD used-mem = '939392'
[offset 80] AUX FIELD aof-base = '0'
[offset 82] Selecting DB ID 0
[offset 109] Checksum OK
[offset 109] \o/ RDB looks OK! \o/
[info] 1 keys read
[info] 0 expires
[info] 0 already expired
```

Then restore:

1. Stop the target server (**redis-cli SHUTDOWN NOSAVE**, so it does not overwrite the file you are about to copy with its own empty dataset).
2. Copy the backup to **dir/dbfilename** and check ownership so the **redis** user can read it.
3. Start the server with **appendonly no** for this first load (see the AOF chapters for why).
4. Verify: **DBSIZE**, a handful of known keys, and **INFO persistence** showing **loading:0**.

Large files take time to load, and while loading, Redis answers commands with **-LOADING Redis is loading the dataset in memory**. A readiness probe should wait for that to clear.

4.5.6 Strengths and weaknesses

Aspect	Advantage	Drawback
Data loss	Bounded by snapshot interval	Everything since the last snapshot is lost on a crash (minutes)
Restart time	Fast: compact, mostly a direct load	Still grows with dataset size
File	One compact file, easy to copy off-site or encrypt	Not human-readable
Performance	Parent does almost no I/O; high throughput	Fork pause and COW memory on large datasets
Backups	Ideal: each file is a complete point-in-time copy	Intervals are coarse

RDB at a glance.

REAL WORLD

A common production use of RDB only Teams that use Redis as a cache of a database, where the data can be rebuilt, often keep RDB with a lazy rule (or `save ""` on replicas) simply so that a restart warms the cache from the last snapshot rather than starting cold. A cold cache can send a stampede of reads to the database.

INTERVIEW ANGLE

How can Redis snapshot a multi-GB dataset without blocking clients? It calls `fork()`. The child process gets a copy-on-write view of memory frozen at fork time and writes the RDB file while the parent keeps serving commands. Pages the parent modifies are copied by the kernel so the child's view stays consistent. The only main-thread pause is the fork call itself (page-table copy), and the cost is extra memory proportional to the writes during the save. The file is written to a temp name and atomically renamed.

KEY POINTS

- An RDB file is a compact point-in-time snapshot; loading is fast, but data after the last snapshot is lost in a crash.
- **BGSAVE** forks a child and uses copy-on-write; **SAVE** blocks the server and is for maintenance only.
- Snapshots are triggered by `save <seconds> <changes>` rules, by **SHUTDOWN** and by replica full syncs.
- Watch `rdb_last_bgsave_status`, `rdb_changes_since_last_save` and `latest_fork_usec`; disable THP and set `vm.overcommit_memory = 1`.
- Keep `dir` on a persistent volume; copy snapshots off the machine for backups.

4.6 The Append-Only File (AOF)

Snapshots lose minutes of writes. For data like order events or carts that is often unacceptable. The **append-only file** (AOF) takes the opposite approach: instead of photographing the dataset, it keeps a *journal* of every write command, in order. To recover, Redis replays the journal from start to end. The loss window shrinks from minutes to about a second, or to zero at the highest cost.

4.6.1 How the AOF works

Every command that modifies data is, after it executes, serialised in the same RESP format clients use and appended to an in-memory AOF buffer. Before the loop sleeps, Redis writes the buffer to the file with `write()`, which hands the bytes to the operating system's page cache. A second step, `fsync()`, forces the OS to push them to the physical disk. When Redis calls `fsync` is the **appendfsync** policy, and it is the single most important durability setting you will touch.

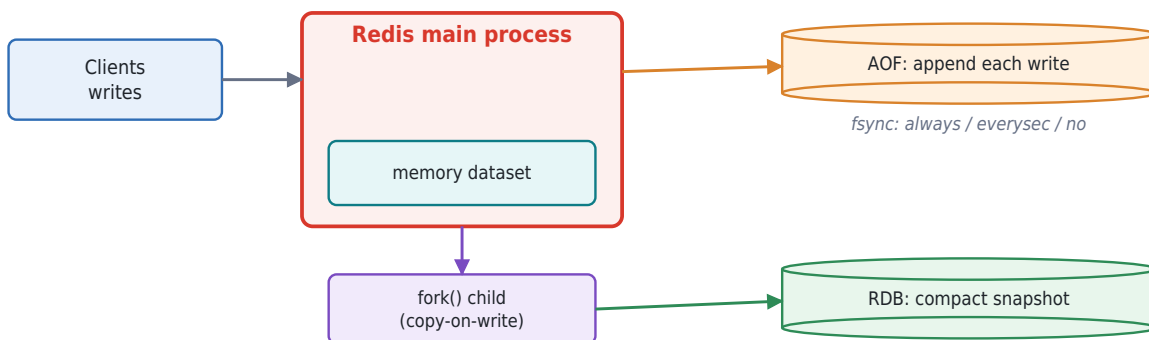


Figure 4.4. Redis can write an RDB snapshot, an append-only log, or both; on restart it loads the AOF if enabled, otherwise the RDB. Notice that the AOF records commands while the RDB records state.

Policy	When fsync runs	Data you can lose in a crash	Cost
always	After every write batch, before replying	Essentially none acknowledged to clients	Slowest; throughput limited by disk fsync latency
everysec	Once per second, on a background thread	About one second (a bit more if the disk stalls)	Small; the recommended default
no	Never; the OS flushes when it wants	Often up to ~30 seconds on Linux	Fastest; weakest guarantee

appendfsync policies.

Important subtleties:

- A process crash (Redis killed with `kill -9`) loses less than a power failure or kernel crash, because bytes already passed to `write()` are in the OS cache and survive the death of the Redis process. `fsync` protects against the machine going down.
- With `everysec`, if the background `fsync` takes longer than a second, Redis may delay writes to the file rather than let the backlog grow unbounded. In that pathological case the loss window can be about two seconds. `INFO persistence` exposes `aof_delayed_fsync`, a counter of such events.
- Disk latency is not Redis latency, but it can become it: a slow shared disk plus `always` makes every write wait.

You turn the AOF on in `redis.conf`; it is **off by default**:

redis.conf

CONF

```
appendonly yes
appendfsync everysec
appenddirname appendonlydir      # directory (inside dir) for the AOF files
aof-use-rdb-preamble yes          # base file is an RDB snapshot (compact, fast to load)
auto-aof-rewrite-percentage 100  # rewrite when file grew by 100% since last rewrite
auto-aof-rewrite-min-size 64mb   # ...and is at least this big
```

You can also start it on a running server with `CONFIG SET appendonly yes`, which triggers a rewrite to create the first base file. Edit `redis.conf` as well (or use `CONFIG REWRITE`) so the setting survives a restart.

4.6.2 Multi-part AOF (Redis 7.0 and later)

Before 7.0 the AOF was one file and rewriting it needed an in-memory "rewrite buffer" plus careful double writing. Since 7.0 the AOF is a **directory** of files tracked by a manifest:

- one **base** file (at most one): a snapshot of the dataset at the time of the last rewrite, in RDB format when `aof-use-rdb-preamble yes` (the default) or as plain commands otherwise;
- one or more **incremental** files: the commands written since that base;
- a **manifest** that lists them in order.

Here is a real directory after a rewrite and one more write, on a test server:

The AOF directory and its manifest

TERMINAL

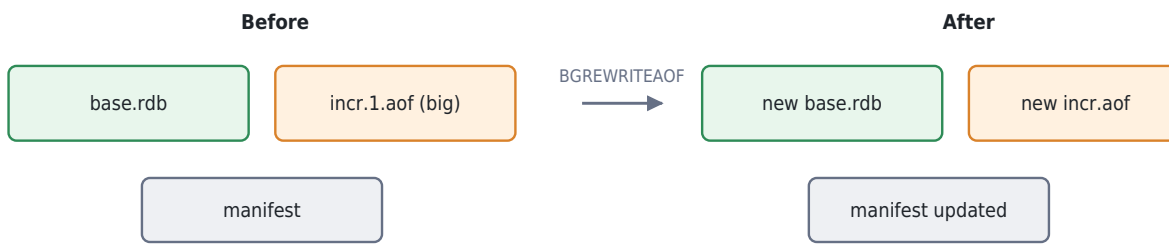
```
$ ls appendonlydir
appendonly.aof.2.base.rdb appendonly.aof.2.incr.aof appendonly.aof.manifest
$ cat appendonlydir/appendonly.aof.manifest
file appendonly.aof.2.base.rdb seq 2 type b
file appendonly.aof.2.incr.aof seq 2 type i
$ head -8 appendonlydir/appendonly.aof.2.incr.aof
*2
$6
SELECT
$1
0
*2
$4
incr
```

The incremental file shows the RESP format: `*2` means an array of two items, `$4` a 4-byte string. It is readable, and it is why people can repair it by hand when needed. A `SELECT 0` line appears because the AOF records which database the following commands apply to.

At startup Redis reads the manifest, loads the base (an RDB load, fast), then replays the incremental files in order.

4.6.3 Rewrite: keeping the file small

The log only grows. `INCR counter` a million times creates a million lines, though the state is one number. **Rewriting** produces a new, minimal base from the current dataset and starts a fresh incremental file.



old files are deleted once the new base is complete; writes during the rewrite go to the new incr file

Figure 4.5. A rewrite forks a child that writes a new base file from memory, while new writes go to a new incremental file; when the child finishes, the manifest switches to the new files and old ones are deleted. Notice that clients are never blocked by the rewrite itself.

Step by step in 7.0 and later:

1. Redis opens a **new incremental file** so that new writes go there from now on, and forks a child (**BGREWRITEAOF**, or automatic).
2. The child writes a **new base file** with the dataset as of the fork. With the RDB preamble this is a standard RDB dump.
3. When the child finishes, the parent updates the manifest atomically: new base plus the new incremental file(s), and marks the old files for deletion.

Automatic rewrites trigger when the AOF has grown by **auto-aof-rewrite-percentage** (100 means it doubled) since the last rewrite and is at least **auto-aof-rewrite-min-size** (64 MB). The cost of a rewrite is the same as **BGSAVE**: a fork, copy-on-write memory, and disk I/O. **aof_rewrite_in_progress** and **aof_last_bgrewrite_status** in **INFO persistence** tell you the state; the rewrite will not start while a **BGSAVE** is running (the request is deferred).

Directives that influence the disk behaviour:

Directive	Default	Purpose
no-appendfsync-on-rewrite	no	If yes, do not fsync the AOF while a rewrite or save is running. Avoids stalls but widens the loss window during that time
aof-rewrite-incremental-fsync	yes	fsync the new file every 4 MB as it is written
aof-timestamp-enabled	no	Writes timestamp annotations in the AOF; helps point-in-time recovery tooling
aof-load-truncated	yes	If the last command is cut short, load what is valid and start
aof-use-rdb-preamble	yes	Base file in RDB format (smaller, faster load)

AOF tuning.

4.6.4 Experiment: kill -9 and recover

Seeing the AOF work is more convincing than reading about it. Start a server with the AOF on, decrement a flash-sale stock counter three times, wait a moment, kill the process without any chance to clean up, and start it again from the same directory.

```

A crash does not lose acknowledged, fsynced writes
$ redis-server --appendonly yes --appendfsync everysec --dir /tmp/aofdemo --daemonize yes
$ redis-cli SET stock:sku42 100
OK
$ redis-cli DECR stock:sku42
(integer) 99
$ redis-cli DECR stock:sku42
(integer) 98
$ redis-cli DECR stock:sku42
(integer) 97
$ sleep 2; pkill -9 redis-server
$ redis-server --appendonly yes --dir /tmp/aofdemo --daemonize yes
$ redis-cli GET stock:sku42
"97"

```

With RDB only and default save rules (nothing triggered by four writes), the same experiment returns (**nil**), because no snapshot had been taken. If you skip the **sleep 2** the result is still very likely 97, since a killed process leaves bytes

already handed to the kernel; to really lose the last second you must cut power to the machine. That distinction between a process crash and a machine crash is the most common source of confusion about **everysec**.

4.6.5 What fsync guarantees (and does not)

fsync asks the operating system to flush a file's data to the storage device and wait. Three caveats keep your expectations realistic.

- Some disks and virtual disks acknowledge a flush while the data still sits in a volatile write cache. The guarantee is only as good as the storage stack; cloud block storage and battery-backed controllers are usually fine, cheap consumer drives may not be.
- **fsync** of the AOF does not make a *replica* consistent. Writes acknowledged by the primary's disk may never have reached any replica when the primary dies.
- Durability and consistency are separate: the AOF can only replay what was logged. A command that failed halfway through (for instance a **MULTI** block that contains an error that appears at runtime) is logged as it executed, which is how Redis transactions behave (see the transactions chapter).

4.6.6 Corruption and repair

The most common problem is a **truncated tail**: the machine lost power while a command was half written. With **aof-load-truncated yes** Redis starts anyway, discards the incomplete last command, and logs what it did:

Redis log when it loads a torn AOF (aof-load-truncated yes)

TERMINAL

```
# !!! Warning: short read while loading the AOF file appendonly.aof.2.incr.aof!!!
# !!! Truncating the AOF appendonly.aof.2.incr.aof at offset 44 !!!
# AOF appendonly.aof.2.incr.aof loaded anyway because aof-load-truncated is enabled
```

Corruption in the middle of the file is different: Redis refuses to start, because loading the rest would produce a dataset with a hole. Use the checker, first without changing anything:

Check and fix a damaged AOF

TERMINAL

```
$ redis-check-aof appendonlydir/appendonly.aof.manifest
Start checking Multi Part AOF
Start to check BASE AOF (RDB format).
RDB preamble is OK, proceeding with AOF tail...
BASE AOF appendonly.aof.2.base.rdb is valid
Start to check INCR files.
0x 44: Expected to read 7 bytes, got 3 bytes
AOF analyzed: filename=appendonly.aof.2.incr.aof, size=71, ok_up_to=44, ok_up_to_line=17, diff=27
AOF appendonly.aof.2.incr.aof is not valid. Use the --fix option to try fixing it.
$ redis-check-aof --fix appendonlydir/appendonly.aof.manifest
...
This will shrink the AOF appendonly.aof.2.incr.aof from 71 bytes, with 27 bytes, to 44 bytes
Continue? [y/N]: y
Successfully truncated AOF appendonly.aof.2.incr.aof
```

(Output condensed; the lines are from a real run on 7.0 except the final confirmation.) The tool reports the offset of the first bad byte (**ok_up_to**) and **--fix** truncates the file there. Read the **diff** number: if it is 27 bytes at the end, you lose one partial command; if it says megabytes, the damage is in the middle and **--fix** discards everything after it. Copy the directory first, always. Related directive: **aof-load-truncated no** makes Redis refuse to start on any truncated tail, if you prefer a human to decide.

WATCH OUT

Check the disk before trusting **everysec**. A full disk makes AOF writes fail. Redis reacts by rejecting writes (**aof_last_write_status:err**) rather than continuing with an incomplete journal. Alert on **aof_last_write_status** and on free disk space, and keep room for a rewrite, which temporarily needs space for the old and new files.

INTERVIEW ANGLE

What is an AOF rewrite and why is it needed? The AOF is an append-only journal that grows with every write, including overwrites and deletes of the same keys. A rewrite builds a compact new base from the current in-memory state in a forked child (as an RDB preamble), while new writes go to a fresh incremental file, then swaps the manifest. This shrinks the file, speeds up restart, and happens in the background. Since 7.0 the AOF is a base file, incremental files and a manifest.

KEY POINTS

- The AOF logs every write command; `appendfsync` decides the loss window: `always` $\approx$ none, `everysec` $\approx$ 1 second, `no` $\approx$ OS-defined.
- It is off by default (`appendonly no`); enable it for data you cannot lose.
- Since 7.0 it is a directory: base file, incremental files and `appendonly.aof.manifest`.
- Rewrites (`BGREWRITEAOF` or `auto-aof-rewrite-*`) shrink it using a fork, like `BGSAVE`.
- A torn last command is handled by `aof-load-truncated`; use `redis-check-aof` (copy first, then `--fix`) for real corruption.

4.7 Choosing a Persistence and Durability Strategy

Persistence settings are a business decision disguised as configuration. Each option moves a point on three scales: how much data you can lose, how fast you recover, and how much performance and operational care you pay. This chapter turns the mechanisms into a decision.

4.7.1 The four configurations

Strategy	Settings	Worst-case data loss	Notes
No persistence	<code>save ""</code> and <code>appendonly no</code>	Everything on restart	Pure cache; restart is instant but cold; fastest and simplest
RDB only	default save rules, <code>appendonly no</code>	Minutes (since last snapshot)	Good for rebuildable data; small files, easy backups
AOF everysec	<code>appendonly yes</code> , <code>appendfsync everysec</code>	About 1 second	The usual choice when Redis holds data of value
AOF always	<code>appendfsync always</code>	Near zero (acknowledged writes)	Expect much lower write throughput; test on your disk
RDB + AOF	both enabled	AOF's window	AOF is used on restart; RDB is a convenient backup format

Persistence strategies compared.

Remember how the restart choice works: if the AOF is enabled and exists, Redis loads **the AOF**, not the RDB, since the AOF is normally more complete. Having both is recommended by the project when you want strong durability, because periodic RDB files give you simple, separate point-in-time backups, while the AOF provides the small loss window.

4.7.2 Matching strategy to data: ShopStream

You do not have to choose one strategy for the whole system. ShopStream applies different rules to different roles, which in practice means separate Redis deployments (or at least separate instances for separate durability needs):

Data	Strategy	Reasoning
Catalogue cache (product:id)	None, or RDB on a replica	Rebuilt from the database; a cold start costs load, not data
Rate-limit counters	None	Short-lived and low value; resetting is harmless
Sessions and carts	AOF everysec	Losing a second of cart edits is acceptable; losing everything is not
Order stream (stream:orders)	AOF everysec, RDB backups, replica, possibly WAIT	These events feed notifications and downstream services
Idempotency keys, locks	AOF everysec or accept loss with TTL logic	Locks expire anyway; lost idempotency keys risk duplicates, so persist them
Source of truth for money or inventory	Not Redis alone	Keep the system of record in the database; Redis is the fast path

ShopStream persistence plan.

NOTE

Persistence is not replication, and neither is a backup. Persistence protects against *process or machine restart on the same disk*. Replication protects against *losing the node*, but replicates mistakes (a bad **FLUSHALL**) instantly. Backups protect against mistakes and disasters. A serious deployment uses all three. Replication is asynchronous, so acknowledged writes can be lost on failover even with **appendfsync always** on the old primary; the replication chapters cover this and **WAIT**.

4.7.3 Diskless replication and persistence-free replicas

When a replica needs a full sync the primary produces an RDB. By default (**repl-diskless-sync yes**) it streams the RDB directly to the replica sockets from the child process instead of writing a file first, which avoids disk I/O on the primary. A common design is to run the primary with persistence and let the replica persist too, or the reverse: persistence only on replicas so the primary never forks for backups. The risk of the second design is a primary restarting empty and replicating emptiness back to its replicas, so it needs failover automation that prevents the restarted primary from coming back as primary. For beginners, the safe default is to persist on the primary.

4.7.4 Backups and restores

A backup you have never restored is a hope. Rules that work:

1. **Take consistent files.** An RDB file is a complete snapshot, so copy it with **cp** or **rsync** after a **BGSAVE** finishes (check **ldb_bgsave_in_progress:0**). A running **BGSAVE** replaces the file atomically, so a copy of **dump.rdb** is always complete. For the AOF directory, copy all files together while no rewrite is running (**aof_rewrite_in_progress:0**); the Redis docs suggest temporarily disabling automatic rewrites with **CONFIG SET auto-aof-rewrite-percentage 0** during the copy, then restoring the value.
2. **Keep several generations:** hourly for a day, daily for a month, for example, as the persistence documentation proposes with hourly and daily directories.
3. **Move copies off the machine** (another host, another region, object storage), at least once a day.
4. **Verify:** compare size and checksum after transfer, run **redis-check-rdb** or **redis-check-aof**, and on a schedule restore into a scratch instance and read some keys.
5. **Remote snapshot:** **redis-cli --rdb backup.rdb** fetches an RDB from a running server over the network without touching its disk layout.
6. **Document the restore:** stop Redis, remove the old data files, place the backup in **dir**, start, check **DBSIZE** and a few known keys. If the backup is an RDB but AOF is enabled in config, start with **appendonly no**, then enable the AOF through **CONFIG SET appendonly yes** after loading; otherwise Redis will find no AOF and start empty.

GOTCHA

The restore that comes back empty. You copy a good **dump.rdb** into the data directory of a server with **appendonly yes**, start Redis, and **DBSIZE** says 0. The server created an empty AOF at first start (or already had one) and loads it in preference to the RDB. Restore the RDB with AOF disabled, verify the data, then turn on the AOF so it generates a base from the loaded dataset.

4.7.5 Putting numbers on it

A useful exercise is to write the recovery requirements as two numbers: **RPO** (recovery point objective, how much data you may lose, in time) and **RTO** (recovery time objective, how long you may be down). Then map them:

- RPO of a day, RTO of minutes: RDB only, cache semantics, cold-start protection in the application.
- RPO of seconds: AOF **everysec** plus a replica for failover.
- RPO of zero for acknowledged writes: Redis is probably not the right place for that data unless you accept the performance cost of **appendfsync always**, plus synchronous replication via **WAIT**, which narrows but does not eliminate the window. Keep the system of record in a database designed for it.
- Restart time (RTO) scales with dataset size: loading roughly takes seconds per few GB for an RDB base, and longer if there is a large incremental tail, so rewrite regularly. Measure on your hardware with a restore drill.

REAL WORLD

The cache that was also the primary store. A frequent incident pattern: a team starts with Redis as a cache, gradually stores sessions, then job queues in the same instance, with no persistence and an `allkeys-lru` policy. A restart or an eviction round then loses user sessions and pending jobs. The fix is organisational, not clever: classify data by loss tolerance, give each class its own instance and policy, and write the persistence strategy next to the key schema.

INTERVIEW ANGLE

How would you configure Redis so you lose at most about a second of data, and what can still go wrong? Enable the AOF with `appendfsync everysec` (optionally with RDB for backups). A crash then loses roughly one second of writes. Things that can still go wrong: the disk stalls so the delay approaches two seconds; replication is asynchronous so a failover can lose recent writes that never reached the replica (mitigate with `min-replicas-to-write` and `WAIT`); a bug or operator mistake (`FLUSHALL`) is persisted too, so you also need point-in-time backups; and a full disk stops writes. For zero loss, use `appendfsync always` and accept the throughput cost, or keep the data in a database whose guarantees match.

KEY POINTS

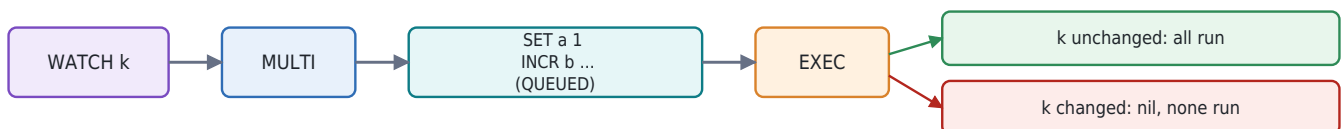
- Choose by loss tolerance and recovery time: none (cache), RDB only (minutes), AOF everysec (about 1 second), AOF always (near zero, slow).
- Restart loads the AOF when it is enabled; to restore an RDB into an AOF-enabled server, load with AOF off first.
- Persistence, replication and backups solve different problems; production needs all three.
- Backups must be complete, off-machine, versioned and test-restored; check them with `redis-check-rdb` and `redis-check-aof`.
- Give different data classes (cache, sessions, order stream) different instances and policies.

4.8 Transactions: MULTI, EXEC and WATCH

Sooner or later an application needs to change two or more keys as one unit. A customer checks out: the cart must be emptied and the order counter must go up. A seat is reserved: the seat key is set and the seat count is decremented. If another client can see the state half-way through, or if one of the commands is lost while the others succeed, the data is wrong. Relational databases solve this with transactions. Redis has transactions too, but they are much simpler than the SQL kind, and the differences are exactly what interviewers and production incidents are made of. This chapter explains what a Redis transaction guarantees, what it does not, and how `WATCH` adds optimistic locking on top.

4.8.1 The core idea: a queued batch executed without interruption

A Redis transaction is a **queue of commands that the server runs back to back, with no other client's command in between**. You open the queue with `MULTI`, send the commands (the server answers `QUEUED` to each instead of running it), and run the whole queue with `EXEC`. Because Redis executes commands on a single main thread (see the chapter on the server architecture), nothing else can interleave while `EXEC` is running its queue. That property is called **isolation**, and Redis gets it for free.



commands in EXEC run one after another with no other client in between; there is no rollback

Figure 4.6. A transaction: commands are queued after `MULTI` and run as one uninterrupted block by `EXEC`. Note that other clients are served before or after the block, never inside it.

A simple transaction

TERMINAL

```
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379> HSET cart:1001 sku:77 2
QUEUED
127.0.0.1:6379> INCR stats:carts:updated
QUEUED
127.0.0.1:6379> EXPIRE cart:1001 86400
QUEUED
127.0.0.1:6379> EXEC
1) (integer) 1
2) (integer) 1
3) (integer) 1
```

EXEC returns an array with one reply per queued command, in order. **DISCARD** throws the queue away and leaves the connection in normal mode. If the connection drops before **EXEC**, the queue is simply forgotten.

DEFINITION

A Redis transaction is a sequence of commands queued with **MULTI** and executed atomically in order by **EXEC**: no other client command runs in between. There is no rollback. Commands that fail at execution time do not undo the commands before them.

4.8.2 Two kinds of errors, two behaviours

This is the most important part of the chapter. A command in a transaction can fail at two different moments, and Redis treats them differently.

Errors while queueing. If a command is syntactically wrong (unknown command, wrong number of arguments), the server refuses it right away with an error. When you later call **EXEC**, Redis refuses to run anything and discards the whole transaction.

Error at queue time: nothing runs

TERMINAL

```
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379> SET order:9:status "PAID"
QUEUED
127.0.0.1:6379> FOOBAR
(error) ERR unknown command 'FOOBAR', with args beginning with:
127.0.0.1:6379> EXEC
(error) EXECABORT Transaction discarded because of previous errors.
127.0.0.1:6379> GET order:9:status
(nil)
```

Errors at execution time. If the command is well formed but fails when it runs, for example **LPUSH** on a key holding a string, then that one command returns an error and **all the others still run**. Redis does not roll back.

Error at execution time: the rest still runs

TERMINAL

```
127.0.0.1:6379> SET a 1
OK
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379> INCR a
QUEUED
127.0.0.1:6379> LPUSH a x
QUEUED
127.0.0.1:6379> GET a
QUEUED
127.0.0.1:6379> EXEC
1) (integer) 2
2) (error) WRONGTYPE Operation against a key holding the wrong kind of value
3) "2"
```

Why no rollback? The Redis authors argue that execution-time errors are programming errors (wrong type, bad argument) that should be caught in development, not at runtime in production, and that skipping rollback keeps Redis simple and fast. You may or may not agree, but you must design for it: a Redis transaction gives you **isolation and ordered execution**, not the "all or nothing" part of ACID.

GOTCHA

"Atomic" in Redis means "no other client's command interleaves", not "all commands succeed or none do". If command 2 of 3 fails at execution time, commands 1 and 3 have already taken effect and stay. Do not rely on **MULTI** to undo partial work. If you need a conditional, all-or-nothing update that depends on the data, use a Lua script or a function (next chapters).

There is one more case worth knowing. If the server crashes or is killed in the middle of **EXEC**, the persistence layer may hold only part of the transaction. With AOF, a half-written transaction is detected at restart and the server refuses to load until you repair the file with **redis-check-aof**. This is rare, but it is the reason the word "atomic" needs the footnote above.

4.8.3 No reads inside the decision: the problem **WATCH** solves

A transaction queues commands; it does not give you their results until **EXEC**. That means you cannot read a value, decide something in your application, and write it back inside the same transaction. Consider decrementing a stock counter only if it is positive:

GET stock:sku-77	→ 1	(client A reads)
GET stock:sku-77	→ 1	(client B reads)
DECR stock:sku-77	→ 0	(A buys the last unit)
DECR stock:sku-77	→ -1	(B buys it too: oversold)

The read and the write are separate steps and another client can slip in between. **MULTI** alone cannot help, because the **GET** result is not available until after **EXEC**. This is a classic **check-then-act race condition**.

WATCH solves it with **optimistic locking**. You tell Redis which keys you care about, then read them normally, decide, and then run a transaction. If any watched key was modified by anyone (including expiry or eviction) between your **WATCH** and your **EXEC**, the **EXEC** does nothing and returns a null reply. You then retry the whole read-decide-write cycle.

WATCH: a successful attempt

TERMINAL

```
127.0.0.1:6379> SET stock:sku-77 1
OK
127.0.0.1:6379> WATCH stock:sku-77
OK
127.0.0.1:6379> GET stock:sku-77
"1"
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379> DECR stock:sku-77
QUEUED
127.0.0.1:6379> EXEC
1) (integer) 0
```

If a second client had run **DECR stock:sku-77** after the **WATCH**, our **EXEC** would have printed **(nil)** and executed nothing. The code that uses this is always a loop:

```
loop (max N times):
    WATCH key
    value = GET key           # plain read, outside MULTI
    if value is not good enough: UNWATCH; give up
    MULTI
    write commands based on value
    result = EXEC
    if result is not null: done
    # otherwise somebody changed the key: try again
```

It is called optimistic because it does not block anyone: it assumes conflicts are rare and pays only when one happens. Under high contention on one key (a flash sale for a single SKU), many attempts fail and retry, which wastes round trips. In that situation a single Lua script (the next chapter) is far better.

Rules of **WATCH**

- **WATCH** must be sent before **MULTI**. Calling it inside **MULTI** is an error.
- **EXEC** and **DISCARD** forget all watched keys, and so does **UNWATCH**. **WATCH** is per connection, so it needs a connection that is not shared between threads at that moment.

- A watched key counts as modified when it is written, deleted, expires, or is evicted. Since Redis 6.0.9, a key that was already expired when **WATCH** ran does not trigger a false abort later. A write that does not change the value (for example **SET** with the same value) still counts as a touch.
- Watching a key in one logical database and switching with **SELECT** is not a safe pattern; use one database.

WATCH OUT

In Redis Cluster all keys of a transaction (and all watched keys) must live in the same hash slot, or you get a **CROSSSLOT** error. Use hash tags such as **cart:{1001}:items** and **cart:{1001}:meta** so related keys land in the same slot. The chapter on Redis Cluster in the scaling part explains slots in detail.

4.8.4 Transactions versus pipelines

Both **MULTI/EXEC** and pipelining send several commands in one network trip, so people confuse them. They solve different problems:

Aspect	MULTI / EXEC	Pipeline
Purpose	Isolation: nobody interleaves	Fewer round trips (speed)
Interleaving with other clients	Not possible inside EXEC	Possible between commands
Failure of one command	Others still run (no rollback)	Others still run
Needs server support	Yes (MULTI, EXEC)	No, purely a client technique
Can be combined	Yes: send MULTI ... EXEC inside a pipeline	Yes

Transaction versus pipeline.

Most client libraries pipeline the whole **MULTI ... EXEC** block automatically, so you pay one round trip. You will see how in the Java client chapters and in the Spring chapter on transactions and scripts.

INTERVIEW ANGLE

Does Redis support transactions? Are they ACID? Redis supports transactions through **MULTI**, **EXEC**, **DISCARD** and **WATCH**. Commands in a transaction run sequentially and in isolation, because the command loop is single threaded, so no other client interleaves. But there is no rollback: a command that fails at execution time does not undo the others, so it is not atomic in the "all or nothing" sense. **WATCH** adds optimistic check-and-set. Durability depends on the persistence settings. For conditional logic that must be atomic, use a Lua script or a function.

KEY POINTS

- **MULTI** queues commands and **EXEC** runs them back to back; nothing from other clients runs in between.
- Syntax errors found while queueing abort the whole transaction; runtime errors (such as **WRONGTYPE**) do not roll anything back.
- You cannot make decisions on read results inside **MULTI**; use **WATCH** for optimistic check-and-set and retry on a null **EXEC** reply.
- Under contention on a single key, prefer a Lua script or function to a **WATCH** retry loop.
- Transactions are not pipelines: one gives isolation, the other saves round trips. In Cluster, keep all keys in one slot.

4.9 Lua Scripting with EVAL

Transactions cannot branch on data. Pipelines cannot either. Many real operations need exactly that: "decrement the stock only if there is enough, and tell me what happened", "release the lock only if I still own it", "refill a token bucket based on elapsed time". Doing these from the client means several round trips and a race condition between them.

Lua scripting lets you ship a small program to the server and run it there, as one indivisible step. It is the main tool for building correct compound operations in Redis, and it appears throughout the patterns and projects parts of this book.

4.9.1 What a script is and how it runs

Redis embeds the Lua 5.1 interpreter. `EVAL` sends the script text, the number of key arguments, the key names, then extra arguments:

```
REDIS
EVAL script numkeys [key [key ...]] [arg [arg ...]]
```

```

Your first script                                TERMINAL
127.0.0.1:6379> SET stock:sku-77 10
OK
127.0.0.1:6379> EVAL "return redis.call('GET', KEYS[1])" 1 stock:sku-77
"10"
127.0.0.1:6379> EVAL "return ARGV[1] .. '-' .. ARGV[2]" 0 hello world
"hello-world"
```

Inside the script, the global tables `KEYS` and `ARGV` hold the key names and the other arguments (Lua arrays start at 1, and every element is a string). The function `redis.call(command, ...)` runs any Redis command and returns its reply converted to a Lua value.

The crucial property: **while a script runs, the server executes nothing else**. It behaves like a transaction that can contain `if` statements and loops. Everything the script reads is guaranteed not to change under its feet, and its writes become visible together.

4.9.2 KEYS versus ARGV

Pass every key the script touches in `KEYS`, and everything else (amounts, tokens, timestamps, TTLs) in `ARGV`. This is not decoration:

- Redis Cluster uses the declared keys to route the call to the right shard and to check that all keys live in one slot.
- Tools, ACLs and replicas use the declared keys to understand what the script accesses.
- It keeps scripts reusable: the same script text works for any key name.

Scripts that build key names dynamically inside the code (for example `redis.call('GET', 'user:' .. ARGV[1])`) may appear to work on a single node and then fail in Cluster. Declare keys properly from day one.

4.9.3 redis.call versus redis.pcall, and return values

If a command run with `redis.call` fails, the error is raised and the script aborts with that error. `redis.pcall` ("protected call") catches the error and returns it as a table with an `err` field, so the script can handle it.

```

call aborts, pcall returns the error                                TERMINAL
127.0.0.1:6379> SET sku "x"
OK
127.0.0.1:6379> EVAL "return redis.call('LPUSH', KEYS[1], 'a')" 1 sku
(error) WRONGTYPE Operation against a key holding the wrong kind of value script: ...
127.0.0.1:6379> EVAL "return redis.pcall('LPUSH', KEYS[1], 'a')" 1 sku
(error) WRONGTYPE Operation against a key holding the wrong kind of value
```

Writes made before an error are **not rolled back**, just as in `MULTI`. Order your script so that checks come first and writes last.

Return values are converted between Lua and the Redis protocol. The table to remember:

Lua value	Redis reply
number (3.7)	integer, fraction cut off (3)
string	bulk string
true	integer 1
false or nil	nil reply
table (array)	array reply; stops at the first nil
table {ok='X'}	status reply
table {err='X'}	error reply

How Lua values become Redis replies.

GOTCHA

Returning a Lua number truncates it to an integer, and `false` becomes a nil reply, not 0. If you need a decimal, return it as a string with `tostring(x)`. If you want to return "false" to the client, return the integer 0 instead. Converting the other way, a Redis integer reply becomes a Lua number, a bulk string becomes a Lua string and a nil bulk reply becomes Lua `false`, so test with `if not value then`.

4.9.4 Script cache: EVALSHA and SCRIPT LOAD

Sending the script text on every call wastes bandwidth. When Redis runs a script, it caches it under its **SHA1 hash**. **SCRIPT LOAD** caches without running and returns the hash; **EVALSHA** runs by hash:

Load once, call by hash

TERMINAL

```
127.0.0.1:6379> SCRIPT LOAD "return redis.call('GET', KEYS[1])"
"d3c21d0c2b9ca22f82737626a27bcaf5d288f99f"
127.0.0.1:6379> EVALSHA d3c21d0c2b9ca22f82737626a27bcaf5d288f99f 1 stock:sku-77
"10"
127.0.0.1:6379> EVALSHA ffffffffffffffffffffffffffffffffffffffffff 0
(error) NOSCRIPT No matching script. Please use EVAL.
```

The cache lives only in memory. It is **not** persisted by RDB, and a restart, a failover to a replica that never ran the script, or **SCRIPT FLUSH** empties it. Client libraries therefore follow the pattern: try **EVALSHA**; on **NOSCRIPT**, send **EVAL** (which also repopulates the cache). Spring's `DefaultRedisScript` and Redisson do this for you. Do not generate script text dynamically with values pasted in; each variant would be cached separately and fill memory. Always pass variable data through **KEYS** and **ARGV**.

4.9.5 Determinism and replication

Older Redis versions replicated scripts by re-running the script text on replicas and in the AOF, so scripts had to be deterministic (same input, same result) and **TIME** or **SRANDMEMBER** before a write was forbidden. Since Redis 5 replication ships by default (and 7.0 removed the old mode) the script's **effects**, that is, the write commands it actually performed. Consequently you can use `redis.call('TIME')` and random commands freely in modern versions. The rule that remains: the script must not depend on the clock of the machine or on global state outside Redis in ways you want replicated, because only the commands it issued are copied.

4.9.6 Limits you must respect

- **A script blocks the server.** Because nothing else runs, a slow script is an outage. The setting `busy-reply-threshold` (older name `lua-time-limit`, default 5000 ms) is not a limit that kills the script: it is the time after which Redis starts replying **BUSY** to other clients and begins to accept **SCRIPT KILL** (for scripts that have not written yet) or **SHUTDOWN NOSAVE**.
- **Keep scripts short.** A handful of commands and loops bounded by a small known number. Never loop over an unbounded collection.
- **No blocking commands.** **BLPOP** and friends behave as their non-blocking counterparts inside scripts.
- **Cluster key rule.** All keys must hash to one slot, and must be declared in **KEYS**.
- **Global variables are forbidden.** Declare everything with `local`. Redis protects its global table and raises an error on accidental globals.
- **Scripts are not a place for business logic.** They are for small atomic read-modify-write operations. Everything else belongs in the application.

A runaway script and how to stop it

TERMINAL

```
127.0.0.1:6379> EVAL "while true do end" 0
# (blocks; from another client, after the threshold)
127.0.0.1:6379> GET stock:sku-77
(error) BUSY Redis is busy running a script. You can only call SCRIPT KILL or SHUTDOWN NOSAVE.
127.0.0.1:6379> SCRIPT KILL
OK
```

WATCH OUT

If a script has already performed a write, **SCRIPT KILL** is refused (to protect atomicity) and your only escape is **SHUTDOWN NOSAVE**, which loses unsaved data. Test every script against realistic data sizes before it goes live.

4.9.7 Pattern 1: compare-and-delete (safe lock release)

A distributed lock is stored as `lock:order:5001` with a random token as its value. To release it, you must delete the key only if it still holds your token; otherwise you might delete a lock that expired and was acquired by someone else. `GET` followed by `DEL` from the client is racy, so do it in Lua. This is the textbook script:

release_lock.lua

LUA

```
-- KEYS[1] = lock key, ARGV[1] = owner token
if redis.call('GET', KEYS[1]) == ARGV[1] then
    return redis.call('DEL', KEYS[1])
else
    return 0
end
```

Run it

TERMINAL

```
127.0.0.1:6379> SET lock:order:5001 "token-abc" NX PX 30000
OK
$ redis-cli --eval release_lock.lua lock:order:5001 , token-xyz
(integer) 0
$ redis-cli --eval release_lock.lua lock:order:5001 , token-abc
(integer) 1
```

NOTE

Redis 8.4 added `DELEX` and the `IFEQ` option of `SET`, which implement compare-and-delete and compare-and-set natively. With them the script above can become a single `DELEX lock:order:5001 IFEQ token-abc`. Check the command pages for your exact version before depending on them, and keep the Lua version if you must support older servers. The chapter on distributed locks in the Patterns part builds on this.

4.9.8 Pattern 2: atomic stock decrement for a flash sale

The ShopStream flash sale must never oversell. The script checks the stock and decrements it in one step, returning the new stock or -1 when there is not enough:

reserve_stock.lua

LUA

```
-- KEYS[1] = stock counter (stock:{sku}), ARGV[1] = quantity wanted
local stock = tonumber(redis.call('GET', KEYS[1]))
local want = tonumber(ARGV[1])
if stock == nil then
    return -2          -- key missing: unknown SKU
end
if stock < want then
    return -1          -- not enough stock, nothing changed
end
return redis.call('DECRBY', KEYS[1], want)
```

What to notice: the read, the comparison and the write are one atomic step, so two buyers can never both take the last unit. The distinct return codes (-2, -1, remaining count) let the caller tell "unknown product" from "sold out" without a second round trip. The full flash-sale project later in the book extends this with reservations and a TTL.

4.9.9 Pattern 3: token bucket rate limiter

A token bucket allows bursts up to a capacity and refills at a steady rate. State (current tokens and last refill time) lives in one hash. The script computes the refill from the time elapsed, so no background process is needed:

token_bucket.lua

LUA

```
-- KEYS[1] = bucket hash, e.g. rl:{ip}:{route}
-- ARGV[1] = capacity, ARGV[2] = refill tokens per second, ARGV[3] = tokens requested
local capacity = tonumber(ARGV[1])
local rate = tonumber(ARGV[2])
local requested = tonumber(ARGV[3])

local t = redis.call('TIME')
local now = t[1] * 1000 + math.floor(t[2] / 1000) -- milliseconds

local data = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(data[1])
local ts = tonumber(data[2])
```

token_bucket.lua (continued)

LUA

```

if tokens == nil then
    tokens = capacity
    ts = now
end

local elapsed = math.max(0, now - ts)
tokens = math.min(capacity, tokens + elapsed * rate / 1000)

local allowed = 0
if tokens >= requested then
    tokens = tokens - requested
    allowed = 1
end

redis.call('HSET', KEYS[1], 'tokens', tostring(tokens), 'ts', now)
redis.call('PEXPIRE', KEYS[1], math.ceil(capacity / rate * 1000) * 2)
return {allowed, math.floor(tokens)}

```

What to notice: **TIME** is read inside the script, so all application servers share one clock (the Redis server's) and clock skew between them does not matter. The **PEXPIRE** makes idle buckets disappear. The token count is stored with **tostring** so that the fractional part is kept as text. The rate limiting chapter in the Patterns part compares this with fixed and sliding windows.

4.9.10 Calling scripts and debugging

From the shell, keep the script in a file and let **redis-cli** read it. Note the comma that separates keys from arguments in this mode:

Run a script file with redis-cli

TERMINAL

```

$ redis-cli --eval token_bucket.lua rl:10.0.0.7:checkout , 20 5 1
1) (integer) 1
2) (integer) 19

```

For stepping through a script, **redis-cli --ldb --eval file.lua keys , args** starts the Redis Lua debugger; use it on a development instance only. Inside scripts, **redis.log(redis.LOG_WARNING, "message")** writes to the server log, and **redis.shalhex(str)** is available. Calling **redis.setresp(3)** switches the reply format that **redis.call** returns, which matters only for RESP3 maps and sets.

INTERVIEW ANGLE

Why use a Lua script instead of MULTI/EXEC with WATCH? A script runs the whole read-decide-write sequence atomically on the server in a single round trip and never retries. **WATCH** needs at least two round trips and retries under contention, and each retry repeats the read. Scripts pay with server time: a long script blocks every other client, so keep them tiny and bounded.

KEY POINTS

- A script runs atomically: no other command executes while it runs, so it can read, decide and write safely.
- Declare every key in **KEYS**, pass everything else in **ARGV**; this is required for Cluster correctness.
- **redis.call** raises errors, **redis.pcall** returns them; there is no rollback of earlier writes.
- Call scripts with **EVALSHA** and fall back to **EVAL** on **NOSCRIPT**; the script cache is lost on restart.
- Keep scripts short and bounded, because a slow script blocks the whole server (**BUSY**).
- Classic uses: compare-and-delete lock release, atomic stock decrement, token bucket.

4.10 Redis Functions

EVAL has an operational weakness: the scripts live in your application code, and each application instance has to send them (or load them into the script cache) by itself. Redis 7.0 introduced **Functions** to treat server-side code as part of the database. A function is stored in the server as a named, versioned-by-you piece of code, persisted and replicated with the data, and called by name. Same Lua engine, same atomic execution, different packaging.

4.10.1 Libraries and registered functions

Functions are grouped in **libraries**. You write a library as a Lua source with a special first line (the shebang) that names the engine and the library, and inside it you register one or more functions:

```
shop.lua LUA

#!/lua name=shop

local function reserve_stock(keys, args)
    local stock = tonumber(redis.call('GET', keys[1]))
    local want = tonumber(args[1])
    if stock == nil then
        return -2
    end
    if stock < want then
        return -1
    end
    return redis.call('DECRBY', keys[1], want)
end

local function stock_level(keys, args)
    return redis.call('GET', keys[1])
end

redis.register_function('reserve_stock', reserve_stock)
redis.register_function{
    function_name = 'stock_level',
    callback = stock_level,
    flags = { 'no-writes' }
}
```

What to notice: a function receives two Lua tables, **keys** and **args**, instead of the globals **KEYS** and **ARGV**. A library must register at least one function, or loading fails. The second registration uses the table form so it can attach **flags**; **no-writes** declares that the function never writes, which allows calling it with **FCALL_R0** and on read-only replicas.

4.10.2 Loading and calling

```
Load a library and call its functions TERMINAL

$ redis-cli -x FUNCTION LOAD REPLACE < shop.lua
"shop"
127.0.0.1:6379> SET stock:sku-77 10
OK
127.0.0.1:6379> FCALL reserve_stock 1 stock:sku-77 3
(integer) 7
127.0.0.1:6379> FCALL reserve_stock 1 stock:sku-77 30
(integer) -1
127.0.0.1:6379> FCALL_R0 stock_level 1 stock:sku-77
"7"
127.0.0.1:6379> FCALL_R0 reserve_stock 1 stock:sku-77 1
(error) ERR Can not execute a script with write flag using *_ro command.
```

FCALL function numkeys key... arg... has the same shape as **EVAL**, but takes a function name instead of source text, so there is no hash to manage and no **NOSCRIPT** fallback. **FUNCTION LOAD** fails if a library with the same name exists unless you pass **REPLACE**.

The management commands you will use:

Command	Purpose
FUNCTION LOAD [REPLACE] code	Load (or replace) a library; returns the library name
FUNCTION LIST [LIBRARYNAME pattern] [WITHCODE]	Show libraries and their functions and flags

Command	Purpose
FUNCTION DELETE library	Remove a library and its functions
FUNCTION DUMP / FUNCTION RESTORE	Serialize all libraries to a payload and load it elsewhere
FUNCTION STATS	Show the running function (if any) and engine counts
FUNCTION FLUSH	Delete all libraries
FUNCTION KILL	Stop a running function that has not yet written (like SCRIPT KILL)

Function management commands.

4.10.3 Persistence, replication and clusters

Unlike the EVAL script cache, loaded functions are **persisted in the AOF and in RDB files and replicated to replicas**. After a restart or a failover they are still there. That removes the NOSCRIPT dance. Two caveats follow:

- **Cache-only instances.** If the instance has no persistence and restarts empty, the functions are gone. Load them again at deployment time, or extract them into an RDB with `redis-cli --functions-rdb` so the server loads them at startup.
- **Redis Cluster.** Functions are replicated from a primary to its replicas, but loading them on every shard is your job. The deployment pipeline must run `FUNCTION LOAD REPLACE` against every primary node, and be able to handle a node added later.

4.10.4 EVAL or FUNCTION?

Aspect	EVAL / EVALSHA	FUNCTION / FCALL
Where code lives	In the client application	In the server, as named libraries
Deployment	Sent at runtime, cached by SHA1	Loaded explicitly during deployment
Survives restart	No (cache is volatile)	Yes (AOF, RDB, replication)
Call style	Script text or hash	Function name
Reuse of helpers	Copy-paste in every script	Shared local functions inside a library
Versioning	Script text is its own version	Library name; you manage upgrades
Best for	Small, app-owned scripts; quick use	Shared, stable logic used by several services

EVAL scripts versus Functions.

The trade-off is governance. With `EVAL` the application and its script are released together and cannot drift apart. With Functions the library is a deployment artefact that must exist in every environment before the application that calls it. For a single service, `EVAL` through a library such as Spring's `RedisScript` is perfectly fine. Teams with several services that share atomic operations (stock handling used by cart, order and inventory services) benefit from Functions. Do this carefully: a function replaced in production changes behaviour for every caller at once, so treat `FUNCTION LOAD REPLACE` like a database migration, and keep the library source in version control.

TIP

Name libraries by domain and bake a version into function names when behaviour changes incompatibly, for example `reserve_stock_v2`. Roll out the new library, switch callers, then delete the old function. That makes rollback a configuration change.

INTERVIEW ANGLE

What is the difference between EVAL and Redis Functions? Both run Lua atomically on the server. `EVAL` scripts are sent by the client and cached by hash in a volatile cache. Functions, added in 7.0, are stored server-side in named libraries, persisted in the AOF and RDB, replicated, and invoked by name with `FCALL`. They suit shared logic managed as part of the database. In Cluster you must load them on every node yourself.

KEY POINTS

- Functions are named Lua code stored in libraries inside Redis, called with **FCALL** and **FCALL_R0**.
- A library starts with a `#!lua name=...` line and registers functions with `redis.register_function`.
- Functions are persisted and replicated, so there is no script-cache loss and no **NOSCRIPT** handling.
- Mark read-only functions with the **no-writes** flag so they can run via **FCALL_R0** and on replicas.
- In Cluster, load libraries on every node; treat library changes as database migrations.

4.11 Pipelining, Blocking Commands and Latency

Redis executes most commands in microseconds, yet a real application often sees milliseconds per call. The missing time is not spent inside Redis; it is spent on the network. This chapter shows how to do the arithmetic, how to cut round trips with **pipelining**, what **blocking commands** do (and do not) block, and how to find the cause when Redis really is slow.

4.11.1 Round-trip time dominates

Every command sent in the usual request-response style costs one **round-trip time (RTT)**: the client writes the command, the packet crosses the network, the server runs it, and the reply crosses back. Within one data centre an RTT is typically a fraction of a millisecond up to about a millisecond; across availability zones or regions it is more. Compare it with the server's own work, which for **GET** or **HSET** is a few microseconds.

Suppose the RTT is 0.5 ms (an illustrative figure) and you need to load 100 product hashes for a category page:

Approach	Round trips	Approximate time
100 separate HGETALL calls	100	50 ms + server work
One pipeline of 100 HGETALL	1	0.5 ms + server work (a little more for sending and parsing)
Single MGET (strings only)	1	0.5 ms + server work

Sequential versus pipelined, illustrative numbers for an RTT of 0.5 ms.

The server time is the same in all cases; only the waiting changes. A single connection that sends one command at a time can never exceed 1 / RTT commands per second, no matter how fast Redis is. With a 0.5 ms RTT that is a ceiling of about 2,000 commands per second per connection. This is the number-one reason people think "Redis is slow" when it is not, and it is also why an N+1 query pattern against Redis hurts just as much as it does against SQL.

4.11.2 Pipelining

Pipelining means sending many commands without waiting for each reply, then reading all the replies together. The server still executes the commands one after another and queues the replies in an output buffer, but the network is used far more efficiently. It also reduces the number of **read** and **write** system calls on the server, which is why pipelined throughput can be several times higher than non-pipelined throughput even on localhost.

Without pipelining <pre> client server --- GET a ---> <-- "1" ----- --- GET b ---> <-- "2" ----- --- GET c ---> <-- "3" ----- 3 round trips </pre>	With pipelining <pre> client server --- GET a ---> --- GET b ---> --- GET c ---> <-- "1" ----- <-- "2" ----- <-- "3" ----- 1 round trip </pre>
--	--

You can try pipelining with nothing but **redis-cli**. The **--pipe** mode reads commands in the raw Redis protocol, or inline text, from standard input and sends them in bulk. It is the fastest way to load a large file of commands:

Bulk load with **redis-cli --pipe**

TERMINAL

```

$ for i in $(seq 1 100000); do echo "SET product:$i:stock 50"; done | redis-cli --pipe
All data transferred. Waiting for the last reply...
Last reply received from server.
errors: 0, replies: 100000

```

In application code you rarely write protocol by hand. Your client library does it. With Lettuce, the commands you issue on one connection through the async API are pipelined automatically; for explicit batching you turn off auto-flush, issue the commands, and flush once:

```

PipelineExample.java
package com.shopstream.demo;

import io.lettuce.core.LettuceFutures;
import io.lettuce.core.RedisFuture;
import io.lettuce.core.api.StatefulRedisConnection;
import io.lettuce.core.api.async.RedisAsyncCommands;

import java.time.Duration;
import java.util.ArrayList;
import java.util.List;

public class PipelineExample {

    public static List<String> loadStock(StatefulRedisConnection<String, String> conn,
                                         List<String> skus) {
        RedisAsyncCommands<String, String> async = conn.async();
        conn.setAutoFlushCommands(false);
        List<RedisFuture<String>> futures = new ArrayList<>();
        try {
            for (String sku : skus) {
                futures.add(async.get("stock:" + sku));
            }
            conn.flushCommands(); // one write for all commands
            LettuceFutures.awaitAll(Duration.ofSeconds(2),
                                    futures.toArray(new RedisFuture[0]));
        } finally {
            conn.setAutoFlushCommands(true);
        }
        List<String> result = new ArrayList<>();
        for (RedisFuture<String> f : futures) {
            try {
                result.add(f.get());
            } catch (Exception e) {
                throw new IllegalStateException(e);
            }
        }
        return result;
    }
}

```

What to notice: the connection is shared, so switching off auto-flush affects every thread using it. That is acceptable in a short-lived, dedicated connection but dangerous on a shared one; the Lettuce chapter in the Java part discusses this trade-off, and Spring's `executePipelined` (see the chapter on transactions and scripts in Spring) hides it behind a safer API.

How big should a pipeline be?

Do not pipeline a million commands in one go. The server must hold all the replies in memory before it can send them, and the client must buffer the commands it has not yet sent. A good rule is batches of a few hundred to a few thousand commands: the gain from batching flattens out quickly, while memory use keeps growing linearly. The `redis-cli --pipe` tool and `redis-benchmark -P` show the effect; the benchmarking lab in the labs part lets you measure it on your own machine.

Pipelines versus transactions, again

A pipeline is not atomic: other clients' commands can be executed between yours. If you need isolation, wrap the batch in `MULTI` and `EXEC` (see the chapter on transactions). If you need a decision inside the batch, use a script. Many libraries combine these (a pipelined transaction), but keep the concepts separate in your head.

4.11.3 Client output buffers

Every connection has an **output buffer** on the server holding replies that have not yet been delivered. A slow reader, a huge pipeline reply, a **KEYS** result or a subscriber that cannot keep up makes this buffer grow, using server memory. To protect itself, Redis puts limits on it through `client-output-buffer-limit`, one line per client class. The usual defaults are:

Class	Hard limit	Soft limit	Soft period
normal	0 (none)	0	0
replica	256 MB	64 MB	60 s
pubsub	32 MB	8 MB	60 s

Default client-output-buffer-limit (hard limit, soft limit, soft seconds).

A client is disconnected when it reaches the hard limit, or stays above the soft limit for the soft period. Normal clients have no limit at all by default, so a client that issues a command with a gigantic reply and reads slowly can consume a lot of memory. Watch the `omem` field of `CLIENT LIST`, the `client_recent_max_output_buffer` value of `INFO clients`, and consider limiting normal clients on shared instances. Also avoid giant replies in the first place: iterate with `SCAN` and its relatives and use `HSCAN/SSCAN` instead of `HGETALL/SMEMBERS` on large collections.

4.11.4 Blocking commands

Some commands wait for data to appear: `BLPOP`, `BRPOP`, `BLMOVE`, `BLMPOP`, `BZPOPMIN`, `BZMPOP`, and `XREAD/XREADGROUP` with the `BLOCK` option. They exist so that a consumer can wait for work without polling. Their syntax always includes a timeout; `0` means wait forever (in seconds for the list commands, in milliseconds for `BLOCK` on streams).

The key point is what exactly blocks. **Only the calling client waits. The server is not blocked:** it keeps processing other clients' commands while the consumer sleeps. Redis parks the blocked connection in a list per key and wakes it, first come first served, when a producer pushes data.

```

A blocked consumer, and a producer waking it
# Terminal A (consumer)
127.0.0.1:6379> BLPOP queue:emails 30
# ... waits ...
# Terminal B (producer)
127.0.0.1:6379> RPUSH queue:emails "welcome:user-42"
(integer) 1
# Terminal A now prints
1) "queue:emails"
2) "welcome:user-42"
(7.45s)

```

If nothing arrives within the timeout the reply is `(nil)`. Things to know when using blocking commands in an application:

- **A blocked connection is occupied.** It cannot run other commands while it waits. Give each blocking consumer its own dedicated connection and never take it from a general-purpose pool, or you will starve ordinary requests. Lettuce users must not run blocking calls on the shared connection that serves the rest of the application.
- **Set the client read timeout above the block time.** If your client's socket timeout is 5 seconds and you block for 30, the client gives up first. Use a finite block time (5 to 30 seconds) in a loop; it makes shutdown and reconnection logic simple.
- **Blocking commands inside `MULTI`, scripts and functions do not block.** They behave like their non-blocking variants and return immediately, otherwise they would freeze the whole server.
- **They are not reliable queues by themselves.** `BLPOP` removes the element before your code processes it; if the worker dies, the job is lost. The patterns part shows `BLMOVE` and streams for reliable processing.

`INFO clients` reports `blocked_clients`, and the `timeout` setting in `redis.conf` closes idle (not blocked) connections; leave it at 0 unless you know why you need it.

4.11.5 Where latency comes from

When someone says "Redis is slow", first find out which of the following it is. Most cases fall into a handful of causes.

Source	How it shows up	What to do
Network RTT and chatty clients	Fast server, slow application; many small calls	Pipeline, use MGET/HMGET, batch work, keep Redis close to the app
Slow commands	Latency spikes, high CPU in the main thread, entries in SLOWLOG	Avoid O(N) commands on big keys; use SCAN family; split big keys
Big keys and big replies	Rare commands take hundreds of ms; output buffers grow	Find with redis-cli --bigkeys; paginate; use UNLINK instead of DEL
Fork for RDB or AOF rewrite	Periodic stalls, worse with large datasets and slow virtualization	Watch latest_fork_usec; give memory headroom; schedule snapshots; see persistence chapters
Disk fsync (AOF)	Latency tied to appendfsync always or a busy disk	Use everysec; faster disk; separate disk
Expiry and eviction bursts	Many keys expiring at the same second; latency at memory limit	Add jitter to TTLs; size maxmemory sensibly
Swapping	Very high, erratic latency; the OS pages Redis memory to disk	Never allow swap on a Redis host; check used_memory against RAM
Transparent Huge Pages	Latency spikes after fork	Disable THP on the host
CPU contention	Noisy neighbours, throttled containers	Dedicated cores or proper CPU limits; avoid overcommit
Scripts and MONITOR	Long BUSY periods, slowed throughput	Keep scripts tiny; never leave MONITOR running

Common sources of latency.

Remember the consequence of the single-threaded command loop: one slow command delays every client queued behind it. A 200 ms **SMEMBERS** on a million-element set is a 200 ms outage for the whole instance, not just for the caller.

4.11.6 Measuring: SLOWLOG, LATENCY and latency stats

Redis ships with tools for finding these problems. Start with the slow log.

SLOWLOG

The slow log records commands whose **execution time** exceeded `slowlog-log-slower-than` microseconds (default 10000, i.e. 10 ms). It keeps the latest `slowlog-max-len` entries (default 128). The time measured is only the time spent running the command; it does not include waiting in the queue, network time or reading the request.

```

Reading the slow log TERMINAL

127.0.0.1:6379> CONFIG SET slowlog-log-slower-than 5000
OK
127.0.0.1:6379> SLOWLOG GET 1
1) 1) (integer) 14
   2) (integer) 1791613380
   3) (integer) 18432
   4) 1) "SMEMBERS"
      2) "tags:all"
   5) "10.0.2.15:52134"
   6) "catalog-service"
127.0.0.1:6379> SLOWLOG LEN
(integer) 14
127.0.0.1:6379> SLOWLOG RESET
OK

```

Each entry has an id, the Unix time, the duration in microseconds (here 18.4 ms), the command with arguments (long arguments are truncated), the client address and the client name set with **CLIENT SETNAME**. Give every service a connection name; it makes the slow log readable. Setting the threshold to 0 logs everything, which is a quick way to see what an application actually sends, but it also costs, so do it briefly. A negative value disables the log.

LATENCY monitor and doctor

The latency monitor records events (fork, AOF fsync, expiry cycles, slow commands) that exceeded a threshold. It is off by default; you switch it on by setting a threshold in milliseconds:

```
Turn on the latency monitor TERMINAL

127.0.0.1:6379> CONFIG SET latency-monitor-threshold 100
OK
127.0.0.1:6379> LATENCY LATEST
1) 1) "fork"
   2) (integer) 1791613400
   3) (integer) 235
   4) (integer) 235
127.0.0.1:6379> LATENCY DOCTOR
Dave, I have observed latency spikes in this Redis instance. ...
```

LATENCY LATEST shows, per event type, the time of the latest spike, its duration and the maximum duration seen, in milliseconds. **LATENCY HISTORY fork** gives time series data and **LATENCY GRAPH fork** draws a small ASCII chart. **LATENCY DOCTOR** writes a human-readable analysis with suggestions. The doctor's tone is jokey, but the content is a useful checklist.

Per-command latency percentiles

Since Redis 7.0, **latency-tracking** is on by default and the server keeps a histogram per command. **INFO latencystats** shows percentiles (by default p50, p99 and p99.9) in microseconds, and **INFO commandstats** shows call counts, total time and microseconds per call. Percentiles tell you far more than averages: a command averaging 20 microseconds can still have a p99.9 of 20 ms because of the occasional big key.

```
Latency from the outside with redis-cli TERMINAL

$ redis-cli --latency
min: 0, max: 1, avg: 0.11 (1372 samples)
$ redis-cli --latency-history -i 5
$ redis-cli --intrinsic-latency 10
Max latency so far: 5 microseconds.
Max latency so far: 28 microseconds.
```

--latency measures round trips with **PING** against the server, so it includes the network. **--intrinsic-latency** measures the baseline delay of the machine itself (the scheduler, virtualization) and must be run **on the server**, not from a client; it is a good check for noisy VMs. Compare the two: if intrinsic latency is a few microseconds, but **--latency** from the application host shows 2 ms, the network is the cost.

TIP

Measure from where the problem is seen. Run **redis-cli --latency** from the application host, check **SLOWLOG** and **LATENCY LATEST** on the server, and compare. If the server side is quiet but the application sees slow calls, look at the client: its connection pool, its timeouts, garbage-collection pauses in the JVM, thread starvation, or the number of calls per request.

INTERVIEW ANGLE

Redis is single-threaded and fast, so why do some of my Redis calls take 50 ms? Common causes: network round trips and many small calls (fix with pipelining and multi-key commands); a slow O(N) command or big key blocking the single command thread so everyone queues behind it (find with **SLOWLOG**, fix the data model); a fork for persistence; swap or Transparent Huge Pages on the host; a client-side pool or GC pause. **SLOWLOG** only measures server execution, so you also compare it with client-side timings.

KEY POINTS

- One round trip per command caps a single connection at 1 / RTT operations per second; pipelining and multi-key commands remove most of that cost.
- Pipelines save time but are not atomic; keep batches to a few hundred or thousand commands.
- Blocking commands block only the caller, but the connection is occupied: use a dedicated connection and a finite timeout.
- Output buffers are unlimited for normal clients by default; avoid giant replies and watch **omem**.
- Diagnose with **SLOWLOG**, the latency monitor, **INFO latencystats** and **redis-cli --latency**; remember that slow commands delay everyone.

4.12 RESP3 and Client-Side Caching

The fastest Redis call is the one you never make. Client-side caching lets an application keep a copy of hot values in its own memory and still be told by Redis when a value changes. The reads then cost nanoseconds instead of a network round trip, and Redis load drops. The feature is built on **RESP3**, the newer version of the Redis wire protocol, so we start there.

4.12.1 RESP2 and RESP3

Clients and servers talk in **RESP** (REdis Serialization Protocol). RESP2 is the classic protocol that everything speaks by default. RESP3, available since Redis 6.0, adds richer reply types and, importantly, **push messages**: data that the server sends to a client without the client having asked for it, on the same connection.

A connection starts in RESP2. To switch, send **HELLO** with the protocol version; the server answers with information about itself:

```

Switching a connection to RESP3 TERMINAL
$ redis-cli -3
127.0.0.1:6379> HELLO 3
1# "server" => "redis"
2# "version" => "8.6.6"
3# "proto" => (integer) 3
4# "id" => (integer) 12
5# "mode" => "standalone"
6# "role" => "master"
7# "modules" => (empty array)

```

What RESP3 changes for applications:

- **Typed replies.** Maps, sets, doubles, booleans and nulls get their own types, so **HGETALL** returns a real map instead of a flat array that the client must pair up, and **ZSCORE** returns a double. Client libraries do this decoding for you.
- **Push messages.** Pub/Sub messages and client-side caching invalidations arrive as pushes. In RESP3 one connection can run normal commands while also receiving pushes, which makes the single-connection design of client-side caching possible.
- **HELLO also authenticates and names the connection:** **HELLO 3 AUTH username password SETNAME catalog-service.**

Lettuce uses RESP3 by default when the server supports it and falls back to RESP2 otherwise; Spring Data Redis and Boot inherit that with Lettuce. For Jedis, check the default protocol of your version and set it explicitly if you need RESP3. Check your client's documentation before depending on RESP3 behaviour.

4.12.2 The idea of client-side caching

Without it, every read of **product:77** goes to Redis. With it:

1. The application asks Redis for **product:77** and stores the value in a local map (for example Caffeine) inside the JVM.
2. Later reads are served from that map. No network.
3. When any client modifies **product:77**, Redis sends an **invalidation** to every connection that may have the key cached. Each of them deletes its local copy.
4. The next read misses locally, fetches the new value from Redis, and caches it again.

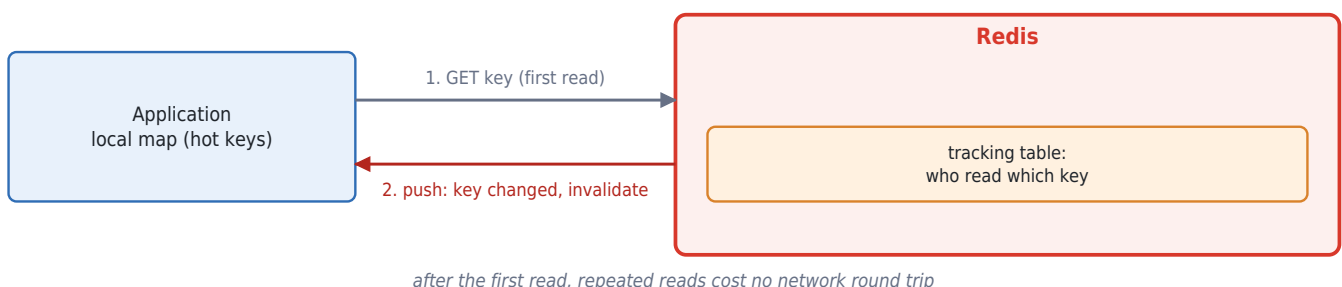


Figure 4.7. Client-side caching: the server remembers which client read which key and sends an invalidation push when the key changes. Note that the application never polls and the local copy is dropped, not updated.

The caching of values is the application's job. Redis only provides the invalidation signal, through the **CLIENT TRACKING** command.

4.12.3 CLIENT TRACKING modes

Default mode

The connection turns tracking on, then reads keys normally. The server remembers, in a global **invalidation table**, which connections have read which keys. When a key is modified, deleted, expires or is evicted, the server pushes an invalidation message to the clients that read it, then forgets that key for those clients (they have to read it again to be tracked again). This mode costs server memory proportional to the number of tracked key and client pairs, and sends messages only to clients that actually may hold the key.

```

Default tracking with two connections (RESP3)
# Connection A
127.0.0.1:6379> HELLO 3
127.0.0.1:6379> CLIENT TRACKING ON
OK
127.0.0.1:6379> GET product:77:name
"Wireless Mouse"
# Connection B
127.0.0.1:6379> SET product:77:name "Wireless Mouse v2"
OK
# Connection A receives a push message (shown here in simplified form)
1) "invalidate"
2) 1) "product:77:name"

```

The push message is of type **invalidate**, carrying an array of key names; a null instead of an array means "everything": the server sends it after **FLUSHALL** and **FLUSHDB**, and the client should clear its whole local cache.

Broadcast mode

CLIENT TRACKING ON BCAST PREFIX product: PREFIX category: takes the opposite approach. The server remembers nothing per key. It simply notifies the connection whenever **any** key starting with a subscribed prefix is touched, whether or not the client ever read it. Server memory cost is almost nothing, but the client receives many more messages. It fits when the cached keys fall under a few clear prefixes and writes are not too frequent.

Opt-in and opt-out

By default, default mode tracks every key the connection reads, which can waste table space on values you do not cache. **OPTIN** flips this: only keys read by the command that immediately follows **CLIENT CACHING YES** are tracked. **OPTOUT** tracks everything except what follows **CLIENT CACHING NO**. Opt-in is the usual choice for applications that cache selectively. **OPTIN** and **OPTOUT** do not work with **BCAST**.

NOLOOP and REDIRECT

By default a client that modifies a key also receives the invalidation for it, which is wasteful if it just wrote the value into its cache. **NOLOOP** suppresses messages caused by the client's own writes. **REDIRECT client-id** sends invalidations to a different connection, which is how RESP2 clients can emulate tracking over Pub/Sub on the channel

__redis__:invalidate. Modern code on RESP3 does not need it.

Option	Meaning
ON / OFF	Enable or disable tracking for this connection
BCAST PREFIX p	Broadcast mode: notify for all keys with prefix p, no per-key memory
OPTIN	Track only keys read right after CLIENT CACHING YES
OPTOUT	Track everything except reads right after CLIENT CACHING NO
NOLOOP	Do not send invalidations caused by this connection's own writes
REDIRECT id	Send invalidations to the connection with this client id (RESP2)

CLIENT TRACKING options.

4.12.4 Memory, limits and correctness

- **The invalidation table is bounded.** `tracking-table-max-keys` (default 1,000,000) limits how many keys are remembered. When it is full, Redis may evict old entries by sending invalidations for keys that did not change, which only costs the clients a cache miss. Correctness is never affected, only hit rate.
- **A race condition exists with separate connections.** If data and invalidations travel on different connections, an invalidation can overtake the reply of the read it refers to, leaving a stale value cached. The documented remedy is to put a placeholder in the local cache before sending the read, and to discard the reply if the placeholder was removed by an invalidation in the meantime. With RESP3 on a single connection the order is guaranteed and the problem disappears.
- **Lost connection means lost invalidations.** If the connection breaks, the client has missed messages for an unknown time and must flush its entire local cache before trusting it again.
- **TTLs still matter.** Keep a local TTL as a safety net even when you use invalidation.
- **It is eventually consistent.** There is a small window between a write and the invalidation arriving. Do not use client-side caching for values where stale reads are unacceptable, such as stock counts at checkout.

4.12.5 Using it from Java

Lettuce supports client-side caching with RESP3 through a helper in `io.lettuce.core.support.caching`. The shape of the API is:

ClientCacheExample.java

JAVA

```
package com.shopstream.demo;

import io.lettuce.core.RedisClient;
import io.lettuce.core.TrackingArgs;
import io.lettuce.core.api.StatefulRedisConnection;
import io.lettuce.core.support.caching.CacheAccessor;
import io.lettuce.core.support.caching.CacheFrontend;
import io.lettuce.core.support.caching.ClientSideCaching;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class ClientCacheExample {

    public static void main(String[] args) {
        RedisClient client = RedisClient.create("redis://localhost:6379");
        StatefulRedisConnection<String, String> connection = client.connect();

        Map<String, String> local = new ConcurrentHashMap<>();
        CacheFrontend<String, String> frontend = ClientSideCaching.enable(
            CacheAccessor.forMap(local), connection, TrackingArgs.Builder.enabled());

        String first = frontend.get("product:77:name"); // network, then cached locally
        String second = frontend.get("product:77:name"); // served from the local map
        System.out.println(first + " " + second);

        frontend.close();
        connection.close();
        client.shutdown();
    }
}
```

What to notice: the connection must speak RESP3 (the default with a recent server). The map is the cache implementation; in production you would plug in a bounded cache with expiry instead of a `ConcurrentHashMap`. Spring's `RedisCacheManager` does not use `CLIENT TRACKING`, so for Spring Cache applications a local Caffeine layer in front of Redis, as described in the multi-level caching chapter of the Patterns part, is the practical combination.

4.12.6 Is it worth it?

Client-side caching pays off for a small set of very hot, read-mostly values read by many requests per second: product pages during a campaign, feature flags, configuration, reference data. It does not pay off when values change constantly, when the hot set is huge, or when every application instance would hold a copy of data it rarely reads (each cached key costs both client memory and an invalidation-table entry). Measure the hit ratio of the local cache first. If it is below about 50 percent, the extra moving parts probably are not worth it.

REAL WORLD

A typical e-commerce site caches the product detail view in Redis. During a sale, a handful of products receive most traffic. Adding a small local cache with tracking-based invalidation for only those hot product keys removes thousands of identical network calls per second per instance, while the long tail still goes straight to Redis. The local cache is deliberately small and short-lived, so a missed invalidation can only cause a brief period of staleness.

INTERVIEW ANGLE

How does Redis client-side caching keep local caches consistent? The client enables **CLIENT TRACKING** on a RESP3 connection. In default mode the server records which keys the connection read, and when any of them changes it pushes an **invalidate** message; the client then drops its copy. In broadcast mode the client subscribes to key prefixes instead and Redis keeps no per-client state. The table of tracked keys is bounded by **tracking-table-max-keys**, and on a lost connection the client must flush everything. It is eventual consistency, not a transaction.

KEY POINTS

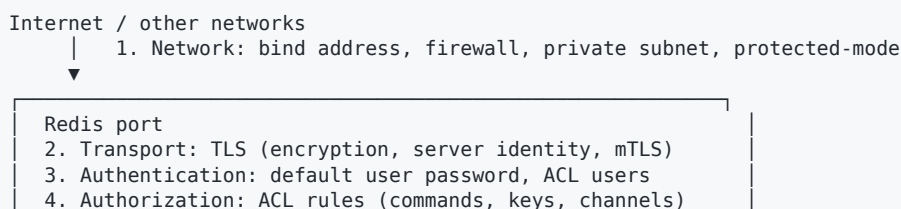
- RESP3 (**HELLO 3**) adds typed replies and push messages on the same connection.
- **CLIENT TRACKING** lets Redis tell clients when a key they cached has changed; the cache itself lives in the application.
- Default mode costs server memory per tracked key; **BCAST** with prefixes costs none but sends more messages; **OPTIN** limits tracking to chosen reads.
- Flush the whole local cache on connection loss and on a null invalidation; keep a local TTL as a safety net.
- Use it for hot, read-mostly, staleness-tolerant data, and measure the local hit ratio first.

4.13 The Redis Security Model

Redis was designed for trusted environments: it assumes that anyone who can send it commands is allowed to read and destroy everything. For years that assumption was the whole security story, and it is why thousands of unprotected instances exposed to the internet were found, wiped and used for crypto-mining. Modern Redis gives you real tools: network binding, passwords, fine-grained **ACLs** (access control lists), and **TLS**. This chapter explains the layers and how they fit together. The operations part returns to security with a full hardening procedure; here we build the mental model you need to configure a safe instance.

4.13.1 Layers of defence

Think of Redis security as four layers, from the outside in. Do not rely on any single one.



A Redis instance should never be reachable from the public internet. That sentence is the single most valuable piece of Redis security advice, and everything else is defence in depth.

4.13.2 Network exposure: bind and protected mode

The `bind` directive lists the network interfaces on which Redis listens. The shipped default config binds to loopback only (`bind 127.0.0.1 -:::1`), so a fresh server on your laptop is reachable only from your own machine. To accept connections from application servers, bind to the private interface address (for example `bind 10.0.2.15`), not to `0.0.0.0` (all interfaces) unless a firewall protects the port.

Protected mode (`protected-mode yes`, the default) is a safety net: if the server has no explicit `bind` setting and no password configured, it answers connections from non-loopback addresses with an error that explains how to fix the configuration, instead of serving them. It protects people who start a default server and forget about the network. It is not a substitute for a firewall.

What protected mode looks like to a remote client

TERMINAL

```
$ redis-cli -h 203.0.113.9 ping
DENIED Redis is running in protected mode because protected mode is enabled and no password is set
for the default user. In this mode connections are only accepted from the loopback interface. ...
```

WATCH OUT

Running Redis in Docker with `-p 6379:6379` publishes the port on every host interface, and Docker edits the firewall rules around your back. On a cloud VM this can expose Redis to the internet even though you "never opened the port". Publish only to loopback (`-p 127.0.0.1:6379:6379`), or use a private Docker network where only application containers can reach it. The same applies to Compose files and Kubernetes services of type `LoadBalancer`.

4.13.3 Authentication and the default user

Out of the box Redis has one user called `default` with no password and full permissions. Setting `requirepass secret` in `redis.conf` simply gives that user a password. Clients then authenticate with `AUTH password`, or with `AUTH username password` for named users.

Authenticating

TERMINAL

```
127.0.0.1:6379> GET product:77
(error) NOAUTH Authentication required.
127.0.0.1:6379> AUTH s3cr3t-passphrase
OK
127.0.0.1:6379> GET product:77
"..."
```

A single shared password is better than nothing, but it has two problems: everybody who knows it can do everything, including `FLUSHALL` and `CONFIG SET`; and rotating it requires touching all clients at once. The ACL system fixes both. Passwords should be long and random (Redis is fast at checking passwords, so an attacker can try a very large number of guesses per second against a short one), and stored in a secret manager, never in the repository. Passwords are stored in memory and in the ACL file as SHA-256 hashes.

GOTCHA

Passing the password on the command line (`redis-cli -a secret`) leaves it in the shell history and in the process list. `redis-cli` warns about it. Use the `REDISCLI_AUTH` environment variable or an interactive `AUTH` instead, and never run `MONITOR` on a production server: it shows every command including authentication calls.

4.13.4 ACLs: users, commands, keys and channels

An **ACL user** is a name plus a set of rules. Rules say whether the user is enabled (`on/off`), which passwords it accepts (`>password` adds one, `<password` removes one, `nopass` allows none), which commands it may run, which keys it may touch and which Pub/Sub channels it may use. You create or modify a user with `ACL SETUSER`:

Create a restricted worker for the order notifier

TERMINAL

```
127.0.0.1:6379> ACL SETUSER notifier on >n0t1f13r-passw0rd ~stream:orders ~notif:* \
&notify:* +@stream +@read +@connection -@dangerous
OK
127.0.0.1:6379> ACL GETUSER notifier
1) "flags"
2) 1) "on"
3) "passwords"
4) 1) "5d4b..."
5) "commands"
```

Create a restricted worker for the order notifier (continued)

TERMINAL

```

6) "-@all +@stream +@read +@connection -@dangerous"
7) "keys"
8) "~stream:orders ~notif:~"
9) "channels"
10) "&notify:~"
11) "selectors"
12) (empty array)

```

The rule language, in the order you would usually write them:

Rule	Meaning
on / off	Enable or disable the user
>pwd / <pwd / nopass / resetpass	Add, remove, disable or clear passwords
~pattern	Allow keys matching the glob pattern (many allowed; ~* is all keys)
%R~pattern / %W~pattern	Allow read-only or write-only access to keys matching the pattern
&pattern	Allow Pub/Sub channels matching the pattern (default: none since 7.0)
+command / -command	Allow or deny one command, or a subcommand as +config get
+@category / -@category	Allow or deny a category such as @read, @write, @list, @dangerous
allkeys / allchannels / allcommands	Aliases for ~, & and +@all
(rules...)	A selector: an extra set of rules, any of which may grant access

Core ACL rules.

Rules are applied **in order**, left to right, and later rules can override earlier ones. A new user starts with no commands, no keys, no channels and is off. The usual recipe is: grant a category with **+@** and then subtract what you do not want, as in **+@all -@dangerous**, or build up from nothing.

Command categories

Every command belongs to one or more categories, and **ACL CAT** lists them: **@keyspace**, **@read**, **@write**, **@set**, **@sortedset**, **@list**, **@hash**, **@string**, **@bitmap**, **@hyperloglog**, **@geo**, **@stream**, **@pubsub**, **@admin**, **@fast**, **@slow**, **@blocking**, **@dangerous**, **@connection**, **@transaction** and **@scripting**. **ACL CAT dangerous** lists the commands in one category. Since Redis 8.0 the commands from the built-in JSON, search, time series and probabilistic modules also belong to these standard categories, so **+@read** covers **JSON.GET** and **+@write** covers **TS.ADD**. Note one subtlety: **+@all** includes module commands, but a narrower category list excludes anything not in it, so test the user after any upgrade.

Key permissions

~jobs:* grants both reading and writing keys that match. **%R~** and **%W~** split these. A command needs **read** permission when it returns or copies the user's data from the key, and **write** permission when it may change the key. **GET** needs read; **LPUSH** needs only write; **LPOP** needs both, because it modifies the list and returns data. Combine with selectors when two key sets need different command sets:

A user that reads catalogue keys and writes only to its own keys

TERMINAL

```

127.0.0.1:6379> ACL SETUSER catalog-svc on >c4t4l0g-passw0rd \
  +@all -@dangerous %R~product:* %RW~session:*
OK

```

Selectors (Redis 7.0 and later)

A **selector** is an additional rule set in parentheses. A command is allowed if the root permissions OR any selector allow it. Example: **ACL SETUSER reporter on >pw +GET ~report:* (+SET ~scratch:*)** lets the user read **report:*** keys and write **scratch:*** keys, but not read **scratch:*** or write **report:***. Selectors let you express "this command on these keys, that command on those keys", which a single flat rule set cannot.

Testing, logging and persisting ACLs

ACL DRYRUN `user command args...` checks whether a user could run a command without executing it, which is excellent for tests and CI. **ACL WHOAMI** names the current user, **ACL LIST** shows all rules, and **ACL DELUSER** removes a user and disconnects its connections.

Dry-run and the ACL log

TERMINAL

```
127.0.0.1:6379> ACL DRYRUN notifier GET other:1
"User notifier has no permissions to access the 'other:1' key"
127.0.0.1:6379> ACL LOG 1
1) 1) "count"
    2) (integer) 1
    3) "reason"
    4) "key"
    5) "context"
    6) "toplevel"
    7) "object"
    8) "other:1"
    9) "username"
    10) "notifier"
    11) "age-seconds"
    12) "0.01"
    13) "client-info"
    14) "id=321 addr=127.0.0.1:54980 ... cmd=get user=notifier ..."
```

The **ACL log** records denied commands, key and channel accesses, and failed **AUTH** attempts, with a reason (**command**, **key**, **channel** or **auth**). Review it in monitoring: a stream of **auth** failures from an unexpected address is an attack or a misconfigured client. The log is capped by **aclog-max-len** (default 128).

ACL rules set with **ACL SETUSER** live in memory and are lost on restart unless they are defined in the config. There are two ways to persist them, and they cannot be mixed. Either put **user ...** lines directly in **redis.conf**, or point **aclfile** **/etc/redis/users.acl** to a dedicated file and use **ACL LOAD** and **ACL SAVE**. The file approach is better for production because **ACL SAVE** writes changes without touching the main config. Note that **CONFIG REWRITE** does not save ACLs.

users.acl

CONF

```
user default off
user admin on #<sha256-of-password> ~* &* +@all
user catalog-svc on #<sha256-of-password> %R~product:* %RW-session:* +@all -@dangerous
user notifier on #<sha256> ~stream:orders ~notif:* &notify:* +@stream +@read +@connection
```

Turning the **default** user off, as the first line does, forces every client to authenticate as a named user. The hash form **#<sha256>** keeps cleartext passwords out of the file.

TIP

Create one ACL user per application or service, named after it, with the minimum commands and key patterns it needs. Then a leaked credential from the catalog service cannot flush the database or read sessions, and **CLIENT LIST** and the ACL log show you exactly which service did what.

4.13.5 TLS and mutual TLS

Without TLS, passwords and data travel in plain text. TLS (transport layer security) encrypts the connection and lets the client verify it is talking to the right server. It is needed whenever traffic crosses a network you do not fully control, and many organisations require it everywhere. Redis supports TLS when built with it (the official packages and images are). The main directives:

Directive	Purpose
tls-port 6379	Port that accepts TLS connections
port 0	Disable the plain-text port when you want TLS only
tls-cert-file / tls-key-file	The server's certificate and private key
tls-ca-cert-file (or tls-ca-cert-dir)	CA certificates used to verify client certificates
tls-auth-clients yes no optional	Require, skip or allow client certificates (default yes)
tls-replication yes	Use TLS for replication links
tls-cluster yes	Use TLS for the cluster bus and cluster traffic

Directive	Purpose
<code>tls-protocols</code>	Restrict TLS versions, for example TLSv1.2 TLSv1.3

Essential TLS settings in redis.conf.

With `tls-auth-clients yes`, the server demands a valid client certificate signed by the configured CA. That is **mutual TLS (mTLS)**: both sides prove their identity. It works well for service-to-service traffic where certificates are issued automatically. Redis 8.6 adds **certificate-based automatic client authentication**: with the `tls-auth-clients-user` directive Redis can map the certificate presented by the client to an ACL user, so that the connection is authenticated by the certificate and no password is needed. Check the 8.6 documentation for the exact mapping rules before relying on it, and watch the `acl_access_denied_tls_cert` statistic for failures.

From the client side, `redis-cli --tls --cacert ca.crt --cert client.crt --key client.key -h redis.internal` connects, and in Spring Boot you enable TLS with `spring.data.redis.ssl.enabled=true` and an SSL bundle (covered in the Spring part).

4.13.6 Dangerous commands and what else to restrict

Some commands are risky in the wrong hands or at the wrong time. A few examples and why:

Command	Danger
<code>FLUSHALL</code> , <code>FLUSHDB</code>	Deletes all data
<code>KEYS</code> , <code>SMEMBERS</code> on huge sets, <code>HGETALL</code> on huge hashes	Blocks the server for a long time
<code>CONFIG SET</code> , <code>CONFIG REWRITE</code>	Changes behaviour, can write files to disk
<code>DEBUG</code> , <code>SHUTDOWN</code> , <code>REPLICAOF</code>	Can crash, stop or reconfigure the node
<code>MONITOR</code>	Streams every command, including secrets; very costly
<code>EVAL</code> , <code>FUNCTION</code> , <code>SCRIPT</code>	Run code; a runaway script blocks everyone
<code>MODULE LOAD</code>	Loads native code into the server process

Commands to restrict.

Older guides recommend `rename-command CONFIG ""` to disable them. That still works but is clumsy, breaks replication and tooling in subtle ways and is discouraged in favour of ACLs. The modern approach: leave application users on a restricted category list (`-@dangerous -@admin`), and keep a separate administrative account whose password only operators hold. Beware that `CONFIG SET dir` plus `CONFIG SET dbfilename` plus `SAVE` was historically used by attackers to write files such as SSH keys or crontabs from an exposed instance; this is one more reason to forbid `CONFIG` for application users and to run Redis as an unprivileged OS user.

4.13.7 A hardening checklist

This is the minimum bar for any instance beyond your laptop. The operations part expands each point.

1. **Never expose Redis to the internet.** Bind to a private interface, use security groups or firewalls that admit only the application subnets.
2. **Disable or password the default user** and create one ACL user per service with minimal permissions.
3. **Use TLS** for client, replication and cluster traffic when crossing untrusted networks; consider mTLS for service identity.
4. **Restrict dangerous commands** through ACL categories rather than renaming.
5. **Run as a non-root user**, with the data directory and config readable only by that user, and no shell access.
6. **Keep secrets out of code and images.** Use a secret manager; rotate passwords by adding the new password with `>new`, rolling out the clients, then removing the old one with `<old`.
7. **Monitor ACL LOG**, failed authentications, connections from unexpected hosts, and `CONFIG` changes.
8. **Patch.** Subscribe to the security announcements for your Redis version and upgrade promptly; several past vulnerabilities concerned Lua scripting and the module APIs, which is another reason to restrict who can run scripts.

INTERVIEW ANGLE

How do you secure a Redis deployment? Layer it. Network first: private subnet, firewall, **bind** to an internal address, **protected-mode** on. Then TLS to encrypt traffic and optionally mTLS for identity. Then authentication with ACL users instead of a shared **requirepass**, one user per service. Then authorization with ACL rules that limit commands, categories, key patterns and channels, and exclude **@dangerous** for apps. Finally operations: run unprivileged, manage secrets, rotate passwords, monitor the ACL log, and keep the software patched.

KEY POINTS

- Redis trusts its clients by design, so keep it off public networks: **bind** to private interfaces, firewall it, and watch Docker port publishing.
- **protected-mode** is a safety net, not a security feature to rely on.
- Replace the shared password with ACL users: one per service, limited by commands or categories, key patterns (**~**, **%R~**, **%W~**) and channels (**&**).
- Persist ACLs in an **aclfile**; test rules with **ACL DRYRUN**; review **ACL LOG**.
- Use TLS (and mTLS where useful); Redis 8.6 can map client certificates to ACL users.
- Restrict **@dangerous** commands with ACLs, run as a non-root user, and never leave **MONITOR** running.

PART 5

Scaling and High Availability

One Redis process is fast, but it is also one machine, one failure and one core. This Part is about copying data, surviving failures and splitting data across machines.

In this part

- 5.1 Replication
- 5.2 Sentinel: automatic failover
- 5.3 Why Cluster: sharding basics
- 5.4 Redis Cluster
- 5.5 Atomic slot migration and operating a cluster
- 5.6 Choosing a topology
- 5.7 Multi-region and managed services

5.1 Replication

A single Redis server is a single point of failure. If the machine dies, the process crashes, or someone runs a bad deployment, every key in memory is gone until you restore from disk, and during that time every service that depends on Redis is degraded. Even when nothing fails, one server has limits: one CPU core executes commands, and the network card of that one machine serves every read.

Replication is the oldest and simplest answer to both problems. You run one **primary** (the older docs and some tools say "master") that accepts writes, and one or more **replicas** (older name: "slaves") that keep a live copy of the primary's data. Replicas are a safety net, a way to spread read traffic, and the building block for everything else in this Part: Sentinel promotes replicas automatically, and Redis Cluster gives every shard its own replicas.

This chapter explains exactly how the copy is made and kept up to date, what the copy does and does not guarantee, and how to read from replicas without being surprised.

5.1.1 The mental model

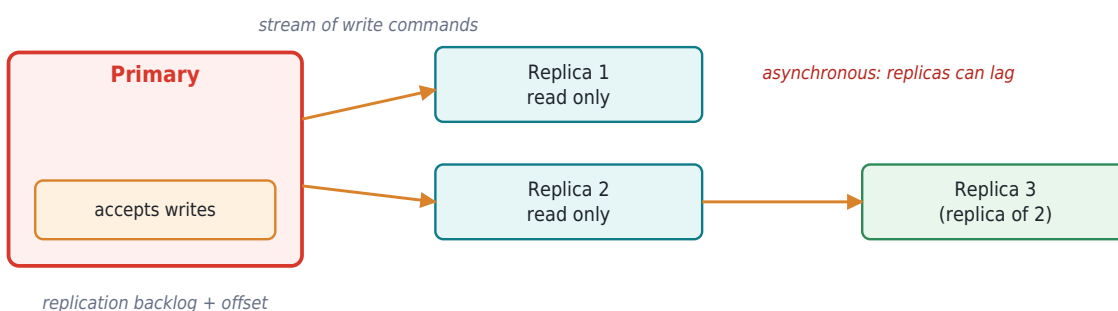


Figure 5.1. Primary-replica replication. Writes go to the primary only; the primary streams every write command to each replica. Notice that the replica can itself have replicas.

Three facts explain almost everything about Redis replication:

1. **Replication is asynchronous.** The primary executes your write, replies **OK** to the client, and only then (a few microseconds or milliseconds later) sends the command to the replicas. The client is not waiting for the replicas.
2. **A replica is a byte-for-byte follower of a command stream.** The primary turns every write into a replication stream (the same RESP format clients use) and sends it to each replica, which applies it in order. A replica never decides anything on its own about the data.
3. **Replication is one-directional.** Data flows primary → replica. By default a replica is read-only and rejects writes.

The second fact has a useful consequence: two instances that have the same *replication ID* and the same *replication offset* hold exactly the same data. We will use that pair in a moment.

5.1.2 Setting up a replica

You turn any Redis server into a replica of another with one command, at runtime, without restarting:

Make a server a replica

TERMINAL

```
127.0.0.1:6380> REPLICAOF 127.0.0.1 6379
OK
127.0.0.1:6380> SET x 1
(error) READONLY You can't write against a read only replica.
```

The equivalent line in `redis.conf` is `replicaof 127.0.0.1 6379`. To turn a replica back into a standalone primary (this is what a manual failover does), run `REPLICAOF NO ONE`. The older command name `SLAVEOF` still works as an alias, but use `REPLICAOF` in everything you write.

If the primary requires authentication, the replica needs credentials too: `masterauth <password>` (and `masteruser <name>` when you use ACL users). Forgetting this is the most common reason a new replica sits in a connecting state forever; the replica's log shows an authentication error.

You can inspect the relationship from either side with `INFO replication` and `ROLE`:

Inspecting replication (typical output, trimmed)

TERMINAL

[illegible]

The field names still use the old words (`connected_slaves`, `slave0`) for compatibility with tools that parse the output. Read them as replicas. Compare `master_repl_offset` on the primary with the replica's `slave_repl_offset`: the difference, in bytes of replication stream, is the replica's lag. The `lag=1` in the `slave0` line is a different measure: seconds since the replica last acknowledged.

5.1.3 Full synchronization

When a replica connects for the first time (or after it has been away too long), it needs a complete copy of the dataset. This is a **full resynchronization**:

1. The replica sends **PSYNC ? -1**, which means "I have no history, give me everything".
2. The primary replies with **+FULLRESYNC <replid> <offset>** and starts a background save (a **fork**, exactly like **BGSAVE**; see the chapter on RDB snapshots in Part 4) to produce an RDB snapshot.
3. While the snapshot is being produced and transferred, the primary keeps accepting writes and **buffers** them in memory in a per-replica output buffer.
4. The replica receives the RDB file, flushes its old dataset, and loads the snapshot into memory. While it does this it cannot serve fresh data.
5. The primary then sends the buffered writes, and from then on streams every new write as it happens.

This works on any primary, but it is expensive: a fork on the primary, a burst of network traffic proportional to the dataset size, a load on the replica, and a memory cost for the buffered writes. On a 20 GB dataset this takes minutes.

Disk-backed versus diskless

By default the primary can write the RDB to its disk and then send the file (disk-backed), or it can have the forked child **stream the RDB straight to the replica sockets** without touching disk (diskless). Diskless is the default in recent versions (`repl-diskless-sync yes`). Because a diskless transfer to several replicas is more efficient when they arrive together, the primary waits `repl-diskless-sync-delay` seconds (default 5) after the first replica asks, to collect more replicas before it starts the transfer. Set it to 0 if you have only one replica and do not want to wait. On the replica side, `repl-diskless-load` controls whether the replica parses the RDB directly from the socket instead of saving it to disk first; the default is `disabled`, and the other values (`swapdb`, `on-empty-db`) trade extra memory or risk for speed, so leave it alone until you have measured a real problem.

NOTE

Diskless replication removes the primary's *disk* from the sync path, not the fork. The child process still needs copy-on-write memory while the primary takes writes, so keep memory headroom (see the chapter on RDB snapshots).

5.1.4 Partial resynchronization

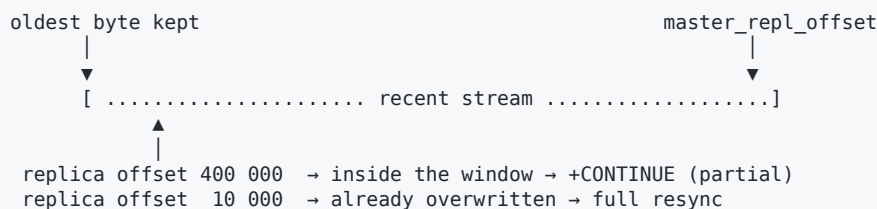
Networks blip. If every disconnection triggered a full sync, a ten-second network glitch would cost you a fork and gigabytes of traffic. Redis avoids this with **partial resynchronization (PSYNC)**.

Three pieces make it work:

- **Replication ID (`replid`)**: a long random string that names one *history* of the dataset. A primary generates a new one when it starts fresh or when a replica is promoted.
- **Replication offset**: a number that increases by the number of bytes of replication stream the primary produces. It increases even when no replica is connected.
- **Replication backlog**: a fixed-size in-memory ring buffer on the primary holding the most recent bytes of the stream (`repl-backlog-size`, default 1 MB).

A replica remembers the `replid` and the offset it has processed. When it reconnects it sends `PSYNC <replid> <offset+1>`. The primary checks two conditions: it recognises that replication ID, and the requested byte is still inside the backlog. If both hold, it answers `+CONTINUE` and sends only the missing bytes. Otherwise it falls back to a full resync.

Backlog (ring buffer, size = `repl-backlog-size`)



The default of 1 MB is small. The right size is: *peak write rate (bytes of stream per second) × the longest disconnection you want to survive without a full resync*, plus a margin. If your primary produces 5 MB/s of replication stream and you want to ride out a 2 minute outage, you need roughly 600 MB. Memory for the backlog is allocated once and shared by all replicas, so one large backlog costs the same with one replica or ten. If the primary has had no replicas for `repl-backlog-ttl` seconds (default 3600) it frees the backlog.

Why there are two replication IDs

After a failover the promoted replica starts a new history, so it picks a new `replid`, but it remembers the old one as `master_replid2` together with the offset at which the switch happened (`second_repl_offset`). The other replicas, which still talk in terms of the old ID, can therefore reconnect to the new primary and get a *partial* sync instead of a full one. Since Redis 4.0 this makes failovers much cheaper: the cost is a few seconds, not a full dataset transfer.

GOTCHA

A replica that restarts and loads from an AOF file cannot do a partial sync, because the AOF does not store the replication ID and offset. A replica that restarts with an RDB file can. This is one reason some operators keep replicas on RDB only even when the primary uses AOF.

5.1.5 What exactly travels on the wire

It helps to know what a replica receives after the initial sync, because it explains several behaviours. The stream is a sequence of ordinary RESP commands, but not always the commands your clients sent:

- **Non-deterministic commands are rewritten.** A command such as `SPOP` removes a random member; replaying it on a replica could remove a different one. The primary therefore propagates the deterministic effect (`SREM` of the member it picked). Relative expiry commands (`EXPIRE key 60`) are propagated as absolute times (`PEXPIREAT`), so a replica that receives them late still expires the key at the same moment.
- **Expiry and eviction become `DEL`.** Only the primary decides a key has expired or must be evicted; it then sends an explicit `DEL` (or `UNLINK`).
- **Lua scripts replicate by effect.** Since Redis 5 the default is to send the commands a script executed, not the script text, so scripts may call random or time commands safely.
- **Heartbeats.** The primary sends a `PING` every `repl-ping-replica-period` seconds (default 10), and each replica sends `REPLCONF ACK <offset>` back about once per second. Those acknowledgements are what `WAIT`, `min-replicas-to-write` and the `lag=` field in `INFO` are built on.
- `SELECT` commands are inserted when the stream switches database.

Because the stream is a single ordered sequence applied by a single thread, one slow command on a replica delays everything behind it, and a replica applies writes strictly in the order the primary produced them. That ordering is the reason a replica is always a consistent (if slightly old) snapshot of the primary rather than a random mixture.

5.1.6 What replication does and does not guarantee

Because the primary replies before the replicas have the write, there is always a window in which a write exists only on the primary. If the primary dies in that window and a replica is promoted, **the write is lost**. In a failover under load this window is usually small (microseconds to milliseconds of stream), but it is not zero, and it grows if a replica is lagging.

Redis gives you two tools to narrow the window. Neither turns Redis into a strongly consistent database.

WAIT: ask for acknowledgements

`WAIT numreplicas timeout` blocks the calling client until all previous writes **of this connection** have been acknowledged by at least `numreplicas` replicas, or until `timeout` milliseconds pass. It returns how many replicas acknowledged. A timeout of 0 blocks forever, so use a real number.

Synchronous-ish writes with `WAIT`

TERMINAL

```
127.0.0.1:6379> SET order:9001:status PAID
OK
127.0.0.1:6379> WAIT 1 100
(integer) 1
127.0.0.1:6379> WAIT 2 100
(integer) 1
```

The second `WAIT` returned 1 after 100 ms because only one replica is connected. Your code must compare the return value with what it asked for, and decide what to do if the write is under-replicated (retry, log, alert, or fail the request). `WAIT` does not roll anything back. Also, replication being acknowledged in memory does not mean it is on disk: since Redis 7.2 there is a companion command, `WAITAOF numlocal numreplicas timeout`, that waits for writes to be fsynced to the AOF locally and/or on replicas.

Even `WAIT` is best effort in a failover. It makes it more likely that the replica chosen by Sentinel or Cluster has your write, but the promotion logic does not guarantee it.

min-replicas-to-write: refuse writes when too few replicas are healthy

The primary can also protect itself. With these two settings it stops accepting writes when too few replicas are healthy:

redis.conf (primary)

CONF

```
min-replicas-to-write 1
min-replicas-max-lag 10
```

Replicas send an acknowledgement to the primary about once per second. If fewer than `min-replicas-to-write` replicas have acknowledged within the last `min-replicas-max-lag` seconds, the primary answers write commands with an error (`NOREPLICAS Not enough good replicas to write.`) instead of executing them. This bounds how much data

a partitioned or isolated primary can accept (and later lose); we return to this in the Sentinel chapter as a split-brain mitigation. The price is availability: if your only replica is down, the primary stops taking writes. Choose it deliberately.

WATCH OUT

Replication is not a backup. A bad `FLUSHALL`, a buggy deployment that deletes keys, or a corrupted value is replicated to every replica within milliseconds. You still need RDB/AOF files and tested restores (see the operations Part).

5.1.7 Reading from replicas

Replicas can serve reads, so adding replicas increases read capacity for read-heavy workloads such as a product catalogue. Two things to understand first.

Reads can be stale

A read from a replica reflects the primary as of a moment ago. Usually that is a few milliseconds; under heavy write load, a slow network or a big full sync, it can be seconds. The classic bug is **read-your-writes**: a user updates their profile (goes to the primary), the page reloads (reads from a replica), and the old value appears. Mitigations: read from the primary for a short time after a write by the same user, route all of a session's traffic to the primary for critical flows, or use `WAIT` before redirecting.

Expiry on replicas

Replicas do not expire keys on their own. When the primary expires or evicts a key it sends a synthetic `DEL` to the replicas. In between, a replica may still hold a key that is logically expired; to avoid returning it, the replica checks the key's TTL against its own clock and answers "not found" for read commands. One consequence is that if the replica's clock differs from the primary's, TTL behaviour can look different; keep clocks synchronised with NTP. After a replica is promoted it starts expiring keys itself.

Other behaviours worth knowing

- `replica-read-only yes` (default) makes replicas reject writes. Writable replicas exist, but data written to them diverges from the primary and is wiped at the next resync. Avoid them.
- `replica-serve-stale-data yes` (default) lets a replica answer reads while its link to the primary is down or while it is doing the initial sync, with whatever it has. Set it to `no` if returning old or empty data is worse than returning an error; the replica then answers most commands with `MASTERDOWN`.
- `replica-priority` (default 100) is used by Sentinel when choosing which replica to promote. A replica with priority 0 is never promoted, which is useful for a reporting replica in another data centre.
- **Chained replication**: a replica can have replicas of its own. This reduces the load on the primary when you have many replicas, at the cost of more lag at the bottom of the chain.
- Containers and NAT: when the primary sees a replica's address as the container or NAT address, Sentinel and tools that read `INFO replication` get wrong endpoints. Set `replica-announce-ip` and `replica-announce-port` on the replica.

For a Spring Boot application using Lettuce, the read preference is a one-line customization (Spring Boot 4 package shown). With a replicated setup (Sentinel or static master/replica configuration), `ReadFrom.REPLICA_PREFERRED` reads from replicas when they are available and from the primary otherwise:

RedisReadConfig.java

JAVA

```
package com.shopstream.catalog.config;

import io.lettuce.core.ReadFrom;
import org.springframework.boot.data.redis.autoconfigure.*;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RedisReadConfig {

    @Bean
    LettuceClientConfigurationBuilderCustomizer readFromReplicas() {
        return builder -> builder.readFrom(ReadFrom.REPLICA_PREFERRED);
    }
}
```

Use this only for reads that tolerate staleness, such as the product catalogue. Writes still go to the primary. The Lettuce chapter in the Java clients Part and the Spring setup chapter explain the other `ReadFrom` choices and how topology changes are handled.

5.1.8 A dangerous combination: no persistence plus automatic restart

WATCH OUT

Suppose the primary has persistence turned off and an init system or orchestrator that restarts crashed processes. The primary crashes and restarts with an **empty dataset**. Its replicas reconnect, see the new empty primary, and faithfully replicate "nothing", deleting their copy of your data. Either keep persistence on in the primary, or disable automatic restart for it (and let Sentinel or an operator promote a replica first).

5.1.9 Promoting a replica by hand

Sooner or later you move the primary role deliberately: a hardware upgrade, an OS patch, a failed disk. Without Sentinel or Cluster, the steps are manual. The safest sequence stops writes first so that the replica is fully caught up:

1. Make sure the replica is healthy and caught up: `master_link_status:up` and `slave_repl_offset` equal to the primary's `master_repl_offset`.
2. Stop writes (pause the application, or put the primary into maintenance). `CLIENT PAUSE <ms>` blocks writes for a limited time without disconnecting anyone.
3. Check again that the offsets match, then run `REPLICAOF NO ONE` on the replica.
4. Point the application (or the DNS name or load balancer) at the new primary, and make the old primary a replica of it with `REPLICAOF`.

Since Redis 6.2 there is a command that coordinates this for you. Run it on the **primary**:

```
Coordinated failover with FAILOVER
127.0.0.1:6379> FAILOVER TO 10.0.1.11 6379 TIMEOUT 5000
OK
127.0.0.1:6379> INFO replication
# Replication
role:master
master_failover_state:waiting-for-sync
...
```

TERMINAL

The primary pauses writes, waits until the chosen replica has caught up to its exact offset, hands over the role, and turns itself into a replica of the new primary. `TIMEOUT` bounds how long it waits, `FORCE` (with `TO`, and `TIMEOUT`) completes the switch even if the replica did not catch up, and `FAILOVER ABORT` cancels one in progress. The `master_failover_state` field you saw in `INFO replication` shows `no-failover`, `waiting-for-sync` or `failover-in-progress`. This is a manual, lossless switch of roles; it is not failure detection, which is what Sentinel adds.

5.1.10 Monitoring replication

Four checks catch most replication problems before users notice:

Signal	What it tells you
<code>master_link_status</code> on the replica	<code>up</code> means connected; <code>down</code> for longer than a few seconds needs investigation (network, auth, primary down)
<code>master_repl_offset</code> minus <code>slave_repl_offset</code>	Lag in bytes of replication stream; a steady increase means the replica cannot keep up
<code>lag=</code> field per replica on the primary	Seconds since the last acknowledgement; above the <code>min-replicas-max-lag</code> value the primary may refuse writes
<code>sync_full</code> and <code>sync_partial_ok</code> in <code>INFO stats</code>	Full syncs on a stable system suggest a too-small backlog or flapping links
<code>connected_slaves</code> on the primary	Fewer replicas than expected

Replication signals worth alerting on.

Byte lag is a more precise health signal than seconds, but remember that both only tell you about the *replication link*. They say nothing about whether the replica's own disk or CPU is the problem; for that use the general monitoring chapter in the operations Part.

5.1.11 Common replication mistakes

- **Treating a replica as a backup.** It faithfully copies deletions and corruption.
- **Leaving the backlog at 1 MB** on a busy primary, so every short disconnection becomes a full sync.
- **Forgetting `masterauth`** (and `masteruser`) on the replicas, so they cannot sync, or cannot follow a new primary after failover.
- **Reading your own writes from a replica** and being surprised by stale data.
- **Running the primary without persistence under an auto-restarting supervisor**, which can erase the replicas.
- **Writable replicas** used for scratch data, which silently diverge.
- **Counting on `WAIT` as consensus.** It tells you how many replicas acknowledged, nothing more.
- **Starting many replicas at once** on a large dataset and saturating the primary's network.
- **Ignoring clocks.** Replica-side logical expiry and TTL reasoning assume reasonably synchronised clocks.

5.1.12 Replication settings at a glance

Setting	Default	Purpose
<code>replicaof</code> host port	none	Make this node a replica of another
<code>masterauth</code> / <code>masteruser</code>	none	Credentials the replica uses to authenticate to the primary
<code>repl-diskless-sync</code>	yes	Child streams the RDB to replicas without writing a file
<code>repl-diskless-sync-delay</code>	5	Seconds to wait for more replicas before starting a diskless transfer
<code>repl-backlog-size</code>	1mb	Size of the partial-resync buffer; size it from write rate x outage time
<code>repl-backlog-ttl</code>	3600	Seconds without replicas before the backlog is freed
<code>repl-timeout</code>	60	Timeout for the replication link (bulk transfer and keepalive)
<code>replica-read-only</code>	yes	Reject writes on replicas
<code>replica-serve-stale-data</code>	yes	Answer reads when the link to the primary is down
<code>replica-priority</code>	100	Promotion preference for Sentinel; 0 means never promote
<code>min-replicas-to-write</code> / <code>min-replicas-max-lag</code>	0 / 10	Refuse writes when too few replicas are in sync

Replication settings that matter most.

5.1.13 Failure scenarios in practice

A replica loses the network for 30 seconds

It reconnects, sends its `replid` and offset, the primary finds those bytes still in the backlog, and a partial sync catches it up in milliseconds. If the primary wrote more than the backlog size in that time, you get a full sync instead. Watch the `sync_full` and `sync_partial_ok` / `sync_partial_err` counters in `INFO stats`: a rising `sync_full` on a stable system means the backlog is too small or the link is flaky.

Full sync makes the primary's memory grow

During a full sync the primary buffers new writes for the replica in its client output buffer. If the replica is slow, that buffer can grow large, and `client-output-buffer-limit replica` (default `256mb 64mb 60`) disconnects the replica when it is exceeded, which restarts the sync, which grows the buffer again. This loop is a classic cause of "replica can never finish syncing". Raise the limit, give the replica more CPU and network, or sync it during a quiet period.

Many replicas at once

If the primary restarts and ten replicas resync simultaneously, the primary produces one RDB (with diskless sync and its delay, or by sharing a disk-based RDB) but must push it ten times. The network card, not Redis, is the bottleneck. Chained replication or staggered restarts help.

REAL WORLD

A typical product-catalogue setup for ShopStream: one primary in the main zone, two replicas in two other availability zones, `min-replicas-to-write` left at 0 because the catalogue is a cache that can be rebuilt, and read traffic from the catalogue service sent to the replicas through the client's `ReadFrom` setting (see the Lettuce chapter). For the order stream, where losing writes hurts, the same topology runs with AOF on and the application uses `WAIT` for the few writes that must survive a failover.

INTERVIEW ANGLE

How does Redis replication work, and can it lose data? It is asynchronous: the primary applies a write, replies to the client, then streams the write to replicas. A new replica gets a full sync (RDB snapshot via fork, plus buffered writes); a reconnecting replica uses `PSYNC` with its replication ID and offset to receive only the missing bytes from the primary's backlog. Because the reply precedes the replication, writes that only reached the primary are lost if it fails and a replica is promoted. `WAIT` and `min-replicas-to-write` narrow that window but do not make Redis strongly consistent.

KEY POINTS

Replication is asynchronous and one-directional: primary → replicas, applied as a command stream.

- Full sync = fork + RDB + buffered writes; partial sync = replication ID + offset + backlog.
- Size `repl-backlog-size` from your write rate and the outage you want to ride out; the 1 MB default is small.
- Replicas can serve reads but those reads can be stale; do not use them for read-your-writes flows.
- `WAIT` and `min-replicas-to-write` reduce, but never eliminate, the window for lost writes.
- Replication is not a backup, and a non-persistent primary that restarts empty can wipe its replicas.

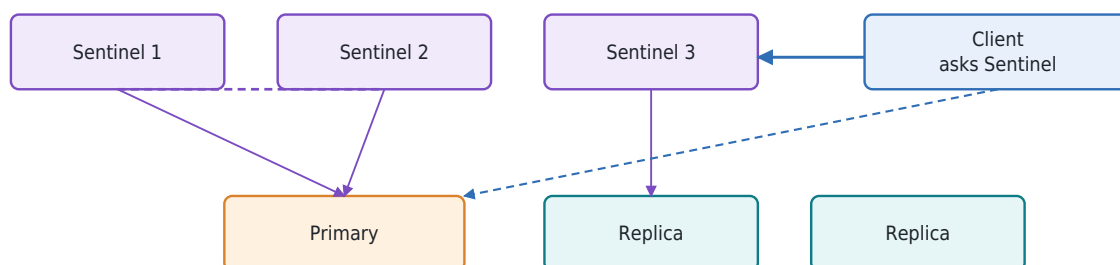
5.2 Sentinel: automatic failover

Replication gives you a copy of the data, but not a decision. When the primary dies at 3 a.m., somebody (or something) must notice, choose the best replica, promote it with `REPLICAOF NO ONE`, point the other replicas at it, and tell every client where the new primary lives. Doing that by hand means an outage that lasts as long as it takes a person to wake up and type correctly.

Redis Sentinel is a separate, small process (it is the same `redis-server` binary started in sentinel mode, or the `redis-sentinel` command) that automates exactly this. A group of Sentinels watches a primary and its replicas, agrees when the primary is down, elects one Sentinel to run the failover, promotes a replica, and acts as a **service-discovery endpoint**: clients ask Sentinel "who is the primary of `mymaster` right now?" instead of hard-coding an address.

Sentinel does not shard data. Every node still holds the full dataset. Sentinel is high availability for a single shard; if your data does not fit on one machine, you need Redis Cluster (later in this Part).

5.2.1 The roles



quorum of Sentinels agrees the primary is down, then one replica is promoted

Figure 5.2. Three Sentinels watching one primary and two replicas. Notice that the Sentinels also monitor each other, and that clients talk to Sentinels only to discover the primary, then to Redis directly for data.

A minimal, sensible deployment has:

- one primary and at least one (usually two) replicas, running as ordinary Redis servers;

- **three or more Sentinel processes**, an odd number, placed on **different machines or availability zones**, so that no single failure takes out a majority of them. Putting three Sentinels on one host protects against a Redis crash but not against that host failing, which defeats the purpose.

Sentinels discover each other and the replicas automatically. You only tell each Sentinel where the primary is; it learns the replicas from the primary's **INFO** output and other Sentinels through a Pub/Sub channel on the primary (`__sentinel__:hello`).

5.2.2 Detecting failure: SDOWN and ODOWN

Every Sentinel sends **PING** to the primary, replicas and other Sentinels once per second. Detection happens in two stages.

1. **Subjectively down (SDOWN)**. If one Sentinel gets no valid reply from an instance for **down-after-milliseconds**, it marks it SDOWN. This is a single Sentinel's opinion. It may be the Sentinel that has a network problem, not the primary.
2. **Objectively down (ODOWN)**. When a Sentinel sees the primary as SDOWN, it asks the other Sentinels (**SENTINEL is-master-down-by-addr**). If at least **quorum** Sentinels agree, the primary is flagged ODOWN and a failover can start.

The **quorum** is the number of Sentinels that must agree the primary is unreachable. This is the point where people get confused, so read this twice: **the quorum is only used to detect the failure**. To actually run a failover, one Sentinel must be elected leader, and that election needs votes from the **majority of all Sentinels** (more than half of those configured), regardless of the quorum setting.

Sentinels	Typical quorum	Majority needed	Consequence
3	2	2	Survives one Sentinel loss
5	3	3	Survives two Sentinel losses
5	2	3	Failure is detected easily, but with 3 Sentinels unreachable no failover can start
2	2	2	Not recommended: losing either one blocks any failover

Quorum and majority for different Sentinel counts.

If the Sentinels that can still talk to each other are fewer than the majority, no failover starts. That is the rule that prevents two halves of a partitioned network from both promoting a primary.

5.2.3 The failover, step by step

1. A Sentinel declares the primary ODOWN (quorum reached) and asks to be leader. Each Sentinel votes once per **configuration epoch** (a monotonically increasing number that versions the configuration). The Sentinel that collects votes from a majority becomes leader for that epoch. If nobody wins, they retry after a randomised delay (the retry interval relates to **failover-timeout**).
2. The leader **selects the best replica**. Replicas that were disconnected from the primary too long are disqualified. The rest are ordered by **replica-priority** (lower is better; priority 0 means never promote), then by replication offset (the one that received the most data wins), then by lexicographically smaller run ID.
3. The leader sends **REPLICAOF NO ONE** to the chosen replica, which becomes the new primary, and waits until it reports role **master**.
4. The leader reconfigures the other replicas to follow the new primary, at most **parallel-syncs** of them at a time, so most replicas keep serving reads while the others resync.
5. The new configuration is published with a higher epoch. All Sentinels adopt the **highest epoch** version of the configuration, which is how they converge on the same view. Sentinel publishes the **+switch-master** event on its Pub/Sub channel with the old and new addresses.
6. When the old primary comes back, Sentinels reconfigure it as a replica of the new primary. It will then do a resync (usually partial, thanks to the second replication ID) and discard any writes it accepted that the new primary never saw.

Typical end-to-end failover time is **down-after-milliseconds** (detection) plus a few seconds for election and promotion. The default **down-after-milliseconds** is 30000 (30 seconds), so out of the box a dead primary is noticed after 30 seconds. Lower values mean faster failover and more false alarms from slow networks, GC-style pauses of the machine, or short blips.

5.2.4 Configuration

Here is a Sentinel configuration for three Sentinels watching ShopStream's session store. All three use the same file (with their own `port` if they share a host).

```
sentinel.conf CONF
port 26379
sentinel monitor shop-sessions 10.0.1.10 6379 2
sentinel down-after-milliseconds shop-sessions 5000
sentinel failover-timeout shop-sessions 60000
sentinel parallel-syncs shop-sessions 1
sentinel auth-pass shop-sessions s3cr3t-change-me
requirepass sentinel-s3cr3t
```

What to notice:

- `sentinel monitor <name> <ip> <port> <quorum>` is the only mandatory line. The name is how clients refer to the primary; the quorum (2 of 3 here) is for failure detection only.
- Every other option has the form `sentinel <option> <name> <value>`. Defaults are `down-after-milliseconds 30000`, `failover-timeout 180000` and `parallel-syncs 1`. The `failover-timeout` bounds how long a failover attempt may take and how long Sentinels wait before retrying a failover that failed.
- `sentinel auth-pass` is the password Sentinel uses to log in to the Redis nodes. It must match `requirepass` (or the ACL user, with `sentinel auth-user`) on every node, replicas included, and the replicas need `masterauth` set so they can follow a new primary. Missing `masterauth` on a replica is a very common cause of a failover that "works" but leaves replicas disconnected.
- `requirepass` here protects the Sentinel itself, which clients and other Sentinels connect to on port 26379.

Start a Sentinel with `redis-sentinel /etc/redis/sentinel.conf` (or `redis-server sentinel.conf --sentinel`).

GOTCHA

Sentinel rewrites its own configuration file. It records the replicas and Sentinels it has discovered, the current epoch and the current primary's address into `sentinel.conf`. The file must therefore be writable, and you must not copy a used `sentinel.conf` to a new machine or bake it into an image: duplicated "myid" lines make Sentinels confuse each other. Give each Sentinel a fresh minimal file.

5.2.5 Asking Sentinel questions

```
Querying a Sentinel TERMINAL
$ redis-cli -p 26379
127.0.0.1:26379> SENTINEL get-master-addr-by-name shop-sessions
1) "10.0.1.10"
2) "6379"
127.0.0.1:26379> SENTINEL replicas shop-sessions
127.0.0.1:26379> SENTINEL sentinels shop-sessions
127.0.0.1:26379> SENTINEL ckquorum shop-sessions
OK 3 usable Sentinels. Quorum and failover authorization can be reached
127.0.0.1:26379> SENTINEL failover shop-sessions
OK
```

`SENTINEL replicas` and `SENTINEL sentinels` list the known nodes with their flags and offsets, `SENTINEL ckquorum` checks that quorum and majority are reachable (run it in monitoring and after changes), and `SENTINEL failover <name>` triggers a failover manually, without needing agreement that the primary is down. The latter is the right way to move the primary before maintenance. The older name for the first command's replica listing was `SENTINEL slaves`; the current command is `SENTINEL replicas`.

You can also subscribe to a Sentinel to receive events such as `+sdown`, `+odown`, `+failover-end` and `+switch-master`:

```
REDIS
PSUBSCRIBE *
# +switch-master shop-sessions 10.0.1.10 6379 10.0.1.11 6379
```


5.2.6 Client discovery

Sentinel only works if your clients cooperate. A Sentinel-aware client is configured with the list of Sentinel addresses and the primary's name, never with the primary's address. On startup, and after a connection failure, the client asks a Sentinel `SENTINEL get-master-addr-by-name`, connects to the answer, and verifies that the node reports itself as primary with `ROLE`. A good client also subscribes to `+switch-master` so it reconnects immediately.

In Spring Boot (Lettuce is the default client) this is configuration only:

```
application.yml
spring:
  data:
    redis:
      password: s3cr3t-change-me
      sentinel:
        master: shop-sessions
        nodes:
          - 10.0.2.21:26379
          - 10.0.2.22:26379
          - 10.0.2.23:26379
      password: sentinel-s3cr3t
```

Notice that `spring.data.redis.password` is for the Redis data nodes and `spring.data.redis.sentinel.password` is for the Sentinels, a distinction that regularly costs an hour. Spring and the Java clients are covered in their own Parts, including how to read from replicas and how clients behave while a failover is in progress (commands fail for a few seconds, so your retry and timeout settings matter).

5.2.7 Following a failover through its events

Every state change in a failover is published as an event on a Sentinel Pub/Sub channel named after the event, which makes failovers observable and scriptable. You will see these, roughly in this order:

Event	Meaning
+sdown / +odown	This Sentinel (or enough Sentinels) consider the primary down
+new-epoch	A new configuration epoch started
+try-failover	This Sentinel wants to run the failover
+vote-for-leader / +elected-leader	The leader election and its winner
+failover-state-select-slave / +selected-slave	The replica to promote was chosen
+failover-state-send-slaveof- noone / +promoted-slave	<code>REPLICAOF NO ONE</code> was sent and the replica became primary
+failover-state-reconf-slaves	Other replicas are being pointed to the new primary
+failover-end	The failover completed
+switch-master	The configuration changed: name, old address, new address. The one most applications care about

Sentinel events in the life of a failover.

The event names still contain the word `slave` in some places for compatibility. Monitoring tools usually alert on `+odown`, `+switch-master` and on any `-failover-abort-*` events (failover attempts that gave up, for example when no suitable replica exists).

5.2.8 Operating Sentinel

Adding and removing nodes

New replicas need no Sentinel configuration: start them with `replicaof` pointing at the current primary and the Sentinels discover them within seconds. A new Sentinel only needs the same `sentinel monitor` line; it finds the others through the primary's hello channel. When you retire a node, Sentinels remember it as down forever unless you tell them to forget: run `SENTINEL RESET <name>` (or `SENTINEL RESET *`) on each Sentinel to clear stale replicas and Sentinels from its state, one Sentinel at a time with a pause in between. To stop monitoring a primary altogether use `SENTINEL REMOVE <name>`.

Changing settings at runtime

`SENTINEL SET <name> down-after-milliseconds 5000` changes a monitored primary's settings on that Sentinel and persists it in the configuration file. Remember that a setting like this is **per Sentinel**: apply it to all of them, or they will disagree. Sentinel-wide parameters can be read and set with `SENTINEL CONFIG GET` and `SENTINEL CONFIG SET` (Redis 6.2 and later).

Scripts and notifications

Sentinel can run an executable on events: `sentinel notification-script <name> <path>` is called for warnings (for example `+sdown`, `+odown`) so you can page someone, and `sentinel client-reconfig-script <name> <path>` is called when the primary changes so you can update a DNS record, a load balancer or a configuration store. Scripts must exit quickly, must be executable, and must exit with code 0 on success. In many modern deployments the clients' Sentinel support and the Pub/Sub events replace the need for these scripts, but they remain the way to integrate non-Sentinel-aware software.

Security

Sentinels expose the same kind of control-plane commands as Redis servers (including `SENTINEL FAILOVER` and `SENTINEL SET`), so protect them like servers: private network only, `requirepass` or ACLs, TLS if your Redis nodes use it, and no exposure to the internet. A client that can reach and command a Sentinel can redirect all traffic to a server of its choice.

5.2.9 What your application sees during a failover

From the application's side, a failover is a window of a few seconds in which:

- commands to the old primary time out or fail with connection errors;
- connections to replicas that are being reconfigured drop and reconnect, because they restart a sync;
- `READONLY` errors can appear if a client still talks to an old primary that has been demoted to a replica;
- in-flight writes whose acknowledgement was lost have an unknown outcome: they might have happened, and might not be on the new primary.

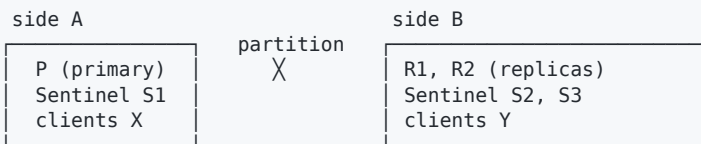
A robust application therefore uses command timeouts that are shorter than the user's patience, retries only operations that are **idempotent** (or protects them with idempotency keys; see the idempotency chapter in the Patterns Part), and has a defined degraded mode if Redis is unavailable. The Java clients Part shows how to configure reconnection and timeouts, and the failover test in the Sentinel lab lets you observe all of this with your own eyes.

REAL WORLD

Teams that run Sentinel in production commonly schedule a **game day**: kill the primary in staging while a load generator runs, and measure three numbers: time until the first successful write on the new primary, how many writes failed, and whether any acknowledged write is missing afterwards. If any of these surprises the team, the application settings (timeouts, retries, idempotency) need work before the next real failure.

5.2.10 Split brain and what Sentinel cannot promise

Imagine primary P with two replicas, and a network partition that isolates P (and one Sentinel) from the other two Sentinels and the replicas:



S2 and S3 reach ODOWN (quorum 2) and form a majority: they promote R1. Now P and R1 are BOTH primaries. Clients X keep writing to P; those writes are lost when the partition heals.

This situation is called **split brain**. Sentinel prevents the minority side from starting a failover, but it cannot stop an isolated old primary from continuing to accept writes from clients that can still reach it. When the partition heals, the old primary is demoted to a replica and resyncs, which **discards** everything it accepted in the meantime.

You can bound the damage with the settings from the replication chapter:

redis.conf (primary and replicas)

CONF

```
min-replicas-to-write 1
min-replicas-max-lag 10
```

With these, an isolated primary stops accepting writes about 10 seconds after it loses its replicas, so the loss window is limited. The trade-off is that if all replicas fail, the primary also stops accepting writes. Even then the data loss is bounded, not eliminated.

Other limitations to keep in mind:

- **No sharding.** Dataset size and write throughput are limited to one node.
- **Failover is not instant.** Expect seconds of failed or retried commands, and design callers for it.
- **Writes can be lost.** Everything from the replication chapter still applies: replication is asynchronous.
- **Addresses must be stable and reachable.** Under Docker, NAT or Kubernetes, the addresses a Sentinel announces (`sentinel announce-ip`, `replica-announce-ip`) must be ones the clients can actually reach. By default Sentinel works with IP addresses; hostnames need `sentinel resolve-hostnames yes` (disabled by default), and `sentinel announce-hostnames yes` if you want hostnames reported to clients.
- **Sentinel can enter TILT mode.** If a Sentinel notices the system clock jumped or its own process was frozen for a long time, it temporarily stops acting (it still monitors but will not start failovers) so that a stalled machine does not trigger a bogus failover.
- **Sentinels need majority.** Losing two of three Sentinels means no automatic failover until one comes back, even though Redis itself keeps serving.

WATCH OUT

Run Sentinel on separate failure domains from the nodes it watches whenever you can, and test it. A failover you have never rehearsed (`SENTINEL failover`, or killing the primary in a staging environment) is a failover you do not know works. The Sentinel lab in the Hands-On Labs Part walks through exactly this.

INTERVIEW ANGLE

What is the difference between quorum and majority in Sentinel? Quorum is how many Sentinels must agree the primary is unreachable before it is marked objectively down and a failover is considered. Majority is how many Sentinels must vote to elect the leader that actually performs the failover; it is more than half of all Sentinels regardless of quorum. Detection needs the quorum, authorization to act needs the majority, which is why a minority partition can never promote a replica.

KEY POINTS

Sentinel gives one shard automatic failover and service discovery; it does not shard data.

- Run three or more Sentinels on different failure domains; quorum detects failure, majority authorizes failover.
- SDOWN is one Sentinel's opinion; ODOWN needs quorum; a leader is elected per configuration epoch.
- The new primary is chosen by replica priority, then replication offset, then run ID.
- Clients must be Sentinel-aware: configure Sentinel addresses and the master name, not the primary's IP.
- Split brain can still lose writes; bound it with `min-replicas-to-write` and `min-replicas-max-lag`.

5.3 Why Cluster: sharding basics

Replication and Sentinel make Redis more available and add read capacity, but every node still holds the **whole dataset** and every write still goes to **one** primary. Three limits remain:

- **Memory.** The dataset must fit in the RAM of one machine (minus headroom for fork and copy-on-write during snapshots). A 400 GB dataset does not fit on a 256 GB server, and very large instances also make restarts, full syncs and backups painfully slow.
- **Write throughput.** One primary executes commands on essentially one core. A single Redis node can do very high operation counts per second, but there is a ceiling, and a few expensive commands can use it up.
- **Network bandwidth.** All traffic to the dataset flows through one network interface.

You can first move the ceiling by scaling **up**: a bigger machine, faster CPU, better network, tuned data structures. That is simple and often the right first move. When it stops being enough, you scale **out**: split the dataset into **shards**, each owned by a different primary, so memory, writes and bandwidth are spread across machines. The word for this is **sharding** (or partitioning).

5.3.1 Three ways to shard

Client-side sharding

The application (or its client library) decides which Redis server holds a key, usually with a hash of the key. The servers know nothing about each other. The simplest form is `hash(key) mod N`, which breaks badly when N changes: adding one server moves almost every key. **Consistent hashing** reduces that to moving roughly $1/N$ of the keys, and is what many older libraries used.

Client-side sharding costs nothing on the server side and adds no extra hop, but every client must share exactly the same view of the topology, there is no automatic failover, and resharding is your problem: you have to copy keys yourself and coordinate the switch.

Proxy-assisted sharding

A **proxy** sits between clients and the shards, speaks the Redis protocol, and routes each command to the right server. Clients talk to the proxy as if it were one big Redis. Well-known examples are Twemproxy (from Twitter) and Envoy's Redis filter. This keeps clients simple, but it adds a network hop (and latency), the proxy is one more component to run and make highly available, and proxies usually restrict multi-key commands, transactions and some other features.

Redis Cluster

Redis Cluster is the **built-in, server-side** solution. The nodes know about each other, divide the key space into **16384 hash slots**, own subsets of the slots, detect failures and promote replicas by themselves, and tell clients (with redirections) when they asked the wrong node. There is no proxy. Clients are "smart": they learn which node owns which slots and talk directly to the right one.

Aspect	Client-side	Proxy	Redis Cluster
Routing done by	Client library	Proxy process	Client, guided by server redirects
Extra network hop	No	Yes	No (in steady state)
Automatic failover	No (add Sentinel per shard)	Depends on setup	Yes, built in
Rebalancing data	Manual	Manual or tool-specific	Built-in slot migration
Multi-key commands	Only on the same server	Limited	Only within one slot
Operational surface	Application code	Proxy fleet + Redis	Redis only

Sharding approaches compared.

5.3.2 Hash slots instead of hashing servers

The key idea of Redis Cluster is an **indirection layer**. Instead of mapping keys directly to servers, keys map to one of 16384 **slots**, and slots map to servers:

key $\xrightarrow{\text{CRC16, mod 16384}}$ slot (0..16383) $\xrightarrow{\text{table lookup}}$ node

Adding a node does not change any key's slot.
It only changes which node owns some of the slots.

This is what makes resizing practical. Moving data from node A to node B means moving a given slot (a bucket of keys), not rehashing everything. A key's slot never changes, so a slot is the unit of ownership, of migration and of failure.

NOTE

Redis Cluster is built for scale and availability, not for transparency. Anything that touches several keys is only guaranteed to work when the keys are in the same slot. The next chapter spells out what that means for your code, and the key-design trick (hash tags) that fixes most cases.

KEY POINTS

Scale up first; scale out when memory, write throughput or bandwidth of one node is the limit.

- Sharding can be done in the client, in a proxy, or by the database (Redis Cluster).
- $\text{hash}(\text{key}) \bmod N$ is a bad scheme because changing N remaps nearly every key.
- Redis Cluster adds an indirection: $\text{key} \rightarrow \text{one of 16384 slots} \rightarrow \text{node}$, so growing the cluster moves slots, not hashes.
- You trade away free multi-key operations in exchange for horizontal scale.

5.4 Redis Cluster

Redis Cluster is a set of Redis nodes that together present one logical database. Every node runs in cluster mode (`cluster-enabled yes`), keeps a full map of which node owns which slot, and talks to all other nodes over a dedicated TCP channel. There is no coordinator and no proxy; the nodes themselves form the control plane.

5.4.1 Slots and key hashing

slot = CRC16(key) mod 16384 (hash tag {...}: only the part in braces is hashed)

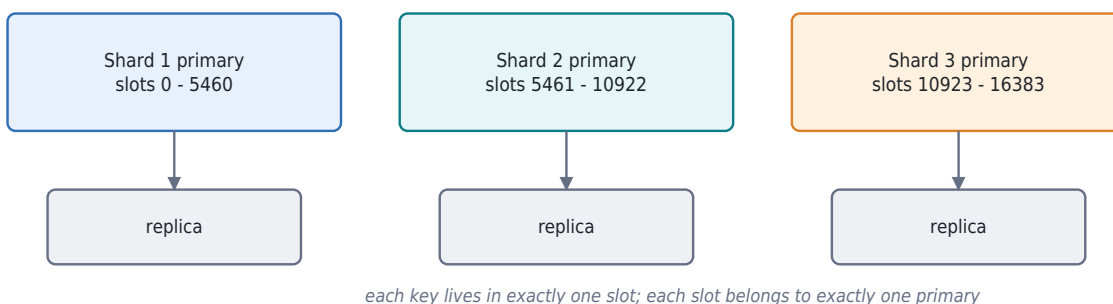


Figure 5.3. The 16384 hash slots divided across three primaries, each with a replica. Notice that slots, not keys, are assigned to nodes.

The slot of a key is computed as:

`HASH_SLOT = CRC16(key) mod 16384`

With three primaries, the usual split is 0 to 5460, 5461 to 10922 and 10923 to 16383. You can ask any node for the slot of a key:

Which slot does a key belong to?

TERMINAL

```
127.0.0.1:17001> CLUSTER KEYSLOT user:42:cart
(integer) 12984
127.0.0.1:17001> SET user:42:cart x
(error) MOVED 12984 127.0.0.1:17003
```

The **SET** was sent to the node that does not own slot 12984, so it answers with a redirection (**MOVED**) rather than guessing. We will come back to that.

Why 16384 and not a larger number? Nodes exchange heartbeat messages that include the sender's slot ownership as a bitmap, and 16384 slots take 2 KB per message. The designers chose a number that keeps messages small and is still far more than the number of primaries (the practical upper limit is in the order of a thousand nodes) needed for even balancing.

Hash tags

By default the whole key is hashed. If the key contains a substring in curly braces, **{...}**, only the part between the **first {** and the **next }** is hashed, provided it is not empty. This lets you force related keys into the same slot:

Hash tags keep related keys together

TERMINAL

```
127.0.0.1:17001> CLUSTER KEYSLOT {user:42}:cart
(integer) 15880
127.0.0.1:17001> CLUSTER KEYSLOT {user:42}:session
(integer) 15880
127.0.0.1:17001> CLUSTER KEYSLOT user:42:cart
(integer) 12984
```

With **{user:42}:cart** and **{user:42}:session** in the same slot, you can read both in one **MGET**, change both in a **MULTI/EXEC** transaction, or touch both from one Lua script. That is how ShopStream keeps a user's cart, session and recent-searches list atomically operable together.

WATCH OUT

Hash tags concentrate load. Everything sharing a tag lives on one node, so a tag like **{tenant:acme}** makes one primary hold, and serve, an entire big tenant, and a tag that is too coarse (**{cart}**) puts everything on one shard and silently destroys the benefit of the cluster. Tag by something with many distinct values (a user id, an order id), and keep the group of keys under the same tag small.

5.4.2 Creating a cluster

A working cluster needs at least **three primaries** (so that a majority can agree a node has failed) and, for high availability, at least one replica per primary: six nodes. Each node needs a config like this:

redis-17001.conf

CONF

```
port 17001
cluster-enabled yes
cluster-config-file nodes-17001.conf
cluster-node-timeout 5000
appendonly yes
```

cluster-config-file is not something you write: the node maintains it automatically (it stores node ids, slots and epochs), and it must be writable and persistent. **cluster-node-timeout** is the central timeout, explained below. The default is 15000 ms; 5000 is a common production value that makes failover faster.

After starting all six, **redis-cli** can wire them up:

Create a 6-node cluster (3 primaries, 3 replicas)

TERMINAL

```
$ redis-cli --cluster create 127.0.0.1:17001 127.0.0.1:17002 127.0.0.1:17003 \
  127.0.0.1:17004 127.0.0.1:17005 127.0.0.1:17006 --cluster-replicas 1
>>> Performing hash slots allocation on 6 nodes...
Master[0] -> Slots 0 - 5460
Master[1] -> Slots 5461 - 10922
Master[2] -> Slots 10923 - 16383
Adding replica 127.0.0.1:17005 to 127.0.0.1:17001
Adding replica 127.0.0.1:17006 to 127.0.0.1:17002
Adding replica 127.0.0.1:17004 to 127.0.0.1:17003
Can I set the above configuration? (type 'yes' to accept): yes
>>> Nodes configuration updated
>>> Assign a different config epoch to each node
>>> Sending CLUSTER MEET messages to join the cluster
Waiting for the cluster to join
.
>>> Performing Cluster Check (using node 127.0.0.1:17001)
```

Create a 6-node cluster (3 primaries, 3 replicas) (continued)

TERMINAL

```
[OK] All nodes agree about slots configuration.
>>> Check for open slots...
>>> Check slots coverage...
[OK] All 16384 slots covered.
```

Here the tool split the slots into three roughly equal ranges, assigned each replica to a primary (preferring a different host from its primary when it can; when everything is on one host, as in this demo, it prints a warning), and made the nodes meet each other with **CLUSTER MEET**. Check the result any time with:

Inspecting the cluster

TERMINAL

```
$ redis-cli --cluster check 127.0.0.1:17001
127.0.0.1:17001> CLUSTER INFO
cluster_state:ok
cluster_slots_assigned:16384
cluster_slots_ok:16384
cluster_slots_pfail:0
cluster_slots_fail:0
cluster_known_nodes:6
cluster_size:3
cluster_current_epoch:6
cluster_my_epoch:1
127.0.0.1:17001> CLUSTER NODES
6c12bd9a... 127.0.0.1:17001@27001 myself,master - 0 0 1 connected 0-5460
e0fc44d2... 127.0.0.1:17002@27002 master - 0 1791613452613 2 connected 5461-10922
6b793cc4... 127.0.0.1:17004@27004 slave 6c12bd9a... 0 1791613452000 1 connected
```

In **CLUSTER NODES**, the **@27001** part is the **cluster bus port**, covered in a moment; **master/slave** (the output still uses the old word) and the slot ranges tell you who owns what. **CLUSTER SHARDS** gives similar information in a structured form, grouped by shard (a primary and its replicas), and is easier for tools to parse.

5.4.3 How requests are routed: MOVED and ASK

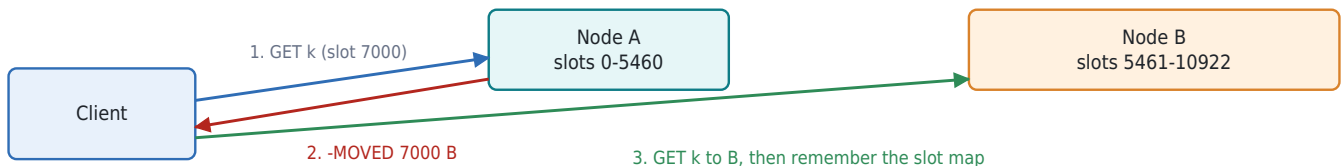


Figure 5.4. A client sends a command to the wrong node, receives **MOVED**, updates its slot map and retries. During a slot migration the old owner answers **ASK** instead. Notice that **MOVED** is permanent information, **ASK** is a one-time redirect.

A client may connect to any node and send any command. The node looks at the key's slot:

- If it owns the slot, it executes the command.
- Otherwise it replies **MOVED <slot> <ip>:<port>**, meaning "this slot is permanently served by that node". A cluster-aware client updates its slot table and re-sends the command there.
- During a migration of the slot, a node that has already moved a key away may reply **ASK <slot> <ip>:<port>**. This is a *temporary* redirect: the client sends **ASKING** followed by the command to the target node, **once**, and does **not** update its table.

redis-cli does not follow redirects unless you start it with **-c** (cluster mode):

Without and with -c

TERMINAL

```
$ redis-cli -p 17001 SET foo bar
(error) MOVED 12182 127.0.0.1:17003
$ redis-cli -c -p 17001 SET foo bar
-> Redirected to slot [12182] located at 127.0.0.1:17003
OK
```

Real clients (Lettuce, Jedis, Redisson, redis-py, go-redis, and so on) do this for you. They fetch the slot map when they connect (**CLUSTER SLOTS** or **CLUSTER SHARDS**), compute **CRC16(key) mod 16384** locally, and send each command directly to the owner, so in steady state there is no extra hop. They refresh the map when they see **MOVED** and, in better

clients, periodically and on topology events. The Java clients Part shows how to turn on topology refresh in Lettuce, which matters during failovers and resharding.

GOTCHA

A client that does not know Redis Cluster (a plain connection to one node) works only for the keys of that node and fails with **MOVED** on the rest. When someone says "Redis works locally but breaks in production", check that production is a cluster and that the client is cluster-aware.

5.4.4 The cluster bus and gossip

Besides the normal port (6379 by default), every node opens a second TCP port, the **cluster bus**. By default it is the data port plus 10000 (6379 → 16379, as 17001 → 27001 above). It can be set with `cluster-port`. Nodes use a binary protocol on this bus to exchange heartbeat and gossip messages, to propagate configuration and slot changes, to run failure detection and to coordinate failovers. Clients must never connect to the bus port, but **both ports must be reachable between all nodes** (firewalls, security groups and Kubernetes network policies are the usual culprits when a new cluster will not form).

Gossip works as follows. Each node continuously sends **PING** to a few other nodes and answers with **PONG**; each message carries the sender's slots, its configuration epoch, its flags and information about a random sample of other nodes it knows. Over time every node learns about every other, and changes spread quickly without any central coordinator.

5.4.5 Failure detection and failover

Failures are detected in two stages that resemble Sentinel's, but all done by the cluster's own nodes.

1. **PFAIL (possible failure)**. A node that has not received a reply from another node for `cluster-node-timeout` milliseconds flags it as PFAIL. This is a local opinion.
2. **FAIL**. When a node learns, through gossip, that a **majority of primaries** consider the same node PFAIL, it marks the node FAIL and broadcasts that to the whole cluster. Only primaries count in this vote.

When a primary is flagged FAIL, its replicas start an election:

1. A replica waits a small delay that depends on how up to date it is (the one with the largest replication offset starts first) plus a random component.
2. It increments the cluster's current epoch and asks the primaries for votes. Each primary votes at most once per epoch.
3. When it collects votes from the **majority of primaries**, it promotes itself, takes over the old primary's slots, and announces the new configuration with a higher config epoch. Other nodes accept the configuration with the higher epoch.
4. The old primary, when it returns, sees that its slots have a newer owner and becomes a replica of the winner.

A replica that has been disconnected from its primary for too long is not allowed to fail over because its data would be too old. The limit is roughly `cluster-node-timeout × cluster-replica-validity-factor` (factor default 10) plus the replica ping period. Setting the factor to 0 means a replica always tries.

In the demo cluster, killing one primary (17003) leads, after about the node timeout plus the election, to this view; its replica (17006) is now a primary owning the slots:

After shutting down 17003 and waiting a few seconds

TERMINAL

```
$ redis-cli -p 17001 CLUSTER NODES | awk '{print $2, $3, $8, $9}'
127.0.0.1:17001@27001 myself,master connected 0-5460
127.0.0.1:17002@27002 master connected 5461-10922
127.0.0.1:17004@27004 slave connected
127.0.0.1:17003@27003 master,fail disconnected
127.0.0.1:17005@27005 slave connected
127.0.0.1:17006@27006 master connected 10923-16383
```

The demo uses a 5 second node timeout, so detection and election took well under ten seconds. The output of a real run varies with timing, but the pattern is always the same: **master**, **fail** for the dead node, and the replica turned **master** with the slot range.

When the cluster stops accepting commands

The cluster as a whole is in state `ok` or `fail`:

- If a **majority of primaries** cannot talk to each other (a partition), the nodes on the minority side stop serving writes after the node timeout. They cannot be sure that the majority side has not promoted a replica, so they refuse rather than risk diverging.
- If some slot has no owner (a primary failed and had **no replica** that could take over), then by default (`cluster-require-full-coverage yes`) the **entire cluster** stops accepting commands, even for keys in healthy slots. Setting it to `no` lets the cluster serve the slots that are still covered, which is often the better trade-off for a cache but wrong for data you need to be consistent.
- `cluster-allow-reads-when-down` (default `no`) can let nodes serve reads while the cluster is marked failed. `cluster-allow-pubsubshard-when-down` (default `yes`) keeps sharded Pub/Sub working.

Like Sentinel, the cluster is asynchronously replicated, so writes acknowledged by a primary that then fails before they reach a replica are lost. `WAIT` helps in the same limited way. There is also a short window in a partition where a primary with clients on its side keeps accepting writes until the node timeout fires; `cluster-node-timeout` is therefore also the knob for how much you lose in such a split.

Replica migration

If one primary loses all its replicas while another has several, a spare replica can move automatically to the orphan. `cluster-migration-barrier` (default 1) is the minimum number of replicas a primary must keep before it donates one, and `cluster-allow-replica-migration` (default `yes`) controls whether nodes may do it. This gives you "N+1" resilience with fewer replicas than a full pair for every primary.

5.4.6 What you give up: multi-key rules

Only commands whose keys are all in the **same slot** work across several keys. Otherwise the server returns:

```
The CROSSLIST error
127.0.0.1:17001> MSET a 1 b 2
(error) CROSSLIST Keys in request don't hash to the same slot
127.0.0.1:17001> MSET {user:42}:a 1 {user:42}:b 2
OK
```

This applies to `MGET`, `MSET`, `DEL` with many keys, set operations (`SINTER`, `SUNION...`), `ZUNIONSTORE`, `RENAME`, `LMOVE`, and also to transactions (`MULTI/EXEC`) and to Lua scripts and functions: every key a script touches must be declared in `KEYS` and be in one slot. A script that builds key names at run time and touches another slot can fail or, worse, behave differently in a cluster than on your laptop.

The other differences you will meet:

- **Only database 0.** `SELECT` with another number is an error.
- **Commands that scan the key space work per node.** `KEYS`, `SCAN`, `DBSIZE`, `FLUSHALL` and `RANDOMKEY` act only on the node you ask; to cover everything you must run them on every primary (tools such as `redis-cli --cluster call` do that).
- **Pub/Sub.** A classic `PUBLISH` is propagated to every node over the bus, which does not scale with the node count. **Sharded Pub/Sub** (`SPUBLISH` / `SSUBSCRIBE`, since Redis 7.0) hashes the channel name to a slot and keeps messages within one shard.
- **Reading from replicas.** A replica redirects reads for its slots to the primary, unless the client first sends `READONLY` on that connection (Lettuce and Jedis can do this when you configure a read preference). Stale reads apply as in the replication chapter.
- **Big keys and hot slots.** A single key never splits across nodes, so a huge sorted set or a hot key stays on one primary. Sharding does not fix those; data modelling does.

GOTCHA

Code that works on a standalone Redis and fails in a cluster is nearly always one of three things: a multi-key command across slots, a Lua script touching undeclared keys, or an assumption that `SCAN/KEYS` see the whole database. Develop and test against a cluster (Testcontainers can start one) if production is one.

5.4.7 Using a cluster from Spring Boot

For application code, the cluster is mostly configuration. List a few seed nodes (the client discovers the rest) and the maximum number of redirects it should follow for one command:

```
application.yml
spring:
  data:
    redis:
      password: s3cr3t-change-me
      cluster:
        nodes:
          - 10.0.1.1:6379
          - 10.0.1.2:6379
          - 10.0.1.3:6379
      max-redirects: 3
```

One detail will bite you in production: by default Lettuce reads the slot map once when it connects and refreshes it only when it needs to. If a primary fails and a replica takes over, or you reshard, a client with stale topology keeps sending commands to the old node. The remedy is to enable periodic and adaptive topology refresh:

```
RedisClusterConfig.java
package com.shopstream.config;

import java.time.Duration;

import io.lettuce.core.cluster.ClusterClientOptions;
import io.lettuce.core.cluster.ClusterTopologyRefreshOptions;
import org.springframework.boot.data.redis.autoconfigure.*;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RedisClusterConfig {

    @Bean
    LettuceClientConfigurationBuilderCustomizer clusterRefresh() {
        ClusterTopologyRefreshOptions refresh = ClusterTopologyRefreshOptions.builder()
            .enablePeriodicRefresh(Duration.ofSeconds(30))
            .enableAllAdaptiveRefreshTriggers()
            .build();
        ClusterClientOptions options = ClusterClientOptions.builder()
            .topologyRefreshOptions(refresh)
            .build();
        return builder -> builder.clientOptions(options);
    }
}
```

What to notice: `enablePeriodicRefresh` re-reads the topology on a timer, and `enableAllAdaptiveRefreshTriggers` makes the client refresh immediately when it sees a `MOVED` redirect, persistent reconnect failures and similar signs of a topology change. Together they shorten the time a client keeps asking the wrong node after a failover. The Lettuce chapter in the Java clients Part goes deeper on these options.

5.4.8 A worked example: flash-sale stock in a cluster

ShopStream's flash sale decrements the stock of a SKU and records the reservation atomically. In a standalone Redis you would use two keys and one Lua script. In a cluster, both keys must share a slot, so they share a hash tag:

```
reserve.lua
-- KEYS[1] = {sku:1001}:stock   KEYS[2] = {sku:1001}:reserved:<orderId>
-- ARGV[1] = quantity          ARGV[2] = reservation TTL in seconds
local stock = tonumber(redis.call('GET', KEYS[1]) or '0')
local qty = tonumber(ARGV[1])
if stock < qty then
    return 0
end
redis.call('DECRBY', KEYS[1], qty)
redis.call('SET', KEYS[2], qty, 'EX', tonumber(ARGV[2]))
return 1
```

Both keys contain `{sku:1001}`, so they hash to the same slot and the script is legal in a cluster. The design consequence is deliberate: all traffic for one SKU is served by one primary. For a million different SKUs that spreads evenly; for one extremely hot SKU it does not, and that single slot becomes the bottleneck (see the flash-sale pattern and project for ways to deal with it, for example splitting stock into several sub-counters, each with its own tag).

5.4.9 Persistence, backups and configuration in a cluster

Each node persists independently. Use the same persistence policy you would use on a standalone node (see the persistence chapters in Part 4), on primaries and, usually, on replicas. A cluster backup is a set of per-node RDB files taken at roughly the same time; there is no single consistent snapshot of the whole cluster, so restoring means restoring every shard from backups of the same period. Treat `nodes.conf` as part of each node's state: keep it on persistent storage (a volume, not the container's temporary file system), otherwise a restarted node comes back with a new identity.

In containers and behind NAT, tell the node what address to advertise to its peers and to clients, with `cluster-announce-ip`, `cluster-announce-port`, `cluster-announce-bus-port`, or `cluster-announce-hostname` together with `cluster-preferred-endpoint-type` (default `ip`). If clients receive addresses in `MOVED` replies or in the topology that they cannot reach (for example container-internal IPs), every redirect will fail.

5.4.10 CLUSTER commands you will meet

Command	Purpose
CLUSTER INFO	State (<code>ok</code> or <code>fail</code>), slot counts, epochs, message statistics
CLUSTER NODES / CLUSTER SHARDS	Topology: node ids, roles, slots, link state; SHARDS groups by shard
CLUSTER SLOTS	Older slot-range-to-node listing, still used by clients
CLUSTER KEYSLOT key	The slot of a key
CLUSTER COUNTKEYSINSLOT slot / GETKEYSINSLOT slot count	Number of keys / some keys in a slot
CLUSTER MYID	This node's id
CLUSTER MEET ip port	Introduce a node to the cluster
CLUSTER REPLICATE node-id	Make this node a replica of a primary
CLUSTER FAILOVER [FORCE or TAKEOVER]	Run on a replica to take over (see the operations chapter)
CLUSTER FORGET node-id	Remove a node from this node's table
CLUSTER RESET [HARD or SOFT]	Reset a node's cluster state (use with care)
CLUSTER SETSLOT slot IMPORTING, MIGRATING, NODE, STABLE	Low-level slot ownership changes, normally driven by tools

Frequently used CLUSTER subcommands.

5.4.11 Resharding with redis-cli

Moving slots between nodes while the cluster serves traffic is called **resharding**. `redis-cli --cluster reshard` does it interactively or from the command line:

```

Move 100 slots from all primaries to 17001 TERMINAL
$ redis-cli --cluster reshard 127.0.0.1:17001 \
  --cluster-from all --cluster-to 6c12bd9ac044ca22cef2c4047c22c16143bc779d \
  --cluster-slots 100 --cluster-yes
...
Moving slot 10970 from 127.0.0.1:17006 to 127.0.0.1:17001:
Moving slot 10971 from 127.0.0.1:17006 to 127.0.0.1:17001:

```

For each slot, the tool marks it as migrating on the source and importing on the target (`CLUSTER SETSLOT`), copies the keys with `MIGRATE` in batches, and finally assigns the slot to the new owner. During that time, a request for a key not yet moved is answered by the source, and a request for a key already moved gets an `ASK` redirect. The next chapter covers the improved mechanism in Redis 8.4 and the day-to-day operations of growing and shrinking a cluster.

redis-cli --cluster at a glance

Subcommand	What it does
create	Build a new cluster from empty nodes (<code>--cluster-replicas N</code> adds N replicas per primary)
check	Verify all slots are covered and that nodes agree on the configuration
info	Short summary: keys and slots per primary
reshard	Move a chosen number of slots from some nodes to another
rebalance	Spread slots evenly (or by <code>--cluster-weight</code>); <code>--cluster-simulate</code> previews it
add-node / del-node	Add a node (as primary, or as replica with <code>--cluster-slave</code>) / remove an empty node
fix	Repair open slots after an interrupted migration
call	Run one command on every node

The subcommands you will use most.

INTERVIEW ANGLE

How does Redis Cluster decide where a key lives, and what happens if a client asks the wrong node? It computes `CRC16(key) mod 16384` to get a slot (using only the `{hash tag}` if the key has one), and each primary owns a set of slots. A node that does not own the slot replies `MOVED slot host:port`, and a cluster-aware client updates its slot map and retries there; during a slot migration the source may reply `ASK`, which is a one-off redirect. Multi-key operations only work when all keys are in the same slot, which is why hash tags exist.

INTERVIEW ANGLE

Why does a Redis Cluster need at least three primaries? Failure of a primary is declared and a replica is elected by a majority of primaries. With one or two primaries there is no meaningful majority that survives the loss of a node.

REAL WORLD

ShopStream keeps `{user:ID}:cart`, `{user:ID}:session` and `{user:ID}:recent` under one hash tag so the cart service can read and update them in one Lua script, while product caches use plain keys (`product:ID`) that spread evenly across all slots. Each of the six nodes in the cluster runs on a different host, with every replica in a different zone from its primary.

KEY POINTS

A key's slot is `CRC16(key) mod 16384`; hash tags in `{ }` force related keys into the same slot.

- Slots are the unit of ownership and migration; three primaries minimum, plus replicas for failover.
- Smart clients cache the slot map; `MOVED` updates it, `ASK` is a one-time redirect during migration.
- Failure detection: PFAIL by one node, FAIL by a majority of primaries, replica election by a majority of primaries.
- By default the whole cluster stops if any slot is uncovered (`cluster-require-full-coverage yes`).
- Cluster means no cross-slot multi-key commands, only database 0, and per-node `SCAN/KEYS`.

5.5 Atomic slot migration and operating a cluster

A cluster is not finished when `cluster_state:ok` appears. Data grows, traffic shifts, hardware ages and versions change, so you will add and remove nodes, move slots, fail over on purpose and upgrade while the system serves traffic. This chapter covers the newer tools (atomic slot migration in Redis 8.4, per-slot statistics in 8.2) and the routine operations.

5.5.1 How classic slot migration works, and what is wrong with it

Until Redis 8.4, moving a slot meant key-by-key copying driven from outside the servers (by `redis-cli --cluster reshard` or your own tooling):

1. Mark the slot `IMPORTING` on the target and `MIGRATING` on the source (`CLUSTER SETSLOT`).
2. List keys of the slot (`CLUSTER GETKEYSINSLOT`) and move batches with `MIGRATE`, which serialises a key, sends it, and deletes it from the source.
3. Once the slot is empty, tell everyone the target owns it (`CLUSTER SETSLOT ... NODE`).

It works and it is online, but it has well-known rough edges. While the slot is half moved, the keys of that slot are split across two nodes, so a multi-key command on keys of that slot may fail with **TRYAGAIN**, and clients see **ASK** redirections for a while. A very large key is moved as one blocking **MIGRATE**, which stalls the source for as long as it takes. And because the whole process is driven by an external tool, a crash of that tool in the middle leaves "open slots" that you repair with `redis-cli --cluster fix`.

5.5.2 Atomic slot migration (Redis 8.4)

Redis Open Source 8.4 introduces **atomic slot migration** with a new command, **CLUSTER MIGRATION**. The idea is to move a whole slot range as one unit of work, driven by the servers rather than by an external script, and to switch ownership in a single step, so clients are not exposed to a half-moved slot.

You run the command on the **destination primary**, naming the slot ranges to pull:

REDIS

```
CLUSTER MIGRATION IMPORT 0 1000 2000 3000
CLUSTER MIGRATION STATUS ALL
CLUSTER MIGRATION STATUS ID 24cf41718b20f7f05901743dff40bc9b15db339
CLUSTER MIGRATION CANCEL ID 24cf41718b20f7f05901743dff40bc9b15db339
CLUSTER MIGRATION CANCEL ALL
```

IMPORT *start-slot end-slot [start-slot end-slot ...]* accepts several ranges (above: slots 0 to 1000 and 2000 to 3000) and returns a **task id**. The destination then takes a snapshot of those slots from their current owner, receives the writes made in the meantime as a stream, and, when the remaining stream is small enough, the source briefly pauses writes to those slots and hands ownership over. After the handoff the source drops the data of the migrated slots.

STATUS lists tasks (all, or one by id) with fields such as **id**, **slots**, **source**, **dest**, **operation**, **state**, **last_error**, **retries**, **create_time**, **start_time**, **end_time** and **write_pause_ms**. A successful task shows state **completed** and an empty **last_error**. **CANCEL** stops one task or all of them; note that cancelling on the source node does not stop the destination, which keeps retrying until it is cancelled there as well, so cancel on both.

Two settings control the handoff:

Setting	Default	Meaning
<code>cluster-slot-migration-handoff-max-lag-bytes</code>	1MB	When the remaining stream is below this, the source pauses writes and hands off
<code>cluster-slot-migration-write-pause-timeout</code>	10 s	Longest the source pauses writes; if the destination has not taken over by then, the source resumes and treats the migration as failed

Atomic slot migration settings.

Things to be aware of, taken from the current documentation:

- While a migration is importing or trimming, keys in slots a node does not own may be filtered out of **KEYS**, **SCAN**, **RANDOMKEY**, **CLUSTER GETKEYSINSLOT**, **DBSIZE** and **CLUSTER COUNTKEYSINSLOT**, while **INFO KEYSPACE** still shows the real number of keys, including those being imported. Do not alarm if the two disagree for a short time.
- Search commands (**FT.SEARCH**, **FT.AGGREGATE**, **FT.CURSOR**, **FT.HYBRID**) and some time series multi-key queries (**TS.MGET**, **TS.MRANGE**, **TS.MREVRANGE**, **TS.QUERYINDEX**) may return partial results or duplicates while a migration is in progress.
- The command is available in Redis Open Source 8.4 and later; the documentation lists it as not supported for the Standard and Active-Active offerings of Redis Software and Redis Cloud. Check the version of every node before you rely on it, and check whether your version of `redis-cli --cluster reshard` uses it or the classic key-by-key path.

NOTE

The honest summary: atomic slot migration removes the long half-migrated state and the dependence on an external driver. It does not make resharding free. Moving data still costs CPU, memory and network on both nodes, and a short write pause on the migrating slots is part of the design (look at **write_pause_ms** in **STATUS**). Do migrations at a quiet time and watch latency.

5.5.3 Finding what to move: CLUSTER SLOT-STATS (Redis 8.2)

Balanced slot counts do not mean balanced load. One slot can hold a hot key or a huge sorted set. Redis 8.2 added `CLUSTER SLOT-STATS` to report per-slot usage on the node you ask:

REDIS

```
CLUSTER SLOT-STATS SLOTSRANGE 0 100
CLUSTER SLOT-STATS ORDERBY CPU-USEC LIMIT 10 DESC
```

You pick exactly one of two forms. `SLOTSRANGE start end` returns the stats for an inclusive range of slots in ascending slot order. `ORDERBY metric [LIMIT n] [ASC | DESC]` sorts all slots by a metric so that the heaviest are first. The metrics are `KEY-COUNT`, `CPU-USEC`, `MEMORY-BYTES`, `NETWORK-BYTES-IN` and `NETWORK-BYTES-OUT`. Everything except `KEY-COUNT` needs `cluster-slot-stats-enabled yes` in the configuration (it adds a little accounting overhead).

A practical workflow: find the node with the highest CPU or memory, ask it for the top slots by `CPU-USEC` or `MEMORY-BYTES`, and migrate those slots to a quieter primary (with `CLUSTER MIGRATION IMPORT` on 8.4 or `reshard` before that). If a single slot dominates, moving it only relocates the problem; the real fix is in the data model (split the hot key, remove a hash tag that is too coarse).

5.5.4 Everyday operations

Adding a node

1. Start the new node with `cluster-enabled yes` and the same security settings (`requirepass/masterauth`, ACLs, TLS) as the others.
2. Join it. As an empty primary: `redis-cli --cluster add-node 10.0.1.9:6379 10.0.1.1:6379` (new node first, then any existing node). As a replica of a specific primary: add `--cluster-slave --cluster-master-id <node-id>`; without the id, the tool picks a primary with the fewest replicas.
3. An empty primary owns no slots and serves nothing. Give it slots with `reshard`, with `rebalance --cluster-use-empty-masters`, or with `CLUSTER MIGRATION IMPORT`.

Rebalancing

`redis-cli --cluster rebalance <host:port>` moves slots until each primary's share is within a threshold of the average (default 2 percent), optionally weighted by `--cluster-weight node=weight`, and `--cluster-simulate` shows what it would do without doing it. Always simulate first. It balances slot counts, not load, so combine it with `SLOT-STATS` where load is uneven.

Removing a node

A replica can be removed at once. A primary must own **no slots** first, so reshard its slots away, check with `--cluster check`, and only then run `redis-cli --cluster del-node <host:port> <node-id>`. The tool tells the other nodes to forget the node and resets it. If you simply switch off a node without this, the others keep it in their tables as failed, and you must remove it with `CLUSTER FORGET <node-id>` on every other node (`redis-cli --cluster call ... cluster forget <id>` can do this).

Planned failover

To take a primary out for maintenance without losing a write, log in to **its replica** and run:

Planned failover

TERMINAL

```
127.0.0.1:17004> CLUSTER FAILOVER
OK
```

The default form is coordinated: the replica asks the primary to pause clients, waits until it has processed every write, starts an election and takes over, and the old primary becomes its replica. Variants: `CLUSTER FAILOVER FORCE` skips the primary's cooperation (use it when the primary is unreachable but not yet marked failed) and `CLUSTER FAILOVER TAKEOVER` takes over without any election and without agreement, a last-resort tool that can lose writes and should be used by an operator who knows why.

Rolling upgrade

1. Upgrade the **replicas** one at a time (stop, replace binary, start, wait until `master_link_status:up` and the cluster is `ok`).
2. For each primary, run **CLUSTER FAILOVER** on its upgraded replica, so the upgraded node becomes primary.
3. Upgrade the old primaries, now replicas.
4. Verify with `redis-cli --cluster check` and **CLUSTER INFO**.

Read the release notes for every version you cross (especially from Redis 7 to 8) and test on a staging cluster first. Do not run mixed-version clusters longer than needed and avoid using new features until all nodes are upgraded. The upgrades chapter in the operations Part has the full checklist.

5.5.5 Health checks and common problems

Watch `cluster_state`, `cluster_slots_fail`, `cluster_slots_pfail` and `cluster_known_nodes` in **CLUSTER INFO**, plus replication lag per shard and `redis-cli --cluster check` in a scheduled job. The table shows the frequent causes of trouble.

Symptom	Likely cause	What to check or do
<code>cluster_state:fail</code>	A slot is uncovered (primary and its replicas lost) or no majority of primaries is reachable	CLUSTER NODES , <code>--cluster check</code> ; restore a node; consider <code>cluster-require-full-coverage no</code> for caches
Nodes never join	Bus port blocked, wrong announced IP behind NAT or in containers	Open port+10000 (or <code>cluster-port</code>); set <code>cluster-announce-ip</code> and <code>cluster-announce-port/cluster-announce-bus-port</code>
Frequent failovers	<code>cluster-node-timeout</code> too low for network or GC-like pauses, slow commands blocking a node	Look at SLOWLOG , latency, raise the timeout moderately
Constant MOVED in application	Client not caching slot map, or topology refresh disabled	Use a cluster-aware client, enable refresh
Open slots after interrupted resharding	Migration tool died mid-way	<code>redis-cli --cluster fix</code>
Replica never promoted	Replica data too old (validity factor) or <code>cluster-replica-no-failover yes</code>	INFO replication , config check

Cluster symptoms and first checks.

WATCH OUT

Never copy a node's `nodes.conf` (the file named by `cluster-config-file`) to another node or restore an old one into a running cluster: node ids and epochs are in it, and duplicates confuse the whole cluster. To replace a failed node, start a fresh node, add it as a replica of the primary, or let it resync.

KEY POINTS

Atomic slot migration (8.4) moves slot ranges with **CLUSTER MIGRATION IMPORT**, run on the destination primary, and avoids the long half-moved state of classic **MIGRATE**.

- **CLUSTER SLOT-STATS** (8.2) shows per-slot key count and, if enabled, CPU, memory and network use, to find what to move.
- Add nodes empty, then give them slots; remove nodes only after emptying them.
- Use **CLUSTER FAILOVER** on a replica for planned, lossless switchovers and rolling upgrades.
- Test everything on a staging cluster and know the cluster-bus ports before you need them.

5.6 Choosing a topology

You now know four ways to run Redis: a single node, a primary with replicas, a primary with replicas and Sentinel, and Redis Cluster. They are not stages you must climb in order. Each is a trade between simplicity, availability, capacity and what your application has to know. Choosing well usually means choosing the simplest one that meets your requirements.

Start with four questions:

1. **What happens to the business if Redis is gone for a minute? For an hour?** (Availability.)
2. **What happens if the last few seconds of writes are lost?** (Durability and consistency.)
3. **Will the data fit in one machine's memory with headroom, and can one primary handle the writes?** (Capacity.)
4. **Which features does the application need that a cluster restricts?** (Multi-key operations, Lua, transactions, database numbers.)

5.6.1 The four topologies

Topology	Strengths	Capacity limit	Failure behaviour	Client requirements
Single node	Simplest; lowest latency; full command set	One machine's memory and one core	Node down = Redis down until restart or restore	Plain client
Primary + replicas (manual failover)	Copy of data; read scaling; offloads backups	Same as one node for memory and writes	Promote by hand; clients need a new address	Plain client (read/write split optional)
Primary + replicas + Sentinel	Automatic failover and discovery	Same as one node	Seconds of errors during failover; async replication can lose recent writes; split brain possible	Sentinel-aware client
Redis Cluster	Memory and write throughput scale out; automatic failover per shard	Many primaries	A shard without any replica makes its slots unavailable; minority partitions stop; async loss as above	Cluster-aware client; same-slot multi-key rules

Topologies and their failure modes.

When a single node is the right answer

Many production systems should stay here for a long time. If the data is a few gigabytes, the load is well below what one core can do, and the data is a rebuildable cache, one node with sensible persistence (or none), monitoring and a documented restore procedure is a legitimate design. Redis restarts quickly with RDB, and the application may already tolerate a cold cache. Do not buy complexity you do not need.

Primary + replicas

Adding one replica turns "lost the machine" from a data-loss event into a "promote and repoint clients" event. Without Sentinel the promotion is manual, which can be acceptable for internal tools and batch systems where a human is around. It also gives you somewhere to take backups and run slow analytic scans without hurting the primary.

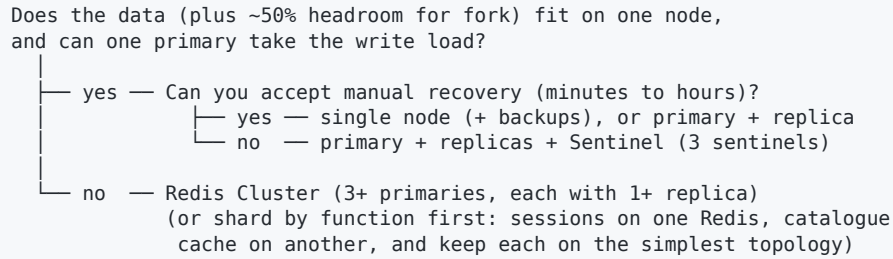
Sentinel

This is the standard answer for "I need automatic failover and my data fits in one node". The cost is three Sentinel processes, a Sentinel-aware client and the discipline to test failovers. Throughput is still bound to a single primary: you add read capacity with replicas, not write capacity.

Cluster

Choose Cluster when one node cannot hold the dataset or serve the writes, or when you want failover without extra processes. Count the costs honestly: a minimum of six nodes for a sane setup, a cluster-aware client, hash tags in your key design, and the restrictions on cross-slot operations. If your code uses scripts, transactions or set intersections across arbitrary keys, you will spend real effort adapting it.

5.6.2 A decision walk-through



The last line of the diagram is important: **splitting by purpose** is often a better first step than a cluster. If ShopStream's session store, catalogue cache and rate limiter share one Redis, they compete for memory and one noisy workload (a cache full of big product JSON) can evict the others' keys. Separate instances per role, each with its own **maxmemory**, eviction policy and persistence setting, are often simpler than one giant cluster and isolate failures.

5.6.3 Sizing rules of thumb

- **Memory headroom.** A primary that forks for RDB or AOF rewrite can use significantly more memory while it takes writes (copy-on-write). Plan **maxmemory** well below the machine's RAM; a common starting point is to leave a large fraction free if the write rate is high. The operations Part has the arithmetic.
- **Replicas need the same memory as the primary**, plus buffers for the replication stream.
- **Cluster shard size.** Smaller shards (for example 10 to 25 GB per primary) restart, sync and migrate faster than a few giant ones. More shards add more nodes and more cross-slot restrictions, so this is a balance, not a rule.
- **Three of everything in different failure domains.** Three Sentinels, primaries and replicas of one shard in different zones or racks.

Failure	Single	Replica + Sentinel	Cluster with replicas
Process crash	Restart, data from disk	Failover in seconds	Shard fails over in seconds
Machine loss	Restore from backup	Failover, small write loss possible	Shard fails over, small write loss possible
Zone loss	Outage	Survives if replicas and a Sentinel majority are in other zones	Survives if each shard has a replica elsewhere and a primary majority survives
Dataset larger than one node	Not possible	Not possible	Possible
Accidental FLUSHALL	Backups only	Replicated everywhere, backups only	Replicated on that shard (and others if sent to all), backups only

What a topology protects you from.

5.6.4 Moving from one topology to the next

Topologies change as a system grows, so choose one you can leave. Moving from a single node to primary-plus-replica or Sentinel is easy: add replicas, add Sentinels, switch the client configuration. Moving to Cluster is a real migration, because the data model must satisfy the same-slot rules and the client must change.

A pragmatic plan looks like this:

1. **Make your key design cluster-ready before you need a cluster.** Put related keys under a hash tag now (**{user:42}:cart**), declare every key a Lua script touches in **KEYS**, and avoid **KEYS/SCAN**-based global logic. These habits cost nothing on a standalone node.
2. **Test against a cluster early**, for example with Testcontainers or a docker-compose cluster, so that cross-slot errors appear in the build and not in production.
3. **Load the cluster in parallel.** For caches, simply deploy the cluster, switch the application over and let it warm up. For data you cannot rebuild, copy it with a migration tool or your own pipeline (scan the old instance, write to the cluster through a cluster-aware client), run both for a while, then cut over.
4. **Keep the rollback path** until the new topology has run through a real failover.

For caches, rebuilding is often the best migration; for state that lives only in Redis, treat the move like any database migration, with a rehearsal and a verified rollback.

INTERVIEW ANGLE

How do you decide between Sentinel and Cluster? If the dataset and write load fit on one primary and you want automatic failover, use replication with Sentinel: simpler, full command set, no key-design restrictions. If memory or write throughput exceeds one node, or you want to grow horizontally, use Cluster and accept same-slot rules, hash tags and a cluster-aware client. Neither removes the possibility of losing recently acknowledged writes in a failover.

KEY POINTS

Pick the simplest topology that meets availability, durability and capacity needs.

- Single node and primary-plus-replica are respectable choices for rebuildable data.
- Sentinel adds automatic failover for one shard; Cluster adds scale-out and per-shard failover.
- Separating workloads by purpose is often better than one big shared cluster.
- Every topology uses asynchronous replication, so none is a zero-loss guarantee.

5.7 Multi-region and managed services

Everything so far assumed that all nodes sit close to one another, with network round trips measured in fractions of a millisecond. Moving data between regions changes the picture. A round trip between continents takes tens to well over a hundred milliseconds, links fail more often, and the speed of light is not negotiable. This chapter gives an overview of the options and trade-offs, then of managed services. It is deliberately descriptive: product features and names change quickly, so check your provider's current documentation before you decide.

5.7.1 Why go beyond one region

- **Latency for users.** A user in another continent should not pay 150 ms for every cache read.
- **Disaster recovery.** A whole region can become unavailable.
- **Data residency.** Some data must stay in a particular jurisdiction.

These goals pull in different directions, and the right design depends on which one matters most.

5.7.2 Patterns

Independent caches per region

The simplest and most common: each region runs its own Redis, populated from the source of truth in that region (or from a regional read replica of the database). Nothing is replicated between Redis instances. A cache miss in region B costs a database read, not a cross-region call. This works well for caches, sessions that are tied to a region, and rate limits that can be per region.

Cross-region replica (disaster recovery or read locality)

A replica in another region is set up with ordinary replication (or the managed equivalent). Reads in that region are fast but stale by at least the network latency plus replication lag; writes still go to the single primary region, so remote writers pay the round trip. For disaster recovery you promote the remote replica manually or with orchestration. Because replication is asynchronous, the recovery point is "whatever had arrived", and you should measure that lag. Putting Sentinel or Cluster members across regions makes the failure detection timeouts and quorum maths harder, since a long inter-region latency spike looks like a failure; this is usually avoided in favour of per-region clusters plus an explicit failover process.

Active-active

In an active-active (multi-writer) design, every region accepts writes to the same logical dataset and changes are merged between regions. Merging requires conflict resolution, and a general-purpose Redis command stream cannot be merged naively (what is the result if two regions **INCR** the same counter, or **SET** the same key, at the same moment?). Active-active products solve this with **conflict-free replicated data types (CRDTs)** that define merge rules per data type, such as counters that add and sets that union. Redis offers this in its commercial products (Redis Software and Redis Cloud); it is not part of Redis Open Source. The consequence for your design is that you must think in terms of eventual consistency and mergeable operations, and some commands and patterns (for example lock-style compare-and-set across regions) do not have a meaningful active-active meaning.

WATCH OUT

A distributed lock across regions is almost never what you want. Round-trip times make lock acquisition slow, and asynchronous replication means two regions can both believe they hold it. Keep locks regional, or move the critical decision into the database that is the system of record.

5.7.3 Managed Redis services

Running Redis well (patching, failover, backups, monitoring, scaling) is real work, and most cloud providers sell it as a service. In general terms, they fall into these groups:

- **Managed open-source-compatible caches.** The provider runs Redis-compatible nodes for you, with a replication group or a cluster-mode option, automatic failover across availability zones, snapshots and a console. Amazon ElastiCache, Azure Cache for Redis (and its successors) and Google Cloud Memorystore are examples of this category.
- **Durable, multi-AZ Redis-compatible databases.** Some services, for example Amazon MemoryDB, keep a multi-AZ transaction log so that acknowledged writes survive a node failure, aiming for use as a primary database rather than a cache. They trade some write latency for that durability.
- **Redis's own cloud and software.** Redis Cloud and Redis Software add features such as active-active geo-distribution and tiering, with their own operational model.
- **Serverless variants.** Some providers offer pay-per-request or auto-scaling flavours that hide the node topology.

Because offerings change, here is what to evaluate rather than what any provider says today:

Question	Why it matters
Which engine and version is it (Redis Open Source, Valkey, or a compatible implementation)?	Command and feature availability (for example Redis 8 data types, Functions, newest commands) may lag or differ
Standalone, replicated or cluster mode?	Decides whether your client must be cluster-aware and how multi-key commands behave
What is the failover time and the write-loss guarantee?	Sets your application's retry and idempotency requirements
Which commands and configuration settings are restricted?	Providers commonly block CONFIG , DEBUG , MODULE and others; some modules or data types may be absent
Persistence and backup options	Cache-only or durable; restore time and cross-region copy
Network model (VPC only, TLS, auth)	Security posture; latency from your compute
Pricing model (instance-hour, per GB, per request, data transfer)	Cross-zone and cross-region traffic is often a significant part of the bill

Questions to ask about any managed service.

Cost and latency trade-offs

Memory is the main cost driver: RAM is expensive, so keeping a large cold dataset in Redis costs far more than in a database. Replicas multiply the memory bill. Cross-zone replication traffic and cross-region transfer are usually billed. Latency changes with placement: a client in another availability zone typically pays a fraction of a millisecond more per round trip than one in the same zone, which matters when an application makes many sequential calls; use pipelining and keep chatty components close to Redis.

5.7.4 A regional disaster recovery runbook

Whatever product you use, a cross-region recovery plan should be written down and rehearsed. A minimal runbook for a primary region lost to an outage has these parts:

1. **Decide and declare.** Who may declare a regional failover, and on what evidence (not on one failed health check). A premature promotion is worse than a short wait, because it creates two divergent datasets.
2. **Fence the old region.** Make sure the old primary cannot accept writes if it comes back (firewall, `REPLICAOF` to the new primary, or shutting it down) to prevent split brain across regions.
3. **Promote the remote replica** (`REPLICAOF NO ONE`, or the provider's promotion call) and note the replication offset or time you recovered to. Writes after that point are lost; record the window for the incident report.
4. **Redirect traffic:** DNS, service discovery or configuration. Remember DNS caching and client reconnect behaviour.
5. **Rebuild redundancy:** add replicas to the new primary and, later, set up the reverse direction for fail-back.
6. **Reconcile:** compare against the system of record and replay or repair anything that matters.

The recovery point is bounded by replication lag at the moment of failure and the recovery time by how automated these steps are. Measure both in a drill; do not guess.

REAL WORLD

For ShopStream, a sensible shape is a managed or self-run replicated Redis per region for sessions and catalogue cache, the catalogue and orders in a regional database, and no attempt to share a Redis dataset across regions. Cross-region concerns (inventory, orders) are handled in the database or by event streams between regions, not by stretching Redis.

KEY POINTS

Multi-region adds latency and failure modes; independent regional caches are the simplest and most robust pattern.

- A cross-region replica gives local reads and disaster recovery, with stale reads and asynchronous loss.
- Active-active needs CRDT-style merge semantics and is a feature of Redis commercial products, not Redis Open Source.
- Managed services remove operations work but vary in version, features, restrictions and cost; check them against your needs.
- Avoid cross-region locks and cross-region synchronous assumptions.

PART 6

Redis from Java

Choosing a client, then Lettuce and Jedis in depth; Redisson and resilience follow.

In this part

- 6.1 Choosing a Java client
- 6.2 Lettuce
- 6.3 Jedis
- 6.4 Redisson
- 6.5 Connection management and resilience

6.1 Choosing a Java client

Redis speaks a simple text protocol (RESP) over TCP, so in principle you could talk to it with a raw socket. In practice you use a client library that opens connections, encodes commands, parses replies, follows cluster redirects, reconnects after failures and maps Redis types onto Java types. Java has three mature, widely used client libraries, and one framework layer (Spring Data Redis) that sits on top of two of them. This chapter helps you choose, and the rest of the Part shows each one in working code.

6.1.1 The candidates at a glance

Library	What it is	Programming model	Typical reason to pick it
Lettuce	Netty-based client, the Spring Boot default	Sync, async (CompletionStage) and reactive (Project Reactor) on one thread-safe connection	You use Spring, or you want one shared connection and non-blocking I/O
Jedis	Small, blocking client; one socket per borrowed connection	Synchronous; pipelines and transactions as objects	You want simple, predictable, easy-to-debug code and a pool
Redisson	Client plus a large library of distributed Java objects	Sync, async and reactive; objects such as <code>RMap</code> , <code>RLock</code> , <code>RRateLimiter</code>	You want ready-made distributed locks, semaphores and collections
Spring Data Redis	Abstraction (templates, repositories, cache, messaging)	<code>RedisTemplate</code> , <code>StringRedisTemplate</code> , repositories	You build a Spring application and want idiomatic integration

The Java Redis ecosystem in one table.

Spring Data Redis is not a competitor to the other three. It is a layer that talks to Redis *through* Lettuce (the default) or Jedis. Redisson also ships Spring integrations, which is why you will sometimes see an application using both Spring Data Redis and Redisson side by side.

6.1.2 How to decide

Use a short decision path rather than a feature checklist.

1. **Building a Spring Boot application?** Start with `spring-boot-starter-data-redis`. You get Lettuce, a configured connection factory and `StringRedisTemplate` without writing any client code. Drop down to the native client only for features the template does not expose.
2. **Need distributed locks, rate limiters, semaphores or a local cache with invalidation, and do not want to write the Lua yourself?** Add Redisson for those objects (see the chapter on Redisson in this Part). Keep ordinary caching and data access on Spring Data Redis.
3. **Writing a plain Java service, a CLI tool or a batch job?** Lettuce or Jedis both work. Choose Jedis if the team prefers blocking code and a connection pool; choose Lettuce if you want one connection shared by many threads or you are already on Reactor.
4. **Need reactive end to end?** Lettuce (directly or through `ReactiveRedisTemplate`). Jedis has no reactive API.

6.1.3 Lettuce versus Jedis: the real difference

The architectural difference matters more than any feature list. Jedis uses one TCP connection per in-flight caller: a thread borrows a connection from a pool, sends a command, blocks until the reply arrives, and returns the connection. Concurrency therefore equals the pool size, and the instance of a raw connection must never be shared between threads.

Lettuce is built on Netty. A single `StatefulRedisConnection` is thread-safe: many threads can submit commands to it concurrently, the commands are written to the socket in order, and replies are matched back to callers in order. Because Redis executes commands on one thread anyway, one well-used connection can keep a server busy. Lettuce still needs extra connections for blocking commands (`BLPOP`, `XREAD BLOCK`) and for transactions, because those occupy the connection.

Two ways to use a connection

Jedis (pool)

```
thread A → borrow → conn 1 → Redis
thread B → borrow → conn 2 → Redis
thread C → borrow → conn 3 → Redis
thread D → waits for a free conn
```

concurrency = pool size

Lettuce (shared connection)

```
thread A ┌
thread B ┤ → one connection → Redis
thread C └ (commands queued, pipelined)
```

concurrency = commands in flight on the socket

Neither model is "faster" in general. A shared Lettuce connection automatically pipelines concurrent callers, which helps throughput; a Jedis pool gives each caller an independent connection, which makes blocking commands and per-connection state trivial. The thing that most often hurts in production is not the model but mis-sized pools and missing timeouts, covered in the chapter on connection management.

NOTE

Version names change quickly. Jedis 7.2 introduced a new entry-point API (`RedisClient`, `RedisClusterClient`, `RedisSentinelClient`) that replaces `JedisPooled`, `JedisPool`, `JedisCluster` and `JedisSentinelPool`, which are now deprecated but still widely used in existing code. This book shows the new API first and points out the legacy names. Always check the release notes of the version you actually use.

INTERVIEW ANGLE

Lettuce or Jedis, and why is a Lettuce connection thread-safe? Spring Boot defaults to Lettuce. A `StatefulRedisConnection` is thread-safe because Netty serialises writes onto an event loop and Lettuce matches replies to a queue of pending commands, so many threads can share one socket. Jedis connections are not thread-safe; you borrow one per operation from a pool. Blocking commands and transactions need a dedicated connection in Lettuce.

KEY POINTS

- Spring Data Redis is a layer over Lettuce or Jedis, not a third client.
- Lettuce shares one thread-safe connection and offers sync, async and reactive APIs; Jedis borrows a blocking connection per call from a pool.
- Redisson adds distributed objects (locks, rate limiters, maps) on top of Redis.
- In a Spring Boot app, start with the starter and reach for native clients only when needed.
- Check which Jedis generation you are on: 7.2 deprecated the older entry classes.

6.2 Lettuce

Lettuce is the client you meet first in a Spring Boot project, so it is worth knowing it directly. This chapter uses the plain library (no Spring) so you see exactly which object does what. Add the dependency `io.lettuce:lettuce-core` to your build; the Redis documentation at the time of writing shows a 6.7.x release in its examples, so use the current release from Maven Central.

6.2.1 The four objects you must know

- **RedisURI**: the address and options for a connection (host, port, database, credentials, SSL, timeout). It can be built with a builder or parsed from a string such as `redis://localhost:6379/0`.
- **RedisClient**: the factory and owner of the heavy resources (Netty event loops, thread pools). Create **one** per application and shut it down at the end.
- **StatefulRedisConnection<K, V>**: one connection to Redis, thread-safe. You obtain command APIs from it.
- The command APIs: `connection.sync()`, `connection.async()` and `connection.reactive()` return **RedisCommands**, **RedisAsyncCommands** and **RedisReactiveCommands**. They all use the same underlying connection.

LettuceHello.java

JAVA

```
package com.shopstream.demo;

import io.lettuce.core.RedisClient;
import io.lettuce.core.RedisURI;
import io.lettuce.core.api.StatefulRedisConnection;
import io.lettuce.core.api.sync.RedisCommands;

public class LettuceHello {

    public static void main(String[] args) {
        RedisURI uri = RedisURI.Builder.redis("localhost", 6379).build();
        RedisClient client = RedisClient.create(uri);
        try (StatefulRedisConnection<String, String> connection = client.connect()) {
            RedisCommands<String, String> redis = connection.sync();
            redis.set("greeting", "hello");
            System.out.println(redis.get("greeting"));
            redis.hset("product:42", "name", "Kettle");
            System.out.println(redis.hgetall("product:42"));
        } finally {
            client.shutdown();
        }
    }
}
```

What to notice: the generic parameters `<String, String>` say that keys and values are Strings; they come from the *codec* (below). Command method names follow the Redis command in lower case (`hset`, `hgetall`, `zadd`), which makes the API easy to guess.

6.2.2 Sync, async and reactive on the same connection

The synchronous API blocks the calling thread until the reply arrives. The asynchronous API returns a **RedisFuture<T>**, which implements **CompletionStage**, so you can chain with `thenApply`, `thenCombine` and so on. The reactive API returns Project Reactor **Mono** and **Flux** types, which only run when subscribed.

ThreeApis.java

JAVA

```
package com.shopstream.demo;

import io.lettuce.core.RedisClient;
import io.lettuce.core.RedisFuture;
import io.lettuce.core.api.StatefulRedisConnection;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutionException;
import reactor.core.publisher.Mono;

public class ThreeApis {

    public static void main(String[] args) throws ExecutionException, InterruptedException {
        RedisClient client = RedisClient.create("redis://localhost:6379");
        try (StatefulRedisConnection<String, String> connection = client.connect()) {

            // 1. synchronous
            String a = connection.sync().get("stock:sku-1");

            // 2. asynchronous: fire several commands without waiting, then collect
            RedisFuture<String> f1 = connection.async().get("stock:sku-1");
            RedisFuture<String> f2 = connection.async().get("stock:sku-2");
            CompletableFuture<String> both =
                f1.toCompletableFuture().thenCombine(f2, (x, y) -> x + "/" + y);
            System.out.println(both.get());
        }
    }
}
```


ThreeApis.java (continued)

JAVA

```

        // 3. reactive: nothing happens until subscribe/block
        Mono<String> m = connection.reactive().get("stock:sku-1");
        System.out.println(m.block());
        System.out.println(a);
    } finally {
        client.shutdown();
    }
}
}

```

Because all three APIs share one connection, firing many async commands back to back is effectively pipelining: Lettuce writes them to the socket without waiting for replies, which cuts round trips (see the chapter on pipelining in the Redis Inside Part for the arithmetic). By default Lettuce flushes each command as it is issued. To batch explicitly, call `connection.setAutoFlushCommands(false)`, queue commands, then call `connection.flushCommands()`; remember to restore auto-flush, and do not do this on a connection shared with other threads.

GOTCHA

Do not call `block()` or `get()` inside a reactive or Netty callback. A reactive chain may run on a Netty event-loop thread. Blocking that thread stalls every command on every connection it serves, and Lettuce or Reactor may throw an exception telling you that blocking is not allowed there. Keep blocking work off the event loop or stay reactive all the way.

6.2.3 Codecs: what goes over the wire

Redis stores bytes. A `RedisCodec<K, V>` converts your key and value types to bytes and back. The default for `client.connect()` is `StringCodec.UTF8`, which is why you saw `<String, String>`. You can ask for other codecs:

Codecs.java

JAVA

```

package com.shopstream.demo;

import io.lettuce.core.RedisClient;
import io.lettuce.core.api.StatefulRedisConnection;
import io.lettuce.core.codec.ByteArrayCodec;
import io.lettuce.core.codec.RedisCodec;
import io.lettuce.core.codec.StringCodec;

public class Codecs {

    public static void main(String[] args) {
        RedisClient client = RedisClient.create("redis://localhost:6379");

        // String keys, byte[] values: a common choice when you serialize objects yourself
        RedisCodec<String, byte[]> mixed = RedisCodec.of(StringCodec.UTF8, ByteArrayCodec.INSTANCE);
        try (StatefulRedisConnection<String, byte[]> connection = client.connect(mixed)) {
            connection.sync().set("blob:1", new byte[] {1, 2, 3});
        } finally {
            client.shutdown();
        }
    }
}

```

For Java objects you normally serialize to JSON (or another format) yourself and store a `String` or `byte[]`. Spring Data Redis wraps this idea in its serializers (see the Spring Part).

6.2.4 Thread safety, blocking commands and pooling

A `StatefulRedisConnection` can be shared by all threads for ordinary commands. Three situations need more than one connection:

- **Blocking commands** (`BLPOP`, `BRPOPLPUSH`, `XREAD` with `BLOCK`, `XREADGROUP` with `BLOCK`) hold the connection until they return. Use a dedicated connection for them.
- **Transactions** (`MULTI/EXEC`) and `WATCH`: the connection is in a stateful mode between `MULTI` and `EXEC`, so other threads must not interleave. Use a dedicated connection per transaction.
- **Pub/Sub subscriptions** switch a connection into subscriber mode; Lettuce provides `StatefulRedisPubSubConnection` for this.

If you need many connections, Lettuce integrates with Apache Commons Pool 2 (add `org.apache.commons:commons-pool2`):

```

LettucePool.java
package com.shopstream.demo;

import io.lettuce.core.RedisClient;
import io.lettuce.core.api.StatefulRedisConnection;
import io.lettuce.core.support.ConnectionPoolSupport;
import org.apache.commons.pool2.impl.GenericObjectPool;
import org.apache.commons.pool2.impl.GenericObjectPoolConfig;

public class LettucePool {

    public static void main(String[] args) throws Exception {
        RedisClient client = RedisClient.create("redis://localhost:6379");
        GenericObjectPoolConfig<StatefulRedisConnection<String, String>> config =
            new GenericObjectPoolConfig<>();
        config.setMaxTotal(8);

        try (GenericObjectPool<StatefulRedisConnection<String, String>> pool =
            ConnectionPoolSupport.createGenericObjectPool(client::connect, config)) {
            try (StatefulRedisConnection<String, String> connection = pool.borrowObject()) {
                connection.sync().set("k", "v");
            } // close() returns the connection to the pool
        } finally {
            client.shutdown();
        }
    }
}

```

Closing a pooled connection *returns* it to the pool rather than closing the socket. Do not pool for ordinary traffic: one shared connection is the intended design, and a pool of idle connections only adds sockets on the server.

6.2.5 Client options, timeouts and reconnection

`ClientOptions` controls behaviour. These facts are from the Lettuce reference: `autoReconnect` is on by default (when a connection is lost without being closed on purpose, the client reconnects and re-issues the queued commands), the default connect timeout in `SocketOptions` is 10 seconds with no keep-alive and no TCP no-delay, and by default **commands do not time out** (`TimeoutOptions` default). That last default surprises people: a command that never gets a reply waits forever unless you set a timeout.

```

LettuceOptions.java
package com.shopstream.demo;

import io.lettuce.core.ClientOptions;
import io.lettuce.core.RedisClient;
import io.lettuce.core.SocketOptions;
import io.lettuce.core.TimeoutOptions;
import java.time.Duration;

public class LettuceOptions {

    public static RedisClient create() {
        RedisClient client = RedisClient.create("redis://localhost:6379");
        client.setOptions(ClientOptions.builder()
            .autoReconnect(true)
            .socketOptions(SocketOptions.builder()
                .connectTimeout(Duration.ofSeconds(2))
                .keepAlive(true)
                .build())
            .timeoutOptions(TimeoutOptions.enabled(Duration.ofMillis(500)))
            .build());
        return client;
    }
}

```

What to notice: `TimeoutOptions.enabled(Duration)` turns on a per-command timeout; without it a lost reply blocks the caller for ever. The `ClientOptions` built-in setting for what happens to commands issued while the connection is down (queued, or rejected immediately) is `disconnectedBehavior`; for latency-sensitive services you usually prefer to

reject quickly rather than queue, so a Redis outage becomes a fast error instead of a growing backlog. Check the Javadoc of your Lettuce version for the exact enum values.

6.2.6 Sentinel, master/replica and cluster

Lettuce discovers topology for you. You hand it addresses; it works out where the primary is.

```

Topologies.java
package com.shopstream.demo;

import io.lettuce.core.ReadFrom;
import io.lettuce.core.RedisClient;
import io.lettuce.core.RedisURI;
import io.lettuce.core.api.StatefulRedisConnection;
import io.lettuce.core.cluster.ClusterClientOptions;
import io.lettuce.core.cluster.ClusterTopologyRefreshOptions;
import io.lettuce.core.cluster.RedisClusterClient;
import io.lettuce.core.cluster.api.StatefulRedisClusterConnection;
import io.lettuce.core.codec.StringCodec;
import io.lettuce.core.masterreplica.MasterReplica;
import io.lettuce.core.masterreplica.StatefulRedisMasterReplicaConnection;
import java.util.concurrent.TimeUnit;
import io.lettuce.core.cluster.ClusterTopologyRefreshOptions.RefreshTrigger;

public class Topologies {

    // Sentinel-managed primary with replica reads
    static StatefulRedisMasterReplicaConnection<String, String> sentinel() {
        RedisURI uri = RedisURI.Builder.sentinel("sentinel-1", 26379, "mymaster")
            .withSentinel("sentinel-2", 26379)
            .withSentinel("sentinel-3", 26379)
            .build();
        RedisClient client = RedisClient.create();
        StatefulRedisMasterReplicaConnection<String, String> connection =
            MasterReplica.connect(client, StringCodec.UTF8, uri);
        connection.setReadFrom(ReadFrom.REPLICA_PREFERRED);
        return connection;
    }

    // Redis Cluster with topology refresh
    static StatefulRedisClusterConnection<String, String> cluster() {
        RedisURI seed = RedisURI.create("redis://node-1:6379");
        RedisClusterClient client = RedisClusterClient.create(seed);
        ClusterTopologyRefreshOptions refresh = ClusterTopologyRefreshOptions.builder()
            .enablePeriodicRefresh(30, TimeUnit.SECONDS)
            .enableAdaptiveRefreshTrigger(RefreshTrigger.MOVED_REDIRECT,
                RefreshTrigger.PERSISTENT_RECONNECTS)
            .build();
        client.setOptions(ClusterClientOptions.builder()
            .topologyRefreshOptions(refresh)
            .build());
        return client.connect();
    }
}

```

The `ReadFrom` settings are `MASTER`, `MASTER_PREFERRED`, `REPLICA`, `REPLICA_PREFERRED`, `LOWEST_LATENCY`, `ANY` and `ANY_REPLICA`. (Some library versions have started to say *upstream* instead of *master* in newer names; use the constants your version offers.) Reading from replicas can return stale data because replication is asynchronous, so use it only for reads that tolerate lag.

For cluster mode, the cluster client keeps a map of slot to node and sends each command to the right node. The map is refreshed when you ask it to. **Periodic refresh is off by default**, so without the options above a client may learn about a failover only after it receives a `MOVED` redirect. Adaptive triggers refresh on `MOVED` redirects or repeated reconnect failures, and a periodic refresh is a cheap safety net.

6.2.7 TLS and authentication

Use `rediss://` in the URI (or `RedisURI.Builder.withSsl(true)`) for TLS, and supply credentials with `withAuthentication(username, password)`. With ACLs enabled, give each application its own user (see the security chapter in the Redis Inside Part). Never put passwords in source files; read them from configuration.

WATCH OUT

Default Lettuce settings are tuned for convenience, not for outages. No command timeout, a 10-second connect timeout and queueing of commands while disconnected can turn a short Redis incident into thread pile-ups and an out-of-memory error in the client. Set command timeouts, decide what should happen while disconnected, and test by stopping Redis.

INTERVIEW ANGLE

How does Lettuce find the new primary after a Sentinel failover? With a Sentinel `RedisURI`, the client asks the sentinels for the primary address and subscribes to their events, then reconnects to the new primary. In cluster mode, it uses cached slot maps, which are refreshed on `MOVED` redirects, on adaptive triggers, or periodically if you enabled periodic refresh.

KEY POINTS

- One `RedisClient`, one shared `StatefulRedisConnection` for ordinary commands; use extra connections for blocking commands, transactions and Pub/Sub.
- Sync, async and reactive APIs share a connection; async commands are effectively pipelined.
- Codecs decide the wire format; the default is UTF-8 strings.
- Commands have no timeout by default; set `TimeoutOptions` yourself.
- For cluster, enable periodic and adaptive topology refresh. Read from replicas only if stale data is acceptable.

6.3 Jedis

Jedis is the oldest and most straightforward Java client. Each call sends a command over a socket and blocks until the reply comes back. There is little to learn, which is its main strength: stack traces are short and behaviour is easy to reason about.

6.3.1 Two generations of API

Jedis 7.2 introduced new entry classes, and the documentation now marks the older ones as deprecated:

Newer API (7.2 and later)	Older API	Use case
<code>RedisClient</code>	<code>JedisPooled</code> , <code>JedisPool</code> , <code>UnifiedJedis</code>	A single server with automatic connection pooling
<code>RedisClusterClient</code>	<code>JedisCluster</code>	Redis Cluster
<code>RedisSentinelClient</code>	<code>JedisSentinelPool</code>	Sentinel-managed primary and replicas

Jedis entry classes.

If your project is on an older release, or you are reading existing code, you will see `JedisPooled` and `JedisPool`. The ideas are identical; only the class names changed.

6.3.2 Basic usage and the connection pool

```
JedisHello.java JAVA

package com.shopstream.demo;

import java.time.Duration;
import redis.clients.jedis.ConnectionPoolConfig;
import redis.clients.jedis.RedisClient;

public class JedisHello {

    public static void main(String[] args) {
        ConnectionPoolConfig pool = new ConnectionPoolConfig();
        pool.setMaxTotal(16);
        pool.setMaxIdle(16);
        pool.setMinIdle(2);
        pool.setBlockWhenExhausted(true);
        pool.setMaxWait(Duration.ofMillis(500));
        pool.setTestWhileIdle(true);

        try (RedisClient jedis = RedisClient.builder()
            .hostAndPort("localhost", 6379)
            .poolConfig(pool)
            .build()) {
            jedis.set("greeting", "hello");
            System.out.println(jedis.get("greeting"));
            jedis.zadd("lb:bestsellers:daily", 12.0, "sku-1");
            System.out.println(jedis.zrevrangeWithScores("lb:bestsellers:daily", 0, 2));
        }
    }
}
```

What to notice: the `RedisClient` object is thread-safe and meant to be shared; each call borrows a connection from the internal pool and returns it. The pool defaults to 8 connections (`maxTotal` and `maxIdle` 8, `minIdle` 0) according to the Jedis docs. `setMaxWait` bounds how long a caller waits for a free connection when all are in use; without a limit threads queue up behind a slow Redis. A short form for scripts is `RedisClient.create("redis://localhost:6379")`.

With the legacy API the same code looks like this (shown only so you recognise it):

```
LegacyJedis.java JAVA

// legacy API, still common in existing code
JedisPooled jedis = new JedisPooled("localhost", 6379);
jedis.set("greeting", "hello");
String value = jedis.get("greeting");
```

6.3.3 Pipelining and transactions

A pipeline buffers commands and sends them in one go; replies are collected when you call `sync()`. Each pipelined call returns a `Response<T>` whose value is available only after `sync()`.

```
JedisPipeline.java JAVA

package com.shopstream.demo;

import java.util.List;
import redis.clients.jedis.AbstractPipeline;
import redis.clients.jedis.AbstractTransaction;
import redis.clients.jedis.RedisClient;
import redis.clients.jedis.Response;

public class JedisPipeline {

    static void pipeline(RedisClient jedis) {
        try (AbstractPipeline pipe = jedis.pipelined()) {
            Response<String> a = pipe.get("stock:sku-1");
            Response<String> b = pipe.get("stock:sku-2");
            pipe.sync();
            System.out.println(a.get() + " " + b.get());
        }
    }

    static void transaction(RedisClient jedis) {
        try (AbstractTransaction trans = jedis.multi()) {
```

JedisPipeline.java (continued)

JAVA

```

        trans.incrBy("counter:orders", 1);
        trans.incrBy("counter:items", 3);
        List<Object> results = trans.exec();
        System.out.println(results);
    }
}

static boolean optimistic(RedisClient jedis) {
    try (AbstractTransaction trans = jedis.transaction(false)) {
        trans.watch("cart:7:version");
        trans.multi();
        Response<Long> r = trans.incr("cart:7:version");
        List<Object> results = trans.exec();
        return results != null && r.get() != null; // null results means aborted: retry
    }
}
}

```

What to notice: always close the pipeline or transaction (try-with-resources) so the connection returns to the pool. `exec()` returns `null` when a **WATCH**ed key changed, and production code retries. A transaction in Redis is atomic with respect to other clients but has no rollback; see the transactions chapter in the Redis Inside Part. In Jedis, pipelining and transactions are separate modes and cannot be mixed on one object.

NOTE

The sample above follows the pipelining page of the Jedis documentation ([AbstractPipeline](#), [AbstractTransaction](#), [jedis.transaction\(false\)](#) for **WATCH**). On older Jedis versions the types are called **Pipeline** and **Transaction**.

6.3.4 Cluster and Sentinel

For cluster, `RedisClusterClient.create(nodes)` takes a set of seed nodes, discovers the slot map and routes commands. Multi-key commands must still respect the same-slot rule (use hash tags like `{cart:7}`). For Sentinel, `RedisSentinelClient` (legacy `JedisSentinelPool`) takes the master name and the sentinel addresses and connects to the current primary. Because the exact builder methods have changed between releases, take the call shape from the Javadoc of the version you depend on; the concepts do not change.

6.3.5 Error handling

When the connection is lost, a command throws `JedisConnectionException`. Jedis does not silently retry for you in the basic client; a caller that wants to retry must run the command again (the pool will provide a fresh connection) and should use backoff. The Jedis documentation shows a loop with a bounded number of attempts and growing sleeps. Never retry non-idempotent commands blindly: if the failure happened after Redis executed **INCR** but before you read the reply, a retry increments twice.

GOTCHA

Forgetting to close what you borrowed. A leaked connection never returns to the pool. After `maxTotal` leaks, every caller blocks for `maxWait` and then fails. Use try-with-resources for pipelines, transactions and any low-level `Jedis` instance you obtain from a pool.

KEY POINTS

- Jedis is simple and blocking; share the pooled `RedisClient` (or legacy `JedisPooled`) across threads.
- Size the pool deliberately and always set a bounded wait.
- Pipelines return `Response` objects filled in by `sync()`; transactions return `null` from `exec()` when a watched key changed.
- Jedis 7.2 deprecated the older entry classes; know both names.
- Retry only idempotent operations, with backoff.

6.4 Redisson

Lettuce and Jedis give you Redis commands as Java methods. Redisson takes a different view: it gives you **Java objects that happen to live in Redis**. You ask for an `RMap`, an `RLock` or an `RRateLimiter` by name, and it implements the Java-style interface (`java.util.Map`, `java.util.concurrent.locks.Lock`) on top of Redis commands, Lua scripts and Pub/Sub. For distributed coordination in particular, this saves you from writing and testing tricky scripts yourself.

6.4.1 Creating a client

RedissonSetup.java

JAVA

```
package com.shopstream.demo;

import org.redisson.Redisson;
import org.redisson.api.RedissonClient;
import org.redisson.config.Config;

public class RedissonSetup {

    public static RedissonClient create() {
        Config config = new Config();
        config.useSingleServer()
            .setAddress("redis://localhost:6379");
        return Redisson.create(config);
    }
}
```

`useSingleServer()` is one of several modes: `useSentinelServers()`, `useClusterServers()`, `useMasterSlaveServers()` and `useReplicatedServers()` cover the other topologies. Maven coordinates for the open-source edition are group `org.redisson`, artifact `redisson`. There is also a Spring Boot starter; check the Redisson documentation for its current artifact name and the properties it reads. A `RedissonClient` is thread-safe and expensive to create: build one per application and shut it down (`shutdown()`) at the end.

Redisson's defaults differ from the other clients and are worth knowing (values from the Redisson configuration page; verify them for your version): a 3000 ms command `timeout`, 4 `retryAttempts` with a jittered delay, a 10000 ms `connectTimeout`, a single-server connection pool of 64, and a `lockWatchdogTimeout` of 30000 ms. Unlike Lettuce, Redisson retries failed commands on its own, which is convenient but means a non-idempotent command can be sent more than once after a timeout.

6.4.2 A tour of the objects

Object	Java analogue	Notes
<code>RBucket</code>	A holder for one value	<code>set</code> , <code>get</code> ; values are encoded by the configured codec
<code>RMap</code>	<code>ConcurrentMap</code>	Stored as a Redis hash; <code>RMapCache</code> adds per-entry TTL
<code>RAtomicallyLong</code>	<code>AtomicLong</code>	Backed by a Redis counter
<code>RLock</code>	<code>Lock</code>	Reentrant distributed lock with an automatic lease-renewing watchdog
<code>RRateLimiter</code>	(none)	Distributed rate limiter, token-bucket style
<code>RSemaphore</code> , <code>RCountDownLatch</code>	<code>Semaphore</code> , <code>CountDownLatch</code>	Distributed coordination
<code>RTopic</code>	Pub/Sub topic	Typed messages through Redis Pub/Sub
<code>RLocalCachedMap</code>	<code>Map</code> with a local near-cache	Invalidated across JVMs through Pub/Sub

A few commonly used Redisson objects.

The Redisson documentation says there are more than fifty such objects and services; the table is a sample.

6.4.3 Locks

The lock is the most used feature. The core API looks like `java.util.concurrent.locks.Lock`:

RedissonLockDemo.java

JAVA

```
package com.shopstream.demo;

import java.util.concurrent.TimeUnit;
import org.redisson.api.RLock;
import org.redisson.api.RedissonClient;

public class RedissonLockDemo {

    private final RedissonClient redisson;

    public RedissonLockDemo(RedissonClient redisson) {
        this.redisson = redisson;
    }

    public void processOrder(String orderId) throws InterruptedException {
        RLock lock = redisson.getLock("lock:order:" + orderId);
        // wait up to 2 s to get the lock; with no lease time the watchdog keeps renewing it
        if (lock.tryLock(2, TimeUnit.SECONDS)) {
            try {
                // critical section
            } finally {
                lock.unlock();
            }
        } else {
            throw new IllegalStateException("order " + orderId + " is being processed");
        }
    }
}
```

What to notice: if you call `tryLock(wait, unit)` without a lease time, Redisson's **watchdog** extends the lock's expiry in the background while the owning JVM is alive (default lock timeout 30 seconds, configurable through `Config.lockWatchdogTimeout`). If the JVM dies, renewals stop and the lock expires, so it cannot be held forever. If you pass an explicit lease time, as in `tryLock(100, 10, TimeUnit.SECONDS)`, there is no renewal and the lock is released after the lease even if your work is still running. Unlocking from a thread that does not own the lock throws `IllegalMonitorStateException`. The patterns chapter on distributed locks explains the failure scenarios (pauses, clock behaviour, fencing tokens) that apply to every Redis lock, Redisson included.

6.4.4 Rate limiter and collections

RedissonMore.java

JAVA

```
package com.shopstream.demo;

import java.time.Duration;
import org.redisson.api.RMap;
import org.redisson.api.RRateLimiter;
import org.redisson.api.RateType;
import org.redisson.api.RedissonClient;

public class RedissonMore {

    static boolean allow(RedissonClient redisson, String apiKey) {
        RRateLimiter limiter = redisson.getRateLimiter("rl:api:" + apiKey);
        // 100 permits per second across all clients; only sets the rate if not set already
        limiter.trySetRate(RateType.OVERALL, 100, Duration.ofSeconds(1));
        return limiter.tryAcquire();
    }

    static void cartExample(RedissonClient redisson) {
        RMap<String, Integer> cart = redisson.getMap("cart:42");
        cart.put("sku-1", 2);
        int qty = cart.get("sku-1");
        System.out.println(qty);
    }
}
```

`RateType.OVERALL` shares the rate across all clients; `PER_CLIENT` applies it per client instance. The `Duration` overload of `trySetRate` is the current one; the overload with a `RateIntervalUnit` is deprecated.

6.4.5 Serialization: a trap

Redisson encodes values with a *codec*. The default in the configuration page I checked is `Kryo5Codec`, a compact binary format that is not human-readable and not readable from other languages or from `redis-cli`. If other systems must read your data, or you want to evolve classes safely, configure a JSON or string-based codec (for example Redisson's Jackson-based codecs) and test round trips. Changing the codec later makes existing values unreadable.

6.4.6 When to use Redisson, and when not to

Use it when you want distributed locks, semaphores, rate limiters or a near-cache with invalidation without writing Lua, and you accept a larger dependency with its own configuration and defaults. Think twice when you only need `GET/SET/HSET`: Spring Data Redis or a plain client gives you readable keys and standard formats, and `redis-cli` will show you what is stored. Mixing both is common: Spring Cache and templates for data, Redisson for locks and rate limits. Keep keys and codecs consistent so each side's data stays inspectable.

NOTE

Licensing and editions. Redisson has a free open-source edition and a commercial PRO edition (different Maven group id) with additional features. Check which features your code depends on before assuming they are in the free edition.

INTERVIEW ANGLE

How does Redisson's lock avoid being held forever by a crashed process? Every lock is a Redis key with an expiry. While the owner is alive, a watchdog task renews the expiry (default timeout 30 seconds). If the process crashes the renewals stop and the key expires. The lock is reentrant: it stores the owner id and a hold count in a hash, and unlocking checks ownership.

KEY POINTS

- Redisson exposes Redis as distributed Java objects: `RMap`, `RLock`, `RRateLimiter`, `RTopic` and many more.
- Create one `RedissonClient` per application; it retries failed commands by default.
- A lock without a lease time is renewed by the watchdog; an explicit lease time disables renewal.
- Check the codec: the default is binary, not JSON.
- Use it for coordination primitives; use Spring Data Redis or a plain client for ordinary data.

6.5 Connection management and resilience

Redis is fast, so developers tend to treat it like a local method call. It is not. It is a network service that can be slow, restarting, failing over, overloaded or unreachable, and the client code decides whether that becomes a one-second blip or an outage of the whole application. This chapter lists the settings and design habits that make the difference. They apply to Lettuce, Jedis, Redisson and Spring alike, even though the setting names differ.

6.5.1 The four timeouts

Timeout	What it bounds	Typical failure if missing
Connect timeout	Opening a TCP connection	Threads hang for the OS default while a node is down
Command (response) timeout	Waiting for a reply to one command	A stalled server blocks callers indefinitely
Pool wait timeout	Waiting for a free pooled connection	Threads pile up behind exhausted pools
Overall deadline	The whole user request, including retries	Retries multiply latency beyond what the caller will wait

The timeouts to set deliberately.

Pick values from your latency budget, not from convenience. If a user request has a 1-second budget and a Redis lookup is only an optimisation, a 50 to 200 ms command timeout is reasonable; if Redis answers normally in under a millisecond, a 500 ms timeout already means something is badly wrong. (These numbers are illustrations, not

recommendations for your system.) Remember the Lettuce default of **no command timeout**, and that Spring Boot exposes `spring.data.redis.timeout` and `spring.data.redis.connect-timeout` for the common cases (the Spring Part shows them).

6.5.2 Reconnection and failover

A good client reconnects by itself. Lettuce does so by default (`autoReconnect`). What you control is what happens in the meantime:

- Commands issued while disconnected can be queued (and may later be executed late) or rejected immediately. For caching and rate limiting, prefer to reject fast.
- During a Sentinel failover or a cluster failover, writes fail for a few seconds until a replica is promoted. Plan for errors in that window rather than hoping for none.
- In cluster mode, make sure topology refresh is enabled (see the Lettuce chapter), otherwise the client keeps talking to a demoted node until redirects teach it otherwise.
- DNS and load balancers: if you connect through a DNS name or proxy, remember that the JVM and the client may cache addresses.

6.5.3 Retries: useful, dangerous, bounded

Retry only when the operation is idempotent or you have made it so. `GET`, `SET` of a fixed value and `DEL` are safe. `INCR`, `LPUSH` and `XADD` are not: if the reply was lost after Redis executed the command, a retry applies it twice (an idempotency key, covered in the patterns Part, is the fix). Use exponential backoff with jitter and a hard cap on attempts and total time. Unbounded or tight-loop retries against a struggling Redis amplify the incident (a retry storm).

6.5.4 Pool sizing

You need a pool when the client is connection-per-call (Jedis) or when you use blocking commands and transactions. A first estimate comes from Little's law: connections in use $\approx$ requests per second $\times$ time each request holds a connection. If 2,000 requests per second each hold a connection for 2 ms, about 4 connections are busy on average. Add headroom for bursts and slow commands, then cap the total so that *all your application instances together* stay well below the server's `maxclients` (default 10000 in a typical configuration; check your server). Forty instances with a pool of 200 each is 8,000 connections, which is a lot of memory and file descriptors for the server.

Symptoms of getting it wrong:

- Pool too small: threads wait for connections, latency rises with load while Redis itself looks idle.
- Pool too large: many idle sockets, connection storms after a restart, and a server that spends effort on connection handling.
- Leaks: the pool slowly drains to zero (see the Jedis chapter).

6.5.5 Graceful degradation

Decide, per use of Redis, what the application does when Redis is not available. The answer is different for each role in ShopStream:

Role	If Redis is down	Reasonable policy
Catalogue cache	Reads can go to the database	Fall back to the database, but protect it (rate limit, short circuit) so it is not flooded
Sessions	Users cannot be identified	Fail the request or force a re-login; do not invent sessions
Rate limiter	No counters	Decide fail-open (allow) or fail-closed (reject); public APIs often choose fail-open, payments often fail-closed
Flash-sale stock	No atomic decrement	Fail closed: refuse the purchase rather than oversell
Distributed lock	Cannot guarantee exclusion	Fail the operation, or rely on a database constraint as the final guard
Leaderboard	Stale or empty	Show cached or empty data; it is not critical

ShopStream roles and their failure behaviour.

The shape of a fallback is simple enough to write by hand:

```
CatalogLookup.java JAVA

package com.shopstream.demo;

import java.util.Optional;
import java.util.function.Function;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.dao.DataAccessException;
import org.springframework.data.redis.core.StringRedisTemplate;

public class CatalogLookup {

    private static final Logger log = LoggerFactory.getLogger(CatalogLookup.class);

    private final StringRedisTemplate redis;
    private final Function<String, String> database;

    public CatalogLookup(StringRedisTemplate redis, Function<String, String> database) {
        this.redis = redis;
        this.database = database;
    }

    public String findProductJson(String id) {
        String key = "product:" + id;
        Optional<String> cached = readQuietly(key);
        if (cached.isPresent()) {
            return cached.get();
        }
        return database.apply(id); // source of truth; may be slower
    }

    private Optional<String> readQuietly(String key) {
        try {
            return Optional.ofNullable(redis.opsForValue().get(key));
        } catch (DataAccessException e) {
            log.warn("Redis read failed for {}: {}", key, e.getMessage());
            return Optional.empty(); // degrade: treat as a cache miss
        }
    }
}
```

What to notice: a cache failure is converted into a miss, logged without a stack trace flood, and the code carries on. Real systems add a **circuit breaker** (for example with Resilience4j, which Spring Boot supports) so that after repeated failures the application stops calling Redis for a while instead of making every request wait for its timeout; this also protects the database from a simultaneous stampede.

WATCH OUT

A cache fallback can take the database down. When Redis fails, every request becomes a database request. If the database was sized for a 95% cache hit ratio, the sudden 20-fold increase can bring it down, turning a cache incident into a total outage. Combine the fallback with load shedding, per-key request coalescing and a circuit breaker.

6.5.6 Observability and testing resilience

Log or meter client-side command latency, timeouts, reconnects and pool usage. Lettuce supports Micrometer metrics and tracing natively (the Spring Part covers wiring). Then test the failure paths: stop the Redis container, pause it (`docker pause`), add network delay, and fail over a Sentinel setup while a load test runs. Many resilience bugs only show up this way.

REAL WORLD

In a typical production incident review, the root cause is rarely "Redis crashed". It is a missing command timeout, a pool that was too small, or a retry loop that turned a 10-second failover into a minute-long outage.

INTERVIEW ANGLE

What do you do in the application when Redis is unavailable? It depends on the role. A cache falls back to the source of truth with load protection; sessions fail or re-authenticate; a rate limiter chooses fail-open or fail-closed by business risk; stock and locks fail closed. In all cases: short timeouts, bounded retries for idempotent calls only, a circuit breaker and alerts.

KEY POINTS

- Set connect, command, pool-wait and overall timeouts explicitly; Lettuce has no command timeout by default.
- Retry only idempotent commands, with backoff, jitter and limits.
- Size pools with Little's law and keep the total across instances below `maxclients`.
- Decide fail-open or fail-closed for each Redis role before the incident.
- Protect the database behind a cache with circuit breakers and load shedding, and test by actually killing Redis.

PART 7

Spring Boot and Spring Data Redis

From a starter dependency to templates, serializers and repositories. Caching, messaging and testing follow.

In this part

- 7.1 Setting up Spring Boot with Redis
- 7.2 StringRedisTemplate, RedisTemplate and serializers
- 7.3 Hash mapping and Redis repositories
- 7.4 Spring Cache with Redis
- 7.5 Pub/Sub with Spring
- 7.6 Streams with Spring
- 7.7 Reactive Redis
- 7.8 Spring Session with Redis
- 7.9 Transactions, Scripts and Pipelines in Spring
- 7.10 Testing Redis Code
- 7.11 Observability and a Production Checklist

7.1 Setting up Spring Boot with Redis

Most Java teams do not use a Redis client directly. They use Spring, which handles the client lifecycle, configuration, exception translation and integration with caching, messaging, sessions and repositories. This chapter builds the foundation: dependencies, configuration for each topology, the choice between Lettuce and Jedis, pooling, TLS and the object at the centre of it all, `RedisConnectionFactory`.

7.1.1 The layers

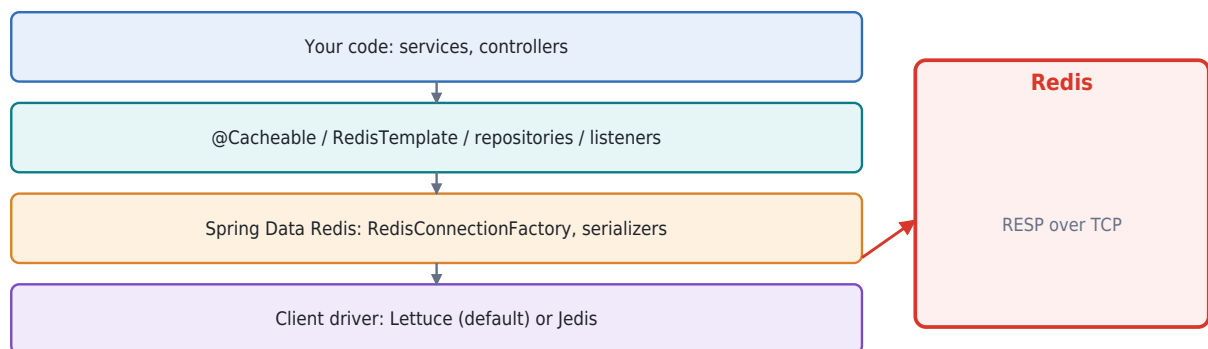


Figure 7.1. The layers between your code and Redis. Your code calls Spring abstractions; the connection factory hands commands to Lettuce (or Jedis), which talks RESP to the server.

Reading the figure from the top: your services use `Spring Cache` annotations or a `StringRedisTemplate`; those use a `RedisConnectionFactory`; the factory is implemented by `LettuceConnectionFactory` or `JedisConnectionFactory`; the client library owns the sockets. Everything above the factory is client-independent, which is why you can usually switch between Lettuce and Jedis by changing a dependency and one property.

7.1.2 Dependencies

With Spring Boot 4.1 and Spring Data Redis 4.1 you add one starter. It brings Spring Data Redis and Lettuce.

pom.xml (fragment)

XML

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
  </dependency>
  <!-- optional: connection pooling for Lettuce or Jedis -->
  <dependency>
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-pool2</artifactId>
  </dependency>
</dependencies>
```

There is also a `spring-boot-starter-data-redis-reactive` starter for the reactive templates (covered later in this Part). Spring Data Redis 4 uses Jackson 3 for its JSON features, which lives in the `tools.jackson` package; the starter and Spring Boot's dependency management take care of the version.

7.1.3 Properties

For the common case of one server, configuration is a few lines of YAML.

application.yml

YAML

```
spring:
  data:
    redis:
      host: localhost
      port: 6379
      database: 0
      username: shopstream      # ACL user, optional
      password: ${REDIS_PASSWORD:}
      timeout: 500ms           # command timeout
      connect-timeout: 2s
```

The Spring Boot reference documents these core keys: `spring.data.redis.host`, `port`, `database`, `username`, `password`, `url`, `ssl.enabled` and `ssl.bundle`. If you set `url` (for example `redis://user:secret@localhost:6379`, or `rediss://` for TLS), the `host`, `port`, `username` and `password` properties are ignored.

WATCH OUT

Check the timeout properties for your Boot version. The Lettuce default is no command timeout (see the previous Part). Spring Boot exposes `spring.data.redis.timeout` and `spring.data.redis.connect-timeout`. Verify in the application-properties appendix of the Boot version you use that they exist as spelled and that your connection factory really applies them: write a test that points the application at a port with nothing behind it, or a paused container, and check how long a call blocks.

For other topologies, Boot offers dedicated property groups. These names are long-standing; confirm them in the properties appendix for your exact Boot version.

application-sentinel.yml

YAML

```
spring:
  data:
    redis:
      sentinel:
        master: mymaster
        nodes:
          - sentinel-1:26379
          - sentinel-2:26379
          - sentinel-3:26379
        password: ${REDIS_PASSWORD:}      # password for the Redis nodes, if different, see docs
```

application-cluster.yml

YAML

```
spring:
  data:
    redis:
      cluster:
        nodes:
          - node-1:6379
          - node-2:6379
          - node-3:6379
```

application-cluster.yml (continued)

YAML

```

    max-redirects: 3
  lettuce:
    cluster:
      refresh:
        adaptive: true
        period: 30s

```

What to notice: for cluster you list a few seed nodes, not every node; the client discovers the rest. The `lettuce.cluster.refresh` group turns on topology refresh (adaptive triggers and a periodic refresh), the setting that makes a client notice failovers promptly. Without it, a cluster client may learn about topology changes only from `MOVED` redirects.

If properties are not expressive enough, Boot lets you define a bean of type `RedisStandaloneConfiguration`, `RedisSentinelConfiguration`, `RedisClusterConfiguration` or `RedisStaticMasterReplicaConfiguration` and uses it instead. The static master/replica configuration is for environments that report non-public addresses through `INFO` (the Spring documentation names AWS as an example) and does not support Pub/Sub, because messages are not propagated across the individual servers. Jedis does not support master/replica mode.

7.1.4 Lettuce or Jedis, and pooling

Lettuce is the default. To use Jedis, exclude `io.lettuce:lettuce-core` from the starter, add `redis.clients:jedis`, and (if both are present) select it with `spring.data.redis.client-type: jedis`. Unless you have a specific reason, stay on Lettuce.

Pooling in Spring behaves differently from what many developers expect:

- With Lettuce, **all** ordinary non-blocking, non-transactional operations share **one** native connection by default (the `shareNativeConnection` setting of `LettuceConnectionFactory` is `true`). Pooling applies to blocking and transactional connections, or to everything if you set `shareNativeConnection` to `false`.
- Boot auto-configures a pooled connection factory only if `commons-pool2` is on the classpath. Pool size is set under `spring.data.redis.lettuce.pool.*` (or `jedis.pool.*`), with `max-active`, `max-idle`, `min-idle` and `max-wait`.
- With Jedis, a pool is the normal way of working.

application.yml (pool fragment)

YAML

```

spring:
  data:
    redis:
      lettuce:
        pool:
          max-active: 16
          max-idle: 16
          min-idle: 2
          max-wait: 500ms

```

7.1.5 Customizing the client

When you need a Lettuce option that has no property (for example a `ClientOptions` timeout setting), register a customizer bean. Boot applies all `LettuceClientConfigurationBuilderCustomizer` beans to the client configuration builder (and `JedisClientConfigurationBuilderCustomizer` for Jedis; `ClientResourcesBuilderCustomizer` customizes Lettuce's shared `ClientResources`).

RedisClientConfig.java

JAVA

```

package com.shopstream.config;

import io.lettuce.core.ClientOptions;
import io.lettuce.core.SocketOptions;
import io.lettuce.core.TimeoutOptions;
import java.time.Duration;
import org.springframework.boot.data.redis.autoconfigure.*;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RedisClientConfig {

```

RedisClientConfig.java (continued)

JAVA

```

@Bean
LettuceClientConfigurationBuilderCustomizer lettuceCustomizer() {
    return builder -> builder.clientOptions(ClientOptions.builder()
        .autoReconnect(true)
        .socketOptions(SocketOptions.builder()
            .connectTimeout(Duration.ofSeconds(2))
            .keepAlive(true)
            .build())
        .timeoutOptions(TimeoutOptions.enabled(Duration.ofMillis(500)))
        .build());
}
}

```

GOTCHA

The customizer's package has moved before. Spring Boot 4 split its auto-configuration into modules, and some classes changed packages compared with Boot 3. If the import above does not resolve in your project, let your IDE find `LettuceClientConfigurationBuilderCustomizer` on the classpath. Behaviour, not the import line, is what to rely on.

When using the Spring Data Redis API directly rather than Boot's auto-configuration, the same options are passed through `LettuceClientConfiguration`:

ManualFactory.java

JAVA

```

package com.shopstream.config;

import io.lettuce.core.ReadFrom;
import java.time.Duration;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.connection.RedisStandaloneConfiguration;
import org.springframework.data.redis.connection.lettuce.LettuceClientConfiguration;
import org.springframework.data.redis.connection.lettuce.LettuceConnectionFactory;

@Configuration
public class ManualFactory {

    @Bean
    LettuceConnectionFactory redisConnectionFactory() {
        LettuceClientConfiguration client = LettuceClientConfiguration.builder()
            .readFrom(ReadFrom.REPLICA_PREFERRED)
            .commandTimeout(Duration.ofMillis(500))
            .useSsl().and()
            .build();
        return new LettuceConnectionFactory(
            new RedisStandaloneConfiguration("redis.internal", 6379), client);
    }
}

```

7.1.6 TLS with SSL bundles

Spring Boot's SSL bundles describe key and trust material once and let components refer to them by name. For Redis you enable SSL and name the bundle:

application-prod.yml

YAML

```

spring:
  ssl:
    bundle:
      pem:
        redis:
          truststore:
            certificate: file:/etc/secrets/redis-ca.pem
  data:
    redis:
      host: redis.internal
      port: 6380
      ssl:
        enabled: true
        bundle: redis

```

The bundle key (`pem`, with a bundle named `redis`) follows Spring Boot's SSL bundle model; for mutual TLS, add a `keystore` with certificate and private key. Server-side TLS configuration is covered in the Operations and Security Part.

7.1.7 The RedisConnectionFactory

`RedisConnectionFactory` is the abstraction every Spring Data Redis component receives: it hands out `RedisConnection` objects and translates client exceptions to Spring's `DataAccessException` hierarchy. In an ordinary Boot application you never create one by hand; you let Boot create it and inject it only where required (for example to build a custom `RedisTemplate` or a message listener container). Use `LettuceConnectionFactory#getConnection` only in infrastructure code, and always close what you get.

A quick way to prove the setup works is a startup check:

RedisStartupCheck.java

JAVA

```
package com.shopstream.config;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.data.redis.core.RedisCallback;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Component;

@Component
public class RedisStartupCheck implements ApplicationRunner {

    private static final Logger log = LoggerFactory.getLogger(RedisStartupCheck.class);

    private final StringRedisTemplate redis;

    public RedisStartupCheck(StringRedisTemplate redis) {
        this.redis = redis;
    }

    @Override
    public void run(ApplicationArguments args) {
        String pong = redis.execute((RedisCallback<String>) connection -> connection.ping());
        log.info("Redis answered {}", pong);
    }
}
```

KEY POINTS

- Add `spring-boot-starter-data-redis`; Lettuce is the default client and Jedis is optional.
- Core properties live under `spring.data.redis.*`; `url` overrides host, port, username and password.
- Use property groups or configuration beans for Sentinel, Cluster and static master/replica setups; enable cluster topology refresh.
- By default Lettuce shares one native connection; pooling needs `commons-pool2` and matters for blocking commands and transactions.
- Set timeouts explicitly and customize the client through customizer beans; use SSL bundles for TLS.

7.2 StringRedisTemplate, RedisTemplate and serializers

`RedisTemplate` is Spring's high-level Redis API. It does three jobs: it borrows and releases connections, it serializes your Java keys and values to bytes and back, and it groups the commands by data type into small interfaces. Learn its shape once and you can drive any data structure.

7.2.1 Which template do I get?

Boot auto-configures two beans:

- `StringRedisTemplate`: keys and values are Strings, using string serializers. It is what you want for counters, flags, IDs, JSON that you serialize yourself, and anything you also want to read in `redis-cli`.
- `RedisTemplate<Object, Object>` (bean name `redisTemplate`): keys and values use **JDK serialization** by default.

WATCH OUT

The default serializer is Java serialization. A `RedisTemplate` you create without configuration stores values with JDK serialization. The values are opaque binary, `redis-cli` shows `"\xac\xed\x00\x05..."`, other languages cannot read them, class changes break deserialization, and the Spring documentation warns that Java native serialization has been used for remote code execution through gadget classes. Always configure explicit serializers, or use `StringRedisTemplate`.

7.2.2 The operations interfaces

The template does not have `get` or `hset` methods itself. You ask it for a typed view, and each view maps to one Redis data type:

Template method	Interface	Redis commands (examples)
<code>opsForValue()</code>	<code>ValueOperations</code>	SET, GET, INCR, SET with NX or EX options
<code>opsForHash()</code>	<code>HashOperations</code>	HSET, HGET, HGETALL, HINCRBY
<code>opsForList()</code>	<code>ListOperations</code>	LPUSH, RPOP, LRange, LTRIM
<code>opsForSet()</code>	<code>SetOperations</code>	SADD, SMEMBERS, SISMEMBER
<code>opsForZSet()</code>	<code>ZSetOperations</code>	ZADD, ZINCRBY, ZRANGE, ZRANK
<code>opsForStream()</code>	<code>StreamOperations</code>	XADD, XREAD, XGROUP, XACK
<code>opsForGeo()</code> , <code>opsForHyperLogLog()</code>	<code>GeoOperations</code> , <code>HyperLogLogOperations</code>	GEOADD, PFADD, PFCOUNT

Template views and the Redis types they drive.

Each also has a *bound* variant (`boundValueOps(key)`, `boundHashOps(key)`, and so on) that fixes the key once, handy when you perform several operations on one key.

7.2.3 A tour with ShopStream data

The next listing uses `StringRedisTemplate` for several of the ShopStream roles. Read it as a sampler: one method per structure.

ShopRedisOps.java

JAVA

```
package com.shopstream.demo;

import java.time.Duration;
import java.util.List;
import java.util.Map;
import java.util.Set;
import org.springframework.data.redis.core.BoundHashOperations;
import org.springframework.data.redis.core.HashOperations;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.core.ZSetOperations.TypedTuple;
import org.springframework.data.redis.connection.stream.RecordId;
import org.springframework.stereotype.Service;

@Service
public class ShopRedisOps {

    private final StringRedisTemplate redis;

    public ShopRedisOps(StringRedisTemplate redis) {
        this.redis = redis;
    }

    // String: flash-sale stock counter, and an idempotency key with TTL
    public long decrementStock(String sku) {
```

ShopRedisOps.java (continued)

JAVA

```

        Long left = redis.opsForValue().decrement("stock:" + sku);
        return left == null ? -1 : left;
    }

    public boolean firstTimeSeen(String idempotencyKey) {
        Boolean created = redis.opsForValue()
            .setIfAbsent("idem:" + idempotencyKey, "1", Duration.ofHours(24));
        return Boolean.TRUE.equals(created);
    }

    // Hash: cart:{userId}
    public void addToCart(String userId, String sku, int quantity) {
        redis.opsForHash().increment("cart:" + userId, sku, quantity);
        redis.expire("cart:" + userId, Duration.ofDays(7));
    }

    public Map<String, String> cart(String userId) {
        HashOperations<String, String, String> hash = redis.opsForHash();
        return hash.entries("cart:" + userId);
    }

    // Bound operations: fix the key once
    public void saveSession(String sid, String userId, String ip) {
        BoundHashOperations<String, String, String> session =
            redis.boundHashOps("session:" + sid);
        session.put("userId", userId);
        session.put("ip", ip);
        session.expire(Duration.ofMinutes(30));
    }

    // List: recent searches, newest first, capped at 10
    public void recordSearch(String userId, String term) {
        String key = "recent:search:" + userId;
        redis.opsForList().leftPush(key, term);
        redis.opsForList().trim(key, 0, 9);
    }

    public List<String> recentSearches(String userId) {
        return redis.opsForList().range("recent:search:" + userId, 0, 9);
    }

    // Set: unique tags
    public Set<String> tags(String sku) {
        redis.opsForSet().add("tags:" + sku, "kitchen", "steel");
        return redis.opsForSet().members("tags:" + sku);
    }

    // Sorted set: daily bestsellers leaderboard
    public void recordSale(String sku, int units) {
        redis.opsForZSet().incrementScore("lb:bestsellers:daily", sku, units);
    }

    public Set<TypedTuple<String>> topSellers() {
        return redis.opsForZSet().reverseRangeWithScores("lb:bestsellers:daily", 0, 9);
    }

    // Stream: order events
    public RecordId publishOrder(String orderId, String userId) {
        return redis.opsForStream()
            .add("stream:orders", Map.of("orderId", orderId, "userId", userId));
    }
}

```

What to notice:

- `setIfAbsent(key, value, Duration)` is `SET key value NX PX ...` in one atomic command. Never write "check then set" as two calls.
- The `increment` methods are atomic server-side operations. `decrementStock` can go below zero; the flash-sale chapter in the Patterns Part shows how to guard against overselling with a script.
- `opsForHash()` is generic in its field and value types, so you declare `HashOperations<String, String, String>` to avoid casting.

- Return values like `Long` and `Boolean` can be `null` when the command is run inside a pipeline or transaction (the call is queued, not executed). Outside those modes they are non-null for these commands, but defensive handling costs little.
- Each call is a network round trip. For several commands in a row, use a pipeline (covered later in this Part) rather than looping.

TIP

Set TTLs when you write. `redis.expire(key, duration)` after `increment` is two commands. Between them the process can die and leave a key with no expiry. For strings, use `set(key, value, Duration)`. For hashes and other structures, run both in a script or transaction when the missing TTL would matter.

7.2.4 Serializers

A `RedisSerializer<T>` has two methods: `serialize(T)` to bytes and `deserialize(byte[])` back. A template has four of them: for keys, values, hash keys and hash values (plus a default serializer used when a specific one is unset). The common rule is **String keys and hash keys, a JSON serializer for values**.

Serializer	What it does	Comment
<code>StringRedisSerializer</code>	UTF-8 strings	Use for keys and hash fields
<code>JdkSerializationRedisSerializer</code>	Java serialization	Default for <code>RedisTemplate</code> ; avoid in new code
<code>JacksonJsonRedisSerializer<T></code>	JSON for one target type, with Jackson 3	Typed and predictable; no type information in the JSON
<code>GenericJacksonJsonRedisSerializer</code>	JSON for any type, with Jackson 3	Default typing is not enabled unless you ask; see below
<code>RedisSerializer.json()</code>	Factory for a generic JSON serializer	Returns the Jackson 3 one in Spring Data Redis 4

Serializers you meet in Spring Data Redis 4.

Typed JSON values

For a cache of one type, the typed serializer is clean: no type information is written, and the JSON is exactly your object.

RedisTemplateConfig.java

JAVA

```
package com.shopstream.config;

import com.shopstream.catalog.Product;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.core.RedisTemplate;
import org.springframework.data.redis.serializer.JacksonJsonRedisSerializer;
import org.springframework.data.redis.serializer.StringRedisSerializer;

@Configuration
public class RedisTemplateConfig {

    @Bean
    RedisTemplate<String, Product> productTemplate(RedisConnectionFactory factory) {
        RedisTemplate<String, Product> template = new RedisTemplate<>();
        template.setConnectionFactory(factory);

        StringRedisSerializer keys = new StringRedisSerializer();
        JacksonJsonRedisSerializer<Product> values =
            new JacksonJsonRedisSerializer<>(Product.class);

        template.setKeySerializer(keys);
        template.setHashKeySerializer(keys);
        template.setValueSerializer(values);
        template.setHashValueSerializer(values);
        template.afterPropertiesSet();
        return template;
    }
}
```

Product.java

JAVA

```
package com.shopstream.catalog;

import java.math.BigDecimal;

public record Product(String id, String name, BigDecimal price, int stock) {
}
```

Using it is then type-safe:

ProductCache.java

JAVA

```
package com.shopstream.catalog;

import java.time.Duration;
import org.springframework.data.redis.core.RedisTemplate;
import org.springframework.stereotype.Component;

@Component
public class ProductCache {

    private final RedisTemplate<String, Product> template;

    public ProductCache(RedisTemplate<String, Product> template) {
        this.template = template;
    }

    public void put(Product p) {
        template.opsForValue().set("product:" + p.id(), p, Duration.ofMinutes(10));
    }

    public Product get(String id) {
        return template.opsForValue().get("product:" + id);
    }
}
```

In `redis-cli` the stored value is plain JSON, for example

`{"id": "sku-1", "name": "Kettle", "price": 24.90, "stock": 120}`. Note that the template's bean name must not clash with Boot's `redisTemplate`; with two `RedisTemplate` beans, injection by generic type (as above) keeps them apart.

A generic JSON serializer

If one template must hold several types, use `GenericJacksonJsonRedisSerializer`. Without default typing, JSON objects deserialize into maps, not into your classes. Spring Data Redis 4's change notes are explicit: unlike its Jackson 2 predecessor, the new serializer does not enable default typing, and you opt in through its builder (`GenericJacksonJsonRedisSerializer.builder().enableDefaultTyping(...)`, with a type validator you choose).

GenericSerializerConfig.java

JAVA

```
package com.shopstream.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.core.RedisTemplate;
import org.springframework.data.redis.serializer.GenericJacksonJsonRedisSerializer;
import org.springframework.data.redis.serializer.RedisSerializer;
import org.springframework.data.redis.serializer.StringRedisSerializer;
import tools.jackson.databind.json.JsonMapper;

@Configuration
public class GenericSerializerConfig {

    @Bean
    RedisTemplate<String, Object> objectTemplate(RedisConnectionFactory factory) {
        RedisTemplate<String, Object> template = new RedisTemplate<>();
        template.setConnectionFactory(factory);
        RedisSerializer<Object> json =
            new GenericJacksonJsonRedisSerializer(JsonMapper.builder().build());
        template.setKeySerializer(new StringRedisSerializer());
        template.setHashKeySerializer(new StringRedisSerializer());
        template.setValueSerializer(json);
        template.setHashValueSerializer(json);
    }
}
```

GenericSerializerConfig.java (continued)

JAVA

```

        template.afterPropertiesSet();
        return template;
    }
}

```

The hard part of polymorphic JSON is security. Default typing writes class names into the data and instantiates classes named in the data when reading it; accepting any class name is dangerous. If you enable it, restrict it with a type validator to your own packages. Often the better design is not to need it: one typed serializer per cache, or store a "type" field yourself.

Migrating from Jackson 2

Applications moving to Spring Data Redis 4 will find the Jackson 2 serializers deprecated for removal:

`GenericJackson2JsonRedisSerializer` becomes `GenericJacksonJsonRedisSerializer`, `Jackson2JsonRedisSerializer` becomes `JacksonJsonRedisSerializer`, and `Jackson2HashMapper` becomes `JacksonHashMapper`. The migration guide warns that Jackson 3 output can differ from Jackson 2 output (and `RedisSerializer.json()` now returns the Jackson 3 serializer), so values written by an old deployment may not be read the same way by a new one. Options: keep the Jackson 2 serializer temporarily so existing entries still deserialize, then migrate by letting entries expire, or write a one-off migration job. For caches the simplest strategy is often to change the key prefix (for example add a version, `product:v2:`), so old and new entries never collide.

7.2.5 Key design in the template layer

Do not build keys by scattering string concatenations around the code. Put key construction in one small class per domain:

Keys.java

JAVA

```

package com.shopstream.common;

public final class Keys {

    private Keys() {
    }

    public static String product(String id) {
        return "product:" + id;
    }

    public static String cart(String userId) {
        return "cart:" + userId;
    }

    public static String session(String sid) {
        return "session:" + sid;
    }
}

```

Central key builders make renames, prefixes per environment and versioned keys (`v2:`) trivial, and let you grep for every use of a key.

7.2.6 Exceptions

Spring translates driver exceptions into its `DataAccessException` hierarchy: `RedisConnectionFailureException` when the connection fails, `QueryTimeoutException` on command timeouts, `RedisSystemException` for others. Catch these (or `DataAccessException`) at the boundary where you decide how to degrade (see the chapter on connection management in the Java clients Part).

INTERVIEW ANGLE

Why does my `RedisTemplate` show strange keys like `"\xac\xed\x00\x05t\x00..."` in `redis-cli`? Because a plain `RedisTemplate` serializes keys with Java's JDK serialization. Use `StringRedisTemplate`, or set `StringRedisSerializer` as key and hash-key serializer and a JSON serializer for values. The same mismatch explains "I wrote with one template and cannot read with another" bugs: two templates with different serializers see different bytes.

KEY POINTS

- `StringRedisTemplate` for strings; `RedisTemplate` needs explicit serializers because its default is JDK serialization.
- Use `opsForValue`, `opsForHash`, `opsForList`, `opsForSet`, `opsForZSet`, `opsForStream`; bound variants fix the key once.
- Keys and hash keys: `StringRedisSerializer`. Values: `JacksonJsonRedisSerializer<T>` for one type, `GenericJacksonJsonRedisSerializer` with care for mixed types.
- Spring Data Redis 4 uses Jackson 3; the Jackson 2 serializers are deprecated for removal and the byte output can differ.
- Centralize key names and handle `DataAccessException` where you decide the degradation policy.

7.3 Hash mapping and Redis repositories

A Redis hash is a natural home for a flat object: field names are property names, field values are property values, and you can read or change one property without touching the others. The question is how a Java object becomes a hash and back. Spring Data Redis answers it at two levels. **Hash mappers** convert an object to a `Map` that you store yourself with `HashOperations`. **Redis repositories** go further: you annotate a class, extend `CrudRepository`, and Spring writes the hash, the indexes and the expiry for you. This chapter shows both, and is candid about when the second is the wrong tool.

7.3.1 Hash mappers

The `HashMapper` interface has two methods: `toHash(object)` and `fromHash(map)`. Spring Data Redis ships several implementations:

Mapper	How it maps	Notes
<code>BeanUtilsHashMapper</code>	Spring BeanUtils; flat bean properties	Simple beans only
<code>ObjectHashMapper</code>	Spring Data's object-to-hash mapping (the engine behind Redis repositories)	Values are <code>byte[]</code> ; stores a <code>_class</code> entry
<code>JacksonHashMapper</code>	Jackson 3	Can flatten nested structures; successor to <code>Jackson2HashMapper</code>
<code>Jackson2HashMapper</code>	Jackson 2	Deprecated and subject to removal

Hash mapper implementations.

The reference documentation shows the pattern with `ObjectHashMapper`, using binary hash fields and values:

HashMapping.java

JAVA

```
package com.shopstream.demo;

import java.util.Map;
import org.springframework.data.redis.core.HashOperations;
import org.springframework.data.redis.hash.HashMapper;
import org.springframework.data.redis.hash.ObjectHashMapper;

public class HashMapping {

    public record Person(String firstname, String lastname) {
    }

    private final HashOperations<String, byte[], byte[]> hashOperations;
    private final HashMapper<Object, byte[], byte[]> mapper = new ObjectHashMapper();

    public HashMapping(HashOperations<String, byte[], byte[]> hashOperations) {
        this.hashOperations = hashOperations;
    }

    public void writeHash(String key, Person person) {
        Map<byte[], byte[]> mappedHash = mapper.toHash(person);
        hashOperations.putAll(key, mappedHash);
    }

    public Person loadHash(String key) {
```

HashMap.java (continued)

JAVA

```

        Map<byte[], byte[]> loadedHash = hashOperations.entries(key);
        return (Person) mapper.fromHash(loadedHash);
    }
}

```

For hashes that humans and other tools should read, you want String fields and values. `JacksonHashMapper` produces a `Map<String, Object>`, and `DecoratingStringHashMapper` converts the values to strings so you can use a `StringRedisTemplate`:

JacksonHashDemo.java

JAVA

```

package com.shopstream.demo;

import com.shopstream.catalog.Product;
import java.util.Map;
import org.springframework.data.redis.core.HashOperations;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.hash.DecoratingStringHashMapper;
import org.springframework.data.redis.hash.HashMapper;
import org.springframework.data.redis.hash.JacksonHashMapper;
import org.springframework.stereotype.Component;

@Component
public class JacksonHashDemo {

    private final HashOperations<String, String, String> hash;
    private final HashMapper<Object, String, String> mapper =
        new DecoratingStringHashMapper<>(JacksonHashMapper.builder().flatten().build());

    public JacksonHashDemo(StringRedisTemplate redis) {
        this.hash = redis.opsForHash();
    }

    public void save(Product product) {
        Map<String, String> fields = mapper.toHash(product);
        hash.putAll("product:" + product.id(), fields);
    }

    public Map<String, String> raw(String id) {
        return hash.entries("product:" + id);
    }
}

```

What to notice: after `save`, `HGETALL product:sku-1` shows readable fields (`id`, `name`, `price`, `stock`). With `flatten()`, nested objects become dotted fields (`address.city`); without it, nested objects are stored as embedded JSON strings. The documentation warns that flattening cannot cope with property names or map keys that contain dots or brackets, and that a flattened hash cannot always be mapped back to an object. If you used `Jackson2HashMapper` before, the migration notes describe a `jackson2CompatibilityMode()` option on the builder to keep the old representation readable; choose `flatten()` or `hierarchical()` to match whatever your old data used.

The advantage of storing a product as a hash rather than a JSON string is partial access: `HINCRBY product:sku-1 stock -1` changes the stock without reading or rewriting the rest. The cost is that nested data is awkward and every field is a string. If you mostly read or write the whole object, a JSON string value is simpler (see the previous chapter).

7.3.2 Redis repositories

A Redis repository stores an annotated Java object as a hash and maintains extra structures for lookups. You get a familiar Spring Data interface; you pay with several Redis commands per save and per query.

UserSession.java

JAVA

```

package com.shopstream.session;

import java.util.Set;
import org.springframework.data.annotation.Id;
import org.springframework.data.redis.core.RedisHash;
import org.springframework.data.redis.core.TimeToLive;
import org.springframework.data.redis.core.index.Indexed;

@RedisHash(value = "session", timeToLive = 1800)
public class UserSession {

```

UserSession.java (continued)

JAVA

```

@Id
private String id;

@Indexed
private String userId;

private String ip;

@TimeToLive
private Long ttlSeconds;

protected UserSession() {
}

public UserSession(String userId, String ip) {
    this.userId = userId;
    this.ip = ip;
}

public String getId() {
    return id;
}

public String getUserId() {
    return userId;
}

public String getIp() {
    return ip;
}

public Long getTtlSeconds() {
    return ttlSeconds;
}
}

```

UserSessionRepository.java

JAVA

```

package com.shopstream.session;

import java.util.List;
import org.springframework.data.repository.CrudRepository;

public interface UserSessionRepository extends CrudRepository<UserSession, String> {

    List<UserSession> findByUserId(String userId);
}

```

The annotations in plain words:

- `@RedisHash("session")` marks the class as a Redis entity and names its **keyspace**, the prefix of its keys. Without a value, the fully qualified class name is the keyspace. The `timeToLive` attribute sets a default expiry in seconds.
- `@Id` marks the identifier. If it is a `String` and null on save, Spring generates a random UUID, as the reference documentation's examples show.
- `@Indexed` on a property maintains a secondary index so that you can query by it. Only indexed properties can be used in derived queries.
- `@TimeToLive` marks a numeric property (or method) that holds the entity's time to live. On read it holds the remaining TTL in seconds; `-1` means no expiry. Do not put `@TimeToLive` on both a property and a method of the same class.

Usage is plain Spring Data:

SessionService.java

JAVA

```

package com.shopstream.session;

import java.util.List;
import org.springframework.stereotype.Service;

@Service
public class SessionService {

```


SessionService.java (continued)

JAVA

```

private final UserSessionRepository repository;

public SessionService(UserSessionRepository repository) {
    this.repository = repository;
}

public String login(String userId, String ip) {
    UserSession saved = repository.save(new UserSession(userId, ip));
    return saved.getId();
}

public List<UserSession> sessionsOf(String userId) {
    return repository.findByUserId(userId);
}
}

```

In a Boot application with Spring Data Redis on the classpath, repository support is auto-configured; the property `spring.data.redis.repositories.enabled` can turn it off. If your application also uses another Spring Data module (JPA, for instance), Spring Data may log that it entered strict repository configuration mode, where it decides which module owns a repository interface by looking at its domain class annotations. Annotating the entity with `@RedisHash` makes the assignment unambiguous. Use `@EnableRedisRepositories` when you need to set options such as the `keyspace-events` mode.

7.3.3 What is actually stored

Redis repositories are easy to use and surprisingly busy. The reference documentation shows the commands issued on insert; for our session they correspond to the following (typical, trimmed output):

What a saved session looks like

TERMINAL

```

127.0.0.1:6379> HGETALL session:7f3a
1) "_class"
2) "com.shopstream.session.UserSession"
3) "id"
4) "7f3a"
5) "userId"
6) "u-42"
7) "ip"
8) "10.0.0.7"
127.0.0.1:6379> SMEMBERS session
1) "7f3a"
127.0.0.1:6379> SMEMBERS session:userId:u-42
1) "7f3a"
127.0.0.1:6379> SMEMBERS session:7f3a:idx
1) "session:userId:u-42"
127.0.0.1:6379> TTL session:7f3a
(integer) 1800

```

Four kinds of key appear:

Key	Type	Purpose
session:{id}	hash	The entity: one field per property, plus <code>_class</code>
session	set	The IDs of all entities in the keyspace; backs <code>findAll</code> , <code>count</code>
session:{property}:{value}	set	A secondary index: the IDs whose property has this value
session:{id}:idx	set	Names of the index sets this entity is in, so they can be cleaned up on update and delete

Keys written by a Redis repository (keyspace "session").

Geo-indexed properties (`@GeoIndexed`) use a Redis geo set instead of a plain set.

A save therefore performs: write the hash, add the ID to the keyspace set, add it to each index set, record the index names. An update replaces the hash and removes stale index entries first (`DEL`, `HMSET`-style write, `SMEMBERS` on the `:idx` set, then removals and re-adds). A finder like `findByUserId` does `SINTER` on the index sets, then one `HGETALL` for **each** matching ID. None of this is wrapped in a transaction by the documentation's description, so a crash in the middle can leave an index and a hash out of step.

7.3.4 Expiry and phantom copies

When a repository entity has a TTL, Spring sets **EXPIRE** on the hash. But Redis cannot expire a single member of a set, so the ID would stay forever in the keyspace set and in the index sets after the hash vanishes. To clean up, Spring Data Redis subscribes to Redis **keyspace notifications** (expired-key events) and removes the dangling index entries when the event arrives. For that it needs the expired object's values, which are gone by then, so it keeps a **phantom copy**: a second copy of the entity that expires five minutes after the original (this is described in the reference documentation; the copy has a `:phantom` suffix in the implementation). The consequences:

- It enables a `RedisKeyExpiredEvent` that your code can listen for, containing a copy of the expired object.
- It doubles the storage for entities with a TTL. The `shadowCopy` setting (the reference documentation writes it as `shadowCopy = OFF`; look up where it is declared in your version) disables the copy, and the expired event then carries only the ID.
- The keyspace listener alters the server setting `notify-keyspace-events` if it is not already set. It does not override an existing value, so set it correctly yourself (the expired-events flag must be on). On managed services where `CONFIG` is blocked (the documentation names AWS ElastiCache) the listener fails, and you set `keyspaceNotificationsConfigParameter` to an empty string and configure the server through the provider's mechanism.
- By default the expiry listener is **not started** with the application. You can start it with the application or on the first insert of an entity with a TTL, through the `enableKeyspaceEvents` option of `@EnableRedisRepositories`.

RedisRepoConfig.java

JAVA

```
package com.shopstream.session;

import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.core.RedisKeyValueAdapter.EnableKeyspaceEvents;
import org.springframework.data.redis.repository.configuration.EnableRedisRepositories;

@Configuration
@EnableRedisRepositories(enableKeyspaceEvents = EnableKeyspaceEvents.ON_STARTUP)
public class RedisRepoConfig {
}
```

Check the Javadoc of `EnableKeyspaceEvents` in your version for the available modes.

WATCH OUT

Pub/Sub is not durable, so index cleanup can be missed. The reference documentation states it plainly: repositories rely on Pub/Sub messages for residual index cleanup. If a key expires while the application is down (or disconnected), the event is lost and index sets may keep references to IDs whose hash no longer exists. Redis also cannot expire individual members of an index set. A query can then return fewer objects than the index suggests, and index sets grow. If this matters, do not use TTLs with indexed entities, or schedule your own cleanup.

7.3.5 Limits and when not to use repositories

Redis has no query language over hashes, so every capability is built from the primitives above, and the costs show up:

- **Write amplification.** One save is several commands, more when indexes change. Compare with one `HSET`.
- **N+1 reads.** A finder loads each matching entity with its own `HGETALL`. A query returning 5,000 sessions is 5,000 reads plus the `SINTER`.
- **Query power.** Equality on indexed properties and simple combinations. No range queries or full text over repository indexes; for those use the Query Engine (see the search chapter of the Patterns Part).
- **Cluster.** The keys involved (entity hash, keyspace set, index sets) hash to different slots, and the multi-key reads and cleanup are hard to fit into a cluster. Check the reference documentation for your version before relying on repositories with Redis Cluster.
- **Big keyspace sets.** The keyspace set grows with the number of entities; `findAll` over millions of entities is a large read.
- **Format.** Data is stored in Spring Data's own hash format with a `_class` field; other services must follow it.

A decision guide:

Need	Good choice	Why
Cache entity by key, read whole object	JSON string via <code>RedisTemplate</code> or Spring Cache	One read, one write, simple
Update single fields often (<code>stock</code> , counters)	Hash with <code>HashOperations</code>	<code>HINCRBY</code> , partial reads
Quick CRUD of small objects with a few lookup fields	Redis repositories	Convenient; accept the overhead
Large result sets, ranges, text search, joins	A database, or Redis Query Engine	Redis repositories are not built for this
Sessions	Spring Session (see the later chapter)	Purpose-built, handles expiry

Choosing a storage style for objects.

NOTE

Why session storage is a good fit and a bad fit. Sessions are looked up by ID, expire on their own and are small, which suits a hash with a TTL. Looking up "all sessions of user u-42" through an index is the part that carries the cleanup problem above. Spring Session (covered later in this Part) is built for this job and is usually the better choice.

LAB 7.1 · Inspect what a repository writes

1. Start a local Redis with `docker run -d --name redis -p 6379:6379 redis:8` and a Spring Boot app with the `UserSession` entity and repository above.
2. Call `login("u-42", "10.0.0.7")` twice from a test or a `CommandLineRunner`.
3. In `redis-cli`, run `SCAN 0 MATCH session* COUNT 100` and note every key type you see. Use `TYPE`, `HGETALL`, `SMEMBERS` and `TTL` on each.
4. Call `findById("u-42")` while running `redis-cli MONITOR` in another terminal (only on a development instance). Count the commands issued and compare with a hand-written `HGETALL` by ID.
5. Wait for the TTL to expire (use a short `timeToLive`, for example 20 seconds) and look again at the key space set and the index set. Which entries are still there? Try again with the expiry listener disabled.

INTERVIEW ANGLE

Why does saving an entity with a Redis repository issue more than one command, and what is the risk? The repository maintains a key space set and secondary index sets in addition to the entity hash, and updates must clean old index entries. These writes are not one atomic operation, and expiry cleanup depends on non-durable Pub/Sub key space events, so indexes can drift from the data. For simple key-based access use `RedisTemplate/HashOperations`; use repositories where convenience beats the overhead.

KEY POINTS

- A `HashMapper` converts objects to hash maps; `ObjectHashMapper` is binary, `JacksonHashMapper` (Jackson 3) can flatten nested objects, and the Jackson 2 mapper is deprecated.
- Hashes allow partial updates (`HINCRBY`); JSON strings are simpler for whole-object access.
- `@RedisHash`, `@Id`, `@Indexed` and `@TimeToLive` define a repository entity; keys are the hash, a key space set, index sets and an `:idx` tracker.
- TTL cleanup relies on key space notifications and phantom copies, which are neither free nor durable.
- Expect write amplification and N+1 reads; avoid repositories for large result sets, complex queries and cluster-heavy designs.

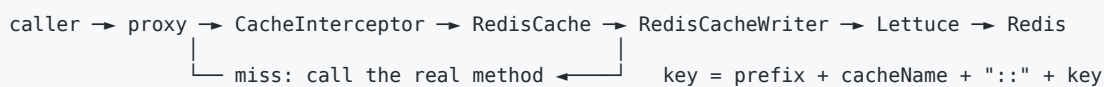
7.4 Spring Cache with Redis

Almost every Spring application eventually asks the same question: "I already computed this, why am I computing it again?" The Spring Cache abstraction answers it with annotations. You mark a method with `@Cacheable`, Spring wraps the bean in a proxy, and the proxy checks a cache before calling the method. Redis is one of the stores that can sit behind that cache. This chapter shows how the pieces fit, how to configure them for a production service such as ShopStream's catalog-service, and which mistakes cost the most time.

The caching patterns themselves (cache-aside, stampede protection, consistency with the database) are covered in the Patterns part. Here we focus on the Spring machinery: what is stored under which key, how long it lives, how it is serialized, and what the annotations do and do not guarantee.

7.4.1 The abstraction in one picture

Spring Cache has two layers. The **annotations** (`@Cacheable`, `@CachePut`, `@CacheEvict`) describe intent on your methods. The **CacheManager** is a registry that hands out named `Cache` objects. When Redis is the backing store, the manager is a `RedisCacheManager` and each named cache is a `RedisCache` whose entries are plain Redis string keys.



Because the mechanism is a proxy, the annotation only works when the call comes from another bean through the Spring-managed reference. We will return to that limitation in the pitfalls section.

7.4.2 Minimal setup

Add `spring-boot-starter-cache` next to `spring-boot-starter-data-redis`, enable caching, and tell Boot to use Redis. With these three steps Boot builds a `RedisCacheManager` for you.

pom.xml (dependencies only)

XML

```

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-cache</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
  </dependency>
</dependencies>

```

application.yml

YAML

```

spring:
  cache:
    type: redis
    redis:
      time-to-live: 10m
      cache-null-values: false
      key-prefix: "shopstream:"
      use-key-prefix: true
      enable-statistics: true
  data:
    redis:
      host: localhost
      port: 6379

```

The `spring.cache.redis.*` properties set defaults for every cache that Boot creates: `time-to-live` is the entry TTL, `cache-null-values` decides whether a `null` result is stored, `key-prefix` and `use-key-prefix` control the prefix, and `enable-statistics` switches on hit and miss counting (used by the metrics section later). These properties are enough for a first version. When you need a different TTL per cache, a different serializer, or jitter, you define the `RedisCacheManager` yourself, which we do below.

CatalogCacheConfig.java

JAVA

```
package com.shopstream.catalog;

import org.springframework.cache.annotation.EnableCaching;
import org.springframework.context.annotation.Configuration;

@Configuration
@EnableCaching
public class CatalogCacheConfig {
}
```

7.4.3 The three annotations

Here is a catalog service that uses all three. The record `ProductDto` is the cached value.

ProductService.java

JAVA

```
package com.shopstream.catalog;

import java.math.BigDecimal;
import org.springframework.cache.annotation.CacheEvict;
import org.springframework.cache.annotation.CachePut;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.stereotype.Service;

record ProductDto(long id, String name, BigDecimal price) {}

@Service
public class ProductService {

    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    @Cacheable(cacheNames = "products", key = "#id", unless = "#result == null")
    public ProductDto findById(long id) {
        return repository.load(id);
    }

    @CachePut(cacheNames = "products", key = "#result.id()")
    public ProductDto update(ProductDto changed) {
        return repository.save(changed);
    }

    @CacheEvict(cacheNames = "products", key = "#id")
    public void delete(long id) {
        repository.delete(id);
    }

    @CacheEvict(cacheNames = "products", allEntries = true)
    public void clearAll() {
    }
}

interface ProductRepository {
    ProductDto load(long id);
    ProductDto save(ProductDto dto);
    void delete(long id);
}
```

What to notice:

- `@Cacheable` looks first. On a hit it returns the cached value and the method body never runs. On a miss it runs the body and stores the result. The Redis key is `products::42` (cache name, two colons, the key).
- `@CachePut` always runs the method and always writes the result. Use it when the method already has the fresh value and you want the cache to hold it.
- `@CacheEvict` removes one entry, or every entry of the cache with `allEntries = true`. Eviction is usually safer than put after a write, because a stale put can overwrite a newer value in a race; the Patterns part discusses this trade-off.

- `unless` is evaluated after the method ran and can see `#result`. `condition` is evaluated before and decides whether caching applies at all.

7.4.4 Keys and key generation

If you give no `key`, Spring uses its default key generator: no arguments gives a fixed internal key, one argument uses that argument, several arguments are combined into a `SimpleKey`. Redis then needs a string, so `RedisCache` converts the key with a conversion service. Simple types such as `Long` and `String` convert naturally. A custom object converts only if it has a useful `toString()`, otherwise you get an exception complaining that the key cannot be converted to a string.

The robust habit is to write the key explicitly with SpEL (`key = "#id"`, `key = "#userId + ':' + #locale"`). It also documents the Redis key you will see in `redis-cli`.

The default prefix is the cache name followed by `::`. You can change it in the configuration with `prefixCacheNameWith("shopstream:")` (a fixed prefix in front of the name) or `computePrefixWith(cacheName -> ...)` (a function). Prefixing matters when several services share one Redis: without a prefix, two services that both define a cache named `products` overwrite each other's entries.

7.4.5 Your own RedisCacheManager: TTLs, serializers, jitter

The following configuration gives the `products` cache a 30 minute TTL with jitter, a short-lived `stock-view` cache of one minute, and JSON values instead of Java serialization.

RedisCacheConfig.java

JAVA

```
package com.shopstream.catalog;

import java.time.Duration;
import java.util.Map;
import java.util.concurrent.ThreadLocalRandom;
import org.springframework.cache.annotation.EnableCaching;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.EnableCaching;
import org.springframework.data.redis.cache.BatchStrategies;
import org.springframework.data.redis.cache.RedisCacheConfiguration;
import org.springframework.data.redis.cache.RedisCacheManager;
import org.springframework.data.redis.cache.RedisCacheWriter;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.serializer.JacksonJsonRedisSerializer;
import org.springframework.data.redis.serializer.RedisSerializationContext.SerializationPair;
import org.springframework.data.redis.serializer.StringRedisSerializer;

@Configuration
@EnableCaching
public class RedisCacheConfig {

    @Bean
    RedisCacheManager cacheManager(RedisConnectionFactory factory) {
        RedisCacheWriter writer =
            RedisCacheWriter.nonLockingRedisCacheWriter(factory, BatchStrategies.scan(1000));

        RedisCacheConfiguration defaults = RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(10))
            .disableCachingNullValues()
            .prefixCacheNameWith("shopstream:")
            .serializeKeysWith(SerializationPair.fromSerializer(StringRedisSerializer.UTF_8));

        RedisCacheConfiguration products = defaults
            .entryTtl((key, value) -> Duration.ofMinutes(30)
                .plusSeconds(ThreadLocalRandom.current().nextLong(0, 300)))
            .serializeValuesWith(SerializationPair.fromSerializer(
                new JacksonJsonRedisSerializer<ProductDto.class>());

        RedisCacheConfiguration stockView = defaults
            .entryTtl(Duration.ofMinutes(1));

        return RedisCacheManager.builder(writer)
            .cacheDefaults(defaults)
            .withInitialCacheConfigurations(Map.of(
                "products", products,
                "stock-view", stockView))
    }
}
```

RedisCacheConfig.java (continued)

JAVA

```

        .enableStatistics()
        .build();
    }
}

```

What to notice:

- `entryTtl(Duration)` sets a fixed TTL. The overload that accepts a function (a `RedisCacheWriter.TtlFunction`) is called per entry with the key and value, and that is how the jitter is produced: every entry gets 30 minutes plus a random 0 to 299 seconds, so a batch of products cached together does not all expire in the same second (the avalanche problem described in the Patterns part).
- `disableCachingNullValues()` means a `null` result is not cached. If a `@Cacheable` method returns `null` and you did not add `unless = "#result == null"`, the put fails with an `IllegalArgumentException` that names the cache. Either choice is valid, but you must pick one on purpose.
- `BatchStrategies.scan(1000)` makes `allEntries = true` use `SCAN` instead of `KEYS`. The default batch strategy uses `KEYS`, which blocks a large key space; this is the one-line fix. With Jedis, `SCAN` is only supported for non-clustered setups.
- `serializeValuesWith` replaces the default value serializer. Out of the box `RedisCache` uses Java native serialization (`JdkSerializationRedisSerializer`), which needs `Serializable` classes, is opaque in `redis-cli`, and breaks when the class changes. JSON is readable and survives refactoring better.
- `enableStatistics()` on the builder turns on local hit and miss counters. It is the same switch as the `spring.cache.redis.enable-statistics` property.

GOTCHA

One serializer per cache, not one for all types. `JacksonJsonRedisSerializer<ProductDto>` knows exactly one target type. If you put it on the defaults, a cache holding a `List<String>` fails on read. Give typed serializers to individual caches as above, or use `GenericJacksonJsonRedisSerializer` for mixed content; remember that it does not write type information unless you enable default typing in its builder, and that default typing has security implications for untrusted data. The template chapter earlier in this Part explains the serializers in detail.

7.4.6 sync = true, locking and the stampede

When a popular entry expires, many threads may miss at once and all call the database. `@Cacheable(sync = true)` tells Spring that only one thread per key may compute the value while others wait for it. The important detail is the scope: this is a lock **inside one JVM**. If catalog-service runs on ten instances, each instance may still compute the value once, so up to ten database calls can happen for one expiry. That is usually acceptable, and far better than hundreds. For cross-instance protection you need a Redis-based lock or single-flight, covered in the Patterns part.

Two further points. With `sync = true` you can use only one cache name, and the `unless` attribute is not supported together with it, so a `null` result must be handled another way. Separately, `RedisCacheWriter.lockingRedisCacheWriter(...)` is a different thing: it locks at the **cache** level (not the entry) with a lock key in Redis to make `putIfAbsent` and `clean` non-overlapping. It costs extra round trips and the reference documentation describes it as an opt-in; most applications leave the default lock-free writer in place.

7.4.7 Pitfalls that bite in practice

Self-invocation

Calling a cached method from another method of the same class bypasses the proxy, so the annotation does nothing.

SelfInvocation.java

JAVA

```

package com.shopstream.catalog;

import org.springframework.cache.annotation.Cacheable;
import org.springframework.stereotype.Service;

@Service
public class PriceService {

    public String describe(long id) {
        // BUG: this is a plain Java call, the proxy is never involved
        return "Price: " + priceOf(id);
    }
}

```


SelfInvocation.java (continued)

JAVA

```

    }

    @Cacheable(cacheNames = "prices", key = "#id")
    public String priceOf(long id) {
        return "9.99";
    }
}

```

The fix is to move the cached method into another bean and inject it, which is also the cleaner design.

Large and dynamic keys

A key such as `#filters` where `filters` is a big object produces a long string key per call, and almost every call is a miss. Keep keys short and deterministic, hash large inputs yourself, and avoid caching results of queries whose parameters are nearly unique (free text search).

Caching mutable or huge objects

A cache entry is serialized on put and deserialized on get, so mutating the returned object does not change the cache (unlike an in-memory map). The flip side: caching a 5 MB list of products makes each hit move 5 MB across the network. Cache the pieces you actually reuse and keep values small.

Transactions

By default a `@CachePut` writes to Redis immediately, even if the surrounding database transaction later rolls back. `RedisCacheManager.builder(...).transactionAware()` makes cache puts and evictions wait until the transaction commits. Even then, cache and database can disagree for a short time; see the consistency chapter in the Patterns part.

Failures of Redis

When Redis is down, a cached method throws, and the user sees an error although the database is healthy. Implement `CachingConfigurer` (or define a `CacheErrorHandler`) that logs and returns, so that a Redis outage degrades to "slower" rather than "broken". Test this path on purpose.

Reactive methods

Spring's cache abstraction can cache methods that return `Mono` or `Flux`; it needs a cache that supports asynchronous retrieval, which `RedisCache` does. We come back to this in the reactive chapter.

INTERVIEW ANGLE

Why did my `@Cacheable` method not cache anything? Most likely one of four causes: caching is not enabled (`@EnableCaching` is missing); the method is called from inside the same class so the proxy is bypassed; the method is not public or the class is final in a way that prevents proxying; or the result is `null` and null caching is disabled. A fifth cause is a key that is different on every call. Check with `redis-cli --scan --pattern 'shopstream:*` to see what really gets stored.

KEY POINTS

- `@Cacheable`, `@CachePut` and `@CacheEvict` work through a proxy; calls from inside the same class are not cached.
- Redis keys look like `cacheName::key`; set the key explicitly with SpEL and use a prefix when services share a Redis.
- Define `RedisCacheManager` yourself for per-cache TTLs, jitter (a TTL function), JSON serializers and a `SCAN` batch strategy for `allEntries`.
- The default value serializer is Java serialization; prefer typed JSON serializers per cache.
- `sync = true` is a per-JVM lock, not a distributed one.
- Add a `CacheErrorHandler` so that a Redis outage does not turn into application errors.

7.5 Pub/Sub with Spring

Redis Pub/Sub lets one client publish a message to a channel and every currently subscribed client receive it. It is fire-and-forget: Redis keeps no copy, so a subscriber that is offline when the message is published never sees it. That makes it a good fit for transient signals (cache invalidation notices, "something changed" nudges, live updates to a dashboard) and a bad fit for work that must not be lost; for that, use Streams. The Pub/Sub concept itself is explained in the data structures part; this chapter shows how Spring wires it up.

7.5.1 The pieces

Type	Role
RedisMessageListenerContainer	Owns the subscription connection and a thread pool; delivers messages to listeners
MessageListener	Interface with <code>onMessage(Message message, byte[] pattern)</code> ; you implement it
MessageListenerAdapter	Wraps a plain object and calls one of its methods, converting the payload for you
ChannelTopic / PatternTopic	What to subscribe to: an exact channel, or a glob pattern such as <code>orders.*</code>
StringRedisTemplate.convertAndSend	Publishes a message to a channel

Spring Data Redis messaging types.

Spring documents the container as the component that "is responsible for all threading of message reception". One connection and one thread are shared by all listeners, subscriptions are created lazily, and you can add or remove listeners while the application is running.

7.5.2 Publishing

Publishing is one line on a template. Under the hood it runs the `PUBLISH` command; treat it as best effort, because nothing tells your code whether anyone was listening.

ProductEventPublisher.java

JAVA

```
package com.shopstream.catalog;

import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Component;

@Component
public class ProductEventPublisher {

    private final StringRedisTemplate redis;

    public ProductEventPublisher(StringRedisTemplate redis) {
        this.redis = redis;
    }

    public void productChanged(long productId) {
        redis.convertAndSend("events.product.changed", String.valueOf(productId));
    }
}
```

7.5.3 Subscribing with a container and a listener

Use this to keep the local in-memory cache of each catalog-service instance in sync: when one instance changes a product, every instance learns about it.

PubSubConfig.java

JAVA

```
package com.shopstream.catalog;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.listener.ChannelTopic;
import org.springframework.data.redis.listener.RedisMessageListenerContainer;
import org.springframework.data.redis.listener.Topic;

@Configuration
```

PubSubConfig.java (continued)

JAVA

```
public class PubSubConfig {

    @Bean
    RedisMessageListenerContainer listenerContainer(
        RedisConnectionFactory factory, LocalCacheInvalidator invalidator) {

        RedisMessageListenerContainer container = new RedisMessageListenerContainer();
        container.setConnectionFactory(factory);
        container.addMessageListener(invalidator, ChannelTopic.of("events.product.changed"));
        container.addMessageListener(invalidator, Topic.pattern("events.price.*"));
        return container;
    }
}
```

LocalCacheInvalidator.java

JAVA

```
package com.shopstream.catalog;

import java.nio.charset.StandardCharsets;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.data.redis.connection.Message;
import org.springframework.data.redis.connection.MessageListener;
import org.springframework.stereotype.Component;

@Component
public class LocalCacheInvalidator implements MessageListener {

    private static final Logger log = LoggerFactory.getLogger(LocalCacheInvalidator.class);

    @Override
    public void onMessage(Message message, byte[] pattern) {
        String channel = new String(message.getChannel(), StandardCharsets.UTF_8);
        String body = new String(message.getBody(), StandardCharsets.UTF_8);
        log.info("channel={} body={} viaPattern={}", channel, body, pattern != null);
        // evict the entry from the local in-memory cache here
    }
}
```

What to notice:

- `onMessage` receives raw bytes. If you published through a `StringRedisTemplate`, decode them as UTF-8.
- The second argument is the pattern that matched, or `null`/empty when the subscription was an exact channel. One listener can serve several topics.
- The container starts automatically with the application context (it is a `SmartLifecycle`) and stops on shutdown.

7.5.4 Subscribing with MessageListenerAdapter

A `MessageListenerAdapter` lets any plain class become a listener, so your business code does not depend on Spring types. The adapter deserializes the payload with its configured serializer and calls the named method.

OrderNotifyConfig.java

JAVA

```
package com.shopstream.notification;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.listener.ChannelTopic;
import org.springframework.data.redis.listener.RedisMessageListenerContainer;
import org.springframework.data.redis.listener.adapter.MessageListenerAdapter;

@Configuration
public class OrderNotifyConfig {

    public static class OrderNotifier {
        public void handleMessage(String orderId) {
            System.out.println("order paid: " + orderId);
        }
    }

    @Bean
```

OrderNotifyConfig.java (continued)

JAVA

```

    MessageListenerAdapter orderListener() {
        return new MessageListenerAdapter(new OrderNotifier(), "handleMessage");
    }

    @Bean
    RedisMessageListenerContainer orderContainer(
        RedisConnectionFactory factory, MessageListenerAdapter orderListener) {
        RedisMessageListenerContainer container = new RedisMessageListenerContainer();
        container.setConnectionFactory(factory);
        container.addMessageListener(orderListener, ChannelTopic.of("orders.paid"));
        return container;
    }
}

```

The adapter's default method name is `handleMessage`, and an exception in your method is caught and handled by the container (by default it is logged), so one bad message does not kill the subscription.

7.5.5 Annotation-driven listeners

Spring Data Redis 4.x also has an annotation style, similar in spirit to Spring's JMS and Kafka listeners. A bean method is marked `@RedisListener`, and a configuration class carries `@EnableRedisListeners` next to a `RedisMessageListenerContainer` bean. The reference documentation shows the attribute as `topic`, with a plain string meaning a channel and a glob meaning a pattern.

AnnotatedListener.java

JAVA

```

package com.shopstream.notification;

import org.springframework.data.redis.annotation.RedisListener;
import org.springframework.stereotype.Component;

@Component
public class AnnotatedListener {

    @RedisListener(topic = "orders.paid")
    public void onOrderPaid(String orderId) {
        System.out.println("paid: " + orderId);
    }
}

```

The reference documentation says nothing about which version introduced it, and at least one blog post writes the attribute as `topics`. Check the Javadoc of the version you use before copying either spelling. The container-based style above works on every version, which is why this book uses it as the primary example.

7.5.6 Reliability limits you must design around

A subscriber only receives messages published while it is connected and subscribed. Four consequences follow.

- **No history, no replay.** A restart, a network blip or a Sentinel failover creates a gap. Messages published in the gap are gone.
- **No acknowledgement and no retry.** Redis does not know whether your listener finished. If `onMessage` throws, the message is not redelivered.
- **No load balancing.** Every subscriber gets every message. Ten notification-service instances on one channel send ten emails. For "one worker handles each message", use a stream with a consumer group.
- **Slow subscribers are cut off.** Redis buffers outgoing messages per client; a subscriber that cannot keep up is disconnected once its output buffer exceeds the `client-output-buffer-limit pubsub` setting.

WATCH OUT

Do not use Pub/Sub for events that must not be lost. Payment confirmations, stock reservations and order state changes belong in a stream (or a database outbox). Use Pub/Sub for hints that can be recomputed: "product 42 changed, drop your local copy". If a missed hint is only a few seconds of staleness, Pub/Sub is the right size of tool.

TIP

Keep the listener fast. The container delivers messages on its own threads. A listener that calls a slow HTTP service blocks delivery of the following messages. Hand the work to a bounded executor of your own, and give the container a suitable `TaskExecutor` through `setTaskExecutor` in anything beyond a demo.

On a Redis Cluster, a classic `PUBLISH` is broadcast to every node, which is costly at volume. Sharded Pub/Sub (`SSUBSCRIBE`, `SPUBLISH`) keeps traffic within one shard; check the support of your Spring Data Redis and client version before relying on it.

INTERVIEW ANGLE

What is the difference between Redis Pub/Sub and Redis Streams? Pub/Sub delivers to the subscribers connected right now and keeps nothing; Streams append messages to a log that is stored, can be read later, and with consumer groups can be shared among workers with acknowledgement and redelivery. Choose Pub/Sub for ephemeral notifications, Streams when a message must be processed at least once.

KEY POINTS

- `RedisMessageListenerContainer` owns the subscription connection and threads; define it as a bean and register listeners with `ChannelTopic.of(...)` or `Topic.pattern(...)`.
- Implement `MessageListener` for raw bytes, or wrap a POJO with `MessageListenerAdapter`; `@RedisListener` with `@EnableRedisListeners` is the annotation style in Spring Data Redis 4.x.
- Publish with `StringRedisTemplate.convertAndSend(channel, message)`.
- Messages are not stored, acknowledged or load balanced. Anything critical goes to Streams.
- Keep listeners fast and give the container a sensible `TaskExecutor`.

7.6 Streams with Spring

A Redis Stream is an append-only log. Producers add entries with `XADD`; consumers read them either independently (`XREAD`) or as a team through a **consumer group** (`XREADGROUP`), where Redis hands each entry to exactly one consumer of the group and remembers it in a **pending entries list** (PEL) until the consumer acknowledges it with `XACK`. The stream data type, groups and the PEL are explained in the data structures part; here we put them to work in ShopStream: order-service appends order events to `stream:orders`, and notification-service, running as several instances in the consumer group `notifier`, sends the emails.

Spring Data Redis supports streams on three levels: `StreamOperations` (one command per method), `StreamMessageListenerContainer` (a polling loop that calls your listener), and the reactive `StreamReceiver` (shown in the next chapter).

7.6.1 Producing events

`opsForStream()` returns the `StreamOperations`. With a `StringRedisTemplate` an entry is a map of strings.

OrderEventProducer.java

JAVA

```
package com.shopstream.order;

import java.util.Map;
import org.springframework.data.redis.connection.stream.RecordId;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Component;

@Component
public class OrderEventProducer {

    public static final String STREAM = "stream:orders";

    private final StringRedisTemplate redis;

    public OrderEventProducer(StringRedisTemplate redis) {
        this.redis = redis;
    }

    public RecordId orderPlaced(long orderId, long userId, String total) {
        Map<String, String> fields = Map.of(
            "type", "ORDER_PLACED",
```

OrderEventProducer.java (continued)

JAVA

```

        "orderId", String.valueOf(orderId),
        "userId", String.valueOf(userId),
        "total", total);
    RecordId id = redis.opsForStream().add(STREAM, fields);
    // keep the stream bounded: approximate trim to about 100000 entries
    redis.opsForStream().trim(STREAM, 100_000, true);
    return id;
}

```

`add` returns the generated ID such as `1760000000000-0`. The last argument of `trim` asks for **approximate** trimming (`MAXLEN ~`), which Redis can do cheaply in whole internal blocks; exact trimming is slower. An unbounded stream is a memory leak that grows slowly, so decide on a trimming policy on day one.

7.6.2 Creating the group

A group must exist before consumers can read through it. Create it once at startup; if it already exists Redis answers with a `BUSYGROUP` error, which you treat as success.

StreamGroupInitializer.java

JAVA

```

package com.shopstream.notification;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.data.redis.connection.stream.ReadOffset;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Component;

@Component
public class StreamGroupInitializer implements ApplicationRunner {

    private static final Logger log = LoggerFactory.getLogger(StreamGroupInitializer.class);

    private final StringRedisTemplate redis;

    public StreamGroupInitializer(StringRedisTemplate redis) {
        this.redis = redis;
    }

    @Override
    public void run(ApplicationArguments args) {
        try {
            redis.opsForStream().createGroup("stream:orders", ReadOffset.from("0"), "notifier");
        } catch (Exception e) {
            // BUSYGROUP: group exists. A missing stream also fails here, see the warning below.
            log.info("createGroup: {}", e.getMessage());
        }
    }
}

```

GOTCHA

The group cannot be created on a stream that does not exist yet. The `createGroup` call of the template fails if `stream:orders` has never received an entry, because it does not use the `MKSTREAM` option of `XGROUP CREATE`. Either create the group after the first event is produced (seed the stream at deployment), or execute `XGROUP CREATE stream:orders notifier 0 MKSTREAM` from a migration script. `ReadOffset.from("0")` means the group starts with the oldest entry; `$` means only new entries.

7.6.3 Consuming with StreamMessageListenerContainer

The container runs a polling loop on its own executor and calls your `StreamListener` for each entry. There are three decisions to take: which consumer name this instance uses, whether the container acknowledges automatically, and what happens on error.

StreamConsumerConfig.java

JAVA

```
package com.shopstream.notification;

import java.net.InetAddress;
import java.time.Duration;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.connection.stream.Consumer;
import org.springframework.data.redis.connection.stream.MapRecord;
import org.springframework.data.redis.connection.stream.ReadOffset;
import org.springframework.data.redis.connection.stream.StreamOffset;
import org.springframework.data.redis.stream.StreamMessageListenerContainer;

@Configuration
public class StreamConsumerConfig {

    private static final Logger log = LoggerFactory.getLogger(StreamConsumerConfig.class);

    @Bean(initMethod = "start", destroyMethod = "stop")
    StreamMessageListenerContainer<String, MapRecord<String, String, String>> streamContainer(
        RedisConnectionFactory factory, OrderEventListener listener) throws Exception {

        var options = StreamMessageListenerContainer.StreamMessageListenerContainerOptions
            .builder()
            .pollTimeout(Duration.ofSeconds(1))
            .batchSize(10)
            .errorHandler(t -> log.warn("stream error: {}", t.toString()))
            .build();

        var container = StreamMessageListenerContainer.create(factory, options);

        String consumerName = InetAddress.getLocalHost().getHostName();
        var request = StreamMessageListenerContainer.StreamReadRequest
            .builder(StreamOffset.create("stream:orders", ReadOffset.lastConsumed()))
            .consumer(Consumer.from("notifier", consumerName))
            .autoAcknowledge(false)
            .cancelOnError(t -> false)
            .build();

        container.register(request, listener);
        return container;
    }
}
```

OrderEventListener.java

JAVA

```
package com.shopstream.notification;

import org.springframework.data.redis.connection.stream.MapRecord;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.stream.StreamListener;
import org.springframework.stereotype.Component;

@Component
public class OrderEventListener
    implements StreamListener<String, MapRecord<String, String, String>> {

    private final StringRedisTemplate redis;
    private final EmailSender emailSender;

    public OrderEventListener(StringRedisTemplate redis, EmailSender emailSender) {
        this.redis = redis;
        this.emailSender = emailSender;
    }

    @Override
```

OrderEventListener.java (continued)

JAVA

```

    public void onMessage(MapRecord<String, String, String> record) {
        emailSender.sendOrderConfirmation(record.getValue().get("orderId"));
        // acknowledge only after the work succeeded
        redis.opsForStream().acknowledge("stream:orders", "notifier", record.getId());
    }
}

interface EmailSender {
    void sendOrderConfirmation(String orderId);
}

```

What to notice:

- `ReadOffset.lastConsumed()` translates to the `>` ID of `XREADGROUP`: "give me entries never delivered to anyone in this group". It does not return entries that are already pending for you after a crash; the recovery section deals with those.
- `autoAcknowledge(false)` means that the listener must call `acknowledge`. The shorter `container.receiveAutoAck(...)` variant acknowledges when the entry is **delivered** to the listener, before your code succeeds. An exception then loses the entry, which is "at most once". For orders, we want "at least once", so we acknowledge manually after the work.
- `cancelOnError(t -> false)` keeps the subscription alive when an error occurs. The default behaviour cancels the subscription for most errors, after which that consumer silently stops reading, a classic production surprise. The `errorHandler` is where you log or count the error.
- The consumer name must be stable and unique per instance (host name, pod name). Two instances using the same name share a PEL and confuse each other; a name that changes at every restart leaves pending entries behind under dead names.
- The container's `start` and `stop` methods are wired as bean init and destroy methods in the example, so the polling thread follows the application lifecycle.

7.6.4 Object records

You can avoid manual maps. `ObjectRecord` carries a Java object that is converted through a `HashMapmer`; read it with a target type. The reference documentation shows `StreamRecords.newRecord().in("my-stream").ofObject(value)` for writing and `opsForStream().read(User.class, StreamOffset.fromStart("my-stream"))` for reading, and a custom mapper such as the `JacksonHashMapmer` (Jackson 3) can be passed to `opsForStream(...)`. For events that several services share, plain string maps with explicit field names are easier to evolve and to inspect with `XRANGE`.

7.6.5 Failure handling: pending entries, retry, dead letters

Two things go wrong in production. A consumer crashes after reading but before acknowledging, leaving entries in the PEL. Or an entry fails repeatedly (a poisoned message) and would be retried forever. Redis does not redeliver pending entries by itself; your code must look at the PEL and claim entries that have been idle for too long.

PendingReclaimer.java

JAVA

```

package com.shopstream.notification;

import java.time.Duration;
import java.util.List;
import org.springframework.data.domain.Range;
import org.springframework.data.redis.connection.stream.MapRecord;
import org.springframework.data.redis.connection.stream.PendingMessage;
import org.springframework.data.redis.connection.stream.PendingMessages;
import org.springframework.data.redis.connection.stream.RecordId;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

@Component
public class PendingReclaimer {

    private static final String STREAM = "stream:orders";
    private static final String GROUP = "notifier";
    private static final String DLQ = "stream:orders:dlq";
    private static final int MAX_DELIVERIES = 5;
}

```


PendingReclaimer.java (continued)

JAVA

```

private final StringRedisTemplate redis;
private final EmailSender emailSender;
private final String consumerName = "reclaimer";

public PendingReclaimer(StringRedisTemplate redis, EmailSender emailSender) {
    this.redis = redis;
    this.emailSender = emailSender;
}

@Scheduled(fixedDelay = 30_000)
public void reclaim() {
    PendingMessages pending = redis.opsForStream()
        .pending(STREAM, GROUP, Range.unbounded(), 100);
    for (PendingMessage p : pending) {
        if (p.getElapsedTimeSinceLastDelivery().compareTo(Duration.ofMinutes(1)) < 0) {
            continue; // still being worked on by its consumer
        }
        RecordId id = p.getId();
        List<MapRecord<String, Object, Object>> claimed = redis.opsForStream()
            .claim(STREAM, GROUP, consumerName, Duration.ofMinutes(1), id);
        for (MapRecord<String, Object, Object> record : claimed) {
            handle(record, p.getTotalDeliveryCount());
        }
    }
}

private void handle(MapRecord<String, Object, Object> record, long deliveries) {
    if (deliveries >= MAX_DELIVERIES) {
        redis.opsForStream().add(DLQ, java.util.Map.of(
            "origId", record.getId().getValue(),
            "orderId", String.valueOf(record.getValue().get("orderId"))));
        redis.opsForStream().acknowledge(STREAM, GROUP, record.getId());
        return;
    }
    try {
        emailSender.sendOrderConfirmation(String.valueOf(record.getValue().get("orderId")));
        redis.opsForStream().acknowledge(STREAM, GROUP, record.getId());
    } catch (RuntimeException e) {
        // leave it pending; the next run tries again and the delivery count grows
    }
}
}

```

For this to run, add `@EnableScheduling` to any configuration class. What the reclaimer does: it lists the first 100 pending entries (`X_PENDING` with a range), skips those idle for less than a minute, claims the rest for itself (`XCLAIM`, which also bumps the delivery counter) and either processes them or, after five deliveries, copies them to a dead-letter stream and acknowledges the original so that it stops cycling. A real implementation would also persist why the entry failed, and an alert should fire when the DLQ grows.

TIP

Idempotent consumers make everything above safe. At-least-once delivery means that after a crash the same order event can be handled twice. Make the handler idempotent: check or set a key such as `idem:email:{orderId}` with `SET ... NX` before sending, or make the effect naturally repeatable. Redis 8.6 introduces idempotent production (`IDMPAUTO`/`IDMP` arguments to `XADD`) to protect against duplicate producers; the consumer side still needs its own protection.

WATCH OUT

The container and many consumers share a connection setup. Each registered subscription issues blocking reads (`XREADGROUP ... BLOCK`). With Lettuce, blocking reads use a dedicated connection rather than the shared one, so a container does not block your other commands, but size your Redis `maxclients` and connection limits knowing that every container adds connections. Do not run the reclaimer and heavy work on the container's executor; give the container its own `executor(...)` in the options if you process slowly.

INTERVIEW ANGLE

How do you get at-least-once processing with Redis Streams in Spring? Create a consumer group, read with `StreamMessageListenerContainer` and manual acknowledgement (`autoAcknowledge(false)`), acknowledge only after the work succeeded, keep the subscription alive on errors, and run a reclaimer that uses `XPENDING` and `XCLAIM` (or `XAUTOCLAIM`) to recover entries abandoned by crashed consumers, with a delivery limit and a dead-letter stream. Make handlers idempotent because duplicates are possible.

KEY POINTS

- `opsForStream().add` appends, `trim(..., true)` bounds the stream approximately; decide on retention from the start.
- Create the group before consuming and make the consumer name stable and unique per instance.
- Acknowledge manually after success; `receiveAutoAck` gives at-most-once.
- Set `cancelOnError(t -> false)` and an `errorHandler`, otherwise a consumer can stop silently.
- Redis never redelivers pending entries on its own: reclaim idle ones, count deliveries, route poison messages to a dead-letter stream.

7.7 Reactive Redis

Lettuce, the default client, is reactive at its core, and Spring Data Redis exposes that through reactive templates. If your service is built on Spring WebFlux (or you simply want non-blocking composition), you use `ReactiveRedisTemplate` instead of `RedisTemplate`: every method returns a `Mono` or `Flux` and nothing happens until someone subscribes. The commands themselves are the same; only the way you wait for the result changes.

Reactive does not make Redis faster, and a single Lettuce connection already multiplexes concurrent commands in the blocking world too. It pays off when your whole request path is non-blocking, so that a thread is never parked waiting for Redis. Mixing a reactive template into a blocking servlet application mostly adds complexity.

7.7.1 Dependency and auto-configuration

Use the reactive starter. It brings Lettuce and Reactor, and Boot auto-configures `ReactiveRedisConnectionFactory`, `ReactiveStringRedisTemplate` and a `ReactiveRedisTemplate<Object, Object>`.

pom.xml (dependencies only)

XML

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis-reactive</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

7.7.2 ReactiveStringRedisTemplate in practice

The reactive operations mirror the blocking ones: `opsForValue()`, `opsForHash()`, `opsForList()`, `opsForZSet()` and so on.

VisitCounter.java

JAVA

```
package com.shopstream.gateway;

import java.time.Duration;
import org.springframework.data.redis.core.ReactiveStringRedisTemplate;
import org.springframework.stereotype.Component;
import reactor.core.publisher.Mono;

@Component
public class VisitCounter {

    private final ReactiveStringRedisTemplate redis;

    public VisitCounter(ReactiveStringRedisTemplate redis) {
        this.redis = redis;
    }
}
```

VisitCounter.java (continued)

JAVA

```

/** Counts a visit and returns the new total for the page. */
public Mono<Long> hit(String page) {
    String key = "visits:" + page;
    return redis.opsForValue().increment(key)
        .flatMap(total -> total == 1
            ? redis.expire(key, Duration.ofDays(1)).thenReturn(total)
            : Mono.just(total));
}

public Mono<String> lastSeenIp(String sessionId) {
    return redis.opsForValue().get("session:" + sessionId + ":ip");
}
}

```

What to notice: `increment` returns `Mono<Long>`; the TTL is set only on the first hit. The two commands are not atomic as a pair (a crash between them leaves a key without TTL), which is why production counters use a Lua script or `SET ... EX` patterns, as the Patterns part shows. The point here is the shape of the code: you compose steps with `flatMap` and `thenReturn`, never with `.block()`.

WATCH OUT

Calling `.block()` inside a reactive chain. `block()` on a Netty event loop thread throws or stalls the loop, and in tests it hides the fact that nothing subscribed. Return the `Mono` to the framework (a WebFlux handler may return it) and let it subscribe.

7.7.3 A typed reactive template

For objects, define a `ReactiveRedisTemplate` with an explicit serialization context. Unlike the auto-configured generic one, it has the serializers you choose.

ReactiveRedisConfig.java

JAVA

```

package com.shopstream.catalog;

import java.math.BigDecimal;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.connection.ReactiveRedisConnectionFactory;
import org.springframework.data.redis.core.ReactiveRedisTemplate;
import org.springframework.data.redis.serializer.JacksonJsonRedisSerializer;
import org.springframework.data.redis.serializer.RedisSerializationContext;
import org.springframework.data.redis.serializer.StringRedisSerializer;

record ProductView(long id, String name, BigDecimal price) {}

@Configuration
public class ReactiveRedisConfig {

    @Bean
    ReactiveRedisTemplate<String, ProductView> productTemplate(
        ReactiveRedisConnectionFactory factory) {

        var valueSerializer = new JacksonJsonRedisSerializer<>(ProductView.class);
        RedisSerializationContext<String, ProductView> context =
            RedisSerializationContext<String, ProductView>newSerializationContext(
                StringRedisSerializer.UTF_8)
                .value(valueSerializer)
                .build();
        return new ReactiveRedisTemplate<>(factory, context);
    }
}

```

ReactiveProductCache.java

JAVA

```

package com.shopstream.catalog;

import java.time.Duration;
import java.util.function.Function;
import org.springframework.data.redis.core.ReactiveRedisTemplate;
import org.springframework.stereotype.Component;
import reactor.core.publisher.Mono;

```

ReactiveProductCache.java (continued)

JAVA

```

@Component
public class ReactiveProductCache {

    private final ReactiveRedisTemplate<String, ProductView> template;

    public ReactiveProductCache(ReactiveRedisTemplate<String, ProductView> template) {
        this.template = template;
    }

    /** Cache-aside: Redis first, loader on a miss, then store with a TTL. */
    public Mono<ProductView> get(long id, Function<Long, Mono<ProductView>> loader) {
        String key = "product:" + id;
        return template.opsForValue().get(key)
            .switchIfEmpty(Mono.defer(() -> loader.apply(id))
                .flatMap(p -> template.opsForValue()
                    .set(key, p, Duration.ofMinutes(10))
                    .thenReturn(p)));
    }
}

```

The key line is `switchIfEmpty(Mono.defer(...))`: a cache miss is an empty `Mono`, and `Mono.defer` makes sure the loader is only called then (without it, the loader would be called eagerly every time the chain is assembled). A Redis error propagates as an error signal; add `onErrorResume` to fall back to the loader when Redis is unavailable.

7.7.4 Reactive caching, scripts and streams

- **Spring Cache** annotations can be used on methods that return `Mono` or `Flux`. The abstraction caches the emitted value and not the publisher, and requires a cache implementation with asynchronous retrieval; `RedisCache` has it. Test your actual cache behaviour in an integration test before trusting it.
- **Scripts**: `ReactiveRedisTemplate.execute(script, keys, args)` returns a `Flux` of results, as in the transactions and scripts chapter later in this Part.
- **Pub/Sub**: `ReactiveRedisMessageListenerContainer` has `receive(ChannelTopic.of("my-channel"))`, which returns a `Flux<ChannelMessage<String, String>>`. It uses the driver threads and needs no executor of its own. `receiveLater` returns a `Mono` that completes once the subscription is confirmed, which is handy to avoid a race between subscribing and publishing in tests.
- **Streams**: `StreamReceiver` (created with `StreamReceiver.create(factory, options)`) returns a `Flux` of records from `receive(StreamOffset.fromStart("stream"))`. It is demand-driven: when the downstream has no demand, polling pauses, and depending on the `ReadOffset` strategy this can cause entries to be skipped. For consumer groups, use the overload that takes a `Consumer` and `StreamOffset` and acknowledge explicitly through `ReactiveStreamOperations.acknowledge`.

NOTE

Low-level reactive access. Under the templates sits `ReactiveRedisConnection`, with command groups such as `stringCommands()` or `keyCommands()` that work on `ByteBuffer`. You rarely need it; use it for commands that the template does not expose.

KEY POINTS

- `ReactiveRedisTemplate` and `ReactiveStringRedisTemplate` return `Mono/Flux`; nothing runs until subscribed.
- Use the reactive starter and keep the whole request path non-blocking; never `block()` inside a chain.
- Define a typed `ReactiveRedisTemplate` with a `RedisSerializationContext` for objects.
- A cache miss is an empty `Mono`: use `switchIfEmpty(Mono.defer(...))` for cache-aside.
- Reactive streams are demand-driven; read the backpressure notes before using `StreamReceiver` with groups.

7.8 Spring Session with Redis

HTTP sessions are the classic reason to put Redis into a web application. By default a servlet container keeps each session in the memory of one server. With two instances behind a load balancer you either pin users to an instance ("sticky sessions") and lose their session when that instance restarts, or you move the session out of the instance. Spring Session replaces the container's `HttpSession` implementation with one that stores the session in an external store; Redis is the most common choice because sessions are small, hot, and naturally expire.

7.8.1 Setting it up in Spring Boot

Add the starter. Boot then auto-configures the session repository for a servlet application (and, in a reactive application, the reactive web session).

pom.xml (dependencies only)

XML

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-session-data-redis</artifactId>
</dependency>
```

application.yml

YAML

```
spring:
  session:
    timeout: 30m
    redis:
      namespace: "shopstream:session"
      flush-mode: on-save
  data:
    redis:
      host: localhost
      port: 6379
server:
  servlet:
    session:
      cookie:
        http-only: true
        secure: true
        same-site: lax
```

The Boot reference documentation states that the timeout is set with `spring.session.timeout`, and that when it is not set a servlet application falls back to `server.servlet.session.timeout`. The Spring Session Boot guide lists `spring.session.redis.namespace` and `spring.session.redis.flush-mode`. Property names under the session prefix have moved between Boot generations, so open the Common Application Properties appendix for your exact Boot version and search for `spring.session` before you deploy. The behaviour described below does not depend on the names.

If you prefer explicit configuration, the annotation `@EnableRedisHttpSession` (or `@EnableRedisIndexedHttpSession`) switches the auto-configuration off and gives you the same settings as attributes. The Boot guide notes that auto-configuration is equivalent to adding the annotation by hand.

7.8.2 What lives in Redis

Each session is stored as a Redis hash under a key made from the namespace and the session ID. A typical session, seen from `redis-cli`, looks like this.

Inspect a stored session

TERMINAL

```
127.0.0.1:6379> KEYS spring:session:*
1) "spring:session:sessions:7c9f1e22-1d1b-4c1b-8a39-93b0c9f0a1de"
2) "spring:session:sessions:expires:7c9f1e22-1d1b-4c1b-8a39-93b0c9f0a1de"
3) "spring:session:expirations:1760001600000"
127.0.0.1:6379> HKEYS spring:session:sessions:7c9f1e22-1d1b-4c1b-8a39-93b0c9f0a1de
1) "creationTime"
2) "lastAccessedTime"
3) "maxInactiveInterval"
4) "sessionAttr:cartId"
127.0.0.1:6379> TTL spring:session:sessions:7c9f1e22-1d1b-4c1b-8a39-93b0c9f0a1de
(integer) 2112
```

(**KEYS** is used here only on an empty demo instance; on a real one use **SCAN**.) The exact set of keys depends on the repository flavour: the indexed repository additionally maintains the **expires** and **expirations** keys so that it can publish session-expired events, and the simpler repository only uses the session hash with a TTL. Attributes are stored with the prefix **sessionAttr:**. The session cookie in the browser (named **SESSION** by default) carries an encoded session ID, never the data.

7.8.3 Serialization

By default session attributes are written with Java serialization, so everything you put into a session must be **Serializable**. That works, but it is opaque and fragile across deployments. You can switch to JSON by defining a bean named **springSessionDefaultRedisSerializer** of type **RedisSerializer<Object>**. Be careful: Spring Security stores objects such as the **SecurityContext** in the session, and a JSON serializer must be able to round-trip them, which usually requires registering Spring Security's Jackson modules. If you do not need it, keep the default and keep session attributes small (an ID, not a whole cart).

7.8.4 Session fixation and cookie safety

Session fixation is an attack where an attacker obtains a valid session ID, makes the victim use it (for instance by planting it in a link or cookie), and then waits for the victim to log in; now the attacker's known ID is an authenticated session. The standard defence is to issue a **new session ID on authentication**. Spring Security does this by default (its session-fixation protection changes the session ID on login while keeping the attributes), and Spring Session supports it, because the repository can rotate IDs. Do not turn that protection off.

Also set the cookie flags shown in the YAML: **http-only** hides the cookie from scripts, **secure** sends it only over HTTPS, and **same-site** reduces cross-site request risk. Never put the session ID in a URL.

7.8.5 Operational notes

- **Memory:** sessions are many small hashes. Measure one with **MEMORY USAGE** and multiply by your peak concurrent sessions before sizing Redis.
- **Expiry:** the TTL on the session key is the idle timeout and is extended on each request. The **flush-mode** decides whether changes are written at once or when the request ends (**on-save**, the default). Attributes you change after the response was committed may not be saved.
- **Persistence and failover:** if Redis loses its data, all users are logged out. With asynchronous replication, a failover can also lose the last writes. For many shops this is acceptable; for others enable AOF or accept re-login.
- **Keyspace notifications:** the indexed repository relies on Redis keyspace events. Spring Session tries to configure **notify-keyspace-events** itself, which fails on managed services that block **CONFIG**. In that case configure the setting in the service console and tell Spring Session not to try (the **ConfigureRedisAction.NO_OP** bean).
- **Redis down means no login:** decide what the application does when it cannot reach the session store, and test it.

REAL WORLD

Sticky sessions versus a shared session store. Teams often start with sticky sessions, then hit rolling deployments that log everyone out, or autoscaling that unbalances load. A shared store makes instances interchangeable, at the price of one extra Redis round trip per request (typically well under a millisecond on the same network) and a new dependency to monitor.

KEY POINTS

- Spring Session moves the **HttpSession** into Redis; with Boot you add **spring-boot-starter-session-data-redis**.
- Set the idle timeout with **spring.session.timeout**; confirm namespace and flush-mode property names for your Boot version.
- Sessions are hashes with a TTL; attributes use Java serialization unless you provide a serializer bean.
- Keep the session ID rotation on login (session fixation protection) and set **HttpOnly**, **Secure** and **SameSite** on the cookie.
- Plan for Redis loss: it means logging users out, so decide if that is acceptable.

7.9 Transactions, Scripts and Pipelines in Spring

Three features let you send several commands as a unit: **MULTI/EXEC** transactions, Lua scripts, and pipelining. They solve different problems, and Spring's template API has a different entry point for each. The engine-level semantics (what **EXEC** guarantees, how **WATCH** works, the script cache) are in the Redis Inside part; this chapter is about calling them correctly from Java.

Need	Use	Spring entry point
Several commands applied together, no interleaving	MULTI/EXEC	<code>SessionCallback</code> , or <code>@Transactional</code> with transaction support
Read, decide, then write, atomically	Lua script	<code>RedisScript</code> with <code>execute(script, keys, args)</code>
Fewer network round trips for independent commands	Pipelining	<code>executePipelined</code>
Optimistic locking on a key	WATCH + MULTI	<code>SessionCallback</code>

Which tool for which need.

7.9.1 Why the template needs a SessionCallback

A Redis transaction is bound to one connection: **MULTI**, the queued commands and **EXEC** must all travel over the same connection. Template methods such as `opsForValue().set(...)` normally borrow a connection, run one command and release it, so a `multi()` followed by separate template calls may land on different connections. A `SessionCallback` solves this: it tells the template to use the same connection for everything inside the callback.

StockReservation.java

JAVA

```
package com.shopstream.inventory;

import java.util.List;
import org.springframework.dao.DataAccessException;
import org.springframework.data.redis.core.RedisOperations;
import org.springframework.data.redis.core.SessionCallback;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Service;

@Service
public class StockReservation {

    private final StringRedisTemplate redis;

    public StockReservation(StringRedisTemplate redis) {
        this.redis = redis;
    }

    /** Reserves one unit with optimistic locking; returns false if someone else changed it. */
    public boolean reserveOne(String sku, String reservationId) {
        String stockKey = "stock:" + sku;
        List<Object> result = redis.execute(new SessionCallback<List<Object>>() {
            @Override
            @SuppressWarnings({"unchecked", "rawtypes"})
            public List<Object> execute(RedisOperations operations) throws DataAccessException {
                operations.watch(stockKey);
                Object current = operations.opsForValue().get(stockKey);
                if (current == null || Long.parseLong(current.toString()) <= 0) {
                    operations.unwatch();
                    return List.of();
                }
                operations.multi();
                operations.opsForValue().decrement(stockKey);
                operations.opsForSet().add("reserved:" + sku, reservationId);
                return operations.exec();
            }
        });
        return result != null && !result.isEmpty();
    }
}
```

What to notice:

- The read happens **before** `multi()`. Inside `MULTI` commands are only queued, so a `get` there would return `null` instead of the value.
- If another client modifies the watched key between `watch` and `exec`, `EXEC` aborts and the template returns an empty or `null` result. The caller must retry or give up. Under contention this loop becomes a retry storm, which is exactly the case for a Lua script instead.
- Redis has no rollback. If a queued command fails at runtime, the others still run. `EXEC` makes the group atomic with respect to other clients, not "all or nothing".

WATCH OUT

Do not call `multi()` on a template outside a `SessionCallback`. A bare `redisTemplate.multi()` followed by other template calls is a bug waiting to happen: the commands may use another connection, or the connection may be returned to the pool with an open transaction. The same applies to the `@Transactional` route below.

7.9.2 @Transactional and transaction support

`RedisTemplate` does not take part in Spring-managed transactions unless you enable it explicitly with `setEnableTransactionSupport(true)` on that template. The reference documentation describes the effect: the connection is bound to the current thread, commands inside the transaction are queued, and when the Spring transaction completes without error the Redis transaction is committed with `EXEC`, otherwise discarded.

There is a catch that surprises people: with transaction support on, **reads inside the transaction return `null`** (they are queued), and the template uses a bound connection that is not returned until the transaction ends. Use it for write-only groups that should follow a database transaction, not for read-modify-write logic. A common alternative is to write to Redis after the database commit with `@TransactionalEventListener(phase = AFTER_COMMIT)`, which avoids binding Redis to the database transaction at all.

7.9.3 Lua scripts with RedisScript

For "check something, then change something" with no gap, a Lua script is the right tool. It runs on the server in one step, so no other command can interleave. This is the flash-sale decrement for `ShopStream`.

scripts/reserve_stock.lua

LUA

```
-- KEYS[1] = stock:{sku}  ARGV[1] = quantity wanted
local stock = tonumber(redis.call('GET', KEYS[1]))
if stock == nil then
    return -1
end
local want = tonumber(ARGV[1])
if stock < want then
    return 0
end
redis.call('DECRBY', KEYS[1], want)
return 1
```

InventoryScripts.java

JAVA

```
package com.shopstream.inventory;

import java.util.List;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.io.ClassPathResource;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.core.script.DefaultRedisScript;
import org.springframework.data.redis.core.script.RedisScript;
import org.springframework.scripting.support.ResourceScriptSource;
import org.springframework.stereotype.Service;

@Configuration
class ScriptConfig {

    @Bean
    RedisScript<Long> reserveStockScript() {
        DefaultRedisScript<Long> script = new DefaultRedisScript<>();
        ClassPathResource file = new ClassPathResource("scripts/reserve_stock.lua");
        script.setScriptSource(new ResourceScriptSource(file));
        script.setResultType(Long.class);
    }
}
```


InventoryScripts.java (continued)

JAVA

```

        return script;
    }
}

@Service
public class InventoryService {

    private final StringRedisTemplate redis;
    private final RedisScript<Long> reserveStockScript;

    public InventoryService(StringRedisTemplate redis, RedisScript<Long> reserveStockScript) {
        this.redis = redis;
        this.reserveStockScript = reserveStockScript;
    }

    public boolean reserve(String sku, int quantity) {
        Long result = redis.execute(
            reserveStockScript, List.of("stock:" + sku), String.valueOf(quantity));
        if (result == null || result < 0) {
            throw new IllegalStateException("unknown SKU " + sku);
        }
        return result == 1L;
    }
}

```

What to notice:

- Keys go in the `keys` list and other values in the arguments. This is not style: in Redis Cluster the keys declared by a script decide which node runs it, and all keys must be in the same hash slot.
- The script bean is a singleton. The reference documentation recommends a single `DefaultRedisScript` instance so that the SHA1 of the script is not recomputed on each call. The default executor first tries `EVALSHA` and falls back to `EVAL` when the script is not yet in the server's script cache, so you get the saving of not re-sending the script text without managing it.
- The result type must be one of `Long`, `Boolean`, `List`, or a deserialized value type. A Lua `nil` or `false` maps to Java `null` or `false`; a script returning the status `OK` can use no result type.
- Beware of long scripts. A script blocks the whole server while it runs. Keep them to a few commands and never loop over big collections.

7.9.4 Pipelining

When you have many independent commands, `executePipelined` sends them without waiting for each reply and returns all results at the end. It is the same idea as `redis-benchmark -P` from the labs.

CartCountLoader.java

JAVA

```

package com.shopstream.cart;

import java.util.List;
import org.springframework.data.redis.core.RedisCallback;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Component;

@Component
public class CartCountLoader {

    private final StringRedisTemplate redis;

    public CartCountLoader(StringRedisTemplate redis) {
        this.redis = redis;
    }

    /** Returns the number of distinct items in each user's cart, one round trip. */
    public List<Object> itemCounts(List<Long> userIds) {
        return redis.executePipelined(
            (RedisCallback<Object>) connection -> {
                for (Long userId : userIds) {
                    connection.hashCommands().hLen(("cart:" + userId).getBytes());
                }
            }
        );
    }
}

```


CartCountLoader.java (continued)

JAVA

```

        return null;
    });
}
}

```

The callback must return `null`; the template collects the replies. The template deserializes the results with the serializers you configured, as the reference documentation notes. A pipeline is **not** atomic: other clients' commands can interleave, and one failing command does not stop the rest. A pipeline also holds all replies in memory, so batch very large jobs (for example 500 to 1000 commands at a time).

GOTCHA

Pipelines and transactions on a shared connection. Do not mix a bound transaction and `executePipelined` casually. With transaction support enabled, the results of a pipeline inside a transaction come back only when the transaction completes, and your code sees surprising values. Keep the two features in separate code paths.

KEY POINTS

- Use a `SessionCallback` so that `WATCH`, `MULTI` and `EXEC` share one connection; read before `multi()`.
- `RedisTemplate` joins `@Transactional` only after `setEnableTransactionSupport(true)`, and reads inside `return null`.
- Use a Lua `RedisScript` for atomic read-decide-write; pass keys as keys, keep one script bean, keep scripts short.
- `executePipelined` cuts round trips but is not atomic.

7.10 Testing Redis Code

Code that talks to Redis needs two kinds of tests. Unit tests check your logic with Redis out of the picture. Integration tests check that your keys, serializers, TTLs and scripts really work against a real Redis server, because the interesting bugs live exactly there: wrong key names, a serializer that cannot read what it wrote, a Lua script with a typo.

7.10.1 Unit tests: keep Redis away

For a service that only uses a template in a few places, mock the template. The code under test does not know whether it talks to Redis or to a fake.

ProductEventPublisherTest.java

JAVA

```

package com.shopstream.catalog;

import static org.mockito.Mockito.mock;
import static org.mockito.Mockito.verify;

import org.junit.jupiter.api.Test;
import org.springframework.data.redis.core.StringRedisTemplate;

class ProductEventPublisherTest {

    @Test
    void publishesProductId() {
        StringRedisTemplate redis = mock(StringRedisTemplate.class);
        ProductEventPublisher publisher = new ProductEventPublisher(redis);

        publisher.productChanged(42L);

        verify(redis).convertAndSend("events.product.changed", "42");
    }
}

```

This is cheap and fast, but it proves nothing about Redis. Keep such tests for branching logic and use real Redis for everything else.

7.10.2 Integration tests with Testcontainers

Testcontainers starts a real Redis in Docker for the test run. It needs a Docker daemon on the machine (and in CI). Spring Boot connects it to your application with `@ServiceConnection`: the container's host and mapped port replace `spring.data.redis.*` for the test context, so you configure nothing by hand.

pom.xml (test dependencies)

XML

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-testcontainers</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>testcontainers</artifactId>
  <scope>test</scope>
</dependency>
```

RedisTestConfig.java

JAVA

```
package com.shopstream.support;

import org.springframework.boot.test.context.TestConfiguration;
import org.springframework.boot.testcontainers.service.connection.ServiceConnection;
import org.springframework.context.annotation.Bean;
import org.testcontainers.containers.GenericContainer;

@TestConfiguration(proxyBeanMethods = false)
public class RedisTestConfig {

    @Bean
    @ServiceConnection(name = "redis")
    GenericContainer<?> redisContainer() {
        return new GenericContainer<>("redis:8").withExposedPorts(6379);
    }
}
```

The `name = "redis"` attribute is required: Spring Boot cannot tell from a `GenericContainer` which image it runs, so the name is the hint that selects the Redis connection details. Because the container is a Spring bean, Spring starts it before the beans that need it and stops it when the context closes; you do not need the JUnit `@Testcontainers` extension.

Now a real test of the stock script and the cache, using the full application context.

InventoryIT.java

JAVA

```
package com.shopstream.inventory;

import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;
import org.springframework.data.redis.core.RedisCallback;
import org.springframework.data.redis.core.StringRedisTemplate;
import com.shopstream.support.RedisTestConfig;

@SpringBootTest
@Import(RedisTestConfig.class)
class InventoryIT {

    @Autowired StringRedisTemplate redis;
    @Autowired InventoryService inventory;

    @BeforeEach
    void clean() {
        redis.execute((RedisCallback<Object>) c -> {
            c.serverCommands().flushDb();
        });
    }
}
```

InventoryIT.java (continued)

JAVA

```

        return null;
    });
}

@Test
void neverOversells() {
    redis.opsForValue().set("stock:sku-1", "3");

    boolean first = inventory.reserve("sku-1", 2);
    boolean second = inventory.reserve("sku-1", 2);

    assertThat(first).isTrue();
    assertThat(second).isFalse();
    assertThat(redis.opsForValue().get("stock:sku-1")).isEqualTo("1");
}
}

```

What to notice: every test starts from an empty database, because leftovers from one test make the next one flaky; flushing is acceptable here because this Redis exists only for the test run. Never run this against a shared instance. The class name ends in **IT** by convention so that a build tool can run it in the integration phase.

If you cannot use `@ServiceConnection`, the older approach works on every version: start the container in a static field and publish its address with a `@DynamicPropertySource` method that calls `registry.add("spring.data.redis.host", ...)` and `registry.add("spring.data.redis.port", ...)`.

7.10.3 Test slices

Boot 4 has a Redis test slice, `@DataRedisTest`, in the module `spring-boot-data-redis-test` (package `org.springframework.boot.data.redis.test.autoconfigure`). It starts only the Redis auto-configuration (connection factory, templates, repositories) and not your whole application, so it is faster for tests that only exercise Redis code. The slice does not component-scan your services, so import what you need.

ProductEventPublisherSliceTest.java

JAVA

```

package com.shopstream.catalog;

import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.data.redis.test.autoconfigure.DataRedisTest;
import org.springframework.context.annotation.Import;
import org.springframework.data.redis.core.StringRedisTemplate;
import com.shopstream.support.RedisTestConfig;

@DataRedisTest
@Import(RedisTestConfig.class)
class ProductEventPublisherSliceTest {

    @Autowired StringRedisTemplate redis;

    @Test
    void roundTrip() {
        redis.opsForValue().set("ff:new-checkout", "on");
        assertThat(redis.opsForValue().get("ff:new-checkout")).isEqualTo("on");
    }
}

```

The slice list in the Boot reference shows that this slice imports the Redis, reactive Redis and Redis repository auto-configurations plus SSL and the optional Testcontainers service-connection auto-configuration. The name of the matching starter artifact for test scope is not something this book could confirm; the module `spring-boot-data-redis-test` is the one that holds the annotation.

7.10.4 Embedded Redis and fakes

Several community libraries start an "embedded" Redis binary inside the test JVM, and there are in-memory fakes that speak the Redis protocol. They look convenient, but they have three problems: they usually lag behind current Redis versions, they implement only part of the commands (rarely Lua details, Streams edge cases, or module commands such as JSON), and they behave differently from the real server in exactly the places you want to test. A container with the same image tag as production avoids all three. Use a fake only if Docker is truly unavailable, and then limit its use to trivial key-value tests.

7.10.5 Making tests reliable

- **Pin the image tag** to the major version you run, for example `redis:8`, so tests change only when you change them.
- **Reuse the container** across test classes: a static or shared configuration starts it once per test run. Starting a container per test method is slow.
- **Test expiry without sleeping**: set a short TTL in the test configuration, or assert on `redis.getExpire(key)`, instead of `Thread.sleep`. For logic that depends on time, inject a `Clock`.
- **Test Redis being down**: stop the container (or point the test at a closed port) and assert that your `CacheErrorHandler` and timeouts keep the service answering.
- **Assert on the keys**: a test that scans for `product:*` and expects exactly one key catches key-name regressions that mocks cannot see.

KEY POINTS

- Unit tests mock the template for logic; integration tests use a real Redis from Testcontainers (`redis:8`).
- `@Bean @ServiceConnection(name = "redis") GenericContainer<?>` wires the container to the application; the `name` is required for `GenericContainer`.
- `@DataRedisTest` is the Redis test slice in Boot 4; `@DynamicPropertySource` is the manual alternative.
- Clean the database before each test and avoid sleeping; test failure paths too.
- Prefer a real container over embedded or fake Redis.

7.11 Observability and a Production Checklist

A Redis-backed Spring service has two places to look when something is wrong: Redis itself (its `INFO` output, slow log, memory; see the Operations part) and the application's view of Redis: how long commands take from the client side, how many connections are open, whether the cache is doing its job. This chapter covers the application side.

7.11.1 Health

With Spring Boot Actuator on the classpath and a Redis connection factory present, a Redis health indicator is registered automatically and appears as the `redis` component of `/actuator/health`. It reports `UP` when it can talk to Redis and includes the server version in the details. Show details only to authorised users:

```
application.yml
management:
  endpoints:
    web:
      exposure:
        include: health,metrics,prometheus
    endpoint:
      health:
        show-details: when-authorized
        probes:
          enabled: true
```

YAML

Think before tying Redis health to your Kubernetes **readiness** or **liveness** probe. If Redis is only a cache and the service can serve from the database, a Redis outage should not take every pod out of rotation (that turns a performance problem into a full outage). If Redis is a hard dependency (a session store), a failing readiness probe is correct. Decide per service, and keep liveness independent from Redis in either case.

7.11.2 Metrics

Three groups of metrics matter.

Cache hit ratio

When statistics are enabled on the cache manager (`enableStatistics()` or `spring.cache.redis.enable-statistics: true`), Micrometer's cache instrumentation binds the caches, and you get meters such as `cache.gets` with a `result` tag of `hit` or `miss`, and `cache.puts`. The hit ratio is hits divided by all gets. Graph it per cache. A cache below, say, 50% for a read-heavy path deserves a look at its TTL, key design or whether it should exist at all. Treat that number as a prompt for investigation, not as a universal target.

Client-side command latency

Lettuce can record command latency through Micrometer. Register your own `ClientResources` bean with a Micrometer recorder; Boot's auto-configured resources step aside when you define one.

LettuceMetricsConfig.java

JAVA

```
package com.shopstream.config;

import io.lettuce.core.metrics.MicrometerCommandLatencyRecorder;
import io.lettuce.core.metrics.MicrometerOptions;
import io.lettuce.core.resource.ClientResources;
import io.micrometer.core.instrument.MeterRegistry;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class LettuceMetricsConfig {

    @Bean(destroyMethod = "shutdown")
    ClientResources lettuceClientResources(MeterRegistry registry) {
        MicrometerOptions options = MicrometerOptions.create();
        return ClientResources.builder()
            .commandLatencyRecorder(new MicrometerCommandLatencyRecorder(registry, options))
            .build();
    }
}
```

The recorder publishes timers with a `lettuce` prefix, tagged by command and host. This is the number that tells you what your application experiences, including network and queueing, which the server-side `SLOWLOG` cannot show. Compare the two when users complain about latency but the slow log is empty. Check the Lettuce reference for the exact meter names of your version before building dashboards on them.

Connections and pool

If you use pooling (`commons-pool2` on the classpath), track active and idle connections with the pool's own metrics or JMX. On the server, watch `connected_clients` in `INFO clients`. A client count that rises with traffic and never falls is a leak or a missing pool limit.

7.11.3 Logging

Keep `io.lettuce.core` and `org.springframework.data.redis` at `INFO` or `WARN` in production. Raise `io.lettuce.core` to `DEBUG` temporarily to see connection and reconnect events when you are chasing a flaky network. Do not enable `TRACE`: it logs every command and can leak data into the logs. Log cache errors from your `CacheErrorHandler` at `WARN` with the cache name and key (never the value), and count them with a metric so that a silent Redis outage shows on a dashboard.

7.11.4 Production checklist

Area	Check
Timeouts	<code>spring.data.redis.timeout</code> (command timeout) and <code>connect-timeout</code> are set to values you chose, not left at defaults; the service fails fast when Redis is slow
Connections	One shared Lettuce connection is usually enough; pooling is configured deliberately if you use blocking calls or transactions
Security	Redis ACL user with only the commands and key patterns needed; TLS enabled (<code>spring.data.redis.ssl.bundle</code>); password in a secret store
Keys	A prefix per service or environment; no KEYS calls; explicit cache keys
TTL	Every cache has a TTL; session and idempotency keys have TTLs; jitter on mass-loaded caches
Serialization	JSON serializers chosen on purpose; no Java serialization in shared data; a plan for schema changes
Failure	<code>CacheErrorHandler</code> in place; behaviour with Redis down tested; circuit breaker or fallback where the user path depends on Redis
Memory	<code>maxmemory</code> and <code>maxmemory-policy</code> set on the server; no unbounded streams, lists or sets
Streams	Group created at startup; manual ack; reclaimer and dead-letter stream; trimming policy
Observability	Health indicator, hit ratio, Lettuce latency, connection count, error counters on a dashboard with alerts
Tests	Testcontainers integration tests with the same Redis major version as production
Deployment	Rolling restarts handled: stable consumer names, graceful container shutdown, reconnect behaviour tested with a failover

Spring and Redis, before go-live.

REAL WORLD

The first outage is rarely Redis being down. Typical first incidents in Spring services: a cache without TTL that fills memory until the eviction policy (or `noeviction` errors) bites; a missing timeout so that threads pile up behind a slow Redis; a stream consumer that stopped after the first exception; a **KEYS** call in an admin endpoint. Every item in the checklist above exists because teams have hit it.

INTERVIEW ANGLE

How would you monitor a Spring Boot service that uses Redis as a cache? From the application: the cache hit ratio per cache (Micrometer, statistics enabled), Lettuce command latency, error counts from the cache error handler, and the health indicator. From Redis: memory use against `maxmemory`, evicted and expired keys, connected clients, slow log and latency. Alert on a falling hit ratio, rising latency percentiles, evictions when you did not expect them, and rising error counts.

KEY POINTS

- The Redis health indicator is automatic with Actuator; think carefully before wiring it into readiness probes.
- Enable cache statistics for hit ratio; register a Micrometer command latency recorder for Lettuce.
- Log cache errors without values, keep **TRACE** off in production, and count failures.
- Walk the checklist: timeouts, ACL and TLS, TTLs on everything, failure behaviour, bounded memory, tested stream consumers.

PART 8

Patterns

Proven recipes that combine the commands you already know: caching, locking, rate limiting, queues, counters and more.

In this part

- 8.1** Caching fundamentals
- 8.2** Cache-aside
- 8.3** Read-through, write-through, write-behind and refresh-ahead
- 8.4** Cache failure modes
- 8.5** Keeping cache and database consistent
- 8.6** Multi-level caching
- 8.7** Distributed locks
- 8.8** Rate limiting
- 8.9** Idempotency keys and deduplication
- 8.10** Queues and background jobs
- 8.11** Delayed and scheduled tasks
- 8.12** Counters, analytics and unique counts
- 8.13** Leaderboards and ranking
- 8.14** Sessions, carts and ephemeral state
- 8.15** Feature flags, configuration and flash-sale stock
- 8.16** Real-time messaging patterns
- 8.17** Search, geo and AI: semantic caching and RAG
- 8.18** Anti-patterns catalogue

8.1 Caching fundamentals

Caching is the single most common reason people install Redis, and also the most common source of subtle production bugs. A cache is a small, fast copy of data that lives somewhere else (the **source of truth**, usually a relational database). Reading from the copy is cheaper than reading from the source, so you do it first. The idea is trivial; the difficulty is entirely in the word "copy". A copy can become wrong, can disappear, can be missing exactly when everybody asks for it, and can quietly cost more than it saves.

This chapter gives you the vocabulary and the decisions that apply to every caching pattern in the rest of this Part: what is worth caching, how to measure whether the cache works, how to choose a TTL, how to name keys, how to serialize values, and how to live with stale data. The next chapters then build the actual patterns (cache-aside, write-through and friends) and the failure modes.

8.1.1 Why a cache helps

A request to a relational database typically costs one network round trip plus query planning, index lookups, possibly disk reads, and row serialization. That is often 1 to 20 milliseconds under light load and much more under load. A Redis **GET** of a small value costs one network round trip plus a hash table lookup, typically well under a millisecond inside the same data centre. The numbers vary with your hardware and network, so treat them as orders of magnitude, but the shape is stable: a cache hit is one or two orders of magnitude cheaper than a database read.

The second benefit matters more in production than the first. A cache **protects the database**. If 95% of reads are answered by Redis, the database sees 5% of the traffic, and a traffic spike that would have flattened it becomes survivable. This is also why cache failures are dangerous: when the cache is cold or down, all the traffic lands on a database that was sized on the assumption that the cache exists.

8.1.2 What to cache, and what not to

Good cache candidates share three properties: they are **read much more often than written**, they are **expensive to compute or fetch**, and they **tolerate being slightly stale**. Typical examples in ShopStream:

- the product detail page (`product:{id}`): read thousands of times per update, a joined query over several tables;
- category listings and the home page "featured" block: expensive aggregations, same answer for every visitor;
- results of a call to a slow third party (shipping rate lookup, currency rates);
- computed values such as "number of reviews and average rating".

Poor candidates:

- data that changes on every request or is unique per request (a cache hit ratio near zero costs memory and adds latency to every miss);
- data that must never be stale: the exact stock of a flash-sale item at the moment of purchase, an account balance before a debit. You can show a cached "about 12 left" on the product page, but the decision to sell must be made against the source of truth (or against Redis used as the primary store with the right atomic operations, see the chapter on flash-sale stock later in this Part);
- huge values that are rarely re-read (a 5 MB report for one user);
- anything cheap. A primary-key lookup on a warm database table may be fast enough that a cache adds complexity for no gain. Measure first.

TIP

Cache the **shape the caller needs**, not the raw rows. A cached `ProductView` record that already contains price, rating and image URLs saves the join and the mapping code. A cached copy of the `products` row saves only one query out of the four you still have to run.

8.1.3 Hit ratio: the one number that matters

The **hit ratio** is the fraction of lookups answered from the cache:

```
hit_ratio = hits / (hits + misses)
```

Redis counts keyspace hits and misses server-wide; you read them from `INFO stats`:

Server-wide hit and miss counters

TERMINAL

```
127.0.0.1:6379> INFO stats
# Stats
...
keyspace_hits:1843920
keyspace_misses:90211
...
```

With those two numbers the ratio is $1843920 / (1843920 + 90211) \approx 0.953$, or 95%. Two cautions. First, the counters are for the whole server and count every command that looks up a key, including `GETs` that your application uses for locks or rate limits, so an application-level metric (a Micrometer counter in your cache layer) is more truthful for a specific cache. Second, the ratio alone does not tell you whether the cache is **worth** having. The useful way to think is expected cost per request:

```
avg_latency = hit_ratio * t_hit + (1 - hit_ratio) * (t_hit + t_source)
```

If a hit costs 0.5 ms, the database read costs 10 ms and the hit ratio is 90%, the average is $0.5 + 0.1 \times 10 = 1.5$ ms, against 10 ms without a cache. At 50% the average is 5.5 ms; at 20% it is 8.5 ms and you are paying memory and complexity for a small gain. These figures are illustrative.

What decides your hit ratio? Mostly three things: the **popularity distribution** of your keys (a few products get most views, which caches love), the **TTL** (shorter TTLs mean more misses) and the **memory budget** (if the working set does not fit, eviction removes entries before they are re-read).

8.1.4 Choosing a TTL

A time-to-live (TTL) is the maximum age of a cached value. It has two roles: it bounds staleness, and it bounds memory (entries nobody asks for again disappear). Choose it from the **business tolerance for stale data**, not from habit:

Data	TTL	Reasoning
Product description and images	1 hour to 1 day	Edited rarely; also explicitly invalidated on edit
Product price	1 to 5 minutes	Wrong prices hurt; invalidate on change as well
Category page with counts	30 to 60 seconds	Expensive query, nobody notices a minute of lag
Result of "not found" lookup	30 to 60 seconds	Short, because the item may appear soon (negative caching)
Third party exchange rate	the provider's update interval	No point in asking more often than the data changes

Illustrative TTL starting points for ShopStream data.

Three practical rules:

1. **Always set a TTL**, even when you also invalidate explicitly. The TTL is your safety net for every bug in the invalidation code. A key without a TTL that is never deleted stays forever; this is how caches silently fill with garbage and why the anti-pattern list at the end of this Part has "no TTL" in it.
2. **Add jitter** when many keys are created at the same moment (a bulk warm-up, a deployment). If 100,000 keys all get exactly 600 seconds, they all expire in the same second. The chapter on cache failure modes explains the damage and the fix.
3. **Do not use one global TTL**. Group data by how stale it may be and give each group its own.

8.1.5 Key design and versioned keys

Cache keys must be **deterministic** (same inputs, same key), **unique** (different inputs, different key) and **readable** (you will debug them with `redis-cli`). Use the `entity:id:field` convention with colons, and put in the key every input that changes the result:

```
product:1042                one product
product:1042:v3              one product, schema version 3
catalog:page:electronics:2:sort=price:lang=en  a listing; every parameter is part of the key
user:77:recommendations
```

Two key-design ideas pay for themselves quickly.

Schema version in the key. When you change the shape of the cached object (add a field, rename a property), old entries in Redis still have the old shape, and the new code cannot read them. Instead of flushing, put a version in a key prefix: `shopstream:v3:product:1042`. Deploy the new code and it reads and writes `v3` keys, ignoring `v2` ones, which age out through their TTL. Rolling deployments, where old and new application instances run side by side, work because each version reads only its own keys.

Version token for group invalidation. Sometimes one change must invalidate many keys: "a category was renamed, so every listing page of that category is wrong". Deleting a set of keys by pattern is expensive (it needs `SCAN`, never `KEYS`). A cleaner trick is to store a small counter `catalog:ver:electronics` and include its value in the listing keys: `catalog:electronics:v{ver}:page:2`. To invalidate the whole group you `INCR` the counter. Old pages are no longer referenced and expire naturally.

GOTCHA

Keys are not free either. A key like `catalog:page:electronics:2:sort=price:lang=en` is fine. But never include unbounded user input directly (a free-text search query of 2,000 characters) in a key: hash it first (for example with SHA-256 and hex-encode) and keep the readable parts short. Also remember that Redis cluster hashes the whole key, or the part in `{...}`, to decide the shard; see the chapter on cluster hash tags in the scaling Part.

8.1.6 Serialization

Redis stores bytes. Your Java object must be turned into bytes on write and back on read. Your choice affects size, speed, readability and compatibility across deploys.

Format	Good	Watch out
JSON (Jackson)	Human readable in <code>redis-cli</code> ; works across languages; tolerant of added fields	Larger than binary; need to pin the types you deserialize
Java serialization	Zero setup (the default for <code>RedisTemplate</code>)	Opaque bytes; breaks when the class changes; security risk on untrusted data; do not use
Binary (Protobuf, MessagePack, Kryo)	Compact and fast	Needs schema discipline; not readable in the CLI
Plain string / hash fields	Smallest and cheapest for tiny values; you can update one field with <code>HSET</code>	You write the mapping yourself

Common serialization choices for cached values.

For a book-sized service the sensible default is **JSON through Jackson**. In Spring Data Redis 4 that means `GenericJacksonJsonRedisSerializer` or the typed `JacksonJsonRedisSerializer<T>` (Jackson 3). The typed one is safer for caches: it knows exactly which class to read, so you never need to embed class names in the JSON, which also avoids the polymorphic deserialization risks that come with default typing. The Spring chapters earlier in this book show how to wire them.

A practical point: design cached objects as **immutable records of simple types**. A record `ProductView(long id, String name, long priceCents, double rating)` serializes the same way in every JVM and survives refactoring much better than an entity with lazy-loaded collections.

8.1.7 Stale data is a feature you manage

A cache that is never wrong is a cache that is never used. Staleness is the price of the speed, so the right questions are: **how stale, for whom, and what happens if it is wrong?** The "5 minutes old price" on the listing page is fine as long as the checkout recomputes the price from the database. Draw this line explicitly in your design: *display paths* may read the cache; *decision paths* (charge, reserve, authorize) read the source of truth.

Common staleness bugs:

- the write path updates the database and forgets to touch the cache (the TTL eventually hides the bug, which makes it hard to find);
- a read racing with a write puts an old value back into the cache after the invalidation (explained in the next chapters);
- a second service writes to the same table and knows nothing about your cache.

8.1.8 Cache warming

A **cold** cache (empty after a restart, a failover to a fresh node, a flush, a new region) has a hit ratio of zero and sends the full load to the database. For small systems this does not matter. For busy systems you **warm** the cache before you take traffic, or you pace the traffic:

- pre-load the hottest keys from a list (the top 10,000 products by views yesterday), with jittered TTLs;
- run the warm-up as a throttled background job, not as a thundering burst of queries against the database;
- keep Redis persistent (RDB or AOF) if re-warming is expensive, so a restart reloads the data from disk rather than from the database;
- when you deploy a new cache key version, warm the new keys before switching the traffic to them.

8.1.9 Configuring Redis to behave as a cache

A Redis instance used purely as a cache wants a different configuration from one that stores primary data. The settings that matter (each is explained in depth in the memory and persistence chapters of the Redis Inside Part):

redis.conf (cache-only instance)

CONF

```
# Hard memory ceiling for the dataset. Leave headroom for buffers and fork copy-on-write.
maxmemory 2gb

# When the ceiling is reached, evict instead of returning errors on writes.
# allkeys-lfu keeps frequently used keys; allkeys-lru is the classic alternative.
maxmemory-policy allkeys-lfu

# Free memory in a background thread when keys are evicted, expired or overwritten.
lazyfree-lazy-eviction yes
lazyfree-lazy-expire yes
lazyfree-lazy-server-del yes

# A pure cache can usually be rebuilt, so persistence is optional.
# Disable snapshots if a cold start after a restart is acceptable:
save ""
appendonly no
```

What to notice. The default `maxmemory-policy` is `noeviction`, which means a full instance answers write commands with an out-of-memory error. That is right for a primary store and wrong for a cache, so a cache needs an eviction policy. With `allkeys-*` policies Redis may evict **any** key, including keys that have no TTL; with `volatile-*` policies only keys that have a TTL are candidates, which is the safer choice if the same instance also holds a few permanent keys (feature flags, for example) that must not disappear. If you disable persistence, remember the consequence: a restart or a failover to an empty node produces a completely cold cache, and the avalanche defences from the failure-modes chapter become important.

TIP

Use one Redis instance (or at least one logical purpose) per role when you can. A cache that is allowed to evict and a lock or stock store that must never lose a key have opposite requirements. Mixing them under `allkeys-lru` means eviction can silently delete a lock or a counter. Either run separate instances, or use `volatile-*` policies and make sure only cache keys carry TTLs.

8.1.10 A first look at a cache in redis-cli

Before writing Java, watch the mechanics in a terminal. This session shows a miss, a fill with a TTL, a hit, the countdown and the expiry.

Cache lifecycle by hand

TERMINAL

```
127.0.0.1:6379> GET shopstream:v1:product:1042
(nil)
127.0.0.1:6379> SET shopstream:v1:product:1042 '{"id":1042,"price":8900}' EX 600
OK
127.0.0.1:6379> GET shopstream:v1:product:1042
{"id":1042,"price":8900}
127.0.0.1:6379> TTL shopstream:v1:product:1042
(integer) 594
127.0.0.1:6379> DEL shopstream:v1:product:1042
(integer) 1
127.0.0.1:6379> GET shopstream:v1:product:1042
(nil)
```

The first `GET` is the miss, `SET ... EX 600` is the fill, the next `GET` is the hit, `TTL` counts down, and `DEL` is the invalidation after a database write. Cache-aside in four commands; everything else in this Part is about doing those four steps safely under concurrency, at scale, and with failures.

INTERVIEW ANGLE

How do you decide whether something should be cached? Cache data that is read far more than it is written, expensive to produce, and tolerant of slight staleness. Then measure: compute the hit ratio and the saved latency. Always set a TTL as a safety net, separate display paths (can be stale) from decision paths (must read the source of truth), and have a plan for a cold cache and for the database load when the cache is down.

KEY POINTS

Caching fundamentals

- A cache is a copy; every caching decision is a decision about how wrong the copy may be.
- Cache read-heavy, expensive, staleness-tolerant data in the shape the caller needs.
- Measure the hit ratio (`keyspace_hits`, `keyspace_misses`, or better an application metric) and think in expected latency.
- Always set a TTL, vary it per data class, add jitter to keys created together.
- Keys are deterministic and include every input; put a schema version in the key to survive deployments.
- Prefer JSON via Jackson with immutable records; plan for a cold cache.

8.2 Cache-aside

Cache-aside (also called lazy loading) is the default caching pattern, and the one you will write most often. The application code, not Redis, is responsible for filling the cache. Redis is a dumb, fast key-value store that sits beside the database, and that is exactly why it is so popular: any application can use it with nothing more than `GET`, `SET` and `DEL`.

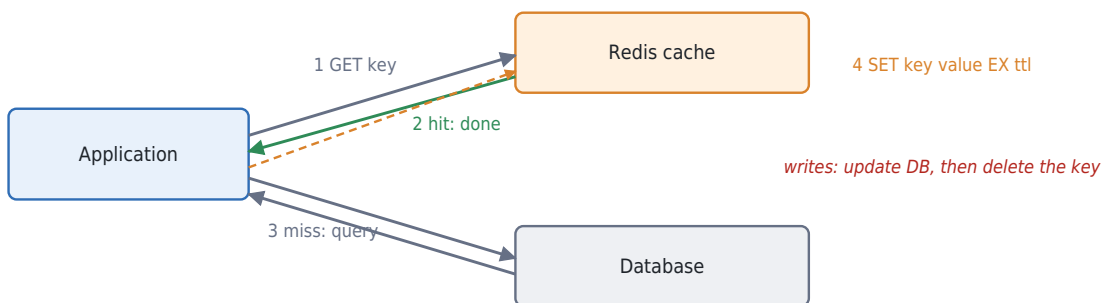


Figure 8.1. Cache-aside: on a read the application checks Redis first and, on a miss, loads from the database and fills the cache; on a write it changes the database and invalidates the cache entry.

8.2.1 The read flow

1. The application computes the cache key and runs `GET` on Redis.
2. **Hit:** a value is returned. Deserialize it and return it. Done.
3. **Miss:** `GET` returns `(nil)`. The application queries the database.
4. The application writes the result to Redis with a TTL (`SET key value EX 600`).
5. It returns the result to the caller.

On the very first request for a key you pay the full price plus one extra round trip to Redis. Every later request, until the entry expires or is removed, is a hit. That is the meaning of "lazy": data enters the cache only when somebody asks for it, so the cache naturally contains the popular data and nothing else.

8.2.2 The write flow

When data changes you update the **database first** and then deal with the cache. You have two options.

- **Invalidate** (delete the cache entry): `DEL product:1042`. The next read misses, loads the fresh row and refills the cache. Simple and robust.
- **Update** (overwrite the cache entry with the new value): needs the application to have the exact new representation, and creates a race when two writers interleave (below).

The strong default is **invalidate, do not update**. Deleting is idempotent (you can safely repeat it), does not require building the cached representation on the write path, and cannot leave a wrong value behind that lives until the TTL. The cost is one extra miss after each write, which is cheap for read-heavy data.

8.2.3 Code: a plain Java cache-aside with StringRedisTemplate

This is a complete, minimal implementation without any Spring Cache annotations, so every step is visible. It uses a Jackson 3 `JsonMapper` and `StringRedisTemplate`.

```
ProductCache.java JAVA

package com.shopstream.catalog;

import java.time.Duration;
import java.util.Optional;
import java.util.concurrent.ThreadLocalRandom;

import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Service;

import tools.jackson.databind.json.JsonMapper;

@Service
public class ProductCache {

    private static final Duration BASE_TTL = Duration.ofMinutes(10);

    private final StringRedisTemplate redis;
    private final JsonMapper json;
    private final ProductRepository repository;

    public ProductCache(StringRedisTemplate redis, JsonMapper json,
        ProductRepository repository) {
        this.redis = redis;
        this.json = json;
        this.repository = repository;
    }

    public Optional<ProductView> find(long id) {
        String key = key(id);
        String cached = redis.opsForValue().get(key); // 1. look in the cache
        if (cached != null) {
            return Optional.of(json.readValue(cached, ProductView.class)); // 2. hit
        }
        Optional<ProductView> loaded = repository.findViewById(id); // 3. miss
        loaded.ifPresent(view ->
            redis.opsForValue().set(key, json.writeValueAsString(view), ttlWithJitter())); // 4
        return loaded; // 5
    }

    public void update(long id, ProductUpdate change) {
        repository.update(id, change); // 1. database first (the source of truth)
        redis.delete(key(id)); // 2. then invalidate
    }

    private static String key(long id) {
        return "shopstream:v1:product:" + id;
    }

    private static Duration ttlWithJitter() {
        long jitterSeconds = ThreadLocalRandom.current().nextLong(0, 60);
        return BASE_TTL.plusSeconds(jitterSeconds);
    }
}
```

What to notice

- `json.readValue` and `writeValueAsString` are Jackson 3 methods and do not throw checked exceptions, so the code stays short. Inject the `JsonMapper` that Spring Boot auto-configures.
- The key carries a schema version (`v1`) and the entity id.
- `ttlWithJitter()` adds up to 60 seconds, so entries created together do not expire together.
- The write path changes the database before it deletes the cache entry. Reversing the order is a bug, as the next section shows.
- A "not found" result is not cached here. The failure-modes chapter shows how to cache negatives safely.

8.2.4 The same thing with Spring's cache annotations

You rarely need to write the code above by hand in a Spring application. `@Cacheable` is cache-aside implemented by a proxy around your method, and `@CacheEvict` is the invalidate step. The Spring chapter on the cache abstraction covers configuration; this is the shape:

```
CatalogService.java JAVA

package com.shopstream.catalog;

import org.springframework.cache.annotation.CacheEvict;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class CatalogService {

    private final ProductRepository repository;

    public CatalogService(ProductRepository repository) {
        this.repository = repository;
    }

    @Cacheable(cacheNames = "products", key = "#id", unless = "#result == null")
    public ProductView getProduct(long id) {
        return repository.findViewById(id).orElse(null);
    }

    @Transactional
    @CacheEvict(cacheNames = "products", key = "#id")
    public void updateProduct(long id, ProductUpdate change) {
        repository.update(id, change);
    }
}
```

The annotation version has two subtle behaviours to know. `@CacheEvict` by default runs **after the method returns successfully**, which is not necessarily after the surrounding transaction commits (see the next section for why that matters). And calling `getProduct` from another method of the same class bypasses the proxy, so the cache is not consulted at all.

8.2.5 Race conditions in cache-aside

Cache-aside is simple, but it is not atomic: "read the database, then write the cache" are two separate steps, and other threads and processes run in between. Three classic races follow. Understand them, because they are the content of most "caching consistency" interview questions.

Race 1: stale fill after a write

Two threads, A reads and B writes, interleave like this:

time	Thread A (reader)	Thread B (writer)
1	GET product:1 -> miss	
2	SELECT row	-> price 10
3		UPDATE price = 12 (commit)
4		DEL product:1 (nothing to delete)
5	SET product:1 {price: 10}	<- stale value written AFTER the delete
6	...	cache now says 10, database says 12

The cache holds a stale value until the TTL expires. The window is narrow (A must be slow between its read and its write), but under load it happens. This is why the TTL is not optional, and why systems that need tighter consistency add a version check or a short lock (see the chapter on keeping cache and database consistent).

Race 2: wrong order of write steps

If the writer deletes the cache entry **before** updating the database, a reader can load the old row in between and put the old value back:

Writer: DEL product:1 -> Reader: miss, SELECT old row, SET old value -> Writer: UPDATE database

Now the stale value is in the cache after the database already changed, and there is no later delete to clean it up. With "database first, delete second" the bad interleaving needs a reader that started *before* the write and finishes *after* the delete (Race 1), which is much less likely. So: **write the database, then delete the cache.**

Race 3: delete before commit

If the delete happens inside a transaction that has not committed yet, a reader can miss, read the old committed row (the new one is not visible yet), and refill the cache with it before the commit. The fix is to delete **after the commit**, for instance with a `@TransactionalEventListener(phase = AFTER_COMMIT)` or Spring's `transactionAware()` cache manager, which postpones cache puts and evictions until the surrounding transaction commits.

WATCH OUT

The order is: commit the database change, then invalidate the cache. Anything else lets a concurrent reader put the old value back. Even in the right order a rare stale fill remains possible, so keep the TTL as a safety net.

8.2.6 What if the cache call itself fails?

A cache must never be able to take your application down. If Redis times out or refuses the connection, the read path should **log, count, and fall back to the database** instead of returning an error. The write path is trickier: if the database write succeeded but the `DEL` failed, you now have a stale entry. Options are to retry the delete a few times, to publish the key to a small retry queue, or to rely on a short TTL. Decide it deliberately; do not leave it to chance. The chapter on connection management in the Java clients Part covers timeouts and graceful degradation.

8.2.7 Caching a value as a hash instead of a JSON string

Storing the whole object as one JSON string is the simplest choice, but Redis hashes offer something a string cannot: you can read or change **one field** without rewriting the whole value. For an entry with a frequently changing field (the stock count shown on the product page, a view counter) a hash is a natural fit.

A product cached as a hash

TERMINAL

```
127.0.0.1:6379> HSET product:1042 name "Trail Shoe" priceCents 8900 rating 4.6 stockHint 12
(integer) 4
127.0.0.1:6379> EXPIRE product:1042 600
(integer) 1
127.0.0.1:6379> HGET product:1042 priceCents
"8900"
127.0.0.1:6379> HINCRBY product:1042 stockHint -1
(integer) 11
127.0.0.1:6379> HGETALL product:1042
1) "name"
2) "Trail Shoe"
3) "priceCents"
4) "8900"
5) "rating"
6) "4.6"
7) "stockHint"
8) "11"
```

The trade-offs: field values are strings, so numbers and nested structures need converting; there is a single TTL per key (Redis 7.4 and later can also expire individual hash fields with `HEXPIRE`, which is rarely needed for caches); and a missing key still means a miss for the whole object. With Spring Data Redis you can map an object to a hash with `HashMapper` implementations (see the hash mapping chapter in the Spring Part). Use a JSON string for "read the whole thing, replace the whole thing" caches and a hash when partial updates are really useful.

8.2.8 Caching lists and pages

Listings (a category page, search results, "customers also bought") need extra care, because they are **derived from many rows** and any of those rows can change.

- **Cache per page, not per query.** Key the page by every parameter: `catalog:electronics:sort=price:page=2:size=20`. A short TTL (30 to 120 seconds) is usually the best invalidation strategy for listings, because a single product edit would otherwise have to invalidate every page that might contain it.
- **Cache ids, not objects.** Store the ordered list of product ids per page (a small value) and resolve each id from the product cache (`MGET` for all ids in one round trip). Now a product edit invalidates one `product:{id}` key, and every

page that shows it automatically displays the new data on its next read. This is the most effective trick for listing caches.

- **Do not cache unbounded results.** Cap page size and total page count. Deep pagination (`page=48000`) is a penetration and big-key risk.

ListingCache.java

JAVA

```
package com.shopstream.catalog;

import java.util.List;
import java.util.Objects;

import org.springframework.data.redis.core.StringRedisTemplate;

public class ListingCache {

    private final StringRedisTemplate redis;

    public ListingCache(StringRedisTemplate redis) {
        this.redis = redis;
    }

    /** Resolve a cached page of product ids into cached product JSON in one round trip. */
    public List<String> productJsonForPage(List<Long> idsOnPage) {
        List<String> keys = idsOnPage.stream()
            .map(id -> "shopstream:v1:product:" + id)
            .toList();
        List<String> values = redis.opsForValue().multiGet(keys); // MGET
        // null entries are per-product misses: load those from the database and fill them
        return values == null ? List.of() : values.stream().filter(Objects::nonNull).toList();
    }
}
```

`multiGet` sends one `MGET`, so a page of 20 products costs one round trip instead of 20. In a Redis Cluster, `MGET` across keys in different slots is not allowed by the server; the Lettuce and Spring clients split such calls per node for you, but the single-round-trip property no longer holds. Keep that in mind when you choose key names (see the cluster chapter).

8.2.9 Pros and cons

Strengths	Weaknesses
Simple; works with any database and any Redis	Application code carries the caching logic (easy to forget or duplicate)
Cache contains only what was requested	First request per key is slow (cold miss)
A Redis outage degrades performance instead of correctness	Race conditions between fill and invalidation
Cache and database can use different data models	Two systems to keep in step; data can be stale up to the TTL

Cache-aside trade-offs.

INTERVIEW ANGLE

Explain cache-aside and its main race condition. Read: check the cache, on a miss load from the database and populate the cache with a TTL. Write: update the database, then delete the cache entry. The main race is a reader that loaded the old row before a write, and puts it into the cache after the writer's delete, leaving stale data until the TTL. Mitigations: always set a TTL, delete after commit, optionally a delayed second delete or a version check.

KEY POINTS

Cache-aside

- The application loads data on a miss and stores it with a TTL; Redis never talks to the database.
- On writes, update the database first and then delete the cache entry; deleting is safer than overwriting.
- Delete after the transaction commits, never before.
- A reader that started before a write can still re-fill a stale value; the TTL bounds the damage.
- Treat Redis failures as cache misses on reads; decide explicitly what to do when an invalidation fails.
- `@Cacheable` and `@CacheEvict` implement this pattern, with self-invocation as the usual trap.

8.3 Read-through, write-through, write-behind and refresh-ahead

Cache-aside puts the caching logic in your application. The other classic patterns move that logic into a **caching layer** that sits between the application and the database: the application talks only to the cache abstraction, and the layer talks to the database. The names describe who does what on a read or on a write.

An important honesty note before the details. Redis itself implements **none** of these patterns. Redis does not know your database and cannot call it. If you want read-through or write-behind, either a library does it for you (Spring Cache, Redisson `RMap` with `MapLoader/MapWriter`, Caffeine `LoadingCache`), or a separate component does (a worker reading a Redis stream and writing to SQL), or you write it yourself. Redis Enterprise and some commercial products offer built-in data integration features; this book covers open source Redis and treats those as outside its scope.

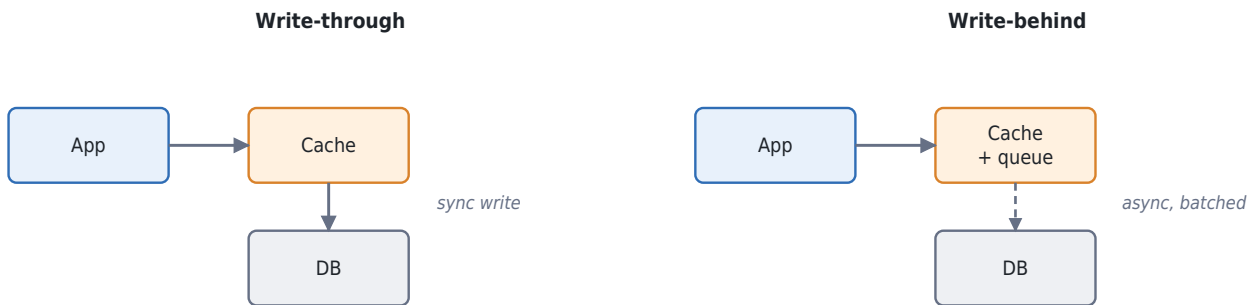


Figure 8.2. Write-through writes the cache and the database in one synchronous step; write-behind acknowledges after the cache write and flushes to the database later.

8.3.1 Read-through

In **read-through**, the application asks the cache for a key. If the entry is missing, the *cache layer* (not the application) loads it from the database, stores it and returns it. From the caller's perspective the cache is the only data source.

The behaviour is the same as cache-aside on the wire, but the loading code lives in one place and is registered once. In Java the best-known form is Caffeine's loading cache, which also gives you **single-flight** for free: if ten threads ask for the same missing key, only one runs the loader and the others wait for its result.

ReadThroughExample.java

JAVA

```
package com.shopstream.catalog;

import java.time.Duration;

import com.github.benmanes.caffeine.cache.Caffeine;
import com.github.benmanes.caffeine.cache.LoadingCache;

public class ReadThroughExample {

    private final LoadingCache<Long, ProductView> cache;

    public ReadThroughExample(ProductRepository repository) {
        this.cache = Caffeine.newBuilder()
            .maximumSize(10_000)
            .expireAfterWrite(Duration.ofMinutes(5))
            .build(id -> repository.findViewById(id).orElse(null)); // the loader
    }

    public ProductView get(long id) {
        return cache.get(id); // loads through the loader on a miss
    }
}
```

This example is an in-process cache, not Redis, but it shows the shape. With Redis you obtain the same shape from Spring's `@Cacheable` (the proxy is the "layer" and your method body is the loader), or from Redisson's `RMapCache` with a `MapLoader`. Read-through suits read-mostly data where the loader is the same for every key.

8.3.2 Write-through

In **write-through**, every write goes to the cache layer first, and the layer writes to both the cache and the database **synchronously** before it acknowledges the caller.

- **Pro:** the cache is always current for written keys; reads right after a write are hits with fresh data; no "invalidate then miss" window.
- **Con:** write latency is the sum of both writes; every write is cached even if nobody reads that key again (wasted memory, which is why it is usually combined with a TTL); you must make the two writes consistent. If the database write succeeds and the Redis write fails (or the reverse) the two disagree, and there is no distributed transaction between Redis and your database.

The application-level version looks like this, and it shows the problem plainly:

```
WriteThroughSketch.java JAVA

package com.shopstream.catalog;

import java.time.Duration;

import org.springframework.data.redis.core.StringRedisTemplate;
import tools.jackson.databind.json.JsonMapper;

public class WriteThroughSketch {

    private final StringRedisTemplate redis;
    private final JsonMapper json;
    private final ProductRepository repository;

    public WriteThroughSketch(StringRedisTemplate redis, JsonMapper json,
                             ProductRepository repository) {
        this.redis = redis;
        this.json = json;
        this.repository = repository;
    }

    public void save(ProductView view) {
        repository.save(view); // 1. database (may throw)
        redis.opsForValue().set("shopstream:vl:product:" + view.id(),
                               json.writeValueAsString(view), Duration.ofHours(1)); // 2. cache
        // If step 2 fails, the database is right and the cache is stale or absent.
    }
}
```

Because the database write goes first, a failure of the cache write leaves a missing or stale entry, not a lost update. That is the same safety posture as cache-aside, with one difference: you overwrite instead of delete, so two concurrent **save** calls for the same product can finish their Redis writes in the opposite order from their database commits, leaving the older value in the cache. For hot rows that are updated concurrently, **invalidation is safer than write-through**.

8.3.3 Write-behind (write-back)

In **write-behind**, the layer writes to the cache and acknowledges at once. The database write happens **later**, **asynchronously**, usually batched. The caller sees Redis-level latency, and the database sees fewer, larger writes.

The price is **durability**. Until the flush, the only copy of the newest data is in Redis memory. If Redis loses it (crash without persistence, failover to a replica that had not received it yet, eviction) the write is gone, although the caller was told "done". Use it only where you can accept that risk, or where you build a durable queue in front of the database.

Typical good fits: counters of page views, "last seen" timestamps, game scores that are rewritten constantly, telemetry. Typical bad fits: orders and payments.

A safer way to build write-behind with Redis is to make the **buffer durable and replayable** by using a stream:

```
request -> app: HSET counters + XADD stream:writes -> reply (fast)
      |
      v
    flusher (consumer group) reads batches
      |
      v
    batch UPSERT into the database,
    XACK only after the commit succeeds
```

With a consumer group (see the Streams chapters in the data structures Part), an entry that was read but not acknowledged is redelivered after a crash, so the flush survives worker failures. You still depend on Redis persistence and replication for the stream itself, and the database write must be idempotent because delivery is at-least-once. The idempotency chapter later in this Part covers that.

WATCH OUT

Write-behind turns Redis into your database for a while. Treat the unflushed window like any other data-loss window: enable AOF with `appendfsync everysec` at least, size and monitor the backlog (stream length, age of the oldest unflushed item), and alert when the flusher stops. A silent flusher failure grows memory until eviction or an out-of-memory error destroys the buffer.

8.3.4 Refresh-ahead

Refresh-ahead (refresh-behind) reloads a hot entry **before** it expires, so users never see the miss. A background task notices that an entry is close to its expiry, loads the fresh value and replaces it while the old value is still being served.

You can implement it with cheap tools:

- store the value with a long hard TTL (say 10 minutes) and remember a *soft* deadline inside the value (say 8 minutes after loading). On read, if now is past the soft deadline, return the current value immediately and trigger one asynchronous reload, guarded by a short `SET refresh:lock NX PX 10000` so that only one instance does it;
- or use `TTL/PTTL` on the key: if the remaining TTL is below a threshold, trigger a refresh;
- or schedule a refresh job for a known list of hot keys.

This is also called **stale-while-revalidate**. Caffeine has `refreshAfterWrite`, which does exactly this for in-process caches. Refresh-ahead costs extra database load (you refresh keys that may never be asked for again) and extra code, so use it only for the few keys where an occasional slow request hurts, such as the home page block.

8.3.5 Choosing a pattern

Pattern	Who loads / writes	Consistency	Use when
Cache-aside	Application	Eventual; stale up to TTL; races possible	Default choice, most read-heavy data
Read-through	Caching layer	Same as cache-aside	You want loading logic in one place (Spring Cache, Caffeine, Redisson)
Write-through	Layer, synchronous to both	Strong for the writing path; two writes can diverge	Data re-read right after writing; low write rate
Write-behind	Layer, async to database	Database lags; risk of loss	Very high write rates, loss tolerable or buffer durable
Refresh-ahead	Background task	Fresh for hot keys	A few hot keys where a miss is costly

Caching patterns compared.

INTERVIEW ANGLE

What is the difference between write-through and write-behind? Both put writes through the cache. Write-through updates cache and database synchronously before acknowledging, so it is simpler to reason about but slower. Write-behind acknowledges after the cache write and persists to the database later in batches: faster and gentler on the database, but any data not yet flushed is lost if the cache node fails. Redis does not implement either by itself; a library or your own worker does.

KEY POINTS

Read-through, write-through, write-behind, refresh-ahead

- These patterns move the caching logic out of the call sites into a layer; Redis does not implement them on its own.
- Read-through equals cache-aside with the loader registered once; in-process caches add single-flight.
- Write-through is synchronous and consistent for the writer, but there is no transaction across Redis and the database; invalidation is safer for hot, concurrently updated rows.
- Write-behind trades durability for speed; make the buffer durable (stream plus persistence) and the flush idempotent.
- Refresh-ahead (stale-while-revalidate) hides misses for a handful of hot keys.

8.4 Cache failure modes

A cache works until the day it does not, and the failure is nearly always the same story: something makes many requests miss at once, and the database, which relied on the cache to absorb them, falls over. The database slowdown makes the requests slower, so more of them pile up, the miss fills never complete, and the system collapses. The five classic names are **stampede**, **penetration**, **avalanche**, **hot key** and **big key**. They are different problems with different cures, and interviewers like to see that you can tell them apart.

Name	What happens	Main cures
Stampede (dogpile, breakdown)	One hot key expires; many concurrent requests miss and all hit the database	Single-flight lock, early/probabilistic refresh, stale-while-revalidate
Penetration	Requests for keys that do not exist miss every time, so the database is hit every time	Negative caching, Bloom filter, input validation
Avalanche	Many keys expire together, or Redis goes down; the database receives a mass of misses	TTL jitter, warming, replication/HA, rate limiting, circuit breakers
Hot key	One key receives so much traffic that a single Redis node (one thread) becomes the bottleneck	Local in-process copy, key replication with suffixes, client-side caching
Big key	One key holds a huge value; operations on it block the server and the network	Split into smaller keys, compress, paginate, use UNLINK

The five cache failure modes at a glance.

8.4.1 Cache stampede (dogpile)

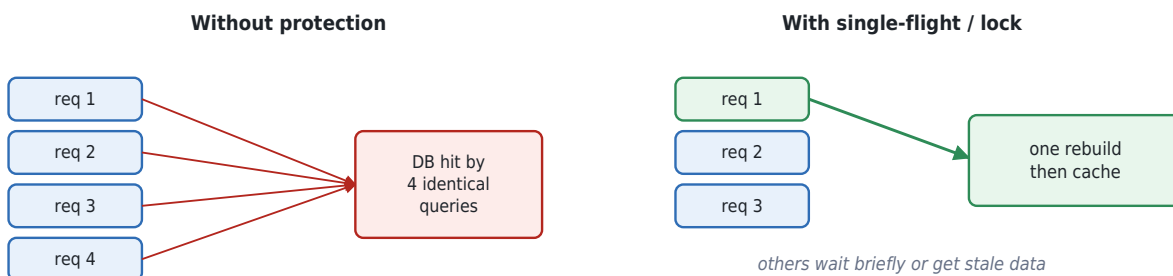


Figure 8.3. A hot key expires; without protection every waiting request queries the database, with a lock only one request rebuilds the value while the others wait or use the stale copy.

Picture the product page of a popular item, read 5,000 times a second. Its cache entry has a 10 minute TTL and the database query takes 200 ms. At the instant the key expires, the next request misses and starts the 200 ms query. During those 200 ms another 1,000 requests arrive, they also miss, and each starts the same query. The database suddenly runs 1,000 identical heavy queries in parallel, gets slower, which lengthens the window, and which brings in even more requests. A single expiry has become an outage.

All the cures reduce the number of concurrent rebuilds of one key to one (or close to it).

Cure 1: single-flight with a lock

On a miss, take a short lock with `SET lock:product:1042 token NX PX 5000`. The winner loads from the database and fills the cache. The losers either wait a few milliseconds and re-read the cache, or return a stale value.

```
StampedeGuard.java
package com.shopstream.catalog;

import java.time.Duration;
import java.util.Optional;
import java.util.UUID;
import java.util.function.Supplier;

import org.springframework.data.redis.core.StringRedisTemplate;

public class StampedeGuard {

    private final StringRedisTemplate redis;
```

JAVA

StampedeGuard.java (continued)

JAVA

```

public StampedeGuard(StringRedisTemplate redis) {
    this.redis = redis;
}

/** Returns the cached string, or rebuilds it once across all instances. */
public String getOrLoad(String key, Duration ttl, Supplier<String> loader)
    throws InterruptedException {
    for (int attempt = 0; attempt < 20; attempt++) {
        String cached = redis.opsForValue().get(key);
        if (cached != null) {
            return cached;
        }
        String lockKey = "lock:" + key;
        String token = UUID.randomUUID().toString();
        Boolean won = redis.opsForValue()
            .setIfAbsent(lockKey, token, Duration.ofSeconds(5));
        if (Boolean.TRUE.equals(won)) {
            try {
                String again = redis.opsForValue().get(key); // double-check after winning
                if (again != null) {
                    return again; // someone filled it meanwhile
                }
                String value = loader.get(); // only the winner hits the DB
                redis.opsForValue().set(key, value, ttl);
                return value;
            } finally {
                // release only our own lock (compare-and-delete, see the locks chapter)
                if (token.equals(redis.opsForValue().get(lockKey))) {
                    redis.delete(lockKey);
                }
            }
        }
        Thread.sleep(25 + (long) (Math.random() * 25)); // lost: wait, then re-read
    }
    return loader.get(); // lock holder is slow or dead: fall back to loading directly
}

```

What to notice

- After winning the lock the code reads the cache **again**. A caller may have seen a miss, been slow, and taken the lock just after another winner filled the cache and released it; the second read stops that caller from rebuilding a value that already exists.
- The lock has its own TTL (5 seconds). If the winner crashes, the lock disappears by itself and another request takes over.
- Losers wait with a small random sleep and re-read the cache instead of querying the database. After a bounded number of attempts they fall back to loading directly, so a stuck lock cannot block users forever.
- The **get-then-delete** release in **finally** is not atomic. It is acceptable here because the lock only protects an optimisation, but the locks chapter shows the correct **Lua** (or **DELEX**) release for cases where it matters.
- **Thread.sleep** blocks a platform thread; with virtual threads on Java 21 (Spring Boot 4 can enable them) that is cheap.

Spring's `@Cacheable(sync = true)` gives the same protection **inside one JVM** (a per-key lock in the cache abstraction). With ten application instances you still get up to ten concurrent rebuilds, which is usually fine. Use a distributed lock like the above when the rebuild is truly expensive.

Cure 2: probabilistic early expiration

Instead of waiting for the key to expire, each reader flips a weighted coin and sometimes refreshes the value *early*. The nearer the expiry, the likelier the refresh. This is the "XFetch" algorithm from a 2015 paper on optimal probabilistic cache stampede prevention. The decision rule is:

```

refresh now if:  now - delta * beta * ln(random()) >= expiry
delta = how long the recompute took (measured when you stored the value)
beta  = tuning factor, 1.0 is a good default
random() in (0, 1]; ln(random()) is negative, so the term adds a random head start

```

Store **delta** (the recompute time) and the expiry timestamp alongside the value, for instance as fields of a JSON wrapper. Slow rebuilds start earlier, fast ones later, and you need no lock at all, because the chance that many readers pick the same instant is tiny. The drawback is that it spends extra recomputations, and the logic is more intricate than a lock.

Cure 3: serve stale, refresh in the background

Store the value with a **soft TTL** inside the payload and a **longer hard TTL** on the key (for example 5 minutes soft, 30 minutes hard). Readers always get a value; when the soft deadline has passed, one reader (chosen by a short **SET NX**) starts an asynchronous refresh and everybody continues to be served the old value. This is the stale-while-revalidate behaviour from the previous chapter, and for read-heavy hot keys it is the most user-friendly option because nobody waits. It trades a bounded amount of staleness for latency.

Cure 4: never expire the hot keys

For a small, known set of keys (home page, top 100 products) you can store them *without* a TTL and refresh them from a scheduled job or on change events. This eliminates the expiry moment entirely but makes a refresh failure a staleness problem, so add monitoring on the age of the data.

8.4.2 Cache penetration

Penetration means the cache cannot help because the requested key **does not exist** in the database either. A request for **product:999999999** misses in Redis, misses in the database, and there is nothing to cache, so the next request repeats the same expensive trip. This happens with innocent bugs (a bad link), crawlers, and deliberately with an attack: a script requests random ids, and every request goes to the database.

Cure 1: negative caching

Cache the *absence*. When the database returns nothing, store a small sentinel for a short time:

NegativeCaching.java

JAVA

```
package com.shopstream.catalog;

import java.time.Duration;
import java.util.Optional;

import org.springframework.data.redis.core.StringRedisTemplate;
import tools.jackson.databind.json.JsonMapper;

public class NegativeCaching {

    private static final String NOT_FOUND = "__NULL__";

    private final StringRedisTemplate redis;
    private final JsonMapper json;
    private final ProductRepository repository;

    public NegativeCaching(StringRedisTemplate redis, JsonMapper json,
        ProductRepository repository) {
        this.redis = redis;
        this.json = json;
        this.repository = repository;
    }

    public Optional<ProductView> find(long id) {
        String key = "shopstream:vl:product:" + id;
        String cached = redis.opsForValue().get(key);
        if (NOT_FOUND.equals(cached)) {
            return Optional.empty(); // known to be missing
        }
        if (cached != null) {
            return Optional.of(json.readValue(cached, ProductView.class));
        }
        Optional<ProductView> loaded = repository.findViewById(id);
        if (loaded.isPresent()) {
            redis.opsForValue().set(key, json.writeValueAsString(loaded.get()),
                Duration.ofMinutes(10));
        } else {
            redis.opsForValue().set(key, NOT_FOUND, Duration.ofSeconds(45)); // short TTL
        }
    }
}
```

NegativeCaching.java (continued)

JAVA

```

    }
    return loaded;
}
}

```

Keep the negative TTL **short** (30 to 60 seconds): the product may be created a moment later, and a long negative entry would hide it. When you create the entity, delete the negative entry. The drawback of negative caching is memory: an attacker who requests a million different random ids creates a million small keys. Combine it with input validation and rate limiting, and consider the next cure.

Cure 2: Bloom filter

A **Bloom filter** is a compact probabilistic set. It answers "is this id possibly in the set?" with either **definitely not** or **probably yes**. It never gives a false negative for items added to it, and rarely gives a false positive. Redis 8 includes Bloom filters as a built-in data type (**BF.RESERVE**, **BF.ADD**, **BF.EXISTS**), no module needed. Fill the filter with all valid product ids at startup and on every create; check it before touching the cache or database.

A Bloom filter guarding product ids

TERMINAL

```

127.0.0.1:6379> BF.RESERVE bf:product:ids 0.001 1000000
OK
127.0.0.1:6379> BF.ADD bf:product:ids 1042
(integer) 1
127.0.0.1:6379> BF.EXISTS bf:product:ids 1042
(integer) 1
127.0.0.1:6379> BF.EXISTS bf:product:ids 999999999
(integer) 0

```

BF.RESERVE key error_rate capacity creates a filter sized for a given false positive rate ($0.001 = 0.1\%$) and expected number of items. If the answer is **0**, return "not found" immediately without any lookup. If it is **1**, proceed to the cache and the database; the 0.1% of false positives still cost one lookup, which is harmless. The costs of a Bloom filter are that deletions are not supported in the standard variant (use a Cuckoo filter if you need deletes), and that it must be filled completely and kept in step with the database. The details of probabilistic structures are in the data structures Part.

Cure 3: validate before you look up

Many penetration requests are malformed: negative ids, ids outside the range your sequence generator has ever produced, strings where numbers belong. Reject them at the controller. Combined with rate limiting (see the rate limiting chapter later in this Part) this is cheap and removes the easy attacks.

8.4.3 Cache avalanche

An **avalanche** is a stampede of many keys at once. Two typical causes:

1. **Mass expiry.** You warm the cache with 200,000 products, all with **EX 3600**. An hour later, all of them expire in the same second and the database receives the entire catalogue load.
2. **Cache outage.** Redis restarts without persistence, or its node dies and a replica takes over late, or the network drops. The cache is empty or unreachable and every request goes to the database.

Defences:

- **TTL jitter:** add a random component to every TTL ($3600 + \text{random}(0, 600)$ seconds). Cheap and always worth doing, as the **ProductCache** example does.
- **Staggered warm-up:** warm in batches with pauses and different TTLs.
- **High availability for Redis:** replication with Sentinel or Cluster, so that a single node failure does not empty the cache (see the scaling Part). Persistence, so a restart loads data from disk.
- **Protect the database:** rate limits or a concurrency limit (bulkhead) in front of the database, plus a circuit breaker that returns a degraded response (stale data, a placeholder) when the database is overloaded. It is better to serve 80% of users well than 100% of users with timeouts.
- **Multi-level caching:** a small in-process cache with a short TTL absorbs the first shock when Redis is cold; see the multi-level caching chapter.

8.4.4 Hot keys

A **hot key** is a single key that receives a disproportionate share of traffic: the product of a viral advertisement, the feature-flag document read on every request, the leaderboard top 10. A Redis instance executes commands on one thread; a cluster hashes a key to exactly one shard. So one key can saturate one node's CPU or network card while other nodes sit idle, and adding shards does not help.

Detect hot keys with `redis-cli --hotkeys` (requires an LFU `maxmemory-policy`), with `MONITOR` briefly on a test system (never casually on a busy production server), or with the `HOTKEYS` command in Redis 8.6, which is designed for exactly this (check its page in the command reference of your version for its options). Your application metrics, which count lookups per key prefix, are often the most practical tool.

Cures:

- **Local copy:** keep the hot value in the application's memory for a few seconds (Caffeine) so most reads never reach Redis. This is the best single fix for read-hot keys. The next chapters on multi-level caching and client-side caching build on it.
- **Replicate the key under several names:** store the value as `home:featured:0` to `home:featured:7` and have each client read a random suffix. In Cluster mode the eight keys land on different slots and therefore possibly different shards. You must update all copies on change.
- **Read from replicas** for read-hot keys, accepting replication lag.
- **Split a hot counter:** for write-hot counters, shard the counter (`views:1042:0..7`) and sum on read.

8.4.5 Big keys

A **big key** is one that holds a very large value: a string of tens of megabytes, a hash with millions of fields, a list or sorted set with millions of members. Because Redis executes commands one at a time on the main thread, an operation that touches the whole key (`HGETALL`, `SMEMBERS`, `LRANGE 0 -1`, `DEL`) blocks every other client for the duration, and ships a huge reply across the network. Replication, persistence and key migration between cluster nodes also suffer. A deletion of a 10 million element collection with `DEL` can stall the server for a noticeable time.

Cures:

- **Prevent:** cap the size of collections that grow with user activity (trim lists with `LTRIM`, bound sorted sets, expire old fields), and never cache a "whole table" as one value.
- **Split:** shard a big hash into buckets by `hash(field) mod N` (`cart-index:0` to `cart-index:63`), page a big list into chunks.
- **Compress** large text values (gzip or LZ4) before storing, if CPU is cheaper than memory and bandwidth for you.
- **Delete without blocking:** use `UNLINK` instead of `DEL` (it frees memory in a background thread), and iterate with `SSCAN`, `HSCAN`, `ZSCAN` in batches.
- **Find them:** `redis-cli --bigkeys` scans and reports the biggest key per type; `--memkeys` reports by memory use; `MEMORY USAGE key SAMPLES 0` gives the size of one key.

8.4.6 Lab: prove the stampede guard works

Do not trust a stampede guard you have not tried to break. This test starts 200 concurrent readers against an empty cache and counts how many times the (slow) loader runs. Without protection the count is close to 200; with the `StampedeGuard` from earlier it should be 1 on a single Redis. It uses virtual threads from Java 21, which make hundreds of blocked callers cheap.

StampedeGuardTest.java

JAVA

```
package com.shopstream.catalog;

import static org.junit.jupiter.api.Assertions.assertEquals;

import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.atomic.AtomicInteger;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
```


StampedeGuardTest.java (continued)

JAVA

```

import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.data.redis.core.StringRedisTemplate;

@SpringBootTest
class StampedeGuardTest {

    @Autowired
    StringRedisTemplate redis;

    @Test
    void onlyOneCallerRebuildsTheValue() throws Exception {
        redis.delete("demo:hot");
        StampedeGuard guard = new StampedeGuard(redis);
        AtomicInteger loaderCalls = new AtomicInteger();

        try (ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor()) {
            List<Future<String>> results = new ArrayList<>();
            for (int i = 0; i < 200; i++) {
                results.add(pool.submit(() -> guard.getOrLoad("demo:hot",
                    Duration.ofSeconds(30), () -> {
                        loaderCalls.incrementAndGet();
                        sleep(300); // pretend the database query is slow
                        return "value";
                    }
                )));
            }
            for (Future<String> f : results) {
                assertEquals("value", f.get());
            }
        }
        assertEquals(1, loaderCalls.get());
    }

    private static void sleep(long millis) {
        try {
            Thread.sleep(millis);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

```

If the assertion fails with 2 or 3 loader calls, check the double-check inside the lock: without it, a caller that saw a miss just before the winner stored the value can take the lock right after the winner released it and rebuild the value again. Even with it, "one rebuild per expiry instead of hundreds" is the goal rather than a hard guarantee, because a lock that expires during a slow load lets a second caller in.

8.4.7 Signals that tell you the cache is in trouble

Each failure mode leaves a fingerprint in metrics. Put these on a dashboard and alert on them before users complain.

Symptom	Likely cause	First check
Database query rate spikes at regular intervals, matching a TTL	Stampede or avalanche on a popular key set	Compare spike period with TTLs; look at jitter and rebuild locking
Miss rate high, but the same missing ids repeat in the logs	Penetration	Top missing keys; add negative caching or a Bloom filter
evicted_keys rising steadily in INFO stats	Cache too small for the working set, or no TTL on junk keys	used_memory , maxmemory , key TTL coverage
One Redis node at high CPU while the others idle	Hot key in a cluster	redis-cli --hotkeys (LFU policy) or the key-level metrics of your app
Latency spikes in Redis and slowlog shows HGETALL , SMEMBERS or DEL	Big key	redis-cli --bigkeys , MEMORY USAGE
Hit ratio drops after each deployment	Cache key schema version changed, cold cache	Warm-up job; roll out version changes gradually

Symptoms and the failure mode they suggest.

WATCH OUT

Do not rely on a single mitigation. A real incident combines modes: an avalanche (mass expiry) turns the hottest keys into stampedes, and the database overload makes every rebuild slower. Layer the defences: jittered TTLs, single-flight on rebuilds, negative caching, rate limits in front of the database, and monitoring that shows the miss rate before the database pages you.

INTERVIEW ANGLE

Distinguish cache penetration, breakdown (stampede) and avalanche. Penetration: the key does not exist anywhere, so every request goes through to the database; cure with negative caching and a Bloom filter. Breakdown or stampede: one hot key expires and many concurrent requests rebuild it; cure with a lock or single-flight, or early refresh. Avalanche: many keys expire at once or the cache is down; cure with TTL jitter, high availability, and protecting the database with rate limits and circuit breakers.

KEY POINTS

Cache failure modes

- **Stampede:** one hot key, many concurrent rebuilds. Use a TTL-bound lock (single-flight), probabilistic early refresh or serve-stale-and-refresh.
- **Penetration:** nonexistent keys. Use short negative caching, a Bloom filter (built in since Redis 8) and input validation.
- **Avalanche:** many keys expire together, or Redis disappears. Jitter TTLs, keep Redis highly available, protect the database.
- **Hot key:** one node is the bottleneck. Add a local copy, key replicas with suffixes, or split the counter.
- **Big key:** operations block the single command thread. Split, bound and `UNLINK`; find them with `--bigkeys`.

8.5 Keeping cache and database consistent

Two stores that hold the same fact will, sooner or later, disagree. There is no setting in Redis that prevents it, because Redis and your database do not share a transaction. What you can do is choose a strategy whose **disagreement window is short, bounded and acceptable** for the data in question, and be honest in the design document about what that window is. This chapter lists the strategies from the simplest to the most elaborate, with their limits.

8.5.1 What "consistent" can realistically mean

For a cache beside a database, three levels are realistic:

- **Eventually consistent with a bound:** the cache may be stale, but never for longer than the TTL (or the invalidation latency). This is what 95% of caches provide, and it is usually enough.
- **Read-your-writes:** the user who just made a change sees it, even if others see the old value for a while. Achieved by invalidating in the write path and, if needed, bypassing the cache for that user for a few seconds.
- **Strongly consistent:** a read never returns a value older than the last committed write. A cache beside a database cannot give you this without making reads wait on the database or on a lock. If you need it, read from the database, or make Redis the primary store for that data.

Everything below aims at the first two.

8.5.2 Strategy 1: TTL only

The simplest strategy: never invalidate; let entries expire. Staleness is bounded by the TTL, there is no invalidation code to get wrong, and it works across services that do not know about each other. It suits data where "a few minutes old" is fine: category pages, aggregated statistics, search suggestions. Its weakness is obvious: the user who edits a product description waits up to the TTL to see it.

8.5.3 Strategy 2: update the database, then delete the key

This is cache-aside's write path from earlier in this Part, and it is the best default for entity caches. Combine it with a TTL (strategy 1) as a safety net. It leaves one residual race: the reader that loaded the old row before the write and fills the cache after your delete.

8.5.4 Strategy 3: delayed double delete

To shrink that residual race, delete the key **twice**: once right after the database commit, and again after a short delay, long enough for any in-flight reader to have finished its fill.

DoubleDelete.java

JAVA

```
package com.shopstream.catalog;

import java.time.Duration;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;

import org.springframework.data.redis.core.StringRedisTemplate;

public class DoubleDelete {

    private final StringRedisTemplate redis;
    private final ScheduledExecutorService scheduler =
        Executors.newSingleThreadScheduledExecutor();

    public DoubleDelete(StringRedisTemplate redis) {
        this.redis = redis;
    }

    /** Call this AFTER the database transaction has committed. */
    public void invalidate(String key) {
        redis.delete(key); // first delete
        scheduler.schedule(() -> redis.delete(key), 500, TimeUnit.MILLISECONDS); // second delete
    }
}
```

The delay should exceed the longest plausible "read the database, write the cache" time of a reader (a few hundred milliseconds in most systems). Weaknesses: the delay is a guess, a process crash can lose the scheduled second delete (use a durable delay queue, such as the sorted-set based one from the delayed-task chapter, if that matters), and it adds load. It reduces the probability of a stale entry; it does not make it impossible. Describe it that way.

8.5.5 Strategy 4: versioned values (reject stale writes)

The cleanest way to defeat out-of-order fills is to give every value a **version** (a row version column, an update timestamp, or a monotonically increasing number) and make the cache accept a write only if its version is newer. Because "compare version and set" must be atomic, do it in a small Lua script:

cache_put_if_newer.lua

LUA

```
-- KEYS[1] = cache key, ARGV[1] = new version, ARGV[2] = payload, ARGV[3] = ttl seconds
local current = redis.call('HGET', KEYS[1], 'v')
if current and tonumber(current) >= tonumber(ARGV[1]) then
    return 0 -- cache already holds an equal or newer version
end
redis.call('HSET', KEYS[1], 'v', ARGV[1], 'data', ARGV[2])
redis.call('EXPIRE', KEYS[1], tonumber(ARGV[3]))
return 1
```

Now Race 1 from the cache-aside chapter cannot overwrite a newer value with an older one, as long as the reader's value carries the version it read from the database. There is one catch: if you *delete* the key on writes, the version information disappears with it, and a stale fill with an old version is accepted into an empty key. To close that gap, either write the new value (with its version) in the write path instead of deleting, or keep a small tombstone **v** for a short time. Versioning is the right tool for hot entities with frequent concurrent updates, at the price of more code and a hash layout instead of a plain string.

8.5.6 Strategy 5: change data capture (CDC) invalidation

All the strategies above require that **every writer** of the data remembers to invalidate. In a system with several services, batch jobs, admin scripts and manual SQL fixes, that is impossible to guarantee. **Change data capture** moves the responsibility to the database: a tool reads the database's transaction log (the PostgreSQL WAL, the MySQL binlog) and publishes one event per committed row change. A small consumer receives each event and deletes or refreshes the matching cache key.

```
any writer --> database --(transaction log)--> CDC connector --> event stream
                                                    |
                                                    cache-invalidator service
                                                    |
                                                    DEL product:1042
```

Advantages: it catches every change regardless of its origin, invalidation happens only for committed data (so no "delete before commit" problem), and the application code stays free of cache bookkeeping. The event stream can be Kafka, or a Redis stream if you already have Redis and the volume is moderate. Debezium is a widely used open source connector for this role.

Disadvantages: more moving parts to run and monitor; invalidation lags behind the commit by the pipeline latency (typically tens to hundreds of milliseconds, sometimes more under load), so the reader that just wrote may briefly see the old value (combine with read-your-writes); and the consumer must be idempotent and handle out-of-order or repeated events. Deleting a key is naturally idempotent, which is one more reason to prefer deletion over updating the cached value.

8.5.7 Strategy 6: publish invalidations to other instances

If every application instance also holds a local in-process cache (the next chapter), deleting the Redis key is not enough: the other instances still hold their own copy. Broadcast the invalidation with Redis pub/sub (or client-side caching tracking) and have every instance evict its local entry. Pub/sub is fire-and-forget, so a short local TTL remains necessary for the instance that missed a message while reconnecting.

8.5.8 A worked example: one price change, three strategies

Make the trade-offs concrete with a single event. The price of product 1042 changes from 89.00 to 79.00 at time $t=0$ in the database. A reader, R, requests the product at about the same time. The cache is filled with the old price (via a miss at $t = -5$ ms, when R began its read).

Strategy	Timeline	Result
Delete only	R reads old row at -5 ms; commit at 0; delete at +1 ms; R writes old price at +3 ms	Stale 89.00 until the TTL expires
Delayed double delete (500 ms)	Same as above, plus a second delete at +501 ms	Stale only for about 500 ms
Versioned put	R's value carries version 7; the writer stored version 8 (or the key holds none and a tombstone remembers 8)	R's put is rejected; cache stays correct or empty
CDC invalidation	Connector emits the change at about +50 ms; consumer deletes the key at +60 ms	Stale until about +60 ms; every writer is covered

What each strategy leaves behind after the same unlucky interleaving.

None of the rows is "perfect", and each costs a different amount of complexity. Choosing is an engineering decision: for a product description, delete plus TTL is plenty; for a price shown on the product page, delete after commit plus a short TTL; for anything that drives money, read the database at decision time.

8.5.9 Putting it together: a practical recipe

For most Spring services, the following layered recipe gives a good result without heroic engineering:

1. Cache-aside, JSON values in immutable records, keys with a schema version.
2. Always a TTL with jitter, per data class.
3. Write path: update the database in a transaction, and delete the cache key **after the commit** (`@TransactionalEventListener(phase = AFTER_COMMIT)` or `transactionAware()`).

- For hot, concurrently updated entities add a version check (strategy 4) or the delayed second delete.
- If many writers exist, invalidate through CDC rather than trying to teach every writer.
- Decision paths (checkout, payment, stock reservation) read the source of truth.
- Monitor staleness: for a sample of keys, compare the cache with the database in a background job and graph the mismatch rate. A number you measure beats an opinion.

WATCH OUT

There is no free lunch. Any design that writes to two systems without a shared transaction can fail between the two writes. The question is never "how do I make it impossible?" but "how often can it happen, how long does it last, and what does it cost the business?". Where a mismatch costs money (prices at checkout, stock, balances) do not rely on a cache for the decision.

INTERVIEW ANGLE

Why delete the cache instead of updating it after a database write? Deleting is idempotent and cheap, and the next read rebuilds the value from the database, so the cache cannot be left holding a value that was computed from stale input. Updating needs the new representation at write time, and when two writers interleave their database commits and cache writes in opposite orders, the older value can win. Delete (after commit), plus a TTL, plus optionally a delayed second delete or a version check, is the standard answer.

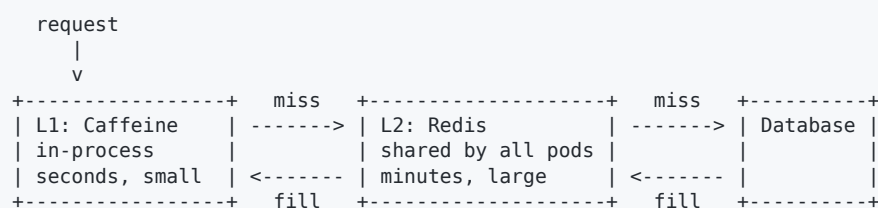
KEY POINTS

Keeping cache and database consistent

- Pick the consistency level per data class; a cache beside a database is eventually consistent by nature.
- TTL is the universal safety net; delete-after-commit is the default write path.
- Delayed double delete and versioned writes reduce, but do not eliminate, stale fills.
- Use CDC when many writers exist; it invalidates exactly the committed changes.
- Decisions that cost money read the source of truth, not the cache.

8.6 Multi-level caching

A single Redis lookup is fast, but it is still a network round trip: typically a few hundred microseconds inside one data centre, plus serialization and deserialization on your side. If the same instance asks for the same hot product 3,000 times a second, you pay that round trip 3,000 times and you push 3,000 requests a second at one Redis key. A **multi-level cache** adds a tiny cache *inside the application process* in front of Redis. Reads hit the local level first (nanoseconds, no serialization if you keep objects), then Redis, then the database.



8.6.1 What each level is for

Aspect	L1: Caffeine in the JVM	L2: Redis
Latency	Tens of nanoseconds, no network	Hundreds of microseconds and more
Capacity	Bounded by heap; keep it small (thousands of entries)	Gigabytes
Sharing	Per instance; each pod has its own copy	Shared by all instances
Staleness	Different instances may disagree briefly	One copy; consistent across instances
Survives restart	No (cold after each deploy)	Yes, if persistent

L1 (in-process) versus L2 (Redis).

The local level absorbs **hot-key** traffic and shields Redis during short incidents; the shared level gives all instances a common, warm, larger cache and shields the database. The price is a new consistency problem: each pod's L1 can hold a different version of an entry, and invalidating Redis does not touch the other pods' memory.

8.6.2 Rules for a safe L1

- **Short TTL, small size.** Five to thirty seconds for L1 is typical, with `maximumSize` bounded. This directly limits how long different pods can disagree: the L1 TTL is your extra staleness.
- **Cache immutable things** (or defensive copies): a mutable object shared by many request threads is a bug magnet. Records are ideal.
- **Broadcast invalidations.** On a write, delete the key in Redis and publish a message on a pub/sub channel (`cache:invalidate`) containing the key. Every pod subscribes and evicts that key from its L1. Pub/sub may lose a message while a pod reconnects, so the short TTL remains the backstop.
- **Do not put everything in L1.** Only the hot subset benefits, and only data that tolerates the L1 TTL of staleness.

8.6.3 Code: a two-level lookup

TwoLevelProductCache.java

JAVA

```

package com.shopstream.catalog;

import java.time.Duration;
import java.util.Optional;

import com.github.benmanes.caffeine.cache.Cache;
import com.github.benmanes.caffeine.cache.Caffeine;
import org.springframework.data.redis.core.StringRedisTemplate;
import tools.jackson.databind.json.JsonMapper;

public class TwoLevelProductCache {

    private final Cache<Long, ProductView> local = Caffeine.newBuilder()
        .maximumSize(5_000)
        .expireAfterWrite(Duration.ofSeconds(10))
        .build();

    private final StringRedisTemplate redis;
    private final JsonMapper json;
    private final ProductRepository repository;

    public TwoLevelProductCache(StringRedisTemplate redis, JsonMapper json,
        ProductRepository repository) {
        this.redis = redis;
        this.json = json;
        this.repository = repository;
    }

    public Optional<ProductView> find(long id) {
        ProductView hit = local.getIfPresent(id);           // L1
        if (hit != null) {
            return Optional.of(hit);
        }
        String key = "shopstream:v1:product:" + id;
        String cached = redis.opsForValue().get(key);       // L2
        if (cached != null) {
            ProductView view = json.readValue(cached, ProductView.class);
            local.put(id, view);
            return Optional.of(view);
        }
        Optional<ProductView> loaded = repository.findViewById(id); // source of truth
        loaded.ifPresent(view -> {
            redis.opsForValue().set(key, json.writeValueAsString(view), Duration.ofMinutes(10));
            local.put(id, view);
        });
        return loaded;
    }

    /** Called when a pub/sub invalidation message for this product arrives. */
    public void evictLocal(long id) {
        local.invalidate(id);
    }
}

```

For production use, rather than hand-rolling this, Spring Cache lets you register a composite or a second `CacheManager`, and libraries exist that package the two-level pattern; whichever you pick, check how they handle invalidation across pods, because that is the part that matters.

8.6.4 Client-side caching in Redis itself

Redis offers a protocol-level version of the same idea: **client-side caching** (the `CLIENT TRACKING` command, RESP3). The client keeps its own local copy of values it has read; the server remembers which keys each client has read and sends an **invalidation message** when one of those keys changes. The pattern removes the hand-built pub/sub broadcast and the guesswork about local TTLs, because staleness is limited by the delivery of the invalidation push rather than by a timer. The mechanics (`DEFAULT`, `BCAST`, `OPTIN` modes, memory cost on the server, the `HELLO 3` handshake) are explained in the chapter on RESP3 and client-side caching in the Redis Inside Part; the figure there shows the flow. Lettuce has support for it in its client API, which you can use to build the local level on top of Redis tracking. Check the Lettuce reference for the version you use before relying on a specific helper class, and test failure behaviour, in particular what happens to the local entries when the connection drops (a correct implementation must flush them).

When is the Redis-native approach better than a TTL-bound Caffeine layer? When you have a modest set of very hot keys, want low staleness without writing broadcast code, and your client library supports it. When is the plain Caffeine layer better? When you want the simplest thing that works everywhere, or when a few seconds of staleness is fine anyway.

8.6.5 Pitfalls

- **Memory per pod times pods.** 5,000 entries of 20 KB each is 100 MB per instance on the heap. Bound by weight if values vary in size.
- **Cold deploys.** A rolling restart empties L1 pod by pod; the L2 cache and the database see a small wave. This is another reason to keep L2.
- **Testing is harder.** Tests that write and then read through two levels can pass or fail depending on timing. Make L1 configurable (a zero-size or no-op version for tests) and test the invalidation path explicitly.
- **Double serialization costs.** L1 stores objects, L2 stores bytes. Do not deserialize on every L1 hit.
- **Unbounded key spaces.** A hot-key problem is solved by an L1; an "all keys are lukewarm" problem is not. If the working set is huge and flat, L1 mostly misses and only adds overhead.

REAL WORLD

In a typical read-heavy storefront the home page "featured products" block is requested by every visitor. A 5 second L1 entry per application instance turns thousands of Redis reads per second per pod into one read per pod every 5 seconds. Product detail pages, which have a long tail of keys, sit in Redis only. The same service uses both levels, chosen by the shape of the traffic.

KEY POINTS

Multi-level caching

- An in-process L1 (Caffeine) in front of Redis removes network cost and hot-key pressure; Redis remains the shared L2.
- L1 needs a short TTL and a size bound; its TTL is the additional staleness you accept.
- Broadcast invalidations (pub/sub) or use Redis client-side caching to evict other pods' copies; keep the TTL as backstop.
- Cache immutable objects; avoid L1 for flat, huge key spaces.
- Test the invalidation path and the connection-loss behaviour.

8.7 Distributed locks

Inside one JVM you protect shared state with `synchronized` or a `ReentrantLock`. When the same code runs on ten instances of a service, those locks protect nothing across processes. A **distributed lock** is a mutual-exclusion primitive that lives outside the application so that only one process at a time enters a critical section: "only one instance runs the nightly price import", "only one worker processes order 42", "only one request rebuilds this cache entry".

Redis is a popular lock service because it is fast and already deployed. But a lock built on Redis is easy to get subtly wrong, and the honest summary of the whole field is: **a distributed lock is a way to reduce duplicate work, and it is only a correctness guarantee if the protected resource cooperates**. This chapter builds the lock step by step, shows where it breaks, and tells you when to avoid it.

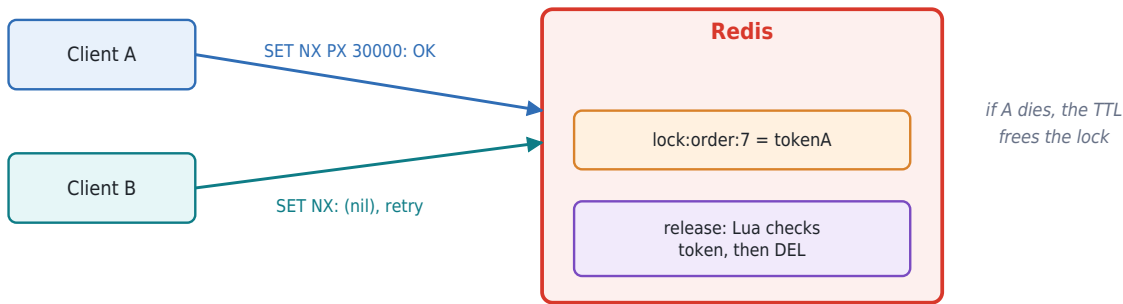


Figure 8.4. Acquire with `SET NX PX` and a unique token, do the work, release with a script that deletes the key only if the token still matches.

8.7.1 The basic lock: `SET NX PX`

The primitive is a single command:

```

Acquire a lock for 30 seconds
127.0.0.1:6379> SET lock:order:42 "worker-7:3f9c1a" NX PX 30000
OK
127.0.0.1:6379> SET lock:order:42 "worker-9:77b2e0" NX PX 30000
(nil)
127.0.0.1:6379> PTTL lock:order:42
(integer) 28431
  
```

Each part has a purpose:

- **NX** means "only set if the key does not exist". Redis executes commands one at a time, so exactly one concurrent caller gets **OK**; the others get **(nil)**. This is the mutual exclusion.
- **PX 30000** sets an expiry of 30,000 milliseconds **in the same atomic command**. If the holder crashes, the lock vanishes by itself and the system recovers. Setting the key and then calling **EXPIRE** as two commands is a classic bug: a crash between them leaves a lock that never expires.
- The **value is a random unique token** (a UUID, or worker id plus random), identifying *who* holds the lock. It is essential for safe release, as the next section shows.

8.7.2 Releasing safely: compare, then delete

The naive release is `DEL lock:order:42`. It is dangerous. Consider this sequence:

1. Worker A takes the lock with a 30 second TTL.
2. A is slow (a long database call, a garbage collection pause). The 30 seconds pass; the lock **expires**.
3. Worker B acquires the lock.
4. A finishes and calls `DEL`. It deletes **B's** lock.
5. Worker C acquires the lock while B is still working. Now two workers are inside the critical section.

The fix is to delete the key only if it still holds *your* token. "Read the value, compare, delete" must be atomic, so it runs as a Lua script:

```

unlock.lua
if redis.call('GET', KEYS[1]) == ARGV[1] then
    return redis.call('DEL', KEYS[1])
else
    return 0
end
  
```

In Redis 8.4 and later there is a native command that does the same thing:

```

Compare-and-delete with DELEX (Redis 8.4+)
127.0.0.1:6379> DELEX lock:order:42 IFEQ "worker-7:3f9c1a"
(integer) 1
127.0.0.1:6379> DELEX lock:order:42 IFEQ "worker-7:3f9c1a"
(integer) 0
  
```

`DELEX key IFEQ value` deletes the key only if the stored string equals `value` and returns `1` if it deleted, `0` otherwise. If your whole fleet runs a Redis version that has it, it replaces the script; otherwise keep the Lua version. The 8.4 `SET` command also gained `IFEQ`, which lets you extend a lock's TTL only if you still own it (`SET key token IFEQ token PX 30000`), though check the command page for your exact version before depending on it. A Lua script for extension works everywhere:

extend.lua

LUA

```
if redis.call('GET', KEYS[1]) == ARGV[1] then
    return redis.call('PEXPIRE', KEYS[1], ARGV[2])
else
    return 0
end
```

8.7.3 A lock helper in Java

RedisLock.java

JAVA

```
package com.shopstream.locks;

import java.time.Duration;
import java.util.List;
import java.util.Optional;
import java.util.UUID;

import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.core.script.DefaultRedisScript;

public class RedisLock {

    private static final DefaultRedisScript<Long> UNLOCK = new DefaultRedisScript<>("""
        if redis.call('GET', KEYS[1]) == ARGV[1] then
            return redis.call('DEL', KEYS[1])
        else
            return 0
        end
        """, Long.class);

    private final StringRedisTemplate redis;

    public RedisLock(StringRedisTemplate redis) {
        this.redis = redis;
    }

    /** Returns a token if the lock was acquired, otherwise empty. */
    public Optional<String> tryLock(String name, Duration ttl) {
        String token = UUID.randomUUID().toString();
        Boolean ok = redis.opsForValue().setIfAbsent("lock:" + name, token, ttl);
        return Boolean.TRUE.equals(ok) ? Optional.of(token) : Optional.empty();
    }

    /** Releases the lock only if we still own it. Returns true if we did. */
    public boolean unlock(String name, String token) {
        Long deleted = redis.execute(UNLOCK, List.of("lock:" + name), token);
        return deleted != null && deleted == 1L;
    }
}
```

OrderProcessor.java

JAVA

```
package com.shopstream.orders;

import java.time.Duration;
import java.util.Optional;

import com.shopstream.locks.RedisLock;
import org.springframework.stereotype.Service;

@Service
public class OrderProcessor {

    private final RedisLock lock;

    public OrderProcessor(RedisLock lock) {
```

OrderProcessor.java (continued)

JAVA

```

        this.lock = lock;
    }

    public void process(long orderId) {
        String name = "order:" + orderId;
        Optional<String> token = lock.tryLock(name, Duration.ofSeconds(30));
        if (token.isEmpty()) {
            return; // someone else is on it: skip, retry later, or fail fast
        }
        try {
            // do the work: must finish well inside the 30 seconds
        } finally {
            lock.unlock(name, token.get());
        }
    }
}

```

What to notice

- `setIfAbsent(key, value, Duration)` is SET key value NX PX ... underneath.
- The script is a constant sent by hash after the first call (EVALSHA), so the release is one cheap round trip.
- `tryLock` does not wait. A waiting variant loops with a small sleep and a deadline; never spin without a pause and without a limit.
- The work inside the lock must be much shorter than the TTL, and the code must be written knowing the lock *can* expire underneath it. Which brings us to the real problem.

8.7.4 The fundamental problem: the lock can expire while you still think you hold it

A lock with a TTL is a **lease**: the holder has the right to the resource only for a limited time. If the holder pauses longer than the lease (a long GC pause, a stalled VM, a slow network call, a blocked thread), the lock expires, someone else takes it, and the original holder wakes up *still believing it owns the lock*. No amount of care in the unlock code fixes this, because the holder does not know that time passed.

Two defences exist, and they solve different parts of the problem.

Defence 1: renewal (watchdog)

A background task extends the lease while the work is in progress: every third of the TTL, run the extend script above; stop when the work finishes. If the holder process dies, the renewals stop and the lock expires, which is what you want. Redisson's **RLock** does this automatically (below). Renewal makes expiry during normal slow work unlikely. It does **not** help against a process that is paused entirely, because a paused process also pauses its watchdog.

Defence 2: fencing tokens

A **fencing token** is a number that increases every time the lock is granted. The holder sends the token with every write to the protected resource, and the resource **rejects any request carrying a token lower than one it has already seen**. A delayed old holder wakes up, sends its old token, and the resource refuses it. This is the only approach that stays safe even when the lock expires unnoticed, but it needs the resource (the database, a storage service) to check tokens. With a relational database it is easy: keep a `last_token` column and update **WHERE** `last_token < ?`.

Redis can mint the token if acquire is atomic with the increment, so use a script:

acquire_with_fence.lua

LUA

```

-- KEYS[1] = lock key, KEYS[2] = fence counter key
-- ARGV[1] = owner token, ARGV[2] = ttl in ms
if redis.call('SET', KEYS[1], ARGV[1], 'NX', 'PX', ARGV[2]) then
    return redis.call('INCR', KEYS[2]) -- returns the fencing token (1, 2, 3, ...)
else
    return 0 -- not acquired
end

```

In a Redis Cluster both keys must be in the same slot, so give them a shared hash tag: `{order:42}:lock` and `{order:42}:fence`. Note also that after a failover to a replica that missed the last increment, the counter could go

backwards, so Redis counters are a pragmatic fencing source, not a mathematically perfect one. A database sequence is stronger when you have one.

8.7.5 Redlock and the debate

A single Redis node (or a primary with asynchronous replicas) has one more weakness: if the primary dies right after granting a lock and before replicating the key, the promoted replica does not know about the lock, and a second client can acquire it. For a lock that must tolerate that, the Redis documentation describes the **Redlock** algorithm. Roughly:

1. Take the current time.
2. Try to acquire the lock (same key, same random token, short TTL) on each of **N independent Redis primaries** (typically 5), one after the other, with a small per-node timeout.
3. The lock is acquired if it was obtained on a **majority** (at least 3 of 5) *and* the elapsed time is less than the lock TTL. The remaining validity is the TTL minus the elapsed time minus a small allowance for clock drift.
4. If acquisition failed, release the lock on all nodes.

The nodes are independent: separate machines, no replication between them.

The algorithm was publicly criticised by Martin Kleppmann, a distributed systems researcher, and defended by Redis's creator Salvatore Sanfilippo. In neutral summary: Kleppmann's argument is that Redlock's safety depends on timing assumptions (bounded process pauses, bounded network delay, reasonably accurate clocks) that real systems violate, so it cannot give correctness when a client pauses after acquiring the lock, and it provides no fencing token; for correctness he recommends a consensus-based system with fencing, and for efficiency a single Redis node is enough. The defenders answered that the algorithm explicitly accounts for elapsed time and drift, and that the pause problem affects every lease-based lock, including those built on consensus systems, unless the resource also cooperates through fencing. Both sides agree on the practical point: **if a duplicate execution would be a correctness disaster, the lock alone is not enough; the resource must defend itself** (fencing, unique constraints, idempotent operations).

A guideline that follows from the debate:

Purpose	Meaning	Reasonable tool
Efficiency	A duplicate run wastes work but does no harm (rebuild a cache entry, run a report twice)	Single-node SET NX PX with token; Redisson RLock
Correctness, soft	A duplicate causes inconsistency that you can repair	Lock plus idempotent operations and unique constraints
Correctness, hard	A duplicate causes money or data loss	Do not rely on Redis locks alone: database constraints or row locks, fencing tokens, or a consensus system

Which lock for which purpose.

8.7.6 Redisson RLock

Redisson (a Java client covered in the Java clients Part) implements locks and the watchdog for you. **RLock** is reentrant (the same thread can lock again), works across JVMs, and when you do not specify a lease time it uses a **watchdog**: the lock is held for **lockWatchdogTimeout** (30 seconds by default) and automatically extended while the owning thread is alive.

RedissonOrderProcessor.java

JAVA

```
package com.shopstream.orders;

import java.util.concurrent.TimeUnit;

import org.redisson.api.RLock;
import org.redisson.api.RedissonClient;
import org.springframework.stereotype.Service;

@Service
public class RedissonOrderProcessor {

    private final RedissonClient redisson;

    public RedissonOrderProcessor(RedissonClient redisson) {
        this.redisson = redisson;
    }
}
```

RedissonOrderProcessor.java (continued)

JAVA

```

    }

    public void process(long orderId) throws InterruptedException {
        RLock lock = redisson.getLock("lock:order:" + orderId);
        // wait up to 2 seconds to get the lock; no lease time => watchdog keeps it alive
        if (!lock.tryLock(2, TimeUnit.SECONDS)) {
            return; // could not get it in time
        }
        try {
            // critical section
        } finally {
            if (lock.isHeldByCurrentThread()) {
                lock.unlock();
            }
        }
    }
}

```

If you pass an explicit lease time (`tryLock(wait, lease, unit)`), the watchdog is **not** used and the lock expires after the lease, which is the right choice when you want a hard upper bound. Redisson also provides a `RedissonMultiLock` and a Redlock-style lock across several Redis instances in its API; check the Redisson documentation for the current status and licensing of each feature before choosing it, since some features are in the paid edition. Also note that Redisson's lock is a Redis hash with a Lua-managed owner id and count, so it is not the simple string key shown above.

8.7.7 Waiting for a lock without hammering Redis

`tryLock` returns immediately. Sometimes you want to **wait** a bounded time for the lock. The naive way is a tight loop of `SET NX`, which sends thousands of useless commands a second when the lock is busy. Use a sleep with exponential backoff and jitter, and a hard deadline:

WaitingLock.java

JAVA

```

package com.shopstream.locks;

import java.time.Duration;
import java.util.Optional;
import java.util.concurrent.ThreadLocalRandom;

public class WaitingLock {

    private final RedisLock lock;

    public WaitingLock(RedisLock lock) {
        this.lock = lock;
    }

    public Optional<String> lockWithin(String name, Duration ttl, Duration maxWait)
        throws InterruptedException {
        long deadline = System.nanoTime() + maxWait.toNanos();
        long sleepMillis = 10;
        while (System.nanoTime() < deadline) {
            Optional<String> token = lock.tryLock(name, ttl);
            if (token.isPresent()) {
                return token;
            }
            long jitter = ThreadLocalRandom.current().nextLong(sleepMillis / 2 + 1);
            Thread.sleep(sleepMillis + jitter);
            sleepMillis = Math.min(sleepMillis * 2, 200); // back off, but cap the pause
        }
        return Optional.empty();
    }
}

```

Redisson avoids polling: its lock waiters subscribe to a pub/sub channel and are woken when the holder releases the lock, which is a better design when many callers contend for one lock.

8.7.8 A renewing lease for scheduled jobs

A common real use is "run this scheduled job on exactly one instance". The job may take longer than any TTL you would like to commit to up front, so the holder renews the lease while it works. This sketch uses the extend script and a scheduled executor. If a renewal reports that the lock is no longer ours, the job must stop, because someone else may now be running it.

```
LeaseHolder.java JAVA

package com.shopstream.locks;

import java.time.Duration;
import java.util.List;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.ScheduledFuture;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicBoolean;

import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.core.script.DefaultRedisScript;

public class LeaseHolder implements AutoCloseable {

    private static final DefaultRedisScript<Long> EXTEND = new DefaultRedisScript<>("""
        if redis.call('GET', KEYS[1]) == ARGV[1] then
            return redis.call('PEXPIRE', KEYS[1], ARGV[2])
        else
            return 0
        end
        """, Long.class);

    private final ScheduledExecutorService scheduler =
        Executors.newSingleThreadScheduledExecutor();
    private final AtomicBoolean lost = new AtomicBoolean(false);
    private final ScheduledFuture<?> renewal;

    public LeaseHolder(StringRedisTemplate redis, String key, String token, Duration ttl) {
        long ttlMillis = ttl.toMillis();
        this.renewal = scheduler.scheduleAtFixedRate(() -> {
            Long ok = redis.execute(EXTEND, List.of(key), token, String.valueOf(ttlMillis));
            if (ok == null || ok == 0L) {
                lost.set(true); // we no longer own the lock
                renewal().cancel(false);
            }
        }, ttlMillis / 3, ttlMillis / 3, TimeUnit.MILLISECONDS);
    }

    private ScheduledFuture<?> renewal() {
        return renewal;
    }

    /** The job should check this between steps and stop when it turns true. */
    public boolean isLost() {
        return lost.get();
    }

    @Override
    public void close() {
        scheduler.shutdownNow();
    }
}
```

What to notice

- Renewal runs at one third of the TTL, so two consecutive renewals can fail before the lease expires.
- `isLost()` is cooperative: the job must poll it. Nothing can forcibly stop code that does not look.
- A Redis outage makes renewals fail (`ok == null`), which marks the lease as lost. That is the conservative choice: stop rather than risk a double run.
- Closing the holder stops the renewals; the lock then expires on its own or is released explicitly with the safe unlock.

8.7.9 Lab: watch a lock being taken, refused, stolen and released

Open two terminals with `redis-cli` (call them A and B). This session shows exactly why the token check matters.

Terminal A takes the lock with a short TTL

TERMINAL

```
127.0.0.1:6379> SET lock:demo "tokenA" NX PX 3000
OK
```

Terminal B tries immediately, then again after the lease expired

TERMINAL

```
127.0.0.1:6379> SET lock:demo "tokenB" NX PX 30000
(nil)
127.0.0.1:6379> SET lock:demo "tokenB" NX PX 30000
OK
```

Terminal A wakes up late and releases with a plain DEL (the bug)

TERMINAL

```
127.0.0.1:6379> DEL lock:demo
(integer) 1
127.0.0.1:6379> GET lock:demo
(nil)
```

A's late `DEL` removed B's lock; B still thinks it holds it. Now repeat the experiment with the safe release: in A use `EVAL "if redis.call('GET', KEYS[1]) == ARGV[1] then return redis.call('DEL', KEYS[1]) else return 0 end" 1 lock:demo tokenA` and observe that it returns `(integer) 0` when B owns the lock, leaving B's lock alone.

8.7.10 Failure scenarios and what they cost

Event	What happens	Mitigation
Holder crashes	Lock expires after the TTL; others proceed	Keep TTL short, renew while alive
Holder pauses longer than the TTL	Two holders at once for a while	Fencing tokens or idempotent work
Redis primary fails, replica promoted	The lock key may be missing on the replica; a second client can acquire it	Accept for efficiency locks; use constraints for correctness; consider <code>WAIT</code> as a partial mitigation
Network partition around the client	Client cannot renew; the lock expires	Stop work when renewal fails
Clock jumps on the Redis server	Expiry may fire early or late	Redis uses its own clock; keep time sync healthy, avoid long jumps
Release called twice	Second call returns 0 and does nothing (with the token check)	None needed

How a Redis lock behaves when things go wrong.

8.7.11 When not to use a lock

Locks are the heaviest tool for coordination. Before reaching for one, check whether a cheaper construction removes the need:

- **Atomic single commands.** Decrementing stock with a Lua script or `DECR` is atomic by itself. No lock needed.
- **Optimistic locking.** `WATCH/MULTI/EXEC` in Redis, or a `version` column in SQL (`UPDATE ... WHERE version = ?`), detects conflicting writers and retries instead of blocking.
- **Unique constraints and idempotency keys.** Let the database reject the duplicate. "Process payment for order 42" guarded by a unique (`order_id`, `kind`) row is safer than any lock.
- **Single consumer by design.** A stream consumer group delivers each entry to one consumer, so "only one worker handles message X" needs no lock.
- **Leader election for scheduled jobs.** A `SET NX PX` lease with renewal is fine for "only one pod runs the nightly import", but libraries like ShedLock (a Spring-oriented scheduler lock) package exactly this for you.
- **Partition the work** instead of locking it: route all events for one order to the same worker (by hash), and the worker handles them in order without cross-process locks.

GOTCHA

Reentrancy and thread identity. A simple string-token lock like `RedisLock` is not reentrant: the same thread asking twice will fail the second time. Redisson locks are reentrant but tied to a thread id; with reactive code, thread hopping or virtual threads, check how the owner is identified before trusting unlock calls from a different thread.

WATCH OUT

Never use a lock with no TTL, and never assume the TTL cannot elapse. A lock without expiry is held forever after one crash. A lock with an expiry is a lease that may end while you are still working. Design the critical section so a second concurrent run is harmless (idempotent steps, version checks, unique constraints), and use fencing tokens where it is not.

REAL WORLD

A typical use in ShopStream is the "rebuild once" lock from the stampede section and a lease for the scheduled nightly catalogue import. Neither needs more than a single-node `SET NX PX` lock, because a rare double run only wastes effort. Stock reservations and payments use atomic Lua scripts and database constraints instead of locks.

INTERVIEW ANGLE

How do you implement a distributed lock with Redis, and what can go wrong? Acquire with `SET key token NX PX ttl` (atomic, with a unique token and an expiry); release with a Lua script or `DELEX IFEQ` that deletes only if the token matches. Problems: the lock can expire while the holder is still working (GC pause, slow call), allowing two holders; a failover can lose a lock that was not yet replicated; clocks and pauses make timing assumptions unsafe. Mitigate with renewal (watchdog), fencing tokens checked by the resource, idempotent operations, and by not using a Redis lock as the only guard for money-critical work. Redlock extends the idea over several independent nodes but remains debated.

KEY POINTS

Distributed locks

- Acquire with one atomic command: `SET key token NX PX ttl`; the token is random and unique per acquisition.
- Release only if the token matches (Lua script, or `DELEX key IFEQ token` on Redis 8.4+); never plain `DEL`.
- A lock with a TTL is a lease and may expire under a paused holder; renewal lowers the odds, fencing tokens checked by the resource remove the danger.
- Redlock uses a majority of independent nodes; the Kleppmann/Sanfilippo debate shows that timing assumptions matter and the protected resource should defend itself.
- Redisson's `RLock` gives reentrancy and a watchdog; an explicit lease time disables the watchdog.
- Prefer atomic commands, optimistic concurrency, unique constraints and partitioning over locks when you can.

8.8 Rate limiting

Every public API eventually meets a client that sends too much: a buggy retry loop, a scraper, a login form under a password-guessing attack, or simply one noisy tenant who starves everyone else. A **rate limiter** decides, for each incoming request, whether the caller is still within its allowance. If not, the service answers quickly and cheaply with `429 Too Many Requests` instead of doing the expensive work.

Redis is the usual home for the counters because a rate limiter has three requirements that match Redis well. The state must be shared by every instance of your service (a counter in JVM memory only sees one instance). The check sits on the hot path of every request, so it must take well under a millisecond. And the state is disposable: counters expire by themselves, so TTLs do the cleanup.

In ShopStream, the gateway limits each client by IP address and each logged-in user by id. The keys look like `rl:{ip}:{route}`, and every algorithm below can use that naming.

8.8.1 What you must decide first

Before choosing an algorithm, answer four questions. They matter more than the algorithm.

- **Who is limited?** An IP address (cheap, but shared by offices and mobile carriers behind NAT), a user id, an API key, a tenant, or a combination such as user plus route.
- **What is the budget?** "100 requests per minute" is a rate; "10 per second with bursts up to 50" is a rate plus a burst allowance. Different algorithms express these differently.
- **What does a rejected request cost?** Reject early and cheaply, ideally at the gateway, before authentication or database access.
- **What if Redis is down? Fail open** (let traffic through, accept the risk) or **fail closed** (reject everything). A public catalogue endpoint should usually fail open; a login or payment endpoint under attack may prefer fail closed. Decide deliberately and write it down.

8.8.2 Fixed window counter

The simplest limiter: one counter per client per time window. Divide time into windows (for example, one minute), and count requests in the current window. When the count exceeds the limit, reject.

Fixed window with INCR and EXPIRE

TERMINAL

```
127.0.0.1:6379> INCR rl:203.0.113.7:/api/search
(integer) 1
127.0.0.1:6379> EXPIRE rl:203.0.113.7:/api/search 60
(integer) 1
127.0.0.1:6379> INCR rl:203.0.113.7:/api/search
(integer) 2
```

The first request creates the key (**INCR** treats a missing key as 0) and sets a 60 second TTL. Every following request in that window only increments. When the key expires, the next request starts a new window. **INCR** is atomic, so two instances can never lose an update.

There is one subtle bug in this two-command version: if the client crashes between **INCR** and **EXPIRE**, the key lives forever and the user is blocked forever. Do both in one atomic step with a short Lua script:

fixed_window.lua

LUA

```
-- KEYS[1] = counter key, ARGV[1] = window length in ms
local count = redis.call('INCR', KEYS[1])
if count == 1 then
    redis.call('PEXPIRE', KEYS[1], ARGV[1])
end
return count
```

The weakness of the fixed window is the **boundary burst**. With a limit of 100 per minute, a client can send 100 requests in the last second of minute one and 100 more in the first second of minute two: 200 requests within two seconds, while never violating the rule. Whether this matters depends on what is behind the API. For "stop abuse" it is often acceptable; for "protect a fragile backend" it is not.

8.8.3 Sliding window log

To remove the boundary burst, remember the exact time of every request and count those in the last N milliseconds. A sorted set fits: the member identifies the request, the score is its timestamp.

The algorithm has four steps: remove members with a score older than **now - window** (**ZREMRANGEBYSCORE**); count what remains (**ZCARD**); if the count is at or above the limit, reject; otherwise add the current request (**ZADD**) and refresh the key TTL. These steps must be one atomic unit, otherwise two concurrent requests can both see 99 and both be admitted. A Lua script does that:

sliding_window_log.lua

LUA

```
-- KEYS[1] = zset key
-- ARGV[1] = window in ms, ARGV[2] = limit, ARGV[3] = unique request id
local window_ms = tonumber(ARGV[1])
local limit = tonumber(ARGV[2])
local member = ARGV[3]

local t = redis.call('TIME')
local now = t[1] * 1000 + math.floor(t[2] / 1000)
```

sliding_window_log.lua (continued)

LUA

```

redis.call('ZREMRANGEBYSCORE', KEYS[1], '-inf', now - window_ms)
local count = redis.call('ZCARD', KEYS[1])
if count >= limit then
    local oldest = redis.call('ZRANGE', KEYS[1], 0, 0, 'WITHSCORES')
    local retry_ms = tonumber(oldest[2]) + window_ms - now
    return {0, 0, retry_ms}
end
redis.call('ZADD', KEYS[1], now, member)
redis.call('PEXPIRE', KEYS[1], window_ms)
return {1, limit - count - 1, 0}

```

What to notice: the script reads the clock with **TIME** inside Redis, so all application instances share one clock and clock skew between servers cannot corrupt the window. The returned array is `{allowed, remaining, retry_after_ms}`. The member must be unique per request (a UUID); if two requests used the same member, the second **ZADD** would overwrite the first and the limiter would under-count.

The log is exact but expensive: memory is proportional to the limit (a limit of 10,000 per hour stores up to 10,000 members per client). Use it for small limits where precision matters, such as "5 login attempts per 15 minutes".

8.8.4 Sliding window counter

A good compromise keeps two counters, for the current and the previous fixed window, and estimates the sliding total by weighting the previous window by how much of it still overlaps the sliding window.

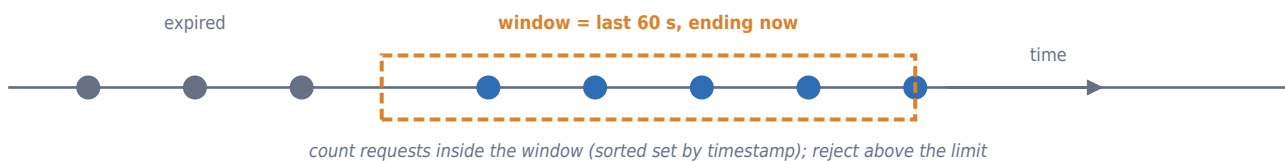


Figure 8.5. Fixed windows versus a sliding window. Notice how the sliding estimate blends the previous and current counters.

If the window is 60 seconds, the previous window had 84 requests, and we are 25 percent into the current window which has 36, the estimate is $84 \times 0.75 + 36 = 99$. It is an approximation (it assumes requests in the previous window were evenly spread), but the error is small in practice and the memory cost is two integers per client.

sliding_window_counter.lua

LUA

```

-- KEYS[1] = key prefix, ARGV[1] = window in ms, ARGV[2] = limit
local window_ms = tonumber(ARGV[1])
local limit = tonumber(ARGV[2])

local t = redis.call('TIME')
local now = t[1] * 1000 + math.floor(t[2] / 1000)
local current_window = math.floor(now / window_ms)
local elapsed_fraction = (now % window_ms) / window_ms

local current_key = KEYS[1] .. ':' .. current_window
local previous_key = KEYS[1] .. ':' .. (current_window - 1)

local current = tonumber(redis.call('GET', current_key) or '0')
local previous = tonumber(redis.call('GET', previous_key) or '0')
local estimated = previous * (1 - elapsed_fraction) + current

if estimated >= limit then
    return {0, 0}
end
redis.call('INCR', current_key)
redis.call('PEXPIRE', current_key, window_ms * 2)
return {1, math.floor(limit - estimated - 1)}

```

GOTCHA

Keys built inside a script This script computes key names (`KEYS[1] .. ':' .. current_window`) that were not declared to Redis. On a single node that works. In Redis Cluster the declared keys decide which node runs the script, and undeclared keys are not guaranteed to live there. If you need this variant in a cluster, put a hash tag in the prefix (for example `rl:{user42}`) so all derived keys share a slot, and test it. The token bucket below avoids the problem by using one key.

8.8.5 Token bucket

The token bucket is the most popular algorithm because it expresses what people actually want: a steady rate plus a burst allowance. Picture a bucket that holds up to `capacity` tokens. Tokens are added at a fixed refill rate until the bucket is full. Each request removes one token (or more for expensive calls); if the bucket is empty, the request is rejected.

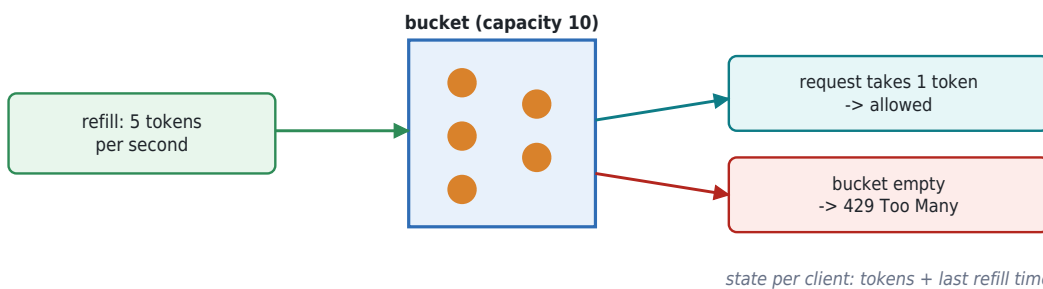


Figure 8.6. Token bucket: refill at a steady rate, spend one token per request, burst up to the bucket capacity.

You do not need a background process to add tokens. Store only two fields, the token count and the time of the last update, and compute the refill lazily when the next request arrives: `tokens = min(capacity, tokens + elapsed × rate)`.

token_bucket.lua

LUA

```
-- KEYS[1] = bucket key (a hash)
-- ARGV[1] = capacity, ARGV[2] = refill tokens per second, ARGV[3] = cost
local capacity = tonumber(ARGV[1])
local refill_per_sec = tonumber(ARGV[2])
local cost = tonumber(ARGV[3])

local t = redis.call('TIME')
local now = t[1] * 1000 + math.floor(t[2] / 1000)

local data = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(data[1])
local ts = tonumber(data[2])
if tokens == nil then
    tokens = capacity
    ts = now
end

local elapsed = math.max(0, now - ts)
tokens = math.min(capacity, tokens + elapsed * refill_per_sec / 1000)

local allowed = 0
local retry_ms = 0
if tokens >= cost then
    tokens = tokens - cost
    allowed = 1
else
    retry_ms = math.ceil((cost - tokens) * 1000 / refill_per_sec)
end

redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', KEYS[1], math.ceil(capacity / refill_per_sec * 1000) * 2)
return {allowed, math.floor(tokens), retry_ms}
```

What to notice:

- A new client starts with a full bucket, so its first burst is allowed.

- The key TTL is twice the time needed to refill an empty bucket. An idle client's key disappears on its own, and a recreated bucket starts full, which is the state the idle client would have reached anyway.
- Fractional tokens are kept in the hash, but the script returns `math.floor(tokens)`, because Redis converts Lua numbers to integers in replies.
- The reply carries `retry_ms`, which the HTTP layer turns into a `Retry-After` header.

With capacity 3 and a refill of 1 token per second, the fourth immediate call is rejected:

Trying the token bucket (typical output)

TERMINAL

```
$ redis-cli --eval token_bucket.lua rl:user:42 , 3 1 1
1) (integer) 1
2) (integer) 2
3) (integer) 0
$ redis-cli --eval token_bucket.lua rl:user:42 , 3 1 1
1) (integer) 1
2) (integer) 1
3) (integer) 0
$ redis-cli --eval token_bucket.lua rl:user:42 , 3 1 1
1) (integer) 1
2) (integer) 0
3) (integer) 0
$ redis-cli --eval token_bucket.lua rl:user:42 , 3 1 1
1) (integer) 0
2) (integer) 0
3) (integer) 986
```

In `--eval`, the comma separates key names from arguments, and spaces around it are required.

8.8.6 Leaky bucket

The **leaky bucket** is the token bucket's mirror image. Requests pour into a bucket that drains at a constant rate; if the bucket overflows, requests are rejected. Used as a *meter*, the level rises with each request and drains over time, and the same lazy-update trick works; the effect is almost identical to a token bucket. Used as a *queue*, it smooths bursts into a constant output rate by delaying requests, which is what traffic shapers do. For API protection most teams pick the token bucket and move on. Choose the queue form only when the downstream system must see a perfectly steady rate.

Algorithm	Memory per client	Strength	Weakness
Fixed window	one counter	simplest, very fast	up to 2× burst at window edges
Sliding window log	one entry per request	exact	memory grows with the limit
Sliding window counter	two counters	smooth, cheap	approximate
Token bucket	one small hash	rate plus burst, intuitive	two parameters to tune
Leaky bucket	one small hash or a queue	constant output rate	extra latency in queue form

Choosing an algorithm.

8.8.7 Calling the script from Spring

Load each script once as a `RedisScript` and call it through `StringRedisTemplate`. Spring computes the SHA-1, tries `EVALSHA`, and falls back to `EVAL` if the server has not seen the script yet.

TokenBucketRateLimiter.java

JAVA

```
package com.shopstream.gateway.ratelimit;

import java.util.List;
import org.springframework.core.io.ClassPathResource;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.core.script.DefaultRedisScript;
import org.springframework.stereotype.Component;

@Component
public class TokenBucketRateLimiter {

    public record Decision(boolean allowed, long remaining, long retryAfterMs) {}
```

TokenBucketRateLimiter.java (continued)

JAVA

```

private final StringRedisTemplate redis;
private final DefaultRedisScript<List> script;

public TokenBucketRateLimiter(StringRedisTemplate redis) {
    this.redis = redis;
    this.script = new DefaultRedisScript<>();
    this.script.setLocation(new ClassPathResource("scripts/token_bucket.lua"));
    this.script.setResultType(List.class);
}

public Decision tryAcquire(String key, int capacity, double refillPerSec, int cost) {
    List<?> reply = redis.execute(script, List.of(key),
        String.valueOf(capacity), String.valueOf(refillPerSec), String.valueOf(cost));
    if (reply == null || reply.size() < 3) {
        return new Decision(true, capacity, 0); // fail open on a malformed reply
    }
    return new Decision(
        ((Number) reply.get(0)).longValue() == 1L,
        ((Number) reply.get(1)).longValue(),
        ((Number) reply.get(2)).longValue());
}
}

```

The class returns a small record instead of throwing, so the caller decides what to do. The script is expected at `src/main/resources/scripts/token_bucket.lua`. Because `List.class` is the result type, the numbers arrive as `Long` values.

8.8.8 A servlet filter at the gateway

Put the limiter in front of everything else. A servlet filter is enough to show the idea; Spring Cloud Gateway or an API management product would apply the same logic in its own filter type.

RateLimitFilter.java

JAVA

```

package com.shopstream.gateway.ratelimit;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

@Component
public class RateLimitFilter extends OncePerRequestFilter {

    private static final int CAPACITY = 20;
    private static final double REFILL_PER_SEC = 5.0;

    private final TokenBucketRateLimiter limiter;

    public RateLimitFilter(TokenBucketRateLimiter limiter) {
        this.limiter = limiter;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
        FilterChain chain) throws ServletException, IOException {
        String key = "rl:" + request.getRemoteAddr() + ":" + request.getRequestURI();

        TokenBucketRateLimiter.Decision d;
        try {
            d = limiter.tryAcquire(key, CAPACITY, REFILL_PER_SEC, 1);
        } catch (RuntimeException redisDown) {
            chain.doFilter(request, response); // fail open
            return;
        }

        response.setHeader("RateLimit-Limit", String.valueOf(CAPACITY));
        response.setHeader("RateLimit-Remaining", String.valueOf(d.remaining()));
        if (!d.allowed()) {
            long seconds = Math.max(1, (d.retryAfterMs() + 999) / 1000);

```

RateLimitFilter.java (continued)

JAVA

```

        response.setHeader("Retry-After", String.valueOf(seconds));
        response.setStatus(429);
        response.getWriter().write("{\"error\":\"rate_limited\"}");
        return;
    }
    chain.doFilter(request, response);
}

```

What to notice: the `catch` implements the fail-open decision explicitly; the key combines client and route so a flood on `/api/search` does not block `/api/cart`; and a rejected request gets 429 plus `Retry-After` in whole seconds. Behind a load balancer, `getRemoteAddr()` is the balancer's address. Configure the container to honour `X-Forwarded-For` from trusted proxies only (Spring Boot has `server.forward-headers-strategy`); trusting that header from arbitrary clients lets attackers choose their own identity.

The 429 status and `Retry-After` are standard HTTP. For the remaining budget, many APIs send `X-RateLimit-Limit`, `X-RateLimit-Remaining` and `X-RateLimit-Reset`; an IETF draft proposes unprefix `RateLimit-*` fields instead. There is no final standard yet, so pick one style, document it, and use it consistently.

INTERVIEW ANGLE

Design a rate limiter for an API with many servers. Keep the counters in a shared store (Redis), key them by client identity, and make the check-and-update atomic with a Lua script so concurrent requests cannot both pass. Token bucket gives rate plus burst; sliding window counter gives smooth limits cheaply. Read time from Redis, not from the app servers. Return 429 with `Retry-After`, and decide fail-open versus fail-closed. In Redis Cluster a client's key lives on one shard, so the limiter scales by client.

WATCH OUT

Rate limiting is not authentication. Limiting by IP protects you from accidents and crude abuse, not from a distributed attacker with many addresses. Combine it with per-account limits, and apply stricter limits to sensitive endpoints such as login and password reset.

KEY POINTS

- Pick the key (IP, user, API key, route) and the failure policy (open or closed) before the algorithm.
- The check and the update must be atomic: `INCR` for the fixed window, a Lua script for everything else.
- Fixed window is cheapest but allows edge bursts; sliding log is exact but memory heavy; sliding counter is the usual compromise; token bucket models rate plus burst.
- Read the clock inside Redis with `TIME`, and let TTLs delete idle clients' state.
- Reject with 429, `Retry-After` and a consistent set of rate-limit headers.

8.9 Idempotency keys and deduplication

Networks fail in a nasty way: the client sends "pay 40 EUR", the server charges the card, and the response is lost. The client cannot tell whether the request arrived, so it retries. Without protection, the customer is charged twice. The same happens with message consumers, which get redelivered after a crash, and with users who double-click a button.

An operation is **idempotent** if performing it several times has the same effect as performing it once. `PUT /users/7/email` is naturally idempotent; `POST /orders` is not. An **idempotency key** makes a non-idempotent operation safe to retry: the client generates a unique key (a UUID) for each *logical* operation and sends it with every attempt, usually in an `Idempotency-Key` header. The server remembers the key and, on a repeat, returns the stored result instead of acting again.

8.9.1 The basic pattern: claim, execute, store

Redis fits because `SET key value NX EX ttl` is an atomic "claim if nobody else has" operation with automatic expiry.

Claiming an idempotency key

TERMINAL

```
127.0.0.1:6379> SET idem:7f3c-91aa "IN_PROGRESS" NX EX 86400
OK
127.0.0.1:6379> SET idem:7f3c-91aa "IN_PROGRESS" NX EX 86400
(nil)
127.0.0.1:6379> GET idem:7f3c-91aa
"IN_PROGRESS"
```

The first caller gets **OK** and owns the operation. The second gets **(nil)**: the key exists, so it must not execute the work. What it does next depends on the stored state:

1. **Key absent:** claim it, do the work, then overwrite the value with the finished result (keeping or resetting the TTL).
2. **Value IN_PROGRESS:** another attempt is still running. Return **409 Conflict**, or let the caller wait and retry shortly.
3. **Value holds a result:** return the stored response, status and body, exactly as the first attempt did. This is the **replay**.

The idempotency key should also be bound to the request that created it. If a client reuses a key with a different payload, that is a client bug and the safest answer is **422** or **400** instead of silently replaying an unrelated result. Store a hash of the request body next to the response and compare it.

8.9.2 Implementation in Spring

IdempotencyService.java

JAVA

```
package com.shopstream.order.idempotency;

import java.time.Duration;
import java.util.Optional;
import java.util.function.Supplier;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Service;

@Service
public class IdempotencyService {

    private static final String IN_PROGRESS = "IN_PROGRESS";
    private static final Duration KEEP = Duration.ofHours(24);
    private static final Duration LOCK_TTL = Duration.ofSeconds(60);

    public sealed interface Outcome permits Executed, Replayed, Busy {}
    public record Executed(String body) implements Outcome {}
    public record Replayed(String body) implements Outcome {}
    public record Busy() implements Outcome {}

    private final StringRedisTemplate redis;

    public IdempotencyService(StringRedisTemplate redis) {
        this.redis = redis;
    }

    public Outcome execute(String idemKey, Supplier<String> work) {
        String key = "idem:" + idemKey;

        Boolean claimed = redis.opsForValue().setIfAbsent(key, IN_PROGRESS, LOCK_TTL);
        if (Boolean.TRUE.equals(claimed)) {
            try {
                String result = work.get();
                redis.opsForValue().set(key, "DONE:" + result, KEEP);
                return new Executed(result);
            } catch (RuntimeException e) {
                redis.delete(key); // allow a clean retry after a failure
                throw e;
            }
        }

        String stored = redis.opsForValue().get(key);
        if (stored == null) {
            return execute(idemKey, work); // expired between SET and GET: try again
        }
        if (IN_PROGRESS.equals(stored)) {
            return new Busy();
        }
        return new Replayed(stored.substring("DONE:".length()));
    }
}
```

What to notice:

- `setIfAbsent(key, value, Duration)` maps to `SET key value NX PX ...`, one atomic command.
- The claim uses a short TTL (60 seconds). If the process dies mid-work, the key vanishes and a retry can proceed. The final result uses a long TTL (24 hours), which is your **retry window**: a retry after that is treated as a new request.
- On failure the key is deleted so that the client's retry is not blocked. If the work failed *after* causing a side effect you cannot undo, deleting is wrong; there, store a failure result instead.
- The sketch is not safe against one thing: the work succeeded, but the process died before the final `set`. Then the claim expires and a retry runs the work a second time. Redis alone cannot close that gap; see the next section.

8.9.3 The exactly-once illusion

No distributed system gives you "exactly once" delivery across a network and a crash. What you can build is **at-least-once delivery plus idempotent processing**, which has the same visible effect. Redis helps with the "remember what I have seen" half, but the guarantee is only as strong as the place where the side effect is recorded.

If the effect is a row in PostgreSQL, the strongest idempotency key is stored in the same database transaction as the effect (a unique constraint on the key). Then a crash either commits both or neither. The Redis key becomes a fast first line of defence that absorbs most duplicates cheaply, while the database constraint is the final authority. For effects on external systems (a payment provider), pass your idempotency key through to them: most payment APIs accept one.

REAL WORLD

Idempotent order creation ShopStream's `order-service` receives `POST /orders` with an `Idempotency-Key` header from the mobile app. The service claims `idem:{key}` in Redis, inserts the order with the same key under a unique constraint, publishes to `stream:orders`, and stores the response. A double tap returns the first response with the same order id; a crash after the insert is handled by the unique constraint.

8.9.4 Deduplicating messages and events

Consumers of queues and streams are redelivered messages. Dedup keys work the same way, with a business id instead of a client header:

REDIS

```
SET dedup:payment-events:evt_9812 1 NX EX 604800
# OK -> first time, process it
# (nil) -> duplicate, skip it
```

Choose the TTL to be longer than the maximum redelivery delay of the source. Count the cost: one small key per event. For a million events per day with a 7-day TTL, that is millions of keys, each with at least tens of bytes of overhead, so size it (see the sizing chapter in the operations part).

A Bloom filter for huge streams

When exact answers are not required, a **Bloom filter** answers "have I seen this id?" in a few bits per item. It can say "definitely not seen" or "probably seen" (false positives, never false negatives). Redis 8 includes it built in, no module needed.

TERMINAL

```
Bloom filter for seen events
127.0.0.1:6379> BF.RESERVE seen:events 0.001 10000000
OK
127.0.0.1:6379> BF.ADD seen:events evt_9812
(integer) 1
127.0.0.1:6379> BF.ADD seen:events evt_9812
(integer) 0
127.0.0.1:6379> BF.EXISTS seen:events evt_5555
(integer) 0
```

`BF.RESERVE` takes a target error rate (0.1 percent here) and an expected capacity. `BF.ADD` returns `1` if the item was newly added and `0` if it was probably already present. A false positive means a *new* event is wrongly treated as duplicate and dropped. That is acceptable for "do not send the same marketing email twice" and unacceptable for "apply this payment". Use a Bloom filter only where a rare loss is tolerable, or as a pre-filter that sends the "probably seen" cases to an exact check.

GOTCHA

Resizing a Bloom filter A Bloom filter sized for 10 million items degrades when you insert far more. Default filters scale by adding sub-filters, which raises the error rate and memory. Estimate capacity honestly and monitor with `BF.INFO`. Also remember that Bloom filters cannot delete items (use a Cuckoo filter if you need that).

INTERVIEW ANGLE

How do you prevent a double charge when a client retries? Require an idempotency key per logical operation. Atomically claim it with `SET NX` and a TTL, execute, store the response, and replay it for repeats. Bind the key to the request payload. Make the final authority a unique constraint in the same transaction as the side effect, because Redis and the database cannot commit atomically together. Delivery is at-least-once; idempotent processing makes the effect exactly-once.

KEY POINTS

- Idempotency means a retry has the same effect as the first attempt; the idempotency key identifies the logical operation.
- **SET key value NX EX** is the atomic claim; store **IN_PROGRESS** with a short TTL, then the result with a long TTL.
- Replay stored responses, reject reuse with a different payload, and delete the claim on clean failure.
- "Exactly once" is at-least-once delivery plus idempotent processing; the final guard belongs in the system that holds the side effect.
- Bloom filters dedup huge streams cheaply but have false positives; do not use them where a dropped event is costly.

8.10 Queues and background jobs

Some work should not happen inside the web request: sending an e-mail, resizing an image, generating an invoice PDF, calling a slow partner API. The request handler puts a description of the work on a **queue** and returns immediately; one or more **workers** take jobs off the queue and do them in the background. The queue absorbs bursts, decouples the speed of producers and consumers, and lets you retry failures without the user waiting.

Redis offers three building blocks for this, with very different guarantees: lists (simple), streams with consumer groups (robust), and pub/sub (not a queue at all, because messages are lost if nobody is listening). This chapter builds the first two properly and tells you when to stop and use a dedicated broker.

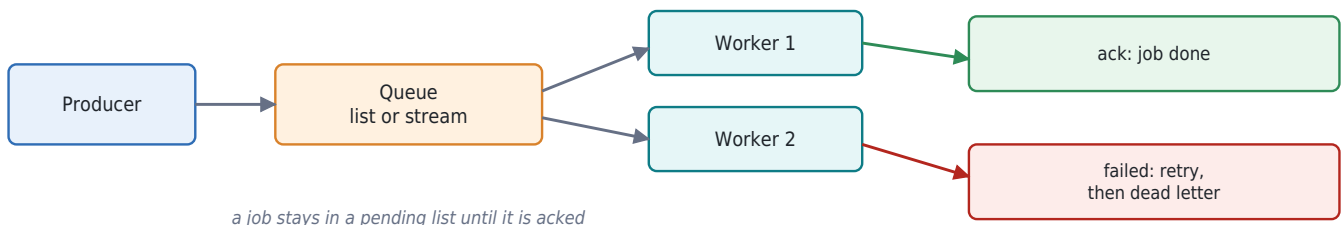


Figure 8.7. A job queue: producers enqueue, workers claim jobs, acknowledge on success, and failed jobs are retried or moved to a dead-letter queue.

8.10.1 What a reliable queue needs

A queue is "reliable" when a job is not lost if a worker crashes halfway through. That requires four things:

- **Claiming:** a job handed to a worker must not be handed to a second worker at the same time.
- **Acknowledgement:** the worker tells the queue "done" only after the work succeeded.
- **Redelivery:** a job claimed but never acknowledged (crashed worker) returns to the queue after a timeout. This timeout is the **visibility timeout** in SQS terms.
- **Poison handling:** a job that fails every time must stop after N attempts and be parked in a **dead-letter queue** (DLQ) for inspection, otherwise it blocks or burns the workers forever.

8.10.2 Level 1: a list as a queue

The simplest queue is a list: producers **LPUSH** at the head, workers **RPOP** from the tail, first in first out. The blocking variant **BRPOP queue timeout** waits until an element is available, so idle workers do not poll in a loop.

A list queue

TERMINAL

```

127.0.0.1:6379> LPUSH queue:emails "{\"to\":\"asha@example.com\",\"template\":\"welcome\"}"
(integer) 1
127.0.0.1:6379> BRPOP queue:emails 5
1) "queue:emails"
2) "{\"to\":\"asha@example.com\",\"template\":\"welcome\"}"
  
```

This is attractively simple and fast, but **BRPOP removes** the job. If the worker crashes right after popping, the job is gone. That is acceptable for jobs you can regenerate (cache warming, metrics flushes) and not for e-mails or payments.

8.10.3 Level 2: reliable list queue with LMOVE

Instead of removing the job, atomically *move* it to a per-worker "processing" list with **LMOVE** (or the blocking **BLMOVE**). The job is now in exactly one list at any moment. When the work finishes, remove it from the processing list with **LREM**.

Reliable queue pattern

TERMINAL

```
127.0.0.1:6379> LPUSH queue:emails job-1 job-2
(integer) 2
127.0.0.1:6379> BLMOVE queue:emails queue:emails:processing:w1 RIGHT LEFT 5
"job-1"
127.0.0.1:6379> LRange queue:emails:processing:w1 0 -1
1) "job-1"
127.0.0.1:6379> LREM queue:emails:processing:w1 1 "job-1"
(integer) 1
```

BLMOVE source destination **RIGHT LEFT** timeout pops from the right of the source (the oldest job, since producers push left) and pushes onto the left of the destination, atomically. **LREM key 1 value** is the acknowledgement. (The older **RPOPLPUSH** and **BRPOPLPUSH** do the same and are deprecated.)

A crashed worker leaves its job in `queue:emails:processing:w1`. A **reaper** must find stuck jobs and push them back. The list does not record when a job was claimed, so you need extra bookkeeping, typically a sorted set `processing:deadlines` with the job id as member and the claim time as score. You are now writing a queue library by hand: claim timestamps, a reaper, retry counters, a DLQ. That is the moment to switch to streams, which have those mechanisms built in.

GOTCHA

Large payloads in lists **LREM** scans the list comparing values. With big payloads or long processing lists it gets slow. Keep the job in the queue small (an id or a compact JSON) and store bulky data elsewhere.

8.10.4 Level 3: streams with consumer groups

A **stream** is an append-only log. A **consumer group** gives each entry to exactly one consumer in the group and tracks, per consumer, which entries were delivered but not yet acknowledged: the **pending entries list** (PEL). That is claiming, acknowledgement and redelivery in one data structure. (Streams and groups are introduced in the data structures part; here we use them as a job queue.)

A stream job queue

TERMINAL

```
127.0.0.1:6379> XGROUP CREATE stream:jobs workers $ MKSTREAM
OK
127.0.0.1:6379> XADD stream:jobs * type email to asha@example.com
"1718000000000-0"
127.0.0.1:6379> XREADGROUP GROUP workers w1 COUNT 1 BLOCK 2000 STREAMS stream:jobs >
1) 1) "stream:jobs"
   2) 1) 1) "1718000000000-0"
        2) 1) "type"
           2) "email"
           3) "to"
           4) "asha@example.com"
127.0.0.1:6379> XACK stream:jobs workers 1718000000000-0
(integer) 1
```

The special id `>` means "entries never delivered to anyone in this group". Until **XACK** runs, the entry stays in the PEL. If worker `w1` dies, the entry sits there with an idle time that keeps growing. Another worker can take it over with **XAUTOCLAIM**:

Taking over abandoned jobs

TERMINAL

```
127.0.0.1:6379> XAUTOCLAIM stream:jobs workers w2 60000 0-0 COUNT 10
```

This transfers entries that have been pending for at least 60,000 ms (the **min-idle-time**, your visibility timeout) to `w2` and returns them, together with a cursor for the next call. Each claim increments the entry's **delivery counter**, visible in `XPENDING stream:jobs workers - + 10`. That counter is what you use to detect poison jobs: when it passes a limit, instead of processing, write the job to a dead-letter stream and acknowledge it.

NOTE

Redis 8.4 added a **CLAIM min-idle-time** option to **XREADGROUP**, which lets a single read return both new entries and abandoned ones. Check the command page before using it; the **XAUTOCLAIM** loop below works on every version.

A worker in Spring

The code below uses `StringRedisTemplate` stream operations directly. A worker thread reads, processes, acknowledges; a second scheduled method reclaims stuck jobs and applies the retry limit.

JobWorker.java

JAVA

```
package com.shopstream.notification.jobs;

import java.time.Duration;
import java.util.List;
import java.util.Map;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.data.domain.Range;
import org.springframework.data.redis.connection.stream.Consumer;
import org.springframework.data.redis.connection.stream.MapRecord;
import org.springframework.data.redis.connection.stream.PendingMessage;
import org.springframework.data.redis.connection.stream.PendingMessages;
import org.springframework.data.redis.connection.stream.ReadOffset;
import org.springframework.data.redis.connection.stream.StreamOffset;
import org.springframework.data.redis.connection.stream.StreamReadOptions;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

@Component
public class JobWorker {

    private static final Logger log = LoggerFactory.getLogger(JobWorker.class);
    private static final String STREAM = "stream:jobs";
    private static final String DLQ = "stream:jobs:dlq";
    private static final String GROUP = "workers";
    private static final long MAX_DELIVERIES = 5;
    private static final Duration VISIBILITY = Duration.ofSeconds(60);

    private final StringRedisTemplate redis;
    private final String consumerName = "worker-" + java.util.UUID.randomUUID();

    public JobWorker(StringRedisTemplate redis) {
        this.redis = redis;
    }

    /** Called in a loop by a dedicated thread. */
    public void pollOnce() {
        List<MapRecord<String, Object, Object>> records = redis.opsForStream().read(
            Consumer.from(GROUP, consumerName),
            StreamReadOptions.empty().count(10).block(Duration.ofSeconds(2)),
            StreamOffset.create(STREAM, ReadOffset.lastConsumed()));
        if (records == null) {
            return;
        }
        for (MapRecord<String, Object, Object> rec : records) {
            try {
                handle(rec.getValue());
                redis.opsForStream().acknowledge(STREAM, GROUP, rec.getId());
            } catch (RuntimeException e) {
                log.warn("job {} failed, will be retried", rec.getId(), e);
                // no XACK: the entry stays pending and is reclaimed after the visibility timeout
            }
        }
    }

    @Scheduled(fixedDelay = 30_000)
    public void reclaimStuckJobs() {
        PendingMessages pending = redis.opsForStream()
            .pending(STREAM, GROUP, Range.unbounded(), 100);
        for (PendingMessage p : pending) {
            if (p.getElapsedTimeSinceLastDelivery().compareTo(VISIBILITY) < 0) {
                continue;
            }
            if (p.getTotalDeliveryCount() >= MAX_DELIVERIES) {
                moveToDeadLetter(p);
            } else {
                redis.opsForStream().claim(STREAM, GROUP, consumerName, VISIBILITY, p.getId());
            }
        }
    }
}
```

JobWorker.java (continued)

JAVA

```

    }

    private void moveToDeadLetter(PendingMessage p) {
        List<MapRecord<String, Object, Object>> found =
            redis.opsForStream().range(STREAM, Range.just(p.getIdAsString()));
        if (found != null && !found.isEmpty()) {
            redis.opsForStream().add(DLQ, found.get(0).getValue());
        }
        redis.opsForStream().acknowledge(STREAM, GROUP, p.getId());
    }

    private void handle(Map<Object, Object> job) {
        // send the e-mail, resize the image, call the partner API ...
    }
}

```

What to notice:

- The worker never acknowledges on failure. Redelivery is automatic and needs no retry bookkeeping of your own: the delivery counter in the PEL is the retry counter.
- `claim(...)` with a minimum idle time only succeeds if the entry has been idle at least that long, so two reclaiming workers cannot both steal the same job at the same moment.
- Dead-lettering copies the payload to `stream:jobs:dlq` and acknowledges the original, so the main PEL does not fill with poison jobs. Someone must monitor the DLQ and decide: fix and replay, or discard.
- A job that takes longer than the visibility timeout will be redelivered while still running. Either set the timeout comfortably above your slowest job or make the handler **idempotent** (see the previous chapter). Treat every queue job as at-least-once.

The method names for pending-entry inspection and claiming (`pending`, `claim`) follow the Spring Data Redis stream API, but exact overloads differ between versions; check them in the reference for your version before copying.

8.10.5 Limits of stream queues

Streams are held in memory, so a backlog costs RAM; trim with `XTRIM MAXLEN ~` or by minimum id, but only after entries are acknowledged, since trimming also deletes entries that are still pending. There is no per-message delay, no priority, no ordering across a consumer group by key, and consumer-group state is replicated asynchronously, so a failover can lose the latest acknowledgements or entries (you may then see a duplicate, which idempotency absorbs, or in rare cases lose an entry).

Property	List with LMOVE	Stream + group	Pub/Sub
Survives worker crash	with a reaper you write	yes (PEL, XAUTOCLAIM)	no
Retry count	you build it	delivery counter built in	none
Several workers share load	yes	yes	no, all receive all
Replay of history	no	yes (log stays until trimmed)	no
Delay or priority	with sorted sets	with sorted sets	no
Best for	simple, loss tolerable	most background jobs	notifications to live listeners

Redis queue options compared.

8.10.6 Redis queues versus Kafka and RabbitMQ

Choose Redis for queues when jobs are small, latency matters, volumes are moderate, and losing a job in a rare failover is tolerable or fixable by idempotent re-enqueue from your database. It is operationally cheap if you already run Redis.

Choose a dedicated broker when you need: durable storage as the point of truth (Kafka's replicated, disk-based log), very large retention and replay for many independent consumers (Kafka), rich routing, per-message TTL, priorities and publisher confirms (RabbitMQ), or strict ordering within partitions at high throughput. Redis is memory-bound and its replication is asynchronous; those are the two facts that decide most comparisons.

Libraries that already did this

You rarely need to write the machinery above for production. Popular job libraries built on Redis include Sidekiq (Ruby), BullMQ (Node.js), RQ and Celery with a Redis broker (Python), and, in the Java world, Redisson's queue and executor classes. They add scheduling, retries with backoff, rate limits and dashboards. Evaluate them against your language and your durability needs; for Spring applications many teams keep a small stream worker like the one above, or use a database-backed scheduler for jobs that must never be lost.

INTERVIEW ANGLE

How do you build a reliable work queue on Redis? Use a stream with a consumer group. Workers read with `XREADGROUP`, process, then `XACK`. Unacknowledged entries stay in the pending list; a reaper uses `XAUTOCLAIM` with a minimum idle time as the visibility timeout. The delivery counter limits retries, and poison jobs go to a dead-letter stream. Delivery is at-least-once, so handlers must be idempotent. With plain lists, use `BLMOVE` into a processing list instead of `BRPOP` so a crash does not lose the job.

KEY POINTS

- A queue is reliable only if claimed jobs are tracked, acknowledged after success, redelivered after a timeout, and parked in a DLQ after repeated failure.
- `BRPOP` loses jobs on a crash; `BLMOVE` into a processing list is the minimal reliable list queue; streams with consumer groups give the PEL, `XAUTOCLAIM` and delivery counts for free.
- Choose the visibility timeout above the longest normal job duration, and make handlers idempotent.
- Trim streams only past acknowledged entries; backlog is RAM.
- Redis suits moderate, latency-sensitive queues; use Kafka or RabbitMQ when durable log retention, replay at scale or rich routing is the requirement.

8.11 Delayed and scheduled tasks

"Send the reminder in 30 minutes." "Cancel the unpaid order after 15 minutes." "Retry in 10 seconds, then 40, then 160." All of these are **delayed tasks**: work that must become eligible at a specific future time. A plain queue cannot express this, since it releases items as soon as they are pushed.

8.11.1 The sorted set as a timer wheel

A sorted set sorted by due time is the standard answer. The member is the task id (or its payload), the score is the due time as epoch milliseconds. Everything with a score in the past is ready.

Scheduling and polling delayed tasks

TERMINAL

```
127.0.0.1:6379> ZADD delayed:jobs 1718003600000 "cancel-order:9001"
(integer) 1
127.0.0.1:6379> ZADD delayed:jobs 1718000900000 "reminder:cart:42"
(integer) 1
127.0.0.1:6379> ZRANGE delayed:jobs 0 0 WITHSCORES
1) "reminder:cart:42"
2) "1718000900000"
127.0.0.1:6379> ZRANGEBYSCORE delayed:jobs -inf 1718001000000 LIMIT 0 10
1) "reminder:cart:42"
```

Looking at the first element (`ZRANGE ... 0 0 WITHSCORES`) tells a poller when the next job is due, so it can sleep exactly that long (capped, say, at one second so new earlier jobs are noticed quickly). `ZRANGEBYSCORE ... -inf now` lists what is ready.

8.11.2 Claiming due tasks atomically

If several pollers run, two of them could read the same due task and both execute it. The fix is to move the task out of the sorted set in one atomic step. A Lua script reads the due entries, removes them, and pushes them onto a ready list (or a stream) from which workers consume with `BLMOVE` or a consumer group.

promote_due.lua

LUA

```
-- KEYS[1] = delayed zset, KEYS[2] = ready list, ARGV[1] = max batch size
local t = redis.call('TIME')
local now = t[1] * 1000 + math.floor(t[2] / 1000)
local due = redis.call('ZRANGEBYSCORE', KEYS[1], '-inf', now, 'LIMIT', 0, ARGV[1])
for _, id in ipairs(due) do
    redis.call('ZREM', KEYS[1], id)
    redis.call('LPUSH', KEYS[2], id)
end
return #due
```

Because scripts run one at a time, two pollers calling this script can never promote the same task. The script returns how many tasks it moved; if that equals the batch size, poll again immediately, otherwise sleep. A Java polling loop is then a few lines:

DelayedPromoter.java

JAVA

```
package com.shopstream.order.jobs;

import java.util.List;
import org.springframework.core.io.ClassPathResource;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.core.script.DefaultRedisScript;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

@Component
public class DelayedPromoter {

    private final StringRedisTemplate redis;
    private final DefaultRedisScript<Long> promote = new DefaultRedisScript<>();

    public DelayedPromoter(StringRedisTemplate redis) {
        this.redis = redis;
        promote.setLocation(new ClassPathResource("scripts/promote_due.lua"));
        promote.setResultType(Long.class);
    }

    public void schedule(String taskId, long dueAtEpochMs) {
        redis.opsForZSet().add("delayed:jobs", taskId, dueAtEpochMs);
    }

    @Scheduled(fixedDelay = 1000)
    public void promoteDue() {
        Long moved;
        do {
            moved = redis.execute(promote, List.of("delayed:jobs", "queue:ready"), "100");
        } while (moved != null && moved >= 100);
    }
}
```

What to notice: scheduling a task with an existing id simply updates its score, which gives you **rescheduling** for free; cancelling is **ZREM**. The promoter runs on every instance, and the script makes that safe. The granularity is the polling interval (one second here). The promoted task goes to a normal queue, so retry and DLQ handling from the previous chapter apply to it unchanged.

Between "ZREM and LPUSH" and the worker finishing there is a window; if you need crash safety there too, promote into a stream rather than a list, and use the consumer-group mechanics.

8.11.3 Retry with backoff

Retries are just delayed tasks: on failure, compute the next delay ($\text{base} \times 2^{\text{attempt}}$ with some random jitter), **ZADD** the job into **delayed:jobs** with **now + delay**, and acknowledge the current delivery. Store the attempt number inside the job payload. After the maximum number of attempts, write to the DLQ instead.

8.11.4 Keyspace notifications are not a scheduler

It is tempting to use key expiry as the timer: set `order:9001:unpaid` with a 15 minute TTL and subscribe to the expired-key event, then cancel the order when it fires. Do not do this for anything that matters. Redis can publish a message when a key expires (keyspace notifications, disabled by default, switched on with `notify-keyspace-events`), but:

- Expiry is not exact. Keys are removed lazily when touched, and by a periodic background sampler, so the event can arrive late, especially on a busy server or when many keys expire together.
- The notification is published over pub/sub, which is fire-and-forget. If your subscriber is disconnected, restarting, or slow when the key expires, the event is gone forever and the order is never cancelled.
- The expired event carries the key name only, not the value, which is already deleted.
- In a cluster, notifications are per node and a subscriber must connect to every node.
- Every instance of your service receives the event, so you must add your own deduplication so the cancel runs once.

WATCH OUT

Using key expiry as a reliable timer A notification-based "cancel unpaid orders" job silently misses orders whenever the listener is down at the wrong moment. Use the sorted set approach (or a database-backed scheduler) as the source of truth. If you also like the expiry event as an optimisation, keep a periodic sweep as the safety net.

For long-term or calendar-style schedules (every night at 02:00, on the first of the month) use a real scheduler: Spring's `@Scheduled` with a cluster lock (see the distributed locks chapter), Quartz, or an external scheduler. Redis timers are best for the many short, per-entity delays.

INTERVIEW ANGLE

How would you cancel unpaid orders 15 minutes after creation? Add the order id to a sorted set with score `createdAt + 15 min`. A poller promotes due entries with an atomic Lua script into a work queue, and the worker checks the order is still unpaid before cancelling (idempotent). Key-expiry notifications are not reliable enough: they are fire-and-forget and can be late or missed.

KEY POINTS

- A sorted set scored by due time (epoch ms) is the Redis delayed-task structure; `ZRANGEBYSCORE -inf now` finds the ready tasks.
- Promote due tasks with a Lua script so concurrent pollers never process the same task twice; rescheduling is `ZADD`, cancelling is `ZREM`.
- Retries with backoff are delayed tasks with an attempt counter in the payload.
- Expired-key notifications are best-effort pub/sub events, not a reliable scheduler.
- Keep a periodic sweep or a database record as the safety net for anything that must happen.

8.12 Counters, analytics and unique counts

"How many page views today?" "How many different visitors?" "Which products are trending this hour?" Questions like these are asked by every product, and answering them from a relational database means a write per event and expensive aggregation queries. Redis answers them with in-memory, atomic counters and compact probabilistic structures, at a cost of a few bytes or kilobytes per metric.

The key is to match the question to the structure. This chapter walks through the common ones, from exact and cheap to approximate and tiny.

8.12.1 Plain counters with INCR

`INCR` and `INCRBY` are atomic: a thousand concurrent clients all incrementing the same key never lose an update, and no read-modify-write cycle is needed. A counter per metric per time bucket is the foundation of most analytics.

Counting page views per day

TERMINAL

```
127.0.0.1:6379> INCR views:product:1001:2026-10-10
(integer) 1
127.0.0.1:6379> INCRBY views:product:1001:2026-10-10 5
(integer) 6
127.0.0.1:6379> EXPIRE views:product:1001:2026-10-10 2592000
(integer) 1
127.0.0.1:6379> GET views:product:1001:2026-10-10
"6"
```

Put the time bucket in the key (`...:2026-10-10`, or `...:2026-10-10T14` for hourly) and give it a TTL so old buckets disappear without a cleanup job. Summing a week is a `MGET` of seven keys, or a pipeline of seven `GET`s.

When one logical event has several dimensions, keep them together in a hash with `HINCRBY`: `HINCRBY stats:2026-10-10 orders:web 1` increments one field of a per-day hash, so one key holds many related counters and a single `HGETALL` reads them all. Hashes of small size are stored compactly (see the chapter on encodings), which makes them more memory efficient than thousands of separate string keys.

GOTCHA

Counters are values, not truth Redis counters are incremented by your application; if a request is retried and the increment runs twice, the count is off by one. For counts that feed billing or compliance, derive them from an event log and treat Redis as a fast approximation, or make the increment idempotent (see the idempotency chapter).

8.12.2 Unique counts with HyperLogLog

"How many distinct visitors today?" Storing every visitor id in a set gives an exact answer but costs memory proportional to the number of visitors: tens of millions of ids are gigabytes. A **HyperLogLog** (HLL) estimates the number of distinct items using a fixed amount of memory, at most about 12 KB per key, with a standard error of roughly 0.81 percent.

Unique visitors per day

TERMINAL

```
127.0.0.1:6379> PFADD uv:2026-10-10 user:42 user:77 user:42
(integer) 1
127.0.0.1:6379> PFADD uv:2026-10-10 user:99
(integer) 1
127.0.0.1:6379> PFCOUNT uv:2026-10-10
(integer) 3
127.0.0.1:6379> PFMERGE uv:week:2026-W41 uv:2026-10-10 uv:2026-10-11
OK
127.0.0.1:6379> PFCOUNT uv:week:2026-W41
(integer) 3
```

`PFADD` returns `1` if the add changed the internal registers (so the estimate may have changed) and `0` otherwise; it does not tell you whether the item was "new". `PFCOUNT` returns the estimate. `PFMERGE` combines several HLLs into one that counts the union: that is how you roll days into weeks without re-reading any visitor ids. You can also give several keys to `PFCOUNT` to count their union directly.

What an HLL cannot do: list members, test membership, or delete an item. Use it when "about 1.2 million unique visitors" is a good enough answer. It is the wrong tool for "has user 42 visited?" (use a set, a bitmap or a Bloom filter) and for small cardinalities where an exact set is cheap.

8.12.3 Bitmaps for daily active users

A bitmap is a string treated as an array of bits. If user ids are small integers, set bit number `userId` to `1` when that user is active that day. One million users cost 125 KB per day, and you get exact answers plus fast set algebra.

Daily active users with bitmaps

TERMINAL

```
127.0.0.1:6379> SETBIT active:2026-10-10 42 1
(integer) 0
127.0.0.1:6379> SETBIT active:2026-10-10 77 1
(integer) 0
127.0.0.1:6379> SETBIT active:2026-10-11 42 1
(integer) 0
127.0.0.1:6379> GETBIT active:2026-10-10 77
(integer) 1
127.0.0.1:6379> BITCOUNT active:2026-10-10
(integer) 2
```

Daily active users with bitmaps (continued)

TERMINAL

```
127.0.0.1:6379> BITOP AND active:both active:2026-10-10 active:2026-10-11
(integer) 10
127.0.0.1:6379> BITCOUNT active:both
(integer) 1
```

SETBIT returns the previous bit value, which doubles as "was this the first visit today?". **BITCOUNT** counts the ones (the daily active users). **BITOP AND** over two days gives the users active on both (retention), **OR** the users active on either. The integer returned by **BITOP** is the length in bytes of the result.

Two limits to know. Bitmap size is determined by the **highest** offset set, so with sparse or huge ids (UUIDs, or ids in the billions) the bitmap wastes memory: setting bit 4,000,000,000 allocates 500 MB. Map users to a dense integer index first. And the first write to a far offset blocks Redis briefly while the string is allocated.

Question	Structure	Accuracy	Memory
Total events	INCR	exact	one small key
Distinct items, huge cardinality	HyperLogLog	about 0.81% error	up to 12 KB per key
Distinct items, dense integer ids	bitmap	exact	1 bit per possible id
Distinct items, need membership or listing	set	exact	proportional to items
Most frequent items	Top-K / sorted set	approximate / exact	bounded by K / by items

Choosing a counting structure.

8.12.4 Trending items: sorted sets and Top-K

For "top 10 products by views this hour", the exact approach is a sorted set per time bucket whose members are product ids:

Trending products

TERMINAL

```
127.0.0.1:6379> ZINCRBY trending:2026-10-10T14 1 product:1001
"1"
127.0.0.1:6379> ZINCRBY trending:2026-10-10T14 1 product:1001
"2"
127.0.0.1:6379> ZINCRBY trending:2026-10-10T14 1 product:2002
"1"
127.0.0.1:6379> ZRANGE trending:2026-10-10T14 0 9 REV WITHSCORES
1) "product:1001"
2) "2"
3) "product:2002"
4) "1"
```

This is exact but holds one entry per distinct product. When there are millions of distinct items and you only need the heaviest few, Redis 8 includes a probabilistic **Top-K** structure that keeps only K candidates:

Top-K (built in since Redis 8)

TERMINAL

```
127.0.0.1:6379> TOPK.RESERVE topk:searches 20
OK
127.0.0.1:6379> TOPK.ADD topk:searches "running shoes" "laptop" "running shoes"
1) (nil)
2) (nil)
3) (nil)
127.0.0.1:6379> TOPK.LIST topk:searches WITHCOUNT
1) "running shoes"
2) (integer) 2
3) "laptop"
4) (integer) 1
```

TOPK.ADD returns, for each item, the item it pushed out of the top K, or **(nil)** if none. Counts are estimates (the structure is a count-min-style sketch with decay), so use it for ranking and dashboards, not for billing. Check the command pages for the optional width, depth and decay parameters before tuning.

8.12.5 Time series

When you need values over time with ranges and aggregation (CPU per minute, orders per second), counters in keys named by timestamp get clumsy. Redis 8 includes a time series data type:

Time series basics

TERMINAL

```
127.0.0.1:6379> TS.CREATE ts:orders:web RETENTION 86400000 DUPLICATE_POLICY SUM
OK
127.0.0.1:6379> TS.ADD ts:orders:web * 1
(integer) 1718000000123
127.0.0.1:6379> TS.RANGE ts:orders:web - + AGGREGATION sum 60000
1) 1) (integer) 1718000000000
2) 1
```

RETENTION is in milliseconds (here 24 hours; older samples are dropped automatically). ***** means "now". **TS.RANGE** with **AGGREGATION sum 60000** returns per-minute sums. The data type also supports labels for querying many series (**TS.MGET**, **TS.MRANGE**) and compaction rules that downsample automatically into coarser series. For heavy long-term analytics with arbitrary queries, a columnar or time-series database remains the better home; Redis shines for recent windows and live dashboards.

REAL WORLD

Counting in ShopStream ShopStream keeps **uv:{date}** HyperLogLogs for unique visitors, **trending:{hour}** sorted sets for the homepage's "trending now" strip, and **INCR**-based order counters per minute. A nightly job reads the finished day's keys, writes the totals to the data warehouse, and lets TTLs delete the Redis copies after a few days.

KEY POINTS

- **INCR** and **HINCRBY** are atomic; put the time bucket in the key and add a TTL.
- HyperLogLog counts distinct items in at most about 12 KB with roughly 0.81% error; **PFMERGE** unions days into weeks. It cannot list or test members.
- Bitmaps give exact per-user flags and set algebra (**BITCOUNT**, **BITOP**) but need dense integer ids.
- A sorted set gives exact trending; Top-K gives approximate heavy hitters in bounded memory.
- Redis counters are fast approximations of your event history; derive billing-grade numbers elsewhere.

8.13 Leaderboards and ranking

A leaderboard answers "who is first, who is next, and where do I stand?" while scores keep changing. Computing that in SQL means sorting millions of rows on every request. A Redis **sorted set** keeps its members ordered by score at all times, so every operation is logarithmic and you never sort on read.

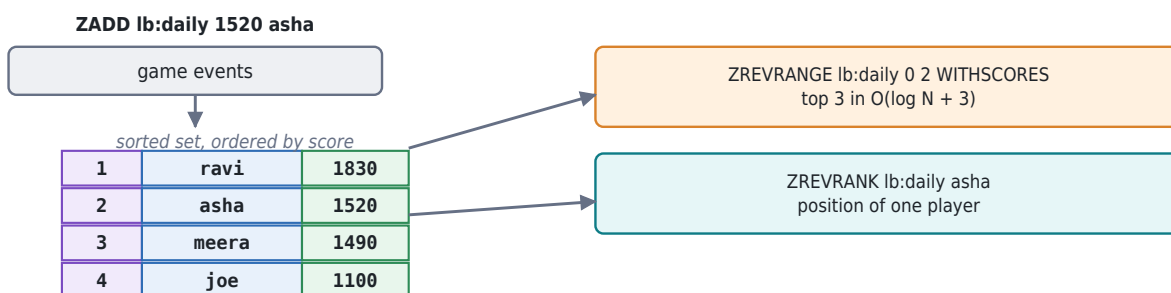


Figure 8.8. A sorted set as a leaderboard: member = player or product, score = points; rank queries read the ordered structure directly.

In ShopStream the board is **lb:bestsellers:daily**: members are product ids, the score is units sold today. The same code serves a game's top players or a "most active reviewers" page.

8.13.1 The core operations

```
A leaderboard in five commands TERMINAL
127.0.0.1:6379> ZADD lb:game alice 3200 bob 4100 carol 2750
(integer) 3
127.0.0.1:6379> ZINCRBY lb:game 500 carol
"3250"
127.0.0.1:6379> ZRANGE lb:game 0 2 REV WITHSCORES
1) "bob"
2) "4100"
3) "carol"
4) "3250"
5) "alice"
6) "3200"
127.0.0.1:6379> ZREVRANK lb:game alice
(integer) 2
127.0.0.1:6379> ZSCORE lb:game alice
"3200"
```

- **ZADD** *key score member* inserts or updates. **ZINCRBY** adds to the score atomically, which is how points accumulate from many servers.
- **ZRANGE** *key 0 9 REV WITHSCORES* returns the top ten, highest first. (The older **ZREVRANGE** is deprecated in favour of **ZRANGE ... REV**.)
- **ZREVRANK** *key member* returns the zero-based position counted from the highest score; add one for a human rank. **ZRANK** counts from the lowest. Since Redis 7.2, **ZRANK** and **ZREVRANK** accept **WITHSCORE** to return rank and score in one call.
- **ZSCORE** returns one member's score.
- **ZADD** options help with updates: **GT** only updates if the new score is greater (a personal best), **NX** only adds new members.

The cost: **ZADD**, **ZINCRBY**, **ZRANK** are $O(\log N)$; **ZRANGE** is $O(\log N + M)$ for M returned items. Even with tens of millions of members these remain fast.

8.13.2 Pagination

For "top 100" pages, use ranks: page p of size n is **ZRANGE** *key $p*n$ $(p+1)*n-1$ REV WITHSCORES*. Rank-based paging is stable enough for leaderboards, because users expect positions to move. For infinite scroll over a rapidly changing board, prefer a cursor made of the last seen (score, member) and ask for items below it with **ZRANGE** *key (score -inf BYSCORE REV LIMIT 0 n*, so a new entry at the top does not shift every page by one.

8.13.3 Around-me queries

"Show me my rank and my neighbours" is the feature players love. Find the player's rank, then fetch a window around it.

```
LeaderboardService.java JAVA

package com.shopstream.catalog.ranking;

import java.util.ArrayList;
import java.util.List;
import java.util.Set;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.core.ZSetOperations.TypedTuple;
import org.springframework.stereotype.Service;

@Service
public class LeaderboardService {

    public record Entry(long rank, String member, double score) {}

    private final StringRedisTemplate redis;

    public LeaderboardService(StringRedisTemplate redis) {
        this.redis = redis;
    }

    public void addPoints(String board, String member, double points) {
        redis.opsForZSet().incrementScore(board, member, points);
    }
}
```

LeaderboardService.java (continued)

JAVA

```

    }

    public List<Entry> top(String board, int n) {
        return window(board, 0, n - 1);
    }

    public List<Entry> aroundMe(String board, String member, int radius) {
        Long rank = redis.opsForZSet().reverseRank(board, member);
        if (rank == null) {
            return List.of();
        }
        long from = Math.max(0, rank - radius);
        return window(board, from, rank + radius);
    }

    private List<Entry> window(String board, long from, long to) {
        Set<TypedTuple<String>> tuples =
            redis.opsForZSet().reverseRangeWithScores(board, from, to);
        List<Entry> result = new ArrayList<>();
        if (tuples == null) {
            return result;
        }
        long rank = from + 1;
        for (TypedTuple<String> t : tuples) {
            result.add(new Entry(rank++, t.getValue(), t.getScore()));
        }
        return result;
    }
}

```

What to notice: `reverseRank` and `reverseRangeWithScores` are the Spring wrappers for `ZREVRANK` and the descending range. The returned set preserves Redis order, so ranks are assigned by counting up from `from + 1`. Between the rank call and the range call the board may change; for a leaderboard that is fine. If you need a consistent snapshot, do both in a short Lua script or accept the tiny skew.

8.13.4 Ties

When two members have the same score, Redis orders them lexicographically by member. That is deterministic but rarely what the product wants: the player who reached the score *first* should usually rank higher. The standard trick is to encode the time into the score so that earlier means better.

A score is a double, which represents integers exactly up to 2^{53} (about 9×10^{15}). A composite score such as `points ×  $2^{32}$  + ( $2^{32}$  - 1 - epochSeconds)` fits comfortably as long as points stay below roughly two million, and it sorts higher points first, then earlier achievement first. Decode with integer division when displaying. If points can be much larger, keep the real score in a separate hash and use the sorted set only for ordering, or accept lexicographic ties.

8.13.5 Resets, windows and scale

- **Daily and weekly boards:** put the period in the key (`lb:bestsellers:2026-10-10`) and `EXPIRE` it a few days after the period ends. A new day starts with an empty board automatically, and no reset job runs.
- **Rolling aggregates:** `ZUNIONSTORE lb:bestsellers:week 7 <seven day keys>` sums the daily boards (it takes a count of keys, then the keys; `WEIGHTS` can age old days down). Run it periodically or on demand and cache the result.
- **All-time boards:** keep a long-lived sorted set updated alongside the daily one.
- **Memory:** a sorted set costs on the order of tens of bytes per member plus the member string. Ten million members is hundreds of megabytes, so size it (see the sizing chapter). To keep only the top 10,000, trim periodically with `ZREMRANGEBYRANK board 0 -10001`.
- **Scale beyond one key:** one sorted set lives on one node. Shard by region, league or season (`lb:eu:...`); cross-shard "global top 10" is then a merge of each shard's top 10, done in the application. A single huge board can also be a hot key under extreme write load.

INTERVIEW ANGLE

How do you build a real-time leaderboard that shows a user's rank? A sorted set with the user as member and the score as the score. Update with **ZINCRBY**, show the top N with **ZRANGE ... REV**, and get a user's rank with **ZREVRANK**; all are $O(\log N)$. For around-me, fetch the rank and read a window of neighbours. Break ties by encoding the achievement time into the score. Use per-period keys with TTL for daily boards, and shard by league if one key becomes too big or too hot.

KEY POINTS

- A sorted set keeps members ordered by score, so rank, top-N and neighbour queries are $O(\log N)$ without sorting on read.
- Use **ZINCRBY** for accumulating points, **ZRANGE ... REV** for the top, **ZREVRANK** for a position; prefer these over deprecated reverse commands.
- Encode time into the score to break ties fairly; keep scores inside the exactly representable range.
- Per-period keys with TTL give automatic resets; **ZUNIONSTORE** builds rolling windows.
- A single sorted set lives on one node; shard by region or league when size or write rate demands.

8.14 Sessions, carts and ephemeral state

Some data is valuable only for a short time and is read on nearly every request: who is logged in, what is in the shopping basket, which step of the checkout the user is on. Putting it in the main database adds load and latency for data that is thrown away within hours or days. This is **ephemeral state**, and Redis with TTLs is its natural store.

8.14.1 Sessions

A web application needs to remember the user between requests. With one server, memory works. With several servers behind a load balancer, either every request must go back to the same server (sticky sessions, which unbalances load and loses sessions on restart) or the session must live in a shared store. A shared Redis session store makes the application servers stateless: any instance can serve any request, and you can add or remove instances freely.

The data model is a hash per session with a TTL:

A session as a hash with a sliding TTL

TERMINAL

```
127.0.0.1:6379> HSET session:9f2c userId 42 role customer cartId c-17
(integer) 3
127.0.0.1:6379> EXPIRE session:9f2c 1800
(integer) 1
127.0.0.1:6379> HGETALL session:9f2c
1) "userId"
2) "42"
3) "role"
4) "customer"
5) "cartId"
6) "c-17"
127.0.0.1:6379> EXPIRE session:9f2c 1800
(integer) 1
```

Every request that reads the session calls **EXPIRE** again, which makes the timeout **sliding**: the session expires after 30 minutes of *inactivity*. If you also want an absolute limit (log out after 12 hours no matter what), store the creation time in the hash and check it. Logging out is a **DEL**. The session id must be long, random and unguessable (at least 128 bits from a secure random generator), delivered in a **Secure, HttpOnly** cookie.

Spring Session

In a Spring application you almost never code this by hand. The **spring-session-data-redis** dependency replaces the servlet container's session with one stored in Redis, with no change to your controllers that use **HttpSession**. Spring Boot auto-configures it when Spring Session and the Redis starter are on the classpath. The session timeout can be set with **spring.session.timeout** (Boot falls back to **server.servlet.session.timeout** when it is absent); check the Boot reference for your version for the full list of **spring.session.redis.*** options. Spring Session stores each session as a hash under a namespace prefix and uses extra bookkeeping keys so that expiry works reliably, so do not write session keys by hand next to it.

WATCH OUT

What a session may and may not hold. Keep sessions small: user id, roles, a few flags. Do not store entire user profiles or large objects, and never anything you cannot afford to lose. A Redis restart without persistence, or a failover that loses the latest writes, logs users out. That is acceptable for sessions. If it is not, enable persistence (see the persistence chapter) and accept the cost.

8.14.2 Shopping carts

A cart is a map from product to quantity, which is exactly a hash: `cart:{userId}` with one field per SKU.

```

CartService.java
package com.shopstream.cart;

import java.time.Duration;
import java.util.HashMap;
import java.util.Map;
import org.springframework.data.redis.core.HashOperations;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Service;

@Service
public class CartService {

    private static final Duration CART_TTL = Duration.ofDays(30);

    private final StringRedisTemplate redis;
    private final HashOperations<String, String, String> hash;

    public CartService(StringRedisTemplate redis) {
        this.redis = redis;
        this.hash = redis.opsForHash();
    }

    public long addItem(String userId, String sku, int quantity) {
        String key = "cart:" + userId;
        Long newQty = hash.increment(key, sku, quantity);
        redis.expire(key, CART_TTL);
        return newQty == null ? 0 : newQty;
    }

    public void removeItem(String userId, String sku) {
        hash.delete("cart:" + userId, sku);
    }

    public Map<String, Integer> view(String userId) {
        Map<String, Integer> items = new HashMap<>();
        hash.entries("cart:" + userId)
            .forEach((sku, qty) -> items.put(sku, Integer.parseInt(qty)));
        return items;
    }

    public void mergeGuestCart(String guestId, String userId) {
        view(guestId).forEach((sku, qty) -> addItem(userId, sku, qty));
        redis.delete("cart:" + guestId);
    }
}

```

What to notice: `increment` is `HINCRBY`, so "add two more of this item" is atomic even if the user taps from two devices at once. Each write refreshes the cart TTL, so abandoned carts disappear after 30 days of inactivity. `mergeGuestCart` handles the classic login moment: the anonymous cart from a guest cookie is merged into the user's cart. Prices and product names are not stored in the cart, only SKU and quantity; the checkout reads current prices from the catalogue, which avoids stale-price bugs.

Should the cart be in Redis at all? If losing a cart is acceptable (the user re-adds items) and read speed matters, yes. If carts are business-critical (marketing re-engagement on abandoned carts, high-value B2B quotes), write them to the database as well, asynchronously through a stream, and use Redis as the fast read path.

8.14.3 Other ephemeral state

The same shape fits many short-lived records:

- **One-time codes and tokens:** e-mail verification, password reset, login OTP: `SET otp:{userId} 493021 EX 300`, with a failed-attempts counter next to it and a rate limit on requests.
- **Checkout or wizard progress:** a hash per flow with a TTL.
- **"Typing" and presence indicators:** `SET presence:{userId} 1 EX 30`, refreshed by a heartbeat; missing key means offline.
- **Recently viewed items:** a capped list, `LPUSH` then `LTRIM recent:{userId} 0 19`, so it never grows beyond 20 entries.
- **Search history:** the same capped list, or a sorted set scored by timestamp.

Hash field expiration

Since Redis 7.4 individual hash fields can have their own TTL (`HEXPIRE key seconds FIELDS count field ...`), and Redis 8.0 added `HGETEX`, `HSETEX` and `HGETDEL`. This helps when one hash holds items with different lifetimes, such as per-SKU reservations inside a cart hash. It is not needed for plain carts; one TTL on the whole key is simpler. If you use field TTLs, check the command pages for the exact syntax and remember that client libraries lag behind the server.

GOTCHA

TTLs and writes Commands that overwrite a key with `SET` remove an existing TTL unless you pass `KEEPTTL`. A cart updated with `DEL` plus recreate, or a session blob rewritten with a plain `SET`, silently becomes immortal. Hash writes (`HSET`, `HINCRBY`) keep the key TTL; set the TTL again explicitly whenever you create the key.

KEY POINTS

- Ephemeral state (sessions, carts, tokens, presence) is read constantly and valuable briefly, which suits Redis with TTLs.
- A session is a small hash with a sliding TTL refreshed on access; use Spring Session instead of coding it by hand.
- A cart is a hash of SKU to quantity; `HINCRBY` makes adds atomic and each write refreshes the TTL.
- Decide explicitly what happens when Redis loses the data; store anything business-critical in the database too.
- Watch out for commands that drop TTLs (`SET` without `KEEPTTL`) and for unbounded growth of per-user structures (capped lists with `LTRIM`).

8.15 Feature flags, configuration and flash-sale stock

This chapter covers three uses that look unrelated but share one trait: a small piece of shared state that many application instances must read quickly and that must change safely. A **feature flag** switches behaviour without a deploy. A **dynamic setting** such as a page size or a maintenance banner changes at runtime. **Stock** is a counter that must never go below zero, however many buyers arrive at once.

8.15.1 Feature flags

The simplest flag is a string key: `SET ff:new-checkout on`. The service reads it with `GET` and branches. That works, but two refinements make flags usable in practice.

Percentage rollouts

To enable a feature for 10 percent of users, you need a stable assignment: a user must see the same variant on every request. Store the percentage in Redis and compute the bucket from the user id in the application:

FeatureFlags.java

JAVA

```
package com.shopstream.gateway.flags;

import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Component;

@Component
public class FeatureFlags {

    private final StringRedisTemplate redis;
```


FeatureFlags.java (continued)

JAVA

```

public FeatureFlags(StringRedisTemplate redis) {
    this.redis = redis;
}

public boolean isEnabled(String flag, String userId) {
    String value = redis.opsForValue().get("ff:" + flag);
    if (value == null || value.equals("off")) {
        return false;
    }
    if (value.equals("on")) {
        return true;
    }
    int percent = Integer.parseInt(value); // e.g. "10" means 10 percent
    int bucket = Math.floorMod((flag + ":" + userId).hashCode(), 100);
    return bucket < percent;
}
}

```

What to notice: the flag name is part of the hash input, so different flags select different 10 percent groups instead of always the same unlucky users. `String.hashCode()` is stable across JVMs for a given string, which is what the assignment needs. Raising the value from 10 to 25 only adds users; nobody who had the feature loses it.

Reading flags cheaply

A flag is read on nearly every request, and a Redis round trip each time adds up. Two techniques keep it cheap. First, cache the value inside the JVM for a short time (a second or five, for example with a Caffeine cache), accepting that a change takes that long to spread. Second, for immediate propagation, publish a message on a pub/sub channel such as `ff:changed` whenever a flag is edited, and have each instance drop its local copy when it hears the message. If a message is missed the short local TTL repairs it, so the pub/sub notification is only an accelerator. The multi-level caching chapter in this Part describes the same combination for cache entries.

Keep the flag state small and authoritative somewhere else too (a database or config repository, with history and audit), and use Redis as the fast distribution layer. A hash `ff:all` with one field per flag lets a service fetch every flag with one `HGETALL` at startup and on a timer.

8.15.2 Dynamic configuration

The same pattern serves settings that operators change at runtime: `HSET config:catalog pageSize 24 bannerText "Free shipping"`. Services refresh the hash on a timer or on a change notification. Be disciplined about what lives there. Redis is a fine place for knobs, thresholds and kill switches. It is a poor place for secrets (use a secrets manager), and a dangerous place for anything that must be consistent across instances at one precise instant, because instances will pick up changes at slightly different times.

8.15.3 Flash-sale stock

A flash sale is a stress test for correctness: 50,000 people try to buy 500 items in the same second. The failure everyone fears is **overselling**: selling 503 items when you own 500.

The naive implementation reads the stock, checks it, and writes the new value:

NaiveStock.java

JAVA

```

public boolean buyNaive(String sku) {
    int stock = Integer.parseInt(redis.opsForValue().get("stock:" + sku));
    if (stock > 0) {
        redis.opsForValue().set("stock:" + sku, String.valueOf(stock - 1)); // WRONG
        return true;
    }
    return false;
}
}

```

Two threads both read 1, both pass the check, both write 0, and one item is sold twice. The read, the check and the write are three separate steps, and anything can happen between them.

Fix 1: decrement first, undo on failure

DECR is atomic and returns the new value. Decrement first, and if the result is negative, put it back:

Optimistic atomic decrement

TERMINAL

```
127.0.0.1:6379> SET stock:sku-1001 1
OK
127.0.0.1:6379> DECR stock:sku-1001
(integer) 0
127.0.0.1:6379> DECR stock:sku-1001
(integer) -1
127.0.0.1:6379> INCR stock:sku-1001
(integer) 0
```

A result of **0** or more means "you got one". A negative result means "sold out; give it back". This never oversells, and it is very fast. Its weakness is that the counter briefly shows negative values, and a crash between the **DECR** and the **INCR** leaves the counter short by one. It also cannot take more than one step: reserving and later paying, with expiry of unpaid reservations, is not a single **DECR**.

Fix 2: a Lua script for check and decrement

A script checks and decrements in one atomic unit, so the counter never goes below zero. For flash sales, ShopStream uses a **reservation** model: a buyer who passes the stock check holds the items for a limited time. If they do not pay in time, the items return to stock.

The data per SKU uses a hash tag in the key names so all three keys share one cluster slot: the counter **stock:{sku}**, a hash of reservations **resv:{sku}** (order id to quantity), and a sorted set **resv:exp:{sku}** (order id scored by expiry time).

reserve.lua

LUA

```
-- KEYS[1] = stock counter, KEYS[2] = reservations hash, KEYS[3] = expiry zset
-- ARGV[1] = quantity, ARGV[2] = order id, ARGV[3] = hold time in ms
local qty = tonumber(ARGV[1])
local order = ARGV[2]
local hold_ms = tonumber(ARGV[3])

if redis.call('HEXISTS', KEYS[2], order) == 1 then
    return {0, tonumber(redis.call('GET', KEYS[1]) or '0')}
end
local stock = tonumber(redis.call('GET', KEYS[1]) or '0')
if stock < qty then
    return {-1, stock}
end

local t = redis.call('TIME')
local now = t[1] * 1000 + math.floor(t[2] / 1000)
redis.call('DECRBY', KEYS[1], qty)
redis.call('HSET', KEYS[2], order, qty)
redis.call('ZADD', KEYS[3], now + hold_ms, order)
return {1, stock - qty}
```

The reply is **{code, remaining}**: **1** reserved, **0** this order already holds a reservation (a safe retry, which makes the call idempotent per order id), **-1** insufficient stock. Because the order id is the hash field, a double click or a retried request cannot reserve twice.

Expired reservations return to stock through a second script run by a periodic sweeper (the same shape as the delayed-task promoter):

release_expired.lua

LUA

```
-- KEYS[1] = stock counter, KEYS[2] = reservations hash, KEYS[3] = expiry zset
-- ARGV[1] = batch size
local t = redis.call('TIME')
local now = t[1] * 1000 + math.floor(t[2] / 1000)
local expired = redis.call('ZRANGEBYSCORE', KEYS[3], '-inf', now, 'LIMIT', 0, ARGV[1])
for _, order in ipairs(expired) do
    local qty = redis.call('HGET', KEYS[2], order)
    if qty then
        redis.call('INCRBY', KEYS[1], qty)
        redis.call('HDEL', KEYS[2], order)
    end
end
```

release_expired.lua (continued)

LUA

```

end
redis.call('ZREM', KEYS[3], order)
end
return #expired

```

Payment success removes the reservation without returning stock: `HDEL resv:{sku} orderId` and `ZREM resv:exp:{sku} orderId` (in a small script of its own, or one `MULTI/EXEC`). If payment succeeds *after* the sweeper already released the reservation, the confirmation must check that the reservation still exists and otherwise re-run the reserve step or refuse the order. Always check the return value of the confirm step.

A sale of two items (typical output from the reserve script)

TERMINAL

```

$ redis-cli SET "stock:{sku1}" 2
OK
$ redis-cli --eval reserve.lua "stock:{sku1}" "resv:{sku1}" "resv:exp:{sku1}" , 1 o1 600000
1) (integer) 1
2) (integer) 1
$ redis-cli --eval reserve.lua "stock:{sku1}" "resv:{sku1}" "resv:exp:{sku1}" , 1 o1 600000
1) (integer) 0
2) (integer) 1
$ redis-cli --eval reserve.lua "stock:{sku1}" "resv:{sku1}" "resv:exp:{sku1}" , 1 o2 600000
1) (integer) 1
2) (integer) 0
$ redis-cli --eval reserve.lua "stock:{sku1}" "resv:{sku1}" "resv:exp:{sku1}" , 1 o3 600000
1) (integer) -1
2) (integer) 0

```

Read it line by line: order `o1` reserves one item (1 left); `o1` again is recognised as already reserved; `o2` takes the last item; `o3` is told there is no stock. The `{sku1}` hash tag in all three key names keeps them on the same node in a cluster.

WATCH OUT

Redis is not the system of record for money and stock. The Redis counter is a fast gatekeeper that decides who gets to try. After a failover or restart without enough persistence it may disagree with reality. Keep the authoritative inventory in the database, reconcile periodically, and make the order service refuse orders the database cannot fulfil. A Redis counter that is slightly high is a business problem; one that is slightly low just loses a few sales. Design the failure mode on purpose.

GOTCHA

Hot key A flash sale for one SKU sends all traffic to one key on one node. Redis handles hundreds of thousands of simple operations per second on a node, so this often suffices, but it cannot be spread across a cluster. If one key is too hot, split the stock into `N` sub-counters (`stock:{sku}:1 ... :N`), give each buyer a random shard, and fall back to other shards when theirs is empty.

INTERVIEW ANGLE

How do you prevent overselling in a flash sale? Make check-and-decrement atomic: `DECR` with a negative check, or a Lua script that tests the stock and decrements in one step, never read-then-write. Hold items with a reservation that expires and returns to stock. Make reservation idempotent per order id. Keep the database as the source of truth and reconcile.

KEY POINTS

- Flags and settings are small, read-mostly state: cache them in the JVM briefly and use pub/sub only to speed up propagation.
- Percentage rollouts use a stable hash of flag plus user id.
- Never read-modify-write stock: use `DECR` plus undo, or a Lua script for check, decrement and reservation.
- Reservations with expiry need a sweeper that returns stock; make reserve and confirm idempotent per order.
- Redis gates the traffic; the database remains the system of record. Shard the counter if a single SKU is too hot.

8.16 Real-time messaging patterns

Redis can move messages between application instances in three ways, with three different guarantees. Choosing the wrong one is a classic production incident: a feature built on pub/sub that "works in testing" and quietly drops messages in production.

8.16.1 Pub/Sub for fan-out

Pub/Sub delivers a message to everyone currently subscribed to a channel. It stores nothing: if no subscriber is connected at that moment, the message vanishes. Delivery is at-most-once.

Publish and subscribe (two terminals)

TERMINAL

```
127.0.0.1:6379> SUBSCRIBE chat:room:42
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "chat:room:42"
3) (integer) 1
1) "message"
2) "chat:room:42"
3) "hello everyone"
```

The publisher side (second terminal)

TERMINAL

```
127.0.0.1:6379> PUBLISH chat:room:42 "hello everyone"
(integer) 1
```

PUBLISH returns the number of subscribers that received the message. **PSubscribe** `chat:room:*` subscribes to a pattern. A subscribed connection is dedicated to subscribing (apart from a few commands), so libraries use a separate connection for it. In Redis Cluster, ordinary **PUBLISH** is broadcast to every node over the cluster bus; with many channels and nodes this is wasteful, and **sharded pub/sub** (**SPUBLISH** and **SSUBSCRIBE**, since Redis 7.0) keeps a channel on the node that owns its hash slot.

Good uses: live notifications where missing one is harmless, cache invalidation hints, "something changed, re-read it" signals. Bad uses: anything that must not be lost, such as orders or payments. For those, use streams.

8.16.2 Pub/Sub in Spring

Spring wraps subscriptions in `RedisMessageListenerContainer`:

ChatConfig.java

JAVA

```
package com.shopstream.notification.chat;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.listener.PatternTopic;
import org.springframework.data.redis.listener.RedisMessageListenerContainer;
import org.springframework.messaging.simp.SimpMessagingTemplate;

@Configuration
public class ChatConfig {

    @Bean
    RedisMessageListenerContainer chatContainer(RedisConnectionFactory factory,
                                                SimpMessagingTemplate websocket) {
        RedisMessageListenerContainer container = new RedisMessageListenerContainer();
        container.setConnectionFactory(factory);
        container.addMessageListener((message, pattern) -> {
            String channel = new String(message.getChannel());
            String roomId = channel.substring(channel.lastIndexOf(':') + 1);
            String body = new String(message.getBody());
            websocket.convertAndSend("/topic/rooms." + roomId, body);
        }, new PatternTopic("chat:room:*"));
        return container;
    }
}
```

And the publishing side is one line, `stringRedisTemplate.convertAndSend("chat:room:" + roomId, json)`.

8.16.3 The WebSocket gateway pattern

Why bring Redis in? Because of horizontal scaling. A user connected over WebSocket to instance A cannot receive a message sent by a user connected to instance B, unless the instances talk to each other. The standard shape:

```

user1 --ws--> [ app instance A ] --PUBLISH chat:room:42--> +-----+
                    ^                                     | Redis |
                    | SUBSCRIBE chat:room:*               +-----+
user2 --ws--> [ app instance B ] <--message-----+

```

Every instance subscribes to the channels (or pattern) it needs and forwards each message to its own locally connected sockets. Redis is only the bus between instances; each instance keeps the table of "which sockets are in which room". If an instance restarts, its users reconnect and simply miss messages sent in between. For a chat product that must show history, persist messages in a database or a stream and let a reconnecting client fetch what it missed since its last seen id.

8.16.4 Presence

Who is online? Combine a heartbeat key with a sorted set or set:

- On connect and every 20 seconds: `SET presence:user:42 1 EX 45`. A missing key means offline; a crash needs no cleanup because the key expires.
- For "who is online in room 7": `ZADD room:7:online <now> user:42` on heartbeat, and `ZRANGEBYSCORE room:7:online <now-45s> +inf` to list those seen recently. Trim old entries with `ZREMRANGEBYSCORE`.
- For a "typing..." indicator, publish a short-lived event and do not store it at all.

8.16.5 Streams for events that must not be lost

When a subscriber must be able to catch up after downtime, or when each event should be processed by one worker from a group, use a stream. A stream stores entries with ids, supports consumer groups with acknowledgement, and lets a new consumer read from any past id. The earlier chapter on queues showed the worker side. For notifications, the pattern in ShopStream is:

- `order-service` appends `{orderId, userId, status}` to `stream:orders` with `XADD`.
- `notification-service` reads through consumer group `notifier`, sends the e-mail or push, and acknowledges.
- Optionally, it also publishes to `PUBLISH user:42:events` so a user with an open browser tab sees the update instantly. If that publish is lost, the e-mail path still delivers, and the page shows the new status on next refresh.

That last point is the useful design rule: **use streams for the guaranteed path and pub/sub as an optional instant-delivery optimisation.**

Mechanism	Stored?	Delivery	Consumers	Typical use
Pub/Sub	no	at-most-once	all subscribers	live updates, invalidation hints
Stream, no group	yes (until trimmed)	readers choose position	each reader sees all	event log, audit feed
Stream + group	yes, with pending list	at-least-once	each entry to one worker	notifications, order events
List with <code>BLMOVE</code>	yes	at-least-once with a reaper	one worker per item	simple job queues

Choosing a messaging mechanism.

WATCH OUT

Slow subscribers A subscriber that reads messages slower than they are published causes Redis to buffer them in memory for that connection. If the buffer passes the `client-output-buffer-limit pubsub` limit (the default is 32 MB hard, or 8 MB sustained for 60 seconds), Redis disconnects the client. Keep message handlers fast (hand work to an executor), and keep messages small.

INTERVIEW ANGLE

Pub/Sub or Streams? Pub/Sub is fire-and-forget fan-out with no storage: a subscriber that is offline misses the message. Streams persist entries, support consumer groups with acknowledgement, and allow replay. Use pub/sub for ephemeral live signals and streams whenever a message must be processed even if the consumer is down.

KEY POINTS

- Pub/Sub is at-most-once, unstored fan-out; use sharded pub/sub (**SPUBLISH**) in a cluster at scale.
- For WebSockets across several instances, every instance subscribes to Redis and forwards to its local sockets.
- Presence is a heartbeat key with a short TTL (or a sorted set scored by last-seen time).
- Anything that must not be lost belongs in a stream with a consumer group; pub/sub can add instant delivery on top.
- Keep subscribers fast, or the server disconnects them when the output buffer limit is hit.

8.17 Search, geo and AI: semantic caching and RAG

Redis 8 ships a query engine (full-text search, secondary indexes, vector search), geospatial commands and probabilistic structures in the standard distribution. This chapter shows the three patterns ShopStream-style applications reach for most: secondary-index search over hashes, nearby-store lookup, and vector search for AI features. The data structure chapters introduce these types; here the focus is on how they appear in application designs.

8.17.1 Searching over your own data

With plain keys you can only look things up by their exact name. The query engine lets you index fields of hashes or JSON documents and then query by text, tag, number range or vector similarity, without maintaining your own index structures.

Index product hashes and search them

TERMINAL

```
127.0.0.1:6379> FT.CREATE idx:products ON HASH PREFIX 1 product: \
    SCHEMA name TEXT category TAG price NUMERIC SORTABLE
OK
127.0.0.1:6379> HSET product:1001 name "Trail running shoes" category shoes price 89
(integer) 3
127.0.0.1:6379> FT.SEARCH idx:products "@name:running @category:{shoes}" LIMIT 0 10
1) (integer) 1
2) "product:1001"
3) 1) "name"
    2) "Trail running shoes"
    3) "category"
    4) "shoes"
    5) "price"
    6) "89"
```

The index is maintained automatically: any hash whose key starts with **product:** is indexed when written, and re-indexed when changed. This is useful for catalogues, admin search and filtering. It is not a replacement for a dedicated search engine when you need heavy relevance tuning, large text corpora far beyond memory, or complex analytics. Remember that the data and the index both live in RAM; index only the fields you query.

8.17.2 Nearby stores with geo commands

Geospatial data is stored in a sorted set whose score encodes longitude and latitude. ShopStream's store locator uses **geo:stores**.

Find stores near a customer

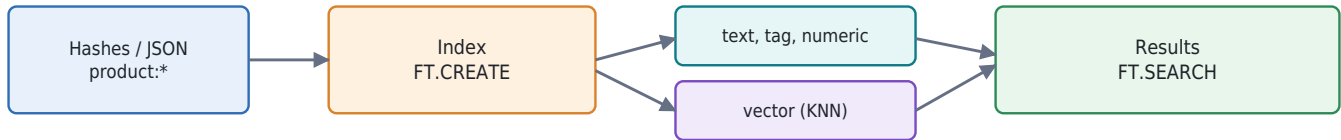
TERMINAL

```
127.0.0.1:6379> GEOADD geo:stores 13.4050 52.5200 store:berlin 11.5820 48.1351 store:munich
(integer) 2
127.0.0.1:6379> GEOSearch geo:stores FROMLONLAT 13.38 52.51 BYRADIUS 50 km ASC WITHDIST
1) 1) "store:berlin"
   2) "2.1637"
```

GEOADD takes longitude first, then latitude. **GEOSearch** finds members within a radius (or box) of a point or of an existing member, optionally sorted by distance. Because the set is an ordinary sorted set, you can also delete a store with **ZREM**. Geo is exact enough for store locators and delivery zones, and handles millions of points; for polygons, routing or map tiles you need a GIS database.

8.17.3 Vector search

AI applications turn text, images or products into **embeddings**: arrays of floating point numbers (for example 384 or 1536 values) where similar meaning gives nearby vectors. **Vector search** finds the K stored vectors nearest to a query vector, which is how you build "similar products", semantic search and retrieval for LLMs.



new keys with the indexed prefix are indexed automatically

Figure 8.9. Vector search: text is embedded into a vector, then the index returns the nearest stored vectors, optionally filtered by tags.

You declare a vector field in the index. **HNSW** is an approximate nearest-neighbour graph (fast, slightly approximate); **FLAT** is exact brute force (fine for small sets).

REDIS

```

FT.CREATE idx:chunks ON HASH PREFIX 1 chunk: SCHEMA \
  text TEXT \
  source TAG \
  embedding VECTOR HNSW 6 TYPE FLOAT32 DIM 384 DISTANCE_METRIC COSINE
  
```

Each chunk hash stores the text and the embedding as a binary blob of 384 little-endian 32-bit floats. A query asks for the nearest neighbours of a query vector, optionally with a pre-filter before the `=>`:

REDIS

```

FT.SEARCH idx:chunks "@source:{handbook}"=>[KNN 3 @embedding $vec AS dist] \
  PARAMS 2 vec "<binary blob>" SORTBY dist LIMIT 0 3 DIALECT 2
  
```

The pre-filter `@source:{handbook}` restricts the candidates, `KNN 3` asks for the three nearest, `AS dist` names the distance field (smaller is closer for cosine distance), and `PARAMS` passes the query vector as a binary blob. Vector queries need **DIALECT 2** or higher.

NOTE

Redis 8 also introduced a beta **vector set** data type with commands such as **VADD** and **VSIM**, a lighter-weight structure for similarity search without declaring an index. It is marked beta, so check the current documentation before relying on it.

8.17.4 Pattern 1: the semantic cache for LLM calls

Calling a large language model is slow and costs money per token. Many user questions are paraphrases of ones you already answered ("how do I return an item" and "what is your returns process"). An exact-match cache misses these; a **semantic cache** matches on meaning.

The flow, per request:

1. Embed the user's question into a vector.
2. Query the cache index for the single nearest stored question (**KNN 1**).
3. If its distance is below a threshold, return the stored answer: a **hit**, in milliseconds, at no model cost.
4. Otherwise call the LLM, return the answer, and store question embedding plus answer in a hash with a TTL.

JAVA

```

SemanticCache.java
package com.shopstream.assistant;

import java.util.Optional;

public class SemanticCache {

    interface Embedder { float[] embed(String text); }
    interface VectorIndex {
        Optional<Hit> nearest(float[] vector);
    }
  
```


SemanticCache.java (continued)

JAVA

```

    void put(String question, float[] vector, String answer, long ttlSeconds);
}
record Hit(String answer, double distance) {}

private static final double MAX_DISTANCE = 0.12; // tune on real data
private static final long TTL_SECONDS = 24 * 3600;

private final Embedder embedder;
private final VectorIndex index;

public SemanticCache(Embedder embedder, VectorIndex index) {
    this.embedder = embedder;
    this.index = index;
}

public String answer(String question, java.util.function.Function<String, String> llm) {
    float[] vector = embedder.embed(question);
    Optional<Hit> hit = index.nearest(vector);
    if (hit.isPresent() && hit.get().distance() <= MAX_DISTANCE) {
        return hit.get().answer();
    }
    String fresh = llm.apply(question);
    index.put(question, vector, fresh, TTL_SECONDS);
    return fresh;
}
}

```

The sketch hides the Redis details behind a `VectorIndex` interface so the logic is visible: `nearest` runs the `FT.SEARCH ... KNN 1` query shown above, and `put` writes a hash and sets its TTL. The threshold is the crucial parameter. Too loose and the cache returns a wrong answer for a different question; too strict and the hit rate collapses. Tune it by labelling a sample of real question pairs, and scope the cache (per tenant, per language, per product) with a tag filter so one customer's answer is never served to another.

WATCH OUT

Semantic caches can return wrong or private answers. Never put user-specific or confidential answers into a shared semantic cache, and expire entries when the underlying documents change. A near match is not an identical question: "refund for order 5" and "refund for order 6" are close in vector space and need different answers. Cache only questions whose answers are generic.

8.17.5 Pattern 2: retrieval-augmented generation (RAG)

An LLM does not know your private documents. **Retrieval-augmented generation** fixes this at query time: retrieve the most relevant passages from your own data and put them in the prompt.

1. **Ingest:** split documents into chunks of a few hundred tokens, embed each chunk, and store `chunk:{id}` hashes with the text, the embedding and metadata (source, version, access group).
2. **Retrieve:** embed the user's question, run a KNN query filtered by the metadata the user is allowed to see, and take the top K chunks.
3. **Generate:** send the question and the retrieved passages to the LLM and ask it to answer from them.

Redis fits because retrieval needs low latency and a metadata filter in the same query. Its limits are the usual ones of an in-memory store: the vectors must fit in RAM (a 384-dimension float32 vector is about 1.5 KB, so a million chunks is on the order of 1.5 GB before graph overhead), and large rebuilds of an index are worth planning.

Spring AI offers a Redis-backed vector store integration, so you can use the framework's `VectorStore` abstraction instead of writing the queries by hand; consult its current reference for the exact dependency and configuration. Keep the embedding model fixed for a given index: vectors from different models are not comparable, so changing models means re-embedding everything.

8.17.6 Pattern 3: agent memory and conversation state

The simplest "memory" for a chat assistant is the recent conversation, stored as a list or a stream per conversation (`chat:{conversationId}`) with a TTL and a length cap, so the next request can rebuild the prompt quickly. Long-term memory (facts about the user) can be stored as short texts with embeddings and retrieved by similarity when relevant. Both are ordinary uses of structures you already know; the AI-specific part is deciding what to keep, what to summarise and what to forget.

REAL WORLD

AI features in ShopStream The shopping assistant uses a semantic cache for generic questions ("what is your return policy"), vector search over product descriptions for "similar items", and a per-conversation list with a 24-hour TTL for chat context. Order-specific answers always bypass the cache.

KEY POINTS

- The query engine indexes hashes or JSON automatically; index only the fields you query, because both data and index live in RAM.
- `GEOADD` and `GEOSEARCH` give store-locator style queries; longitude comes first.
- Vector search finds nearest embeddings with `FT.SEARCH ... KNN`, optionally pre-filtered by tags, and requires `DIALECT 2`.
- A semantic cache returns stored LLM answers for near-identical questions; its distance threshold and scoping decide whether it helps or harms.
- RAG is ingest, retrieve, generate; Redis supplies the low-latency, filterable retrieval step.

8.18 Anti-patterns catalogue

Most Redis outages are not caused by exotic bugs. They come from a short list of avoidable mistakes. This chapter collects them in one place, each with the symptom, the reason, and the fix. Use it as a review checklist for any code or design that touches Redis.

Anti-pattern	Why it hurts	Instead
<code>KEYS *</code> in production	blocks the single command thread	<code>SCAN</code> with <code>COUNT</code>
Huge values or collections	latency spikes, slow replication, memory	split, compress, store elsewhere
Hot key	one node saturated	replicate reads, shard the key, local cache
Only copy, no persistence	restart loses data	persistence or database as source of truth
Unbounded growth	memory fills, eviction or OOM	TTLs, <code>LTRIM</code> , <code>XTRIM</code> , <code>maxmemory</code>
Slow or blocking Lua	stalls every client	short scripts, move work out
Wrong eviction policy	surprise data loss or write errors	match policy to use case
No TTL on cache keys	stale data, memory leak	always TTL, with jitter
N+1 round trips	latency multiplies	pipelining, <code>MGET</code> , scripts
Pub/Sub for critical events	messages lost	streams with groups

The anti-pattern summary.

8.18.1 1. KEYS in production (and other O(N) commands)

`KEYS` pattern walks the whole keyspace in one command. Redis executes commands one at a time, so with ten million keys every other client waits for seconds. The same applies to `SMEMBERS` on a huge set, `HGETALL` on a huge hash, `LRANGE 0 -1` on a long list, `DEL` of a gigantic key, and `FLUSHALL` run at the wrong moment. Use the cursor-based `SCAN`, `SSCAN`, `HSCAN`, `ZSCAN`, which return a little at a time, and `UNLINK` instead of `DEL` for big keys (it frees memory in a background thread).

Iterating safely

TERMINAL

```
127.0.0.1:6379> SCAN 0 MATCH session:* COUNT 100
1) "17"
2) 1) "session:9f2c"
   2) "session:41aa"
```

Repeat with the returned cursor until it is `"0"`. `SCAN` may return an element more than once, and elements added or removed during the scan may or may not appear, so make processing idempotent. In Spring, `redisTemplate.keys("session:*")` issues `KEYS`; use `redisTemplate.scan(ScanOptions.scanOptions().match("session:*").count(100).build())` instead and close the cursor.

8.18.2 2. Huge values and big keys

A single string of 100 MB, or a hash with five million fields, causes trouble in many places: reading it saturates the network and delays other clients, deleting it blocks, replicating or persisting it makes spikes, and in a cluster it unbalances a shard. Aim for values in the kilobytes. Find offenders with `redis-cli --bigkeys` or `--memkeys` and `MEMORY USAGE key`. Fix by splitting a large hash into buckets (`cart:{id}` is fine, `all-carts` is not), storing blobs in object storage with only a reference in Redis, compressing, and paginating collections.

8.18.3 3. Hot keys

One key that receives a large fraction of all traffic (a global counter, a celebrity's profile, the flash-sale SKU) pins one CPU core on one node. More cluster nodes do not help because the key lives on exactly one. Mitigate: a small in-process cache with a short TTL in front of the key, replicated reads from replicas where staleness is acceptable, splitting a counter into N sub-counters and summing, and, for reads, replicating the value under several key names. Redis 8.6 added a `HOTKEYS` command to help you find them.

8.18.4 4. Redis as the only copy

Redis is in memory first. By default it uses RDB snapshots, and replication is asynchronous, so after a crash or a failover the most recent writes can be lost. If the data is also in a database, that is fine: it is a cache. If Redis is the only place the data exists, you need to choose persistence deliberately (AOF with a suitable `appendfsync`, replicas, backups) and still accept a window of possible loss. For money, orders and inventory of record, keep the system of record in a durable database. The persistence part explains the exact guarantees.

8.18.5 5. Unbounded growth

Structures that only ever grow: lists of events, streams without trimming, sets of "seen" ids, sorted sets of every session, per-user history. Memory climbs until `maxmemory` is reached or the process is killed by the operating system. For every structure, ask "what removes old entries?" and answer with a TTL, `LTRIM` for capped lists, `XTRIM MAXLEN ~ N` or `MINID` for streams, `ZREMRANGEBYRANK` for leaderboards, and a monitoring alert on memory growth. Always set `maxmemory`; without it a Redis server will consume memory until the host runs out.

8.18.6 6. Long-running or blocking scripts

A Lua script or a `MULTI/EXEC` block runs without interruption. A loop over a million elements, or a call to a slow function in a script, freezes the server for its duration, and other clients receive `BUSY` errors once the script has run longer than `busy-reply-threshold` (formerly `lua-time-limit`, default 5000 ms). Keep scripts short, bounded and free of unbounded loops. Move heavy computation to the application. Blocking commands such as `BLPOP` block only the calling client, not the server, so they are fine.

8.18.7 7. Wrong eviction policy

The default `maxmemory-policy` is `noeviction`: when memory is full, writes fail with an `OOM` error while reads continue. That is right for a primary store and wrong for a pure cache, which should use `allkeys-lru` or `allkeys-lfu`. The mirror mistake: `allkeys-lru` on an instance that also holds sessions, locks and queues will silently evict them under pressure. The `volatile-*` policies evict only keys that have a TTL, which protects keys without one but turns a missing TTL on cache keys into an out-of-memory error. Use separate Redis instances for caches and for data that must not be evicted.

8.18.8 8. No TTL

Cache entries without a TTL never refresh and never leave. They serve stale data forever and slowly fill memory. Set a TTL when you write, with random jitter so many keys written together do not expire together. Hunt for the omission: periodically check keys without expiry with **SCAN** plus **TTL** (returns **-1** for no expiry) on a sample.

8.18.9 9. N+1 round trips

Every command costs a network round trip, typically a few hundred microseconds inside a data centre. Fetching 200 products in a loop with **GET** is 200 round trips; the same data in one **MGET** or one pipeline is one. The slow loop is invisible in a unit test against localhost and obvious in production.

Pipelining.java

JAVA

```
// Anti-pattern: one round trip per key
for (String id : ids) {
    products.add(redis.opsForValue().get("product:" + id));
}

// Better: one round trip for all keys
List<String> keys = ids.stream().map(id -> "product:" + id).toList();
List<String> products = redis.opsForValue().multiGet(keys);
```

multiGet is **MGET**. For mixed commands use **redis.executePipelined(...)**, which sends them in one batch. Note that **MGET** across keys in different cluster slots needs the client to split the request; the Lettuce and Spring clients do this, at the cost of several round trips.

8.18.10 10. Fire-and-forget pub/sub for critical events

Publishing an "order paid" event with **PUBLISH** means that if the consumer is restarting, slow, or disconnected, the event is gone, with no error anywhere. Use a stream with a consumer group (or a real broker) for events that must be processed; keep pub/sub for ephemeral signals.

8.18.11 Other mistakes worth a checklist line

- **Treating Redis like a relational database:** joins, ad-hoc queries over unindexed data and multi-key constraints do not exist. Model data around access patterns.
- **Using the JDK serializer by default:** **RedisTemplate** serializes objects with Java serialization unless you configure a serializer; the bytes are unreadable from other tools, bloated, and tied to your class versions. Use String or JSON serializers deliberately.
- **Exposing Redis to the internet or leaving it without authentication:** a recurring cause of real breaches. Bind to private interfaces, use ACLs and TLS (see the security chapter).
- **Ignoring connection limits:** a new connection per request exhausts **maxclients**; use a shared client or a pool.
- **Mixing workloads:** one instance for cache, queues, locks and leaderboards means one memory policy for all. Separate by durability and eviction needs.
- **Big MULTI/EXEC batches and WATCH storms on hot keys:** contention causes retries that amplify load; prefer a script.
- **No monitoring:** run with **INFO** metrics, latency tracking and slow log from day one (see the operations part).

INTERVIEW ANGLE

What are common Redis mistakes in production? Using **KEYS** or other O(N) commands on big data, storing huge values or hot keys, relying on Redis as the only copy of important data, forgetting TTLs and **maxmemory** so memory grows unbounded, picking an eviction policy that deletes the wrong things, running long Lua scripts, making one round trip per key instead of pipelining, and using pub/sub for events that must not be lost.

KEY POINTS

- Replace **KEYS** and other full scans with cursor-based **SCAN** family commands and **UNLINK**; keep values small and collections bounded.
- Hot keys cannot be fixed by adding nodes; cache locally, split the key, or read from replicas.
- Redis is fast storage with asynchronous replication, not automatically a system of record; decide durability on purpose.
- Every structure needs a way to shrink: TTL, trimming, **maxmemory** and a matching eviction policy.
- Batch commands with **MGET** and pipelines; keep Lua short; use streams, not pub/sub, for critical events.

PART 9

Where Redis Is Used

Use cases, trade-offs and reference architectures, described by category of system rather than by company.

In this part

- 9.1 The Use-Case Map
- 9.2 E-Commerce
- 9.3 Gaming and Social Applications
- 9.4 Fintech, Ride-Hailing and Logistics
- 9.5 Real-Time Analytics and IoT
- 9.6 Microservices Glue
- 9.7 AI Applications
- 9.8 Decision Guide: When to Use Redis and When Not To
- 9.9 Reference Architectures for ShopStream

9.1 The Use-Case Map

You now know what Redis can do. The harder question, the one that decides whether a project succeeds, is when Redis is the right tool and what role it should play. This Part walks through the industries and system types where Redis shows up, and then gives you a decision guide for the cases where it should not.

A note on sources. Redis is widely deployed, but this book does not make claims about what any named company runs, because such claims go stale or cannot be verified. Instead every example here is phrased as "a typical e-commerce site" or "a ride-hailing backend". The designs are the ones that follow from Redis's properties, and you can check each one against the chapters of Parts 3 to 8.

9.1.1 Redis plays four roles

Almost every real use of Redis falls into one of four roles. Naming the role first tells you how much durability and consistency you must demand.

Role	What it means	Consequence
Cache	A faster copy of data that lives elsewhere	Losing it is fine; the source of truth rebuilds it
Ephemeral state	Data that is only meaningful for a short time (sessions, locks, rate counters)	Losing it hurts a little; TTLs bound its life
Derived or hot data	Data computed from a durable source and kept for speed (leaderboards, counters, feeds)	Rebuildable, but rebuilding may be slow, so persistence helps
Messaging and coordination	Streams, pub/sub, queues, locks between processes	Delivery guarantees matter; choose the primitive carefully

The four roles of Redis.

Redis as the only copy of important business data (the "system of record") is possible, with AOF and replication, but it is the role that needs the most care and is discussed honestly in the decision chapter later in this Part.

9.1.2 The map

The table below is the reference for the rest of the Part. Read it as: for this use case, reach for these structures, using this pattern, and expect this consistency need and lifetime. Sizes and TTLs are typical starting points, not rules.

Use case	Structures	Pattern	Consistency need	Typical TTL / size
Page or object cache	String, hash, JSON	Cache-aside	Eventual; stale OK	Seconds to hours
Session store	Hash	Key per session, sliding TTL	Read-your-writes	15 to 60 min
Shopping cart	Hash	Hash per user	Per-user ordering	Days; small

Use case	Structures	Pattern	Consistency need	Typical TTL / size
Rate limiting	String counter, sorted set	Fixed or sliding window, Lua	Atomic check-and-increment	Window length
Leaderboard	Sorted set	ZADD, ZREVRANGE	Eventual OK	Per season; up to millions
Flash-sale stock	String counter	Lua atomic decrement	Strict: never oversell	Sale duration
Distributed lock	String	SET NX PX, token check	Safety needs care	Seconds
Idempotency key	String	SET NX EX	Atomic first-writer-wins	Hours to days
Job queue	List, stream	Consumer group, ack	At-least-once	Until processed
Pub/sub fan-out	Pub/sub channel	Fire and forget	At-most-once	None
Unique counts	HyperLogLog, bitmap	PFADD, SETBIT	Approximate OK	Day or month
Geo search	Geo (sorted set)	GEOSEARCH	Eventual	Live positions, short
Time series	Time series, stream	Append and range query	Eventual	Retention window
Semantic cache, RAG	Search index with vectors	KNN query	Approximate by design	Hours to days

Use case to structure map.

Notice what the table says about durability: most rows tolerate loss, and the rows that do not (flash-sale stock, idempotency keys, queues) are exactly the ones where you must design the recovery path. Keep that in mind in every chapter that follows.

KEY POINTS

- Name the role (cache, ephemeral state, derived data, messaging) before choosing structures or persistence.
- Most Redis use cases tolerate loss; the ones that do not need an explicit recovery design.
- Map each use case to a structure and a pattern, then to a consistency need and a TTL.
- This book describes categories of systems, not claims about named companies.

9.2 E-Commerce

E-commerce is the home turf of our running example, ShopStream. A shop has a read-heavy catalogue, a write-heavy cart, bursty traffic during sales and strict rules around money and stock. Redis fits several of these jobs at once, and a typical site uses it in five places.

9.2.1 Catalogue cache

Product pages are read thousands of times for every one change. The database can serve them, but at peak the same few hundred hot products dominate the load. Cache-aside (Part 8) puts a copy in Redis: the service reads `product:{id}` first, falls back to the database on a miss, and stores the result with a TTL.

Two details matter in e-commerce specifically. First, price and stock must not be stale for long, so catalogue descriptions get a long TTL (hours) while price and availability are either short-lived (seconds) or fetched separately. Splitting the cached object into a stable part and a volatile part is a common design. Second, add a small random jitter to TTLs so that products cached in the same minute do not all expire in the same second.

REDIS

```
# stable product data, cached for hours
HSET product:1001 name "Trail Shoe" brand "Strider" category "shoes"
EXPIRE product:1001 14400
# volatile data, cached for seconds
SET product:1001:price 5999 EX 30
```

9.2.2 Carts

A cart belongs to one user, is updated item by item and must feel instant. A hash per user is the natural shape: the field is the SKU, the value is the quantity.

A cart as a hash

TERMINAL

```
127.0.0.1:6379> HINCRBY cart:42 sku:1001 1
(integer) 1
127.0.0.1:6379> HINCRBY cart:42 sku:2002 2
(integer) 2
127.0.0.1:6379> HGETALL cart:42
1) "sku:1001"
2) "1"
3) "sku:2002"
4) "2"
127.0.0.1:6379> EXPIRE cart:42 604800
(integer) 1
```

Each **HINCRBY** is atomic, so two browser tabs adding the same item do not lose updates. The **EXPIRE** gives abandoned carts a lifetime of seven days; refresh it whenever the cart changes. Anonymous carts use a random cart id as the key suffix and are merged into the user cart at login (read both hashes, add quantities, delete the anonymous one).

GOTCHA

A cart in Redis is not an order. Treat the cart as convenient, mostly durable state. The moment the user checks out, write the order to your database (the system of record) and only then clear the cart. Do not let an eviction or a failover decide whether an order existed.

9.2.3 A cart service in Java

The same cart with Spring Data Redis. The service owns the key naming and the TTL, so no other class needs to know how a cart is stored.

CartService.java

JAVA

```
package com.shopstream.cart;

import java.time.Duration;
import java.util.Map;
import java.util.stream.Collectors;

import org.springframework.data.redis.core.HashOperations;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Service;

@Service
public class CartService {

    private static final Duration CART_TTL = Duration.ofDays(7);

    private final StringRedisTemplate redis;

    public CartService(StringRedisTemplate redis) {
        this.redis = redis;
    }

    private String key(long userId) {
        return "cart:" + userId;
    }

    public void addItem(long userId, String sku, int quantity) {
        HashOperations<String, String, String> ops = redis.opsForHash();
        ops.increment(key(userId), sku, quantity);
        redis.expire(key(userId), CART_TTL);
    }

    public Map<String, Integer> items(long userId) {
        HashOperations<String, String, String> ops = redis.opsForHash();
        return ops.entries(key(userId)).entrySet().stream()
            .collect(Collectors.toMap(Map.Entry::getKey,
                e -> Integer.parseInt(e.getValue())));
    }
}
```

CartService.java (continued)

JAVA

```

public void clear(long userId) {
    redis.delete(key(userId));
}
}

```

What to notice: `increment` maps to `HINCRBY`, so concurrent adds are safe without any Java locking. `expire` is called on every change, which gives the sliding seven-day lifetime. The two calls are two round trips; that is fine here, and a pipeline or a small Lua script can merge them if the cart path ever becomes hot. The service stores strings only, so the cart is readable from `redis-cli` while debugging.

9.2.4 Sessions

Web sessions stored in Redis let any application instance serve any request, which is what makes horizontal scaling and rolling deploys painless. A session is a hash under `session:{sid}` with a TTL that you slide forward on each request. Spring Session (see the Spring Part) does this for you. Keep sessions small: an id, a user id, and a few flags. Large objects in sessions are copied on every request and make failover painful.

9.2.5 Flash sales

A flash sale is the hardest e-commerce load: ten thousand people want the last two hundred units in the same few seconds. The database cannot arbitrate this cheaply, and a read-then-write in application code oversells. Redis solves it because a Lua script runs atomically on the single command thread.

stock_reserve.lua

LUA

```

-- KEYS[1] = stock:{sku}, ARGV[1] = quantity wanted
local stock = tonumber(redis.call('GET', KEYS[1]) or '0')
local want = tonumber(ARGV[1])
if stock >= want then
    redis.call('DECRBY', KEYS[1], want)
    return 1
end
return 0

```

What to notice: the check and the decrement are one atomic step, so stock can never go negative. The script returns 1 or 0 and the caller decides what to tell the user. The full production version, with reservation TTLs, an order stream and idempotency, is built as a project later in this book.

The honest limits: the Redis counter is a fast gate, not the ledger. After a successful reservation you write the order durably, and a periodic job reconciles the counter with the database. If Redis restarts and loses a few seconds of decrements (with default `appendfsync everysec`), the reconciliation corrects the counter. Decide in advance whether a small oversell followed by an apology, or a small undersell, is acceptable for your business; both are cheaper than not selling.

9.2.6 Recommendations and "recently viewed"

Simple recommendations do not need machine learning infrastructure at request time. A list holds the last N products a user viewed, capped with `LTRIM`. A sorted set holds the bestsellers per category, updated with `ZINCRBY` on every purchase. "Customers also bought" is often computed offline and written as a set or a sorted set per product, so the page only does one `ZREVRANGE`. Redis serves the result; the offline job computes it. Vector search (see the AI chapter in this Part) adds similarity-based recommendations when you need them.

REDIS

```

LPUSH recent:42 sku:1001
LTRIM recent:42 0 19
ZINCRBY lb:bestsellers:shoes 1 sku:1001
ZREVRANGE lb:bestsellers:shoes 0 9 WITHSCORES

```

REAL WORLD

On a typical sale day The catalogue cache absorbs most reads, the cart hash absorbs write bursts, and the stock counter is protected by a script. The database sees only misses, checkouts and the reconciliation job. That asymmetry, not raw speed, is why the architecture survives the spike.

KEY POINTS

- Cache the catalogue with TTL jitter; split stable data from volatile price and stock.
- A hash per user is the cart; **HINCRBY** is atomic and the cart gets a sliding TTL.
- Sessions in Redis make app instances stateless; keep them small.
- Flash-sale stock is decremented by a Lua script; the database remains the ledger.
- Orders are written to the database, never only to Redis.

9.3 Gaming and Social Applications

Games and social products have two things in common: huge numbers of small, frequently changing facts (a score, a status, a position) and users who notice latency immediately. These are in-memory data store workloads.

9.3.1 Leaderboards

The leaderboard is the classic. A sorted set keeps members ordered by score, and the operations players care about are all logarithmic: add or update a score, get the top N, get a player's rank, get the players around a rank.

```

A game leaderboard
127.0.0.1:6379> ZADD lb:season:7 4200 "asha" 3900 "ben" 5100 "chen"
(integer) 3
127.0.0.1:6379> ZINCRBY lb:season:7 200 "ben"
"4100"
127.0.0.1:6379> ZREVRANGE lb:season:7 0 2 WITHSCORES
1) "chen"
2) "5100"
3) "asha"
4) "4200"
5) "ben"
6) "4100"
127.0.0.1:6379> ZREVRANK lb:season:7 "ben"
(integer) 2

```

Ties are broken by lexicographic order of the member, which may not be what your game wants; if first-to-reach wins, encode a timestamp into the score. To show "ten players around me", take the player's rank with **ZREVRANK** and then call **ZREVRANGE** for rank minus five to rank plus five. At very large scale (tens of millions of players), keep the full ranking in one sorted set if memory allows, or shard by region or league and merge only for the global top N.

9.3.2 Matchmaking

Matching players of similar skill is a range query: a sorted set of waiting players scored by rating lets you ask **ZRANGEBYSCORE** for everyone within a window around your rating, then widen the window the longer a player has waited. The match is claimed atomically (a Lua script that removes both players or neither), so one player cannot land in two matches. Redis is the waiting room; the game server owns the match itself.

9.3.3 Presence and "who is online"

Presence is a heartbeat problem. Every client refreshes **presence:{userId}** with a short TTL (say 60 seconds) every 20 to 30 seconds. If the key exists the user is online; if the client crashes the key simply expires, with no cleanup job. For "how many users are online right now" a sorted set scored by last-seen timestamp works: **ZADD online <now> <user>**, then **ZCOUNT online <now-60> +inf**, and **ZREMRANGEBYSCORE** to trim old entries. For a friends-list view, ask for the presence keys of friends in one pipelined batch rather than one round trip per friend.

9.3.4 Feeds and timelines

A social feed can be built two ways, and Redis supports both. In **fan-out on write**, publishing a post pushes its id into the timeline list or sorted set of every follower (**ZADD timeline:{follower} <timestamp> <postId>**), so reading a feed is one cheap **ZREVRANGE**. In **fan-out on read**, the reader fetches recent posts of everyone followed and merges them at request time. Fan-out on write makes reads fast and writes expensive, which hurts for accounts with millions of followers. Typical systems mix the two: push for normal accounts, pull for very popular ones, merged at read time. Cap each timeline (**ZREMRANGEBYRANK timeline:42 0 -1001** keeps the newest 1000) so memory stays bounded. The posts themselves live in your database; Redis holds ids.

9.3.5 Counters and reactions

Likes, views and follower counts are **INCR** on a string or a field in a hash. The write rate on a viral post can be very high, and the exact count is rarely critical. A common design counts in Redis and flushes totals to the database every few seconds, accepting that a crash loses a few seconds of increments. Whether to also keep a set of who liked (to prevent double likes) depends on the product: a set per post is exact but costs memory per user; a Bloom filter is approximate and compact.

INTERVIEW ANGLE

How would you build a global leaderboard for 50 million players? Use a sorted set keyed by season, **ZADD** or **ZINCRBY** on score change, **ZREVRANGE** for the top N and **ZREVRANK** for a player's rank, all $O(\log N)$. Estimate memory first (tens of bytes per member plus the member string), shard by region if one node cannot hold it, and persist with AOF or rebuild from the durable score table. For the "around me" view use the rank then a range query.

KEY POINTS

- Sorted sets give you rank, top N and around-me queries in logarithmic time.
- Presence is a TTL key refreshed by heartbeat; expiry is the cleanup.
- Feeds: push ids to followers for normal accounts, pull for celebrities, cap every timeline.
- Count in Redis, flush to the database in batches, and accept bounded loss for non-critical counters.

9.4 Fintech, Ride-Hailing and Logistics

These industries move money, vehicles and parcels. Mistakes are expensive, so the first rule is blunt: **Redis is not the system of record for balances, payments or shipments**. The ledger lives in a transactional database with strong guarantees. Redis earns its place around the ledger, in the parts that need speed, expiry and coordination: rate limits, idempotency, locks, risk counters and live positions.

9.4.1 Rate limits and abuse control

Payment and login endpoints are attacked and mis-called constantly. A rate limiter in front of them, built on atomic counters or a token bucket in Lua (see the rate limiting chapter in Part 8), costs microseconds per call. Limits are layered: per IP at the gateway, per account for sensitive actions such as password attempts, per API key for partners. A fail-open or fail-closed decision is part of the design: if Redis is down, should payments be blocked or allowed? Most teams fail closed for login and fail open for read-only APIs, and write the choice down.

9.4.2 Idempotency keys

Networks retry. A mobile app that times out sends the payment request again, and the customer must not be charged twice. The client sends an **Idempotency-Key** header; the server claims it atomically before doing the work.

REDIS

```
# claim the key; only one caller wins
SET idem:pay:7f3a NX EX 86400 "in-progress"
# after the payment is stored durably, record the response for replays
SET idem:pay:7f3a "{\"status\":201,\"paymentId\":\"p-889\"}" XX EX 86400
```

If **SET ... NX** returns **OK** you own the request. If it returns **(nil)**, either the first attempt is still running (answer 409 or make the caller wait) or it finished and you replay the stored response. The key's TTL bounds how long a retry is recognised. The database should still have a unique constraint on the business operation, because the Redis key can be lost in a failover; Redis makes duplicates rare and cheap to detect, the constraint makes them impossible.

9.4.3 Locks around money

Locks serialise work on one entity, for example "only one settlement run per merchant". The simple pattern is **SET lock:merchant:9 <token> NX PX 30000** and a Lua compare-and-delete to release. Be honest about what that gives you: mutual exclusion that is correct while the lock holder finishes within the TTL and Redis does not fail over at the wrong moment. A paused process (a long garbage collection, a network stall) can still act after its lock expired. For anything where a double action is harmful, the real protection is a **fencing token**: the lock service hands out an increasing number and the database rejects writes that carry an older one. Treat the Redis lock as an efficiency tool and let the database enforce correctness.

9.4.4 Risk and fraud counters

Fraud rules are often counting questions: how many cards has this device tried in ten minutes, how many failed logins per account today, how much has this user transferred in the last hour. Sliding windows over a sorted set (**ZADD** with timestamps, **ZREMRANGEBYSCORE** to trim, **ZCARD** to count) or small time-bucketed counters answer them in one round trip, and the rule engine reads them while a transaction is being authorised. These counters are derived data: if lost, the system is briefly less strict, not wrong about money.

9.4.5 Live positions and geo

A ride-hailing backend asks the same question all day: which available drivers are within two kilometres of this rider? The geo commands store positions in a sorted set and answer radius queries.

Nearby drivers

TERMINAL

```
127.0.0.1:6379> GEOADD drivers:available 77.5946 12.9716 "d:17" 77.6101 12.9352 "d:21"
(integer) 2
127.0.0.1:6379> GEOSearch drivers:available FROMLONLAT 77.5900 12.9700 BYRADIUS 3 km ASC
1) "d:17"
```

Drivers send a position every few seconds, so the write rate is high but each write is tiny. Remove a driver from the set when they accept a ride (**ZREM**), and set a companion key with a TTL so that drivers whose app died drop out of the available pool. Logistics systems do the same with vehicles and parcels: the latest position lives in Redis, the history is appended to a stream or time series and then to the warehouse, and the delivery status of record lives in the database. In one large region the geo set can get hot; shard by city or by cell id so that one key does not carry the whole country.

WATCH OUT

Money in a cache If an interviewer or a design review asks "where is the balance stored?", the answer must never be only "in Redis". Redis may hold a cached balance for display, with the ledger as the truth. A cached copy used to authorise a debit needs a compare against the ledger.

KEY POINTS

- Redis surrounds the ledger (limits, idempotency, locks, risk counters, live positions); it is not the ledger.
- Idempotency keys use **SET NX EX** and a stored response; a database constraint is the backstop.
- A lock is an efficiency tool; fencing tokens and database constraints give correctness.
- Decide fail-open or fail-closed per endpoint before the outage, not during it.
- Geo sets answer "who is near me" fast; shard them when one key gets hot.

9.5 Real-Time Analytics and IoT

Analytics workloads count, aggregate and look at the recent past. Redis suits the real-time edge of this work: what happened in the last minute, hour or day, while the long-term history goes to a column store or data warehouse.

9.5.1 Counting things cheaply

Three structures cover most "how many" questions.

Question	Structure	Cost	Accuracy
Page views	INCR on a string per page and time bucket	1 key per bucket	Exact
Unique visitors per day	HyperLogLog (PFADD , PFCOUNT)	About 12 KB per key at most	Approximate (standard error $\approx 0.81\%$)
Did user X visit today	Bitmap (SETBIT , GETBIT), bit index = user number	1 bit per user id	Exact

Counting structures.

Daily active users from a bitmap: **SETBIT** `dau:2026-10-09 <userNumber> 1`, then **BITCOUNT** `dau:2026-10-09`. Bitmaps for several days combine with **BITOP AND** or **OR** to answer "active every day this week". This needs dense, small integer user ids; with sparse ids a bitmap wastes memory and a HyperLogLog or set is better.

Unique visitors with HyperLogLog

TERMINAL

```
127.0.0.1:6379> PFADD uv:2026-10-09 "u1" "u2" "u3"
(integer) 1
127.0.0.1:6379> PFADD uv:2026-10-09 "u2"
(integer) 0
127.0.0.1:6379> PFCOUNT uv:2026-10-09
(integer) 3
```

9.5.2 Time series

Redis 8 includes a time series data type (formerly a module). It stores samples of (timestamp, value) per key with a retention period, and supports range queries and aggregation, for example the average per five minutes.

REDIS

```
TS.CREATE sensor:7:temp RETENTION 86400000 LABELS site plant1 kind temperature
TS.ADD sensor:7:temp * 21.4
TS.RANGE sensor:7:temp - + AGGREGATION avg 300000
```

Here `*` means "use the server time", `RETENTION` is in milliseconds (one day), and the aggregation bucket is 300000 ms, five minutes. Labels let you query many series at once (`TS.MGET`, `TS.MRANGE` with a filter such as `kind=temperature`). Compaction rules can roll fine data into coarser series automatically. Check the commands page for the exact options available in your version before relying on one.

9.5.3 Ingesting events with streams

Sensors, clickstreams and application logs arrive as a firehose. A stream is an append-only log in Redis, and consumer groups let several workers share the processing with acknowledgements (see the streams chapters in Parts 3 and 8). A common shape is:

```
devices → gateway → XADD events:telemetry MAXLEN ~ 1000000
                        |
                        +→ group "aggregator" → TS.ADD / HINCRBY (live view)
                        +→ group "archiver"   → batch write to warehouse
```

Two independent groups read the same stream: one maintains the live aggregates in Redis for dashboards, the other writes batches to long-term storage. Capping the stream with `MAXLEN ~` bounds memory; size the cap for how far behind your slowest consumer may fall, because trimmed entries are gone. If the data must be kept for days or replayed at will, or the volume is far beyond one node's memory, a log system built for that (such as Kafka, discussed in the next chapter) is the better buffer, with Redis serving the hot aggregates.

GOTCHA

Memory is the budget Redis keeps everything in RAM. Estimate before you commit: a day of one-second samples for 10,000 sensors is 864 million points. Aggregate, downsample, set retention, and keep raw data elsewhere.

KEY POINTS

- Use `INCR` for exact counts, HyperLogLog for approximate unique counts, bitmaps for dense yes/no per user.
- Redis time series gives retention, labels and aggregation for recent data.
- Streams ingest events and let independent consumer groups build live views and archives.
- Bound everything (retention, `MAXLEN`) because RAM is finite; send long history to a warehouse.

9.6 Microservices Glue

In a system of many small services, Redis often becomes shared infrastructure: a place where independent processes meet. That is useful and also dangerous, because a shared Redis is a shared dependency. This chapter covers the legitimate uses and the failure modes.

9.6.1 Shared cache and sessions

Several instances of a service, or several services, can share one cache so that a value computed by one is available to all. Per-instance in-memory caches (Caffeine) are faster but each holds its own copy and drifts out of sync. A two-level cache (a small local cache in front of Redis, invalidated by Redis messages or client-side caching tracking) combines both. Sessions and short-lived tokens follow the same logic: store them centrally so any instance can validate them.

9.6.2 Configuration and feature flags

Feature flags are keys: `ff:new-checkout` with a value of `on`, `off` or a percentage. Services read them with a short local cache, and pub/sub or keyspace tracking tells instances to refresh. Redis is fine as the fast read path for flags; keep the authoritative flag definitions, audit trail and approval workflow in a proper config system and have it write to Redis.

9.6.3 Service discovery: use with caution

It is tempting to register service instances as keys with TTL heartbeats, and it works for small setups. In production, prefer the platform's own discovery (Kubernetes services, a service mesh, or a dedicated registry): they handle health checks, load balancing and consistency better than a hand-rolled Redis registry, and they do not make your cache a single point of failure for every call between services.

9.6.4 Messaging between services

Redis offers three ways to move messages, and they differ in what happens when a consumer is away.

Primitive	Delivery	Persistence	Use for
Pub/sub	At-most-once, to whoever is connected	None	Cache invalidation, live notifications, presence
List with LMOVE	At-least-once with a processing list	In the keyspace	Simple work queues
Stream + consumer group	At-least-once with ack and replay of pending	Yes, with trimming	Event processing, order events, background jobs

Messaging options inside Redis.

Use pub/sub only for messages you can afford to lose. For order events that a notification service must not miss, use a stream with a consumer group, and design the consumer to be idempotent because at-least-once delivery means duplicates.

9.6.5 Redis and Kafka: complements, not rivals

A question every architect gets: "we have Kafka, do we need Redis?" They solve different problems. Kafka is a distributed, replicated, disk-based log built for high-throughput, durable event streaming with long retention and replay by many independent consumers. Redis is an in-memory data store built for sub-millisecond access to current state and small, fast coordination primitives. A common combination: Kafka carries the durable stream of business events between services; consumers build read-optimised views of that stream in Redis (the current price of each product, the live leaderboard, the session state) for fast lookups. Redis streams cover smaller, simpler pipelines where one tool is easier to run than two, with the caveat that capacity is bounded by memory.

9.6.6 API gateway rate limiting

The gateway is the natural place for coarse limits per client. It keeps counters in Redis so that all gateway instances share the same count (a local counter per instance would multiply the allowed rate by the instance count). Return **429 Too Many Requests** with a **Retry-After** header, and keep the Lua script small so that gateway latency stays low.

REAL WORLD

Blast radius If one Redis serves the cache, the sessions, the rate limiter and the queue, a memory spike in the cache can evict sessions and block logins. Production setups usually run separate instances (or at least separate eviction policies) for cache data, which may be evicted, and for state that must not be: use **allkeys-lru** on the cache and **noeviction** on the queue and session instance.

INTERVIEW ANGLE

Why not use Redis pub/sub as the event bus between microservices? Pub/sub is at-most-once: a subscriber that is down, slow or reconnecting misses messages and nothing is stored. It is excellent for invalidation and live signals, and wrong for events that must be processed. Use streams with consumer groups or a durable broker, with idempotent consumers.

KEY POINTS

- Share caches and sessions through Redis; add a local cache layer when latency demands it.
- Prefer platform service discovery over a hand-built Redis registry.
- Pub/sub is lossy; streams with consumer groups give at-least-once delivery.
- Kafka is the durable event backbone, Redis the fast current-state store; they complement each other.
- Isolate workloads (cache vs state) into separate instances to limit blast radius.

9.7 AI Applications

Applications built on large language models (LLMs) have three recurring needs: remembering things cheaply, finding relevant information by meaning, and avoiding repeated expensive model calls. Redis 8 includes a query engine with vector search in the standard distribution, which makes it a practical single place for all three. This chapter stays close to what the features do and avoids performance claims; benchmark with your own data and embedding model.

9.7.1 Vectors in two minutes

An **embedding** is a list of numbers (a vector, for example 768 or 1536 floats) produced by a model so that texts with similar meaning land close together. **Vector search** finds the stored vectors nearest to a query vector. Exact search compares against everything (the **FLAT** index type); approximate search trades a little accuracy for speed (the **HNSW** index type). Closeness is measured by a distance metric such as **COSINE**, **L2** or **IP** (inner product). Redis stores the vector as a field of a hash or JSON document and the search index makes it queryable.

REDIS

```
FT.CREATE idx:docs ON HASH PREFIX 1 doc: SCHEMA
  title TEXT
  tenant TAG
  embedding VECTOR HNSW 6 TYPE FLOAT32 DIM 768 DISTANCE_METRIC COSINE
```

The **6** after **HNSW** is the number of attribute tokens that follow (three name-value pairs). Documents under the prefix **doc:** are indexed automatically when written. A query asks for the K nearest vectors, optionally filtered by other fields:

REDIS

```
FT.SEARCH idx:docs "(@tenant:{acme})=>[KNN 5 @embedding $vec AS dist]"
PARAMS 2 vec "<binary FLOAT32 blob>" SORTBY dist RETURN 2 title dist DIALECT 2
```

The query vector is passed as the raw bytes of the float array (little-endian float32), not as text. In Java you build it with a **ByteBuffer**; client libraries and Spring AI provide helpers. Filtering on tags while searching by vector is how multi-tenant applications keep one customer's documents out of another's results.

9.7.2 Semantic cache

LLM calls are slow and cost money per token. Users ask the same question in many wordings: "how do I reset my password" and "I forgot my password, what now". An ordinary cache keyed by exact text misses these. A **semantic cache** embeds the incoming prompt, searches for a stored prompt within a distance threshold, and returns the stored answer on a hit.

```
prompt → embed → KNN 1 in Redis — distance ≤ threshold → return cached answer
                        |
                        | otherwise → call LLM → store prompt vector + answer (TTL)
```

Design points that decide whether it works:

- **The threshold is a product decision.** Too loose and users get an answer to a different question; too strict and the cache never hits. Tune it on real traffic and log near-misses.
- **Scope the cache.** Include tenant, language and model version as filter fields; a cached answer is only valid in the context that produced it.
- **Set a TTL** so stale answers age out, and delete entries when the underlying knowledge changes.
- Do not cache personalised or sensitive answers.

9.7.3 Retrieval-augmented generation (RAG)

A model only knows its training data. RAG gives it your data at question time: split your documents into chunks, embed and store each chunk in Redis, and on each question embed the question, fetch the K nearest chunks, and pass them to the model as context. Redis plays the retrieval role. Ingestion (chunking, embedding, refreshing) is a batch or streaming job you write, and quality depends more on chunking and the embedding model than on the database.

Redis also supports combining text relevance with vector similarity. Release 8.4 added **FT.HYBRID**, a hybrid search command that combines the two result lists (Reciprocal Rank Fusion is one of the supported ways). Hybrid search helps when exact terms matter, such as product codes or error numbers, which embeddings alone tend to blur. Check the command page for the options available in your version.

9.7.4 Agent memory and sessions

An AI agent needs working state: the conversation so far, tool results, user preferences. This is session data again, and Redis handles it as it handles any session:

- **Short-term memory:** a list or stream per conversation (`chat:{id}`) holding recent turns, trimmed to a window, with a TTL so abandoned chats disappear.
- **Long-term memory:** facts and summaries stored with embeddings so the agent can recall relevant ones by meaning, scoped per user with a tag filter.
- **Coordination:** locks, queues and streams to run tool calls and background tasks reliably.

Redis 8 also introduced a beta vector set data type designed for storing and querying vectors directly; its commands (prefixed **V**, such as **VADD** and **VSIM**) are young, so confirm their current status in the documentation before building on them.

WATCH OUT

Memory cost and embeddings Each vector costs its dimensions × 4 bytes for float32 before index overhead. One million 768-dimension vectors is roughly 3 GB of raw vector data (1,000,000 × 768 × 4 bytes) plus the HNSW graph and your other fields. Estimate, then test with a realistic sample. Some index options reduce memory by compressing vectors, with an accuracy cost.

GOTCHA

Changing the embedding model Vectors from different models are not comparable, and the dimension is fixed at index creation. A model change means re-embedding everything and building a new index. Store the model name in the document and create the new index alongside the old one for a smooth switch.

KEY POINTS

- Vector search finds nearest embeddings; it is built into Redis 8 (**FT.CREATE** with a **VECTOR** field, **FT.SEARCH** with **KNN**).
- A semantic cache returns stored LLM answers for similar prompts; the distance threshold and the scoping filters make or break it.
- RAG uses Redis as the retrieval store; quality depends mostly on chunking and the embedding model.
- Agent memory is session data plus embeddings; keep it scoped, trimmed and expiring.
- Budget memory (dimensions × 4 bytes per vector) and plan for model changes.

9.8 Decision Guide: When to Use Redis and When Not To

Every tool is wrong for some job. This chapter gives you a checklist for deciding, an honest list of cases where Redis is the wrong choice, and a table of alternatives. It is also what a design review or an interview wants to hear: not enthusiasm, but reasoning.

9.8.1 Five questions before you put data in Redis

1. **How big is the data, now and in a year?** Everything lives in RAM, and RAM costs more per gigabyte than disk. A working set of a few gigabytes to a few hundred is comfortable; terabytes of cold data are not. Multiply by overhead (typically well above the raw payload) and leave headroom for forks and replication buffers.
2. **What happens if this data is lost?** If the answer is "we rebuild it", Redis is easy. If the answer is "we lose money or legal records", Redis can be part of the design only with a durable source of truth behind it.
3. **What consistency do you need?** Single commands and scripts are atomic on one node. Replication is asynchronous, so after a failover the last few writes may be gone, and in a cluster, multi-key operations are limited to keys in one slot. If you need multi-row transactions with rollback, constraints and joins, you need a relational database.
4. **How will you query it?** Redis is excellent when you know the key or the range (by key, score, time, location, vector). It is poor at ad hoc questions across arbitrary fields (the query engine helps, but it is an index you maintain, not a general SQL engine).
5. **Can you operate it?** Memory sizing, persistence settings, failover and monitoring are your job, or you pay a managed service to do them.

9.8.2 Worked example: will it fit?

Question 2 above asks about size, so do the arithmetic once. ShopStream expects 2 million active sessions, 500,000 catalogue entries and 3 million carts at peak. The numbers below are illustrative estimates, not measurements.

Data	Count	Rough size each	Rough total
Sessions (hash, 6 small fields)	2,000,000	400 bytes	0.8 GB
Catalogue (hash, 15 fields)	500,000	1.5 KB	0.75 GB
Carts (hash, 4 items)	3,000,000	300 bytes	0.9 GB
Subtotal	-	-	about 2.5 GB

Illustrative memory estimate.

Then add margin: key overhead and allocator fragmentation (plan for well above the raw payload), replication and client buffers, and room for the copy-on-write cost during a background save or AOF rewrite, which in the worst case approaches the size of the dataset if writes touch most pages. A sensible plan for this data is a node with 8 GB or more and **maxmemory** set noticeably below the physical RAM. The point is the method: count, size, add overhead, add fork headroom, then measure with **MEMORY USAGE** on real keys and revise.

9.8.3 Signals that Redis is a good fit

- Access is by key or by simple ordering, and latency is measured in milliseconds or less.
- The data is a copy, a counter, a window, a queue entry or a short-lived state.
- Operations are naturally atomic and small (increment, push, add to set, claim a lock).
- A TTL describes the data's real lifetime.
- Read volume far exceeds what the primary store should serve.

9.8.4 Signals that it is the wrong tool

Situation	Why Redis struggles	Consider
Large, cold, mostly-archival data	RAM cost; most of it is never read	Relational database, object store, column store
Complex relational queries, joins, reporting	No joins; no ad hoc SQL	PostgreSQL or another relational database
Multi-row ACID transactions with rollback	MULTI has no rollback, scripts are not undone on error	Relational database
Financial ledger, orders, legal records	Asynchronous replication can lose acknowledged writes	Relational database with synchronous replication
Durable high-volume event log with long replay	Bounded by memory, trimmed streams drop history	Kafka or a similar log system
Full-text search over a large corpus with rich analysis	Query engine is capable but not a search platform for every need	Dedicated search engine, or test the Redis query engine on your data
Graph traversal as the core workload	No native graph model in core Redis	Graph database
Blob storage of large files	Large values block the single thread and burn RAM	Object storage; keep the URL in Redis

When Redis is the wrong primary store, and what to consider instead.

9.8.5 Redis as the primary store: a sober view

It is possible. With **appendonly yes**, an **appendfsync** policy that matches your loss tolerance, replicas, and tested backups, Redis can hold the primary copy of some data, and teams do run it that way for things like sessions or leaderboards that they treat as the authoritative copy. But understand what you accept: with **everysec** a crash can lose about a second of writes; replication is asynchronous, so a failover can lose more; **WAIT** narrows but does not eliminate the window; and a dataset must fit in memory on the node or shard. If those terms are acceptable for this data, it is a valid choice. If you hesitate, add a database behind it.

9.8.6 The cache-vs-store test

Ask: "if I flushed Redis right now, what breaks and for how long?" Group your keys into three buckets and handle each differently:

- **Pure cache:** nothing breaks except speed. Use **allkeys-lru** or **allkeys-lfu**, no persistence needed (or RDB for faster warm-up).
- **State with a source of truth:** users are inconvenienced (logged out, carts empty) and recovery is possible. Persistence on, **noeviction** or **volatile-*** policies, TTLs everywhere that makes sense.
- **State with no other copy:** treat it like a database: AOF, replicas, backups, alerts and restore drills.

Mixing all three on one instance with one eviction policy is the most common architecture mistake. Separate them.

9.8.7 Alternatives, case by case

Need	Redis	Alternative and when it wins
Application cache	Shared, networked, rich structures	In-process cache (Caffeine) when one JVM and nanosecond reads matter; Memcached for plain, simple string caching
Session store	TTL, hashes, fast	A database table when sessions must survive anything and traffic is small
Message queue	Lists and streams, simple	RabbitMQ for routing and acknowledgements semantics; Kafka for durable, replayable event streams
Counters and analytics	Atomic, fast, approximate options	A time-series or column store for long history and ad hoc analysis
Leaderboards	Sorted sets are ideal	A database with an index, when rankings are rare and small
Locks	Fast, simple, needs care	Database row locks or advisory locks, or a consensus store such as ZooKeeper or etcd, when correctness is critical
Search	Query engine with vectors built in	Dedicated search engine for large, text-heavy workloads

Redis compared to common alternatives.

Valkey, the open-source fork of Redis created from Redis 7.2.4, speaks the same core protocol and commands. The newer Redis 8 data types and the query engine are specific to Redis; check the license terms and feature set against your needs when choosing a distribution or a managed service.

INTERVIEW ANGLE

When would you not use Redis? When the data is large and cold, when I need relational queries or multi-row transactions with rollback, or when losing acknowledged writes is unacceptable and there is no durable store behind it. I would use Redis around the primary database as a cache, a limiter, a queue or a coordination layer, and keep the system of record where it has strong durability and constraints.

KEY POINTS

- Ask about size, loss tolerance, consistency, query shape and operability before choosing Redis.
- Redis is right for key-based, small, atomic, TTL-shaped, latency-sensitive workloads.
- It is wrong as the sole store for ledgers, large cold data, relational queries and replayable event history.
- Separate pure cache, state with a source of truth, and state with no other copy, with different policies.
- Compare against the specific alternative for each case instead of choosing by habit.

9.9 Reference Architectures for ShopStream

This chapter assembles the pieces. ShopStream, our running example, is a mid-sized online shop with a gateway and five services: catalog, cart, order, inventory and notification. Below are three reference architectures, from the smallest that is sensible to one that survives a regional sale. They are generic designs for a typical company at each stage; adapt the numbers to your traffic.

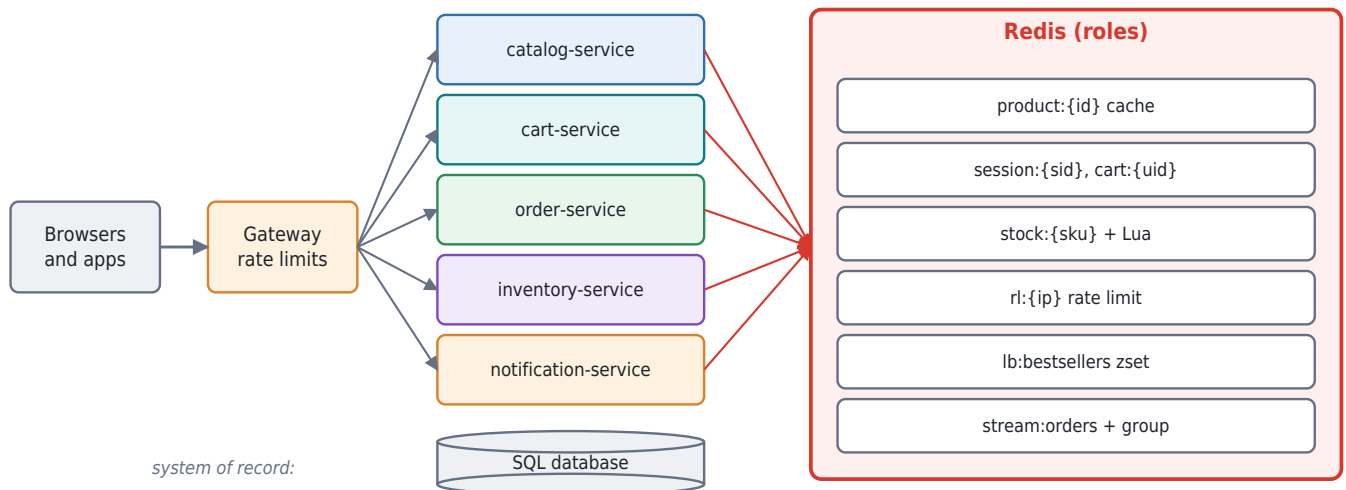


Figure 9.1. Redis roles in ShopStream: notice that every Redis role sits beside a durable database, never in place of it.

9.9.1 Which key family belongs to which service

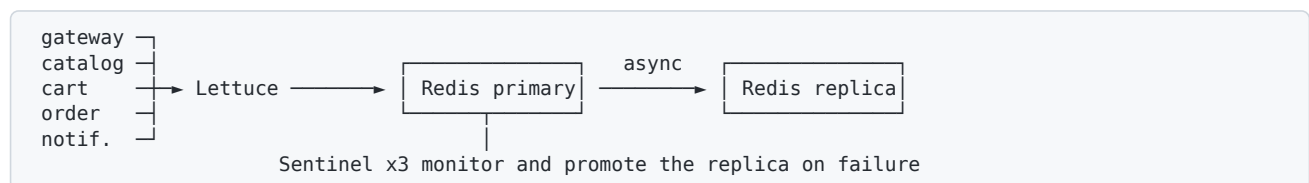
Before choosing a topology, decide ownership. A shared Redis does not mean shared keys: each key family has one owning service, a naming convention and a lifetime.

Service	Keys	Structure	Lifetime and loss policy
catalog	`product:{id}`	Hash or JSON	TTL hours with jitter; rebuildable
cart	`cart:{userId}`	Hash	7 days sliding; low-value loss
gateway	`session:{sid}`, `rl:{ip}:{route}`	Hash, string	Minutes; loss logs users out
inventory	`stock:{sku}`	String counter	Sale duration; reconciled with database
order	`idem:{key}`, `lock:order:{id}`	String	Hours / seconds; database constraint as backstop
order, notification	`stream:orders`	Stream, group notifier	Days, trimmed; must not be silently lost
analytics	`uv:{date}`, `lb:bestsellers:daily`	HyperLogLog, sorted set	Days; rebuildable
catalog	`geo:stores`	Geo	Long; reloaded from database
config	`ff:{name}`	String	Long; owned by the config system

ShopStream key families.

9.9.2 Architecture A: one primary, one replica (start here)

For a young shop or a staging environment, a single primary with one replica and Sentinel is enough.



Settings that matter: **appendonly** yes with **appendfsync** everysec because the instance holds carts, sessions and the order stream; **maxmemory** with **maxmemory-policy** volatile-lru so that only keys with a TTL are evicted and stream and counters are protected (every cache key must therefore have a TTL); and a monitored memory alert well before the limit. All workloads share one instance, which is the weakness: a cache memory spike can push the instance into eviction trouble. Acceptable at small scale, and the next step fixes it.

9.9.3 Architecture B: separate cache and state (the usual production shape)

Split by what happens on loss. A cache instance (or cluster) holds only rebuildable data and evicts freely. A state instance holds sessions, carts, stock, idempotency keys and the order stream, with persistence and no eviction.

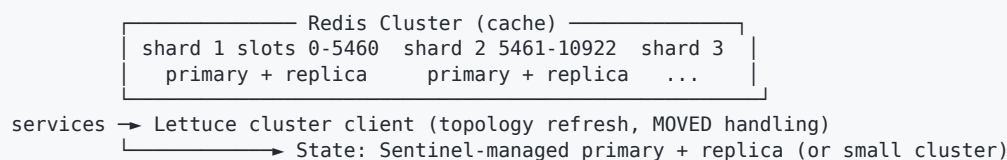
Setting	Cache instance	State instance
Keys	product, search results, bestsellers	session, cart, stock, idem, stream:orders
maxmemory-policy	allkeys-lru or allkeys-lfu	noeviction
Persistence	None or RDB only	AOF everysec plus RDB
Replication / HA	Replica optional	Replica and Sentinel, or managed HA
On memory full	Evicts old entries	Writes fail loudly; alert and scale

Two instances with different policies.

In Spring Boot (see the Spring Part) this means two connection factories and two templates, one per instance, with `@Qualifier` names such as `cacheRedisTemplate` and `stateRedisTemplate`. `RedisCacheManager` is wired to the cache connection only. This separation is the single biggest reliability improvement for the least effort.

9.9.4 Architecture C: sharded for a big sale

When the state no longer fits in one node's memory, or one node's single command thread becomes the bottleneck, move to Redis Cluster for the dataset that needs it (usually the cache and the sessions, which are plentiful and independent per key).



Design rules for the cluster: keep every multi-key operation inside one slot with a hash tag, for example `cart:{42}` and `cart:{42}:meta` together, because cross-slot commands fail. Make the stock key `stock:{sku}` so that one SKU is always a single-shard operation; a flash sale for one hot SKU still funnels to one shard, so for extreme cases split the stock into several buckets (`stock:{sku}:1` to `:8`) and draw from a random bucket. Use `readFrom` replica preference in the client only for data where slightly stale reads are fine. Rehearse a node failure and a resharding before the sale, not during it.

9.9.5 The sale-day checklist

Whatever the architecture, work through this before a big event:

1. Load-test the hot paths with realistic key distributions, not one key repeated.
2. Pre-warm the catalogue cache for the featured products.
3. Confirm that every cache key has a TTL and that the state instance runs `noeviction` with an alert at 70 to 80 percent memory.
4. Check `latency` and `slowlog` baselines; no command should be a surprise.
5. Verify client timeouts, retry limits and the fail-open or fail-closed choice per endpoint.
6. Test failover and a restore from backup in a non-production copy, with a stopwatch.
7. Have a runbook for: Redis down, memory full, replica lagging, hot key.

9.9.6 Putting it together

The common thread across all three: Redis sits next to the durable stores, each key family has an owner, a lifetime and a loss policy, and the architecture is chosen by asking what happens when each piece disappears. If you can answer that for every key in the table at the start of this chapter, you understand Redis well enough to design with it.

REAL WORLD

Growing in steps Teams rarely start at Architecture C. A typical path is A for the first launch, B after the first scare about eviction or sessions, and C only when measured memory or throughput requires it. Each step is a response to a measurement, not a guess.

INTERVIEW ANGLE

How would you use Redis in an e-commerce platform and what would you keep out of it? In: catalogue cache with jittered TTLs, sessions, carts as hashes, a Lua-guarded stock counter for sales, rate limits and idempotency keys at the gateway, a bestseller sorted set, and an order stream with a consumer group for notifications. Out: the order ledger, payments and anything that must survive a failover without loss, which belong in the database. I would separate cache and state instances because they need different eviction and persistence.

KEY POINTS

- Assign every key family to one owning service, with a lifetime and a loss policy.
- Start with a primary, a replica and Sentinel; separate cache from state next; shard only when measured.
- Cache instances evict and skip persistence; state instances use **noeviction**, AOF and replicas.
- In a cluster, design keys with hash tags so multi-key operations stay in one slot.
- Prepare for events with load tests, pre-warming, alerts, failover rehearsals and a runbook.

PART 10

Projects

Four complete Spring Boot services. Build them, break them, extend them.

In this part

- 10.1** How to Work Through the Projects
- 10.2** Project A: A Production-Grade Catalogue Cache
- 10.3** Project B: An API Rate Limiter Service
- 10.4** Project C: A Flash-Sale Service That Never Oversells
- 10.5** Project D: A Live Leaderboard with Notifications

10.1 How to Work Through the Projects

The earlier Parts explained Redis one idea at a time. This Part puts the ideas together. You will build four small but complete services, each one a slice of ShopStream: a catalogue service with a hardened cache, a rate limiter, a flash-sale service that must never oversell, and a leaderboard with live notifications. Each project is small enough to finish in an evening and realistic enough to teach you the problems you will meet at work.

Every project follows the same shape, so you always know where you are:

1. **Goal and requirements:** what the service must do, stated so you can test it.
2. **Architecture and layout:** a sketch and the file tree.
3. **Build and run files:** `pom.xml` dependencies, `docker-compose.yml`, `application.yml`.
4. **The code:** the key classes in full, followed by "what to notice".
5. **Verify with redis-cli:** look at the keys your code created.
6. **Tests:** integration tests against a real Redis in a container (Testcontainers).
7. **Demo and extensions:** a behaviour or load demo, then ideas to push further.

10.1.1 Common setup for all four projects

All four projects use Java 21, Maven, Spring Boot 4.1 and Spring Data Redis 4.1 with the default Lettuce client. You need a JDK 21, Maven 3.9+ and Docker. Each project is a separate Maven module with its own `pom.xml`; they share nothing except the conventions below, so you can build them in any order.

Convention	Meaning
Package root	<code>com.shopstream.<project></code> , for example <code>com.shopstream.catalog</code>
Redis image	<code>redis:8</code> , which includes the Bloom filter, JSON and Search commands without extra modules
Key names	<code>entity:id:field</code> with colons; every project documents its keys in a table
Redis access	<code>StringRedisTemplate</code> for strings, counters and Lua; JSON is written by hand with the Boot-provided <code>JsonMapper</code>
Tests	<code>@SpringBootTest</code> plus a Redis container declared as a bean with <code>@ServiceConnection</code>

Conventions shared by the projects.

NOTE

Version numbers in the `pom.xml` listings are the ones the Spring Boot parent manages. Use the latest 4.1.x patch release of `spring-boot-starter-parent`; do not pin Spring Data Redis, Lettuce, Micrometer or Testcontainers yourself.

Two choices recur and are worth explaining once. First, we call Lua scripts through `DefaultRedisScript` with only string arguments. `StringRedisTemplate` serialises script arguments with its string serializer, so a `Long` argument would fail; always pass `String.valueOf(n)`. Second, our tests start a real Redis through Testcontainers rather than mocking `StringRedisTemplate`. A mock cannot tell you whether your Lua script, your key expiry or your Bloom filter command works, and those are the parts that break.

TIP

Type the code yourself the first time, at least for the core classes. The compile errors and the first failing test teach more than a pasted listing does. Compare with the listing afterwards.

KEY POINTS

Each project is a vertical slice: HTTP endpoint, Redis data model, atomic logic, tests.

- Run Redis in Docker; run the service on your machine so you can use a debugger.
- Document every key you create; a service whose keys you cannot list is a service you cannot operate.
- Test against a real Redis; mock only the slow database.
- Pass only strings to scripts executed through `StringRedisTemplate`.

10.2 Project A: A Production-Grade Catalogue Cache

10.2.1 Goal and requirements

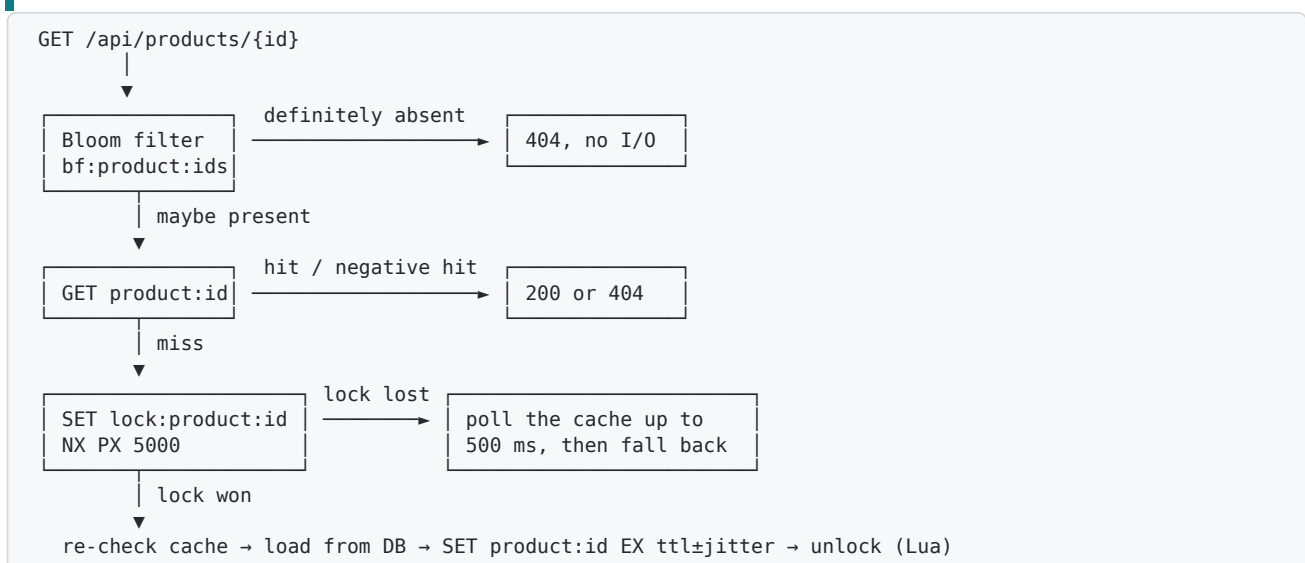
ShopStream's catalog-service answers `GET /api/products/{id}` thousands of times per second, and the product database is slow (we will simulate 50 ms per lookup). The cache-aside pattern from the Patterns Part is the starting point. In this project we harden it against the four failures that bite real caches.

Requirement	What it means
Cache-aside read	Look in Redis first, load from the database on a miss, store the result
TTL with jitter	Base TTL 10 minutes plus or minus 20 percent, so keys loaded together do not expire together
Negative caching	A product that does not exist is remembered for 60 seconds as a marker value
Bloom filter	Ids that were never created are rejected before they reach the cache or the database
Stampede protection	When a hot key expires, only one request reloads it; the rest wait briefly and read the result
Metrics	Counters for hit, miss, negative and Bloom rejections, and a counter of database loads

Requirements.

The last metric is the one that proves the work. If 200 concurrent requests for a cold key cause exactly one database load, the design works.

10.2.2 Architecture



The Bloom filter has one-sided error: it can say "maybe present" for an id that does not exist (a false positive), but it never says "absent" for an id that was added. That is exactly the property we need to shield the database from random ids, and the negative cache covers the false positives.

10.2.3 Project layout

```

catalog-cache/
├── pom.xml
├── docker-compose.yml
├── src
│   ├── main
│   │   ├── java/com/shopstream/catalog
│   │   │   ├── CatalogApplication.java
│   │   │   ├── CatalogProperties.java
│   │   │   ├── Product.java
│   │   │   ├── ProductRepository.java
│   │   │   ├── SlowProductRepository.java
│   │   │   ├── BloomFilter.java
│   │   │   ├── SingleFlightLock.java
│   │   │   ├── ProductCache.java
│   │   │   ├── CatalogService.java
│   │   │   ├── CatalogMetrics.java
│   │   │   └── ProductController.java
│   │   └── resources/application.yml
│   └── test/java/com/shopstream/catalog
│       ├── RedisTestConfig.java
│       └── CatalogServiceIT.java

```

10.2.4 Build and run files

The `pom.xml` below shows only the parts you must write; the parent and `<properties>` (Java 21) are the usual Boot boilerplate.

pom.xml (dependencies) XML

```

<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.0</version>
</parent>
<properties><java.version>21</java.version></properties>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-testcontainers</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

```

GOTCHA

Spring Boot 4 split its auto-configuration into smaller modules and renamed some starters (the web starter is now `spring-boot-starter-webmvc`, with a matching `spring-boot-starter-webmvc-test`). If a tutorial written for Boot 3 does not compile, check starter names and test-annotation packages first. The listings in this Part avoid `MockMvc`-style test slices so they depend on as little of that as possible.

docker-compose.yml

YAML

```

services:
  redis:
    image: redis:8
    container_name: shopstream-redis
    ports:
      - "6379:6379"
    command: ["redis-server", "--appendonly", "no", "--maxmemory", "256mb",
              "--maxmemory-policy", "allkeys-lru"]
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 2s
      retries: 10

```

We set `maxmemory` and an eviction policy because a cache without a memory limit is not a cache, it is a slow memory leak. `allkeys-lru` is the usual choice for a pure cache. Note it can also evict the Bloom filter and lock keys; the extension ideas at the end of the chapter show how to cope.

application.yml

YAML

```

spring:
  application:
    name: catalog-cache
  data:
    redis:
      host: localhost
      port: 6379
      timeout: 500ms

  management:
    endpoints:
      web:
        exposure:
          include: health,metrics

  catalog:
    ttl: 10m
    ttl-jitter: 0.2
    negative-ttl: 60s
    lock-ttl: 5s
    wait-max: 500ms
    db-latency-ms: 50
    seed-products: 1000

```

10.2.5 The domain and the slow database

Product.java

JAVA

```

package com.shopstream.catalog;

import java.math.BigDecimal;

public record Product(String id, String name, BigDecimal price, int stock) {
}

```

ProductRepository.java

JAVA

```

package com.shopstream.catalog;

import java.util.Collection;
import java.util.Optional;

public interface ProductRepository {

    Optional<Product> findById(String id);

    void save(Product product);

    Collection<String> allIds();

    /** How many times findById ran. Tests use this to count database loads. */
    int loadCount();
}

```

SlowProductRepository.java

JAVA

```

package com.shopstream.catalog;

import java.math.BigDecimal;
import java.util.Collection;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;
import org.springframework.stereotype.Repository;

@Repository
public class SlowProductRepository implements ProductRepository {

    private final Map<String, Product> table = new ConcurrentHashMap<>();
    private final AtomicInteger loads = new AtomicInteger();
    private final long latencyMs;

    public SlowProductRepository(CatalogProperties props) {
        this.latencyMs = props.dbLatencyMs();
        for (int i = 1; i <= props.seedProducts(); i++) {
            String id = "p-" + i;
            table.put(id, new Product(id, "Product " + i, BigDecimal.valueOf(i * 3L, 2), 100));
        }
    }

    @Override
    public Optional<Product> findById(String id) {
        loads.incrementAndGet();
        try {
            Thread.sleep(latencyMs);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        return Optional.ofNullable(table.get(id));
    }

    @Override
    public void save(Product product) {
        table.put(product.id(), product);
    }

    @Override
    public Collection<String> allIds() {
        return table.keySet();
    }

    @Override
    public int loadCount() {
        return loads.get();
    }
}

```

CatalogProperties.java

JAVA

```

package com.shopstream.catalog;

import java.time.Duration;
import org.springframework.boot.context.properties.ConfigurationProperties;

@ConfigurationProperties(prefix = "catalog")
public record CatalogProperties(
    Duration ttl,
    double ttlJitter,
    Duration negativeTtl,
    Duration lockTtl,
    Duration waitMax,
    long dbLatencyMs,
    int seedProducts) {
}

```

10.2.6 The Bloom filter and the lock

Both helpers are thin wrappers over commands run through Lua scripts. Going through `execute(RedisScript, ...)` gives us a typed result (`Long`) without touching low-level connection APIs. The Bloom filter commands (`BF.RESERVE`, `BF.ADD`, `BF.EXISTS`) are built into Redis 8.

```
BloomFilter.java JAVA

package com.shopstream.catalog;

import java.util.List;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.core.script.DefaultRedisScript;
import org.springframework.stereotype.Component;

@Component
public class BloomFilter {

    static final String KEY = "bf:product:ids";

    // BF.RESERVE fails with "item exists" when the key exists, so ignore that error.
    private static final DefaultRedisScript<Long> RESERVE = new DefaultRedisScript<>("""
        pcall(redis.call, 'BF.RESERVE', KEYS[1], ARGV[1], ARGV[2])
        return 1
        """, Long.class);

    private static final DefaultRedisScript<Long> ADD = new DefaultRedisScript<>("return redis.call('BF.ADD', KEYS[1], ARGV[1])", Long.class);

    private static final DefaultRedisScript<Long> EXISTS = new DefaultRedisScript<>("return redis.call('BF.EXISTS', KEYS[1], ARGV[1])", Long.class);

    private final StringRedisTemplate redis;

    public BloomFilter(StringRedisTemplate redis) {
        this.redis = redis;
    }

    /** Create the filter sized for 'capacity' ids at the given false-positive rate. */
    public void reserve(double errorRate, long capacity) {
        redis.execute(RESERVE, List.of(KEY), String.valueOf(errorRate), String.valueOf(capacity));
    }

    public void add(String id) {
        redis.execute(ADD, List.of(KEY), id);
    }

    public boolean mightContain(String id) {
        Long r = redis.execute(EXISTS, List.of(KEY), id);
        return r != null && r == 1L;
    }
}
```

WATCH OUT

If the Bloom key does not exist (for example it was evicted, or you flushed the database), `BF.EXISTS` returns 0 for everything and your service would answer 404 for every product. The startup runner below rebuilds the filter, but in production you also want an alert on `EXISTS bf:product:ids` and a way to rebuild it without a restart. Either exclude the key from eviction by using a `volatile-*` policy (it has no TTL) or run the filter on a Redis that never evicts.

```
SingleFlightLock.java JAVA

package com.shopstream.catalog;

import java.time.Duration;
import java.util.List;
import java.util.UUID;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.core.script.DefaultRedisScript;
import org.springframework.stereotype.Component;

@Component
public class SingleFlightLock {

    // Delete the lock only if we still own it (compare token, then delete, atomically).
```

SingleFlightLock.java (continued)

JAVA

```

private static final DefaultRedisScript<Long> RELEASE = new DefaultRedisScript<>("""
    if redis.call('GET', KEYS[1]) == ARGV[1] then
        return redis.call('DEL', KEYS[1])
    end
    return 0
    """, Long.class);

private final StringRedisTemplate redis;
private final Duration ttl;

public SingleFlightLock(StringRedisTemplate redis, CatalogProperties props) {
    this.redis = redis;
    this.ttl = props.lockTtl();
}

/** Returns an owner token if we got the lock, otherwise null. */
public String tryAcquire(String productId) {
    String token = UUID.randomUUID().toString();
    Boolean ok = redis.opsForValue().setIfAbsent(key(productId), token, ttl);
    return Boolean.TRUE.equals(ok) ? token : null;
}

public void release(String productId, String token) {
    redis.execute(RELEASE, List.of(key(productId)), token);
}

private static String key(String productId) {
    return "lock:product:" + productId;
}
}

```

What to notice: `setIfAbsent(key, value, Duration)` sends a single `SET key value NX PX ms` command, so the lock and its expiry are set atomically. The lock has a TTL (5 seconds) so a crashed instance cannot block a key forever, and the release script checks the token so a slow request cannot delete a lock that has since passed to someone else. This is the single-key lock from the distributed locking chapter, deliberately kept simple: here a lost or expired lock only means one extra database read, not corrupted data.

10.2.7 The cache layer

`ProductCache` knows how products are stored: as JSON text under `product:{id}`, with a short marker for "this product does not exist". It returns a three-state result, so the caller can tell "not cached" from "cached as absent".

ProductCache.java

JAVA

```

package com.shopstream.catalog;

import java.time.Duration;
import java.util.Optional;
import java.util.concurrent.ThreadLocalRandom;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Component;
import tools.jackson.databind.json.JsonMapper;

@Component
public class ProductCache {

    static final String ABSENT = "__absent__";

    public enum Status { MISS, HIT, NEGATIVE }

    public record Lookup(Status status, Product product) {
        static final Lookup MISS = new Lookup(Status.MISS, null);
        static final Lookup NEGATIVE = new Lookup(Status.NEGATIVE, null);

        public Optional<Product> asOptional() {
            return Optional.ofNullable(product);
        }
    }

    private final StringRedisTemplate redis;
    private final JsonMapper json;
    private final CatalogProperties props;
}

```

ProductCache.java (continued)

JAVA

```

public ProductCache(StringRedisTemplate redis, JsonMapper json, CatalogProperties props) {
    this.redis = redis;
    this.json = json;
    this.props = props;
}

public Lookup get(String id) {
    String raw = redis.opsForValue().get(key(id));
    if (raw == null) {
        return Lookup.MISS;
    }
    if (ABSENT.equals(raw)) {
        return Lookup.NEGATIVE;
    }
    return new Lookup(Status.HIT, json.readValue(raw, Product.class));
}

public void put(Product product) {
    redis.opsForValue().set(key(product.id()), json.writeValueAsString(product),
        withJitter(props.ttl(), props.ttlJitter()));
}

public void putAbsent(String id) {
    redis.opsForValue().set(key(id), ABSENT, props.negativeTtl());
}

public void evict(String id) {
    redis.delete(key(id));
}

static Duration withJitter(Duration base, double fraction) {
    double factor = 1.0 + ThreadLocalRandom.current().nextDouble(-fraction, fraction);
    return Duration.ofMillis(Math.round(base.toMillis() * factor));
}

static String key(String id) {
    return "product:" + id;
}
}

```

NOTE

Jackson 3 lives in the `tools.jackson` packages, and its exceptions are unchecked, which is why `readValue` needs no `throws` clause. Spring Boot 4 auto-configures a `JsonMapper` bean, so we simply inject it. Java records are supported by Jackson without extra annotations.

Jitter deserves a number. With a fixed 600-second TTL, 10,000 products loaded in the same minute would all expire in the same minute, and the database would see a wave of 10,000 loads. With plus or minus 20 percent the expiry times spread over a 240-second window (480 to 720 seconds), which flattens the wave to roughly 40 loads per second on average. The exact figures are illustrative; the idea is to turn a spike into a slope.

10.2.8 The service: putting the defences in order

The order of the checks matters: cheapest and most selective first.

CatalogService.java

JAVA

```

package com.shopstream.catalog;

import java.time.Duration;
import java.util.Optional;
import org.springframework.stereotype.Service;

@Service
public class CatalogService {

    private final ProductRepository repository;
    private final ProductCache cache;
    private final BloomFilter bloom;
    private final SingleFlightLock lock;
    private final CatalogMetrics metrics;
}

```

CatalogService.java (continued)

JAVA

```

private final Duration waitMax;

public CatalogService(ProductRepository repository, ProductCache cache, BloomFilter bloom,
    SingleFlightLock lock, CatalogMetrics metrics, CatalogProperties props) {
    this.repository = repository;
    this.cache = cache;
    this.bloom = bloom;
    this.lock = lock;
    this.metrics = metrics;
    this.waitMax = props.waitMax();
}

public Optional<Product> find(String id) {
    if (!bloom.mightContain(id)) {
        metrics.result("bloom_reject");
        return Optional.empty();
    }
    ProductCache.Lookup first = cache.get(id);
    if (first.status() != ProductCache.Status.MISS) {
        metrics.result(first.status() == ProductCache.Status.HIT ? "hit" : "negative");
        return first.asOptional();
    }
    metrics.result("miss");
    return loadOnce(id);
}

private Optional<Product> loadOnce(String id) {
    String token = lock.tryAcquire(id);
    if (token != null) {
        try {
            ProductCache.Lookup again = cache.get(id); // someone may have filled it
            if (again.status() != ProductCache.Status.MISS) {
                return again.asOptional();
            }
            return loadFromDatabase(id);
        } finally {
            lock.release(id, token);
        }
    }
    return waitForOwner(id);
}

private Optional<Product> waitForOwner(String id) {
    long deadline = System.nanoTime() + waitMax.toNanos();
    while (System.nanoTime() < deadline) {
        try {
            Thread.sleep(20);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            break;
        }
        ProductCache.Lookup l = cache.get(id);
        if (l.status() != ProductCache.Status.MISS) {
            return l.asOptional();
        }
    }
    return loadFromDatabase(id); // owner was too slow or died: do the work ourselves
}

private Optional<Product> loadFromDatabase(String id) {
    Optional<Product> found = metrics.timeDbLoad(() -> repository.findById(id));
    if (found.isPresent()) {
        cache.put(found.get());
    } else {
        cache.putAbsent(id);
    }
    return found;
}

public Product save(Product product) {
    repository.save(product);
    bloom.add(product.id()); // a new id must pass the filter
}

```

CatalogService.java (continued)

JAVA

```

        cache.evict(product.id());    // delete after the write; the next read repopulates
        return product;
    }
}

```

CatalogMetrics.java

JAVA

```

package com.shopstream.catalog;

import io.micrometer.core.instrument.MeterRegistry;
import io.micrometer.core.instrument.Timer;
import java.util.function.Supplier;
import org.springframework.stereotype.Component;

@Component
public class CatalogMetrics {

    private final MeterRegistry registry;
    private final Timer dbTimer;

    public CatalogMetrics(MeterRegistry registry) {
        this.registry = registry;
        this.dbTimer = Timer.builder("catalog.db.loads").register(registry);
    }

    public void result(String kind) {
        registry.counter("catalog.cache", "result", kind).increment();
    }

    public <T> T timeDbLoad(Supplier<T> work) {
        return dbTimer.record(work);
    }
}

```

What to notice in `CatalogService`:

- **Bloom first.** One cheap `BF.EXISTS` round trip protects the cache and database from ids that were never valid.
- **Double-check after winning the lock.** Between our cache miss and our lock acquisition another request may have finished loading. Without this second `get`, every request that queued behind the first would reload the same row one after another.
- **Waiters poll, they do not block on the lock.** A waiter checks the cache every 20 ms for up to `wait-max`. If the owner is slow or dead, the waiter falls back to loading itself, so the lock can only make things faster, never wedge a request.
- **Write path deletes, it does not update.** After `save` we delete the cached entry and let the next read repopulate it. This is simpler than writing the new value and avoids storing a value computed from stale data.
- `Timer.record(Supplier)` counts every database load, and its `count` statistic is our proof of single-flight behaviour.

GOTCHA

Delete-after-write still has a tiny race: a reader that loaded the old row just before the write can put the old value back just after your delete. The jittered TTL bounds the damage (the stale entry expires), and for most catalogues that is acceptable. If it is not, version the entries or use short TTLs for frequently edited products.

10.2.9 The HTTP layer and startup warm-up

ProductController.java

JAVA

```
package com.shopstream.catalog;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final CatalogService service;

    public ProductController(CatalogService service) {
        this.service = service;
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> get(@PathVariable String id) {
        return service.find(id)
            .map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PostMapping("/{id}")
    public Product put(@PathVariable String id, @RequestBody Product body) {
        return service.save(new Product(id, body.name(), body.price(), body.stock()));
    }
}
```

CatalogApplication.java

JAVA

```
package com.shopstream.catalog;

import org.springframework.boot.ApplicationRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
@EnableConfigurationProperties(CatalogProperties.class)
public class CatalogApplication {

    public static void main(String[] args) {
        SpringApplication.run(CatalogApplication.class, args);
    }

    /** Rebuilds the Bloom filter from the database every time the service starts. */
    @Bean
    ApplicationRunner warmBloomFilter(ProductRepository repository, BloomFilter bloom) {
        return args -> {
            bloom.reserve(0.01, 100_000);
            repository.allIds().forEach(bloom::add);
        };
    }
}
```

The filter is sized for 100,000 ids at a 1 percent false-positive rate. If more ids are added later, the filter auto-scales by creating sub-filters (the default unless `NONSCALING` is passed), at the cost of a somewhat higher combined error rate and more memory.

10.2.10 Verify with redis-cli

Start Redis with `docker compose up -d`, start the service with `mvn spring-boot:run`, then:

Watch the cache fill and the negative cache work

TERMINAL

```
$ curl -s localhost:8080/api/products/p-42
{"id":"p-42","name":"Product 42","price":1.26,"stock":100}
$ curl -s -o /dev/null -w "%{http_code}\n" localhost:8080/api/products/zzz-unknown
404
$ docker exec -it shopstream-redis redis-cli
127.0.0.1:6379> GET product:p-42
"{\"id\":\"p-42\",\"name\":\"Product 42\",\"price\":1.26,\"stock\":100}"
127.0.0.1:6379> TTL product:p-42
(integer) 541
127.0.0.1:6379> BF.EXISTS bf:product:ids p-42
(integer) 1
127.0.0.1:6379> BF.EXISTS bf:product:ids zzz-unknown
(integer) 0
127.0.0.1:6379> EXISTS product:zzz-unknown
(integer) 0
```

The unknown id left no trace: the Bloom filter rejected it, so there is no `product:zzz-unknown` key and no database load. Now simulate a deleted product, which is in the filter but not in the table. Remove it from the repository (or use a false-positive id) and request it twice; the second request is answered from the marker:

Negative marker

TERMINAL

```
127.0.0.1:6379> GET product:p-999999
"__absent__"
127.0.0.1:6379> TTL product:p-999999
(integer) 57
```

10.2.11 Tests with Testcontainers

The test configuration declares the Redis container as a Spring bean. Spring Boot's `@ServiceConnection` then points `spring.data.redis.host` and `port` at the mapped container port automatically. Because `GenericContainer` does not reveal which service it runs, we give the hint with `name = "redis"`.

RedisTestConfig.java

JAVA

```
package com.shopstream.catalog;

import org.springframework.boot.test.context.TestConfiguration;
import org.springframework.boot.testcontainers.service.connection.ServiceConnection;
import org.springframework.context.annotation.Bean;
import org.testcontainers.containers.GenericContainer;

@TestConfiguration(proxyBeanMethods = false)
public class RedisTestConfig {

    @Bean
    @ServiceConnection(name = "redis")
    GenericContainer<?> redisContainer() {
        return new GenericContainer<>("redis:8").withExposedPorts(6379);
    }
}
```

CatalogServiceIT.java

JAVA

```
package com.shopstream.catalog;

import static org.assertj.core.api.Assertions.assertThat;

import java.util.List;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.stream.IntStream;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;
import org.springframework.data.redis.core.StringRedisTemplate;
```

CatalogServiceIT.java (continued)

JAVA

```

@SpringBootTest(properties = {"catalog.seed-products=50", "catalog.db-latency-ms=100"})
@Import(RedisTestConfig.class)
class CatalogServiceIT {

    @Autowired CatalogService service;
    @Autowired ProductRepository repository;
    @Autowired StringRedisTemplate redis;
    @Autowired BloomFilter bloom;

    @Test
    void secondReadIsServedFromCache() {
        int before = repository.loadCount();
        assertThat(service.find("p-1")).isPresent();
        assertThat(service.find("p-1")).isPresent();
        assertThat(repository.loadCount() - before).isEqualTo(1);
        Long ttl = redis.getExpire("product:p-1");
        assertThat(ttl).isBetween(470L, 730L); // 600 s plus or minus 20 percent
    }

    @Test
    void unknownIdNeverReachesTheDatabase() {
        int before = repository.loadCount();
        assertThat(service.find("does-not-exist-" + System.nanoTime())).isEmpty();
        assertThat(repository.loadCount()).isEqualTo(before);
    }

    @Test
    void concurrentMissesCauseOneDatabaseLoad() {
        redis.delete("product:p-7");
        int before = repository.loadCount();
        ExecutorService pool = Executors.newFixedThreadPool(40);
        try {
            List<CompletableFuture<Boolean>> calls = IntStream.range(0, 40)
                .mapToObj(i -> CompletableFuture.supplyAsync(
                    () -> service.find("p-7").isPresent(), pool))
                .toList();
            calls.forEach(c -> assertThat(c.join()).isTrue());
        } finally {
            pool.shutdown();
        }
        assertThat(repository.loadCount() - before).isEqualTo(1);
    }

    @Test
    void removedProductIsRememberedAsAbsent() {
        // An id that is in the Bloom filter but not in the table: add it to the filter only.
        String ghost = "ghost-" + System.nanoTime();
        bloom.add(ghost);
        int before = repository.loadCount();
        assertThat(service.find(ghost)).isEmpty();
        assertThat(service.find(ghost)).isEmpty();
        assertThat(repository.loadCount() - before).isEqualTo(1);
        assertThat(redis.opsForValue().get("product:" + ghost)).isEqualTo("__absent__");
    }
}

```

WATCH OUT

The stampede test is only meaningful if the database call is slower than the time it takes the threads to start, which is why the test property sets `catalog.db-latency-ms=100`. If you set it to zero, the test passes even without the lock because the first load finishes before the others miss. A test that cannot fail is not a test; break `loadOnce` on purpose (skip the lock) and confirm the test goes red.

10.2.12 Demo: watch the database load counter

With the service running, fire 200 requests with 50 in parallel at a cold key, then read the counter:

Single-flight under load

TERMINAL

```
$ docker exec shopstream-redis redis-cli DEL product:p-500
(integer) 1
$ seq 1 200 | xargs -P 50 -I{} curl -s -o /dev/null localhost:8080/api/products/p-500
$ curl -s localhost:8080/actuator/metrics/catalog.db.loads | grep -o '"COUNT","value":[0-9.]*'
"COUNT","value":1.0
$ curl -s "localhost:8080/actuator/metrics/catalog.cache?tag=result:hit" | grep -o '"value":[0-9.]*'
"value":199.0
```

The exact split between hits and misses varies from run to run (a few early requests may be counted as misses while they wait); the number to watch is the single database load. Repeat the experiment after commenting out the lock in `loadOnce` and the counter will jump to the number of requests that arrived in the first 50 ms.

INTERVIEW ANGLE

How do you protect a database from a cache stampede, and from cache penetration? A stampede is many concurrent misses on one hot key (usually just after it expires); protect with single-flight loading (a short Redis lock or an in-process lock so one request reloads), TTL jitter so keys do not expire together, and optionally serving slightly stale data while one request refreshes. Penetration is repeated lookups for keys that do not exist; protect with negative caching (short-TTL marker), a Bloom filter of valid ids, and input validation.

10.2.13 Extension ideas

- **Stale-while-revalidate.** Store a soft expiry inside the JSON and keep the Redis TTL longer. Serve the stale value immediately and let one request refresh in the background.
- **Two-level cache.** Add a small in-JVM Caffeine cache in front of Redis with a 5 second TTL, and compare latency. Then add client-side caching invalidation from the Redis chapter on tracking.
- **Warm the hot set.** On start, preload the top 500 products from the leaderboard of most viewed items.
- **Measure.** Add a `redis-benchmark` style load with 95 percent reads on 100 hot ids and chart p99 latency with and without jitter and single-flight.
- **Survive eviction.** Move the Bloom filter to its own Redis instance with `noeviction`, or rebuild it when `BF.EXISTS` on a known id returns 0.

KEY POINTS

What this project proves:

- Cache-aside needs more than get/set: jittered TTLs, negative caching and single-flight loading each close a specific failure.
- A Bloom filter answers "definitely not here" for almost no memory and no database load.
- A lock used to save work can be short and lossy; a lost lock costs a duplicate read, so wait with a bounded timeout and fall back.
- Count database loads; that metric is the proof the cache does its job.
- Test with a slow fake database and a real Redis; both are needed to see stampedes.

10.3 Project B: An API Rate Limiter Service

10.3.1 Goal and requirements

ShopStream's gateway must stop one noisy client from starving everyone else. Customers who pay get a larger allowance than anonymous visitors. You will build a rate limiter as a Spring MVC interceptor backed by one Lua script, so that the check-and-update is atomic no matter how many gateway instances run.

Requirement	What it means
Algorithm	Token bucket: a bucket holds up to <code>capacity</code> tokens, refills at <code>refill</code> tokens per second, and each request spends <code>cost</code> tokens

Requirement	What it means
Atomic	Read, refill, spend and write happen in one Lua script; two instances can never both spend the last token
Per-plan limits	Each API key maps to a plan (anonymous , free , pro) with its own capacity and refill rate
Weighted routes	An expensive route such as search can cost several tokens
Response contract	429 Too Many Requests with Retry-After , plus X-RateLimit-Limit and X-RateLimit-Remaining on every response
Failure policy	If Redis is unreachable the limiter fails open (requests pass) and counts the event
Clock	The script uses the Redis TIME command, so application servers with skewed clocks do not matter

Requirements.

10.3.2 Why a token bucket

The Patterns Part compared three algorithms. As a reminder of the trade-offs that led to this choice:

Algorithm	Strength	Weakness
Fixed window counter	Trivial: INCR plus EXPIRE	A client can send 2× the limit across a window boundary
Sliding window log	Exact	One sorted-set member per request; memory grows with the limit
Token bucket	Allows controlled bursts, two numbers of state per client	Slightly harder to explain; needs the script below

Algorithms at a glance.

The bucket is a hash with two fields: **tokens** (a number that may be fractional) and **ts** (the millisecond time of the last update). On each request the script computes how many tokens have accumulated since **ts**, caps the total at **capacity**, tries to spend **cost**, and writes the new state back. Idle clients cost nothing: the hash gets a TTL slightly longer than the time needed to refill a full bucket, after which a fresh client is indistinguishable from a forgotten one.

```
capacity = 10, refill = 5 tokens/s, cost = 1
```

```
time    0.0s   0.2s   0.4s   ...  1.0s       2.0s
tokens  10 → 9 → 8 → ... ▶ (burst of 10 sent, bucket empty)
                        0 → +5 per second → steady 5 req/s
11th request at 0.0s → denied, Retry-After = ceil((1 - 0) / 5) = 1 s (rounded up)
```

10.3.3 Project layout

```
rate-limiter/
├── pom.xml                (same dependencies as Project A)
├── docker-compose.yml     (same Redis service as Project A)
├── src
│   ├── main
│   │   ├── java/com/shopstream/ratelimit
│   │   │   ├── RateLimiterApplication.java
│   │   │   ├── RateLimitProperties.java
│   │   │   ├── TokenBucketLimiter.java
│   │   │   ├── RateLimitInterceptor.java
│   │   │   ├── WebConfig.java
│   │   │   └── DemoController.java
│   │   ├── resources
│   │   │   ├── application.yml
│   │   │   └── scripts/token_bucket.lua
│   └── test/java/com/shopstream/ratelimit
│       ├── RedisTestConfig.java
│       └── RateLimiterIT.java
```

The **pom.xml** needs **spring-boot-starter-webmvc**, **spring-boot-starter-data-redis**, **spring-boot-starter-actuator**, and for tests **spring-boot-starter-test** and **spring-boot-testcontainers**, exactly as in Project A. The **docker-compose.yml** can be reused unchanged; for a rate limiter you may prefer **--maxmemory-policy volatile-ttl**, because every bucket key has a TTL.

10.3.4 Configuration

The limits are data, not code, so they live in `application.yml` and bind to a record.

```
application.yml
spring:
  application:
    name: rate-limiter
  data:
    redis:
      host: localhost
      port: 6379
      timeout: 200ms

  ratelimit:
    fail-open: true
    plans:
      anonymous: { capacity: 10, refill-per-second: 2 }
      free:      { capacity: 30, refill-per-second: 10 }
      pro:       { capacity: 200, refill-per-second: 100 }
    keys:
      demo-free: free
      demo-pro:  pro
    route-costs:
      "/api/search": 5

  management:
    endpoints:
      web:
        exposure:
          include: health,metrics
```

NOTE

The Redis `timeout: 200ms` is deliberate. A rate limiter is on the hot path of every request; if Redis stalls, you want the call to fail within a fraction of your request budget and trigger the fail-open policy, not hang for the default of one minute.

```
RateLimitProperties.java
package com.shopstream.ratelimit;

import java.util.Map;
import org.springframework.boot.context.properties.ConfigurationProperties;

@ConfigurationProperties(prefix = "ratelimit")
public record RateLimitProperties(
    boolean failOpen,
    Map<String, Plan> plans,
    Map<String, String> keys,
    Map<String, Integer> routeCosts) {

    public record Plan(int capacity, double refillPerSecond) {
    }
}
```

10.3.5 The Lua script

The whole algorithm is one script. Read it top to bottom; there is no branching you cannot follow.

```
scripts/token_bucket.lua
-- KEYS[1] = bucket key
-- ARGV[1] = capacity (tokens)
-- ARGV[2] = refill rate (tokens per second)
-- ARGV[3] = cost of this request (tokens)
-- Returns { allowed (1/0), remaining tokens (floored), retry-after ms, ms until full }
local capacity = tonumber(ARGV[1])
local refill    = tonumber(ARGV[2])
local cost      = tonumber(ARGV[3])

local t = redis.call('TIME')
local now = t[1] * 1000 + math.floor(t[2] / 1000) -- milliseconds

local data = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
```

scripts/token_bucket.lua (continued)

LUA

```

local tokens = tonumber(data[1])
local ts     = tonumber(data[2])
if tokens == nil then
    tokens = capacity
    ts = now
end

local elapsed = math.max(0, now - ts)
tokens = math.min(capacity, tokens + elapsed * refill / 1000)

local allowed = 0
local retry = 0
if tokens >= cost then
    tokens = tokens - cost
    allowed = 1
else
    retry = math.ceil((cost - tokens) * 1000 / refill)
end

redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
local fullMs = math.ceil((capacity - tokens) * 1000 / refill)
redis.call('PEXPIRE', KEYS[1], math.ceil(capacity * 1000 / refill) + 1000)

return { allowed, math.floor(tokens), retry, fullMs }

```

What to notice:

- **TIME instead of the client clock.** Every application server shares Redis's single clock, so skewed server clocks cannot give one instance extra tokens. Since Redis 5, scripts are replicated by their effects (the resulting writes), so calling **TIME** in a script is safe.
- **State is written even when the request is denied.** Updating **ts** on a denial is correct because the refill has been accounted into **tokens**; skipping the write would make the next call recompute the same refill, which is also correct but wastes the TTL refresh.
- **Floating point stays inside the script.** Lua numbers are doubles. A Lua number returned to the client is truncated to an integer, which is why we return **math.floor** values and let Java deal with whole numbers.
- **One key, so no cluster problem.** Everything touches **KEYS[1]** only. In Redis Cluster that is required: a script may only touch keys that live in one slot, and it must declare them in **KEYS**.
- **Self-cleaning.** The **PEXPIRE** is slightly longer than the time to refill from empty. When it fires, the client's state is gone and the next request starts with a full bucket, which is exactly what the maths would have given.

10.3.6 The limiter component

```
TokenBucketLimiter.java JAVA

package com.shopstream.ratelimit;

import java.util.List;
import org.springframework.core.io.ClassPathResource;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.core.script.DefaultRedisScript;
import org.springframework.stereotype.Component;

@Component
public class TokenBucketLimiter {

    public record Decision(boolean allowed, long remaining, long retryAfterMs, long fullInMs) {
    }

    private final StringRedisTemplate redis;
    private final DefaultRedisScript<List> script = new DefaultRedisScript<>();

    public TokenBucketLimiter(StringRedisTemplate redis) {
        this.redis = redis;
        script.setLocation(new ClassPathResource("scripts/token_bucket.lua"));
        script.setResultType(List.class);
    }

    public Decision tryAcquire(String identity, RateLimitProperties.Plan plan, int cost) {
        @SuppressWarnings("unchecked")
        List<Long> r = (List<Long>) redis.execute(script, List.of("rl:" + identity),
            String.valueOf(plan.capacity()),
            String.valueOf(plan.refillPerSecond()),
            String.valueOf(cost));
        return new Decision(r.get(0) == 1L, r.get(1), r.get(2), r.get(3));
    }
}
```

`DefaultRedisScript` computes the script's SHA-1 once. Spring Data Redis sends `EVALSHA` and, if the server answers with `NOSCRIPT` (after a restart or `SCRIPT FLUSH`), transparently falls back to `EVAL` and the server caches the script again. You never manage the SHA yourself.

10.3.7 The interceptor

A `HandlerInterceptor` runs before the controller and sees the request and response, which is exactly the right place for a limiter that must add headers. A servlet `Filter` would also work and runs earlier (before Spring MVC mapping), which matters if you want to protect static resources too.

```
RateLimitInterceptor.java JAVA

package com.shopstream.ratelimit;

import io.micrometer.core.instrument.MeterRegistry;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.dao.DataAccessException;
import org.springframework.stereotype.Component;
import org.springframework.web.servlet.HandlerInterceptor;

@Component
public class RateLimitInterceptor implements HandlerInterceptor {

    private static final Logger log = LoggerFactory.getLogger(RateLimitInterceptor.class);

    private final TokenBucketLimiter limiter;
    private final RateLimitProperties props;
    private final MeterRegistry metrics;

    public RateLimitInterceptor(TokenBucketLimiter limiter, RateLimitProperties props,
        MeterRegistry metrics) {
        this.limiter = limiter;
        this.props = props;
        this.metrics = metrics;
    }
}
```

RateLimitInterceptor.java (continued)

JAVA

```

@Override
public boolean preHandle(HttpServletRequest req, HttpServletResponse resp, Object handler)
    throws Exception {
    String apiKey = req.getHeader("X-API-Key");
    String identity;
    String planName;
    if (apiKey == null) {
        identity = "ip:" + req.getRemoteAddr();
        planName = "anonymous";
    } else {
        planName = props.keys().get(apiKey);
        if (planName == null) {
            resp.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Unknown API key");
            return false;
        }
        identity = "key:" + apiKey;
    }
    RateLimitProperties.Plan plan = props.plans().get(planName);
    int cost = props.routeCosts().getOrDefault(req.getRequestURI(), 1);

    TokenBucketLimiter.Decision d;
    try {
        d = limiter.tryAcquire(identity, plan, cost);
    } catch (DataAccessException e) {
        metrics.counter("ratelimit.errors").increment();
        log.warn("Rate limiter unavailable: {}", e.toString());
        if (props.failOpen()) {
            return true;
        }
        resp.sendError(HttpServletResponse.SC_SERVICE_UNAVAILABLE, "Rate limiter unavailable");
        return false;
    }

    resp.setHeader("X-RateLimit-Limit", String.valueOf(plan.capacity()));
    resp.setHeader("X-RateLimit-Remaining", String.valueOf(d.remaining()));
    if (d.allowed()) {
        metrics.counter("ratelimit.requests", "plan", planName, "result", "allowed")
            .increment();
        return true;
    }
    long retryAfterSeconds = Math.max(1, (d.retryAfterMs() + 999) / 1000);
    resp.setHeader("Retry-After", String.valueOf(retryAfterSeconds));
    resp.setStatus(429);
    resp.setContentType("application/json");
    resp.getWriter().write(
        "{\"error\":\"rate_limited\",\"retryAfterMs\":\"" + d.retryAfterMs() + "\"}");
    metrics.counter("ratelimit.requests", "plan", planName, "result", "denied")
        .increment();
    return false;
}
}

```

WebConfig.java

JAVA

```

package com.shopstream.ratelimit;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    private final RateLimitInterceptor interceptor;

    public WebConfig(RateLimitInterceptor interceptor) {
        this.interceptor = interceptor;
    }

    @Override

```


WebConfig.java (continued)

JAVA

```

    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(interceptor).addPathPatterns("/api/**");
    }
}

```

DemoController.java

JAVA

```

package com.shopstream.ratelimit;

import java.util.Map;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api")
public class DemoController {

    @GetMapping("/ping")
    public Map<String, String> ping() {
        return Map.of("pong", "ok");
    }

    @GetMapping("/search")
    public Map<String, String> search() {
        return Map.of("results", "expensive");
    }
}

```

RateLimiterApplication.java

JAVA

```

package com.shopstream.ratelimit;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.EnableConfigurationProperties;

@SpringBootApplication
@EnableConfigurationProperties(RateLimitProperties.class)
public class RateLimiterApplication {

    public static void main(String[] args) {
        SpringApplication.run(RateLimiterApplication.class, args);
    }
}

```

What to notice in the interceptor:

- **Identity before limiting.** An unknown API key is a **401**, not a free pass to the anonymous plan; otherwise attackers could rotate fake keys to get fresh buckets.
- **Fail-open versus fail-closed is a business decision.** For a public catalogue, blocking all traffic because the limiter is down is worse than briefly having no limit. For a login or payment endpoint you may choose fail-closed. The code makes it one property.
- **Headers on every response, Retry-After only on 429.** Clients can pace themselves before they are blocked. **Retry-After** is rounded up to whole seconds, as the HTTP header requires.
- **The route cost is looked up by exact URI.** For real routes with path variables, use the matched pattern from the request attribute instead; see the extensions.

WATCH OUT

`req.getRemoteAddr()` behind a load balancer or gateway is the balancer's address, so every anonymous user would share one bucket. Behind a trusted proxy, read **X-Forwarded-For** (or configure `server.forward-headers-strategy`), and never trust that header from the open internet, where clients can forge it.

10.3.8 Verify with redis-cli

Start Redis and the service, then spend the anonymous bucket (capacity 10) quickly:

Burst, then denial

TERMINAL

```
$ for i in $(seq 1 12); do \
  curl -s -o /dev/null -w "%{http_code}" localhost:8080/api/ping; done; echo
200 200 200 200 200 200 200 200 200 200 200 429 429
$ curl -si localhost:8080/api/ping | head -8
HTTP/1.1 429
X-RateLimit-Limit: 10
X-RateLimit-Remaining: 0
Retry-After: 1
Content-Type: application/json

{"error":"rate_limited","retryAfterMs":388}
```

Inspect the bucket

TERMINAL

```
127.0.0.1:6379> KEYS rl:*
1) "rl:ip:0:0:0:0:0:0:0:1"
127.0.0.1:6379> HGETALL rl:ip:0:0:0:0:0:0:0:1
1) "tokens"
2) "0.4"
3) "ts"
4) "1760100000123"
127.0.0.1:6379> PTTL rl:ip:0:0:0:0:0:0:0:1
(integer) 5601
```

NOTE

KEYS is fine on an empty development instance and is the quickest way to look; on a busy server use **SCAN 0 MATCH rl:* COUNT 100** instead. The IPv6 loopback address in the key (**0:0:0:0:0:0:0:1**) is what you see when **curl** connects to **localhost** over IPv6; the exact values in your output will differ.

Now try the paid plan and the weighted route:

Plans and weights

TERMINAL

```
$ curl -s -H "X-API-Key: demo-pro" -o /dev/null -D - localhost:8080/api/search | grep X-Rate
X-RateLimit-Limit: 200
X-RateLimit-Remaining: 195
$ curl -s -o /dev/null -w "%{http_code}\n" -H "X-API-Key: wrong" localhost:8080/api/ping
401
```

The search call cost 5 tokens, so 200 became 195.

10.3.9 Tests

We reuse the same **RedisTestConfig** pattern as Project A (a **GenericContainer<>("redis:8")** bean with **@ServiceConnection(name = "redis")**). The tests set tiny plans so they run in milliseconds. To call the running server we use the JDK **HttpClient** and the **local.server.port** property, which keeps the test independent of Spring Boot 4's reorganised web-test packages.

RateLimiterIT.java

JAVA

```
package com.shopstream.ratelimit;

import static org.assertj.core.api.Assertions.assertThat;

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
```

RateLimiterIT.java (continued)

JAVA

```

import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;

@SpringBootTest(
    webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT,
    properties = {
        "ratelimit.plans.tiny.capacity=5",
        "ratelimit.plans.tiny.refill-per-second=0.001",
        "ratelimit.plans.quick.capacity=2",
        "ratelimit.plans.quick.refill-per-second=20",
        "ratelimit.keys.k-tiny=tiny"
    })
@Import(RedisTestConfig.class)
class RateLimiterIT {

    @Value("${local.server.port}") int port;
    @Autowired TokenBucketLimiter limiter;
    @Autowired RateLimitProperties props;

    @Test
    void bucketAllowsCapacityThenDenies() {
        var plan = props.plans().get("tiny");
        for (int i = 0; i < 5; i++) {
            assertThat(limiter.tryAcquire("t:basic", plan, 1).allowed()).isTrue();
        }
        var denied = limiter.tryAcquire("t:basic", plan, 1);
        assertThat(denied.allowed()).isFalse();
        assertThat(denied.retryAfterMs()).isPositive();
    }

    @Test
    void bucketRefillsOverTime() throws Exception {
        var plan = props.plans().get("quick"); // 20 tokens per second
        limiter.tryAcquire("t:refill", plan, 1);
        limiter.tryAcquire("t:refill", plan, 1);
        assertThat(limiter.tryAcquire("t:refill", plan, 1).allowed()).isFalse();
        Thread.sleep(150); // about 3 tokens, capped at 2
        assertThat(limiter.tryAcquire("t:refill", plan, 1).allowed()).isTrue();
    }

    @Test
    void concurrentCallersNeverExceedCapacity() {
        var plan = props.plans().get("tiny"); // 5 tokens, practically no refill
        AtomicInteger allowed = new AtomicInteger();
        ExecutorService pool = Executors.newFixedThreadPool(32);
        try {
            List<CompletableFuture<Void>> calls = new ArrayList<>();
            for (int i = 0; i < 200; i++) {
                calls.add(CompletableFuture.runAsync(() -> {
                    if (limiter.tryAcquire("t:race", plan, 1).allowed()) {
                        allowed.incrementAndGet();
                    }
                }, pool));
            }
            calls.forEach(CompletableFuture::join);
        } finally {
            pool.shutdown();
        }
        assertThat(allowed.get()).isEqualTo(5);
    }

    @Test
    void httpContractIncludesHeadersAnd429() throws Exception {
        HttpClient http = HttpClient.newHttpClient();
        URI uri = URI.create("http://localhost:" + port + "/api/ping");
        HttpRequest req = HttpRequest.newBuilder(uri)
            .header("X-API-Key", "k-tiny").build();
        HttpResponse<String> last = null;
        for (int i = 0; i < 6; i++) {
            last = http.send(req, HttpResponse.BodyHandlers.ofString());
        }
        assertThat(last.statusCode()).isEqualTo(429);
    }
}

```

RateLimiterIT.java (continued)

JAVA

```

    assertThat(last.headers().firstValue("Retry-After")).isPresent();
    assertThat(last.headers().firstValue("X-RateLimit-Remaining")).hasValue("0");
}
}

```

The third test is the important one. It fires 200 calls from 32 threads at a bucket of 5 tokens and asserts that exactly 5 were allowed. If you replace the Lua script with a Java read-modify-write (**HGET**, compute, **HSET**), this test will fail intermittently and show you the race that the script prevents.

10.3.10 Demo: two clients, one noisy

Run two terminals. In the first, hammer the API as an anonymous client; in the second, use the pro key and watch it keep succeeding:

Isolation between clients

TERMINAL

```

$ seq 1 100 | xargs -P 10 -I{} curl -s -o /dev/null -w "%{http_code}\n" localhost:8080/api/ping \
  | sort | uniq -c
   11 200
   89 429
$ curl -s -o /dev/null -w "%{http_code}\n" -H "X-API-Key: demo-pro" localhost:8080/api/ping
200
$ curl -s "localhost:8080/actuator/metrics/ratelimit.requests?tag=result:denied" \
  | grep -o '"value":[0-9.]*'
"value":89.0

```

The anonymous count is 11 rather than 10 because a fraction of a token refills while the 100 requests are being sent. Your exact numbers will vary slightly.

INTERVIEW ANGLE

Why implement a rate limiter in a Lua script instead of Java code with GET and SET? Read-modify-write across the network is a race: two app servers read the same token count and both spend the last token. Redis runs a script as one atomic unit on its single command thread, so no other command interleaves between the read and the write. The alternative is optimistic locking with **WATCH/MULTI** and retries, which is slower under contention. A script also saves round trips: one instead of two or three.

REAL WORLD

Production limiters usually sit in the API gateway or a sidecar, not inside every service, and often combine a coarse per-IP limit at the edge with finer per-key limits behind it. The Redis part is almost always some variant of this script. The cost to watch is one Redis round trip (typically well under a millisecond on a local network) added to every request; keep Redis in the same zone as the gateway.

10.3.11 Extension ideas

- **Per-route buckets.** Use the matched pattern (`HandlerMapping.BEST_MATCHING_PATTERN_ATTRIBUTE`) in the key, giving `rl:{identity}:{route}`, so a busy route cannot exhaust a quiet one.
- **Local fallback.** When Redis is down, fall back to a small in-process Guava or Caffeine-based limiter rather than no limit at all.
- **Sliding window variant.** Implement the sorted-set sliding window log as a second script and compare memory with `MEMORY USAGE` for 1,000 clients.
- **Dynamic plans.** Store plans in a Redis hash (`plan:{name}`) so an operator can change limits without a deploy.
- **Cluster test.** Run the tests against a three-node Redis Cluster and confirm all keys of one identity still live in one slot (they do, since each script touches one key).

KEY POINTS

What this project proves:

- A token bucket needs two numbers of state per client and one atomic script; Lua gives you atomicity without locks.
- Use the Redis **TIME** command for time arithmetic inside the script; do not trust application clocks.
- Decide the failure policy (fail open or closed) explicitly and give Redis a short client timeout.
- Give clients the information to behave: **Retry-After** and remaining-count headers.
- Prove atomicity with a concurrency test that would fail for a non-atomic implementation.

10.4 Project C: A Flash-Sale Service That Never Oversells

10.4.1 Goal and requirements

At 12:00 sharp ShopStream drops 100 limited sneakers at half price, and 20,000 people click at once. Three things must hold, even with several service instances and a worker that may crash mid-job:

1. **Never oversell.** Exactly 100 units can be reserved, no matter how many requests race.
2. **Holds expire.** A shopper who reserves a pair has two minutes to pay; if they do not, the pair goes back on sale.
3. **Orders are processed exactly once in effect.** Confirmed reservations become order events on a Redis stream, consumed by a worker in a consumer group that survives crashes and redelivery.

Requirement	Design choice
Atomic stock check and decrement	One Lua script (reserve.lua) decides: sold out, already bought, or reserved
One unit per shopper	A set buyers:{sku} checked and updated in the same script
Reservation with deadline	Sorted set resvexp:{sku} scored by deadline in milliseconds, plus a hash with the details
Expiry that returns stock	A scheduled reaper finds due reservations; release.lua claims each one atomically and restores stock
Confirm exactly once	confirm.lua claims the reservation with ZREM ; whoever removes it wins, a retry gets "already confirmed"
Idempotent API	Idempotency-Key header: replaying a request returns the first answer and does not spend stock twice
Order pipeline	confirm.lua appends to stream:orders ; a worker reads it as a member of group fulfilment , then XACKs

Requirements.

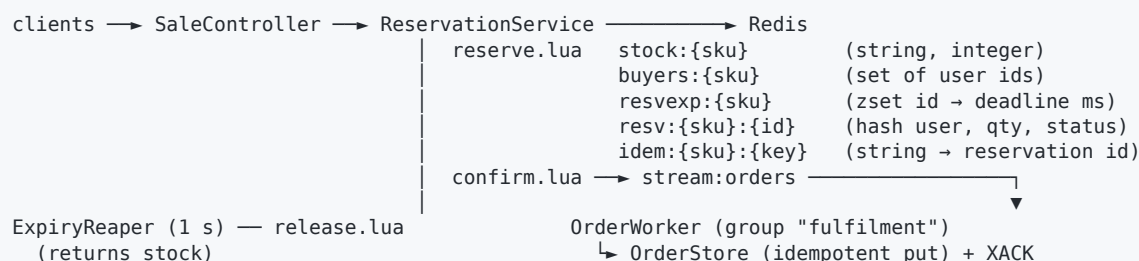
NOTE

In key names below, **{sku}** and **{id}** are placeholders, not Redis Cluster hash tags. This project is written for a single Redis (or primary with replicas), where scripts may touch any keys. To run it on Redis Cluster you would put a hash tag around the SKU in every key a script touches (for example **stock:{sku}**, literally with braces) so they share a slot, and send **XADD** from Java instead of from the script. The Scaling Part explains why.

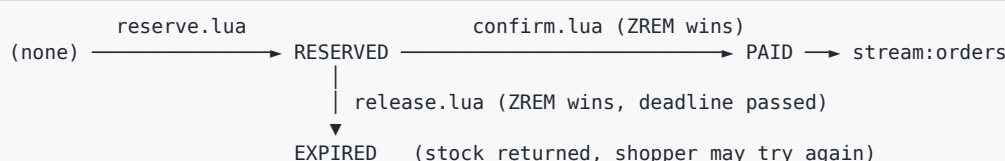
10.4.2 Why a script, not WATCH or a lock

There are three ways to decrement stock safely. A plain **DECR** and a check afterwards oversells briefly (the counter goes negative, and other requests see it). **WATCH** with **MULTI/EXEC** is correct but retries under contention, and a flash sale is pure contention: with 1,000 competing clients most **EXEC** calls fail and retry. A distributed lock works but serialises requests through an extra round trip each. A Lua script runs as one atomic step on Redis's single command thread: no retries, no lock, one round trip. A single Redis instance can run a very large number of such tiny scripts per second; in practice the network and the web tier usually saturate first, but measure your own setup with **redis-benchmark** or a load test.

10.4.3 Architecture



The reservation lifecycle is a tiny state machine. The sorted set is the arbiter: a reservation is "live" exactly while its id is in `resvexp:{sku}`. Both `confirm.lua` and `release.lua` try to `ZREM` the id first; Redis runs scripts one at a time, so exactly one of them succeeds. That removes the classic race between "shopper pays at 1:59" and "reaper expires at 2:00".



10.4.4 Project layout

```

flash-sale/
├── pom.xml                      (same dependencies as Project A, web + data-redis + test)
├── docker-compose.yml          (same Redis service)
├── src
│   ├── main
│   │   ├── java/com/shopstream/flashsale
│   │   │   ├── FlashSaleApplication.java
│   │   │   ├── SaleProperties.java
│   │   │   ├── ReservationService.java
│   │   │   ├── ExpiryReaper.java
│   │   │   ├── OrderWorker.java
│   │   │   ├── OrderStore.java
│   │   │   └── SaleController.java
│   │   └── resources
│   │       ├── application.yml
│   │       └── scripts/{reserve,confirm,release}.lua
│   └── test/java/com/shopstream/flashsale
│       ├── RedisTestConfig.java
│       └── FlashSaleIT.java
  
```

For this project use `appendonly yes` in `docker-compose.yml` (`command: ["redis-server", "--appendonly", "yes"]`): reservations and stock are state you do not want to lose on a restart. Remember that even with the AOF on `everysec` you can lose about a second of writes in a crash, so for money-adjacent state the order of record should still be your database, fed from the stream.

application.yml

YAML

```

spring:
  application:
    name: flash-sale
  data:
    redis:
      host: localhost
      port: 6379
  sale:
    hold: 2m
    stream: stream:orders
    group: fulfilment
  
```

SaleProperties.java

JAVA

```
package com.shopstream.flashsale;

import java.time.Duration;
import org.springframework.boot.context.properties.ConfigurationProperties;

@ConfigurationProperties(prefix = "sale")
public record SaleProperties(Duration hold, String stream, String group) {
}
```

10.4.5 The three Lua scripts

`reserve.lua` is the heart of the project. Read it as a decision list: first match wins.

scripts/reserve.lua

LUA

```
-- KEYS: 1 stock, 2 buyers, 3 expiry zset, 4 reservation hash, 5 idempotency key
-- ARGV: 1 reservation id, 2 user id, 3 quantity, 4 hold time in ms
-- Returns { code, reservationId }
-- 1 reserved | 3 duplicate request (replayed) | -1 already bought | -2 sold out | -3 no such sale
local previous = redis.call('GET', KEYS[5])
if previous then
    return { 3, previous }
end

local stock = redis.call('GET', KEYS[1])
if not stock then
    return { -3, '' }
end
if redis.call('SISMEMBER', KEYS[2], ARGV[2]) == 1 then
    return { -1, '' }
end
local qty = tonumber(ARGV[3])
if tonumber(stock) < qty then
    return { -2, '' }
end

local t = redis.call('TIME')
local now = t[1] * 1000 + math.floor(t[2] / 1000)

redis.call('DECRBY', KEYS[1], qty)
redis.call('SADD', KEYS[2], ARGV[2])
redis.call('ZADD', KEYS[3], now + tonumber(ARGV[4]), ARGV[1])
redis.call('HSET', KEYS[4], 'user', ARGV[2], 'qty', ARGV[3], 'status', 'RESERVED')
redis.call('EXPIRE', KEYS[4], 86400)
redis.call('SET', KEYS[5], ARGV[1], 'EX', 86400)
return { 1, ARGV[1] }
```

scripts/confirm.lua

LUA

```
-- KEYS: 1 expiry zset, 2 reservation hash, 3 orders stream
-- ARGV: 1 reservation id, 2 sku
-- Returns 1 confirmed | 2 already confirmed | 0 expired or unknown
local deadline = redis.call('ZSCORE', KEYS[1], ARGV[1])
if not deadline then
    if redis.call('HGET', KEYS[2], 'status') == 'PAID' then
        return 2
    end
    return 0
end
local t = redis.call('TIME')
local now = t[1] * 1000 + math.floor(t[2] / 1000)
if tonumber(deadline) < now then
    return 0
end

redis.call('ZREM', KEYS[1], ARGV[1])
local f = redis.call('HMGET', KEYS[2], 'user', 'qty')
redis.call('HSET', KEYS[2], 'status', 'PAID')
redis.call('XADD', KEYS[3], '*', 'orderId', ARGV[1], 'sku', ARGV[2], 'user', f[1], 'qty', f[2])
return 1
```

scripts/release.lua

LUA

```
-- KEYS: 1 stock, 2 buyers, 3 expiry zset, 4 reservation hash
-- ARGV: 1 reservation id
-- Returns 1 if this call returned the stock, 0 if it was not due or someone else won
local deadline = redis.call('ZSCORE', KEYS[3], ARGV[1])
if not deadline then
    return 0
end
local t = redis.call('TIME')
local now = t[1] * 1000 + math.floor(t[2] / 1000)
if tonumber(deadline) > now then
    return 0
end

redis.call('ZREM', KEYS[3], ARGV[1])
local f = redis.call('HMGET', KEYS[4], 'user', 'qty')
redis.call('INCRBY', KEYS[1], f[2])
redis.call('SREM', KEYS[2], f[1])
redis.call('HSET', KEYS[4], 'status', 'EXPIRED')
return 1
```

What to notice:

- **Everything in `reserve.lua` is decided before anything is written.** All checks run first, then the five writes; there is no early return after a partial write, so the script never leaves half-applied state. (Redis scripts are atomic in the sense that nothing else runs in between, but they are not rolled back if a later command errors, so design them check-then-write.)
- **`confirm.lua` re-checks the deadline.** Between the reaper's runs, an overdue reservation still sits in the sorted set. Checking the time inside the script means a payment at second 121 is refused even if the reaper has not noticed yet.
- **Idempotency is part of the same script.** The first thing `reserve.lua` does is look up the replay key. Doing that in Java before the script would leave a window where two identical requests both pass the check.
- **Quantities are strings in the hash.** `HMGET` returns strings; `INCRBY` accepts a numeric string. Redis would reject a value that is not an integer, which would abort the script, so validate `qty` in Java first.
- **Long-running scripts block everything.** These run in microseconds. Never put a loop over thousands of reservations into one script; the reaper processes ids one script call at a time for this reason.

10.4.6 The reservation service

ReservationService.java

JAVA

```
package com.shopstream.flashsale;

import java.time.Duration;
import java.util.List;
import java.util.Set;
import java.util.UUID;
import org.springframework.core.io.ClassPathResource;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.core.script.DefaultRedisScript;
import org.springframework.stereotype.Service;

@Service
public class ReservationService {

    public enum Outcome { RESERVED, DUPLICATE_REQUEST, ALREADY_BOUGHT, SOLD_OUT, NO_SUCH_SALE }

    public enum ConfirmOutcome { CONFIRMED, ALREADY_CONFIRMED, EXPIRED }

    public record Reservation(Outcome outcome, String reservationId) {
    }

    private final StringRedisTemplate redis;
    private final SaleProperties props;
    private final DefaultRedisScript<List> reserve = script("reserve");
    private final DefaultRedisScript<Long> confirm = longScript("confirm");
    private final DefaultRedisScript<Long> release = longScript("release");

    public ReservationService(StringRedisTemplate redis, SaleProperties props) {
        this.redis = redis;
        this.props = props;
    }

    private static DefaultRedisScript<List> script(String name) {
        var s = new DefaultRedisScript<List>();
        s.setLocation(new ClassPathResource("scripts/" + name + ".lua"));
        s.setResultType(List.class);
        return s;
    }

    private static DefaultRedisScript<Long> longScript(String name) {
        var s = new DefaultRedisScript<Long>();
        s.setLocation(new ClassPathResource("scripts/" + name + ".lua"));
        s.setResultType(Long.class);
        return s;
    }

    /** (Re)opens a sale with the given stock. Resets buyers and pending holds. */
    public void openSale(String sku, int units) {
        redis.delete(List.of("buyers:" + sku, "resvexp:" + sku));
        redis.opsForValue().set("stock:" + sku, String.valueOf(units));
        redis.opsForSet().add("sales:active", sku);
    }

    public long stock(String sku) {
        String v = redis.opsForValue().get("stock:" + sku);
        return v == null ? 0 : Long.parseLong(v);
    }

    public Set<String> activeSales() {
        Set<String> s = redis.opsForSet().members("sales:active");
        return s == null ? Set.of() : s;
    }

    public Reservation reserve(String sku, String userId, int qty, String idempotencyKey) {
        String id = UUID.randomUUID().toString();
        String idem = idempotencyKey != null ? idempotencyKey : id;
        @SuppressWarnings("unchecked")
        List<Object> r = (List<Object>) redis.execute(reserve,
            List.of("stock:" + sku, "buyers:" + sku, "resvexp:" + sku,
                "resv:" + sku + ":" + id, "idem:" + sku + ":" + idem),
            id, userId, String.valueOf(qty), String.valueOf(props.hold().toMillis()));
        long code = (Long) r.get(0);
        String reservationId = (String) r.get(1);
    }
}
```

ReservationService.java (continued)

JAVA

```

        Outcome outcome = switch ((int) code) {
            case 1 -> Outcome.RESERVED;
            case 3 -> Outcome.DUPLICATE_REQUEST;
            case -1 -> Outcome.ALREADY_BOUGHT;
            case -2 -> Outcome.SOLD_OUT;
            default -> Outcome.NO_SUCH_SALE;
        };
        return new Reservation(outcome, reservationId);
    }

    public ConfirmOutcome confirm(String sku, String reservationId) {
        Long r = redis.execute(confirm,
            List.of("resvexp:" + sku, "resv:" + sku + ":" + reservationId, props.stream()),
            reservationId, sku);
        return r == null || r == 0 ? ConfirmOutcome.EXPIRED
            : r == 1 ? ConfirmOutcome.CONFIRMED : ConfirmOutcome.ALREADY_CONFIRMED;
    }

    /** Called by the reaper. Returns how many reservations had their stock returned. */
    public int releaseExpired(String sku) {
        long horizon = System.currentTimeMillis() + 1000; // script re-checks with Redis TIME
        Set<String> due = redis.opsForZSet().rangeByScore("resvexp:" + sku, 0, horizon, 0, 100);
        int released = 0;
        if (due == null) {
            return 0;
        }
        for (String id : due) {
            Long r = redis.execute(release,
                List.of("stock:" + sku, "buyers:" + sku, "resvexp:" + sku,
                    "resv:" + sku + ":" + id), id);
            if (r != null && r == 1L) {
                released++;
            }
        }
        return released;
    }
}

```

ExpiryReaper.java

JAVA

```

package com.shopstream.flashsale;

import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

@Component
public class ExpiryReaper {

    private final ReservationService reservations;

    public ExpiryReaper(ReservationService reservations) {
        this.reservations = reservations;
    }

    @Scheduled(fixedDelay = 1000)
    public void run() {
        for (String sku : reservations.activeSales()) {
            reservations.releaseExpired(sku);
        }
    }
}

```

What to notice: the reaper is safe to run on every instance at once. If two instances pick the same expired reservation, `release.lua` lets only one succeed (`ZREM` wins). That is the same trick as the lock-free claims in the job-queue pattern, and it means you do not need leader election. The `horizon` is slightly ahead of local time only to fetch candidates; the script is the authority on whether the deadline has passed.

10.4.7 The controller

```

SaleController.java
package com.shopstream.flashsale;

import java.util.Map;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/sales/{sku}")
public class SaleController {

    public record ReserveRequest(String userId, int qty) {
    }

    private final ReservationService service;

    public SaleController(ReservationService service) {
        this.service = service;
    }

    @PostMapping
    public Map<String, Object> open(@PathVariable String sku, @RequestParam int stock) {
        service.openSale(sku, stock);
        return Map.of("sku", sku, "stock", stock);
    }

    @GetMapping
    public Map<String, Object> state(@PathVariable String sku) {
        return Map.of("sku", sku, "stock", service.stock(sku));
    }

    @PostMapping("/reservations")
    public ResponseEntity<Map<String, Object>> reserve(
        @PathVariable String sku, @RequestBody ReserveRequest body,
        @RequestHeader(value = "Idempotency-Key", required = false) String key) {
        if (body.userId() == null || body.qty() < 1 || body.qty() > 2) {
            return ResponseEntity.badRequest().body(Map.of("error", "invalid_request"));
        }
        var r = service.reserve(sku, body.userId(), body.qty(), key);
        return switch (r.outcome()) {
            case RESERVED -> ResponseEntity.status(201)
                .body(Map.of("reservationId", r.reservationId()));
            case DUPLICATE_REQUEST -> ResponseEntity.ok(Map.of("reservationId", r.reservationId()));
            case ALREADY_BOUGHT -> ResponseEntity.status(409)
                .body(Map.of("error", "already_bought"));
            case SOLD_OUT -> ResponseEntity.status(409).body(Map.of("error", "sold_out"));
            case NO_SUCH_SALE -> ResponseEntity.status(404).body(Map.of("error", "no_such_sale"));
        };
    }

    @PostMapping("/reservations/{id}/confirm")
    public ResponseEntity<Map<String, Object>> confirm(@PathVariable String sku,
        @PathVariable String id) {
        return switch (service.confirm(sku, id)) {
            case CONFIRMED, ALREADY_CONFIRMED -> ResponseEntity.ok(Map.of("status", "PAID"));
            case EXPIRED -> ResponseEntity.status(410).body(Map.of("error", "reservation_expired"));
        };
    }
}

```

Why 409 Conflict for sold out and 410 Gone for an expired hold? They are conventions, not rules. The point is that a client can tell "you lost the race" (stop, show the sold-out page) from "your hold lapsed" (offer to try again) without parsing messages.

10.4.8 The order pipeline: stream and consumer group

`confirm.lua` appends one entry per paid reservation to `stream:orders`. The worker is a member of consumer group `fulfilment`. A stream gives you what a plain list cannot: entries stay after delivery, each consumer has a list of delivered-but-unacknowledged entries (the pending entries list, PEL), and a crashed worker's work can be picked up again.

OrderStore.java

JAVA

```
package com.shopstream.flashsale;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import org.springframework.stereotype.Component;

/** Stands in for the orders table. putIfAbsent makes processing idempotent. */
@Component
public class OrderStore {

    public record Order(String orderId, String sku, String userId, int qty) {
    }

    private final Map<String, Order> orders = new ConcurrentHashMap<>();

    public boolean save(Order order) {
        return orders.putIfAbsent(order.orderId(), order) == null;
    }

    public int count() {
        return orders.size();
    }
}
```

OrderWorker.java

JAVA

```
package com.shopstream.flashsale;

import jakarta.annotation.PostConstruct;
import java.util.List;
import java.util.Map;
import java.util.UUID;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.data.redis.RedisSystemException;
import org.springframework.data.redis.connection.stream.Consumer;
import org.springframework.data.redis.connection.stream.MapRecord;
import org.springframework.data.redis.connection.stream.ReadOffset;
import org.springframework.data.redis.connection.stream.StreamOffset;
import org.springframework.data.redis.connection.stream.StreamReadOptions;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

@Component
public class OrderWorker {

    private static final Logger log = LoggerFactory.getLogger(OrderWorker.class);

    private final StringRedisTemplate redis;
    private final SaleProperties props;
    private final OrderStore store;
    private final Consumer consumer;

    public OrderWorker(StringRedisTemplate redis, SaleProperties props, OrderStore store) {
        this.redis = redis;
        this.props = props;
        this.store = store;
        this.consumer = Consumer.from(props.group(), "worker-" + UUID.randomUUID());
    }

    @PostConstruct
    void createGroup() {
        try {
            redis.opsForStream().createGroup(props.stream(), ReadOffset.from("0"), props.group());
        } catch (RedisSystemException e) {
            String msg = String.valueOf(e.getMessage()) + e.getCause();
        }
    }
}
```

OrderWorker.java (continued)

JAVA

```

        if (!msg.contains("BUSYGROUP")) {
            throw e; // any error other than "group already exists" is real
        }
    }
}

@Scheduled(fixedDelay = 200)
public void poll() {
    // 1. entries delivered to this consumer earlier but never acknowledged (offset "0")
    handle(read(ReadOffset.from("0")));
    // 2. new entries nobody has received yet (">")
    handle(read(ReadOffset.lastConsumed()));
}

private List<MapRecord<String, Object, Object>> read(ReadOffset offset) {
    var options = StreamReadOptions.empty().count(50);
    return redis.opsForStream().read(consumer, options,
        StreamOffset.create(props.stream(), offset));
}

private void handle(List<MapRecord<String, Object, Object>> records) {
    if (records == null) {
        return;
    }
    for (var rec : records) {
        Map<Object, Object> f = rec.getValue();
        var order = new OrderStore.Order((String) f.get("orderId"), (String) f.get("sku"),
            (String) f.get("user"), Integer.parseInt((String) f.get("qty")));
        if (!store.save(order)) {
            log.info("Order {} was already processed; acknowledging again", order.orderId());
        }
        redis.opsForStream().acknowledge(props.stream(), props.group(), rec.getId());
    }
}
}

```

What to notice in the worker:

- **Pending first, then new.** Reading offset 0 returns this consumer's own unacknowledged entries; reading > returns entries never delivered to anyone. If the process dies between saving an order and **XACK**, the entry is delivered again on restart. Because the consumer name is random in this demo, a restarted worker is a new consumer and would not see the old PEL; in production use a stable name (the pod name) or run **XAUTOCLAIM** to adopt entries idle for too long. See the extensions.
- **Processing is idempotent.** **putIfAbsent** on the order id means a redelivered entry cannot create a second order. Streams give you at-least-once delivery; idempotent handlers turn that into exactly-once effect.
- **Acknowledge only after the work.** **XACK** follows the save. Reversing the order would lose the order on a crash between the two.
- **createGroup creates the stream if needed.** Spring Data Redis issues **XGROUP CREATE** with **MKSTREAM**, so starting the worker before any sale works. A restart finds the group already present and the server answers **BUSYGROUP**, which we tolerate.

GOTCHA

A consumer group that nobody reads is not free: unacknowledged entries stay in the PEL and the stream itself keeps growing. Trim the stream (**XTRIM**, or **XADD ... MAXLEN ~**) once you are sure every group has acknowledged. Since Redis 8.2 you can also use **XACKDEL** and the **ACKED** option to delete entries only once all groups have acknowledged them; check the command pages for exact syntax before relying on it.

FlashSaleApplication.java

JAVA

```

package com.shopstream.flashsale;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.scheduling.annotation.EnableScheduling;

@SpringBootApplication
@EnableScheduling
@EnableConfigurationProperties(SaleProperties.class)

```

FlashSaleApplication.java (continued)

JAVA

```
public class FlashSaleApplication {

    public static void main(String[] args) {
        SpringApplication.run(FlashSaleApplication.class, args);
    }
}
```

10.4.9 Verify with redis-cli

Open a sale of three units, reserve and confirm one, and look at the state:

Walk one reservation through Redis

TERMINAL

```
$ curl -s -X POST "localhost:8080/api/sales/sneaker-1?stock=3"
{"sku":"sneaker-1","stock":3}
$ curl -s -X POST localhost:8080/api/sales/sneaker-1/reservations \
  -H 'Content-Type: application/json' -d '{"userId":"u1","qty":1}'
{"reservationId":"c0a8f1de-5d3c-4b6e-9a53-7a4e2c1b9f10"}
127.0.0.1:6379> GET stock:sneaker-1
"2"
127.0.0.1:6379> SMEMBERS buyers:sneaker-1
1) "u1"
127.0.0.1:6379> ZRANGE resvexp:sneaker-1 0 -1 WITHSCORES
1) "c0a8f1de-5d3c-4b6e-9a53-7a4e2c1b9f10"
2) "1760100120123"
127.0.0.1:6379> HGETALL resv:sneaker-1:c0a8f1de-5d3c-4b6e-9a53-7a4e2c1b9f10
1) "user"
2) "u1"
3) "qty"
4) "1"
5) "status"
6) "RESERVED"
```

After the shopper pays

TERMINAL

```
$ curl -s -X POST \
  localhost:8080/api/sales/sneaker-1/reservations/c0a8f1de-5d3c-4b6e-9a53-7a4e2c1b9f10/confirm
{"status":"PAID"}
127.0.0.1:6379> XLEN stream:orders
(integer) 1
127.0.0.1:6379> XINFO GROUPS stream:orders
1) 1) "name"
   2) "fulfilment"
   3) "consumers"
   4) (integer) 1
   5) "pending"
   6) (integer) 0
   7) "last-delivered-id"
   8) "1760100005432-0"
   9) "entries-read"
  10) (integer) 1
  11) "lag"
  12) (integer) 0
127.0.0.1:6379> ZCARD resvexp:sneaker-1
(integer) 0
```

`pending 0` and `lag 0` mean the worker has read and acknowledged everything. If the worker is stopped, new confirmations raise `lag`; if it reads but fails before `XACK`, they raise `pending`. Those two numbers are the first things to monitor on a stream pipeline.

10.4.10 Tests: the oversell test

The test that matters is brutal and short: 500 distinct shoppers race for 100 units from 64 threads.

FlashSaleIT.java

JAVA

```
package com.shopstream.flashsale;

import static org.assertj.core.api.Assertions.assertThat;
import static org.awaitility.Awaitility.await;

import java.time.Duration;
```

FlashSaleIT.java (continued)

JAVA

```

import java.util.List;
import java.util.Queue;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.stream.IntStream;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;
import org.springframework.data.redis.core.StringRedisTemplate;

@SpringBootTest(properties = {"sale.hold=2s", "spring.task.scheduling.pool.size=2"})
@Import(RedisTestConfig.class)
class FlashSaleIT {

    @Autowired ReservationService service;
    @Autowired OrderStore orders;
    @Autowired StringRedisTemplate redis;

    @Test
    void neverOversells() throws Exception {
        service.openSale("race", 100);
        Queue<ReservationService.Reservation> results = new ConcurrentLinkedQueue<>();
        ExecutorService pool = Executors.newFixedThreadPool(64);
        try {
            List<Future<?>> futures = IntStream.range(0, 500)
                .mapToObj(i -> pool.submit(
                    () -> results.add(service.reserve("race", "user-" + i, 1, null))))
                .toList();
            for (Future<?> f : futures) {
                f.get();
            }
        } finally {
            pool.shutdown();
        }
        long reserved = results.stream()
            .filter(r -> r.outcome() == ReservationService.Outcome.RESERVED).count();
        long soldOut = results.stream()
            .filter(r -> r.outcome() == ReservationService.Outcome.SOLD_OUT).count();
        assertEquals(reserved, 100);
        assertEquals(soldOut, 400);
        assertEquals(service.stock("race"), 0);
    }

    @Test
    void sameIdempotencyKeyDoesNotSpendStockTwice() {
        service.openSale("idem", 5);
        var first = service.reserve("idem", "alice", 1, "req-123");
        var replay = service.reserve("idem", "alice", 1, "req-123");
        assertEquals(first.outcome(), ReservationService.Outcome.RESERVED);
        assertEquals(replay.outcome(), ReservationService.Outcome.DUPLICATE_REQUEST);
        assertEquals(replay.reservationId(), first.reservationId());
        assertEquals(service.stock("idem"), 4);
    }

    @Test
    void oneUnitPerShopper() {
        service.openSale("once", 5);
        service.reserve("once", "bob", 1, null);
        assertEquals(service.reserve("once", "bob", 1, null).outcome(),
            ReservationService.Outcome.ALREADY_BOUGHT);
    }

    @Test
    void expiredHoldReturnsStockAndCannotBeConfirmed() throws Exception {
        service.openSale("hold", 1);
        var r = service.reserve("hold", "carol", 1, null);
        assertEquals(service.stock("hold"), 0);
        Thread.sleep(2300); // hold is 2 s
        assertEquals(service.confirm("hold", r.reservationId()),
            ReservationService.Outcome.EXPIRED);
        service.releaseExpired("hold"); // the background reaper may have won; either is fine
        assertEquals(service.stock("hold"), 1);
    }
}

```


FlashSaleIT.java (continued)

JAVA

```

    var retry = service.reserve("hold", "carol", 1, null); // carol may try again
    assertThat(retry.outcome()).isEqualTo(ReservationService.Outcome.RESERVED);
}

@Test
void confirmedReservationsBecomeOrdersExactlyOnce() {
    service.openSale("order", 10);
    int before = orders.count();
    for (int i = 0; i < 10; i++) {
        var r = service.reserve("order", "buyer-" + i, 1, null);
        service.confirm("order", r.reservationId());
        service.confirm("order", r.reservationId()); // a retry must be harmless
    }
    await().atMost(Duration.ofSeconds(10))
        .untilAsserted(() -> assertThat(orders.count() - before).isEqualTo(10));
    var summary = redis.opsForStream().pending("stream:orders", "fulfilment");
    Long pending = summary.getTotalPendingMessages();
    assertThat(pending).isZero();
}
}

```

Because the stream and sale keys persist inside one container for the whole test class, every test uses its own SKU (*race, idem, once, hold, order*) so tests do not disturb each other. The 500-way race uses distinct user ids on purpose; with a single user id the "one per shopper" rule would reject all but one request and the test would prove nothing about stock.

WATCH OUT

`StringRedisTemplate.opsForStream().pending(...)` is shown here to assert the worker acknowledged everything; if your Spring Data Redis version names this method differently, `X_PENDING` through a `RedisCallback` is an equally good check. Keep the assertion: a worker that processes but forgets `XACK` passes every other test and slowly fills the PEL in production.

10.4.11 Demo: the thundering herd

With the service running, open a sale of 100 and let 2,000 requests from 100 different users hit it with 50 in parallel:

Oversell check from the outside

TERMINAL

```

$ curl -s -X POST "localhost:8080/api/sales/air-max?stock=100"
{"sku":"air-max","stock":100}
$ seq 1 2000 | xargs -P 50 -I{} sh -c \
'curl -s -o /dev/null -w "%{http_code}\n" -X POST \
  localhost:8080/api/sales/air-max/reservations \
  -H "Content-Type: application/json" -d "{\"userId\":\"u{}\", \"qty\":1}\"' | sort | uniq -c
100 201
1900 409
$ curl -s localhost:8080/api/sales/air-max
{"sku":"air-max","stock":0}

```

Exactly 100 201 responses, no matter how often you repeat it. Now do the experiment that teaches the most: change `reserve.lua` into three separate Java calls (`GET`, compare, `DECRBY`) and run the demo again. You will see more than 100 successes and a negative stock. That is the oversell the script exists to prevent.

INTERVIEW ANGLE

How would you prevent overselling in a flash sale? Make the check and the decrement a single atomic operation. In Redis that is a Lua script (or `DECR` with a check of the returned value and compensation, which briefly shows negative stock). Avoid read-then-write from the application. Add reservation holds with a deadline so abandoned carts return stock, idempotency keys so retries do not double-spend, and treat Redis as the fast gate while the order database remains the system of record, reconciled from a durable stream.

REAL WORLD

Real sales add layers in front of this: a queue or waiting room that admits users at a controlled rate, bot protection, CDN caching of the product page so only the reserve call reaches Redis, and reconciliation jobs that compare Redis stock with the database. The atomic script at the centre looks very much like the one here.

10.4.12 Extension ideas

- **Stable consumer identity plus `XAUTOCLAIM`.** Name consumers after the pod, and add a second scheduled task that calls `XAUTOCLAIM` for entries idle more than 60 seconds so a dead worker's backlog is adopted by a live one.
- **Dead-letter stream.** After N delivery attempts (see `XPENDING` delivery counts), move the entry to `stream:orders:dead` and alert.
- **Persist to a database.** Replace `OrderStore` with a JPA repository and a unique constraint on the order id; the constraint is the final idempotency guard.
- **Waiting room.** Add a sorted set of queued users and admit 200 per second into the reserve endpoint.
- **Reconciliation.** A nightly job compares `stock + reserved + paid` with the initial stock for every sale and alerts on drift.

KEY POINTS

What this project proves:

- Atomic Lua scripts beat `WATCH` retries and locks for hot-counter workloads; design them check-first, write-last.
- Use one structure as the arbiter of state (here the expiry sorted set) so competing paths, confirm versus expire, cannot both win.
- Put idempotency inside the atomic step and make consumers idempotent as well; streams deliver at least once.
- Acknowledge after the work; watch `lag` and `pending` on every consumer group.
- Prove correctness with a concurrency test that would fail for the naive implementation.

10.5 Project D: A Live Leaderboard with Notifications

10.5.1 Goal and requirements

ShopStream runs a daily "Shopper Rewards" challenge: every purchase earns points, and a live leaderboard shows the top shoppers. When someone enters the top 10 or takes the lead, they get a notification; everybody watching the page sees the board update without refreshing. This project combines four Redis features that usually appear separately: a sorted set for ranking, a stream as the durable event log, Pub/Sub for live fan-out, and a HyperLogLog for counting unique players.

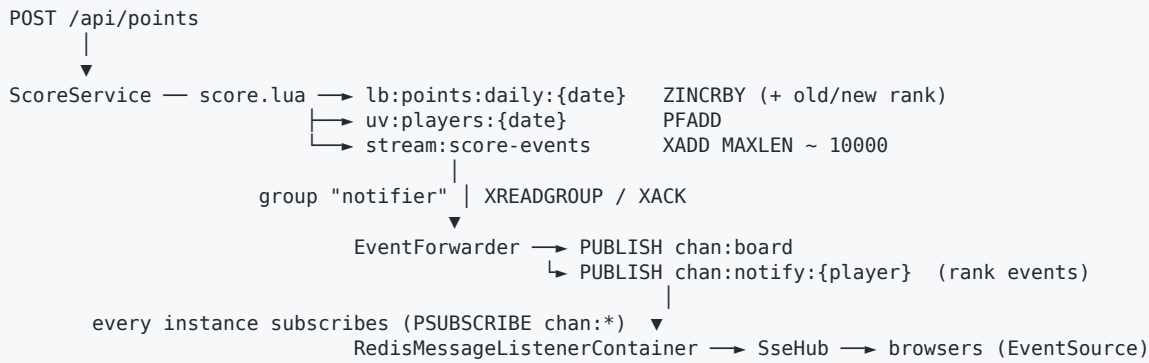
Requirement	Design choice
Ranked scores	Sorted set <code>lb:points:daily:{date}</code> ; <code>ZINCRBY</code> adds points, <code>ZREVRANGE</code> reads the top
Rank and neighbours	<code>ZREVRANK</code> for "you are 7th", then a short range around that position
Unique players per day	HyperLogLog <code>uv:players:{date}</code> with <code>PFADD</code> and <code>PFCOUNT</code> , about 12 KB per key at most
Durable event log	Stream <code>stream:score-events</code> , trimmed approximately to 10,000 entries
Reliable forwarding	A worker in consumer group <code>notifier</code> reads the stream and publishes to Pub/Sub, acknowledging after publish
Live push	Pub/Sub channels <code>chan:board</code> (everyone) and <code>chan:notify:{player}</code> (one player), received by every service instance and pushed to browsers over Server-Sent Events (SSE)
Atomicity	One Lua script updates the board, the HyperLogLog and the stream and reports the old and new rank

Requirements.

NOTE

Why not push straight from the web request to the browsers? Because with two or more service instances, the instance that handles the purchase is rarely the one holding a given browser's SSE connection. Pub/Sub is the bus that reaches every instance. And why the stream in the middle? Pub/Sub is fire-and-forget: a message published while an instance is restarting is gone. The stream keeps every event until a consumer acknowledges it, so the forwarding step can retry.

10.5.2 Architecture



10.5.3 Project layout

```
leaderboard/
├── pom.xml                (as Project A, web + data-redis + actuator + test)
├── docker-compose.yml     (same Redis service)
├── src
│   ├── main
│   │   ├── java/com/shopstream/leaderboard
│   │   │   ├── LeaderboardApplication.java
│   │   │   ├── ScoreService.java
│   │   │   ├── EventForwarder.java
│   │   │   ├── PubSubConfig.java
│   │   │   ├── SseHub.java
│   │   │   └── LeaderboardController.java
│   │   └── resources
│   │       ├── application.yml
│   │       ├── scripts/score.lua
│   │       └── static/index.html
│   └── test/java/com/shopstream/leaderboard
│       ├── RedisTestConfig.java
│       └── LeaderboardIT.java
```

The Redis keys of this project:

Key	Type	Purpose
lb:points:daily:{date}	zset	Player to points for one day, expires after 8 days
uv:players:{date}	HLL	Approximate count of distinct players who scored that day
stream:score-events	stream	Every scoring event, trimmed to about 10,000 entries
chan:board	channel	Live board updates for all viewers
chan:notify:{player}	channel	Personal notifications for one player

Keys.

10.5.4 The scoring script

Two things need atomicity: the old rank must be read before the increment and the new rank after it, with no other score change in between, and the three structures must be updated together.

scripts/score.lua

LUA

```
-- KEYS: 1 daily leaderboard, 2 unique-players HLL, 3 event stream
-- ARGV: 1 player id, 2 points to add, 3 board TTL in seconds
-- Returns { new score (string), old rank (0-based, -1 if new), new rank (0-based) }
local oldRank = redis.call('ZREVRANK', KEYS[1], ARGV[1])
local score = redis.call('ZINCRBY', KEYS[1], ARGV[2], ARGV[1])
local newRank = redis.call('ZREVRANK', KEYS[1], ARGV[1])
redis.call('EXPIRE', KEYS[1], tonumber(ARGV[3]))
redis.call('PFADD', KEYS[2], ARGV[1])
redis.call('EXPIRE', KEYS[2], tonumber(ARGV[3]))

local old = -1
if oldRank then
```

scripts/score.lua (continued)

LUA

```

    old = oldRank
end
redis.call('XADD', KEYS[3], 'MAXLEN', '~', '10000', '*',
           'player', ARGV[1], 'points', ARGV[2], 'score', score,
           'oldRank', tostring(old), 'newRank', tostring(newRank))
return { score, old, newRank }

```

What to notice: **ZREVRANK** returns a nil reply for a player not in the set, which a Lua script sees as **false**, so we convert it to **-1**. Ranks are 0-based in Redis; the Java side adds one for humans. **XADD ... MAXLEN ~ 10000** trims approximately, which is cheaper than exact trimming: Redis removes whole internal blocks, so the stream may hold a little more than 10,000 entries.

GOTCHA

Sorted sets order players with equal scores by member name (lexicographically). If "fairness" matters, the player who reached 500 points first should rank higher. A common trick encodes the time in the fractional part of the score, for example **points + (1 - timestamp / 1e13)**. Doubles hold about 15 to 16 significant digits, so check that your points and timestamps fit before you try it.

10.5.5 The score service

ScoreService.java

JAVA

```

package com.shopstream.leaderboard;

import java.time.Clock;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import java.util.Set;
import org.springframework.core.io.ClassPathResource;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.core.ZSetOperations.TypedTuple;
import org.springframework.data.redis.core.script.DefaultRedisScript;
import org.springframework.stereotype.Service;

@Service
public class ScoreService {

    public static final String STREAM = "stream:score-events";
    private static final long BOARD_TTL_SECONDS = 8L * 24 * 3600;

    public record Entry(long rank, String player, double score) {}

    public record ScoreResult(String player, double score, long oldRank, long newRank) {}

    private final StringRedisTemplate redis;
    private final Clock clock;
    private final DefaultRedisScript<List> script = new DefaultRedisScript<>();

    public ScoreService(StringRedisTemplate redis, Clock clock) {
        this.redis = redis;
        this.clock = clock;
        script.setLocation(new ClassPathResource("scripts/score.lua"));
        script.setResultType(List.class);
    }

    public ScoreResult addPoints(String player, long points) {
        LocalDate today = LocalDate.now(clock);
        @SuppressWarnings("unchecked")
        List<Object> r = (List<Object>) redis.execute(script,
            List.of(boardKey(today), uniqueKey(today), STREAM),
            player, String.valueOf(points), String.valueOf(BOARD_TTL_SECONDS));
        return new ScoreResult(player, Double.parseDouble((String) r.get(0)),
            (Long) r.get(1), (Long) r.get(2));
    }

    public List<Entry> top(int size) {
        String key = boardKey(LocalDate.now(clock));
        var tuples = redis.opsForZSet().reverseRangeWithScores(key, 0, size - 1);
        return toEntries(tuples, 1);
    }

    /** The player's rank (1-based) plus two neighbours above and below. */
    public Optional<List<Entry>> around(String player) {
        String key = boardKey(LocalDate.now(clock));
        Long rank = redis.opsForZSet().reverseRank(key, player);
        if (rank == null) {
            return Optional.empty();
        }
        long start = Math.max(0, rank - 2);
        var tuples = redis.opsForZSet().reverseRangeWithScores(key, start, rank + 2);
        return Optional.of(toEntries(tuples, start + 1));
    }

    public long uniquePlayersToday() {
        Long n = redis.opsForHyperLogLog().size(uniqueKey(LocalDate.now(clock)));
        return n == null ? 0 : n;
    }

    private static List<Entry> toEntries(Set<TypedTuple<String>> tuples, long firstRank) {
        List<Entry> out = new ArrayList<>();
        if (tuples == null) {

```

ScoreService.java (continued)

JAVA

```

        return out;
    }
    long rank = firstRank;
    for (TypedTuple<String> t : tuples) { // the returned Set keeps Redis order
        out.add(new Entry(rank++, t.getValue(), t.getScore() == null ? 0 : t.getScore()));
    }
    return out;
}

static String boardKey(LocalDate d) {
    return "lb:points:daily:" + d;
}

static String uniqueKey(LocalDate d) {
    return "uv:players:" + d;
}
}

```

What to notice: the sorted set operations cost $O(\log N)$ for the rank and $O(\log N + M)$ for a range of M entries, so a board with 10 million players answers "top 10" and "my rank" in microseconds to low milliseconds on the server. The `Set` returned by `reverseRangeWithScores` is a `LinkedHashSet`, so iteration order is Redis order; if you ever copy it into a `HashSet` the order is lost.

10.5.6 From stream to Pub/Sub

The forwarder is a consumer-group worker, structured exactly like the order worker in the flash-sale project: pending entries first, then new ones, acknowledge after the work. The "work" here is publishing.

EventForwarder.java

JAVA

```

package com.shopstream.leaderboard;

import jakarta.annotation.PostConstruct;
import java.util.List;
import java.util.Map;
import java.util.UUID;
import org.springframework.data.redis.RedisSystemException;
import org.springframework.data.redis.connection.stream.Consumer;
import org.springframework.data.redis.connection.stream.MapRecord;
import org.springframework.data.redis.connection.stream.ReadOffset;
import org.springframework.data.redis.connection.stream.StreamOffset;
import org.springframework.data.redis.connection.stream.StreamReadOptions;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import tools.jackson.databind.json.JsonMapper;

@Component
public class EventForwarder {

    static final String GROUP = "notifier";

    public record BoardEvent(String player, String score, int rank) {}

    public record Notification(String player, String message, int rank) {}

    private final StringRedisTemplate redis;
    private final JsonMapper json;
    private final Consumer consumer = Consumer.from(GROUP, "fwd-" + UUID.randomUUID());

    public EventForwarder(StringRedisTemplate redis, JsonMapper json) {
        this.redis = redis;
        this.json = json;
    }

    @PostConstruct
    void createGroup() {
        try {
            redis.opsForStream().createGroup(ScoreService.STREAM, ReadOffset.from("0"), GROUP);
        } catch (RedisSystemException e) {
            if (!(String.valueOf(e.getMessage()) + e.getCause()).contains("BUSYGROUP")) {}
        }
    }
}

```

EventForwarder.java (continued)

JAVA

```

        throw e;
    }
}

@Scheduled(fixedDelay = 100)
public void poll() {
    forward(read(ReadOffset.from("0")));           // our own unacknowledged entries
    forward(read(ReadOffset.lastConsumed()));      // new entries
}

private List<MapRecord<String, Object, Object>> read(ReadOffset offset) {
    return redis.opsForStream().read(consumer, StreamReadOptions.empty().count(100),
        StreamOffset.create(ScoreService.STREAM, offset));
}

private void forward(List<MapRecord<String, Object, Object>> records) {
    if (records == null) {
        return;
    }
    for (var rec : records) {
        Map<Object, Object> f = rec.getValue();
        String player = (String) f.get("player");
        int oldRank = Integer.parseInt((String) f.get("oldRank"));
        int newRank = Integer.parseInt((String) f.get("newRank"));

        var board = new BoardEvent(player, (String) f.get("score"), newRank + 1);
        redis.convertAndSend("chan:board", json.writeValueAsString(board));

        String message = null;
        if (newRank == 0 && oldRank != 0) {
            message = "You took the lead!";
        } else if (newRank < 10 && (oldRank < 0 || oldRank >= 10)) {
            message = "You entered the top 10!";
        }
        if (message != null) {
            redis.convertAndSend("chan:notify:" + player,
                json.writeValueAsString(new Notification(player, message, newRank + 1)));
        }
        redis.opsForStream().acknowledge(ScoreService.STREAM, GROUP, rec.getId());
    }
}
}

```

If the process dies after `convertAndSend` but before `acknowledge`, the entry is delivered again and viewers see a duplicate board update. For a leaderboard that is harmless (the board message carries the absolute score and rank, not a delta). Notifications are more sensitive; if duplicates would annoy users, record `"notified:{player}:{date}"` with `SET NX` before sending.

WATCH OUT

`Pub/Sub` delivers only to currently connected subscribers and does not buffer. If nobody is subscribed, `PUBLISH` returns `0` and the message is gone. That is why the durable record is the stream and why a browser that reconnects must reload `GET /api/leaderboard` to resynchronise instead of expecting missed events.

10.5.7 Receiving messages and pushing to browsers

Spring Boot does not create a `RedisMessageListenerContainer` for you. Declare one, subscribe with a pattern, and hand every message to the hub that owns the SSE connections.

PubSubConfig.java

JAVA

```

package com.shopstream.leaderboard;

import java.nio.charset.StandardCharsets;
import java.time.Clock;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.connection.Message;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.listener.PatternTopic;
import org.springframework.data.redis.listener.RedisMessageListenerContainer;

```

PubSubConfig.java (continued)

JAVA

```

@Configuration
public class PubSubConfig {

    @Bean
    Clock clock() {
        return Clock.systemDefaultZone();
    }

    @Bean
    RedisMessageListenerContainer listenerContainer(RedisConnectionFactory factory, SseHub hub) {
        var container = new RedisMessageListenerContainer();
        container.setConnectionFactory(factory);
        container.addMessageListener((Message message, byte[] pattern) -> {
            String channel = new String(message.getChannel(), StandardCharsets.UTF_8);
            String body = new String(message.getBody(), StandardCharsets.UTF_8);
            if (channel.equals("chan:board")) {
                hub.broadcast("leaderboard", body);
            } else if (channel.startsWith("chan:notify:")) {
                hub.send(channel.substring("chan:notify:".length()), "notification", body);
            }
        }, new PatternTopic("chan:*"));
        return container;
    }
}

```

SseHub.java

JAVA

```

package com.shopstream.leaderboard;

import java.io.IOException;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;
import org.springframework.stereotype.Component;
import org.springframework.web.servlet.mvc.method.annotation.SseEmitter;

/** Holds the SSE connections of THIS instance only. Other instances hold their own. */
@Component
public class SseHub {

    private record Subscriber(SseEmitter emitter, String player) {
    }

    private final List<Subscriber> subscribers = new CopyOnWriteArrayList<>();

    public SseEmitter subscribe(String player) {
        SseEmitter emitter = new SseEmitter(30 * 60 * 1000L); // 30 min
        Subscriber s = new Subscriber(emitter, player);
        subscribers.add(s);
        emitter.onCompletion(() -> subscribers.remove(s));
        emitter.onTimeout(() -> subscribers.remove(s));
        emitter.onError(e -> subscribers.remove(s));
        send(s, "hello", "connected"); // commits headers right away
        return emitter;
    }

    public void broadcast(String eventName, String data) {
        subscribers.forEach(s -> send(s, eventName, data));
    }

    public void sendTo(String player, String eventName, String data) {
        subscribers.stream().filter(s -> player.equals(s.player()))
            .forEach(s -> send(s, eventName, data));
    }

    public int connections() {
        return subscribers.size();
    }

    private void send(Subscriber s, String eventName, String data) {
        try {
            s.emitter().send(SseEmitter.event().name(eventName).data(data));
        } catch (IOException | IllegalStateException e) {
        }
    }
}

```

SseHub.java (continued)

JAVA

```
        subscribers.remove(s);                // client went away
    }
}
```

What to notice:

- **One pattern subscription, two kinds of channel.** `PSubscribe chan:*` receives `chan:board` and every `chan:notify:{player}`. Pattern subscriptions are convenient; with hundreds of thousands of personal channels you may prefer to subscribe per connected player instead, so Redis does less matching.
- **The container uses its own connection.** A connection in subscribe mode can run no other commands, so the container holds a dedicated one and delivers messages on a task executor thread. Your normal template calls use other connections.
- **The hub is local state.** A connection to instance A is only known to A. This is why every instance must subscribe: each instance forwards the Pub/Sub message to its own browsers.
- **Cleanup on every exit path.** Without `onCompletion`, `onTimeout` and `onError`, closed browsers leak emitters forever, and each broadcast then throws on dead connections.

10.5.8 The controller and a tiny front end

LeaderboardController.java

JAVA

```
package com.shopstream.leaderboard;

import java.util.List;
import java.util.Map;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.mvc.method.annotation.SseEmitter;

@RestController
@RequestMapping("/api")
public class LeaderboardController {

    public record PointsRequest(String playerId, long points) {
    }

    private final ScoreService scores;
    private final SseHub hub;

    public LeaderboardController(ScoreService scores, SseHub hub) {
        this.scores = scores;
        this.hub = hub;
    }

    @PostMapping("/points")
    public ResponseEntity<?> add(@RequestBody PointsRequest body) {
        if (body.playerId() == null || body.points() < 1 || body.points() > 10_000) {
            return ResponseEntity.badRequest().body(Map.of("error", "invalid_request"));
        }
        return ResponseEntity.ok(scores.addPoints(body.playerId(), body.points()));
    }

    @GetMapping("/leaderboard")
    public List<ScoreService.Entry> top(@RequestParam(defaultValue = "10") int size) {
        return scores.top(Math.min(Math.max(size, 1), 100));
    }

    @GetMapping("/players/{id}/rank")
    public ResponseEntity<List<ScoreService.Entry>> around(@PathVariable String id) {
        return scores.around(id).map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }

    @GetMapping("/stats/unique-players")
    public Map<String, Object> unique() {
        return Map.of("uniquePlayersToday", scores.uniquePlayersToday());
    }

    @GetMapping(path = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public SseEmitter stream(@RequestParam(required = false) String player) {
        return hub.subscribe(player);
    }
}
```

LeaderboardApplication.java

JAVA

```
package com.shopstream.leaderboard;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;

@SpringBootApplication
@EnableScheduling
public class LeaderboardApplication {
}
```

LeaderboardApplication.java (continued)

JAVA

```

    public static void main(String[] args) {
        SpringApplication.run(LeaderboardApplication.class, args);
    }
}

```

The input is validated at the edge (**points** between 1 and 10,000) because the board trusts its callers: one bug that posts a million points ruins the day's board.

The front end is one static page that opens an **EventSource** and re-renders the top 10 whenever a **Leaderboard** event arrives:

src/main/resources/static/index.html

```

<!doctype html>
<meta charset="utf-8">
<title>Shopper Rewards</title>
<h1>Today's top shoppers</h1>
<ol id="board"></ol>
<div id="note" style="color:#b00"></div>
<script>
    const player = new URLSearchParams(location.search).get('player') || '';
    async function refresh() {
        const rows = await fetch('/api/leaderboard?size=10').json();
        document.getElementById('board').innerHTML =
            rows.map(r => `<li>${r.player}: ${r.score}</li>`).join('');
    }
    const es = new EventSource('/api/stream' + (player ? '?player=' + player : ''));
    es.addEventListener('leaderboard', refresh);
    es.addEventListener('notification', e =>
        document.getElementById('note').textContent = JSON.parse(e.data).message);
    es.onopen = refresh; // resync after every (re)connect: Pub/Sub does not replay
    refresh();
</script>

```

Note **es.onopen = refresh**: the browser reconnects automatically after a dropped connection, and re-reading the snapshot closes the gap that fire-and-forget Pub/Sub leaves.

10.5.9 Verify with redis-cli

Start the service, post a few scores, and inspect:

Scores, ranks and unique players

TERMINAL

```

$ add() { curl -s -X POST localhost:8080/api/points -H 'Content-Type: application/json' \
  -d "{\"playerId\":\"$1\",\"points\":$2}"; echo; }
$ add ana 50; add ben 80; add cy 20; add ana 60
{"player":"ana","score":50.0,"oldRank":-1,"newRank":0}
{"player":"ben","score":80.0,"oldRank":-1,"newRank":0}
{"player":"cy","score":20.0,"oldRank":-1,"newRank":2}
{"player":"ana","score":110.0,"oldRank":1,"newRank":0}
127.0.0.1:6379> ZREVRANGE lb:points:daily:2026-10-10 0 2 WITHSCORES
1) "ana"
2) "110"
3) "ben"
4) "80"
5) "cy"
6) "20"
127.0.0.1:6379> ZREVRANK lb:points:daily:2026-10-10 ben
(integer) 1
127.0.0.1:6379> PFCOUNT uv:players:2026-10-10
(integer) 3
127.0.0.1:6379> XLEN stream:score-events
(integer) 4
127.0.0.1:6379> TTL lb:points:daily:2026-10-10
(integer) 691190

```

Ana posted twice but **PFCOUNT** says 3 players, as it should. In a second terminal, watch the live channels while you post more scores:

Watch Pub/Sub from redis-cli

TERMINAL

```
127.0.0.1:6379> PSUBSCRIBE chan:*
Reading messages... (press Ctrl-C to quit)
1) "psubscribe"
2) "chan:*"
3) (integer) 1
1) "pmessage"
2) "chan:*"
3) "chan:board"
4) "{\"player\":\"cy\", \"score\": \"120\", \"rank\": 1}"
1) "pmessage"
2) "chan:*"
3) "chan:notify:cy"
4) "{\"player\":\"cy\", \"message\": \"You took the lead!\", \"rank\": 1}"
```

(The message was produced by a post that lifted **cy** from 20 to 120 points. The **rank** field is 1-based in the JSON.) Open <http://localhost:8080/?player=cy> in a browser and post points for **cy**: the page updates and shows the notification text with no refresh.

NOTE

HyperLogLog trades accuracy for size. **PFCOUNT** has a standard error of about 0.81 percent, and the structure never exceeds roughly 12 KB per key regardless of how many players it has seen. For a few players the estimate is exact, which is why the sample shows 3. If you need the exact set of distinct ids, use a **SET**, and pay memory proportional to the number of players.

10.5.10 Tests

The test class reuses the **RedisTestConfig** container bean from the earlier projects. The SSE test opens a real HTTP connection, waits for the greeting, posts points over HTTP, and waits for the push.

LeaderboardIT.java

JAVA

```
package com.shopstream.leaderboard;

import static org.assertj.core.api.Assertions.assertThat;

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.List;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.TimeUnit;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@Import(RedisTestConfig.class)
class LeaderboardIT {

    @Value("${local.server.port}") int port;
    @Autowired ScoreService scores;

    @Test
    void ranksFollowPointsAndReportOldAndNewRank() {
        scores.addPoints("t-ann", 100);
        scores.addPoints("t-ben", 300);
        var result = scores.addPoints("t-ann", 250); // ann 350 passes ben 300
        assertThat(result.oldRank()).isGreaterThan(result.newRank());
        List<ScoreService.Entry> top = scores.top(100);
        int ann = top.stream().map(ScoreService.Entry::player).toList().indexOf("t-ann");
        int ben = top.stream().map(ScoreService.Entry::player).toList().indexOf("t-ben");
        assertThat(ann).isLessThan(ben);
        assertThat(scores.around("t-ann")).isPresent();
    }

    @Test
    void uniquePlayersCountsEachPlayerOnce() {
```

LeaderboardIT.java (continued)

JAVA

```

    long before = scores.uniquePlayersToday();
    scores.addPoints("t-u1", 1);
    scores.addPoints("t-u1", 1);
    scores.addPoints("t-u2", 1);
    assertThat(scores.uniquePlayersToday() - before).isEqualTo(2);
}

@Test
void notificationReachesTheBrowserOverSse() throws Exception {
    HttpClient http = HttpClient.newHttpClient();
    BlockingQueue<String> lines = new LinkedBlockingQueue<>();
    HttpRequest sse = HttpRequest.newBuilder(
        URI.create("http://localhost:" + port + "/api/stream?player=sse-dana"))
        .header("Accept", "text/event-stream").build();
    http.sendAsync(sse, HttpResponse.BodyHandlers.ofLines())
        .thenAccept(r -> r.body().forEach(lines::add));
    assertThat(waitFor(lines, "data:connected")).isTrue();

    URI pointsUri = URI.create("http://localhost:" + port + "/api/points");
    HttpRequest post = HttpRequest.newBuilder(pointsUri)
        .header("Content-Type", "application/json")
        .POST(HttpRequest.BodyPublishers.ofString(
            "{\"playerId\":\"sse-dana\",\"points\":9000}"));
    var response = http.send(post, HttpResponse.BodyHandlers.ofString());
    assertThat(response.statusCode()).isEqualTo(200);

    assertThat(waitFor(lines, "event:notification")).isTrue();
}

private static boolean waitFor(BlockingQueue<String> lines, String needle) throws Exception {
    long deadline = System.nanoTime() + TimeUnit.SECONDS.toNanos(10);
    while (System.nanoTime() < deadline) {
        String line = lines.poll(200, TimeUnit.MILLISECONDS);
        if (line != null && line.contains(needle)) {
            return true;
        }
    }
    return false;
}
}

```

The SSE test crosses every layer: the HTTP post, the Lua script, the stream, the scheduled forwarder, **PUBLISH**, the listener container, the hub and the HTTP response. It is slower than the others (about 100 ms of polling latency) but it is the one that fails when you misspell a channel name or forget `@EnableScheduling`.

WATCH OUT

The notification assertion depends on `sse-dana` being the first player on a fresh board when the test runs, or at least entering the top 10. With 9,000 points she takes the lead on any board the other tests have built, because they award at most a few hundred points; if you add tests with larger awards, give this one more points or a separate Redis database.

10.5.11 Demo: two instances, one board

The architecture claim is that any instance can handle the purchase while any instance serves the viewer. Prove it by running two copies on different ports against the same Redis:

Cross-instance push

TERMINAL

```

$ mvn spring-boot:run -Dspring-boot.run.arguments=--server.port=8081 &
$ mvn spring-boot:run -Dspring-boot.run.arguments=--server.port=8082 &
$ curl -N "localhost:8082/api/stream?player=ana" &
event:hello
data:connected

$ curl -s -X POST localhost:8081/api/points -H 'Content-Type: application/json' \
  -d '{"playerId":"ana","points":500}'
{"player":"ana","score":500.0,"oldRank":-1,"newRank":0}
event:leaderboard
data:{"player":"ana","score":500.0,"rank":1}

event:notification
data:{"player":"ana","message":"You took the lead!","rank":1}

```

The purchase went to port 8081, a forwarder in one of the two instances published it, and the SSE connection on port 8082 received the push. If both instances run the forwarder, the consumer group makes sure each stream entry is forwarded by exactly one of them (barring a crash), so viewers do not see duplicates in normal operation.

INTERVIEW ANGLE

When would you use Pub/Sub, and when a stream, for notifications? Pub/Sub is for live fan-out where losing a message while nobody is listening is acceptable (a UI update, a cache-invalidation hint). A stream is for events you must not lose: it persists entries, supports consumer groups with acknowledgements, and lets a crashed consumer resume. Production designs often use both: the stream as the durable log and a forwarder that republishes to Pub/Sub for the low-latency, many-listener part.

REAL WORLD

Leaderboards in games and apps are one of the best-known uses of sorted sets because `ZADD/ZINCRBY` and rank queries are logarithmic. At very large scale teams shard boards (by region, season or score bucket) and keep an approximate global rank; the principle, one sorted set per board and a stream as the audit log, stays the same.

10.5.12 Extension ideas

- **Weekly and all-time boards.** Add `ZUNIONSTORE` with the last 7 daily boards into `lb:points:weekly`, cached for a minute, and compare its cost with an incrementally maintained weekly board.
- **Last-Event-ID replay.** Put the stream entry id into the SSE `id` field. On reconnect the browser sends `Last-Event-ID`; read the missed entries with `XRANGE` and replay them.
- **Per-player channels on demand.** Subscribe to `chan:notify:{player}` only when a player connects, with `RedisMessageListenerContainer.addListener` and `removeMessageListener`.
- **Sharded Pub/Sub.** Replace `PUBLISH` with `SPUBLISH` and `SSUBSCRIBE` to scale Pub/Sub on a Redis Cluster.
- **Tie-break by time.** Implement the fractional-score trick and write a test showing that two players with equal points rank by who got there first.
- **Backpressure.** Limit each SSE connection to one event per 250 ms by coalescing board updates in the hub.

KEY POINTS

What this project proves:

- A sorted set gives you ranking, rank lookup and neighbour queries with logarithmic cost; `ZINCRBY` is the scoring primitive.
- Run several related updates in one Lua script when they must agree, here the board, the HyperLogLog and the event log.
- Combine a stream (durable, acknowledged) with Pub/Sub (instant, lossy) instead of choosing one; every instance subscribes, each browser connects to just one.
- Clients must resynchronise after a reconnect because Pub/Sub does not replay.
- HyperLogLog counts distinct items in a small fixed size with a small, known error.

PART 11

Operations and Security

Running Redis in production: deploy it, size it, watch it, fix it, lock it down, back it up, and upgrade it.

In this part

- 11.1** Deployment Options and a Production `redis.conf`
- 11.2** Sizing and Capacity Planning
- 11.3** Monitoring Redis
- 11.4** Troubleshooting Playbook
- 11.5** Security Hardening
- 11.6** Backup, Restore and Disaster Recovery
- 11.7** Upgrades and Migrations

11.1 Deployment Options and a Production `redis.conf`

Everything so far ran on a laptop. A production Redis has to survive restarts, keep its data directory on disk that outlives the process, be reachable only by the services that need it, and tell you when it is unhappy. This chapter covers the common ways to run Redis and then walks through a sane, annotated `redis.conf` that you can adapt.

11.1.1 Choosing how to run Redis

There is no single right answer. The question is who owns the operating system, the disk and the failover logic.

Option	Good for	Watch out for
Docker / Compose	Local development, CI, small single-host setups	Data lost if no volume is mounted; no automatic failover
systemd on a VM	One or a few servers you manage yourself	You own patching, kernel settings, backups, failover
Kubernetes (StatefulSet or an operator)	Teams that already run everything on Kubernetes	Storage class, pod rescheduling, and memory limits need care
Managed service from a cloud or Redis vendor	Teams that do not want to run Redis	Cost, feature lag, vendor limits on commands and modules

Deployment options at a glance.

If your whole team's job is not "run databases", strongly consider a managed service for production and use the rest of this Part as the checklist of what you are paying someone else to do. If you run it yourself, the rest of the chapter applies directly.

11.1.2 Docker and Docker Compose

The official image is `redis` on Docker Hub. For ShopStream development, the following Compose file gives you a Redis with a named volume, a mounted config file and a health check.

`docker-compose.yml`

YAML

```
services:
  redis:
    image: redis:8
    container_name: shopstream-redis
    command: ["redis-server", "/usr/local/etc/redis/redis.conf"]
    ports:
      - "127.0.0.1:6379:6379"
    volumes:
      - redis-data:/data
      - ./redis.conf:/usr/local/etc/redis/redis.conf:ro
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 3s
      retries: 5
```

docker-compose.yml (continued)

YAML

```
restart: unless-stopped

volumes:
  redis-data:
```

What to notice

- `127.0.0.1:6379:6379` publishes the port only on the host's loopback interface. Writing just `6379:6379` publishes it on every interface of the host, which is how many accidental exposures begin.
- `/data` is the image's working directory, so `dir /data` in the config (or the default) writes RDB and AOF files into the named volume. Without the volume, `docker rm` deletes your data.
- Pin a major version tag (`redis:8`) rather than `latest` so an image pull does not upgrade you by surprise. For production pin a full version.
- The container needs the config to bind to an address the other containers can reach (for example `bind * -::*` inside the container) because Docker networking, not Redis, then decides who can connect. Keep the host-side publish restricted as shown above.

11.1.3 Running under systemd on a VM

When you install Redis from a package manager or compile it, run it as an unprivileged `redis` user under `systemd`. Packages ship a unit file, so you rarely write it by hand, but you should know what matters in it.

/etc/systemd/system/redis.service

CONF

```
[Unit]
Description=Redis in-memory data store
After=network.target

[Service]
User=redis
Group=redis
ExecStart=/usr/bin/redis-server /etc/redis/redis.conf --supervised systemd
Type=notify
Restart=always
LimitNOFILE=65535
RuntimeDirectory=redis
ReadWritePaths=/var/lib/redis /var/log/redis

[Install]
WantedBy=multi-user.target
```

`--supervised systemd` (the config directive is `supervised`) makes Redis tell `systemd` when it is ready. `LimitNOFILE` matters because Redis wants a file descriptor per client connection plus some of its own; with fewer descriptors than `maxclients` allows, Redis lowers its effective client limit at startup and logs a warning.

Linux kernel settings

Redis prints warnings at startup for the settings that matter most. Do not ignore them.

Setting	Why	Typical value
<code>vm.overcommit_memory</code>	A background save forks; with strict overcommit the fork can fail on a large dataset even though copy-on-write needs little memory	1
Transparent Huge Pages	Huge pages make copy-on-write copy 2 MB instead of 4 KB per touched page and cause latency spikes during forks	never (disable)
<code>net.core.somaxconn</code>	Caps the listen backlog; Redis's <code>tcp-backlog</code> (default 511) is silently reduced if this is lower	1024 or higher
Swap	A Redis whose pages are swapped out becomes extremely slow in an unpredictable way	Small or none, and size RAM so it is not used

Host settings Redis complains about if they are wrong.

11.1.4 Redis on Kubernetes

Redis is a stateful, long-lived process with a disk, so the Kubernetes primitive that fits is a **StatefulSet**: pods get stable names and each keeps its own PersistentVolumeClaim across restarts. A **headless Service** gives each pod a stable DNS name, which Sentinel and Cluster need because nodes identify each other by address.

Rather than hand-writing replication, Sentinel and Cluster bootstrapping, many teams install a Helm chart or an **operator** (a controller that watches a custom resource such as "a Redis cluster with 3 shards and 1 replica each" and creates and repairs the pods). Several community and vendor operators exist, and which is best depends on your requirements, so evaluate them against these questions rather than by name:

- Does it handle failover and re-sharding, or only create the pods?
- How does it do upgrades: one pod at a time, replica before primary?
- Where are backups stored and have you restored one?
- Is the project maintained and does it support the Redis version you want?

Points that bite regardless of tooling:

- **Memory limits.** If the pod's memory limit is close to `maxmemory`, the fork for a background save plus copy-on-write pushes the container over its limit and the kernel kills it (out-of-memory kill). Leave headroom, as the sizing chapter explains.
- **Storage.** Use a storage class with predictable latency. AOF `fsync` on slow network disks shows up directly as command latency.
- **Pod rescheduling and addresses.** A restarted pod may get a new IP. Clients and cluster nodes should use DNS names, and a Redis Cluster must be configured to announce an address peers can reach.
- **Disruption budgets.** Add a PodDisruptionBudget so a node drain cannot take the primary and all replicas at once.

11.1.5 A production `redis.conf`, annotated

The file below is a starting point for a single primary with one replica, persistence on, and a trusted private network. Every directive is a real Redis directive; the values are sensible defaults to adapt, not magic numbers.

`redis.conf`

CONF

```
# --- Network ---
bind 10.0.1.15 127.0.0.1
protected-mode yes
port 6379
tcp-backlog 511
tcp-keepalive 300
timeout 0
maxclients 10000

# --- Security (details in the security chapter) ---
aclfile /etc/redis/users.acl
enable-debug-command no
enable-module-command no

# --- Memory ---
maxmemory 12gb
maxmemory-policy allkeys-lfu
maxmemory-samples 5
lazyfree-lazy-eviction yes
lazyfree-lazy-expire yes
lazyfree-lazy-server-del yes
replica-lazy-flush yes

# --- Persistence ---
dir /var/lib/redis
save 3600 1 300 100 60 10000
stop-writes-on-bgsave-error yes
rdbcompression yes
rdbchecksum yes
appendonly yes
appendfsync everysec
no-appendfsync-on-rewrite no
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 64mb

# --- Replication ---
```


redis.conf (continued)

CONF

```

repl-backlog-size 256mb
min-replicas-to-write 1
min-replicas-max-lag 10

# --- Observability ---
slowlog-log-slower-than 10000
slowlog-max-len 256
latency-monitor-threshold 100
loglevel notice
logfile /var/log/redis/redis.log

# --- Client protection ---
client-output-buffer-limit normal 0 0 0
client-output-buffer-limit replica 512mb 128mb 60
client-output-buffer-limit pubsub 32mb 8mb 60

```

Reading the file section by section

- **Network.** `bind` lists the interfaces to listen on; give it the private address of the server and loopback, never a public one. `protected-mode yes` is the safety net: with no password and no explicit bind it only answers loopback clients. `tcp-keepalive 300` sends TCP keepalives so dead peers are detected. `timeout 0` never closes idle clients; that is right for connection pools. `maxclients 10000` is the ceiling on simultaneous connections.
- **Memory.** `maxmemory` must be set on any instance used as a cache, and also on a data store if you prefer a clean error to the machine running out of memory. The policy is covered in the eviction chapter in Part 4; `allkeys-lfu` is a good default for caches. The `lazyfree-*` options make the server free big values in a background thread instead of blocking the main one, which smooths latency for large keys.
- **Persistence.** Several `save` rules in one line are allowed (a snapshot after 3600 s if at least 1 key changed, 300 s and 100 keys, 60 s and 10000 keys). `appendonly yes` with `appendfsync everysec` is the usual balance: at most about one second of writes lost on a crash. Redis defaults to `appendonly no`, so you must turn this on. See the persistence chapter in Part 4 for the trade-offs.
- **Replication.** A backlog large enough to cover the longest replica outage you want to ride through with a partial resync. Think "write rate in bytes per second times the seconds of outage". `min-replicas-to-write 1` makes the primary refuse writes if it has fewer than one replica with lag under `min-replicas-max-lag` seconds, trading availability for less data loss.
- **Observability.** `slowlog-log-slower-than` is in microseconds, so 10000 is 10 ms. `latency-monitor-threshold 100` records latency events above 100 ms for `LATENCY DOCTOR`.
- **Client protection.** `client-output-buffer-limit` format is `class hard-limit soft-limit soft-seconds`. A replica or Pub/Sub subscriber that cannot keep up would otherwise make the primary buffer unbounded data in RAM; the limit disconnects it. Raise the replica limit if a large dataset makes full syncs fail.

WATCH OUT

Do not copy `maxmemory 12gb` blindly. It must be derived from your machine's RAM minus operating system, fork headroom and replication buffers. The next chapter shows the arithmetic. A `maxmemory` equal to total RAM is a recipe for an out-of-memory kill during a snapshot.

TIP

Change settings live, then persist them. `CONFIG SET` changes most directives without a restart, and `CONFIG REWRITE` writes the running values back to `redis.conf`. If you manage the file with Ansible or a ConfigMap, change it there instead so the next deploy does not undo your tuning.

11.1.6 Verifying the deployment

After starting, check the facts instead of assuming them:

Post-deploy smoke checks

TERMINAL

```
$ redis-cli -h 10.0.1.15 ping
PONG
$ redis-cli -h 10.0.1.15 config get appendonly
1) "appendonly"
2) "yes"
$ redis-cli -h 10.0.1.15 info server | grep -E "redis_version|tcp_port|config_file"
redis_version:8.0.0
tcp_port:6379
config_file:/etc/redis/redis.conf
$ redis-cli -h 10.0.1.15 info replication | grep -E "^role|connected_slaves"
role:master
connected_slaves:1
```

Note that **INFO** still prints **role:master** and **connected_slaves** for historical reasons, even though the documentation says primary and replica. Also confirm from a machine that should not have access that the port is unreachable; that test matters more than any config line.

KEY POINTS

Deployment and configuration

- Always persist the data directory on a volume or disk that survives the process, and bind only to private interfaces.
- Run as an unprivileged user; set **vm.overcommit_memory=1**, disable Transparent Huge Pages and raise the file descriptor limit.
- On Kubernetes use a StatefulSet or a maintained operator, leave memory headroom above **maxmemory**, and add a disruption budget.
- Set **maxmemory**, a persistence mode, a replication backlog, slow log and client output buffer limits explicitly rather than trusting defaults.
- Verify the running configuration with **CONFIG GET** and **INFO**, and test unreachability from the outside.

11.2 Sizing and Capacity Planning

"How big a server do I need?" is the first question operations asks. Redis keeps the whole dataset in RAM, so memory is the main constraint, but connections, network bandwidth and the CPU core that runs commands also set limits. This chapter gives you a way to estimate, a way to measure, and a list of what to leave out of the budget.

11.2.1 Memory math

Start with the data: number of keys times average size per key. The size per key has two parts, the payload and the overhead.

Per-key overhead

Every key costs memory beyond its value: the key string itself, an entry in the main dictionary, an object header, and (if the key has a TTL) an entry in the expires dictionary. On a test with Redis 7.0 on a 64-bit Linux machine, 100,000 tiny keys such as **key:1** holding a 10-byte value cost about 90 bytes per key all in. Treat "50 to 100 bytes per key, plus the data" as an order-of-magnitude rule, and measure your own data, because versions and allocators differ.

Per-structure behaviour

The same logical data can cost very different amounts depending on the encoding. Small hashes, sorted sets and lists use the compact **listpack** encoding; larger ones switch to hash tables or skip lists with a pointer per element. **MEMORY USAGE** reports the bytes used by one key, including overhead:

Measure real keys instead of guessing

TERMINAL

```
127.0.0.1:6379> SET a b
OK
127.0.0.1:6379> MEMORY USAGE a
(integer) 64
127.0.0.1:6379> HSET h f v
(integer) 1
127.0.0.1:6379> MEMORY USAGE h
(integer) 64
127.0.0.1:6379> OBJECT ENCODING h
"listpack"
```

(Exact byte counts vary by version; the point is that a hash with a single small field costs only a few dozen bytes more than a string.)

TIP

Many small hashes beat millions of tiny top-level keys. If you store 10 attributes of a user, one hash `user:42` with 10 fields uses less memory than 10 separate keys, because there is one set of per-key overhead, not ten. The trade-off: hash fields can have their own TTL only on Redis 7.4 and later, while separate keys can always expire independently.

A worked estimate

Suppose ShopStream expects 2 million products cached as hashes of about 400 bytes of payload, 5 million active sessions of about 300 bytes, and a few small structures. A first estimate:

Dataset	Count	Bytes each (payload + overhead)	Total
Product hashes	2,000,000	400 + 150 = 550	1.1 GB
Sessions	5,000,000	300 + 150 = 450	2.25 GB
Carts, rate limits, misc	n/a	n/a	0.5 GB
Estimated dataset	n/a	n/a	about 3.9 GB

Illustrative first-pass memory estimate (labelled guess, not measurement).

The "150" per item bundles key, dictionary, expiry and per-field overhead and is an assumption. Before trusting it, load a representative sample (say 100,000 products) into a test instance and compare `used_memory` before and after. Scale up from the measured number and add 20 to 30 percent margin for growth.

11.2.2 What else uses memory

`used_memory` in `INFO memory` is what Redis has allocated. Beyond the dataset it includes buffers you must budget for:

- **Client buffers.** Each connection has input and output buffers. Pipelining huge batches or slow readers (Pub/Sub subscribers, replicas) grow them. `maxmemory-clients` (Redis 7.0+) can cap total client memory, as a percentage or an absolute size.
- **Replication backlog and replica output buffers**, sized by `repl-backlog-size` and the output limits shown earlier.
- **AOF buffers** during rewrite.
- **Fragmentation.** The operating system may hold more RSS (resident memory) than Redis has live data. Check `mem_fragmentation_ratio` in `INFO memory`: RSS divided by `used_memory`. Around 1.0 to 1.5 is normal; well above 1.5 means memory is wasted by fragmentation (consider `activedefrag yes` or a restart through failover); below 1.0 means the OS has swapped part of Redis out, which is an emergency. On a tiny, nearly empty instance the ratio is meaningless: a fresh test instance can show a ratio of 5 or more simply because the executable's baseline memory dominates.
- **Eviction does not count these buffers**, which is one reason `maxmemory` must be well below physical RAM.

Headroom for fork and copy-on-write

When Redis takes an RDB snapshot or rewrites the AOF it calls `fork()`. The child shares memory pages with the parent; each page the parent modifies after the fork is copied (copy-on-write). In the worst case, a write-heavy workload during a long snapshot can duplicate nearly the entire dataset. A rule of thumb for planning:

Physical RAM $\geq$ `maxmemory` $\times$ (1 + COW factor) + buffers + OS

read-mostly workload: COW factor about 0.1 to 0.3

write-heavy workload: COW factor up to 1.0 in the worst case

That is why Redis machines often have `maxmemory` at 50 to 70 percent of RAM when persistence is on and writes are heavy. You can measure your own COW usage: after a background save, look at `current_cow_size` in `INFO persistence` while a save is running, and `rdb_last_cow_size` afterwards.

GOTCHA

A cache on a 16 GB box with `maxmemory 15gb` can still get killed. Eviction keeps the dataset under 15 GB, but the snapshot child, client buffers and fragmentation sit on top. When the box runs out, the Linux out-of-memory killer picks the biggest process, which is Redis. Look for it in `dmesg` or the system log.

11.2.3 Connections and CPU

Connections

Each client connection is a file descriptor and some memory. Typical Spring Boot applications use a handful of connections each (Lettuce shares one connection among threads for most commands; Jedis pools use one per concurrent command). The danger is multiplication: 200 pods times 20 connections is 4000 connections. Keep `maxclients` (default 10000) above the sum of all pool sizes with margin, and confirm with `INFO clients`:

```
Connection headroom
127.0.0.1:6379> INFO clients
# Clients
connected_clients:1
cluster_connections:0
maxclients:10000
client_recent_max_input_buffer:0
client_recent_max_output_buffer:0
blocked_clients:0
tracking_clients:0
```

TERMINAL

The single command thread

Commands run on one thread, so one Redis instance can never use more than about one core for command execution, however many cores the machine has. With I/O threads enabled (`io-threads`, reworked in Redis 8.0) socket reading and writing are spread over more cores, which helps for many connections and large payloads but does not parallelize command execution itself. If one instance is CPU bound on the main thread, the answer is to shard the data across several instances (Redis Cluster, or client-side partitioning), not a bigger machine.

Network

A `GET` of a 10 KB value at 100,000 operations per second is about 1 GB per second, roughly 8 Gbit/s, which can saturate a 10 Gbit link. Large values and fan-out reads (replicas, Pub/Sub) are bandwidth problems before they are CPU problems. Multiply payload size by operations per second to estimate.

11.2.4 Benchmarking

`redis-benchmark` ships with Redis and measures raw server throughput for common commands. It is useful for comparing configurations, not for predicting your application's behaviour.

```
redis-benchmark usage
$ redis-benchmark -h 10.0.1.15 -t set,get -n 1000000 -c 50 -d 256 -P 16 -q
SET: 612745.12 requests per second, p50=0.623 msec
GET: 701331.25 requests per second, p50=0.543 msec
```

TERMINAL

(The numbers above are illustrative; yours depend entirely on hardware and network.) The options: `-t` selects tests, `-n` is the total number of requests, `-c` the number of parallel connections, `-d` the value size in bytes, `-P` the pipeline depth and `-q` quiet output. Always benchmark from a separate machine on the same network as your application, with your real value sizes, and with and without pipelining, because a benchmark that runs on the same host as the server mostly measures loopback.

For a more realistic load profile (key distributions, ratios of reads and writes, many connections and threads) a dedicated tool such as `memtier_benchmark` is commonly used. It is a separate project from Redis; check its documentation for options.

REAL WORLD

Capacity reviews that work Teams that avoid surprises review three numbers per instance every quarter: peak `used_memory` as a percentage of `maxmemory`, peak connected clients as a percentage of `maxclients`, and p99 command latency. If any one is trending toward its ceiling, they resize or shard before the incident, not during it.

INTERVIEW ANGLE

How do you size a Redis instance? Estimate keys times (payload plus per-key overhead), validate with **MEMORY USAGE** on real samples, add growth margin, then add headroom for fork copy-on-write, client and replication buffers and fragmentation, so **maxmemory** is typically well below physical RAM. Check connection counts against **maxclients**, and check bandwidth and the single main thread, which cap throughput on one instance and are solved by sharding.

KEY POINTS

Sizing

- Memory is the primary constraint; per-key overhead is tens of bytes and small compact encodings save a lot, but measure with **MEMORY USAGE** and a load test.
- **maxmemory** is not physical RAM: leave room for fork copy-on-write, buffers and fragmentation.
- Watch **mem_fragmentation_ratio**; far above 1.5 means waste, below 1.0 means swapping.
- One instance runs commands on one core and shares one network link: scale out by sharding.
- Benchmark from another host with realistic value sizes and pipelining, and treat the result as a comparison tool.

11.3 Monitoring Redis

A Redis server that nobody watches fails in the same predictable ways: memory fills, latency creeps up, a replica falls behind, connections pile up. Almost every one of those failures is visible in **INFO** long before it becomes an outage. This chapter shows which numbers to watch, where to find them, what thresholds are reasonable as starting guidance, and how to get them into a dashboard.

11.3.1 INFO: the single source of truth

INFO returns server statistics grouped in sections. Ask for one section at a time instead of dumping everything:

REDIS

```
INFO server
INFO clients
INFO memory
INFO persistence
INFO stats
INFO replication
INFO cpu
INFO commandstats
INFO keyspace
```

INFO everything adds all sections, and **INFO commandstats** and **INFO latencystats** show per-command call counts, time and (with **latency-tracking yes**, the default since Redis 7.0) percentile latencies. Fields are **name:value** lines, so they are easy to parse with a script or an exporter.

Field	Section	What it tells you
used_memory	memory	Bytes allocated by Redis; compare with maxmemory
mem_fragmentation_ratio	memory	RSS divided by used_memory; high means wasted memory, below 1 means swapping
connected_clients	clients	Current connections; compare with maxclients
blocked_clients	clients	Clients waiting in blocking calls such as BLPOP or XREAD BLOCK
rejected_connections	stats	Connections refused because maxclients was reached
keyspace_hits / keyspace_misses	stats	Used to compute the cache hit ratio
evicted_keys	stats	Keys removed by maxmemory eviction
expired_keys	stats	Keys removed by TTL
instantaneous_ops_per_sec	stats	Recent throughput
latest_fork_usec	stats	Duration of the last fork in microseconds

Field	Section	What it tells you
rdb_last_bgsave_status / aof_last_write_status	persistence	Must be ok
master_link_status	replication	On a replica: up or down
master_repl_offset / slave_repl_offset	replication	Compare primary and replica to see lag in bytes

The INFO fields that matter most.

11.3.2 Key metrics and starting thresholds

Thresholds depend on your system. The table gives guidance for a typical cache or session store, so use it as a starting point and tune it with your own history.

Metric	How to compute	Starting alert idea
Memory use	$\text{used_memory} \div \text{maxmemory}$	Warn above 80%, page above 90%
Cache hit ratio	$\text{hits} \div (\text{hits} + \text{misses})$ over a window	Alert on a sudden drop, not on a fixed value
Evictions	rate of evicted_keys	Any sustained non-zero rate on a store that must keep data; on a cache watch for a rising trend
Rejected connections	rate of rejected_connections	Any non-zero value
Clients	$\text{connected_clients} \div \text{maxclients}$	Warn above 80%
Fork time	latest_fork_usec	Warn when it grows into hundreds of milliseconds
Replica lag	primary offset minus replica offset, or seconds since last interaction	Alert when lag keeps growing
Persistence errors	rdb_last_bgsave_status, aof_last_write_status	Alert on anything other than ok
Slow commands	rate of new SLOWLOG entries	Alert on a sudden increase
Latency	p99 of client-side timings	Alert on your SLO, for example above 5 ms

Metrics, how to compute them, and a starting alert rule (guidance, not law).

The hit ratio deserves a note. Over the lifetime of the server it is a useless average. Compute it over a recent window, from the difference of the counters, and remember that a low ratio is only a problem if misses are expensive (they go to a slow database) or if it changed recently. A cache full of one-time keys has a low ratio and is working as designed.

GOTCHA

INFO counters only go up. `keyspace_hits`, `evicted_keys` and friends are cumulative since start (or since `CONFIG RESETSTAT`). A dashboard that plots the raw value is not showing what is happening now. Plot the rate of change.

11.3.3 Prometheus and Grafana

The most common way to collect these numbers is a **Prometheus exporter**: a small process that connects to Redis, runs `INFO` and friends, and exposes the values over HTTP for Prometheus to scrape. The widely used one is `redis_exporter`. A typical Compose fragment next to ShopStream's Redis looks like this:

`docker-compose.monitoring.yml`

YAML

```
services:
  redis-exporter:
    image: oliver006/redis_exporter:latest
    environment:
      REDIS_ADDR: "redis://redis:6379"
      REDIS_PASSWORD: "${REDIS_PASSWORD}"
    ports:
      - "127.0.0.1:9121:9121"
    depends_on:
      - redis
```

Prometheus then scrapes port 9121. Pin a specific image version in real deployments. Metric names follow the exporter's `redis_` prefix, for example `redis_memory_used_bytes`, `redis_memory_max_bytes`,

`redis_connected_clients`, `redis_keyspace_hits_total`, `redis_evicted_keys_total`; confirm the exact names against the exporter's README for the version you deploy. Example alert expressions in Prometheus query language:

```
# memory above 90% of maxmemory (only meaningful if maxmemory > 0)
redis_memory_used_bytes / redis_memory_max_bytes > 0.9

# hit ratio over the last 5 minutes
rate(redis_keyspace_hits_total[5m])
/ (rate(redis_keyspace_hits_total[5m]) + rate(redis_keyspace_misses_total[5m]))

# any evictions happening now
rate(redis_evicted_keys_total[5m]) > 0
```

Also monitor the Java side. Spring Boot Actuator with Micrometer exposes Lettuce command latency and connection metrics when Lettuce's metrics are enabled, and your own cache hit and miss counters tell you about the application-level hit ratio, which matters more to users than the server's. The Spring chapters in Part 7 cover application-side metrics.

11.3.4 SLOWLOG

The slow log records commands whose **execution time** (not including network or queueing time) exceeded a threshold. It is the first tool for finding "who is running the expensive command".

Reading the slow log

```
127.0.0.1:6379> CONFIG GET slowlog-log-slower-than
1) "slowlog-log-slower-than"
2) "10000"
127.0.0.1:6379> CONFIG SET slowlog-log-slower-than 5000
OK
127.0.0.1:6379> SLOWLOG GET 1
1) 1) (integer) 14
   2) (integer) 1760000000
   3) (integer) 31254
   4) 1) "SORT"
      2) "biglist"
   5) "10.0.1.20:51234"
   6) ""
127.0.0.1:6379> SLOWLOG LEN
(integer) 3
127.0.0.1:6379> SLOWLOG RESET
OK
```

Each entry is: a unique id, the Unix timestamp, the duration in microseconds (31254 μ s is about 31 ms here), the command and its arguments (long arguments are truncated), the client address and the client name. The log keeps only the most recent `slowlog-max-len` entries in memory, so export it periodically if you want history.

11.3.5 LATENCY tools

Three layers help separate "Redis is slow" from "the network or machine is slow":

- **Client-side timing.** Measure round-trip time from the application host: `redis-cli --latency -h host` samples `PING` repeatedly and prints min, max and average in milliseconds.
- **Intrinsic latency.** `redis-cli --intrinsic-latency 100` run on the server host measures the delay the kernel or hypervisor itself adds (the number is the test duration in seconds). You cannot get better latency than this baseline.
- **The latency monitor.** After `CONFIG SET latency-monitor-threshold 100` Redis records events (such as fork, AOF fsync, expiry cycles) that take longer than 100 ms. `LATENCY LATEST`, `LATENCY HISTORY event` and `LATENCY DOCTOR` then report them, the last in plain English with suggestions.

Latency doctor

```
127.0.0.1:6379> CONFIG SET latency-monitor-threshold 100
OK
127.0.0.1:6379> LATENCY LATEST
1) 1) "fork"
   2) (integer) 1760000100
   3) (integer) 312
   4) (integer) 312
127.0.0.1:6379> LATENCY DOCTOR
```


The **LATENCY LATEST** columns are the event name, the time of the latest sample, the latest latency in milliseconds, and the all-time maximum. On an idle test instance **LATENCY DOCTOR** says there is too little data for its advice, which is normal.

11.3.6 Clients, keyspace and memory commands

CLIENT LIST prints one line per connection with its address, age, idle time, current command, buffer memory (**qbuf**, **omem**) and name. Name your connections (**CLIENT SETNAME**, or **client-name** in Lettuce and Jedis configuration) so you can tell the cart-service pods from the notification-service ones. Sort by **tot-mem** or **omem** to find a client with a runaway buffer, and by **age** plus **idle** to find leaked connections.

INFO keyspace shows per-database key counts, how many have an expiry and the average TTL:

Keyspace summary

TERMINAL

```
127.0.0.1:6379> INFO keyspace
# Keyspace
db0:keys=2041557,expires=1984210,avg_ttl=1352211
```

avg_ttl is in milliseconds. If **keys** is much larger than **expires** on a cache, you are probably missing TTLs, which is how a cache grows until it hits the memory ceiling. **MEMORY STATS** and **MEMORY DOCTOR** give a breakdown of the instance's memory use and simple advice.

REAL WORLD

A dashboard that earns its keep Six panels cover most incidents: memory used versus **maxmemory**, hit ratio over a recent window, ops per second, connected clients, evictions plus expired keys per second, and p99 latency seen from the client. Add replication lag and fork duration for any instance that persists or replicates.

KEY POINTS

Monitoring

- **INFO** is the source of truth; most monitoring is turning its counters into rates and ratios.
- Alert on memory versus **maxmemory**, rejected connections, persistence errors, replication lag, fork time and evictions where they should not occur.
- Use **redis_exporter** with Prometheus and Grafana, plus application-side cache metrics.
- **SLOWLOG** shows expensive commands (execution time only); **LATENCY** tools and **--intrinsic-latency** separate Redis from environment problems.
- Name your clients and keep TTLs on cache keys, then **CLIENT LIST** and **INFO keyspace** answer many questions fast.

11.4 Troubleshooting Playbook

When something is wrong at 3 a.m. you want a short route from symptom to cause to command to fix. This chapter is organized that way. For each area, first reproduce the symptom with a measurement, then change one thing at a time.

11.4.1 Method

1. State the symptom with a number ("p99 went from 2 ms to 80 ms at 14:05").
2. Check the cheap global signals: **INFO** (memory, clients, stats, persistence, replication), **SLOWLOG GET**, **LATENCY LATEST**.
3. Correlate with events: deploy, traffic spike, snapshot start, failover, a new batch job.
4. Form one hypothesis, test it with a command, and only then change something.

11.4.2 High latency

Symptom	Likely cause	How to confirm	Fix
Periodic spikes every few minutes	Fork for RDB snapshot or AOF rewrite	<code>latest_fork_usec</code> in <code>INFO stats</code> ; <code>LATENCY LATEST</code> shows "fork"	Disable Transparent Huge Pages; use faster VM type; make instances smaller by sharding; schedule heavy writes away from snapshots
Spikes with every write, disk busy	AOF fsync on slow disk	<code>strace -T -e trace=fdatsync</code> on the server; <code>aof_delayed_fsync</code> in <code>INFO persistence</code>	Use <code>appendfsync everysec</code> ; faster disk; keep AOF on a separate volume
Constant moderate latency, a few clients to blame	Slow O(N) commands	<code>SLOWLOG GET</code> ; <code>INFO commandstats</code>	Replace with incremental alternatives (<code>SCAN</code> instead of <code>KEYS</code>); cap collection sizes; split big keys
Latency grows with traffic	Single thread saturated	<code>INFO cpu</code> , <code>top</code> shows one core at 100%	Pipeline; reduce payload sizes; enable I/O threads; shard across instances
Sudden huge latency, memory pressure	Swapping	<code>vmstat 1</code> shows swap in/out; <code>mem_fragmentation_ratio</code> below 1	Add RAM, lower <code>maxmemory</code> , stop other memory consumers, no swap
Network-looking delays	Packet loss, noisy neighbours, large replies	<code>redis-cli --latency</code> from client host; compare <code>--intrinsic-latency</code> on server	Fix network; move clients closer; reduce reply sizes
Stalls when many keys expire together	Expiry cycle bursts	<code>LATENCY LATEST</code> event "expire-cycle"	Add jitter to TTLs so keys do not expire at the same instant

Latency symptoms.

WATCH OUT

Never use `KEYS` on a production instance. `KEYS pattern` scans the entire keyspace in one command and blocks every other client for as long as that takes. Use `SCAN` with a cursor and a `COUNT` hint, which returns small batches and lets other commands run between them.

11.4.3 Memory growth and out-of-memory

Symptom	Likely cause	How to confirm	Fix
<code>used_memory</code> climbs without bound on a cache	Keys without TTL, no <code>maxmemory</code>	<code>INFO keyspace</code> (keys much larger than expires); <code>maxmemory</code> is 0	Always set TTLs; set <code>maxmemory</code> and an eviction policy
Writes fail with <code>OOM command not allowed when used memory > 'maxmemory'</code>	<code>noeviction</code> policy (the default) and the memory is full	<code>CONFIG GET maxmemory-policy</code>	For a cache choose <code>allkeys-lru</code> or <code>allkeys-lfu</code> ; for a store add capacity or delete data
RSS much higher than <code>used_memory</code>	Fragmentation	<code>mem_fragmentation_ratio</code> above 1.5	Enable <code>activedefrag yes</code> ; or fail over to a fresh replica and restart the old primary
Process killed by Linux	OOM killer; fork plus copy-on-write plus buffers exceeded RAM	<code>dmesg</code> or journal mentions "Out of memory: Killed process"	Lower <code>maxmemory</code> ; add RAM; limit client buffers
One client's memory huge	Slow Pub/Sub subscriber or huge pipeline	<code>CLIENT LIST</code> large <code>omem</code> or <code>qbuf</code>	Set <code>client-output-buffer-limit</code> ; fix the consumer; set <code>maxmemory-clients</code>
Memory not freed after deleting data	Allocator keeps freed pages	<code>INFO memory</code> allocator fields	Normal; it is reused. Defrag or restart if truly needed

Memory symptoms.

11.4.4 Big keys and hot keys

A **big key** is one whose value is huge (a list with ten million items, a hash with a million fields). Deleting or reading it blocks the single thread for a long time, and in a cluster it can make one shard far larger than the others. A **hot key** is one that receives a disproportionate share of traffic; in a cluster all its traffic lands on one shard.

Finding big keys

`redis-cli --bigkeys` samples the keyspace with `SCAN` and reports the largest key of each type. `--memkeys` does the same by memory usage. Both are safe on a live server (they use `SCAN`) but they do add load, so prefer a replica or off-peak.

```

Scan for big keys TERMINAL

$ redis-cli -h 10.0.1.15 --bigkeys
# Scanning the entire keyspace to find biggest keys as well as
# average sizes per key type.

[00.00%] Biggest hash    found so far "product:991" with 12 fields
[41.25%] Biggest list    found so far "search:recent:u7" with 9001 items

----- summary -----

Sampled 2041557 keys in the keyspace!
Total key length in bytes is 31204112 (avg len 15.28)

Biggest  list found "search:recent:u7" has 9001 items
Biggest  hash found "product:991" has 12 fields

```

(Output shortened and illustrative.) To delete a big collection without blocking, use `UNLINK key`, which reclaims memory in a background thread, or delete incrementally with `HSCAN` plus `HDEL` batches.

Finding hot keys

Before Redis 8.6 the options were awkward: `redis-cli --hotkeys` works only when an LFU eviction policy (`allkeys-lfu` or `volatile-lfu`) is active, sampling access frequency counters; `MONITOR` shows everything but costs a lot and must never be left running. Redis 8.6 adds a purpose-built `HOTKEYS` command family that tracks the hottest keys over a time window:

```

REDIS

# track the top 10 keys by CPU and by network bytes, sampling 1 in 10 commands, for 60 seconds
HOTKEYS START METRICS 2 CPU NET COUNT 10 DURATION 60 SAMPLE 10
# later, read the results of the current or most recent session
HOTKEYS GET
# stop early, or clear the stored results
HOTKEYS STOP
HOTKEYS RESET

```

`METRICS 2 CPU NET` says that two metrics follow (CPU time and network bytes; at least one is required). `COUNT` is how many keys to report and `SAMPLE ratio` samples each command with probability 1 in ratio: a higher ratio costs less performance but may miss keys. `SLOTS` restricts tracking to chosen hash slots in cluster mode. Only one session can run at a time. `HOTKEYS GET` returns fields such as `tracking-active`, `collection-duration-ms`, `by-cpu-time-us` (key and microseconds pairs) and `by-net-bytes` (key and byte pairs). The commands are in the `@admin` and `@dangerous` ACL categories, so restrict who can run them.

If you must live without it, find hot keys with application-side counters and metrics, and mitigate by replicating the value (read from replicas), by caching it in the application process (client-side caching or a small local cache), or by splitting it into several keys (`stock:{sku}:0` to `stock:{sku}:7`) that you sum or choose from at random.

11.4.5 Fork stalls and persistence problems

A fork copies the process's page tables. On big instances or slow hypervisors this takes tens to hundreds of milliseconds, during which Redis serves nothing. Check `latest_fork_usec` and keep instances to a few tens of gigabytes rather than hundreds. Disable Transparent Huge Pages (described in the deployment chapter), and avoid triggering `BGSAVE` and `BGREWRITEAOF` at once.

If `ldb_last_bgsave_status` is `err`, the usual causes are: no disk space in `dir`, wrong permissions on `dir`, or the fork failing for lack of memory (see `vm.overcommit_memory`). With `stop-writes-on-bgsave-error yes` (the default) Redis

then rejects writes with a **MISCONF** error until a save succeeds. Fix the disk or memory problem first; do not just turn the safeguard off, because it is telling you that you are not persisting.

11.4.6 Connection storms

Symptom	Likely cause	How to confirm	Fix
ERR max number of clients reached	Pools too large, connection leak, or a restart wave of many pods	INFO clients; rejected_connections; CLIENT LIST grouped by address/name	Reduce pool sizes; fix leaks (always return pooled connections); raise maxclients and the file descriptor limit
Application times out after Redis restarts	Thundering herd: every client reconnects and retries at once	Many new connections in total_connections_received rate	Backoff with jitter on reconnect; Lettuce reconnects automatically, tune timeouts
Idle connections vanish through a firewall or load balancer	Idle timeout shorter than client keepalive	Errors on first command after quiet period	Keep tcp-keepalive below the device timeout; enable client-side keepalive and periodic PING
Many connections from one client	Creating a new client per request	CLIENT LIST shows churn	Create one shared client or a pool at startup

Connection symptoms.

11.4.7 Replication problems

Symptom	Likely cause	How to confirm	Fix
Replica shows master_link_status:down	Network, wrong password, replica cannot reach primary	INFO replication on the replica; replica log	Check masterauth and masteruser , firewall, TLS settings
Replica endlessly full-resyncs	Backlog too small for the disconnection; output buffer limit closing the link	sync_full and sync_partial_ok / sync_partial_err in INFO stats ; primary log	Raise repl-backlog-size and the replica client-output-buffer-limit
Replication lag grows	Replica too slow, big commands, or network bandwidth	Compare master_repl_offset with slave_repl_offset	Faster replica; smaller values; avoid huge writes
Reads from replica return stale data	Replication is asynchronous	Expected	Read from primary for read-your-writes; use WAIT when needed
Primary refuses writes with NOREPLICAS	min-replicas-to-write not satisfied	INFO replication shows fewer good replicas	Restore the replicas; this protection is by design

Replication symptoms.

For Sentinel and Cluster specific operations (failover, slot repair), see the chapters on them in Part 5. The cluster symptom table follows.

11.4.8 Cluster fail states

Symptom	Likely cause	How to confirm	Fix
CLUSTERDOWN The cluster is down	<code>cluster_state:fail</code> ; some slots unassigned or their owners unreachable	<code>CLUSTER INFO</code> ; <code>redis-cli --cluster check host:port</code>	Restore the failed primary or promote its replica; with <code>cluster-require-full-coverage yes</code> the whole cluster stops when any slot is uncovered
Many MOVED errors	Client does not follow cluster topology	Client logs; <code>CLUSTER SHARDS</code> or <code>CLUSTER NODES</code>	Use a cluster-aware client with topology refresh (Lettuce: enable adaptive refresh)
CROSSSLOT errors	A multi-key command or transaction touches keys in different slots	Error text	Use hash tags such as <code>{user:42}:cart</code> to colocate keys
One shard overloaded	Hot key or big key, or uneven slot spread	<code>INFO</code> per node; <code>CLUSTER SLOT-STATS</code> (8.2+); <code>HOTKEYS</code>	Rebalance slots; split hot keys
Node flapping, failovers without cause	<code>cluster-node-timeout</code> too short for the latency; long fork	Logs show PFAIL/FAIL then recovery	Raise <code>cluster-node-timeout</code> ; fix the stalls that cause it
Slots stuck migrating or importing	Interrupted resharding	<code>redis-cli --cluster check</code> reports open slots	<code>redis-cli --cluster fix host:port</code> , or on Redis 8.4 and later use <code>CLUSTER MIGRATION</code> for atomic slot migration

Cluster symptoms.

INTERVIEW ANGLE

Redis became slow in production. What do you check first? Look at `INFO` and `SLOWLOG` for the global picture: is memory near the limit or swapping, did a fork or AOF fsync coincide with the spikes (`latest_fork_usec`, `LATENCY LATEST`), is a slow O(N) command in the slow log, are clients or CPU saturated? Then separate environment from Redis with `redis-cli --latency` from the client and `--intrinsic-latency` on the server, and change one thing at a time.

KEY POINTS

Troubleshooting

- Measure first: `INFO`, `SLOWLOG`, `LATENCY`, `CLIENT LIST` answer most questions in a minute.
- Latency spikes usually come from forks, fsync, slow O(N) commands, swapping or a saturated single thread.
- Cache memory problems come from missing TTLs, no `maxmemory`, or the wrong eviction policy; the OOM killer needs fork headroom.
- Find big keys with `--bigkeys/--memkeys`, delete them with `UNLINK`; find hot keys with `HOTKEYS` (8.6) or application metrics.
- Connection storms and cluster `CROSSSLOT`, `MOVED` and fail states have well-known causes; know the table, then the command.

11.5 Security Hardening

Redis was designed for trusted networks. It speaks a simple text protocol, it is very fast at guessing-friendly operations like `AUTH` attempts, and a single `FLUSHALL` from an attacker deletes everything. The official security guidance is blunt: access to the Redis port should be denied to everybody but trusted clients. Exposed, unauthenticated Redis servers on the public internet are found and abused routinely, so treat the steps in this chapter as the minimum, not extras.

Think in layers: the network decides who can connect at all, authentication decides who you are, ACLs decide what you may do, TLS protects the bytes on the wire, and operational hygiene covers secrets, dangerous commands and auditing.

11.5.1 Layer 1: network

- **Never expose Redis to the internet.** Put it in a private subnet or an internal Kubernetes namespace and allow only the application's addresses through security groups, firewall rules or `NetworkPolicy`.
- **Bind narrowly.** `bind 10.0.1.15 127.0.0.1` listens only on those interfaces. Docker needs care: `-p 6379:6379` publishes on all host interfaces, so use `-p 127.0.0.1:6379:6379` for local use.
- **Keep protected-mode yes.** Since version 3.2.0, when Redis runs with the default configuration (binding all interfaces) and no password, protected mode makes it answer only loopback clients. It is a safety net against an accidental misconfiguration, not a security design.
- **Do not run as root.** Redis does not need root privileges. A dedicated `redis` user limits the damage if someone gets command execution.
- **Cluster and replication ports.** The cluster bus port (the data port plus 10000) and the replication link also need to be reachable only between your nodes.

11.5.2 Layer 2: authentication and ACL design

Redis 6 introduced Access Control Lists. A fresh server has one user, `default`, equivalent to `user default on nopass ~* &* +@all`: enabled, no password, all keys, all channels, all commands. The old `requirepass` directive still works but only sets a password on that all-powerful `default` user, which is why it is the legacy method. The recommended approach is one user per application with only what it needs.

The ACL rule language

Rules are applied left to right and are incremental:

Rule	Meaning
on / off	Enable or disable authentication as this user
>password	Add a password (stored as a SHA-256 hash)
#hash	Add a password by its SHA-256 hex digest instead of plain text
~pattern	Allow keys matching the pattern; <code>~*</code> is all keys
%R~pattern, %W~pattern	Read-only or write-only access to matching keys (Redis 7.0+)
&pattern	Allow Pub/Sub channels matching the pattern
+@category, -@category	Add or remove a command category such as <code>@read</code> , <code>@write</code> , <code>@dangerous</code>
+command, -command	Add or remove one command; subcommands as <code>+config get</code> (7.0+)
(rules)	A selector: an extra independent rule set (7.0+)
reset	Return the user to its freshly created state

ACL rules you will use most.

A user with no key pattern cannot touch any key. Key patterns restrict only commands that name keys: `FLUSHALL`, `FLUSHDB` and `SWAPDB` are not limited by them, so exclude them explicitly or exclude `@dangerous`.

Users for ShopStream

/etc/redis/users.acl

CONF

```
user default off nopass sanitize-payload ~* &* -@all
user catalog on #HASH_CATALOG ~product:* ~cache:* +@read +@write +@connection -@dangerous
user cart on #HASH_CART ~cart:* ~session:* +@read +@write +@connection -@dangerous
user notifier on #HASH_NOTIFIER ~stream:orders &orders:* +@read +@write +@pubsub -@dangerous
user monitoring on #HASH_MON ~* +info +ping +slowlog +latency +client|list +config|get +memory|usage
user admin on #HASH_ADMIN ~* &* +@all
```

Replace the `HASH_...` placeholders with real SHA-256 hex digests of each password (64 hex characters). What to notice:

- The `default` user is switched `off` so a connection that does not authenticate can do nothing. Applications use their own users.
- Each service sees only its own key prefixes, so a bug or a leaked credential in the cart service cannot read the catalogue or delete streams.

- `-@dangerous` removes commands such as `FLUSHALL`, `KEYS`, `CONFIG`, `DEBUG`, `SHUTDOWN` and `REPLICAOF` from application users in one rule. After that, add back individual commands only if needed. Check the exact category membership of a command with `COMMAND INFO`.
- The monitoring user can read statistics but not application data.
- Selectors and `%R~` let you give read-only access to a prefix (for example a reporting job reading `~product:*` but not writing it).

Users can be defined inline in `redis.conf` or in an external file referenced by `aclfile`; the two methods cannot be mixed. With an ACL file, `ACL LOAD` reloads it (and keeps the old rules if any line is invalid) and `ACL SAVE` writes the current rules back. `CONFIG REWRITE` does not save ACLs.

Working with ACLs live

TERMINAL

```
127.0.0.1:6379> ACL SETUSER cart on >s3cret ~cart:* +@read +@write -@dangerous
OK
127.0.0.1:6379> ACL WHOAMI
"default"
127.0.0.1:6379> AUTH cart s3cret
OK
127.0.0.1:6379> SET cart:42 "x"
OK
127.0.0.1:6379> SET product:1 "x"
(error) NOPERM No permissions to access a key
127.0.0.1:6379> FLUSHALL
(error) NOPERM this user has no permissions to run the 'flushall' command
127.0.0.1:6379> ACL LOG 1
```

(The wording of NOPERM messages differs between versions.) Other commands worth knowing: `ACL LIST` and `ACL GETUSER name` show the effective rules, `ACL DRYRUN user command [arg ...]` tells you whether a user could run a command without running it, `ACL LOG` lists recent denied commands, failed `AUTH` attempts and key/channel violations (its size is `acllog-max-len`), and `ACL DELUSER` removes a user and closes its connections.

WATCH OUT

Redis 8 changed ACL behaviour for module commands. Commands from the built-in modules (JSON, Search, Time Series, probabilistic structures) now fall under the standard ACL categories. According to the 8.0 notes, a user with `+@all -@write` can no longer run `JSON.SET`, which it could before. Review every user you wrote before Redis 8 and add explicit rules where a service lost access, or gained it unintentionally through `+@read`.

TIP

Test every ACL user from the application's point of view. Write an integration test (Testcontainers is ideal) that starts Redis with your real `users.acl` and runs the service's commands as that user. A restrictive ACL that breaks the app in production is a common 3 a.m. surprise: Lua scripts, Streams commands and the commands your client library issues at connection time (for example `CLIENT SETINFO`, `HELLO`) all need to be allowed.

11.5.3 Layer 3: TLS

Without TLS, passwords and data cross the network in clear text, and `AUTH` itself is just another unencrypted command. Redis supports TLS on client connections, replication links and the cluster bus. It requires a Redis build with TLS (official packages and images include it).

redis.conf (TLS section)

CONF

```
port 0
tls-port 6379
tls-cert-file /etc/redis/tls/redis.crt
tls-key-file /etc/redis/tls/redis.key
tls-ca-cert-file /etc/redis/tls/ca.crt
tls-auth-clients yes
tls-replication yes
tls-cluster yes
```

`port 0` disables the plain TCP port entirely so nobody can fall back to it. `tls-auth-clients yes` is the default and enforces **mutual TLS**: clients must present a certificate signed by the CA. Set it to `no` if you want server-only TLS with passwords or ACL users for client identity. `tls-replication yes` on replicas uses TLS for the link to the primary, and `tls-cluster yes` encrypts the cluster bus. The command line client takes `--tls --cert --key --cacert`.

For Spring Boot, enable TLS and point at an SSL bundle (the bundle holds truststore and keystore details):

```

application.yml
YAML

spring:
  data:
    redis:
      host: redis.internal.shopstream.example
      port: 6379
      username: cart
      password: ${REDIS_PASSWORD}
      ssl:
        enabled: true
        bundle: redis-client
    ssl:
      bundle:
        pem:
          redis-client:
            truststore:
              certificate: file:/etc/ssl/redis/ca.crt
            keystore:
              certificate: file:/etc/ssl/redis/client.crt
              private-key: file:/etc/ssl/redis/client.key

```

The property names `spring.data.redis.ssl.enabled` and `spring.data.redis.ssl.bundle` come from Spring Boot's Redis support; confirm the PEM bundle property layout in the Spring Boot SSL reference for your exact version. TLS adds CPU cost on both sides; Redis 8 supports I/O threading together with TLS, which helps on busy servers.

11.5.4 Layer 4: secrets, dangerous commands, audit

Secrets

- Never commit passwords to Git or bake them into images. Inject them from a secrets manager, Kubernetes Secret or environment variable (`${REDIS_PASSWORD}` as above).
- Redis config and ACL files hold passwords or password hashes. Restrict their permissions (for example mode 640, owned by `redis`) and prefer `#sha256hash` entries in ACL files so the plain text is not stored at all.
- Rotate: ACL lets a user hold two passwords at once. Add the new one with `ACL SETUSER cart >new`, roll out the application, then remove the old one with `ACL SETUSER cart <old`.
- Use long random passwords. Redis can check passwords very quickly, so brute-force protection depends on password strength and on network restriction.

Dangerous commands

The old approach of renaming commands with `rename-command` is deprecated according to the current security documentation, which recommends ACL rules instead. Remove from application users the commands that can destroy or reconfigure the server: `FLUSHALL`, `FLUSHDB`, `CONFIG`, `DEBUG`, `SHUTDOWN`, `REPLICAOF`, `MODULE`, `KEYS`, `ACL`. Keep `enable-debug-command no` and `enable-module-command no` (the defaults) in the server configuration. `CONFIG SET` can change `dir` and `dbfilename`, which an attacker may use to make Redis write files to arbitrary locations, so never let untrusted clients have `CONFIG`.

Audit

Open source Redis has no complete built-in audit log of every command. What you can do:

- Read `ACL LOG` regularly (or export it) for denied operations and failed logins.
- Send the server log (`logfile`) to central logging and alert on authentication failures, replication changes and persistence errors.
- Log at the network and orchestration layer: who can reach port 6379, who ran `kubectl exec` into the pod.
- Use short, named connections (`CLIENT SETNAME`) so `CLIENT LIST` shows which service owns each connection.
- Avoid `MONITOR` in production: it streams every command, including passwords sent with `AUTH`, and slows the server.

REAL WORLD

A hardening checklist before go-live Private network only; `protected-mode yes`; `default` user off; per-service ACL users with key prefixes; TLS everywhere including replication; secrets from a manager; `-@dangerous` for apps; separate admin credentials; persistence and backups configured; alerts for `ACL LOG` growth. If you cannot tick every item, say which ones you accepted as risk.

INTERVIEW ANGLE

How do you secure a Redis instance? Isolate it on a private network and bind it to specific interfaces; never expose it publicly. Disable the passwordless `default` user and give each application an ACL user limited to its key patterns and command categories (excluding `@dangerous`). Encrypt connections with TLS, including replication and cluster traffic, keep secrets out of source control, and watch `ACL LOG`. `requirepass` alone is the legacy minimum, and `rename-command` is deprecated in favour of ACLs.

KEY POINTS

Security

- Redis assumes a trusted network: restrict reachability first, and never expose it to the internet.
- Replace the open `default` user with per-service ACL users limited by key pattern and command category.
- Use TLS for clients, replication and the cluster bus; mutual TLS adds client identity.
- Keep secrets out of code; store ACL passwords as hashes; rotate by overlapping two passwords.
- Remove destructive commands from application users with ACLs; do not rely on `rename-command`.
- Redis 8 module commands fall under standard ACL categories: re-test users after upgrading.

11.6 Backup, Restore and Disaster Recovery

Replication is not a backup. If someone runs `FLUSHALL` or a buggy deployment deletes keys, the replicas faithfully delete them too within milliseconds. A backup is a separate, point-in-time copy stored somewhere else, and it only counts once you have restored from it successfully.

11.6.1 Decide what you need to protect

Not all Redis data needs backup. Ask for each ShopStream use:

Data	If lost	Backup need
Catalogue cache, rate-limit counters	Rebuilt from the database as traffic arrives	None; plan for warm-up load on the database
Sessions	Users must log in again	Persistence on, backups optional
Carts, idempotency keys, order streams	Customer-visible loss or double processing	Persistence and regular backups
Primary store use (Redis is the system of record)	Business data lost	Full backup, tested restore, off-site copy

What deserves a backup.

Two numbers define the plan: **RPO** (recovery point objective: how much recent data you can afford to lose) and **RTO** (recovery time objective: how long you can be down). `appendfsync everysec` gives an RPO of about a second against a crash, while hourly RDB copies give an RPO of an hour against a deletion or corruption. RTO depends on dataset size: Redis loads an RDB file at roughly the speed of your disk and CPU, so a large instance may need minutes to start.

11.6.2 RDB backups

An RDB file is never modified after it is produced: Redis writes a temporary file and renames it atomically when the snapshot is complete, so copying the file while the server runs is safe. The official recommendation:

- A cron job that copies `dump.rdb` hourly into one directory and daily into another, with the date in the file name.
- Delete old copies with `find` (for example hourly for 48 hours, daily for one or two months).
- At least once a day, move a copy outside the machine, ideally outside the data centre, and encrypt it.
- Verify the transfer (size and checksum), and have an independent alert if fresh backups stop arriving.

Taking and checking an RDB backup

TERMINAL

```
127.0.0.1:6379> BGSAVE
Background saving started
127.0.0.1:6379> LASTSAVE
(integer) 1760000200
$ cp /var/lib/redis/dump.rdb /backup/dump-$(date +%Y%m%d-%H%M).rdb
$ redis-check-rdb /backup/dump-20261010-0200.rdb
[offset 0] Checking RDB file /backup/dump-20261010-0200.rdb
[offset 26] AUX FIELD redis-ver = '8.0.0'
[offset 0] \o/ RDB looks OK! \o/
```

(Output shortened.) `LASTSAVE` returns the Unix time of the last successful save; compare it before and after `BGSAVE` to know it finished, or watch `rdb_bgsave_in_progress` in `INFO persistence`. To pull a snapshot from a remote server without shell access to it, `redis-cli --rdb /backup/remote.rdb -h host` fetches an RDB over the replication protocol.

11.6.3 AOF backups

Since Redis 7.0 the append-only file is a directory (`appenddirname`) containing a base file, incremental files and a manifest. Copying the directory is a valid backup unless a rewrite happens while you copy, which could leave an inconsistent set. The documented procedure:

1. Turn off automatic rewrites: `CONFIG SET auto-aof-rewrite-percentage 0`, and do not run `BGREWRITEAOF`.
2. Check `aof_rewrite_in_progress` in `INFO persistence` is 0; if it is 1, wait.
3. Copy the files in the AOF directory (or create hard links and copy those, so rewrites can be re-enabled sooner).
4. Restore the setting: `CONFIG SET auto-aof-rewrite-percentage 100` (your previous value).

If you run with both RDB snapshots and AOF, backing up the RDB files is usually simpler and the copy is a single compact file.

11.6.4 Restoring, and the tested restore

Restoring an RDB

1. Stop the Redis process (or a new instance you are restoring into).
2. Put the RDB file in the configured `dir` with the configured `dbfilename`.
3. Start Redis and check `INFO keypace` and a few known keys.

GOTCHA

Restoring an RDB into an instance with AOF enabled. When `appendonly yes` is set, Redis loads from the AOF files at startup and ignores `dump.rdb`. If the AOF directory is empty, it starts empty and your RDB is not loaded. Restore into a server started with `appendonly no`, check the data, then run `CONFIG SET appendonly yes` so Redis writes an AOF from the in-memory data, and finally set `appendonly yes` in the config file.

Repairing a damaged AOF

A server that crashed mid-write may leave a truncated last command. `redis-check-aof --fix` on the affected file truncates the file to the last valid command (Redis can also do this itself with `aof-load-truncated yes`, the default). Always copy the files before running `--fix`, because it drops data.

Prove it with a restore drill

An untested backup is a hope. Schedule a drill, at least quarterly, that does the following automatically if possible:

1. Pull the latest off-site backup, verify its checksum, and decrypt it.
2. Start a scratch Redis from it (a container is ideal).
3. Run sanity checks: key counts within an expected range from `DBSIZE`, a handful of known keys, `redis-check-rdb` passes.
4. Record how long it took. That is your measured RTO.

11.6.5 Disaster recovery across regions

Disaster recovery adds copying backups to other locations, and optionally keeping a standby copy ready.

- **Backup shipping.** Encrypted snapshots to object storage in another region, or to servers at another provider. This is the cheapest and always worth doing.
- **Cross-region replica.** A replica in another region follows the primary asynchronously over a long link, so it lags and is behind by definition: it reduces RTO but its RPO is not zero. Use TLS on the link, enlarge `repl-backlog-size` to survive link outages and expect a full resync after a long one. Promote it manually (`REPLICAOF NO ONE`) when the region is lost, then point applications at it.
- **Active-active with conflict resolution** is a feature of commercial Redis offerings, not of the open-source server; if you need that, evaluate it separately.
- **Runbooks.** Write down the failover steps, who may decide, how clients find the new primary (DNS, Sentinel, configuration), and how you fail back. Rehearse it.

WATCH OUT

Persistence off plus automatic restart can erase your replicas. The replication documentation describes this trap: if a primary has persistence disabled and an automatic restart brings it back empty, its replicas resynchronize from the empty primary and destroy their copies of the data. If persistence is off on a primary that matters, disable automatic restart, or keep persistence on the primary.

INTERVIEW ANGLE

Is a replica a backup? No. Replication copies every change, including accidental deletions and corruption, almost instantly. A backup is an independent point-in-time copy, such as an RDB snapshot stored off the machine, that you have tested restoring. Replicas protect against hardware failure and give availability; backups protect against data mistakes and disasters.

KEY POINTS

Backup and recovery

- Replication is for availability; backups are separate point-in-time copies kept elsewhere.
- RDB files can be copied safely while Redis runs; for AOF directories, pause rewrites while copying.
- Ship encrypted copies off the machine, verify them, and alert when shipping stops.
- A backup is real only after a restore drill; measure RTO.
- With `appendonly yes`, Redis loads the AOF, not `dump.rdb`: restore through a server with AOF off, then enable it.

11.7 Upgrades and Migrations

Upgrading Redis is routine if you do it one node at a time with a rollback plan, and painful if you do it all at once on a Friday. This chapter gives procedures for each topology and notes the changes to watch for when moving from Redis 7 to Redis 8, from Redis Stack to Redis Open Source, or between Redis and other stores.

11.7.1 Before any upgrade

1. **Read the release notes** of every version between yours and the target, looking for breaking changes, changed defaults, removed commands and deprecated configuration.
2. **Back up** and perform a test restore (see the previous chapter).
3. **Rehearse in staging** with a copy of production data, and run your application's integration tests against the new version. Testcontainers makes it easy to run the same tests against two image tags.
4. **Check client compatibility:** the Redis client library (Lettuce, Jedis, Redisson) and Spring Data Redis versions must support the new server. New server versions generally stay backward compatible with older clients, but commands you plan to use must exist in the client.
5. **Plan rollback.** Newer RDB and AOF files may not be readable by older versions, so once data has been written by the new version you may not be able to go back to the old binary with the same files. Keep the pre-upgrade backup and decide the point of no return in advance.

11.7.2 Rolling upgrade: primary with replicas

The principle: upgrade replicas first, then switch roles, then upgrade the old primary. Clients should keep working throughout.

1. Upgrade one replica. Stop it gracefully with `SHUTDOWN SAVE` (a replica stopped cleanly stores in its RDB file the information needed to resync), start it on the new version, and wait until `master_link_status:up` and the offsets match the primary.
2. Repeat for the other replicas, one at a time.
3. Fail over. With Sentinel, run `SENTINEL FAILOVER mymaster`, so a replica running the new version is promoted and clients follow. Without Sentinel, promote a replica with `REPLICAOF NO ONE`, point clients to it, and make the old primary a replica of it.
4. Upgrade the old primary (now a replica) the same way as step 1.

GOTCHA

A replica restarted from an AOF file cannot do a partial resync. The replication documentation notes that a replica that restarted via AOF cannot be partially synchronized. Switch the replica to RDB persistence before shutting it down, restart, then enable AOF again, or accept a full resync. For large datasets, a full resync costs a fork on the primary and heavy bandwidth, so schedule it off-peak.

11.7.3 Rolling upgrade: Redis Cluster

Do the same dance per shard, one shard at a time:

1. Upgrade the replicas of a shard (restart each on the new version and let it resync).
2. On an upgraded replica, run `CLUSTER FAILOVER` to promote it; this is a coordinated manual failover that waits for the replica to catch up, so no writes are lost.
3. Upgrade the former primary, which is now a replica.
4. Check `CLUSTER INFO` (`cluster_state:ok`) and `redis-cli --cluster check` before moving to the next shard.

Avoid resharding or adding nodes at the same time as upgrading. From Redis 8.4, atomic slot migration (`CLUSTER MIGRATION`) makes moving slots between nodes safer, but keep that a separate change from the version upgrade.

11.7.4 Moving from Redis 7 to Redis 8

What to check, based on the 8.0 release notes and what-is-new page:

- **Modules are built in.** Redis 8 merges Redis Stack and Redis Community Edition into one distribution, Redis Open Source, with JSON, Search and Query, Time Series and the probabilistic data structures included. If your configuration contains `loadmodule` lines for those modules, remove them rather than loading old module files over built-in ones, and re-read the release notes before starting.
- **ACLs.** Module commands fall under standard categories now. Re-test users that rely on negative rules such as `+@all -@write`.
- **Search behaviour.** The 8.0 notes list stricter validation of some `FT.` arguments, errors instead of empty results for invalid input, changed default reducer values, and BM25 as the default scorer instead of TF-IDF. If you run search queries, re-run your tests and compare results.
- **Persistence and replication** features such as multi-part AOF (7.0) are unchanged in concept; the new version still reads the previous version's files, but test this on your own data first.
- **Newer features are optional.** Hash field commands such as `HGETEX`, `HSETEX` and `HGETDEL` (8.0), `XDELEX` and `XACKDEL` (8.2), and the 8.4 and 8.6 additions need a matching server, so do not use them in application code until every environment, including developers' containers and CI, runs a server that supports them.
- **License.** Check the license terms of the version you deploy and whether they fit your organization and distribution model.

11.7.5 Moving from Redis Stack or standalone modules

If you ran the `redis-stack` image or loaded Redisearch or RedisJSON modules into Redis 7, the target is the Redis 8 image, which contains them. The sequence: take a backup, start a Redis 8 instance on a copy of the data, verify that indexes (`FT.INFO`), JSON documents (`JSON.GET`) and time series are present and queries behave the same, then switch applications over. If the data does not load or an index is missing, treat it as a migration problem and consult the release notes; do not try to guess file compatibility.

11.7.6 Moving data between instances and other stores

Approach	When to use	Notes
Replication (<code>REPLICAOF</code> new instance, then promote)	Same engine, minimal downtime, e.g. moving to new hardware or another cloud	The new node syncs everything and keeps up with writes; cut over when lag is zero
RDB file restore	Planned downtime acceptable	Simple and verifiable; copy the file, start the new server
<code>redis-cli --pipe</code>	Bulk load from a generated stream of commands in RESP	Fast mass insertion; generate the protocol file from the source
<code>SCAN</code> plus <code>DUMP/RESTORE/MIGRATE</code>	Selective copy between instances	Avoids loading all keys at once; handle TTLs and keys written during the copy
Dedicated migration tools	Cross-vendor or live sync with checks	Evaluate their documentation for supported versions and cluster modes
Application dual-writes	Replacing Redis usage with another store (or the reverse)	Write to both, read from the new one, compare, then drop the old

Migration approaches.

When moving a cache, you may not need to migrate data at all: start the new instance empty and accept a period of low hit ratio, protecting the database with a stampede guard (see the caching patterns in Part 8) and a gradual traffic shift.

When migrating data **from** a relational database into Redis (or between Redis and another store), decide the source of truth, make the load idempotent (repeatable without duplicates), and verify counts and sample records before switching reads.

11.7.7 Valkey and other compatible servers

Valkey is an open-source fork of Redis created from version 7.2.4 under the Linux Foundation, and other Redis-protocol compatible servers exist. At the protocol and core command level (strings, hashes, lists, sets, sorted sets, streams, replication, Cluster, Sentinel) applications written for Redis 7.2 generally work, and Redis clients like Lettuce, Jedis and Redisson speak to them. The two projects have since added different features, so compatibility is not guaranteed beyond the shared base. Features specific to newer Redis versions, such as the Redis 8 built-in JSON and search commands, vector sets or `HGETEX`, may be missing or implemented differently elsewhere. Before moving in either direction, list the commands and modules you use, test them against the target, and read both projects' documentation. Also read the license terms of each option for your use case.

REAL WORLD

An upgrade timeline that rarely surprises Week 1: read notes, upgrade staging, run tests. Week 2: upgrade one production replica and watch it for several days. Week 3: failover and upgrade the rest. At each step you can stop. The riskiest upgrades are the ones squeezed into a single maintenance window with no rehearsal.

INTERVIEW ANGLE

How do you upgrade a Redis primary-replica pair with minimal downtime? Upgrade the replica first and let it resync, then fail over so that the upgraded replica becomes the primary (with Sentinel or a manual promotion), then upgrade the old primary, which is now a replica. Take a backup first, read the release notes, rehearse in staging, and be aware that newer data files may not be readable by older versions, so rollback means restoring the pre-upgrade backup.

KEY POINTS

Upgrades and migrations

- Read release notes, back up, rehearse, and decide the point of no return before touching production.
- Upgrade replicas first, fail over, then upgrade the old primary; in a cluster do it one shard at a time with **CLUSTER FAILOVER**.
- Redis 8 includes the former Stack modules and changes ACL categories and some search behaviour; remove old `loadmodule` lines and re-test users and queries.
- Do not use new commands in code until every environment runs a server that supports them.
- Choose a migration method by downtime tolerance: replication for live moves, RDB for planned downtime, tools or dual-writes across stores.
- Compatibility with Valkey and other servers holds for the shared core, not for newer or module features; test your command set.

PART 12

Revision Kit and Interview Preparation

Condensed reference sheets, decision guides and interview practice. Use it after the main chapters, not instead of them.

In this part

- 12.1 The One-Page Mental Model
- 12.2 Command Cheat Sheets by Structure
- 12.3 Configuration and Metrics Cheat Sheet
- 12.4 Decision Guides
- 12.5 Interview Q&A: Fundamentals
- 12.6 Interview Q&A: Data Structures
- 12.7 Interview Q&A: Internals and Persistence
- 12.8 Interview Q&A: Scaling and High Availability
- 12.9 Interview Q&A: Caching and Patterns
- 12.10 Interview Q&A: Java and Spring
- 12.11 System-Design Scenarios
- 12.12 Common Mistakes and Myths
- 12.13 A 30-Day Study Plan

12.1 The One-Page Mental Model

This chapter compresses the whole book into one picture. If you can explain every box below from memory, you understand Redis well enough to build with it and to defend your choices in an interview.

12.1.1 What Redis is, in five sentences

Redis is an in-memory data structure server: clients send commands over TCP (the RESP protocol), and the server keeps strings, hashes, lists, sets, sorted sets, streams and more in RAM. One main thread executes commands one at a time, so each command is atomic and there are no locks inside the engine. Because data lives in memory, typical operations take microseconds, and the network round trip usually dominates. Durability is optional and tunable (RDB snapshots, the append-only file, or both). Scale-out and availability come from replication, Sentinel and Cluster, each with its own trade-offs.

12.1.2 The map

The whole book on one page

```

CLIENTS (Lettuce / Jedis / Redisson / Spring Data Redis)
  | RESP over TCP, pipelining, pooling or one shared connection
  v
+----- ONE REDIS SERVER -----+
| event loop: read -> parse -> execute (single thread) -> reply |
| I/O threads (optional): socket read/write only                |
| background threads: lazy free, AOF fsync, close               |
|                                                                 |
| keyspace (db 0..15; Cluster: db 0 only)                       |
|   key -> value of one TYPE, stored in one ENCODING            |
|   string | list | hash | set | zset | stream | bitmap | HLL | geo |
|   JSON | Search | time series | Bloom/Cuckoo/CMS/Top-K | vector set |
|                                                                 |
| per-key: TTL (lazy + active expiry)      global: maxmemory + eviction |
| atomic extras: MULTI/EXEC/WATCH, Lua (EVAL), Functions (FCALL) |
+-----+-----+-----+
|               |               |
| persistence (disk)      replication (network)
| RDB snapshot / AOF      async primary -> replicas
|                           |
| Sentinel: failover for ONE shard
| Cluster: 16384 slots over MANY shards

```

12.1.3 Five laws worth memorising

1. **One thread, so slow commands hurt everyone.** An $O(N)$ command on a big key blocks all clients. Avoid **KEYS**, huge **SMEMBERS**, long Lua scripts.
2. **Memory is the budget.** Everything is in RAM, so you size for data plus overhead plus fork headroom, and you decide what happens at **maxmemory** (eviction or errors).
3. **Replication is asynchronous.** A failover can lose the last acknowledged writes. **WAIT** narrows the window but does not make Redis a consensus system.
4. **Persistence is a dial, not a switch.** No persistence, RDB, AOF with **everysec**, AOF with **always**: each has a different data-loss window and cost.
5. **Redis is usually a cache or a fast secondary store.** Keep a system of record elsewhere unless you have deliberately designed and tested the durability story.

12.1.4 ShopStream in one table

Key	Type	Purpose
product:{id}	hash or JSON	Catalogue cache (cache-aside, TTL with jitter)
session:{sid}	hash + TTL	Login sessions
cart:{userId}	hash	Shopping cart
stock:{sku}	string counter	Flash-sale stock, decremented atomically in Lua
rl:{ip}:{route}	string or hash	Rate limit state
lb:bestsellers:daily	sorted set	Leaderboard
stream:orders	stream + group notifier	Order events, at-least-once processing
uv:{date}	HyperLogLog	Unique visitors
geo:stores	geo (sorted set)	Store locator
idem:{key}	string NX + TTL	Idempotency keys
lock:order:{id}	string NX PX	Short distributed lock

Redis roles in the running example.

KEY POINTS

- Redis = single-threaded command execution over in-memory structures, with optional persistence and asynchronous replication.
- Every design question reduces to: which structure, how big, how long it lives, what happens when memory is full, and what you can afford to lose.
- Atomicity comes from the single thread: single commands, **MULTI/EXEC**, Lua and Functions are all executed without interleaving.

12.2 Command Cheat Sheets by Structure

Complexity is shown as Big-O where **N** is the size of the collection and **M** the number of elements touched. Commands marked "legacy" still work but have a modern replacement.

12.2.1 Keys, expiry and server basics

Command	What it does	Cost
DEL key / UNLINK key	Delete a key (UNLINK frees memory in a background thread)	O(1) / O(N) for DEL on big values
EXISTS key	Count how many of the given keys exist	O(1) per key
TYPE key	Type name of the value	O(1)
EXPIRE key sec / PEXPIRE	Set TTL (options NX, XX, GT, LT)	O(1)
TTL key / PTTL key	Remaining time; -1 no TTL, -2 no key	O(1)
PERSIST key	Remove the TTL	O(1)
SCAN cursor MATCH p COUNT n	Incremental iteration, safe for production	O(1) per call
RENAME a b / COPY a b	Rename or copy a key	O(1) / O(N)
OBJECT ENCODING key	Internal encoding (listpack, hashtable, ...)	O(1)
MEMORY USAGE key	Bytes used by a key and its value	O(N)

Generic key commands.

12.2.2 Strings and counters

Command	What it does	Cost
SET k v [NX or XX] [EX s or PX ms or KEPTTL] [GET]	Store a value with optional condition and expiry	O(1)
GET k / GETDEL k / GETEX k EX s	Read; read and delete; read and change TTL	O(1)
MSET / MGET	Many keys in one round trip	O(N)
INCR / INCRBY / DECR / INCRBYFLOAT	Atomic counters	O(1)
APPEND / STRLEN / GETRANGE / SETRANGE	Byte-level edits	O(1) or O(N)
SETBIT / GETBIT / BITCOUNT / BITOP	Bitmaps (DAU, flags)	O(1) / O(N)
SETEX, SETNX, GETSET (legacy)	Replaced by SET options	O(1)

String commands.

12.2.3 Hashes, lists and sets

Command	What it does	Cost
HSET k f v [f v ...]	Set fields (HMSET is legacy)	O(N) fields given
HGET / HMGET / HGETALL	Read one, some, or all fields	O(1) / O(N)
HDEL / HEXISTS / HLEN / HKEYS / HVALS	Field maintenance and listing	O(N) for listing
HINCRBY / HINCRBYFLOAT	Atomic field counters	O(1)
HSCAN	Incremental field iteration	O(1) per call
HEXPIRE / HPEXPIRE / HTTL	Per-field TTL (7.4+)	O(N) fields
HGETEX / HSETEX / HGETDEL	Read or write fields together with TTL changes (8.0+)	O(N) fields

Hash commands.

Command	What it does	Cost
LPUSH / RPUSH	Add at head or tail	O(1) per element
LPOP / RPOP [count]	Remove from head or tail	O(1) per element
LRANGE k start stop	Read a slice (0 -1 means all)	O(S+N)

Command	What it does	Cost
LLEN / LINDEX / LSET / LREM / LTRIM	Inspect and edit	LINDEX is $O(N)$
BLPOP / BRPOP / BLMOVE	Blocking pop and reliable move	$O(1)$
LMOVE src dst LEFT RIGHT	Atomic move between lists (reliable queue)	$O(1)$

List commands.

Command	What it does	Cost
SADD / SREM / SISMEMBER	Add, remove, test membership	$O(1)$ per element
SMEMBERS / SCARD / SSCAN	List all (careful), count, iterate	$O(N)$ / $O(1)$
SINTER / SUNION / SDIFF (+STORE)	Set algebra	$O(N*M)$ worst case
SINTERCARD numkeys keys...	Size of the intersection, no materialisation	$O(N*M)$
SPOP / SRANDMEMBER	Random elements	$O(1)$ or $O(N)$

Set commands.

12.2.4 Sorted sets and streams

Command	What it does	Cost
ZADD k [NX or XX] [GT or LT] score member	Add or update with a score	$O(\log N)$
ZINCRBY k inc member	Atomic score change	$O(\log N)$
ZRANK / ZREVRANK / ZSCORE	Position and score of a member	$O(\log N)$ / $O(1)$
ZRANGE k start stop [REV] [BYScore or BYLEX] [LIMIT] [WITHSCORES]	Unified range query (since 6.2)	$O(\log N + M)$
ZREM / ZREMRANGEBYRANK / ZREMRANGEBYSCORE	Remove	$O(\log N + M)$
ZCARD / ZCOUNT	Size, count in score range	$O(1)$ / $O(\log N)$
ZPOPMIN / ZPOPMAX / BZPOPMIN	Pop extremes (priority queue)	$O(\log N)$
ZUNIONSTORE / ZINTERSTORE	Combine sets with weights	$O(N) + O(M \log M)$

Sorted set commands.

Command	What it does	Cost
XADD k [MAXLEN ~ n] * f v ...	Append an entry, auto-generated ID	$O(1)$
XRANGE / XREVRANGE	Read by ID range	$O(N)$
XREAD [BLOCK ms] STREAMS k id	Read as a plain log	$O(N)$
XGROUP CREATE k g id [MKSTREAM]	Create a consumer group	$O(1)$
XREADGROUP GROUP g c [COUNT n] [BLOCK ms] STREAMS k >	Read new entries for a consumer	$O(M)$
XACK k g id	Acknowledge, remove from the PEL	$O(1)$ per ID
XPENDING / XCLAIM / XAUTOCLAIM	Inspect and take over stuck entries	varies
XTRIM / XLEN / XINFO	Trim, length, introspection	varies

Stream commands.

12.2.5 Probabilistic, geo and other types

Command	What it does	Cost
PFADD / PFCOUNT / PFMERGE	HyperLogLog unique count, about 0.81% standard error	O(1)
GEOADD / GEOSEARCH / GEODIST	Geospatial index on a sorted set	O(log N)
BF.ADD / BF.EXISTS / BF.RESERVE	Bloom filter (no false negatives)	O(k)
JSON.SET / JSON.GET	JSON documents with path access	depends on path
FT.CREATE / FT.SEARCH / FT.AGGREGATE	Query engine: full text, tags, vectors	depends on query
TS.ADD / TS.RANGE	Time series	O(1) / O(N)

Other commands.

12.2.6 Transactions, scripting, pub/sub, admin

Command	Use
MULTI / EXEC / DISCARD / WATCH	Queue commands and run them without interleaving; WATCH gives optimistic locking; no rollback
EVAL script numkeys keys... args... / EVALSHA	Run Lua atomically; pass key names as KEYS , values as ARGV
FUNCTION LOAD / FCALL	Named, versioned server-side functions (7.0+)
PUBLISH / SUBSCRIBE / PSUBSCRIBE	Fire-and-forget messaging, no persistence
SPUBLISH / SSUBSCRIBE	Sharded pub/sub (7.0+)
INFO [section] / CONFIG GET / CONFIG SET	Observe and tune a running server
SLOWLOG GET / LATENCY DOCTOR	Find slow commands and latency spikes
CLIENT LIST / CLIENT KILL	Connections and their state
BGSAVE / BGREWRITEAOF / LASTSAVE	Persistence control
REPLICAOF host port / WAIT n ms	Replication control, wait for replica acks
ACL SETUSER / ACL LIST / ACL WHOAMI	Users and permissions

Everything else.

KEY POINTS

- Learn complexity per command: **O(1)** and **O(log N)** are safe; **O(N)** on unbounded collections is where incidents come from.
- Prefer the modern forms: **SET** options over **SETNX/SETEX**, **HSET** over **HMSET**, **ZRANGE** over **ZREVRANGE**, **SCAN** over **KEYS**.
- When unsure, look at the command page: **COMMAND DOCS name** in redis-cli prints a summary and the complexity.

12.3 Configuration and Metrics Cheat Sheet

12.3.1 Configuration directives you must know

Directive	Default	Why it matters
bind / protected-mode	local / yes	Keep Redis off public networks
port	6379	Cluster bus uses port + 10000
requirepass / ACL	none	Authentication; prefer ACL users
maxmemory	0 (unlimited)	Set it, or the OS OOM killer decides
maxmemory-policy	noeviction	Writes fail at the limit unless you choose an eviction policy
maxmemory-samples	5	Accuracy of approximated LRU/LFU
appendonly	no	Turns the AOF on
appendfsync	everysec	always, everysec or no
auto-aof-rewrite-percentage	100	Rewrite when the AOF doubles
auto-aof-rewrite-min-size	64mb	Do not rewrite tiny files
save	3600 1 300 100 60 10000	RDB snapshot rules (seconds, changes)
repl-backlog-size	1mb	Larger backlog lets replicas do partial resyncs
min-replicas-to-write	0	Refuse writes when too few replicas are in sync
cluster-enabled	no	Starts the node in cluster mode
cluster-require-full-coverage	yes	Stop serving if any slot is uncovered
maxclients	10000	Connection limit (also bounded by file descriptors)
slowlog-log-slower-than	10000	Microseconds; commands slower than this are logged
io-threads	1	Extra threads for socket I/O only
lazyfree-lazy-eviction and friends	see docs	Free memory in the background
hash-max-listpack-entries	512	Compact encoding threshold for hashes

Core configuration directives (defaults as documented for recent releases; verify with CONFIG GET on your build).

NOTE

Defaults change between releases. The **save** default and some **lazyfree-*** defaults differ across versions, so check **CONFIG GET** on the server you actually run and the release notes before relying on a number from this table.

12.3.2 Metrics worth watching

Field (section)	Look for	Meaning
used_memory / used_memory_rss (memory)	RSS far above used_memory	Fragmentation or allocator overhead
mem_fragmentation_ratio (memory)	above about 1.5 or below 1.0	Fragmentation, or swapping if below 1
evicted_keys (stats)	rising	Memory pressure; check policy and sizing
expired_keys (stats)	trend	Normal TTL activity
keyspace_hits / keyspace_misses (stats)	hit ratio = hits / (hits + misses)	Cache effectiveness
instantaneous_ops_per_sec (stats)	sudden drop or spike	Load change or blocked server
connected_clients / blocked_clients (clients)	growth	Connection leaks or blocking commands
rejected_connections (stats)	any value above 0	maxclients reached
latest_fork_usec (stats)	large values	Fork stall on big datasets
rdb_last_bgsave_status / aof_last_write_status (persistence)	not ok	Persistence is failing
master_repl_offset, slave_repl_offset (replication)	gap	Replication lag in bytes
connected_slaves (replication)	fewer than expected	A replica dropped (field name kept for compatibility)

INFO fields and what they tell you.

12.3.3 First-aid commands

REDIS

```
# What is slow right now?
SLOWLOG GET 10
LATENCY DOCTOR
# Who is connected and what are they doing?
CLIENT LIST
# Where is the memory?
MEMORY DOCTOR
MEMORY USAGE product:42
# Big keys, from the shell: redis-cli --bigkeys (uses SCAN, safe-ish)
INFO memory
INFO stats
INFO replication
```

KEY POINTS

- Always set **maxmemory** and a deliberate **maxmemory-policy**.
- Hit ratio, evictions, latency, fragmentation, replication offset gap and persistence status are the six numbers to graph.
- Verify defaults on your own build with **CONFIG GET** instead of trusting memory.

12.4 Decision Guides

12.4.1 Which data structure?

You need	Use	Why
Opaque value, counter, flag, lock token	String	Simplest, atomic INCR , SET NX
An object with several fields, partial updates	Hash	Field-level access, compact when small
Queue, recent items, capped history	List or Stream	List is simple, Stream adds groups and acks
Unique membership, tags, set algebra	Set	O(1) membership
Ranking, priority, time-ordered index, range by score	Sorted set	Order maintained on write
Durable event log with consumers	Stream	Consumer groups, pending list, replay
Approximate unique count	HyperLogLog	12 KB at most, small error
Yes/no per user id	Bitmap	1 bit per id
Nearby places	Geo	Radius and box search
Existence check with tolerated false positives	Bloom filter	No false negatives
Nested document with queries	JSON + Search	Query without a separate engine

Choosing a structure.

12.4.2 Which persistence mode?

Situation	Choice	Data-loss window
Pure cache, can be rebuilt	None, or RDB for warm restarts	Everything on crash (acceptable)
Fast restarts, some loss acceptable	RDB only	Minutes since last snapshot
Typical primary data store use	AOF everysec (with RDB preamble)	About one second
Strict durability	AOF always	Last command at most, slower writes
Maximum safety	AOF + replicas + backups	Depends on replica lag and failover

Choosing persistence.

12.4.3 Which topology?

Need	Topology	Main trade-off
Dev, test, small cache	Single node	No failover
Read scaling, a hot standby	Primary + replicas	Manual failover, stale reads possible
Automatic failover, data fits one node	Sentinel	Still one write node; async replication
Data or throughput beyond one node	Cluster	Multi-key limits to one slot, client must be cluster aware
Managed operations	Cloud service	Cost, feature differences, vendor limits

Choosing a topology.

12.4.4 Which cache pattern?

Pattern	Use when	Watch out for
Cache-aside	Default; reads dominate; app owns the logic	Stampede, stale data after writes
Read-through	A library should hide loading	Less control, same stampede risk
Write-through	Reads must see fresh data	Slower writes, cache churn
Write-behind	Very high write rate, tolerates loss	Data loss on crash, ordering
Refresh-ahead	Hot, predictable keys	Wasted refreshes

Choosing a caching pattern.

12.4.5 Lock or no lock?

Ask in this order, and stop at the first yes:

1. Can a single atomic command or a Lua script do the job (**INCR**, **SET NX**, stock decrement)? Use that, no lock.
2. Can the database enforce it (unique constraint, optimistic version column)? Use that.
3. Is the consequence of two holders running at once only wasted work (a cache rebuild)? A simple **SET NX PX** lock with a token is enough.
4. Does correctness depend on mutual exclusion (money, inventory ledgers)? Do not trust a timed lock alone; add fencing tokens checked by the protected resource, or serialise through a queue.

KEY POINTS

- Choose structures by access pattern, not by habit.
- Persistence and topology choices are about what you can afford to lose and how long you can afford to be down.
- The best lock is the one you designed away.

12.5 Interview Q&A: Fundamentals

How to use the next chapters: cover the answer, say yours out loud in under a minute, then compare. Each answer is deliberately short; in a real interview add one example from a project. Questions are numbered within their group only by order, never by a fixed number, so you can shuffle them.

Q: What is Redis, and what problem does it solve?

Redis is an in-memory data structure server. It stores values such as strings, hashes, lists, sets, sorted sets and streams under keys and exposes operations on those structures as commands. It solves the problem of needing very low latency access to shared state (caches, sessions, counters, queues, leaderboards) that would be too slow or too contended in a disk-based relational database. It is usually deployed beside a primary database, not instead of it.

Q: Why is Redis fast?

Data lives in RAM, so no disk seek is needed for reads. Data structures are chosen so that common operations are $O(1)$ or $O(\log N)$. Command execution is single-threaded, which removes lock contention and context switching. The protocol is simple and supports pipelining, and I/O multiplexing (**epoll**) lets one thread serve thousands of connections. In practice the network round trip is the dominant cost.

Q: If Redis is single-threaded, how does it handle many clients?

One event loop multiplexes all sockets: it reads ready sockets, parses commands, executes them one by one, and writes replies. Because each command is short, the queue drains quickly. Since Redis 6 you can enable **io-threads** to parallelise socket reading and writing, but commands still execute on the main thread. Background threads handle things like lazy freeing of big values, closing files and AOF fsync.

Q: What is the downside of single-threaded execution?

Any slow command blocks every client. A **KEYS *** on millions of keys, **SMEMBERS** on a huge set, **DEL** of a giant key, or a long Lua script stalls the whole server. You mitigate with **SCAN**, **UNLINK**, small keys, short scripts and monitoring **SLOWLOG**. Also, one instance uses one core for commands, so scaling CPU means sharding with Cluster.

Q: Is Redis a cache or a database?

Both, depending on configuration and use. With no persistence and an eviction policy it is a cache. With AOF persistence, replication and backups it can be a primary store for data that fits in memory and tolerates its consistency model. The honest answer: it is a data structure server; you decide the role, and for important data you should know exactly what you can lose.

Q: What are the main use cases?

Caching, session storage, rate limiting, counters and analytics (including HyperLogLog), leaderboards with sorted sets, queues and event streams, pub/sub, distributed locks, geospatial lookups, and with the query engine, JSON documents, full-text and vector search for AI applications.

Q: What is RESP?

The Redis Serialization Protocol: a simple, text-framed binary-safe protocol with typed replies (simple strings, errors, integers, bulk strings, arrays). RESP3 (enabled with `HELLO 3`) adds richer types such as maps, sets and doubles, and push messages, which client-side caching uses for invalidations. It is easy to implement and to debug with `telnet` or `redis-cli`.

Q: What is pipelining and when do you use it?

Pipelining sends many commands without waiting for each reply, then reads all the replies. It removes the per-command network round trip, which is often 100 times larger than server execution time. Use it for bulk loads and independent commands. It is not atomic: other clients' commands can interleave. For atomicity use `MULTI/EXEC` or Lua.

Q: What are the logical databases?

A standalone Redis has 16 numbered databases (0 to 15), selected with `SELECT`. They share the same process, memory and config, so they provide namespacing only, not isolation. Cluster supports only database 0. Most teams use key prefixes (`cart:42`) rather than numbered databases.

Q: How do you name keys?

Use a consistent hierarchy with colons, such as `entity:id:field` (`user:42:profile`). Keep keys short but readable, because keys cost memory on every copy and in replication. Include a version or tenant when needed (`v2:product:7`). In Cluster, use hash tags (`{user:42}:cart`) when several keys must live in one slot.

Q: What happens when Redis runs out of memory?

It depends on `maxmemory` and `maxmemory-policy`. With the default `noeviction`, write commands that would need more memory return an `OOM` error while reads continue. With a policy such as `allkeys-lru` or `allkeys-lfu`, Redis evicts keys to stay under the limit. If `maxmemory` is unset, the process keeps growing until the operating system kills it or starts swapping.

Q: How do you check that Redis is alive and healthy?

`PING` returns `PONG` for liveness. `INFO` gives memory, clients, persistence, replication and stats. For deeper checks: `SLOWLOG GET`, `LATENCY DOCTOR`, `MEMORY DOCTOR`, and metrics exported to Prometheus. A good health check also writes and reads a test key, because a replica in a bad state can still answer `PING`.

Q: What is the difference between Redis and Memcached?

Memcached is a simple, multi-threaded key-value cache for opaque strings. Redis offers rich structures, persistence options, replication, Sentinel and Cluster, scripting, pub/sub and streams. Pick Memcached only when you need a pure, simple cache and its multi-threaded model; Redis covers most other needs.

Q: What is the licensing situation?

Redis changed its license in March 2024 (version 7.4) to a source-available dual license (RSALv2 and SSPLv1), and added AGPLv3 as an option with Redis 8. Valkey is a Linux Foundation fork of the last BSD-licensed code. For commercial use, read the current license terms for the exact version you ship.

Q: What is new in Redis 8?

Redis 8.0 merged Redis Stack into Redis Open Source, so JSON, the query engine (search and vector), time series and the probabilistic structures are built in, and it added hash-field commands such as `HGETEX` and `HSETEX`. Later releases added things like `XDELEX` and atomic cluster slot migration (8.4), and performance and memory improvements plus new eviction policies (8.6). Check the release notes for your exact version.

Q: How do you secure a Redis instance?

Bind to private interfaces and keep `protected-mode` on, never expose it to the internet, use ACL users with minimal permissions instead of a shared password, enable TLS between clients and servers, restrict dangerous commands (`FLUSHALL`, `CONFIG`, `DEBUG`, `KEYS`) through ACLs, and isolate the network with firewalls or security groups.

KEY POINTS

- Fundamentals answers revolve around: in-memory, single-threaded command execution, rich types, optional persistence.
- Always mention the trade-off after the benefit: speed versus memory cost, simplicity versus the blocking risk.

12.6 Interview Q&A: Data Structures

Q: Name the core data types and one use for each.

String (counters, tokens, cached blobs), list (queues, recent items), hash (objects, carts), set (tags, unique members), sorted set (leaderboards, delayed jobs), stream (event log with consumer groups), bitmap (daily active users), HyperLogLog (approximate unique counts), geo (nearby search). Redis 8 adds built-in JSON, time series, Bloom filters and more.

Q: What is the maximum size of a string value?

512 MB. In practice keep values small (kilobytes), because a large value blocks the single thread while it is copied and serialised, and it hurts replication and memory fragmentation.

Q: How do you implement an atomic counter?

`INCR key` (or `INCRBY`). It is atomic because command execution is serial, so there are no lost updates even with thousands of concurrent clients. Add `EXPIRE` for windows, usually in a Lua script or by setting the TTL only when the result is 1.

Q: Hash versus storing JSON in a string?

A hash gives field-level reads, writes and atomic increments (`HINCRBY`) without rewriting the whole object, and small hashes use a compact encoding. A JSON string is simplest when you always read and write the whole object and need nested structure. With Redis 8 the JSON type adds path-based access. Hashes cannot nest.

Q: List versus stream for a queue?

A list gives a simple FIFO with `LPUSH/BRPOP`, but a popped item is gone, so a crashed worker loses it unless you use the reliable `LMOVE` pattern. A stream keeps entries, supports consumer groups, tracks pending entries until acknowledged (`XACK`), allows replay and reclaiming (`XAUTOCLAIM`). Use streams when you need at-least-once delivery.

Q: How does a sorted set work internally and what is its complexity?

Small sorted sets use a listpack; larger ones use a skip list plus a hash table. The skip list keeps members ordered by score, giving $O(\log N)$ insert, rank and range start; the hash table gives $O(1)$ score lookup by member. Ranges cost $O(\log N + M)$ for M returned elements.

Q: How would you build a leaderboard?

A sorted set per board: `ZINCRBY lb:daily 10 user:42` to add points, `ZREVRANGE` or `ZRANGE ... REV WITHSCORES` for the top N , `ZREVRANK` for a user's position. For "around me" fetch a window from rank minus k to rank plus k . For ties, encode a tiebreaker into the score or use lexicographic ordering. Reset daily boards by key name (date in the key) with a TTL.

Q: What is a HyperLogLog and what are its trade-offs?

A probabilistic structure for counting distinct elements in a fixed small memory (up to about 12 KB) with a standard error of about 0.81%. **PFADD** adds, **PFCOUNT** estimates, **PFMERGE** unions. You cannot list members or remove them. Great for unique visitors; not for exact billing.

Q: When use a bitmap?

For dense boolean data indexed by small integers, such as "did user N log in on this date". One bit per user: 10 million users take about 1.2 MB. **SETBIT**, **GETBIT**, **BITCOUNT** and **BITOP** combine days (for example users active on all 7 days). It wastes space for sparse or huge ids.

Q: What is a Bloom filter and where does it help?

A compact probabilistic set with no false negatives but a tunable false-positive rate. In Redis 8 it is built in (**BF.RESERVE**, **BF.ADD**, **BF.EXISTS**). It protects a database from cache penetration: if the filter says an id definitely does not exist, skip the lookup.

Q: How does Geo work?

Geo commands store members in a sorted set with a score derived from a 52-bit geohash of longitude and latitude. **GEOADD** inserts, **GEOSEARCH** finds members within a radius or box, **GEODIST** computes distance. Because it is a sorted set, you can also use **ZREM** and **ZRANGE**.

Q: Why is SMEMBERS or HGETALL dangerous?

They return the entire collection in one command, **O(N)**. With a million elements that blocks the server and floods the output buffer. Use **SSCAN/HSCAN** for incremental iteration, or design so collections stay small.

Q: How does SCAN guarantee anything while keys change?

SCAN returns a cursor and a batch. It guarantees that every element present for the entire iteration is returned at least once, but it may return an element several times and may or may not return elements added or deleted during the scan. Your processing must be idempotent. **COUNT** is a hint, not a limit.

Q: What is an encoding, and how do you see it?

An encoding is the internal representation of a value, e.g. **listpack** for small hashes and sorted sets, **quicklist** for lists, **hashtable**, **skiplist**, **intset**, **embstr**, **int**. **OBJECT ENCODING** key shows it. Redis converts automatically as data grows past thresholds like **hash-max-listpack-entries**.

Q: Can you store the same object as a hash and index it for queries?

With plain commands you maintain secondary indexes yourself (sets or sorted sets pointing to keys). With Redis 8's built-in query engine you create an index with **FT.CREATE** over hashes or JSON documents and query with **FT.SEARCH**. Spring Data Redis repositories (**@Indexed**) do simple equality indexes with sets, which is rarely worth it for large data.

KEY POINTS

- For every structure be ready with: use case, main commands, complexity, and the failure mode (big key, **O(N)** command).
- Show that you pick structures by access pattern, and that you know encodings explain memory behaviour.

12.7 Interview Q&A: Internals and Persistence

Q: How does Redis execute a command, step by step?

The event loop sees a readable socket, reads bytes into the client's query buffer, parses a command, looks it up in the command table, checks ACLs, memory and state, executes it on the main thread, appends the reply to the client's output buffer, propagates writes to the AOF and replicas, and later writes the reply to the socket (optionally by an I/O thread).

Q: How do expiry and eviction differ?

Expiry removes keys whose TTL passed; it is about time. Eviction removes keys when memory exceeds `maxmemory`; it is about space. Expiry happens lazily (when a key is accessed) and actively (a periodic sampler deletes expired keys). Eviction picks victims by a policy such as LRU, LFU, random or TTL.

Q: Are LRU and LFU exact in Redis?

No, they are approximations. Redis samples `maxmemory-samples` keys (default 5) and evicts the best candidate among them, saving the memory of a global ordering. Raising the sample count improves accuracy at some CPU cost. LFU keeps a small logarithmic counter with decay, which handles frequency better than recency for skewed workloads.

Q: Which eviction policy should you choose for a cache?

Usually `allkeys-lru` or `allkeys-lfu` when every key is cacheable. Use `volatile-*` policies only if you mix cache keys (with TTL) and keys that must never be evicted, and remember that if nothing has a TTL they behave like `noeviction`. Redis 8.6 adds least-recently-modified variants (`allkeys-lrm`, `volatile-lrm`).

Q: Explain RDB.

RDB writes a compact point-in-time snapshot of the dataset to a file. `BGSAVE` forks a child process, which writes the snapshot while the parent keeps serving, using copy-on-write memory pages. It is compact, fast to load and good for backups, but you lose everything since the last snapshot, and the fork of a large dataset can pause the server briefly.

Q: Explain AOF.

The append-only file logs every write command. On restart Redis replays the log. `appendfsync` controls durability: `always` (fsync each write, safest, slowest), `everysec` (default, lose up to about a second), `no` (let the OS decide). The log grows, so Redis rewrites it in the background (`BGrewriteAOF`) into a minimal equivalent; since 7.0 it is a multi-part AOF with a base file, incremental files and a manifest.

Q: RDB or AOF?

RDB alone: fast restarts, simple backups, minutes of possible loss. AOF with `everysec`: about a second of loss, larger files, slower restart. Many production systems enable both: the AOF for durability (with an RDB preamble to speed loading) and periodic RDB files for backups. A pure cache may use neither.

Q: What is copy-on-write and why does it affect memory sizing?

After `fork`, parent and child share memory pages. When the parent modifies a page, the OS copies it. With heavy writes during a snapshot, many pages get copied, so peak memory can approach twice the dataset in the worst case. Leave headroom beyond `maxmemory` and watch `latest_fork_usec` and RSS.

Q: Can Redis lose acknowledged writes even with AOF?

Yes. With `everysec` a crash can lose the last second. Even with `always`, a replica-based failover can lose writes that the primary acknowledged but had not replicated. Persistence protects against restarts of one node; replication is asynchronous, so it does not give zero loss either. `WAIT` improves the odds but does not guarantee consensus.

Q: What are Redis transactions, and is there rollback?

`MULTI` starts queueing; `EXEC` runs the queue atomically, without interleaving from other clients. There is no rollback: if one command fails at run time (wrong type), the others still run; syntax errors at queue time abort the whole transaction. `WATCH` adds optimistic locking: if a watched key changes before `EXEC`, the transaction aborts and you retry.

Q: Lua scripts versus MULTI/EXEC?

Both are atomic. A transaction cannot read a value and branch on it in the middle; a Lua script can. So read-modify-write logic (token bucket, compare-and-delete lock release, stock decrement) belongs in Lua or a Function. Scripts must be short, deterministic in effect and declare keys via `KEYS` so Cluster can route them.

Q: What are Redis Functions?

Introduced in 7.0, Functions are named libraries loaded with `FUNCTION LOAD` and called with `FCALL`. Unlike `EVAL` scripts, they are part of the dataset (persisted and replicated), versioned by library, and do not depend on each client caching script SHAs.

Q: What is a big key and why is it a problem?

A key holding a very large value or collection (a list with millions of items, a 50 MB string). Reading, deleting or expiring it blocks the single thread, it creates replication and network spikes, and it unbalances Cluster shards. Find them with `redis-cli --bigkeys` or `--memkeys`; split them; delete with `UNLINK`.

Q: What is a hot key?

A key accessed far more than others, which saturates one thread or one Cluster shard. Fixes: replicate the value across several keys (suffix 1..N) and read a random one, add a local in-process cache, use client-side caching, or move the logic out of Redis. Redis 8.6 adds a `HOTKEYS` command to help find them.

Q: What does fragmentation mean here?

`mem_fragmentation_ratio` is RSS divided by `used_memory`. A ratio well above 1 means the allocator holds memory it cannot return to the OS (after many deletes of varied sizes). Active defragmentation (`activedefrag yes`) can reclaim it. A ratio below 1 can mean swapping, which is disastrous for latency.

Q: How do you debug a latency spike?

Check `SLOWLOG GET` for slow commands, `LATENCY DOCTOR` and `LATENCY LATEST` for events, `INFO stats` for `latest_fork_usec`, `INFO persistence` for AOF fsync delays, the `transparent huge pages` setting, swapping, and client side issues such as connection storms or network RTT. Compare client-measured latency with server-side execution time.

KEY POINTS

- Internals answers: one thread, approximated LRU/LFU, fork plus copy-on-write, AOF fsync policies, and no rollback in transactions.
- State the data-loss window every time you discuss persistence.

12.8 Interview Q&A: Scaling and High Availability

Q: How does replication work?

A replica connects to its primary with `REPLICAOF`. On first contact it does a full sync: the primary produces an RDB snapshot (written to disk or streamed directly with diskless replication) and sends it, buffering new writes meanwhile; then the replica receives the stream of write commands. After a short disconnect it attempts a partial resync using the replication ID, offset and the primary's backlog buffer (`repl-backlog-size`).

Q: Is replication synchronous?

No, it is asynchronous: the primary acknowledges the client before replicas confirm. `WAIT numreplicas timeout` blocks until enough replicas have acknowledged, which narrows the loss window but is not a guarantee against all failure sequences. `min-replicas-to-write` makes the primary refuse writes when too few replicas are healthy.

Q: Can you read from replicas?

Yes, but reads may be stale because of replication lag. Use replica reads for data that tolerates staleness (catalogue cache) and not for read-your-writes flows such as "update then immediately show". Client libraries expose read preferences (Lettuce `ReadFrom`).

Q: What does Sentinel do?

Sentinel is a set of processes that monitor a primary and its replicas, detect failure, elect a leader sentinel, promote a replica and reconfigure the others, and tell clients where the current primary is (`SENTINEL get-master-addr-by-name`). It provides high availability for a single shard; it does not shard data.

Q: What is the difference between quorum and majority in Sentinel?

The quorum is the number of sentinels that must agree the primary is unreachable to mark it objectively down (ODOWN). To actually start a failover, a sentinel must also be authorised by a majority of all sentinels. So deploy at least three sentinels on independent failure domains.

Q: What is split-brain and how do you limit it?

A network partition leaves the old primary alive with some clients while a new primary is promoted elsewhere; both accept writes and the old one's writes are lost when it rejoins as a replica. Limit it with `min-replicas-to-write` and `min-replicas-max-lag`, so an isolated primary stops accepting writes, and with sentinels placed sensibly.

Q: How does Redis Cluster shard data?

The keyspace is divided into 16384 hash slots. A key's slot is `CRC16(key) mod 16384`; each primary owns a subset of slots. Clients cache the slot map and send each command to the right node; a wrong node replies `MOVED`, and during migration `ASK`. Each primary can have replicas for failover.

Q: What are hash tags?

If a key contains `{...}`, only the part inside the first braces is hashed. `{user:42}:cart` and `{user:42}:profile` land in the same slot, so multi-key commands, transactions and scripts that touch both work. Overuse creates hot shards.

Q: What limitations does Cluster impose?

Multi-key commands and scripts need all keys in one slot (otherwise `CROSSSLOT`); only database 0 exists; clients must be cluster-aware; pub/sub is broadcast across the cluster (sharded pub/sub since 7.0 scales better); and `cluster-require-full-coverage yes` stops serving writes if any slot has no owner.

Q: How does Cluster detect failures and fail over?

Nodes gossip over the cluster bus (port + 10000). If a node is unreachable for `cluster-node-timeout`, others flag it as possibly failing (PFAIL); when a majority of primaries agree, it is marked FAIL. Its replicas then hold an election and one is promoted. If a shard loses both primary and replicas, its slots are uncovered.

Q: How do you add capacity to a Cluster?

Add new nodes, then move slots with `redis-cli --cluster reshard` or `rebalance`. Slot migration moves keys in batches with `MIGRATE`, and clients are redirected with `ASK` meanwhile. Redis 8.4 introduced atomic slot migration (`CLUSTER MIGRATION`) that moves whole slots as a unit with less disruption.

Q: Sentinel or Cluster?

Sentinel when the dataset and write throughput fit a single node and you want automatic failover with simple semantics (multi-key operations everywhere). Cluster when you need to scale memory or writes horizontally and can design keys around slots. Many apps are fine on one primary with replicas for years.

Q: How do clients find the primary in each topology?

Standalone: a fixed address. Sentinel: the client asks the sentinels for the current primary address and follows changes. Cluster: the client connects to seed nodes, downloads the slot map with `CLUSTER SHARDS` (or `CLUSTER SLOTS`) and refreshes it on `MOVED` or periodically.

Q: How do managed services like ElastiCache differ?

They handle provisioning, patching, backups and failover, often with their own limits and sometimes different feature sets than open source Redis. Check version, supported modules and commands, persistence options and pricing model before depending on a feature.

Q: How do you scale reads versus writes?

Reads: add replicas and route read-only traffic to them, or add a local cache. Writes: shard with Cluster, or reduce write volume (batching, pipelining, coarser counters). There is no way to scale writes of a single key beyond one core, so split hot counters into sub-keys.

Q: What happens to a replica that is too far behind?

If the primary's backlog no longer contains the replica's offset, partial resync fails and a full sync starts, which costs a fork, disk or network bandwidth, and memory. Size `repl-backlog-size` for the longest outage you want to survive with partial resync, and watch replication offsets.

KEY POINTS

- Replication is asynchronous; Sentinel gives HA without sharding; Cluster gives sharding with 16384 slots and per-slot key constraints.
- Always connect a topology answer to its failure mode and to what the client must do.

12.9 Interview Q&A: Caching and Patterns

Q: Explain cache-aside.

The application checks the cache first; on a miss it reads the database, writes the result into the cache with a TTL, and returns it. On writes it updates the database and then deletes (or updates) the cache entry. It is the default pattern because the cache is optional: if Redis is down the application still works, only slower.

Q: Why delete the cache entry on write instead of updating it?

Deleting is simpler and avoids a race where two concurrent writers update the database in one order and the cache in another, leaving a stale value until the TTL. After a delete the next read repopulates from the database. It still has a small race (a reader loading an old row just before the write commits), which is why a TTL is always the safety net.

Q: What is a cache stampede and how do you prevent it?

When a hot key expires, many requests miss at once and all hit the database. Prevent it with single-flight per key (an in-process lock, or a short `SET NX PX` lock in Redis so one caller rebuilds), TTL jitter so keys do not expire together, probabilistic early refresh, or serving stale data while one caller refreshes.

Q: What is cache penetration?

Requests for keys that do not exist in the database bypass the cache every time, so each one reaches the database. Fix with negative caching (store a short-lived "not found" marker), input validation, and a Bloom filter of valid ids in front of the cache.

Q: What is cache avalanche?

Many keys expire at the same moment, or the cache goes down, and the database is hit by the combined load. Add random jitter to TTLs, warm important keys, use replicas or Sentinel/Cluster for availability, and protect the database with rate limits and circuit breakers.

Q: Write-through versus write-behind?

Write-through writes to the cache and the database synchronously, so reads are fresh but writes are slower.

Write-behind acknowledges after writing to the cache and flushes to the database asynchronously, which is fast but can lose data if Redis fails before the flush and complicates ordering and failure handling. Use it only when that risk is acceptable.

Q: How do you choose a TTL?

Balance staleness tolerance against hit ratio and database load. Ask how wrong the data can be and for how long. Use short TTLs for volatile data, longer for reference data, add jitter, and combine with explicit invalidation for data that must change quickly. Measure hit ratio before and after changes.

Q: How do you keep cache and database consistent?

You cannot get perfect consistency with two independent stores without distributed transactions. Practical options: delete-after-write plus a TTL, delayed double delete, invalidation driven by change data capture or an outbox, and versioned keys or values. State the tolerated staleness explicitly.

Q: How do you implement a distributed lock?

`SET lock:key <random-token> NX PX 30000` acquires it; release with a Lua script that deletes the key only if the stored token equals yours, so you never delete someone else's lock. Handle expiry during long work with renewal (a watchdog) or by keeping the work short.

Q: Is a Redis lock safe for correctness?

Not on its own. With an expiry, a paused process (long GC, network delay) can keep working after its lock expired while another process holds it. Fencing tokens, where the protected resource rejects stale lock holders, solve this. Redlock (locks on several independent nodes) is debated: critics say timing assumptions make it unsafe for strict correctness, defenders say it is fine for efficiency use cases. For money or inventory, prefer database constraints or serialisation.

Q: How would you implement rate limiting?

Fixed window: `INCR` a key per window with `EXPIRE`; simple but allows bursts at window edges. Sliding window log with a sorted set is accurate but memory heavy. Sliding window counter approximates using two windows. Token bucket in Lua allows bursts and a steady rate. Do the check-and-update atomically in a script and return remaining tokens for response headers.

Q: How do you make an API idempotent with Redis?

The client sends an idempotency key. The server does `SET idem:<key> in-progress NX EX 86400`; if it succeeds, process and store the result under the key; if it fails, return the stored result or a conflict while in progress. Redis helps with the fast path; the database must still enforce uniqueness for money-critical operations.

Q: Pub/Sub versus Streams?

Pub/Sub is fire-and-forget: subscribers that are offline miss messages and nothing is stored. Streams persist entries, support consumer groups, acknowledgement and replay. Use pub/sub for ephemeral notifications such as cache invalidation or live UI updates, and streams for work that must not be lost.

Q: How do you build a delayed job queue?

Put job ids in a sorted set scored by due timestamp. A worker polls with `ZRANGE ... BYSCORE` for due items and claims each atomically (a Lua script that removes from the zset and pushes to a ready list). Avoid relying on key expiry notifications, which are not guaranteed delivery.

Q: How would you prevent overselling in a flash sale?

Keep stock in a Redis counter and decrement it in a Lua script that checks `stock > 0` and decrements in one atomic step; reject if zero. Record the reservation (with a TTL) and emit an order event to a stream; confirm against the database asynchronously. Use idempotency keys so retries do not double-reserve.

Q: What are the common Redis anti-patterns?

KEYS in production, huge keys and values, no TTL on cache data, unbounded lists or streams, using Redis as the only copy of important data without persistence, blocking Lua, wrong eviction policy, N+1 round trips instead of pipelines, and fire-and-forget pub/sub for critical events.

KEY POINTS

- For every cache question, mention TTL as a safety net, the stampede risk, and how stale the data may be.
- For every concurrency question, ask "is there a single atomic command or script that does this?" before reaching for a lock.

12.10 Interview Q&A: Java and Spring

Q: Lettuce versus Jedis versus Redisson?

Lettuce (Spring's default) is built on Netty, supports sync, async and reactive APIs, and a single connection is thread-safe and can be shared. Jedis is a simple blocking client; a `Jedis` connection is not thread-safe, so you use a pool (`JedisPool`, or `JedisPooled`). Redisson adds distributed objects and services (locks, maps, rate limiters, queues) on top of Redis. Choose Lettuce by default, Redisson when you want its higher-level structures.

Q: Why does Lettuce not need a connection pool by default?

Its connections are thread-safe and multiplexed with pipelining at the protocol level, so one connection can serve many threads. A pool is only needed for blocking or transactional operations that hold a connection exclusively (`BLPOP`, `MULTI` on a shared connection). Spring Boot enables pooling only when `commons-pool2` is on the classpath and pool settings are configured.

Q: What is the difference between StringRedisTemplate and RedisTemplate?

`StringRedisTemplate` uses string serializers for keys and values, which is what you want for plain strings and for inspecting data in `redis-cli`. `RedisTemplate` is generic; its default serializer is JDK serialization, which produces unreadable, class-coupled bytes. Configure explicit serializers, typically a string key serializer and a JSON value serializer.

Q: How do you serialise objects as JSON with Spring Data Redis 4?

Use the Jackson 3 based serializers: `JacksonJsonRedisSerializer` for a known type, or `GenericJacksonJsonRedisSerializer` for generic values. The older Jackson 2 serializers are deprecated. Default typing is not enabled by default in the generic one, so enable it deliberately and restrict allowed types, since polymorphic deserialization is a security risk.

Q: How does Spring's cache abstraction work with Redis?

`@EnableCaching` activates proxies. `@Cacheable` checks the cache before calling the method, `@CachePut` always writes, `@CacheEvict` removes. A `RedisCacheManager` creates `RedisCache` instances; keys look like `cacheName::key`. Configure TTL per cache with `RedisCacheConfiguration.entryTtl`, choose serializers, and decide on null caching.

Q: What are the common pitfalls of @Cacheable?

Self-invocation (calling a cached method from the same class bypasses the proxy), caching without TTL, default JDK serialization, caching huge collections, unstable key generation (non-`equals` objects), caching nulls unintentionally, and stampedes. `sync = true` adds per-key locking, but only inside one JVM.

Q: How do you listen to Pub/Sub or Streams in Spring?

Pub/Sub: register a `MessageListener` with a `RedisMessageListenerContainer` for channel or pattern topics. Streams: use a `StreamMessageListenerContainer` with a consumer group and `Consumer.from(group, name)`, acknowledge with `XACK` after successful processing, and handle pending entries with claim logic. Make handlers idempotent.

Q: How do you execute a Lua script from Spring?

Wrap it in a `DefaultRedisScript<T>` with the result type and call `redisTemplate.execute(script, keys, args...)`. The template uses `EVALSHA` and falls back to `EVAL` if the script is not cached. Keep scripts in resource files, and keep key names in `KEYS` for Cluster compatibility.

Q: How does a transaction work with RedisTemplate?

`MULTI/EXEC` must run on one connection. Use a `SessionCallback` so all commands share a connection, or enable transaction support on the template. Remember: reads inside `MULTI` return null until `EXEC`, so read-modify-write logic should be Lua or use `WATCH`.

Q: How do you test code that uses Redis?

Use Testcontainers with a real `redis:8` container (a generic container exposing 6379) and point `spring.data.redis.host/port` at it, using Spring Boot's dynamic property or service-connection support. Avoid embedded fake Redis servers: they differ in behaviour, especially for Lua, streams and Cluster.

Q: What should happen if Redis is down?

Decide per feature. A cache should degrade: catch connection errors, log, and read from the database (with a circuit breaker so you do not wait on timeouts). Rate limiting may fail open or closed depending on risk. Sessions or locks may need to fail the request. Always set connect and command timeouts so threads are not blocked indefinitely.

Q: How do you size timeouts and pools?

Set a command timeout a few times above your p99 latency (often hundreds of milliseconds, not seconds), a short connect timeout, and enable topology refresh for Sentinel/Cluster so clients follow failovers. For pools, size by the number of concurrent blocking operations, not by request volume.

Q: What is Spring Session with Redis?

It stores HTTP sessions in Redis instead of the servlet container's memory, so any application instance can serve any user and sessions survive restarts. The session is stored as a hash with a TTL. Cost: serialisation overhead and a Redis dependency on the login path.

KEY POINTS

- In Java answers, name the client, its thread-safety model, the serializer and the timeout policy.
- Never leave `RedisTemplate` on JDK serialization in an answer you would defend.

12.11 System-Design Scenarios

Each scenario below follows one shape you can reuse on a whiteboard: requirements, Redis structures, the critical operation, failure modes, and what you would say about scale.

12.11.1 Design a rate limiter

Requirements: 100 requests per minute per API key, low overhead, correct across many gateway instances. Use a token bucket stored in a hash (`rl:{apiKey}` with `tokens` and `ts`), updated by a Lua script so refill and take are atomic.

token_bucket.lua

LUA

```
-- KEYS[1] = bucket key; ARGV = capacity, refill per second, now (ms), cost
local capacity = tonumber(ARGV[1])
local rate = tonumber(ARGV[2])
local now = tonumber(ARGV[3])
local cost = tonumber(ARGV[4])
local state = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(state[1]) or capacity
local ts = tonumber(state[2]) or now
tokens = math.min(capacity, tokens + (now - ts) / 1000 * rate)
local allowed = 0
if tokens >= cost then
    tokens = tokens - cost
    allowed = 1
end
redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', KEYS[1], math.ceil(capacity / rate * 1000) + 1000)
return {allowed, math.floor(tokens)}
```

Trade-offs: pass the time from the application (or use `TIME` inside the script) and be aware of clock skew between gateways. In Cluster, the single key keeps the script on one slot. If Redis fails, choose fail-open for public traffic or fail-closed for sensitive endpoints. Return `X-RateLimit-Remaining` and `Retry-After` headers.

12.11.2 Design a leaderboard

Use one sorted set per board and period (`lb:bestsellers:2026-10-10`). Update with `ZINCRBY`, read the top 100 with `ZRANGE key 0 99 REV WITHSCORES`, find a user with `ZREVRANK`. Daily boards expire by TTL; all-time boards can be rebuilt from the database. Paginate by rank. Around-me queries fetch rank plus or minus 5. At very large scale, shard the board by region and merge top-N lists, because one sorted set lives on one node. Persist the source of truth (events) elsewhere.

12.11.3 Design a URL shortener cache

Reads vastly outnumber writes. The database stores `code -> longUrl`; Redis caches `url:{code}` as a string with a TTL of hours plus jitter. Generate codes with an atomic `INCR` encoded in base 62 (collision free) or random codes checked with `SET NX`. Add negative caching for unknown codes and a Bloom filter to keep scanners from hitting the database. Hot links: add an in-process cache in front. Analytics clicks: `INCR` counters or a stream consumed in batches, never a database write per click.

12.11.4 Design a session store

Store `session:{sid}` as a hash with a sliding TTL (`EXPIRE` on each access, say 30 minutes). Use a long random session id (at least 128 bits). Persistence: AOF `everysec` plus replicas so a restart does not log everyone out; accept that a failover may lose the last second of updates. Index user to sessions with a set (`user:42:sessions`) for "log out everywhere". Serialize compactly and avoid storing large objects.

12.11.5 Design a distributed lock service

For efficiency locks (avoid duplicate work), use **SET NX PX** with a random token and Lua release; renew while working. For correctness, add a fencing token (an **INCR** counter value returned with the lock) that the downstream resource checks and rejects if older than the last seen. Mention Redlock and its critique, mention that the best design avoids needing the lock, and describe behaviour on failover: a lock acquired on the primary but not yet replicated can be granted twice.

12.11.6 Design a flash sale

Pre-load stock into **stock:{sku}**. A Lua script atomically checks and decrements; a **reservation:{orderId}** key with a TTL holds the item for the payment window; a stream **stream:orders** carries order events to workers in a consumer group. Idempotency keys stop duplicate submissions. A rate limiter and a waiting-room queue protect the system. Reconcile stock with the database afterwards; when a reservation expires, a compensating script returns the unit. Expect hot-key pressure on one SKU: shard the counter into buckets if needed.

12.11.7 Design a news feed

Fan-out on write: for each post, push the post id into followers' timelines (**feed:{userId}**) as a capped list or sorted set scored by time, trimmed with **LTRIM** or **ZREMRANGEBYRANK**. Fan-out on read: build the feed at request time by merging recent posts from followed accounts. Hybrid: fan-out on write for ordinary users and on read for celebrities with millions of followers. Store post bodies in a cache or database, and only ids in the timelines. Timelines are rebuildable, so persistence can be light.

KEY POINTS

- A good design answer names the structure, the atomic operation, the TTL and the failure behaviour.
- Say what you would measure: hit ratio, p99 latency, memory growth, evictions and replication lag.

12.12 Common Mistakes and Myths

Myth	Reality
Redis is single-threaded, so it cannot use many cores	Commands run on one thread, but I/O threads and background threads exist, and Cluster uses one process per shard
Redis is not durable	It is as durable as you configure: AOF with always or everysec , plus replicas and backups
Replication gives zero data loss	It is asynchronous; failover can lose acknowledged writes
KEYS is fine if the dataset is small	Datasets grow; use SCAN
Transactions roll back on error	There is no rollback in MULTI/EXEC
A TTL guarantees prompt deletion	Expired keys are removed lazily and by sampling; memory is not freed at the exact second
LRU evicts the true least recently used key	It samples a few keys
Cluster gives you transparent scaling	Multi-key operations are limited to a slot, and keys must be designed for it
More replicas always mean more safety	They add read capacity and copies, but not consensus
Redis is only a cache	It is a general data structure server with streams, JSON, search and more
Pub/Sub never loses messages	Offline subscribers miss them; there is no storage

Myths and their corrections.

Mistakes seen in real projects

- Running without **maxmemory** and finding the host killed by the OS.
- Caching without TTLs so the cache slowly becomes a stale second database.
- Using one giant hash or list for everything, creating a big key and a hot key.
- Using **RedisTemplate** with default JDK serialization and then being unable to read keys from other services.
- Holding a lock around a slow remote call with a TTL shorter than the call.
- Treating pub/sub as a queue.

- Using **FLUSHALL** on the wrong environment because dev and prod share a network.
- Forgetting that **INFO** fields and defaults differ between versions and cloud providers.

KEY POINTS

- Most Redis incidents come from unbounded growth, slow commands, missing TTLs or unexamined durability assumptions.
- A candidate who names these unprompted sounds like someone who has operated Redis.

12.13 A 30-Day Study Plan

This plan assumes about 90 minutes a day. Each week ends with a build task so you do not just read. Chapter names point to this book; "Lab" means the hands-on labs and "Project" means the end-to-end projects.

Week	Learn	Build or practice
1	Fundamentals; install Redis with Docker; redis-cli tour; strings, hashes, lists, sets	Complete the first labs; implement a visit counter, a todo list with a hash and a list, and unique tags with a set
2	Sorted sets, streams, bitmaps, HyperLogLog, geo; expiry and eviction; RDB and AOF	Build a leaderboard; run a producer and a consumer group; restart the server with RDB then AOF and observe what survives
3	Transactions, Lua, pipelining; replication, Sentinel, Cluster; security	Write the token bucket script; start a primary with a replica and kill the primary; start a 3-shard Cluster and trigger MOVED
4	Java and Spring Data Redis; caching patterns; operations; the interview chapters	Work through the catalogue cache Project and the rate limiter Project; then do the flash-sale Project if time allows

Four-week plan.

Daily rhythm

1. 30 minutes reading a chapter, taking notes in your own words.
2. 40 minutes in **redis-cli** or code: type every example, then change it and predict the result.
3. 20 minutes answering three interview questions out loud from the previous chapters, without looking.

Exercises that pay off

- Fill a database with a million keys, then compare **MEMORY USAGE** for the same data as strings, hashes and JSON.
- Set **maxmemory** low and watch evictions under each policy; chart **evicted_keys**.
- Kill -9 the server under **appendfsync always, everysec** and with no AOF; count lost writes.
- Run **redis-benchmark** with and without pipelining and explain the difference.
- Inject a 2-second delay on a replica link (for example with a proxy) and watch **master_repl_offset**.
- Write a stampede test: 200 threads loading one expired key, once with and once without single-flight.

Before an interview

Re-read the decision guides, the system-design scenarios and the myths table. Practise one scenario aloud with a timer. Prepare two short stories from your own projects: one where Redis solved a problem, and one where a design choice (TTL, eviction, locking) had a trade-off you can explain.

KEY POINTS

- Consistency beats intensity: type the commands, break things on purpose, and explain trade-offs aloud.
- Finish with one project you can demo and discuss for ten minutes.